

Linux Software (DM35424)

Generated by Doxygen 1.15.0

1 Topic Index	1
1.1 Topics	1
2 Data Structure Index	3
2.1 Data Structures	3
3 File Index	5
3.1 File List	5
4 Topic Documentation	7
4.1 DM35424 ADC Library Constants	7
4.1.1 Detailed Description	10
4.1.2 Enumeration Type Documentation	10
4.1.2.1 DM35424_Adc_Clock_Events	10
4.1.2.2 DM35424_Gains	11
4.1.2.3 DM35424_Input_Mode	11
4.1.2.4 DM35424_Input_Ranges	12
4.1.2.5 DM35424_Sampling_Mode	13
4.2 DM35424 ADC Public Library Functions	13
4.2.1 Detailed Description	16
4.2.2 Function Documentation	16
4.2.2.1 DM35424_Adc_Ad_Config_Get_Mode()	16
4.2.2.2 DM35424_Adc_Ad_Config_Set_Mode()	17
4.2.2.3 DM35424_Adc_Channel_Find_Interrupt()	18
4.2.2.4 DM35424_Adc_Channel_Get_Filter()	19
4.2.2.5 DM35424_Adc_Channel_Get_Front_End_Config()	21
4.2.2.6 DM35424_Adc_Channel_Get_Last_Sample()	22
4.2.2.7 DM35424_Adc_Channel_Get_Thresholds()	23
4.2.2.8 DM35424_Adc_Channel_Interrupt_Clear_Status()	24
4.2.2.9 DM35424_Adc_Channel_Interrupt_Get_Config()	25
4.2.2.10 DM35424_Adc_Channel_Interrupt_Get_Status()	26
4.2.2.11 DM35424_Adc_Channel_Interrupt_Set_Config()	27
4.2.2.12 DM35424_Adc_Channel_Reset()	29
4.2.2.13 DM35424_Adc_Channel_Set_Filter()	30
4.2.2.14 DM35424_Adc_Channel_Set_High_Threshold()	31
4.2.2.15 DM35424_Adc_Channel_Set_Low_Threshold()	32
4.2.2.16 DM35424_Adc_Channel_Setup()	33
4.2.2.17 DM35424_Adc_Fifo_Channel_Read()	35
4.2.2.18 DM35424_Adc_Get_Clock_Source_Global()	36
4.2.2.19 DM35424_Adc_Get_Clock_Src()	37
4.2.2.20 DM35424_Adc_Get_Mode_Status()	38
4.2.2.21 DM35424_Adc_Get_Post_Stop_Samples()	39
4.2.2.22 DM35424_Adc_Get_Pre_Trigger_Samples()	40

4.2.2.23	DM35424_Adc_Get_Sample_Count()	41
4.2.2.24	DM35424_Adc_Get_Start_Trigger()	42
4.2.2.25	DM35424_Adc_Get_Stop_Trigger()	43
4.2.2.26	DM35424_Adc_Initialize()	44
4.2.2.27	DM35424_Adc_Interrupt_Clear_Status()	45
4.2.2.28	DM35424_Adc_Interrupt_Get_Config()	46
4.2.2.29	DM35424_Adc_Interrupt_Get_Status()	47
4.2.2.30	DM35424_Adc_Interrupt_Set_Config()	48
4.2.2.31	DM35424_Adc_Open()	49
4.2.2.32	DM35424_Adc_Pause()	50
4.2.2.33	DM35424_Adc_Reset()	50
4.2.2.34	DM35424_Adc_Sample_To_Volts()	51
4.2.2.35	DM35424_Adc_Set_Clk_Divider()	52
4.2.2.36	DM35424_Adc_Set_Clock_Source_Global()	53
4.2.2.37	DM35424_Adc_Set_Clock_Src()	54
4.2.2.38	DM35424_Adc_Set_Post_Stop_Samples()	55
4.2.2.39	DM35424_Adc_Set_Pre_Trigger_Samples()	56
4.2.2.40	DM35424_Adc_Set_Sample_Rate()	57
4.2.2.41	DM35424_Adc_Set_Start_Trigger()	58
4.2.2.42	DM35424_Adc_Set_Stop_Trigger()	59
4.2.2.43	DM35424_Adc_Start()	60
4.2.2.44	DM35424_Adc_Start_Rearm()	61
4.2.2.45	DM35424_Adc_Uninitialize()	62
4.2.2.46	DM35424_Adc_Volts_To_Sample()	63
4.3	DM35424 Board Access Structures	64
4.3.1	Detailed Description	65
4.3.2	Function Documentation	65
4.3.2.1	DM35424_Board_Close()	65
4.3.2.2	DM35424_Board_Open()	65
4.3.2.3	DM35424_Dma()	66
4.3.2.4	DM35424_Modify()	67
4.3.2.5	DM35424_Read()	68
4.3.2.6	DM35424_Write()	69
4.4	DM35424 PCI Region Structures	69
4.4.1	Detailed Description	70
4.4.2	Enumeration Type Documentation	70
4.4.2.1	DM35424_DMA_FUNCTIONS	70
4.4.2.2	dm35424_pci_region_access_size	70
4.4.2.3	dm35424_pci_region_num	71
4.5	DM35424 DAC Library Constants	71
4.5.1	Detailed Description	72
4.5.2	Enumeration Type Documentation	72

4.5.2.1 DM35424_Dac_Clock_Events	72
4.6 DM35424 DAC Library Public Functions	73
4.6.1 Detailed Description	74
4.6.2 Function Documentation	75
4.6.2.1 DM35424_Dac_Channel_Clear_Marker_Status()	75
4.6.2.2 DM35424_Dac_Channel_Get_Marker_Config()	76
4.6.2.3 DM35424_Dac_Channel_Get_Marker_Status()	77
4.6.2.4 DM35424_Dac_Channel_Set_Marker_Config()	78
4.6.2.5 DM35424_Dac_Conv_To_Volts()	79
4.6.2.6 DM35424_Dac_Fifo_Channel_Write()	79
4.6.2.7 DM35424_Dac_Get_Clock_Div()	80
4.6.2.8 DM35424_Dac_Get_Clock_Src()	81
4.6.2.9 DM35424_Dac_Get_Conversion_Count()	82
4.6.2.10 DM35424_Dac_Get_Last_Conversion()	83
4.6.2.11 DM35424_Dac_Get_Mode_Status()	84
4.6.2.12 DM35424_Dac_Get_Post_Stop_Conversion_Count()	85
4.6.2.13 DM35424_Dac_Get_Start_Trigger()	86
4.6.2.14 DM35424_Dac_Get_Stop_Trigger()	87
4.6.2.15 DM35424_Dac_Interrupt_Clear_Status()	88
4.6.2.16 DM35424_Dac_Interrupt_Get_Config()	89
4.6.2.17 DM35424_Dac_Interrupt_Get_Status()	90
4.6.2.18 DM35424_Dac_Interrupt_Set_Config()	91
4.6.2.19 DM35424_Dac_Open()	92
4.6.2.20 DM35424_Dac_Pause()	93
4.6.2.21 DM35424_Dac_Reset()	93
4.6.2.22 DM35424_Dac_Set_Clock_Div()	94
4.6.2.23 DM35424_Dac_Set_Clock_Source_Global()	95
4.6.2.24 DM35424_Dac_Set_Clock_Src()	96
4.6.2.25 DM35424_Dac_Set_Conversion_Rate()	97
4.6.2.26 DM35424_Dac_Set_Last_Conversion()	98
4.6.2.27 DM35424_Dac_Set_Post_Stop_Conversion_Count()	100
4.6.2.28 DM35424_Dac_Set_Start_Trigger()	101
4.6.2.29 DM35424_Dac_Set_Stop_Trigger()	102
4.6.2.30 DM35424_Dac_Start()	103
4.6.2.31 DM35424_Dac_Volts_To_Conv()	103
4.7 DM35424 DIO Public	104
4.7.1 Detailed Description	105
4.7.2 Function Documentation	105
4.7.2.1 DM35424_Dio_Get_Direction()	105
4.7.2.2 DM35424_Dio_Get_Input_Value()	106
4.7.2.3 DM35424_Dio_Get_Output_Value()	107
4.7.2.4 DM35424_Dio_Open()	108

4.7.2.5 DM35424_Dio_Set_Direction()	109
4.7.2.6 DM35424_Dio_Set_Output_Value()	110
4.8 DM35424 DMA Public Library Constants	111
4.8.1 Detailed Description	112
4.8.2 Enumeration Type Documentation	112
4.8.2.1 DM35424_Fifo_States	112
4.9 DM35424 DMA Public Library Functions	113
4.9.1 Detailed Description	114
4.9.2 Function Documentation	114
4.9.2.1 DM35424_Dma_Buffer_Setup()	114
4.9.2.2 DM35424_Dma_Buffer_Status()	116
4.9.2.3 DM35424_Dma_Check_Buffer_Used()	117
4.9.2.4 DM35424_Dma_Check_For_Error()	118
4.9.2.5 DM35424_Dma_Clear()	120
4.9.2.6 DM35424_Dma_Clear_Interrupt()	121
4.9.2.7 DM35424_Dma_Configure_Interrupts()	122
4.9.2.8 DM35424_Dma_Find_Interrupt()	124
4.9.2.9 DM35424_Dma_Get_Current_Buffer_Count()	125
4.9.2.10 DM35424_Dma_Get_Errors()	126
4.9.2.11 DM35424_Dma_Get_Fifo_Counts()	128
4.9.2.12 DM35424_Dma_Get_Fifo_State()	129
4.9.2.13 DM35424_Dma_Get_Interrupt_Configuration()	130
4.9.2.14 DM35424_Dma_Pause()	131
4.9.2.15 DM35424_Dma_Reset_Buffer()	132
4.9.2.16 DM35424_Dma_Setup()	134
4.9.2.17 DM35424_Dma_Setup_Set_Direction()	135
4.9.2.18 DM35424_Dma_Setup_Set_Used()	136
4.9.2.19 DM35424_Dma_Start()	137
4.9.2.20 DM35424_Dma_Status()	138
4.9.2.21 DM35424_Dma_Stop()	141
4.10 DM35424 Driver Constants	142
4.10.1 Detailed Description	142
4.11 DM35424 Driver Enumerations	142
4.11.1 Detailed Description	142
4.11.2 Enumeration Type Documentation	142
4.11.2.1 dm35424_pci_region_access_dir	142
4.12 DM35424 Driver Structures	143
4.12.1 Detailed Description	143
4.13 DM35424 Example Programs Constants	143
4.13.1 Detailed Description	145
4.13.2 Enumeration Type Documentation	145
4.13.2.1 Help_Options	145

4.14 DM35424 Board Macros	147
4.14.1 Detailed Description	147
4.14.2 Macro Definition Documentation	147
4.14.2.1 CLK_40MHZ	147
4.15 DM35424 Board Library Public Functions	147
4.15.1 Detailed Description	148
4.15.2 Function Documentation	148
4.15.2.1 DM35424_Function_Block_Open()	148
4.15.2.2 DM35424_Function_Block_Open_Module()	149
4.15.2.3 DM35424_Gbc_Ack_Interrupt()	150
4.15.2.4 DM35424_Gbc_Board_Reset()	150
4.15.2.5 DM35424_Gbc_Get_Format()	151
4.15.2.6 DM35424_Gbc_Get_Fpga_Build()	152
4.15.2.7 DM35424_Gbc_Get_Pdp_Number()	152
4.15.2.8 DM35424_Gbc_Get_Revision()	153
4.15.2.9 DM35424_Gbc_Get_Sys_Clock_Freq()	154
4.16 DM35424 ioctl macros	155
4.16.1 Detailed Description	155
4.17 DM35424 Board Access Public Library Functions	155
4.17.1 Detailed Description	156
4.17.2 Function Documentation	156
4.17.2.1 DM35424_Dma_Initialize()	156
4.17.2.2 DM35424_Dma_Read()	157
4.17.2.3 DM35424_Dma_Write()	159
4.17.2.4 DM35424_General_InstallISR()	160
4.17.2.5 DM35424_General_RemoveISR()	161
4.17.2.6 DM35424_General_SetISRPriority()	162
4.17.2.7 DM35424_General_WaitForInterrupt()	162
4.18 DM35424 Reference Adjustment Library Constants	163
4.18.1 Detailed Description	164
4.18.2 Enumeration Type Documentation	164
4.18.2.1 DM35424_Copy_Directions	164
4.19 DM35424 Reference Adjustment Public Library Functions	164
4.19.1 Detailed Description	165
4.19.2 Function Documentation	165
4.19.2.1 DM35424_Ref_Adjust_Copy_Data()	165
4.19.2.2 DM35424_Ref_Adjust_Open()	166
4.19.2.3 DM35424_Ref_Adjust_Write_Adc_To_NonVolatile()	167
4.19.2.4 DM35424_Ref_Adjust_Write_Adc_To_Volatile()	168
4.19.2.5 DM35424_Ref_Adjust_Write_Dac_To_NonVolatile()	169
4.19.2.6 DM35424_Ref_Adjust_Write_Dac_To_Volatile()	170
4.20 DM35424 Register Offsets	171

4.20.1 Detailed Description	175
4.20.2 Macro Definition Documentation	175
4.20.2.1 DM35424_OFFSET_FB_ADC_FIFO	175
4.21 DM35424 Temperature	175
4.21.1 Detailed Description	175
4.21.2 Function Documentation	175
4.21.2.1 DM35424_Temperature_Open()	175
4.21.2.2 DM35424_Temperature_Read()	176
4.22 DM35424 Board Types	177
4.22.1 Detailed Description	179
4.23 DM35424 Utility Library Functions	179
4.23.1 Detailed Description	179
4.23.2 Enumeration Type Documentation	179
4.23.2.1 DM35424_Waveforms	179
4.23.3 Function Documentation	180
4.23.3.1 DM35424_Check_Result()	180
4.23.3.2 DM35424_Generate_Signal_Data()	180
4.23.3.3 DM35424_Get_Maskable()	182
4.23.3.4 DM35424_Get_Time_Diff()	183
4.23.3.5 DM35424_Micro_Sleep()	183
5 Data Structure Documentation	185
5.1 DM35424_Board_Descriptor Struct Reference	185
5.1.1 Detailed Description	185
5.1.2 Field Documentation	185
5.1.2.1 file_descriptor	185
5.1.2.2 isr	186
5.1.2.3 pid	186
5.2 dm35424_device_descriptor Struct Reference	186
5.2.1 Detailed Description	187
5.2.2 Field Documentation	187
5.2.2.1 cdev	187
5.2.2.2 device_id	187
5.2.2.3 device_index	187
5.2.2.4 device_lock	187
5.2.2.5 dma_descr_list	187
5.2.2.6 dma_wait_queue	188
5.2.2.7 int_queue_count	188
5.2.2.8 int_queue_in_marker	188
5.2.2.9 int_queue_missed	188
5.2.2.10 int_queue_out_marker	188
5.2.2.11 int_wait_queue	188

5.2.2.12 interrupt_fb	189
5.2.2.13 irq_number	189
5.2.2.14 name	189
5.2.2.15 pci	189
5.2.2.16 pdev	189
5.2.2.17 reference_count	189
5.2.2.18 remove_isr_flag	190
5.3 DM35424_DMA_Descriptor Struct Reference	190
5.3.1 Detailed Description	190
5.3.2 Field Documentation	190
5.3.2.1 buffer_start_offset	190
5.3.2.2 control_offset	190
5.3.2.3 num_buffers	191
5.4 dm35424_dma_descriptor Struct Reference	191
5.4.1 Detailed Description	191
5.4.2 Field Documentation	191
5.4.2.1 buffer	191
5.4.2.2 buffer_size	192
5.4.2.3 bus_addr	192
5.4.2.4 channel	192
5.4.2.5 fb_num	192
5.4.2.6 list	192
5.4.2.7 virt_addr	192
5.5 DM35424_Function_Block Struct Reference	193
5.5.1 Detailed Description	193
5.5.2 Field Documentation	193
5.5.2.1 control_offset	193
5.5.2.2 dma_channel	193
5.5.2.3 dma_offset	194
5.5.2.4 fb_num	194
5.5.2.5 fb_offset	194
5.5.2.6 num_dma_buffers	194
5.5.2.7 num_dma_channels	194
5.5.2.8 ordinal_fb_type_num	195
5.5.2.9 sub_type	195
5.5.2.10 type	195
5.5.2.11 type_revision	195
5.6 dm35424_ioctl_argument Union Reference	195
5.6.1 Detailed Description	196
5.6.2 Field Documentation	196
5.6.2.1 dma	196
5.6.2.2 interrupt	196

5.6.2.3 modify	196
5.6.2.4 readwrite	196
5.7 dm35424_ioctl_dma Struct Reference	197
5.7.1 Detailed Description	197
5.7.2 Field Documentation	197
5.7.2.1 buffer	197
5.7.2.2 buffer_ptr	197
5.7.2.3 buffer_size	197
5.7.2.4 channel	198
5.7.2.5 fb_num	198
5.7.2.6 function	198
5.7.2.7 num_buffers	198
5.7.2.8 pci	198
5.8 dm35424_ioctl_interrupt_info_request Struct Reference	198
5.8.1 Detailed Description	199
5.8.2 Field Documentation	199
5.8.2.1 error_occurred	199
5.8.2.2 interrupt_fb	199
5.8.2.3 interrupts_remaining	199
5.8.2.4 valid_interrupt	200
5.9 dm35424_ioctl_region_modify Struct Reference	200
5.9.1 Detailed Description	200
5.9.2 Field Documentation	200
5.9.2.1 access	200
5.9.2.2 [union]	201
5.9.2.3 mask16	201
5.9.2.4 mask32	201
5.9.2.5 mask8	201
5.10 dm35424_ioctl_region_readwrite Struct Reference	201
5.10.1 Detailed Description	202
5.10.2 Field Documentation	202
5.10.2.1 access	202
5.11 dm35424_pci_access_request Struct Reference	202
5.11.1 Detailed Description	202
5.11.2 Field Documentation	203
5.11.2.1 [union]	203
5.11.2.2 data16	203
5.11.2.3 data32	203
5.11.2.4 data8	203
5.11.2.5 offset	203
5.11.2.6 region	203
5.11.2.7 size	204

5.12 dm35424_pci_region Struct Reference	204
5.12.1 Detailed Description	204
5.12.2 Field Documentation	204
5.12.2.1 allocated	204
5.12.2.2 io_addr	204
5.12.2.3 length	205
5.12.2.4 phys_addr	205
5.12.2.5 virt_addr	205
6 File Documentation	207
6.1 examples/_non_public/dm35424_adc_test.c File Reference	207
6.1.1 Detailed Description	208
6.1.2 Macro Definition Documentation	209
6.1.2.1 BUFFER_SIZE_BYTES	209
6.1.2.2 BUFFER_SIZE_SAMPLES	209
6.1.2.3 DAT_FILE_NAME_PREFIX	209
6.1.2.4 DAT_FILE_NAME_SUFFIX	209
6.1.2.5 DEFAULT_RATE	209
6.1.3 Function Documentation	210
6.1.3.1 ISR()	210
6.1.3.2 main()	210
6.1.3.3 output_channel_status()	211
6.1.3.4 sigint_handler()	212
6.1.4 Variable Documentation	213
6.1.4.1 board	213
6.1.4.2 buffer_count	213
6.1.4.3 buffer_size_bytes	213
6.1.4.4 dma_has_error	213
6.1.4.5 exit_program	213
6.1.4.6 local_buffer	214
6.1.4.7 my_adc	214
6.1.4.8 program_name	214
6.2 dm35424_adc_test.c	214
6.3 examples/_non_public/dm35424_board_checkout.c File Reference	227
6.3.1 Detailed Description	228
6.3.2 Function Documentation	229
6.3.2.1 run_test_9()	229
6.3.2.2 sigint_handler()	229
6.3.3 Variable Documentation	230
6.3.3.1 program_name	230
6.4 dm35424_board_checkout.c	230
6.5 examples/_non_public/dm35424_calibrate.c File Reference	322

6.5.1 Detailed Description	322
6.5.2 Function Documentation	323
6.5.2.1 main()	323
6.5.2.2 sigint_handler()	324
6.5.3 Variable Documentation	325
6.5.3.1 exit_program	325
6.5.3.2 program_name	325
6.6 dm35424_calibrate.c	325
6.7 examples/_non_public/dm35424_oven_test.c File Reference	336
6.7.1 Detailed Description	336
6.7.2 Function Documentation	337
6.7.2.1 sigint_handler()	337
6.7.3 Variable Documentation	337
6.7.3.1 program_name	337
6.8 dm35424_oven_test.c	338
6.9 examples/_non_public/dm35424_ref_adjust_test.c File Reference	358
6.9.1 Detailed Description	359
6.9.2 Function Documentation	359
6.9.2.1 main()	359
6.9.2.2 sigint_handler()	360
6.9.3 Variable Documentation	361
6.9.3.1 exit_program	361
6.9.3.2 program_name	361
6.10 dm35424_ref_adjust_test.c	361
6.11 examples/dm35424_adc.c File Reference	370
6.11.1 Detailed Description	371
6.11.2 Macro Definition Documentation	372
6.11.2.1 BUFFER_SIZE_BYTES	372
6.11.2.2 BUFFER_SIZE_SAMPLES	372
6.11.2.3 DAC_RATE	372
6.11.2.4 DEFAULT_RATE	372
6.11.3 Function Documentation	373
6.11.3.1 main()	373
6.11.3.2 sigint_handler()	374
6.11.4 Variable Documentation	374
6.11.4.1 exit_program	374
6.11.4.2 interrupt_count	374
6.11.4.3 program_name	375
6.12 dm35424_adc.c	375
6.13 examples/dm35424_adc_continuous_dma.c File Reference	388
6.13.1 Detailed Description	389
6.13.2 Macro Definition Documentation	390

6.13.2.1 ASCII_FILE_NAME	390
6.13.2.2 BIN_FILE_NAME	390
6.13.2.3 BUFFER_SIZE_BYTES	390
6.13.2.4 BUFFER_SIZE_SAMPLES	390
6.13.2.5 DEFAULT_RATE	391
6.13.3 Function Documentation	391
6.13.3.1 convert_bin_to_txt()	391
6.13.3.2 ISR()	391
6.13.3.3 main()	392
6.13.3.4 output_channel_status()	393
6.13.3.5 setup_adc()	393
6.13.3.6 setup_ctrlc_handler()	394
6.13.3.7 setup_dacs_and_start()	394
6.13.3.8 sigint_handler()	395
6.13.4 Variable Documentation	395
6.13.4.1 board	395
6.13.4.2 buffer_count	396
6.13.4.3 buffer_size_bytes	396
6.13.4.4 dma_has_error	396
6.13.4.5 exit_program	396
6.13.4.6 local_buffer	396
6.13.4.7 my_adc	396
6.13.4.8 next_buffer	397
6.13.4.9 program_name	397
6.14 dm35424_adc_continuous_dma.c	397
6.15 examples/dm35424_dac.c File Reference	409
6.15.1 Detailed Description	410
6.15.2 Macro Definition Documentation	410
6.15.2.1 DEFAULT_DAC_NUM	410
6.15.3 Function Documentation	411
6.15.3.1 main()	411
6.15.4 Variable Documentation	411
6.15.4.1 program_name	411
6.16 dm35424_dac.c	412
6.17 examples/dm35424_dac_dma.c File Reference	417
6.17.1 Detailed Description	418
6.17.2 Macro Definition Documentation	419
6.17.2.1 BUFFER_SIZE_BYTES	419
6.17.2.2 BUFFER_SIZE_SAMPLES	419
6.17.2.3 DEFAULT_DAC_TO_USE	419
6.17.2.4 DEFAULT_RATE	419
6.17.2.5 NUM_BUFFERS_TO_USE	419

6.17.3 Function Documentation	420
6.17.3.1 main()	420
6.17.3.2 sigint_handler()	421
6.17.4 Variable Documentation	421
6.17.4.1 exit_program	421
6.17.4.2 program_name	421
6.18 dm35424_dac_dma.c	422
6.19 examples/dm35424_dio.c File Reference	428
6.19.1 Detailed Description	429
6.19.2 Macro Definition Documentation	430
6.19.2.1 DM35424_DIO_DIRECTION	430
6.19.3 Function Documentation	430
6.19.3.1 main()	430
6.19.4 Variable Documentation	431
6.19.4.1 board	431
6.19.4.2 my_dio	431
6.19.4.3 program_name	431
6.20 dm35424_dio.c	431
6.21 examples/dm35424_list_fb.c File Reference	434
6.21.1 Detailed Description	434
6.21.2 Function Documentation	435
6.21.2.1 main()	435
6.21.3 Variable Documentation	435
6.21.3.1 program_name	435
6.22 dm35424_list_fb.c	436
6.23 examples/dm35424_ref_adjust.c File Reference	439
6.23.1 Detailed Description	440
6.23.2 Macro Definition Documentation	441
6.23.2.1 DEFAULT_RATE	441
6.23.2.2 MAX_REF_ADJUST	441
6.23.3 Function Documentation	441
6.23.3.1 getch()	441
6.23.3.2 keyboard_hit()	441
6.23.3.3 main()	442
6.23.3.4 sigint_handler()	443
6.23.4 Variable Documentation	443
6.23.4.1 exit_program	443
6.23.4.2 program_name	443
6.24 dm35424_ref_adjust.c	444
6.25 examples/dm35424_temperature.c File Reference	450
6.25.1 Detailed Description	450
6.25.2 Macro Definition Documentation	451

6.25.2.1 TEMP_FB_TO_OPEN	451
6.25.3 Function Documentation	451
6.25.3.1 main()	451
6.25.3.2 sigint_handler()	452
6.25.4 Variable Documentation	452
6.25.4.1 exit_program	452
6.25.4.2 program_name	452
6.26 dm35424_temperature.c	453
6.27 include/dm35424.h File Reference	455
6.27.1 Detailed Description	456
6.28 dm35424.h	456
6.29 include/dm35424_adc_library.h File Reference	457
6.29.1 Detailed Description	463
6.30 dm35424_adc_library.h	463
6.31 include/dm35424_board_access.h File Reference	470
6.31.1 Detailed Description	471
6.31.2 Macro Definition Documentation	471
6.31.2.1 DM35424LIB_API	471
6.32 dm35424_board_access.h	472
6.33 include/dm35424_board_access_structs.h File Reference	473
6.33.1 Detailed Description	474
6.34 dm35424_board_access_structs.h	474
6.35 include/dm35424_dac_library.h File Reference	476
6.35.1 Detailed Description	479
6.36 dm35424_dac_library.h	480
6.37 include/dm35424_dio_library.h File Reference	483
6.37.1 Detailed Description	484
6.38 dm35424_dio_library.h	484
6.39 include/dm35424_dma_library.h File Reference	485
6.39.1 Detailed Description	488
6.40 dm35424_dma_library.h	488
6.41 include/dm35424_driver.h File Reference	492
6.41.1 Detailed Description	493
6.42 dm35424_driver.h	493
6.43 include/dm35424_examples.h File Reference	495
6.43.1 Detailed Description	497
6.44 dm35424_examples.h	497
6.45 include/dm35424_gbc_library.h File Reference	500
6.45.1 Detailed Description	501
6.46 dm35424_gbc_library.h	501
6.47 include/dm35424_ioctl.h File Reference	502
6.47.1 Detailed Description	503

6.48 dm35424_ioctl.h	503
6.49 include/dm35424_os.h File Reference	504
6.49.1 Detailed Description	505
6.50 dm35424_os.h	505
6.51 include/dm35424_ref_adjust_library.h File Reference	506
6.51.1 Detailed Description	508
6.52 dm35424_ref_adjust_library.h	508
6.53 include/dm35424_registers.h File Reference	509
6.53.1 Detailed Description	513
6.54 dm35424_registers.h	514
6.55 include/dm35424_temperature_library.h File Reference	517
6.55.1 Detailed Description	517
6.56 dm35424_temperature_library.h	518
6.57 include/dm35424_types.h File Reference	518
6.57.1 Detailed Description	520
6.57.2 Enumeration Type Documentation	520
6.57.2.1 DM35424_Clock_Buses	520
6.57.2.2 DM35424_Clock_Sources	521
6.58 dm35424_types.h	522
6.59 include/dm35424_util_library.h File Reference	523
6.59.1 Detailed Description	524
6.60 dm35424_util_library.h	524
Index	527

Chapter 1

Topic Index

1.1 Topics

Here is a list of all topics with brief descriptions:

DM35424 ADC Library Constants	7
DM35424 ADC Public Library Functions	13
DM35424 Board Access Structures	64
DM35424 PCI Region Structures	69
DM35424 DAC Library Constants	71
DM35424 DAC Library Public Functions	73
DM35424 DIO Public	104
DM35424 DMA Public Library Constants	111
DM35424 DMA Public Library Functions	113
DM35424 Driver Constants	142
DM35424 Driver Enumerations	142
DM35424 Driver Structures	143
DM35424 Example Programs Constants	143
DM35424 Board Macros	147
DM35424 Board Library Public Functions	147
DM35424 ioctl macros	155
DM35424 Board Access Public Library Functions	155
DM35424 Reference Adjustment Library Constants	163
DM35424 Reference Adjustment Public Library Functions	164
DM35424 Register Offsets	171
DM35424 Temperature	175
DM35424 Board Types	177
DM35424 Utility Library Functions	179

Chapter 2

Data Structure Index

2.1 Data Structures

Here are the data structures with brief descriptions:

DM35424_Board_Descriptor	DM35424 board descriptor. This structure holds information about the board as a whole. It holds the file descriptor and ISR callback function, if applicable	185
dm35424_device_descriptor	DM35424 Device Descriptor. The identifying info for this particular board	186
DM35424_DMA_Descriptor	Descriptor for the DMA on this board	190
dm35424_dma_descriptor	DM35424 DMA descriptor. This structure holds information about a single DMA buffer	191
DM35424_Function_Block	DM35424 function block descriptor. This structure holds information about a function block, including type, number of DMA channels and buffers, descriptors for each DMA channel, and memory offsets to various control locations	193
dm35424_ioctl_argument	ioctl() request structure encapsulating all possible requests. This is what gets passed into the kernel from user space on the ioctl() call	195
dm35424_ioctl_dma	ioctl() request structure for DMA	197
dm35424_ioctl_interrupt_info_request	ioctl() request structure for interrupt	198
dm35424_ioctl_region_modify	ioctl() request structure for PCI region read/modify/write	200
dm35424_ioctl_region_readwrite	ioctl() request structure for read from or write to PCI region	201
dm35424_pci_access_request	PCI region access request descriptor. This structure holds information about a request to read data from or write data to one of a device's PCI regions	202
dm35424_pci_region	DM35424 PCI region descriptor. This structure holds information about one of a device's PCI memory regions	204

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

examples/dm35424_adc.c	Example program which demonstrates the use of the ADC, setting and responding to interrupts	370
examples/dm35424_adc_continuous_dma.c	Example program which demonstrates the use of the ADC and DMA	388
examples/dm35424_dac.c	Example program which demonstrates the use of the DAC	409
examples/dm35424_dac_dma.c	Example program which demonstrates the use of the DAC and DMA	417
examples/dm35424_dio.c	Example program which demonstrates the use of the DIO	428
examples/dm35424_list_fb.c	Example program which demonstrates use of the library to open a function block for use	434
examples/dm35424_ref_adjust.c	Example program which demonstrates the reference voltage adjustment function blocks	439
examples/dm35424_temperature.c	Example program for read the temperature sensor	450
examples/_non_public/dm35424_adc_test.c	Example program which demonstrates the use of the ADC and DMA	207
examples/_non_public/dm35424_board_checkout.c	Example program which checks out the functionality of all of the basic parts of the board	227
examples/_non_public/dm35424_calibrate.c	Example program which demonstrates the reference voltage adjustment function blocks	322
examples/_non_public/dm35424_oven_test.c		336
examples/_non_public/dm35424_ref_adjust_test.c	Example program which demonstrates the use of the digital potentiometer, to adjust the reference voltage of the DACs and ADCs on the board	358
include/dm35424.h	Defines for the DM35424 (Device-specific values)	455
include/dm35424_adc_library.h	Definitions for the DM35424 ADC Library	457
include/dm35424_board_access.h	Structures for the DM35424 Board Access Library	470
include/dm35424_board_access_structs.h	Structures for the DM35424 Board Access Library	473
include/dm35424_dac_library.h	Definitions for the DM35424 DAC Library	476

include/dm35424_dio_library.h	
Definitions for the DM35424 DIO Library	483
include/dm35424_dma_library.h	
Definitions for the DM35424 DMA Library	485
include/dm35424_driver.h	
Structures and defines for the DM35424 driver module	492
include/dm35424_examples.h	
Defines for the DM35424 Example programs. Commonly used constants for the example programs included with the software package	495
include/dm35424_gbc_library.h	
Definitions for the DM35424 Board Library, a library for accessing the board registers	500
include/dm35424_ioctl.h	
DM35424 Low level ioctl() request descriptor structure and request code definitions	502
include/dm35424_os.h	
Function declarations for the DM35424 that are Linux specific	504
include/dm35424_ref_adjust_library.h	
Definitions for the DM35424 Reference Adjustment Library	506
include/dm35424_registers.h	
Defines for the DM35424 Registers (Offsets)	509
include/dm35424_temperature_library.h	
Definitions for the DM35424 Temperature Library	517
include/dm35424_types.h	
Defines for the DM35424. Values for the general board, not specific to a particular function block	518
include/dm35424_util_library.h	
Definitions for the DM35424 Utilities library, various helper functions	523

Chapter 4

Topic Documentation

4.1 DM35424 ADC Library Constants

Macros

- **#define DM35424_ADC_MODE_RESET** 0x00
Register value for ADC Mode Reset.
- **#define DM35424_ADC_MODE_PAUSE** 0x01
Register value for ADC Mode Pause.
- **#define DM35424_ADC_MODE_GO_SINGLE_SHOT** 0x02
Register value for ADC Mode Go (Single Shot).
- **#define DM35424_ADC_MODE_GO_REARM** 0x03
Register value for ADC Mode Go (Rearm after Stop).
- **#define DM35424_ADC_MODE_UNINITIALIZED** 0x04
Register value for ADC Mode Uninitialized.
- **#define DM35424_ADC_STAT_STOPPED** 0x00
Register value for ADC Status - Stopped.
- **#define DM35424_ADC_STAT_FILLING_PRE_TRIG_BUFF** 0x01
Register value for ADC Status - Filling Pre-Start Buffer.
- **#define DM35424_ADC_STAT_WAITING_START_TRIG** 0x02
Register value for ADC Status - Waiting for Start Trigger.
- **#define DM35424_ADC_STAT_SAMPLING** 0x03
Register value for ADC Status - Sampling Data.
- **#define DM35424_ADC_STAT_FILLING_POST_TRIG_BUFF** 0x04
Register value for ADC Status - Filling Post-Stop Buffer.
- **#define DM35424_ADC_STAT_WAIT_REARM** 0x05
Register value for ADC Status - Wait for Rearm.
- **#define DM35424_ADC_STAT_DONE** 0x07
Register value for ADC Status - Done.
- **#define DM35424_ADC_STAT_UNINITIALIZED** 0x08
Register value for ADC Status - Uninitialized.
- **#define DM35424_ADC_STAT_INITIALIZING** 0x09
Register value for ADC Status - Initializing.
- **#define DM35424_ADC_INT_SAMPLE_TAKEN_MASK** 0x01
Register value for Interrupt Mask - Sample Taken.
- **#define DM35424_ADC_INT_CHAN_THRESHOLD_MASK** 0x02

- Register value for Interrupt Mask - Channel Threshold Exceeded.*

 - **#define DM35424_ADC_INT_PRE_BUFF_FULL_MASK** 0x04
- Register value for Interrupt Mask - Pre-Start Buffer Filled.*

 - **#define DM35424_ADC_INT_START_TRIG_MASK** 0x08
- Register value for Interrupt Mask - Start Trigger Occurred.*

 - **#define DM35424_ADC_INT_STOP_TRIG_MASK** 0x10
- Register value for Interrupt Mask - Stop Trigger Occurred.*

 - **#define DM35424_ADC_INT_POST_BUFF_FULL_MASK** 0x20
- Register value for Interrupt Mask - Post-Stop Buffer Filled.*

 - **#define DM35424_ADC_INT_SAMP_COMPL_MASK** 0x40
- Register value for Interrupt Mask - Sampling Complete.*

 - **#define DM35424_ADC_INT_PACER_TICK_MASK** 0x80
- Register value for Interrupt Mask - Pacer Clock Tick Occurred.*

 - **#define DM35424_ADC_INT_ALL_MASK** 0xFF
- Register value for Interrupt Mask - All Bits.*

 - **#define DM35424_ADC_CHAN_INTR_LOW_THRESHOLD_MASK** 0x01
- Register value for Channel Low Threshold Interrupt.*

 - **#define DM35424_ADC_CHAN_INTR_HIGH_THRESHOLD_MASK** 0x02
- Register value for Channel High Threshold Interrupt.*

 - **#define DM35424_ADC_CHAN_FILTER_ORDER0** 0x0
- Register value for Channel Filter Order 0.*

 - **#define DM35424_ADC_CHAN_FILTER_ORDER1** 0x1
- Register value for Channel Filter Order 1.*

 - **#define DM35424_ADC_CHAN_FILTER_ORDER2** 0x2
- Register value for Channel Filter Order 2.*

 - **#define DM35424_ADC_CHAN_FILTER_ORDER3** 0x3
- Register value for Channel Filter Order 3.*

 - **#define DM35424_ADC_CHAN_FILTER_ORDER4** 0x4
- Register value for Channel Filter Order 4.*

 - **#define DM35424_ADC_CHAN_FILTER_ORDER5** 0x5
- Register value for Channel Filter Order 5.*

 - **#define DM35424_ADC_CHAN_FILTER_ORDER6** 0x6
- Register value for Channel Filter Order 6.*

 - **#define DM35424_ADC_CHAN_FILTER_ORDER7** 0x7
- Register value for Channel Filter Order 7.*

 - **#define DM35424_ADC_FE_CONFIG_POWER_ACTIVE** 0x80
- Register value for setting channel power to active.*

 - **#define DM35424_ADC_FE_CONFIG_PGA_ACTIVE** 0x40
- Register value for setting channel PGA to active.*

 - **#define DM35424_ADC_FE_CONFIG_IN_SWITCH_ENABLED** 0x20
- Register value for setting channel input switch to enabled.*

 - **#define DM35424_ADC_FE_CONFIG_DAC_LOOPBACK** 0x00
- Register value for measuring between DACx Chy and VREF (2.5V).*

 - **#define DM35424_ADC_FE_CONFIG_SNGL_END_POS** 0x08
- Register value for measuring InP - VREF (singled ended).*

 - **#define DM35424_ADC_FE_CONFIG_SNGL_END_NEG** 0x10
- Register value for measuring VREF - InN (singled ended).*

 - **#define DM35424_ADC_FE_CONFIG_DIFFERENTIAL** 0x18
- Register value for setting channel to In(Positive) - In(Negative) connection. This is the most common.*

 - **#define DM35424_ADC_FE_CONFIG_GAIN_1** 0x00
- Register value for setting a Gain of 1.*

- **#define DM35424_ADC_FE_CONFIG_GAIN_2** 0x04
Register value for setting a Gain of 2.
- **#define DM35424_ADC_FE_CONFIG_GAIN_4** 0x02
Register value for setting a Gain of 4.
- **#define DM35424_ADC_FE_CONFIG_GAIN_8** 0x06
Register value for setting a Gain of 8.
- **#define DM35424_ADC_FE_CONFIG_GAIN_16** 0x01
Register value for setting a Gain of 16.
- **#define DM35424_ADC_FE_CONFIG_GAIN_32** 0x05
Register value for setting a Gain of 32.
- **#define DM35424_ADC_FE_CONFIG_GAIN_64** 0x03
Register value for setting a Gain of 64.
- **#define DM35424_ADC_FE_CONFIG_GAIN_128** 0x07
Register value for setting a Gain of 128.
- **#define DM35424_ADC_FE_CONFIG_POWER_MASK** 0x80
Bit mask for the channel power bit of the FE Config.
- **#define DM35424_ADC_FE_CONFIG_PGA_MASK** 0x40
Bit mask for the channel PGA bit of the FE Config.
- **#define DM35424_ADC_FE_CONFIG_INPUT_SW_ENABLE_MASK** 0x20
Bit mask for the channel input switch bit of the FE Config.
- **#define DM35424_ADC_FE_CONFIG_INPUT_LINE_MASK** 0x18
Bit mask for the channel input line bits of the FE Config.
- **#define DM35424_ADC_FE_CONFIG_GAIN_MASK** 0x07
Bit mask for the channel gain bits of the FE Config.
- **#define DM35424_ADC_THRESHOLD_MAX** 8388607L
Maximum allowable value to write to the threshold register.
- **#define DM35424_ADC_THRESHOLD_MIN** -8388608L
Minimum allowable value to write to the threshold register.
- **#define DM35424_ADC_HIGH_SPD_MIN_DIV** 2
Minimum divider allowed for HIGH SPEED mode.
- **#define DM35424_ADC_HIGH_RES_MIN_DIV** 2
Minimum divider allowed for HIGH RES mode.
- **#define DM35424_ADC_LOW_POW_MIN_DIV** 4
Minimum divider allowed for LOW POWER mode.
- **#define DM35424_ADC_LOW_SPD_MIN_DIV** 10
Minimum divider allowed for LOW SPEED mode.
- **#define DM35424_ADC_MAX_RATE** 106000
Max rate of the ADC (Hz).
- **#define DM35424_ADC_MAX_VALUE** 8388607
Max possible value for ADC.
- **#define DM35424_ADC_MIN_VALUE** -8388608
Min possible value for ADC.

Enumerations

- **enum DM35424_Adc_Clock_Events** {
DM35424_ADC_CLK_BUS_SRC_DISABLE = 0x00 , **DM35424_ADC_CLK_BUS_SRC_SAMPLE_TAKEN** =
0x80 , **DM35424_ADC_CLK_BUS_SRC_CHAN_THRESH** = 0x81 , **DM35424_ADC_CLK_BUS_SRC_PRE_START_BUFF_FU**
= 0x82 ,
DM35424_ADC_CLK_BUS_SRC_START_TRIG = 0x83 , **DM35424_ADC_CLK_BUS_SRC_STOP_TRIG** =
0x84 , **DM35424_ADC_CLK_BUS_SRC_POST_STOP_BUFF_FULL** = 0x85 , **DM35424_ADC_CLK_BUS_SRC_SAMPLING_C**
= 0x86 ,
DM35424_ADC_CLK_BUS_SRC_PACER_TICK = 0x87 }

Clock events for the global source clocks.

- enum `DM35424_Input_Ranges` {
`DM35424_ADC_RNG_BIPOLAR_2_5V`, `DM35424_ADC_RNG_BIPOLAR_1_25V`, `DM35424_ADC_RNG_BIPOLAR_625mV`,
`DM35424_ADC_RNG_BIPOLAR_312mV`,
`DM35424_ADC_RNG_BIPOLAR_156mV`, `DM35424_ADC_RNG_BIPOLAR_78mV`, `DM35424_ADC_RNG_BIPOLAR_39mV`,
`DM35424_ADC_RNG_BIPOLAR_19mV`,
`DM35424_ADC_RNG_UNIPOLAR_5V` }

Input range of the ADC input pin. This combines polarity and gain into a single enumeration, and is the preferred way of setting polarity and gain.

- enum `DM35424_Input_Mode` { `DM35424_ADC_INPUT_DIFFERENTIAL`, `DM35424_ADC_INPUT_SINGLE_ENDED_POS`,
`DM35424_ADC_INPUT_SINGLE_ENDED_NEG`, `DM35424_ADC_INPUT_DAC_LOOPBACK` }

Input mode of the ADC pin.

- enum `DM35424_Gains` {
`DM35424_ADC_GAIN_05`, `DM35424_ADC_GAIN_1`, `DM35424_ADC_GAIN_2`, `DM35424_ADC_GAIN_4`,
`DM35424_ADC_GAIN_8`, `DM35424_ADC_GAIN_16`, `DM35424_ADC_GAIN_32`, `DM35424_ADC_GAIN_64`,
`DM35424_ADC_GAIN_128` }

Input gain to apply to the incoming signal. Note that the preferred method of setting the gain is through the input range enumeration.

- enum `DM35424_Sampling_Mode` { `DM35424_ADC_MODE_CONFIG_HIGH_SPEED` =0x01, `DM35424_ADC_MODE_CONFIG_HIGH_SPEED` =0x03, `DM35424_ADC_MODE_CONFIG_LOW_POWER` =0x04, `DM35424_ADC_MODE_CONFIG_LOW_SPEED` =0x06 }

Sampling mode for the AD Config Register.

4.1.1 Detailed Description

4.1.2 Enumeration Type Documentation

4.1.2.1 DM35424_Adc_Clock_Events

enum `DM35424_Adc_Clock_Events`

Clock events for the global source clocks.

Enumerator

<code>DM35424_ADC_CLK_BUS_SRC_DISABLE</code>	Register value for Clock Event - Disabled.
<code>DM35424_ADC_CLK_BUS_SRC_SAMPLE_TAKEN</code>	Register value for Clock Event - Sample Taken.
<code>DM35424_ADC_CLK_BUS_SRC_CHAN_THRESH</code>	Register value for Clock Event - Channel Threshold Exceeded.
<code>DM35424_ADC_CLK_BUS_SRC_PRE_START_BUFF_FULL</code>	Register value for Clock Event - Pre-Start Buffer Full.
<code>DM35424_ADC_CLK_BUS_SRC_START_TRIG</code>	Register value for Clock Event - Start Trigger Occurred.
<code>DM35424_ADC_CLK_BUS_SRC_STOP_TRIG</code>	Register value for Clock Event - Stop Trigger Occurred.
<code>DM35424_ADC_CLK_BUS_SRC_POST_STOP_BUFF_FULL</code>	Register value for Clock Event - Post-Stop Buffer Full.

DM35424_ADC_CLK_BUS_SRC_SAMPLING_COMPLETE	Register value for Clock Event - Sampling Complete.
DM35424_ADC_CLK_BUS_SRC_PACER_TICK	Register value for Clock Event - Pacer Tick Occurred.

Definition at line [425](#) of file [dm35424_adc_library.h](#).

4.1.2.2 DM35424_Gains

```
enum DM35424_Gains
```

Input gain to apply to the incoming signal. Note that the preferred method of setting the gain is through the input range enumeration.

Note

Not all values in this enumeration may apply to your board, as this is a shared library. Please consult the board manual for legal values.

Enumerator

DM35424_ADC_GAIN_05	Input Half-Gain
DM35424_ADC_GAIN_1	Input Gain of 1
DM35424_ADC_GAIN_2	Input Gain of 2
DM35424_ADC_GAIN_4	Input Gain of 4
DM35424_ADC_GAIN_8	Input Gain of 8
DM35424_ADC_GAIN_16	Input Gain of 16
DM35424_ADC_GAIN_32	Input Gain of 32
DM35424_ADC_GAIN_64	Input Gain of 64
DM35424_ADC_GAIN_128	Input Gain of 128

Definition at line [591](#) of file [dm35424_adc_library.h](#).

4.1.2.3 DM35424_Input_Mode

```
enum DM35424_Input_Mode
```

Input mode of the ADC pin.

Note

Not all values in this enumeration may apply to your board,as this is a shared library. Please consult the board manual for valid values.

Enumerator

DM35424_ADC_INPUT_DIFFERENTIAL	Differential Operation
DM35424_ADC_INPUT_SINGLE_ENDED_POS	Single-Ended operation, input connected to positive ADC input.
DM35424_ADC_INPUT_SINGLE_ENDED_NEG	Single-Ended operation, input connected to negative ADC input.
DM35424_ADC_INPUT_DAC_LOOPBACK	DAC Output internally looped-back to ADC input. See hardware manual for more info.

Definition at line 553 of file [dm35424_adc_library.h](#).

4.1.2.4 DM35424_Input_Ranges

enum [DM35424_Input_Ranges](#)

Input range of the ADC input pin. This combines polarity and gain into a single enumeration, and is the preferred way of setting polarity and gain.

Note

Not all values in this enumeration may apply to your board,as this is a shared library. Please consult the board manual for legal values.

Enumerator

DM35424_ADC_RNG_BIPOLAR_2_5V	Bipolar Mode, -2.5 to 2.5 V
DM35424_ADC_RNG_BIPOLAR_1_25V	Bipolar Mode, -1.25 to 1.25 V
DM35424_ADC_RNG_BIPOLAR_625mV	Bipolar Mode, -625 mV to 625 mV
DM35424_ADC_RNG_BIPOLAR_312mV	Bipolar Mode, -312.5 mV to 312.5 mV
DM35424_ADC_RNG_BIPOLAR_156mV	Bipolar Mode, -156.25 mV to 156.25 mV
DM35424_ADC_RNG_BIPOLAR_78mV	Bipolar Mode, -78.125 mV to 78.125 mV
DM35424_ADC_RNG_BIPOLAR_39mV	Bipolar Mode, -39.0626 mV to 39.0626 mV
DM35424_ADC_RNG_BIPOLAR_19mV	Bipolar Mode, -19.53125 mV to 19.53125 mV
DM35424_ADC_RNG_UNIPOLAR_5V	Unipolar Mode, 0 to 5 V

Definition at line 494 of file [dm35424_adc_library.h](#).

4.1.2.5 DM35424_Sampling_Mode

enum [DM35424_Sampling_Mode](#)

Sampling mode for the AD Config Register.

Enumerator

DM35424_ADC_MODE_CONFIG_HIGH_SPEED	Register value for ADC AD Config - High Speed.
DM35424_ADC_MODE_CONFIG_HIGH_RES	Register value for ADC AD Config - High Resolution.
DM35424_ADC_MODE_CONFIG_LOW_POWER	Register value for ADC AD Config - Low Power.
DM35424_ADC_MODE_CONFIG_LOW_SPEED	Register value for ADC AD Config - Low Speed.

Definition at line 645 of file [dm35424_adc_library.h](#).

4.2 DM35424 ADC Public Library Functions

Functions

- [DM35424LIB_API](#) int [DM35424_Adc_Open](#) (struct [DM35424_Board_Descriptor](#) *handle, unsigned int number_of_type, struct [DM35424_Function_Block](#) *func_block)
Open the ADC indicated, and determine register locations of control blocks needed to control it.
- [DM35424LIB_API](#) int [DM35424_Adc_Get_Start_Trigger](#) (struct [DM35424_Board_Descriptor](#) *handle, struct [DM35424_Function_Block](#) *func_block, uint8_t *start_trigger)
Get the start trigger for data collection.
- [DM35424LIB_API](#) int [DM35424_Adc_Set_Start_Trigger](#) (struct [DM35424_Board_Descriptor](#) *handle, struct [DM35424_Function_Block](#) *func_block, uint8_t start_trigger)
Set the start trigger for data collection.
- [DM35424LIB_API](#) int [DM35424_Adc_Get_Stop_Trigger](#) (struct [DM35424_Board_Descriptor](#) *handle, struct [DM35424_Function_Block](#) *func_block, uint8_t *stop_trigger)
Get the stop trigger for data collection.
- [DM35424LIB_API](#) int [DM35424_Adc_Set_Stop_Trigger](#) (struct [DM35424_Board_Descriptor](#) *handle, struct [DM35424_Function_Block](#) *func_block, uint8_t stop_trigger)
Set the stop trigger for data collection.
- [DM35424LIB_API](#) int [DM35424_Adc_Get_Pre_Trigger_Samples](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t *count)
Get the amount of data to capture prior to start trigger.
- [DM35424LIB_API](#) int [DM35424_Adc_Set_Pre_Trigger_Samples](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t count)
Set the amount of data to capture prior to start trigger.
- [DM35424LIB_API](#) int [DM35424_Adc_Get_Post_Stop_Samples](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t *count)
Get the amount of data to capture after stop trigger.
- [DM35424LIB_API](#) int [DM35424_Adc_Set_Post_Stop_Samples](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t count)
Set the amount of data to capture after stop trigger.
- [DM35424LIB_API](#) int [DM35424_Adc_Get_Clock_Src](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, enum [DM35424_Clock_Sources](#) *source)
Get the clock source for the ADC.

- [DM35424LIB_API](#) [int](#) [DM35424_Adc_Set_Clock_Src](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, enum [DM35424_Clock_Sources](#) source)
Set the clock source for the ADC.
- [DM35424LIB_API](#) [int](#) [DM35424_Adc_Initialize](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block)
Prepare the ADC for actual data collection. Moves the ADC from uninitialized to stopped.
- [DM35424LIB_API](#) [int](#) [DM35424_Adc_Set_Clk_Divider](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t divider)
Set the Clock Divider for the ADC function block.
- [DM35424LIB_API](#) [int](#) [DM35424_Adc_Set_Sample_Rate](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t rate, uint32_t *actual_rate)
Set the sampling rate for the ADC.
- [DM35424LIB_API](#) [int](#) [DM35424_Adc_Channel_Get_Front_End_Config](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, uint16_t *fe_config)
Get the front-end config register contents.
- [DM35424LIB_API](#) [int](#) [DM35424_Adc_Ad_Config_Set_Mode](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, enum [DM35424_Sampling_Mode](#) mode)
Set the Configuration of the AD Mode.
- [DM35424LIB_API](#) [int](#) [DM35424_Adc_Ad_Config_Get_Mode](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint16_t *mode)
Get AD Config register.
- [DM35424LIB_API](#) [int](#) [DM35424_Adc_Interrupt_Set_Config](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint16_t int_source, int enable)
Configure the interrupts for the ADC.
- [DM35424LIB_API](#) [int](#) [DM35424_Adc_Interrupt_Get_Config](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint16_t *interrupt_ena)
Get the interrupt configuration for the ADC.
- [DM35424LIB_API](#) [int](#) [DM35424_Adc_Start](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block)
Set the ADC mode to Start.
- [DM35424LIB_API](#) [int](#) [DM35424_Adc_Start_Rearm](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block)
Set the ADC mode to Start-Rearm.
- [DM35424LIB_API](#) [int](#) [DM35424_Adc_Reset](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block)
Set the ADC mode to Reset.
- [DM35424LIB_API](#) [int](#) [DM35424_Adc_Pause](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block)
Set the ADC mode to Pause.
- [DM35424LIB_API](#) [int](#) [DM35424_Adc_Uninitialize](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block)
Set the ADC mode to Uninitialized.
- [DM35424LIB_API](#) [int](#) [DM35424_Adc_Get_Mode_Status](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint8_t *mode_status)
Get the ADC mode-status value.
- [DM35424LIB_API](#) [int](#) [DM35424_Adc_Channel_Get_Last_Sample](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, int32_t *value)
Get the last sample taken from the ADC.
- [DM35424LIB_API](#) [int](#) [DM35424_Adc_Get_Sample_Count](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t *value)
Get the count of number of samples taken.
- [DM35424LIB_API](#) [int](#) [DM35424_Adc_Interrupt_Get_Status](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint16_t *value)

Get the interrupt status register.

- **DM35424LIB_API** int **DM35424_Adc_Interrupt_Clear_Status** (struct **DM35424_Board_Descriptor** *handle, const struct **DM35424_Function_Block** *func_block, uint16_t value)

Clear the interrupt status register.

- **DM35424LIB_API** int **DM35424_Adc_Channel_Setup** (struct **DM35424_Board_Descriptor** *handle, struct **DM35424_Function_Block** *func_block, unsigned int channel, enum **DM35424_Input_Ranges** input_range, enum **DM35424_Input_Mode** input_mode)

Setup the channel input for the ADC.

- **DM35424LIB_API** int **DM35424_Adc_Channel_Reset** (struct **DM35424_Board_Descriptor** *handle, const struct **DM35424_Function_Block** *func_block, unsigned int channel)

Reset the channel front-end config.

- **DM35424LIB_API** int **DM35424_Adc_Channel_Interrupt_Set_Config** (struct **DM35424_Board_Descriptor** *handle, const struct **DM35424_Function_Block** *func_block, unsigned int channel, uint8_t interrupts_to_↵ set, int enable)

Setup the channel interrupts.

- **DM35424LIB_API** int **DM35424_Adc_Channel_Interrupt_Get_Config** (struct **DM35424_Board_Descriptor** *handle, const struct **DM35424_Function_Block** *func_block, unsigned int channel, uint8_t *chan_intr_↵ enable)

Get the channel interrupt configuration.

- **DM35424LIB_API** int **DM35424_Adc_Channel_Interrupt_Get_Status** (struct **DM35424_Board_Descriptor** *handle, const struct **DM35424_Function_Block** *func_block, unsigned int channel, uint8_t *chan_intr_↵ status)

Get the channel interrupt status.

- **DM35424LIB_API** int **DM35424_Adc_Channel_Interrupt_Clear_Status** (struct **DM35424_Board_Descriptor** *handle, const struct **DM35424_Function_Block** *func_block, unsigned int channel, uint8_t chan_intr_status)

Clear the interrupt status for this channel.

- **DM35424LIB_API** int **DM35424_Adc_Channel_Find_Interrupt** (struct **DM35424_Board_Descriptor** *handle, const struct **DM35424_Function_Block** *func_block, unsigned int *channel_with_interrupt, int *channel_↵ has_interrupt, uint8_t *channel_intr_status, uint8_t *channel_intr_enable)

Find the first channel with an interrupt. Note that this is only useful when looking for a threshold interrupt.

- **DM35424LIB_API** int **DM35424_Adc_Channel_Set_Filter** (struct **DM35424_Board_Descriptor** *handle, const struct **DM35424_Function_Block** *func_block, unsigned int channel, uint8_t chan_filter)

Set the filter value for the channel.

- **DM35424LIB_API** int **DM35424_Adc_Channel_Get_Filter** (struct **DM35424_Board_Descriptor** *handle, const struct **DM35424_Function_Block** *func_block, unsigned int channel, uint8_t *chan_filter)

Get the filter value for the channel.

- **DM35424LIB_API** int **DM35424_Adc_Channel_Set_Low_Threshold** (struct **DM35424_Board_Descriptor** *handle, const struct **DM35424_Function_Block** *func_block, unsigned int channel, int32_t threshold)

Set the lower threshold for this channel.

- **DM35424LIB_API** int **DM35424_Adc_Channel_Set_High_Threshold** (struct **DM35424_Board_Descriptor** *handle, const struct **DM35424_Function_Block** *func_block, unsigned int channel, int32_t threshold)

Set the high threshold for this channel.

- **DM35424LIB_API** int **DM35424_Adc_Channel_Get_Thresholds** (struct **DM35424_Board_Descriptor** *handle, const struct **DM35424_Function_Block** *func_block, unsigned int channel, int32_t *low_threshold, int32_t *high_threshold)

Get both thresholds for this channel.

- **DM35424LIB_API** int **DM35424_Adc_Fifo_Channel_Read** (struct **DM35424_Board_Descriptor** *handle, const struct **DM35424_Function_Block** *func_block, unsigned int channel, int32_t *value)

Read an ADC sample stored in the onboard FIFO.

- **DM35424LIB_API** int **DM35424_Adc_Set_Clock_Source_Global** (struct **DM35424_Board_Descriptor** *handle, const struct **DM35424_Function_Block** *func_block, enum **DM35424_Clock_Sources** clock_↵ select, enum **DM35424_Adc_Clock_Events** clock_driver)

Set the global clock source for the ADC.

- [DM35424LIB_API](#) `int DM35424_Adc_Get_Clock_Source_Global` (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, int clock_select, int *clock_source)
Get the global clock source for the selected clock.
- [DM35424LIB_API](#) `int DM35424_Adc_Sample_To_Volts` (enum [DM35424_Input_Ranges](#) input_range, int32_t adc_sample, float *volts)
Convert an ADC sample to a volts value.
- [DM35424LIB_API](#) `int DM35424_Adc_Volts_To_Sample` (enum [DM35424_Input_Ranges](#) input_range, float volts, int32_t *adc_sample)
Convert volts to an ADC value.

4.2.1 Detailed Description

DM35424_Adc_Library_Constants

4.2.2 Function Documentation

4.2.2.1 DM35424_Adc_Ad_Config_Get_Mode()

```
DM35424LIB_API int DM35424_Adc_Ad_Config_Get_Mode (
    struct DM35424\_Board\_Descriptor * handle,
    const struct DM35424\_Function\_Block * func_block,
    uint16_t * mode)
```

Get AD Config register.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>mode</i>	
-------------	--

Returned AD_Config register value

Note

This library function will only apply to some ADC subtypes, and may not be applicable to the DM35424.

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

Failure.

errno may be set as follows:

- `EINVAL` Invalid mode requested, or does not apply to this ADC. `ENODEV` Attempted to set for an invalid ADC

References [DM35424LIB_API](#).

4.2.2.2 DM35424_Adc_Ad_Config_Set_Mode()

```
DM35424LIB_API int DM35424_Adc_Ad_Config_Set_Mode (
    struct DM35424_Board_Descriptor * handle,
    const struct DM35424_Function_Block * func_block,
    enum DM35424_Sampling_Mode mode)
```

Set the Configuration of the AD Mode.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>mode</i>	
-------------	--

Mode to set for the AD Config

Note

This library function will only apply to some ADC subtypes, and may not be applicable to the DM35424.

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

Failure.

errno may be set as follows:

- `EINVAL` Invalid mode requested, or does not apply to this ADC. `ENODEV` Attempted to set for an invalid ADC

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), and [setup_adc\(\)](#).

4.2.2.3 DM35424_Adc_Channel_Find_Interrupt()

```
DM35424LIB_API int DM35424_Adc_Channel_Find_Interrupt (
    struct DM35424_Board_Descriptor * handle,
    const struct DM35424_Function_Block * func_block,
    unsigned int * channel_with_interrupt,
    int * channel_has_interrupt,
    uint8_t * channel_intr_status,
    uint8_t * channel_intr_enable)
```

Find the first channel with an interrupt. Note that this is only useful when looking for a threshold interrupt.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>channel_with_interrupt</i>	
-------------------------------	--

Pointer to the returned channel that has an interrupt (if any)

Parameters

<i>channel_has_interrupt</i>	
------------------------------	--

Pointer to boolean indicating whether returned channel has interrupt or not.

Parameters

<i>channel_intr_status</i>	
----------------------------	--

Pointer to the channel's interrupt status.

Parameters

<i>channel_intr_enable</i>	
----------------------------	--

Pointer to the channel's interrupt enable register.

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

Failure.

References [DM35424LIB_API](#).

4.2.2.4 DM35424_Adc_Channel_Get_Filter()

```
DM35424LIB_API int DM35424_Adc_Channel_Get_Filter (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,
```

```
unsigned int channel,  
uint8_t * chan_filter)
```

Get the filter value for the channel.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>channel</i>	
----------------	--

Channel to get filter of

Parameters

<i>chan_filter</i>	
--------------------	--

Pointer to returned channel filter value. Reference the user's manual for valid filter values.

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

Failure.

errno may be set as follows:

- `EINVAL` Invalid channel requested.

References [DM35424LIB_API](#).

4.2.2.5 DM35424_Adc_Channel_Get_Front_End_Config()

```
DM35424LIB_API int DM35424_Adc_Channel_Get_Front_End_Config (
    struct DM35424_Board_Descriptor * handle,
    const struct DM35424_Function_Block * func_block,
    unsigned int channel,
    uint16_t * fe_config)
```

Get the front-end config register contents.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>channel</i>	
----------------	--

Channel the FE Config is requested for.

Parameters

<i>fe_config</i>	
------------------	--

Pointer to the returned FE Config register value

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

Failure.

errno may be set as follows:

- `EINVAL` Invalid channel requested.

References [DM35424LIB_API](#).

4.2.2.6 DM35424_Adc_Channel_Get_Last_Sample()

```
DM35424LIB_API int DM35424_Adc_Channel_Get_Last_Sample (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    unsigned int channel,  
    int32_t * value)
```

Get the last sample taken from the ADC.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>channel</i>	
----------------	--

Channel to get sample from.

Parameters

<i>value</i>	
--------------	--

Pointer to returned sample value.

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), and [run_test_9\(\)](#).

4.2.2.7 DM35424_Adc_Channel_Get_Thresholds()

```
DM35424LIB_API int DM35424_Adc_Channel_Get_Thresholds (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    unsigned int channel,  
    int32_t * low_threshold,  
    int32_t * high_threshold)
```

Get both thresholds for this channel.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>channel</i>	
----------------	--

Channel to set the high threshold of.

Parameters

<i>low_threshold</i>	
----------------------	--

Pointer to signed integer value of threshold.

Parameters

<i>high_threshold</i>	
-----------------------	--

Pointer to signed integer value of threshold.

Note

The threshold register is only 16-bits. Thus, the threshold value really only represents the top 16-bits of the 24-bit ADC value. For convenience, the threshold parameters are returned as 32-bit integers. After getting the value from the register, it will be left-shifted 16-bits.

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

Failure.

errno may be set as follows:

- `EINVAL` Invalid channel requested.

References [DM35424LIB_API](#).

4.2.2.8 DM35424_Adc_Channel_Interrupt_Clear_Status()

```
DM35424LIB_API int DM35424_Adc_Channel_Interrupt_Clear_Status (
    struct DM35424_Board_Descriptor * handle,
    const struct DM35424_Function_Block * func_block,
    unsigned int channel,
    uint8_t chan_intr_status)
```

Clear the interrupt status for this channel.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>channel</i>	
----------------	--

Channel to clear.

Parameters

<i>chan_intr_status</i>	
-------------------------	--

Bit mask indicating which interrupts to clear.

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

Failure.

errno may be set as follows:

- `EINVAL` Invalid channel requested.

References [DM35424LIB_API](#).

Referenced by [run_test_9\(\)](#).

4.2.2.9 DM35424_Adc_Channel_Interrupt_Get_Config()

```
DM35424LIB_API int DM35424_Adc_Channel_Interrupt_Get_Config (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    unsigned int channel,  
    uint8_t * chan_intr_enable)
```

Get the channel interrupt configuration.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>channel</i>	
----------------	--

Channel to get configuration.

Parameters

<i>chan_intr_enable</i>	
-------------------------	--

Pointer to interrupt configuration being returned.

Return values

<i>0</i>	
----------	--

Success.

Return values

<i>Non-zero</i>	
-----------------	--

Failure.

errno may be set as follows:

- `EINVAL` Invalid channel requested.

References [DM35424LIB_API](#).

4.2.2.10 DM35424_Adc_Channel_Interrupt_Get_Status()

```
DM35424LIB_API int DM35424_Adc_Channel_Interrupt_Get_Status (
    struct DM35424_Board_Descriptor * handle,
    const struct DM35424_Function_Block * func_block,
    unsigned int channel,
    uint8_t * chan_intr_status)
```

Get the channel interrupt status.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>channel</i>	
----------------	--

Channel to get configuration.

Parameters

<i>chan_intr_status</i>	
-------------------------	--

Pointer to interrupt status being returned.

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

Failure.

errno may be set as follows:

- `EINVAL` Invalid channel requested.

References [DM35424LIB_API](#).

4.2.2.11 DM35424_Adc_Channel_Interrupt_Set_Config()

```
DM35424LIB_API int DM35424_Adc_Channel_Interrupt_Set_Config (
    struct DM35424_Board_Descriptor * handle,
    const struct DM35424_Function_Block * func_block,
    unsigned int channel,
    uint8_t interrupts_to_set,
    int enable)
```

Setup the channel interrupts.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>channel</i>	
----------------	--

Channel to configure.

Parameters

<i>interrupts_to_set</i>	
--------------------------	--

A bit mask indicating which interrupts to set

Parameters

<i>enable</i>	
---------------	--

A boolean value indicating if selected interrupts should be enabled or disabled.

Return values

<i>0</i>	
----------	--

Success.

Return values

<i>Non-zero</i>	
-----------------	--

Failure.

errno may be set as follows:

- `EINVAL` Invalid channel requested, or requested input mode is not possible on this ADC subtype.

References [DM35424LIB_API](#).

Referenced by [run_test_9\(\)](#).

4.2.2.12 DM35424_Adc_Channel_Reset()

```
DM35424LIB_API int DM35424_Adc_Channel_Reset (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    unsigned int channel)
```

Reset the channel front-end config.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>channel</i>	
----------------	--

Channel to reset.

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

Failure.

errno may be set as follows:

- `EINVAL` Invalid channel requested.

References [DM35424LIB_API](#).

4.2.2.13 DM35424_Adc_Channel_Set_Filter()

```
DM35424LIB_API int DM35424_Adc_Channel_Set_Filter (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    unsigned int channel,  
    uint8_t chan_filter)
```

Set the filter value for the channel.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>channel</i>	
----------------	--

Channel to set filter

Parameters

<i>chan_filter</i>	
--------------------	--

Channel filter value. Reference the user's manual for valid filter values.

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

Failure.

errno may be set as follows:

- `EINVAL` Invalid channel requested.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#).

4.2.2.14 DM35424_Adc_Channel_Set_High_Threshold()

```
DM35424LIB_API int DM35424_Adc_Channel_Set_High_Threshold (
    struct DM35424_Board_Descriptor * handle,
    const struct DM35424_Function_Block * func_block,
    unsigned int channel,
    int32_t threshold)
```

Set the high threshold for this channel.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>channel</i>	
----------------	--

Channel to set the high threshold of.

Parameters

<i>threshold</i>	
------------------	--

Signed high threshold value for this channel.

Note

The threshold register is only 16-bits. Thus, the threshold value really only represents the top 16-bits of the 24-bit ADC value. For convenience, the threshold parameter is accepted as a 32-bit integer. Before writing the value, it will be right-shifted 16 bits.

Return values

0	
---	--

Success.

Return values

<i>Non-zero</i>	
-----------------	--

Failure.

errno may be set as follows:

- `EINVAL` Invalid channel requested.

References [DM35424LIB_API](#).

Referenced by [run_test_9\(\)](#).

4.2.2.15 DM35424_Adc_Channel_Set_Low_Threshold()

```
DM35424LIB_API int DM35424_Adc_Channel_Set_Low_Threshold (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    unsigned int channel,  
    int32_t threshold)
```

Set the lower threshold for this channel.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>channel</i>	
----------------	--

Channel to set the lower threshold of.

Parameters

<i>threshold</i>	
------------------	--

Signed lower threshold value for this channel.

Note

The threshold register is only 16-bits. Thus, the threshold value really only represents the top 16-bits of the 24-bit ADC value. For convenience, the threshold parameter is accepted as a 32-bit integer. Before writing the value, it will be right-shifted 16 bits.

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

Failure.

errno may be set as follows:

- `EINVAL` Invalid channel requested.

References [DM35424LIB_API](#).

Referenced by [run_test_9\(\)](#).

4.2.2.16 DM35424_Adc_Channel_Setup()

```
DM35424LIB_API int DM35424_Adc_Channel_Setup (  
    struct DM35424_Board_Descriptor * handle,  
    struct DM35424_Function_Block * func_block,  
    unsigned int channel,  
    enum DM35424_Input_Ranges input_range,  
    enum DM35424_Input_Mode input_mode)
```

Setup the channel input for the ADC.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>channel</i>	
----------------	--

Channel to configure.

Parameters

<i>input_range</i>	
--------------------	--

An enumerated value representing the input voltage range of the input.

Parameters

<i>input_mode</i>	
-------------------	--

An enumerated value representing the mode to set the input line to.

Note

The input line mode and input voltage ranges available for the board is dependent on the ADC subtype on the board. Review the user's guide to see what values the ADC can be set to.

Return values

0	
---	--

Success.

Return values

<i>Non-zero</i>	
-----------------	--

Failure.

errno may be set as follows:

- `EINVAL` Invalid channel requested, or requested mode/range is not possible on this ADC subtype.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), and [setup_adc\(\)](#).

4.2.2.17 DM35424_Adc_Fifo_Channel_Read()

```
DM35424LIB_API int DM35424_Adc_Fifo_Channel_Read (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    unsigned int channel,  
    int32_t * value)
```

Read an ADC sample stored in the onboard FIFO.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>channel</i>	
----------------	--

Channel to get sample from.

Parameters

<i>value</i>	
--------------	--

Pointer to returned sample value.

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

Failure.

References [DM35424LIB_API](#).

4.2.2.18 DM35424_Adc_Get_Clock_Source_Global()

```
DM35424LIB_API int DM35424_Adc_Get_Clock_Source_Global (
    struct DM35424_Board_Descriptor * handle,
    const struct DM35424_Function_Block * func_block,
    int clock_select,
    int * clock_source)
```

Get the global clock source for the selected clock.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>clock_select</i>	
---------------------	--

Which global clock source to get

Parameters

<i>clock_source</i>	
---------------------	--

Pointer to the returned clock source for the selected global clock

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

Failure.

errno may be set as follows:

- `EINVAL` Invalid clock select or source..

References [DM35424LIB_API](#).

4.2.2.19 DM35424_Adc_Get_Clock_Src()

```
DM35424LIB_API int DM35424_Adc_Get_Clock_Src (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    enum DM35424_Clock_Sources * source)
```

Get the clock source for the ADC.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>source</i>	
---------------	--

Pointer to returned clock source.

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

Failure.

References [DM35424LIB_API](#).

4.2.2.20 DM35424_Adc_Get_Mode_Status()

```
DM35424LIB_API int DM35424_Adc_Get_Mode_Status (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    uint8_t * mode_status)
```

Get the ADC mode-status value.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>mode_status</i>	
--------------------	--

Pointer to the mode_status value to return.

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [run_test_9\(\)](#).

4.2.2.21 DM35424_Adc_Get_Post_Stop_Samples()

```
DM35424LIB_API int DM35424_Adc_Get_Post_Stop_Samples (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    uint32_t * count)
```

Get the amount of data to capture after stop trigger.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>count</i>	
--------------	--

Pointer to the returned count.

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

Failure.

References [DM35424LIB_API](#).

4.2.2.22 DM35424_Adc_Get_Pre_Trigger_Samples()

```
DM35424LIB_API int DM35424_Adc_Get_Pre_Trigger_Samples (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    uint32_t * count)
```

Get the amount of data to capture prior to start trigger.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>count</i>	
--------------	--

Pointer to the returned capture count

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

Failure.

References [DM35424LIB_API](#).

4.2.2.23 DM35424_Adc_Get_Sample_Count()

```
DM35424LIB_API int DM35424_Adc_Get_Sample_Count (
    struct DM35424_Board_Descriptor * handle,
    const struct DM35424_Function_Block * func_block,
    uint32_t * value)
```

Get the count of number of samples taken.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>value</i>	
--------------	--

Pointer to returned sample count.

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), and [run_test_9\(\)](#).

4.2.2.24 DM35424_Adc_Get_Start_Trigger()

```
DM35424LIB_API int DM35424_Adc_Get_Start_Trigger (  
    struct DM35424_Board_Descriptor * handle,  
    struct DM35424_Function_Block * func_block,  
    uint8_t * start_trigger)
```

Get the start trigger for data collection.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>start_trigger</i>	
----------------------	--

Pointer to the returned trigger value.

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

Failure.

References [DM35424LIB_API](#).

4.2.2.25 DM35424_Adc_Get_Stop_Trigger()

```
DM35424LIB_API int DM35424_Adc_Get_Stop_Trigger (  
    struct DM35424_Board_Descriptor * handle,  
    struct DM35424_Function_Block * func_block,  
    uint8_t * stop_trigger)
```

Get the stop trigger for data collection.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>stop_trigger</i>	
---------------------	--

Pointer to the returned trigger value

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

Failure.

References [DM35424LIB_API](#).

4.2.2.26 DM35424_Adc_Initialize()

```
DM35424LIB_API int DM35424_Adc_Initialize (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block)
```

Prepare the ADC for actual data collection. Moves the ADC from uninitialized to stopped.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Note

In many cases, several other steps have to occur before initialization is attempted, or the device will not initialize correctly or at all. Please review the user's manual for the correct steps to take.

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

Failure.

errno may be set as follows:

- `EPERM` Attempted to initialize an ADC with no active channels. `EBUSY` Device did not complete initialization in the time expected (timeout).

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), and [setup_adc\(\)](#).

4.2.2.27 DM35424_Adc_Interrupt_Clear_Status()

```
DM35424LIB_API int DM35424_Adc_Interrupt_Clear_Status (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    uint16_t value)
```

Clear the interrupt status register.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>value</i>	
--------------	--

Bit mask of which interrupts to clear.

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#).

4.2.2.28 DM35424_Adc_Interrupt_Get_Config()

```
DM35424LIB_API int DM35424_Adc_Interrupt_Get_Config (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    uint16_t * interrupt_ena)
```

Get the interrupt configuration for the ADC.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>interrupt_ena</i>	
----------------------	--

Pointer to the interrupt configuration register.

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#).

4.2.2.29 DM35424_Adc_Interrupt_Get_Status()

```
DM35424LIB_API int DM35424_Adc_Interrupt_Get_Status (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    uint16_t * value)
```

Get the interrupt status register.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>value</i>	
--------------	--

Pointer to returned interrupt status.

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#).

4.2.2.30 DM35424_Adc_Interrupt_Set_Config()

```
DM35424LIB_API int DM35424_Adc_Interrupt_Set_Config (
    struct DM35424_Board_Descriptor * handle,
    const struct DM35424_Function_Block * func_block,
    uint16_t int_source,
    int enable)
```

Configure the interrupts for the ADC.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>int_source</i>	
-------------------	--

The interrupts to configure. The bits indicate specific interrupts. Consult the user's manual for a description.

Parameters

<i>enable</i>	
---------------	--

Boolean indicating to enable or disable the selected interrupts.

Return values

<i>0</i>	
----------	--

Success.

Return values

<i>Non-zero</i>	
-----------------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#).

4.2.2.31 DM35424_Adc_Open()

```
DM35424LIB_API int DM35424_Adc_Open (  
    struct DM35424_Board_Descriptor * handle,  
    unsigned int number_of_type,  
    struct DM35424_Function_Block * func_block)
```

Open the ADC indicated, and determine register locations of control blocks needed to control it.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>number_of_type</i>	
-----------------------	--

Which ADC to open. The first ADC on the board will be 0.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Return values

0	
---	--

Success.

Return values

-1	
----	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), and [setup_adc\(\)](#).

4.2.2.32 DM35424_Adc_Pause()

```
DM35424LIB_API int DM35424_Adc_Pause (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block)
```

Set the ADC mode to Pause.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

Failure.

References [DM35424LIB_API](#).

4.2.2.33 DM35424_Adc_Reset()

```
DM35424LIB_API int DM35424_Adc_Reset (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block)
```

Set the ADC mode to Reset.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Return values

0	
---	--

Success.

Return values

<i>Non-zero</i>	
-----------------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), and [run_test_9\(\)](#).

4.2.2.34 DM35424_Adc_Sample_To_Volts()

```
DM35424LIB_API int DM35424_Adc_Sample_To_Volts (  
    enum DM35424_Input_Ranges input_range,  
    int32_t adc_sample,  
    float * volts)
```

Convert an ADC sample to a volts value.

Parameters

<i>input_range</i>	
--------------------	--

Enumerated value indicating what range the ADC channel has been set to, or NULL if the ADC does not have selectable input ranges.

Parameters

<i>adc_sample</i>	
-------------------	--

Signed value from the ADC that we want to convert to volts.

Parameters

<i>volts</i>	
--------------	--

Pointer to the returned float value in volts.

Return values

<i>0</i>	
----------	--

Success.

Return values

<i>Non-zero</i>	
-----------------	--

Failure.

errno may be set as follows:

- `ENODEV` The function block passed in is the wrong subtype

References [DM35424LIB_API](#).

Referenced by [main\(\)](#).

4.2.2.35 DM35424_Adc_Set_Clk_Divider()

```
DM35424LIB_API int DM35424_Adc_Set_Clk_Divider (
    struct DM35424_Board_Descriptor * handle,
    const struct DM35424_Function_Block * func_block,
    uint32_t divider)
```

Set the Clock Divider for the ADC function block.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>divider</i>	
----------------	--

The requested clock divider.

Return values

0	
---	--

Success.

Return values

-1	
----	--

Failure.

References [DM35424LIB_API](#).

4.2.2.36 DM35424_Adc_Set_Clock_Source_Global()

```
DM35424LIB_API int DM35424_Adc_Set_Clock_Source_Global (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    enum DM35424_Clock_Sources clock_select,  
    enum DM35424_Adc_Clock_Events clock_driver)
```

Set the global clock source for the ADC.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>clock_select</i>	
---------------------	--

Which global clock source to set

Parameters

<i>clock_driver</i>	
---------------------	--

Source to set global clock to (what is driving it?)

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

Failure.

errno may be set as follows:

- `EINVAL` Invalid clock select or source..

References [DM35424LIB_API](#).

Referenced by [run_test_9\(\)](#).

4.2.2.37 DM35424_Adc_Set_Clock_Src()

```
DM35424LIB_API int DM35424_Adc_Set_Clock_Src (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    enum DM35424_Clock_Sources source)
```

Set the clock source for the ADC.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>source</i>	
---------------	--

Clock source to use for the ADC. Consult the user's manual for the list of available sources.

Return values

0	
---	--

Success.

Return values

<i>Non-zero</i>	
-----------------	--

Failure.

errno may be set as follows:

- `EINVAL` The clock source selected is not valid.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), and [setup_adc\(\)](#).

4.2.2.38 DM35424_Adc_Set_Post_Stop_Samples()

```
DM35424LIB_API int DM35424_Adc_Set_Post_Stop_Samples (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    uint32_t count)
```

Set the amount of data to capture after stop trigger.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>count</i>	
--------------	--

Number of samples to capture after the stop trigger.

Return values

<i>0</i>	
----------	--

Success.

Return values

<i>Non-zero</i>	
-----------------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), and [run_test_9\(\)](#).

4.2.2.39 DM35424_Adc_Set_Pre_Trigger_Samples()

```
DM35424LIB_API int DM35424_Adc_Set_Pre_Trigger_Samples (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    uint32_t count)
```

Set the amount of data to capture prior to start trigger.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>count</i>	
--------------	--

Number of samples to capture prior to the start trigger.

Note

The amount of data that can be captured prior to the start trigger is limited by the size of the FIFO. Consult the user's manual for this information.

Return values

0	
---	--

Success.

Return values

<i>Non-zero</i>	
-----------------	--

Failure.

errno may be set as follows:

- `EINVAL` The size is not within the valid value range.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), and [run_test_9\(\)](#).

4.2.2.40 DM35424_Adc_Set_Sample_Rate()

```
DM35424LIB_API int DM35424_Adc_Set_Sample_Rate (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    uint32_t rate,  
    uint32_t * actual_rate)
```

Set the sampling rate for the ADC.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>rate</i>	
-------------	--

The requested sampling rate for the ADC (Hz).

Parameters

<i>actual_rate</i>	
--------------------	--

Pointer to the actual rate achieved by the ADC (Hz). Due to divider and clock values, the actual rate will rarely ever be the exact same as the requested rate.

Return values

<i>0</i>	
----------	--

Success.

Return values

<i>Non-zero</i>	
-----------------	--

Failure.

errno may be set as follows:

- `EINVAL` Asked for an invalid sampling rate (negative or 0)
- `ERANGE` Requested sampling rate is outside of the possible range for this ADC.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), and [setup_adc\(\)](#).

4.2.2.41 DM35424_Adc_Set_Start_Trigger()

```
DM35424LIB_API int DM35424_Adc_Set_Start_Trigger (
    struct DM35424_Board_Descriptor * handle,
    struct DM35424_Function_Block * func_block,
    uint8_t start_trigger)
```

Set the start trigger for data collection.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>start_trigger</i>	
----------------------	--

Trigger to start capturing values. See the hardware manual for valid trigger values.

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

Failure.

errno may be set as follows:

- `EINVAL` An invalid value was passed for a start trigger

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), [run_test_9\(\)](#), and [setup_adc\(\)](#).

4.2.2.42 DM35424_Adc_Set_Stop_Trigger()

```
DM35424LIB_API int DM35424_Adc_Set_Stop_Trigger (  
    struct DM35424_Board_Descriptor * handle,  
    struct DM35424_Function_Block * func_block,  
    uint8_t stop_trigger)
```

Set the stop trigger for data collection.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>stop_trigger</i>	
---------------------	--

Trigger to stop capturing values. See the hardware manual for valid trigger values.

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

Failure.

errno may be set as follows:

- `EINVAL` An invalid value was passed for a stop trigger

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), [run_test_9\(\)](#), and [setup_adc\(\)](#).

4.2.2.43 DM35424_Adc_Start()

```
DM35424LIB_API int DM35424_Adc_Start (
    struct DM35424_Board_Descriptor * handle,
    const struct DM35424_Function_Block * func_block)
```

Set the ADC mode to Start.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), and [run_test_9\(\)](#).

4.2.2.44 DM35424_Adc_Start_Rearm()

```
DM35424LIB_API int DM35424_Adc_Start_Rearm (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block)
```

Set the ADC mode to Start-Rearm.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Return values

0	
---	--

Success.

Return values

<i>Non-zero</i>	
-----------------	--

Failure.

References [DM35424LIB_API](#).

4.2.2.45 DM35424_Adc_Uninitialize()

```
DM35424LIB_API int DM35424_Adc_Uninitialize (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block)
```

Set the ADC mode to Uninitialized.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Return values

0	
---	--

Success.

Return values

<i>Non-zero</i>	
-----------------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#).

4.2.2.46 DM35424_Adc_Volts_To_Sample()

```
DM35424LIB_API int DM35424_Adc_Volts_To_Sample (  
    enum DM35424_Input_Ranges input_range,  
    float volts,  
    int32_t * adc_sample)
```

Convert volts to an ADC value.

Parameters

<i>input_range</i>	
--------------------	--

Enumerated value indicating what range the ADC channel has been set to

Parameters

<i>volts</i>	
--------------	--

Value to be converted to counts.

Parameters

<i>adc_sample</i>	
-------------------	--

Pointer to the returned ADC count value.

Return values

<i>0</i>	
----------	--

Success.

Return values

Non-zero	
----------	--

Failure.

errno may be set as follows:

- `ENODEV` The function block passed in is the wrong subtype

4.3 DM35424 Board Access Structures

Data Structures

- struct [DM35424_DMA_Descriptor](#)
Descriptor for the DMA on this board.
- struct [DM35424_Function_Block](#)
DM35424 function block descriptor. This structure holds information about a function block, including type, number of DMA channels and buffers, descriptors for each DMA channel, and memory offsets to various control locations.

Functions

- [DM35424LIB_API](#) int [DM35424_Board_Open](#) (uint8_t dev_num, struct [DM35424_Board_Descriptor](#) **handle)
Open the board, providing the file descriptor that all future operations will reference. Also allocate memory for the device descriptor.
- [DM35424LIB_API](#) int [DM35424_Board_Close](#) (struct [DM35424_Board_Descriptor](#) *handle)
Close the board, closing the open handle for the device file, and freeing the memory allocated for the descriptor.
- [DM35424LIB_API](#) int [DM35424_Read](#) (struct [DM35424_Board_Descriptor](#) *handle, union [dm35424_ioctl_argument](#) *ioctl_request)
Read from the board.
- [DM35424LIB_API](#) int [DM35424_Write](#) (struct [DM35424_Board_Descriptor](#) *handle, union [dm35424_ioctl_argument](#) *ioctl_request)
Write to the board.
- [DM35424LIB_API](#) int [DM35424_Modify](#) (struct [DM35424_Board_Descriptor](#) *handle, union [dm35424_ioctl_argument](#) *ioctl_request)
Read/Modify/Write to the board.
- int [DM35424_Dma](#) (struct [DM35424_Board_Descriptor](#) *handle, union [dm35424_ioctl_argument](#) *ioctl_request)
Perform a DMA operation.

4.3.1 Detailed Description

4.3.2 Function Documentation

4.3.2.1 DM35424_Board_Close()

```
DM35424LIB_API int DM35424_Board_Close (  
    struct DM35424_Board_Descriptor * handle)
```

Close the board, closing the open handle for the device file, and freeing the memory allocated for the descriptor.

Parameters

<i>handle</i>	
---------------	--

Pointer to the device descriptor, which contains the open file id.

Return values

0	
---	--

Success.

Return values

-1	
----	--

Failure. errno may be set as follows:

- ENODATA Device handle is null.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), and [run_test_9\(\)](#).

4.3.2.2 DM35424_Board_Open()

```
DM35424LIB_API int DM35424_Board_Open (  
    uint8_t dev_num,  
    struct DM35424_Board_Descriptor ** handle)
```

Open the board, providing the file descriptor that all future operations will reference. Also allocate memory for the device descriptor.

Parameters

<i>dev_num</i>	
----------------	--

The minor number of the device being opened.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Return values

0	
---	--

Success.

Return values

-1	
----	--

Failure. `errno` may be set as follows:

- `EBUSY` Cannot open specified device. `ENOMEM` Cannot allocate memory for device descriptor.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), and [run_test_9\(\)](#).

4.3.2.3 DM35424_Dma()

```
int DM35424_Dma (  
    struct DM35424_Board_Descriptor * handle,  
    union dm35424_ioctl_argument * ioctl_request)
```

Perform a DMA operation.

Parameters

<i>handle</i>	
---------------	--

Pointer to the device descriptor, which contains the open file id.

Parameters

<i>ioctl_request</i>	
----------------------	--

Structure holding all required information for request to complete, including a DMA descriptor.

Return values

0	
---	--

Success.

Return values

-1	
----	--

Failure.

Warning

This function is not compatible with the Windows driver package and is therefore not included in the Windows DLL.

4.3.2.4 DM35424_Modify()

```
DM35424LIB_API int DM35424_Modify (
    struct DM35424_Board_Descriptor * handle,
    union dm35424_ioctl_argument * ioctl_request)
```

Read/Modify/Write to the board.

Parameters

<i>handle</i>	
---------------	--

Pointer to the device descriptor, which contains the open file id.

Parameters

<i>ioctl_request</i>	
----------------------	--

Structure holding all required information for the modify to complete, including register offset, data size, PCI region number, value mask, and data to write

Return values

0	
---	--

Success.

Return values

-1	
----	--

Failure.

4.3.2.5 DM35424_Read()

```
DM35424LIB_API int DM35424_Read (  
    struct DM35424_Board_Descriptor * handle,  
    union dm35424_ioctl_argument * ioctl_request)
```

Read from the board.

Parameters

<i>handle</i>	
---------------	--

Pointer to the device descriptor, which contains the open file id.

Parameters

<i>ioctl_request</i>	
----------------------	--

Structure holding all required information for the read to complete, including register offset, data size, PCI region number, and pointer for returned data.

Return values

0	
---	--

Success.

Return values

-1	
----	--

Failure.

References [DM35424LIB_API](#).

4.3.2.6 DM35424_Write()

```
DM35424LIB_API int DM35424_Write (
    struct DM35424_Board_Descriptor * handle,
    union dm35424_ioctl_argument * ioctl_request)
```

Write to the board.

Parameters

<i>handle</i>	
---------------	--

Pointer to the device descriptor, which contains the open file id.

Parameters

<i>ioctl_request</i>	
----------------------	--

Structure holding all required information for the write to complete, including register offset, data size, PCI region number, and data to write.

Return values

0	
---	--

Success.

Return values

-1	
----	--

Failure.

References [DM35424LIB_API](#).

4.4 DM35424 PCI Region Structures

Data Structures

- struct [dm35424_pci_access_request](#)
PCI region access request descriptor. This structure holds information about a request to read data from or write data to one of a device's PCI regions.
- struct [dm35424_ioctl_region_readwrite](#)
ioctl() request structure for read from or write to PCI region
- struct [dm35424_ioctl_region_modify](#)
ioctl() request structure for PCI region read/modify/write
- struct [dm35424_ioctl_interrupt_info_request](#)
ioctl() request structure for interrupt
- struct [dm35424_ioctl_dma](#)
ioctl() request structure for DMA
- union [dm35424_ioctl_argument](#)
ioctl() request structure encapsulating all possible requests. This is what gets passed into the kernel from user space on the ioctl() call.

Enumerations

- enum `dm35424_pci_region_num` { `DM35424_PCI_REGION_GBC` = 0 , `DM35424_PCI_REGION_GBC2` , `DM35424_PCI_REGION_FB` }
Standard PCI region number.
- enum `dm35424_pci_region_access_size` { `DM35424_PCI_REGION_ACCESS_8` = 0 , `DM35424_PCI_REGION_ACCESS_16` , `DM35424_PCI_REGION_ACCESS_32` }
Desired size in bits of access to standard PCI region.
- enum `DM35424_DMA_FUNCTIONS` { `DM35424_DMA_INITIALIZE` , `DM35424_DMA_READ` , `DM35424_DMA_WRITE` }

4.4.1 Detailed Description

4.4.2 Enumeration Type Documentation

4.4.2.1 DM35424_DMA_FUNCTIONS

enum `DM35424_DMA_FUNCTIONS`

Enumeration for DMA functions that can be requested for the driver to perform.

Enumerator

<code>DM35424_DMA_INITIALIZE</code>	Initialize the DMA buffers
<code>DM35424_DMA_READ</code>	Read from the DMA buffers (transfer to user space)
<code>DM35424_DMA_WRITE</code>	Write to the DMA buffers (transfer from user space)

Definition at line 96 of file `dm35424_board_access_structs.h`.

4.4.2.2 dm35424_pci_region_access_size

enum `dm35424_pci_region_access_size`

Desired size in bits of access to standard PCI region.

Enumerator

<code>DM35424_PCI_REGION_ACCESS_8</code>	8-bit access
<code>DM35424_PCI_REGION_ACCESS_16</code>	16-bit access
<code>DM35424_PCI_REGION_ACCESS_32</code>	32-bit access

Definition at line 68 of file `dm35424_board_access_structs.h`.

4.4.2.3 dm35424_pci_region_num

enum [dm35424_pci_region_num](#)

Standard PCI region number.

Enumerator

DM35424_PCI_REGION_GBC	General Board Control Registers (BAR0)
DM35424_PCI_REGION_GBC2	General Board Control Registers (64-bit) (BAR1)
DM35424_PCI_REGION_FB	Functional Blocks Registers (BAR2)

Definition at line 40 of file [dm35424_board_access_structs.h](#).

4.5 DM35424 DAC Library Constants

Macros

- **#define DM35424_DAC_INT_CONVERSION_SENT_MASK** 0x01
Register value for Interrupt Mask - Conversion Sent.
- **#define DM35424_DAC_INT_CHAN_MARKER_MASK** 0x02
Register value for Interrupt Mask - Channel has enabled marker.
- **#define DM35424_DAC_INT_START_TRIG_MASK** 0x08
Register value for Interrupt Mask - Start Trigger Occurred.
- **#define DM35424_DAC_INT_STOP_TRIG_MASK** 0x10
Register value for Interrupt Mask - Stop Trigger Occurred.
- **#define DM35424_DAC_INT_POST_STOP_DONE_MASK** 0x20
Register value for Interrupt Mask - Post-Stop Conversions Completed.
- **#define DM35424_DAC_INT_PACER_TICK_MASK** 0x80
Register value for Interrupt Mask - Pacer Clock Tick.
- **#define DM35424_DAC_INT_ALL_MASK** 0xBB
Register value for Interrupt Mask - All Bits.
- **#define DM35424_DAC_MODE_RESET** 0x00
Register value for Mode - Reset.
- **#define DM35424_DAC_MODE_PAUSE** 0x01
Register value for Mode - Pause.
- **#define DM35424_DAC_MODE_GO_SINGLE_SHOT** 0x02
Register value for Mode - Go (Single Shot).
- **#define DM35424_DAC_MODE_GO_REARM** 0x03
Register value for Mode - Go (Re-arm).
- **#define DM35424_DAC_STATUS_STOPPED** 0x00
Register value for DAC Status - Stopped.
- **#define DM35424_DAC_STATUS_WAITING_START_TRIG** 0x02
Register value for DAC Status - Waiting for Start Trigger.
- **#define DM35424_DAC_STATUS_CONVERTING** 0x03
Register value for DAC Status - Converting Data.
- **#define DM35424_DAC_STATUS_OUTPUT_POST** 0x04
Register value for DAC Status - Outputting Post-Stop conversions.
- **#define DM35424_DAC_STATUS_WAITING_REARM** 0x05

- Register value for DAC Status - Waiting for Re-Arm.*
 - #define **DM35424_DAC_STATUS_DONE** 0x07
- Register value for DAC Status - Done.*
 - #define **DM35424_DAC_MAX_RATE** 106000
- Max allowable rate for the DAC (Hz).*
 - #define **DM35424_DAC_MAX_VALUE** 32767
- Max value of the DAC.*
 - #define **DM35424_DAC_MIN_VALUE** -32768
- Min value of the DAC.*
 - #define **DM35424_DAC_LSB_AT_MIN_RANGE** 0.000152587890625f
- DAC LSB (at lowest voltage range).*

Enumerations

- enum [DM35424_Dac_Clock_Events](#) {
[DM35424_DAC_CLK_BUS_SRC_DISABLE](#) = 0x00 , [DM35424_DAC_CLK_BUS_SRC_CONVERSION_SENT](#)
 = 0x80 , [DM35424_DAC_CLK_BUS_SRC_CHAN_MARKER](#) = 0x81 , [DM35424_DAC_CLK_BUS_SRC_START_TRIG](#)
 = 0x83 ,
[DM35424_DAC_CLK_BUS_SRC_STOP_TRIG](#) = 0x84 , [DM35424_DAC_CLK_BUS_SRC_CONV_COMPL](#)
 = 0x85 }
Clocking events that can be used as the global clock sources.

4.5.1 Detailed Description

Functions

4.5.2 Enumeration Type Documentation

4.5.2.1 DM35424_Dac_Clock_Events

enum [DM35424_Dac_Clock_Events](#)

Clocking events that can be used as the global clock sources.

Enumerator

DM35424_DAC_CLK_BUS_SRC_DISABLE	Register value for Clock Event - Disabled.
DM35424_DAC_CLK_BUS_SRC_CONVERSION_SENT	Register value for Clock Event - Conversion Sent.
DM35424_DAC_CLK_BUS_SRC_CHAN_MARKER	Register value for Clock Event - Channel has enabled marker.
DM35424_DAC_CLK_BUS_SRC_START_TRIG	Register value for Clock Event - Start Trigger Occurred.
DM35424_DAC_CLK_BUS_SRC_STOP_TRIG	Register value for Clock Event - Stop Trigger Occurred.
DM35424_DAC_CLK_BUS_SRC_CONV_COMPL	Register value for Clock Event - Conversions Complete.

Definition at line 181 of file [dm35424_dac_library.h](#).

4.6 DM35424 DAC Library Public Functions

Functions

- `DM35424LIB_API int DM35424_Dac_Open (struct DM35424_Board_Descriptor *handle, unsigned int number_of_type, struct DM35424_Function_Block *func_block)`
Open the DAC indicated, and determine register locations of control blocks needed to control it.
- `DM35424LIB_API int DM35424_Dac_Set_Clock_Src (struct DM35424_Board_Descriptor *handle, const struct DM35424_Function_Block *func_block, enum DM35424_Clock_Sources source)`
Set the clock source of the DAC.
- `DM35424LIB_API int DM35424_Dac_Get_Clock_Src (struct DM35424_Board_Descriptor *handle, const struct DM35424_Function_Block *func_block, enum DM35424_Clock_Sources *source)`
Get the clock source of the DAC.
- `DM35424LIB_API int DM35424_Dac_Get_Clock_Div (struct DM35424_Board_Descriptor *handle, const struct DM35424_Function_Block *func_block, uint32_t *divider)`
Get the clock divider value.
- `DM35424LIB_API int DM35424_Dac_Set_Clock_Div (struct DM35424_Board_Descriptor *handle, const struct DM35424_Function_Block *func_block, uint32_t divider)`
Set the clock divider value.
- `DM35424LIB_API int DM35424_Dac_Set_Conversion_Rate (struct DM35424_Board_Descriptor *handle, const struct DM35424_Function_Block *func_block, uint32_t requested_rate, uint32_t *actual_rate)`
Set the conversion rate of this DAC.
- `DM35424LIB_API int DM35424_Dac_Interrupt_Set_Config (struct DM35424_Board_Descriptor *handle, const struct DM35424_Function_Block *func_block, uint16_t interrupt_src, int enable)`
Set the interrupt configuration for this DAC.
- `DM35424LIB_API int DM35424_Dac_Interrupt_Get_Config (struct DM35424_Board_Descriptor *handle, const struct DM35424_Function_Block *func_block, uint16_t *interrupt_ena)`
Get the interrupt configuration for this DAC.
- `DM35424LIB_API int DM35424_Dac_Set_Start_Trigger (struct DM35424_Board_Descriptor *handle, const struct DM35424_Function_Block *func_block, uint8_t trigger_value)`
Set the start trigger.
- `DM35424LIB_API int DM35424_Dac_Set_Stop_Trigger (struct DM35424_Board_Descriptor *handle, const struct DM35424_Function_Block *func_block, uint8_t trigger_value)`
Set the stop trigger.
- `DM35424LIB_API int DM35424_Dac_Get_Start_Trigger (struct DM35424_Board_Descriptor *handle, const struct DM35424_Function_Block *func_block, uint8_t *trigger_value)`
Get the start trigger.
- `DM35424LIB_API int DM35424_Dac_Get_Stop_Trigger (struct DM35424_Board_Descriptor *handle, const struct DM35424_Function_Block *func_block, uint8_t *trigger_value)`
Get the stop trigger.
- `DM35424LIB_API int DM35424_Dac_Start (struct DM35424_Board_Descriptor *handle, const struct DM35424_Function_Block *func_block)`
Set the DAC Mode to Start.
- `DM35424LIB_API int DM35424_Dac_Reset (struct DM35424_Board_Descriptor *handle, const struct DM35424_Function_Block *func_block)`
Set the DAC Mode to Reset.
- `DM35424LIB_API int DM35424_Dac_Pause (struct DM35424_Board_Descriptor *handle, const struct DM35424_Function_Block *func_block)`
Set the DAC Mode to Pause.
- `DM35424LIB_API int DM35424_Dac_Get_Mode_Status (struct DM35424_Board_Descriptor *handle, const struct DM35424_Function_Block *func_block, uint8_t *mode_status)`
Get the Mode and Status of the DAC.

- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Get_Last_Conversion](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, uint8_t *marker, int16_t *value)
Get the value of the last conversion of the DAC.
- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Set_Last_Conversion](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, uint8_t marker, int16_t value)
Set a value to be converted by the DAC immediately.
- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Get_Conversion_Count](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t *value)
Get a count of the number of conversions that DAC has executed.
- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Interrupt_Get_Status](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint16_t *value)
Get a interrupt status register of the DAC.
- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Interrupt_Clear_Status](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint16_t value)
Clear the interrupt status register of the DAC.
- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Set_Post_Stop_Conversion_Count](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t value)
Set the number of conversions the DAC will make after a stop trigger.
- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Get_Post_Stop_Conversion_Count](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t *value)
Get the number of conversions the DAC will make after a stop trigger.
- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Set_Clock_Source_Global](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, enum [DM35424_Clock_Sources](#) clock, enum [DM35424_Dac_Clock_Events](#) clock_driver)
Set the source that will drive the global clock.
- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Channel_Set_Marker_Config](#) (struct [DM35424_Board_Descriptor](#) *handle, struct [DM35424_Function_Block](#) *func_block, unsigned int channel, uint8_t marker_enable)
Set the configuration of the marker interrupts for this channel.
- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Channel_Get_Marker_Config](#) (struct [DM35424_Board_Descriptor](#) *handle, struct [DM35424_Function_Block](#) *func_block, unsigned int channel, uint8_t *marker_enable)
Get the configuration of the marker interrupts for this channel.
- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Channel_Get_Marker_Status](#) (struct [DM35424_Board_Descriptor](#) *handle, struct [DM35424_Function_Block](#) *func_block, unsigned int channel, uint8_t *marker_status)
Get the status of the marker interrupts for this channel.
- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Channel_Clear_Marker_Status](#) (struct [DM35424_Board_Descriptor](#) *handle, struct [DM35424_Function_Block](#) *func_block, unsigned int channel, uint8_t marker_to_clear)
Clear the marker interrupts for this channel.
- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Fifo_Channel_Write](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, int32_t value)
Write a value to the onboard FIFO.
- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Volts_To_Conv](#) (float volts, int16_t *dac_conversion)
Convert a value in volts to a DAC equivalent signed value.
- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Conv_To_Volts](#) (int16_t conversion, float *volts)
Convert a DAC conversion value to volts.

4.6.1 Detailed Description

DM35424_Dac_Library_Constants

4.6.2 Function Documentation

4.6.2.1 DM35424_Dac_Channel_Clear_Marker_Status()

```
DM35424LIB_API int DM35424_Dac_Channel_Clear_Marker_Status (  
    struct DM35424_Board_Descriptor * handle,  
    struct DM35424_Function_Block * func_block,  
    unsigned int channel,  
    uint8_t marker_to_clear)
```

Clear the marker interrupts for this channel.

Parameters

<i>handle</i>	
---------------	--

Pointer to the board handle

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block.

Parameters

<i>channel</i>	
----------------	--

The channel to change.

Parameters

<i>marker_to_clear</i>	
------------------------	--

Bit values indicating which bits in register to clear.

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

References [DM35424LIB_API](#).

4.6.2.2 DM35424_Dac_Channel_Get_Marker_Config()

```
DM35424LIB_API int DM35424_Dac_Channel_Get_Marker_Config (  
    struct DM35424_Board_Descriptor * handle,  
    struct DM35424_Function_Block * func_block,  
    unsigned int channel,  
    uint8_t * marker_enable)
```

Get the configuration of the marker interrupts for this channel.

Parameters

<i>handle</i>	
---------------	--

Pointer to the board handle

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block.

Parameters

<i>channel</i>	
----------------	--

The channel to change.

Parameters

<i>marker_enable</i>	
----------------------	--

Pointer to returned marker interrupt config.

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

References [DM35424LIB_API](#).

4.6.2.3 DM35424_Dac_Channel_Get_Marker_Status()

```
DM35424LIB_API int DM35424_Dac_Channel_Get_Marker_Status (  
    struct DM35424_Board_Descriptor * handle,  
    struct DM35424_Function_Block * func_block,  
    unsigned int channel,  
    uint8_t * marker_status)
```

Get the status of the marker interrupts for this channel.

Parameters

<i>handle</i>	
---------------	--

Pointer to the board handle

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block.

Parameters

<i>channel</i>	
----------------	--

The channel to change.

Parameters

<i>marker_status</i>	
----------------------	--

Pointer to returned marker status.

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

References [DM35424LIB_API](#).

4.6.2.4 DM35424_Dac_Channel_Set_Marker_Config()

```
DM35424LIB_API int DM35424_Dac_Channel_Set_Marker_Config (  
    struct DM35424_Board_Descriptor * handle,  
    struct DM35424_Function_Block * func_block,  
    unsigned int channel,  
    uint8_t marker_enable)
```

Set the configuration of the marker interrupts for this channel.

Parameters

<i>handle</i>	
---------------	--

Pointer to the board handle

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block.

Parameters

<i>channel</i>	
----------------	--

The channel to change.

Parameters

<i>marker_enable</i>	
----------------------	--

Bit values indicating whether to enable marker interrupts (1) or disable (0).

Return values

0	
---	--

Success.

Return values

<i>Non-zero</i>	
-----------------	--

References [DM35424LIB_API](#).

Referenced by [run_test_9\(\)](#).

4.6.2.5 DM35424_Dac_Conv_To_Volts()

```
DM35424LIB_API int DM35424_Dac_Conv_To_Volts (  
    int16_t conversion,  
    float * volts)
```

Convert a DAC conversion value to volts.

Parameters

<i>conversion</i>	
-------------------	--

DAC converter signed value.

Parameters

<i>volts</i>	
--------------	--

The volts equivalent of the converter value.

Return values

0	
---	--

Success.

Return values

<i>Non-zero</i>	
-----------------	--

Failure.

errno may be set as follows:

- `EINVAL` Function called by an unsupported function block.

Referenced by [main\(\)](#).

4.6.2.6 DM35424_Dac_Fifo_Channel_Write()

```
DM35424LIB_API int DM35424_Dac_Fifo_Channel_Write (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    unsigned int channel,  
    int32_t value)
```

Write a value to the onboard FIFO.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

<i>channel</i>	
----------------	--

Channel to write the data to.

Parameters

<i>value</i>	
--------------	--

value to write.

Return values

0	
---	--

Success.

Return values

Non-zero	
----------	--

Failure.

References [DM35424LIB_API](#).

4.6.2.7 DM35424_Dac_Get_Clock_Div()

```
DM35424LIB_API int DM35424_Dac_Get_Clock_Div (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    uint32_t * divider)
```


Get the clock divider value.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>divider</i>	
----------------	--

Pointer to the clock divider returned.

Return values

0	
---	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

References [DM35424LIB_API](#).

4.6.2.8 DM35424_Dac_Get_Clock_Src()

```
DM35424LIB_API int DM35424_Dac_Get_Clock_Src (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    enum DM35424_Clock_Sources * source)
```

Get the clock source of the DAC.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>source</i>	
---------------	--

Pointer to the returned clock set for this DAC.

Return values

0	
---	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

References [DM35424LIB_API](#).

4.6.2.9 DM35424_Dac_Get_Conversion_Count()

```
DM35424LIB_API int DM35424_Dac_Get_Conversion_Count (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    uint32_t * value)
```

Get a count of the number of conversions that DAC has executed.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>value</i>	
--------------	--

Pointer to the returned count of conversions executed.

Return values

0	
---	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [run_test_9\(\)](#).

4.6.2.10 DM35424_Dac_Get_Last_Conversion()

```
DM35424LIB_API int DM35424_Dac_Get_Last_Conversion (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    unsigned int channel,  
    uint8_t * marker,  
    int16_t * value)
```

Get the value of the last conversion of the DAC.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>channel</i>	
----------------	--

DAC channel that we want the last conversion value from.

Parameters

<i>marker</i>	
---------------	--

Pointer to the returned value for the marker byte.

Parameters

<i>value</i>	
--------------	--

Pointer to the returned signed value of the last conversion.

Return values

0	
---	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#).

4.6.2.11 DM35424_Dac_Get_Mode_Status()

```
DM35424LIB_API int DM35424_Dac_Get_Mode_Status (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    uint8_t * mode_status)
```

Get the Mode and Status of the DAC.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>mode_status</i>	
--------------------	--

Pointer to the value of the returned mode_status register.

Return values

0	
---	--

Success.

Return values

Non-Zero	
----------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [run_test_9\(\)](#).

4.6.2.12 DM35424_Dac_Get_Post_Stop_Conversion_Count()

```
DM35424LIB_API int DM35424_Dac_Get_Post_Stop_Conversion_Count (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    uint32_t * value)
```

Get the number of conversions the DAC will make after a stop trigger.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>value</i>	
--------------	--

Pointer to the returned number of conversions.

Return values

0	
---	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

References [DM35424LIB_API](#).

4.6.2.13 DM35424_Dac_Get_Start_Trigger()

```
DM35424LIB_API int DM35424_Dac_Get_Start_Trigger (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    uint8_t * trigger_value)
```

Get the start trigger.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>trigger_value</i>	
----------------------	--

Pointer to the returned trigger value (event) that will initiate conversions on this DAC.

Return values

0	
---	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

References [DM35424LIB_API](#).

4.6.2.14 DM35424_Dac_Get_Stop_Trigger()

```
DM35424LIB_API int DM35424_Dac_Get_Stop_Trigger (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    uint8_t * trigger_value)
```

Get the stop trigger.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>trigger_value</i>	
----------------------	--

Pointer to the returned trigger value (event) that will halt conversions on this DAC.

Return values

0	
---	--

Success.

Return values

Non-Zero	
----------	--

Failure.

References [DM35424LIB_API](#).

4.6.2.15 DM35424_Dac_Interrupt_Clear_Status()

```
DM35424LIB_API int DM35424_Dac_Interrupt_Clear_Status (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    uint16_t value)
```

Clear the interrupt status register of the DAC.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>value</i>	
--------------	--

Bitmask indicating which interrupts to clear.

Return values

0	
---	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

References [DM35424LIB_API](#).

4.6.2.16 DM35424_Dac_Interrupt_Get_Config()

```
DM35424LIB_API int DM35424_Dac_Interrupt_Get_Config (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    uint16_t * interrupt_ena)
```

Get the interrupt configuration for this DAC.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>interrupt_ena</i>	
----------------------	--

Pointer to the returned bitmask indicating which interrupts are set.

Return values

<i>0</i>	
----------	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

References [DM35424LIB_API](#).

4.6.2.17 DM35424_Dac_Interrupt_Get_Status()

```
DM35424LIB_API int DM35424_Dac_Interrupt_Get_Status (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    uint16_t * value)
```

Get a interrupt status register of the DAC.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>value</i>	
--------------	--

Pointer to the returned interrupt status.

Return values

0	
---	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

References [DM35424LIB_API](#).

4.6.2.18 DM35424_Dac_Interrupt_Set_Config()

```
DM35424LIB_API int DM35424_Dac_Interrupt_Set_Config (
    struct DM35424_Board_Descriptor * handle,
    const struct DM35424_Function_Block * func_block,
    uint16_t interrupt_src,
    int enable)
```

Set the interrupt configuration for this DAC.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>interrupt_src</i>	
----------------------	--

Bitmask indicating which interrupts to set.

Parameters

<i>enable</i>	
---------------	--

A boolean value indicating whether selected interrupts are to be enabled or disabled.

Return values

0	
---	--

Success.

Return values

Non-Zero	
----------	--

Failure.

References [DM35424LIB_API](#).

4.6.2.19 DM35424_Dac_Open()

```
DM35424LIB_API int DM35424_Dac_Open (  
    struct DM35424_Board_Descriptor * handle,  
    unsigned int number_of_type,  
    struct DM35424_Function_Block * func_block)
```

Open the DAC indicated, and determine register locations of control blocks needed to control it.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>number_of_type</i>	
-----------------------	--

Which DAC to open. The first DAC on the board will be 0.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Return values

0	
---	--

Success.

Return values

Non-Zero	
----------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), and [setup_dacs_and_start\(\)](#).

4.6.2.20 DM35424_Dac_Pause()

```
DM35424LIB_API int DM35424_Dac_Pause (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block)
```

Set the DAC Mode to Pause.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Return values

0	
---	--

Success.

Return values

Non-Zero	
----------	--

Failure.

References [DM35424LIB_API](#).

4.6.2.21 DM35424_Dac_Reset()

```
DM35424LIB_API int DM35424_Dac_Reset (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block)
```

Set the DAC Mode to Reset.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Return values

0	
---	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), and [run_test_9\(\)](#).

4.6.2.22 DM35424_Dac_Set_Clock_Div()

```
DM35424LIB_API int DM35424_Dac_Set_Clock_Div (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    uint32_t divider)
```

Set the clock divider value.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>divider</i>	
----------------	--

Divider value to set this DAC clock to.

Return values

0	
---	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [run_test_9\(\)](#).

4.6.2.23 DM35424_Dac_Set_Clock_Source_Global()

```
DM35424LIB_API int DM35424_Dac_Set_Clock_Source_Global (
    struct DM35424_Board_Descriptor * handle,
    const struct DM35424_Function_Block * func_block,
    enum DM35424_Clock_Sources clock,
    enum DM35424_Dac_Clock_Events clock_driver)
```

Set the source that will drive the global clock.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>clock</i>	
--------------	--

Which global clock to set.

Parameters

<i>clock_driver</i>	
---------------------	--

Source to drive global clock.

Return values

0	
---	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure. `errno` may be set as follows:

- `EINVAL` Invalid clock or source requested.

References [DM35424LIB_API](#).

Referenced by [run_test_9\(\)](#).

4.6.2.24 DM35424_Dac_Set_Clock_Src()

```
DM35424LIB_API int DM35424_Dac_Set_Clock_Src (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    enum DM35424_Clock_Sources source)
```

Set the clock source of the DAC.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>source</i>	
---------------	--

The clock source that we want to set for this DAC.

Return values

<i>0</i>	
----------	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), [run_test_9\(\)](#), and [setup_dacs_and_start\(\)](#).

4.6.2.25 DM35424_Dac_Set_Conversion_Rate()

```
DM35424LIB_API int DM35424_Dac_Set_Conversion_Rate (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    uint32_t requested_rate,  
    uint32_t * actual_rate)
```

Set the conversion rate of this DAC.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>requested_rate</i>	
-----------------------	--

Requested rate of conversion for the DAC (Hz).

Parameters

<i>actual_rate</i>	
--------------------	--

Pointer to the returned value of the actual rate achieved (Hz).

Note

The actual obtainable rate depends on many board-specific values and clocks, and so the returned rate will rarely be the exact same as the requested rate.

Return values

0	
---	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

errno may be set as follows:

- `EINVAL` Invalid rate requested.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), and [setup_dacs_and_start\(\)](#).

4.6.2.26 DM35424_Dac_Set_Last_Conversion()

```
DM35424LIB_API int DM35424_Dac_Set_Last_Conversion (
    struct DM35424_Board_Descriptor * handle,
    const struct DM35424_Function_Block * func_block,
    unsigned int channel,
    uint8_t marker,
    int16_t value)
```

Set a value to be converted by the DAC immediately.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>channel</i>	
----------------	--

DAC channel that we want the last conversion set to.

Parameters

<i>marker</i>	
---------------	--

Value of the marker bits (top 8 bits)

Parameters

<i>value</i>	
--------------	--

Value to be converted by DAC and set on its output pin.

Note

The DAC will set its output value to the last conversion register value only if the DAC is in Reset mode.

Return values

<i>0</i>	
----------	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), and [run_test_9\(\)](#).

4.6.2.27 DM35424_Dac_Set_Post_Stop_Conversion_Count()

```
DM35424LIB_API int DM35424_Dac_Set_Post_Stop_Conversion_Count (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    uint32_t value)
```

Set the number of conversions the DAC will make after a stop trigger.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>value</i>	
--------------	--

Number of conversions.

Return values

0	
---	--

Success.

Return values

Non-Zero	
----------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [run_test_9\(\)](#).

4.6.2.28 DM35424_Dac_Set_Start_Trigger()

```
DM35424LIB_API int DM35424_Dac_Set_Start_Trigger (
    struct DM35424_Board_Descriptor * handle,
    const struct DM35424_Function_Block * func_block,
    uint8_t trigger_value)
```

Set the start trigger.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>trigger_value</i>	
----------------------	--

Trigger value (event) that will initiate conversions on this DAC.

Return values

<i>0</i>	
----------	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), [run_test_9\(\)](#), and [setup_dacs_and_start\(\)](#).

4.6.2.29 DM35424_Dac_Set_Stop_Trigger()

```
DM35424LIB_API int DM35424_Dac_Set_Stop_Trigger (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    uint8_t trigger_value)
```

Set the stop trigger.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>trigger_value</i>	
----------------------	--

Trigger value (event) that will halt conversions on this DAC.

Return values

0	
---	--

Success.

Return values

Non-Zero	
----------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), [run_test_9\(\)](#), and [setup_dacs_and_start\(\)](#).

4.6.2.30 DM35424_Dac_Start()

```
DM35424LIB_API int DM35424_Dac_Start (
    struct DM35424_Board_Descriptor * handle,
    const struct DM35424_Function_Block * func_block)
```

Set the DAC Mode to Start.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Return values

<i>0</i>	
----------	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), [run_test_9\(\)](#), and [setup_dacs_and_start\(\)](#).

4.6.2.31 DM35424_Dac_Volts_To_Conv()

```
DM35424LIB_API int DM35424_Dac_Volts_To_Conv (
    float volts,
    int16_t * dac_conversion)
```

Convert a value in volts to a DAC equivalent signed value.

Parameters

<i>volts</i>	
--------------	--

The volts value we want the DAC to output.

Parameters

<code>dac_conversion</code>	
-----------------------------	--

Pointer to signed value representing the equivalent of the volts.

Return values

<code>0</code>	
----------------	--

Success.

Return values

<code>Non-zero</code>	
-----------------------	--

Failure.

errno may be set as follows:

- `EINVAL` Function called by an unsupported function block.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), and [setup_dacs_and_start\(\)](#).

4.7 DM35424 DIO Public

Functions

- [DM35424LIB_API](#) `int DM35424_Dio_Open` (struct [DM35424_Board_Descriptor](#) *handle, unsigned int number_of_type, struct [DM35424_Function_Block](#) *func_block)
Open the DIO indicated, and determine register locations of control blocks needed to control it.
- [DM35424LIB_API](#) `int DM35424_Dio_Set_Direction` (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t direction)
Set the direction of the DIO pins.
- [DM35424LIB_API](#) `int DM35424_Dio_Get_Direction` (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t *direction)
Get the direction of the DIO pins.
- [DM35424LIB_API](#) `int DM35424_Dio_Get_Input_Value` (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t *value)
Get the input value of the DIO.
- [DM35424LIB_API](#) `int DM35424_Dio_Get_Output_Value` (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t *value)
Get the current value of the output register.
- [DM35424LIB_API](#) `int DM35424_Dio_Set_Output_Value` (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t value)
Set the value to be put on output pins.

4.7.1 Detailed Description

Library Functions

4.7.2 Function Documentation

4.7.2.1 DM35424_Dio_Get_Direction()

```
DM35424LIB_API int DM35424_Dio_Get_Direction (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    uint32_t * direction)
```

Get the direction of the DIO pins.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block representing the DIO.

Parameters

<i>direction</i>	
------------------	--

Bitmask representing the directions of the pins. (0 = input, 1 = output)

Return values

0	
---	--

Success.

Return values

Non-Zero	
----------	--

Failure.

References [DM35424LIB_API](#).

4.7.2.2 DM35424_Dio_Get_Input_Value()

```
DM35424LIB_API int DM35424_Dio_Get_Input_Value (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    uint32_t * value)
```

Get the input value of the DIO.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block representing the DIO.

Parameters

<i>value</i>	
--------------	--

Pointer to returned value.

Note

The value of pins that are set to output will be zero.

Return values

0	
---	--

Success.

Return values

Non-Zero	
----------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#).

4.7.2.3 DM35424_Dio_Get_Output_Value()

```
DM35424LIB_API int DM35424_Dio_Get_Output_Value (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    uint32_t * value)
```

Get the current value of the output register.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block representing the DIO.

Parameters

<i>value</i>	
--------------	--

Pointer to returned value.

Note

The value of pins that are set to input will be zero.

Return values

0	
---	--

Success.

Return values

Non-Zero	
----------	--

Failure.

References [DM35424LIB_API](#).

4.7.2.4 DM35424_Dio_Open()

```
DM35424LIB_API int DM35424_Dio_Open (
    struct DM35424_Board_Descriptor * handle,
    unsigned int number_of_type,
    struct DM35424_Function_Block * func_block)
```

Open the DIO indicated, and determine register locations of control blocks needed to control it.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>number_of_type</i>	
-----------------------	--

Which DIO to open. The first DIO on the board will be 0.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Return values

0	
---	--

Success.

Return values

Non-Zero	
----------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#).

4.7.2.5 DM35424_Dio_Set_Direction()

```
DM35424LIB_API int DM35424_Dio_Set_Direction (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    uint32_t direction)
```

Set the direction of the DIO pins.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block representing the DIO.

Parameters

<i>direction</i>	
------------------	--

Bitmask representing the directions to set the pins. (0 = input, 1 = output)

Return values

0	
---	--

Success.

Return values

Non-Zero	
----------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#).

4.7.2.6 DM35424_Dio_Set_Output_Value()

```
DM35424LIB_API int DM35424_Dio_Set_Output_Value (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    uint32_t value)
```

Set the value to be put on output pins.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block representing the DIO.

Parameters

<i>value</i>	
--------------	--

Value to be written to output pins.

Note

Writing a bit to a pin set to input will have no effect.

Return values

0	
---	--

Success.

Return values

Non-Zero	
----------	--

Failure.

Referenced by [main\(\)](#).

4.8 DM35424 DMA Public Library Constants

Macros

- **#define DM35424_DMA_ACTION_CLEAR** 0x00
Register value for DMA clear action.
- **#define DM35424_DMA_ACTION_GO** 0x01
Register value for DMA go action.
- **#define DM35424_DMA_ACTION_PAUSE** 0x02
Register value for DMA pause action.
- **#define DM35424_DMA_ACTION_HALT** 0x03
Register value for DMA halt action.
- **#define DM35424_DMA_SETUP_DIRECTION_READ** 0x04
Register value to set DMA to READ direction.
- **#define DM35424_DMA_SETUP_DIRECTION_WRITE** 0x00
Register value to set DMA to WRITE direction.
- **#define DM35424_DMA_SETUP_DIRECTION_MASK** 0x04
Register value to set DMA to READ direction.
- **#define DM35424_DMA_SETUP_IGNORE_USED** 0x08
Register value to tell DMA to ignore used buffers.
- **#define DM35424_DMA_SETUP_NOT_IGNORE_USED** 0x00
Register value to tell DMA to not ignore used buffers.
- **#define DM35424_DMA_SETUP_IGNORE_USED_MASK** 0x08
Bit mask for Ignore Used bit in setup register.
- **#define DM35424_DMA_SETUP_INT_ENABLE** 0x01
Register value to enable interrupts in the setup register.
- **#define DM35424_DMA_SETUP_INT_DISABLE** 0x00
Register value to disable interrupts in the setup register.
- **#define DM35424_DMA_SETUP_INT_MASK** 0x01
Bit mask for the interrupt bit in the setup register.
- **#define DM35424_DMA_SETUP_ERR_INT_ENABLE** 0x02
Register value to enable the error interrupt.
- **#define DM35424_DMA_SETUP_ERR_INT_DISABLE** 0x00
Register value to disable the error interrupt.
- **#define DM35424_DMA_SETUP_ERR_INT_MASK** 0x02
Bit mask for the error interrupt bit in the setup register.
- **#define DM35424_DMA_STATUS_CLEAR** 0x00
Register value to write to status registers to clear them.
- **#define DM35424_DMA_CTRL_CLEAR** 0x00
Register value to write to control register to clear it.
- **#define DM35424_DMA_BUFFER_STATUS_CLEAR** 0x00
Register value to write to the buffer status register to clear it.
- **#define DM35424_DMA_BUFFER_CTRL_CLEAR** 0x00
Register value to write to the buffer control register to clear it.
- **#define DM35424_DMA_BUFFER_STATUS_USED_MASK** 0x01
Bit mask for the used buffer bit in the buffer status register.
- **#define DM35424_DMA_BUFFER_STATUS_TERM_MASK** 0x02
Bit mask for the terminated buffer bit in the buffer status register.
- **#define DM35424_DMA_BUFFER_CTRL_VALID** 0x01
Register value to write to buffer control register to mark it as valid.

- **#define DM35424_DMA_BUFFER_CTRL_HALT** 0x02
Register value to write to buffer control register to tell DMA to halt after processing this buffer.
- **#define DM35424_DMA_BUFFER_CTRL_LOOP** 0x04
Register value to write to buffer control register to tell DMA to loop back to buffer 0 after using this buffer.
- **#define DM35424_DMA_BUFFER_CTRL_INTR** 0x08
Register value to write to buffer control register to tell DMA to issue an interrupt after using this buffer.
- **#define DM35424_DMA_BUFFER_CTRL_PAUSE** 0x10
Register value to write to buffer control register to tell DMA to pause after processing this buffer.
- **#define DM35424_DMA_CTRL_BLOCK_SIZE** 0x10
Constant value indicating DMA control block size.
- **#define DM35424_DMA_BUFFER_CTRL_BLOCK_SIZE** 0x10
Constant value indicating DMA buffer control block size.
- **#define DM35424_BIT_MASK_DMA_BUFFER_SIZE** 0x0FFFFFFF
Bit mask for the DMA buffer size, since it is 24-bits of a 32-bit register.

Enumerations

- enum [DM35424_Fifo_States](#) { [DM35424_FIFO_UNKNOWN](#) , [DM35424_FIFO_EMPTY](#) , [DM35424_FIFO_FULL](#) , [DM35424_FIFO_HAS_DATA](#) }
Descriptions of the possible states the FIFO might be in.

4.8.1 Detailed Description

4.8.2 Enumeration Type Documentation

4.8.2.1 DM35424_Fifo_States

enum [DM35424_Fifo_States](#)

Descriptions of the possible states the FIFO might be in.

Enumerator

DM35424_FIFO_UNKNOWN	State of FIFO is unknown.
DM35424_FIFO_EMPTY	FIFO is empty.
DM35424_FIFO_FULL	FIFO is full
DM35424_FIFO_HAS_DATA	FIFO is between empty and full

Definition at line [237](#) of file [dm35424_dma_library.h](#).

4.9 DM35424 DMA Public Library Functions

Functions

- `DM35424LIB_API int DM35424_Dma_Start` (struct `DM35424_Board_Descriptor` *handle, const struct `DM35424_Function_Block` *func_block, unsigned int channel)
Start the DMA.
- `DM35424LIB_API int DM35424_Dma_Stop` (struct `DM35424_Board_Descriptor` *handle, const struct `DM35424_Function_Block` *func_block, unsigned int channel)
Stop the DMA.
- `DM35424LIB_API int DM35424_Dma_Pause` (struct `DM35424_Board_Descriptor` *handle, const struct `DM35424_Function_Block` *func_block, unsigned int channel)
Pause the DMA.
- `DM35424LIB_API int DM35424_Dma_Clear` (struct `DM35424_Board_Descriptor` *handle, const struct `DM35424_Function_Block` *func_block, unsigned int channel)
Clear the DMA.
- `DM35424LIB_API int DM35424_Dma_Get_Fifo_Counts` (struct `DM35424_Board_Descriptor` *handle, const struct `DM35424_Function_Block` *func_block, unsigned int channel, uint16_t *write_count, uint16_t *read_count)
Get the Read and Write FIFO count values.
- `DM35424LIB_API int DM35424_Dma_Get_Fifo_State` (struct `DM35424_Board_Descriptor` *handle, const struct `DM35424_Function_Block` *func_block, unsigned int channel, enum `DM35424_Fifo_States` *state)
Get the state of the FIFO.
- `DM35424LIB_API int DM35424_Dma_Configure_Interrupts` (struct `DM35424_Board_Descriptor` *handle, const struct `DM35424_Function_Block` *func_block, unsigned int channel, int enable, int error_enable)
Configure the interrupts for the DMA channel.
- `DM35424LIB_API int DM35424_Dma_Get_Interrupt_Configuration` (struct `DM35424_Board_Descriptor` *handle, const struct `DM35424_Function_Block` *func_block, unsigned int channel, int *enable, int *error_enable)
Get the configuration of the interrupts for the DMA channel.
- `DM35424LIB_API int DM35424_Dma_Setup` (struct `DM35424_Board_Descriptor` *handle, const struct `DM35424_Function_Block` *func_block, unsigned int channel, int direction, int ignore_used)
Setup the DMA channel, specifically the direction and if used buffers are ignored.
- `DM35424LIB_API int DM35424_Dma_Setup_Set_Direction` (struct `DM35424_Board_Descriptor` *handle, const struct `DM35424_Function_Block` *func_block, unsigned int channel, int direction)
Set the direction of the DMA, read or write.
- `DM35424LIB_API int DM35424_Dma_Setup_Set_Used` (struct `DM35424_Board_Descriptor` *handle, const struct `DM35424_Function_Block` *func_block, unsigned int channel, int ignore_used)
Set the DMA channel to ignore or not ignore a used buffer. Ignoring used buffers is mostly useful when outputting a repeating data cycle.
- `DM35424LIB_API int DM35424_Dma_Get_Errors` (struct `DM35424_Board_Descriptor` *handle, const struct `DM35424_Function_Block` *func_block, unsigned int channel, int *stat_overflow, int *stat_underflow, int *stat_used, int *stat_invalid)
Get the current value of the DMA channel error registers.
- `DM35424LIB_API int DM35424_Dma_Status` (struct `DM35424_Board_Descriptor` *handle, const struct `DM35424_Function_Block` *func_block, unsigned int channel, uint32_t *current_buffer, uint32_t *current_count, int *current_action, int *stat_overflow, int *stat_underflow, int *stat_used, int *stat_invalid, int *stat_complete)
Get the current status of the DMA channel. Determine which buffer it is using, what its current action is, and the state of all error conditions and normal interrupt conditions.
- `DM35424LIB_API int DM35424_Dma_Get_Current_Buffer_Count` (struct `DM35424_Board_Descriptor` *handle, const struct `DM35424_Function_Block` *func_block, unsigned int channel, uint32_t *current_buffer, uint32_t *current_count)

Get the current buffer and buffer count in use by the DMA.

- [DM35424LIB_API](#) [int](#) [DM35424_Dma_Check_For_Error](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, int *has_error)

Check the DMA channel for any error conditions. This just returns a simple boolean as quickly as possible. If there is an error condition, you will have to query the DMA again to determine what the error is.

- [DM35424LIB_API](#) [int](#) [DM35424_Dma_Buffer_Setup](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, unsigned int buffer, uint8_t ctrl)

Setup the DMA buffer for use.

- [DM35424LIB_API](#) [int](#) [DM35424_Dma_Buffer_Status](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, unsigned int buffer, uint8_t *status, uint8_t *control, uint32_t *size)

Get the status of the buffer. This gets the status, control, and size registers.

- [DM35424LIB_API](#) [int](#) [DM35424_Dma_Check_Buffer_Used](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, unsigned int buffer_num, int *is_used)

Check if the indicated buffer has the "Used" flag set.

- [int](#) [DM35424_Dma_Find_Interrupt](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int *channel, int *channel_complete, int *channel_error)

Find which DMA channel has an interrupt condition, whether from using a buffer with interrupt set, or from an error.

DMA channels are evaluated starting at Channel 0.

- [int](#) [DM35424_Dma_Clear_Interrupt](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, int clear_overflow, int clear_underflow, int clear_used, int clear_invalid, int clear_complete)

Clear the interrupt flag from a DMA channel. Clearing the flags will allow another interrupt of the same type to occur again, and is the normal operation after handling the interrupt itself.

- [int](#) [DM35424_Dma_Reset_Buffer](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, unsigned int buffer)

Reset the DMA buffer, preparing it to be used again by the DMA engine.

4.9.1 Detailed Description

DM35424_Dma_Library_Constants

4.9.2 Function Documentation

4.9.2.1 DM35424_Dma_Buffer_Setup()

```
DM35424LIB_API int DM35424_Dma_Buffer_Setup (
    struct DM35424_Board_Descriptor * handle,
    const struct DM35424_Function_Block * func_block,
    unsigned int channel,
    unsigned int buffer,
    uint8_t ctrl)
```

Setup the DMA buffer for use.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>channel</i>	
----------------	--

DMA Channel to set.

Parameters

<i>buffer</i>	
---------------	--

DMA buffer to set.

Parameters

<i>ctrl</i>	
-------------	--

Unsigned short containing control bits. Will be written to control register.

Return values

0	
---	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure. `errno` may be set as follows:

- `EINVAL` Invalid channel or buffer requested.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), [setup_adc\(\)](#), and [setup_dacs_and_start\(\)](#).

4.9.2.2 DM35424_Dma_Buffer_Status()

```
DM35424LIB_API int DM35424_Dma_Buffer_Status (
    struct DM35424_Board_Descriptor * handle,
    const struct DM35424_Function_Block * func_block,
    unsigned int channel,
    unsigned int buffer,
    uint8_t * status,
    uint8_t * control,
    uint32_t * size)
```

Get the status of the buffer. This gets the status, control, and size registers.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>channel</i>	
----------------	--

DMA Channel to get.

Parameters

<i>buffer</i>	
---------------	--

DMA buffer to get.

Parameters

<i>status</i>	
---------------	--

Pointer to returned DMA buffer status register value.

Parameters

<i>control</i>	
----------------	--

Pointer to returned DMA buffer control register value.

Parameters

<i>size</i>	
-------------	--

Pointer to returned DMA buffer size (in bytes).

Return values

<i>0</i>	
----------	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure. `errno` may be set as follows:

- `EINVAL` Invalid channel or buffer requested.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), and [setup_adc\(\)](#).

4.9.2.3 DM35424_Dma_Check_Buffer_Used()

```
DM35424LIB_API int DM35424_Dma_Check_Buffer_Used (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    unsigned int channel,  
    unsigned int buffer_num,  
    int * is_used)
```

Check if the indicated buffer has the "Used" flag set.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>channel</i>	
----------------	--

DMA Channel to get.

Parameters

<i>buffer_num</i>	
-------------------	--

Which buffer in the DMA channel to check.

Parameters

<i>is_used</i>	
----------------	--

Pointer to returned boolean indicating if the buffer has the Used flag set.

Return values

0	
---	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure. errno may be set as follows:

- `EINVAL` Invalid channel requested.

4.9.2.4 DM35424_Dma_Check_For_Error()

```
DM35424LIB_API int DM35424_Dma_Check_For_Error (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    unsigned int channel,  
    int * has_error)
```

Check the DMA channel for any error conditions. This just returns a simple boolean as quickly as possible. If there is an error condition, you will have to query the DMA again to determine what the error is.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>channel</i>	
----------------	--

DMA Channel to set.

Parameters

<i>has_error</i>	
------------------	--

Pointer to returned boolean value indicating if the DMA channel has an error condition.

Return values

0	
---	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure. errno may be set as follows:

- `EINVAL` Invalid channel requested.

References [DM35424LIB_API](#).

4.9.2.5 DM35424_Dma_Clear()

```
DM35424LIB_API int DM35424_Dma_Clear (
    struct DM35424_Board_Descriptor * handle,
    const struct DM35424_Function_Block * func_block,
    unsigned int channel)
```

Clear the DMA.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>channel</i>	
----------------	--

DMA Channel to clear.

Return values

0	
---	--

Success.

Return values

-1	
----	--

Failure. errno may be set as follows:

- `EBUSY` The action was not executed before timeout. `EINVAL` Invalid channel requested.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#).

4.9.2.6 DM35424_Dma_Clear_Interrupt()

```
int DM35424_Dma_Clear_Interrupt (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    unsigned int channel,  
    int clear_overflow,  
    int clear_underflow,  
    int clear_used,  
    int clear_invalid,  
    int clear_complete)
```

Clear the interrupt flag from a DMA channel. Clearing the flags will allow another interrupt of the same type to occur again, and is the normal operation after handling the interrupt itself.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>channel</i>	
----------------	--

DMA channel to clear.

Parameters

<i>clear_overflow</i>	
-----------------------	--

Boolean indicating whether or not to clear the overflow interrupt status.

Parameters

<i>clear_underflow</i>	
------------------------	--

Boolean indicating whether or not to clear the underflow interrupt status.

Parameters

<i>clear_used</i>	
-------------------	--

Boolean indicating whether or not to clear the used interrupt status.

Parameters

<i>clear_invalid</i>	
----------------------	--

Boolean indicating whether or not to clear the invalid buffer interrupt status.

Parameters

<i>clear_complete</i>	
-----------------------	--

Boolean indicating whether or not to clear the buffer completed interrupt status.

Return values

0	
---	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure. `errno` may be set as follows:

- `EINVAL` Invalid channel requested.

Warning

This function is not compatible with the Windows driver package and is therefore not included in the Windows DLL.

Referenced by [ISR\(\)](#).

4.9.2.7 DM35424_Dma_Configure_Interrupts()

```
DM35424LIB_API int DM35424_Dma_Configure_Interrupts (
    struct DM35424_Board_Descriptor * handle,
    const struct DM35424_Function_Block * func_block,
    unsigned int channel,
    int enable,
    int error_enable)
```

Configure the interrupts for the DMA channel.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>channel</i>	
----------------	--

DMA Channel to configure interrupts for.

Parameters

<i>enable</i>	
---------------	--

Boolean value indicating if interrupt is to be enabled or disabled.

Parameters

<i>error_enable</i>	
---------------------	--

Boolean value indicating if interrupts for error conditions are to be enabled or disabled.

Return values

0	
---	--

Success.

Return values

-1	
----	--

Failure. errno may be set as follows:

- `EINVAL` Invalid channel requested.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), and [setup_adc\(\)](#).

4.9.2.8 DM35424_Dma_Find_Interrupt()

```
int DM35424_Dma_Find_Interrupt (
    struct DM35424_Board_Descriptor * handle,
    const struct DM35424_Function_Block * func_block,
    unsigned int * channel,
    int * channel_complete,
    int * channel_error)
```

Find which DMA channel has an interrupt condition, whether from using a buffer with interrupt set, or from an error. DMA channels are evaluated starting at Channel 0.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>channel</i>	
----------------	--

Pointer to returned DMA channel with interrupt, if any.

Parameters

<i>channel_complete</i>	
-------------------------	--

Pointer to returned boolean indicating that the interrupt on this channel was from using a buffer with the interrupt bit set.

Parameters

<i>channel_error</i>	
----------------------	--

Pointer to returned boolean indicating that the interrupt on this channel was from an error condition.

Return values

0	
---	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

Warning

This function is not compatible with the Windows driver package and is therefore not included in the Windows DLL.

Referenced by [ISR\(\)](#).

4.9.2.9 DM35424_Dma_Get_Current_Buffer_Count()

```
DM35424LIB_API int DM35424_Dma_Get_Current_Buffer_Count (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    unsigned int channel,  
    uint32_t * current_buffer,  
    uint32_t * current_count)
```

Get the current buffer and buffer count in use by the DMA.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>channel</i>	
----------------	--

Channel to get buffer info from.

Parameters

<i>current_buffer</i>	
-----------------------	--

Pointer to the returned current buffer the DMA is using.

Parameters

<i>current_count</i>	
----------------------	--

Pointer to the returned count for the current buffer. This indicates how far into the buffer the DMA is.

Return values

0	
---	--

Success.

Return values

Non-Zero	
----------	--

Failure.

References [DM35424LIB_API](#).

4.9.2.10 DM35424_Dma_Get_Errors()

```
DM35424LIB_API int DM35424_Dma_Get_Errors (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    unsigned int channel,  
    int * stat_overflow,  
    int * stat_underflow,  
    int * stat_used,  
    int * stat_invalid)
```

Get the current value of the DMA channel error registers.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>channel</i>	
----------------	--

DMA Channel to configure interrupts for.

Parameters

<i>stat_overflow</i>	
----------------------	--

Pointer to the returned boolean indicating if overflow has occurred.

Parameters

<i>stat_underflow</i>	
-----------------------	--

Pointer to the returned boolean indicating if underflow has occurred.

Parameters

<i>stat_used</i>	
------------------	--

Pointer to the returned boolean indicating if the DMA attempted to use an already used buffer.

Parameters

<i>stat_invalid</i>	
---------------------	--

Pointer to the returned boolean indicating if the DMA attempted to use an invalid buffer.

Return values

<i>0</i>	
----------	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure. `errno` may be set as follows:

- `EINVAL` Invalid channel requested.

References [DM35424LIB_API](#).

4.9.2.11 DM35424_Dma_Get_Fifo_Counts()

```
DM35424LIB_API int DM35424_Dma_Get_Fifo_Counts (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    unsigned int channel,  
    uint16_t * write_count,  
    uint16_t * read_count)
```

Get the Read and Write FIFO count values.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>channel</i>	
----------------	--

DMA Channel to get counts for.

Parameters

<i>write_count</i>	
--------------------	--

Pointer to the returned number of bytes of space available in the FIFO.

Parameters

<i>read_count</i>	
-------------------	--

Pointer to the returned number of bytes of data available in the FIFO.

Note

These counts are valid regardless of the direction of the DMA.

Return values

0	
---	--

Success.

Return values

-1	
----	--

Failure. `errno` may be set as follows:

- `EINVAL` Invalid channel requested.

References [DM35424LIB_API](#).

4.9.2.12 DM35424_Dma_Get_Fifo_State()

```
DM35424LIB_API int DM35424_Dma_Get_Fifo_State (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    unsigned int channel,  
    enum DM35424_Fifo_States * state)
```

Get the state of the FIFO.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>channel</i>	
----------------	--

DMA Channel to get state for.

Parameters

<i>state</i>	
--------------	--

Pointer to the returned FIFO state enumeration.

Note

This is just a convenience function that infers a FIFO state from the FIFO counts.

Return values

<i>0</i>	
----------	--

Success.

Return values

<i>-1</i>	
-----------	--

Failure. `errno` may be set as follows:

- `EINVAL` Invalid channel requested.

References [DM35424LIB_API](#).

4.9.2.13 DM35424_Dma_Get_Interrupt_Configuration()

```
DM35424LIB_API int DM35424_Dma_Get_Interrupt_Configuration (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    unsigned int channel,  
    int * enable,  
    int * error_enable)
```

Get the configuration of the interrupts for the DMA channel.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>channel</i>	
----------------	--

DMA Channel to configure interrupts for.

Parameters

<i>enable</i>	
---------------	--

Pointer to returned value indicating if interrupt is enabled or disabled.

Parameters

<i>error_enable</i>	
---------------------	--

Pointer to returned boolean value indicating if interrupts for error conditions are enabled or disabled.

Return values

0	
---	--

Success.

Return values

-1	
----	--

Failure. errno may be set as follows:

- `EINVAL` Invalid channel requested.

References [DM35424LIB_API](#).

4.9.2.14 DM35424_Dma_Pause()

```
DM35424LIB_API int DM35424_Dma_Pause (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    unsigned int channel)
```

Pause the DMA.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>channel</i>	
----------------	--

DMA Channel to pause.

Return values

0	
---	--

Success.

Return values

-1	
----	--

Failure. `errno` may be set as follows:

- `EBUSY` The action was not executed before timeout. `EINVAL` Invalid channel requested.

References [DM35424LIB_API](#).

4.9.2.15 DM35424_Dma_Reset_Buffer()

```
int DM35424_Dma_Reset_Buffer (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    unsigned int channel,  
    unsigned int buffer)
```

Reset the DMA buffer, preparing it to be used again by the DMA engine.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>channel</i>	
----------------	--

DMA channel containing the buffer.

Parameters

<i>buffer</i>	
---------------	--

Buffer in channel to clear.

Return values

0	
---	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure. `errno` may be set as follows:

- `EINVAL` Invalid channel or buffer requested.

Warning

This function is not compatible with the Windows driver package and is therefore not included in the Windows DLL.

References [DM35424LIB_API](#).

Referenced by [ISR\(\)](#).

4.9.2.16 DM35424_Dma_Setup()

```
DM35424LIB_API int DM35424_Dma_Setup (
    struct DM35424_Board_Descriptor * handle,
    const struct DM35424_Function_Block * func_block,
    unsigned int channel,
    int direction,
    int ignore_used)
```

Setup the DMA channel, specifically the direction and if used buffers are ignored.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>channel</i>	
----------------	--

DMA Channel to set.

Parameters

<i>direction</i>	
------------------	--

Direction for DMA. See the DMA constants for possible values.

Parameters

<i>ignore_used</i>	
--------------------	--

Boolean value indicating if used buffers should be ignored.

Note

This is a convenience function that accomplishes two of the setup steps in one function. This function is identical to calling the direction and ignore used library functions separately.

Return values

0	
---	--

Success.

Return values

-1	
----	--

Failure. `errno` may be set as follows:

- `EINVAL` Invalid channel requested, or wrong direction requested for function block type.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), [setup_adc\(\)](#), and [setup_dacs_and_start\(\)](#).

4.9.2.17 DM35424_Dma_Setup_Set_Direction()

```
DM35424LIB_API int DM35424_Dma_Setup_Set_Direction (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    unsigned int channel,  
    int direction)
```

Set the direction of the DMA, read or write.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>channel</i>	
----------------	--

DMA Channel to set.

Parameters

<i>direction</i>	
------------------	--

Direction for DMA. See the DMA constants for possible values.

Return values

<i>0</i>	
----------	--

Success.

Return values

<i>-1</i>	
-----------	--

Failure. `errno` may be set as follows:

- `EINVAL` Invalid channel requested, or wrong direction requested for function block type.

References [DM35424LIB_API](#).

4.9.2.18 DM35424_Dma_Setup_Set_Used()

```
DM35424LIB_API int DM35424_Dma_Setup_Set_Used (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    unsigned int channel,  
    int ignore_used)
```

Set the DMA channel to ignore or not ignore a used buffer. Ignoring used buffers is mostly useful when outputting a repeating data cycle.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>channel</i>	
----------------	--

DMA Channel to set.

Parameters

<i>ignore_used</i>	
--------------------	--

Boolean value indicating if used buffers should be ignored.

Return values

<i>0</i>	
----------	--

Success.

Return values

<i>-1</i>	
-----------	--

Failure. `errno` may be set as follows:

- `EINVAL` Invalid channel requested.

References [DM35424LIB_API](#).

4.9.2.19 DM35424_Dma_Start()

```
DM35424LIB_API int DM35424_Dma_Start (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    unsigned int channel)
```

Start the DMA.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>channel</i>	
----------------	--

DMA Channel to start.

Return values

0	
---	--

Success.

Return values

-1	
----	--

Failure. `errno` may be set as follows:

- `EBUSY` The action was not executed before timeout. `EINVAL` Invalid channel requested.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), and [setup_dacs_and_start\(\)](#).

4.9.2.20 DM35424_Dma_Status()

```
DM35424LIB_API int DM35424_Dma_Status (
    struct DM35424_Board_Descriptor * handle,
    const struct DM35424_Function_Block * func_block,
    unsigned int channel,
    uint32_t * current_buffer,
    uint32_t * current_count,
    int * current_action,
    int * stat_overflow,
    int * stat_underflow,
    int * stat_used,
    int * stat_invalid,
    int * stat_complete)
```

Get the current status of the DMA channel. Determine which buffer it is using, what its current action is, and the state of all error conditions and normal interrupt conditions.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>channel</i>	
----------------	--

DMA Channel to configure interrupts for.

Parameters

<i>current_buffer</i>	
-----------------------	--

Pointer to the returned current buffer the DMA is using.

Parameters

<i>current_count</i>	
----------------------	--

Pointer to the returned count for the current buffer. This indicates how far into the buffer the DMA is.

Parameters

<i>current_action</i>	
-----------------------	--

Pointer to the returned action the DMA is currently taking.

Parameters

<i>stat_overflow</i>	
----------------------	--

Pointer to the returned boolean indicating if overflow has occurred.

Parameters

<i>stat_underflow</i>	
-----------------------	--

Pointer to the returned boolean indicating if underflow has occurred.

Parameters

<i>stat_used</i>	
------------------	--

Pointer to the returned boolean indicating if the DMA attempted to use an already used buffer.

Parameters

<i>stat_invalid</i>	
---------------------	--

Pointer to the returned boolean indicating if the DMA attempted to use an invalid buffer.

Parameters

<i>stat_complete</i>	
----------------------	--

Pointer to the returned boolean indicating if the DMA has completed using a buffer that had an interrupt set.

Return values

0	
---	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure. `errno` may be set as follows:

- `EINVAL` Invalid channel requested.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), and [output_channel_status\(\)](#).

4.9.2.21 DM35424_Dma_Stop()

```
DM35424LIB_API int DM35424_Dma_Stop (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    unsigned int channel)
```

Stop the DMA.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>channel</i>	
----------------	--

DMA Channel to stop.

Return values

0	
---	--

Success.

Return values

-1	
----	--

Failure. errno may be set as follows:

- EBUSY The action was not executed before timeout. EINVAL Invalid channel requested.

References [DM35424LIB_API](#).

4.10 DM35424 Driver Constants

Macros

- `#define DM35424_MAX_NUM_DEVICES 8`
DM35424 Maximum number of devices.
- `#define DM35424_NUM_MINORS_PER_DEVICE 1`
DM35424 Number of minors per device.
- `#define DM35424_NAME_LENGTH 200`
DM35424 Max possible board name length.
- `#define DM35424_PCI_NUM_REGIONS PCI_ROM_RESOURCE`
Number of standard PCI regions.
- `#define DM35424_INT_QUEUE_SIZE 256`
Number of interrupts to hold in a queue for processing.

4.10.1 Detailed Description

4.11 DM35424 Driver Enumerations

Enumerations

- enum `dm35424_pci_region_access_dir` { `DM35424_PCI_REGION_ACCESS_READ = 0`, `DM35424_PCI_REGION_ACCESS_WRITE` }
Direction of access to standard PCI region.

4.11.1 Detailed Description

DM35424_Driver_Constants

4.11.2 Enumeration Type Documentation

4.11.2.1 dm35424_pci_region_access_dir

```
enum dm35424_pci_region_access_dir
```

Direction of access to standard PCI region.

Enumerator

DM35424_PCI_REGION_ACCESS_READ	Read from the region
DM35424_PCI_REGION_ACCESS_WRITE	Write to the region

Definition at line 92 of file [dm35424_driver.h](#).

4.12 DM35424 Driver Structures

Data Structures

- struct [dm35424_pci_region](#)
DM35424 PCI region descriptor. This structure holds information about one of a device's PCI memory regions.
- struct [dm35424_dma_descriptor](#)
DM35424 DMA descriptor. This structure holds information about a single DMA buffer.
- struct [dm35424_device_descriptor](#)
DM35424 Device Descriptor. The identifying info for this particular board.

Variables

- static struct file_operations **dm35424_file_ops**
Placeholder prototype for file ops struct.

4.12.1 Detailed Description

DM35424_Driver_Enumerations

4.13 DM35424 Example Programs Constants

Macros

- #define **BUFFER_VALID** 1
Boolean indicating buffer valid.
- #define **BUFFER_NO_VALID** 0
Boolean indicating buffer not valid.
- #define **BUFFER_HALT** 1
Boolean indicating buffer halt set.
- #define **BUFFER_NO_HALT** 0
Boolean indicating buffer halt not set.
- #define **BUFFER_LOOP** 1
Boolean indicating buffer loop set.
- #define **BUFFER_NO_LOOP** 0
Boolean indicating buffer loop not set.
- #define **BUFFER_INTERRUPT** 1
Boolean indicating buffer interrupt.
- #define **BUFFER_NO_INTERRUPT** 0
Boolean indicating no buffer interrupt.
- #define **BUFFER_PAUSE** 1
Boolean indicating buffer should pause when filled.
- #define **BUFFER_NO_PAUSE** 0
Boolean indicating buffer should not pause when filled.
- #define **IGNORE_USED** 1
Boolean indicating ignore used buffers.
- #define **NOT_IGNORE_USED** 0

- Boolean indicating not ignore used buffers.*

 - **#define CLEAR_INTERRUPT 1**

Boolean indicating to clear an interrupt.
- **#define NO_CLEAR_INTERRUPT 0**

Boolean indicating to not clear an interrupt.
- **#define INTERRUPT_ENABLE 1**

Boolean indicating interrupt enable.
- **#define INTERRUPT_DISABLE 0**

Boolean indicating interrupt disable.
- **#define ERROR_INTR_ENABLE 1**

Boolean indicating error interrupt enable.
- **#define ERROR_INTR_DISABLE 0**

Boolean indicating error interrupt disable.
- **#define SYNCBUS_NONE 0**

Value indicating no Syncbus option was chosen.
- **#define SYNCBUS_MASTER 1**

Value indicating Syncbus Master was chosen.
- **#define SYNCBUS_SLAVE 2**

Value indicating Syncbus Slave was chosen.
- **#define CHANNEL_0 0**

Constant for selecting Channel 0.
- **#define CHANNEL_1 1**

Constant for selecting Channel 1.
- **#define CHANNEL_2 2**

Constant for selecting Channel 2.
- **#define CHANNEL_3 3**

Constant for selecting Channel 3.
- **#define BUFFER_0 0**

Constant for selecting Buffer 0.
- **#define BUFFER_1 1**

Constant for selecting Buffer 1.
- **#define ADC_0 0**

Constant for selecting ADC 0.
- **#define ADC_1 1**

Constant for selecting ADC 1.
- **#define DAC_0 0**

Constant for selecting DAC 0.
- **#define DAC_1 1**

Constant for selecting DAC 1.
- **#define DAC_2 2**

Constant for selecting DAC 2.
- **#define DAC_3 3**

Constant for selecting DAC 3.
- **#define REF_0 0**

Constant for selecting REF 0.
- **#define REF_1 1**

Constant for selecting REF 1.
- **#define DIO_0 0**

Constant for selecting DIO 0.
- **#define ADIO_0 0**

Constant for selecting ADIO 0.

- `#define ENABLED 1`
Constant to indicate an Enabled value.
- `#define DISABLED 0`
Constant to indicate a Disabled value.

Enumerations

- `enum Help_Options {`
`HELP_OPTION = 1 , MINOR_OPTION , RATE_OPTION , CHANNELS_OPTION ,`
`FILE_OPTION , START_OPTION , WAVE_OPTION , TEST_OPTION ,`
`NOSTOP_OPTION , SYNCBUS_OPTION , DUMP_OPTION , HOURS_OPTION ,`
`OUTPUT_RMS_OPTION , OUTPUT_ADC_OPTION , ADC_NUM_OPTION , DAC_NUM_OPTION ,`
`ADC_OPTION , DAC_OPTION , PATTERN_OPTION , SAMPLES_OPTION ,`
`MODE_OPTION , AD_MODE_OPTION , REF_NUM_OPTION , BINARY_OPTION ,`
`SENDER_OPTION , RECEIVER_OPTION , RANGE_OPTION , REFILL_FIFO_OPTION ,`
`LOW_THRESHOLD_OPTION , PORT_OPTION , BAUD_OPTION , EXTERNAL_OPTION ,`
`SIZE_OPTION , VERBOSE_OPTION , USER_ID_OPTION , COUNT_OPTION ,`
`NUM_OPTION , SYNC_TERM_OPTION , BIN2TXT_OPTION , STORE_OPTION ,`
`TERM_OPTION , REFCLK_OPTION , OFIELD_OPTION , PACKED_OPTION ,`
`MASTER_OPTION , SLAVE_OPTION , SYNC_CONN_OPTION }`
Constants used for parsing command line parameters of example programs.

4.13.1 Detailed Description

4.13.2 Enumeration Type Documentation

4.13.2.1 Help_Options

```
enum Help_Options
```

Constants used for parsing command line parameters of example programs.

Note

This value won't be seen by the user (except in code), so the value can be used for any desired option.

Enumerator

<code>HELP_OPTION</code>	Command line parameter <code>-help</code> .
<code>MINOR_OPTION</code>	Command line parameter <code>-minor</code> .
<code>RATE_OPTION</code>	Command line parameter <code>-rate</code> .
<code>CHANNELS_OPTION</code>	Command line parameter <code>-chan</code> .
<code>FILE_OPTION</code>	Command line parameter for including a file.
<code>START_OPTION</code>	Command line parameter <code>-start</code> .
<code>WAVE_OPTION</code>	Command line parameter <code>-wave</code> .
<code>TEST_OPTION</code>	Command line parameter <code>-test</code> .
<code>NOSTOP_OPTION</code>	Command line parameter <code>-nostop</code> .

SYNCBUS_OPTION	Command line parameter –syncbus.
DUMP_OPTION	Command line parameter –dump.
HOURS_OPTION	Command line parameter –hours.
OUTPUT_RMS_OPTION	Command line parameter –output_rms.
OUTPUT_ADC_OPTION	Command line parameter –output_adc.
ADC_NUM_OPTION	Command line parameter –num_adc.
DAC_NUM_OPTION	Command line parameter –num_dac.
ADC_OPTION	Command line parameter –adc.
DAC_OPTION	Command line parameter –dac.
PATTERN_OPTION	Command line parameter –pattern.
SAMPLES_OPTION	Command line parameter –samples.
MODE_OPTION	Command line parameter –mode.
AD_MODE_OPTION	Command line parameter –ad_mode.
REF_NUM_OPTION	Command line parameter –ref.
BINARY_OPTION	Command line parameter –binary.
SENDER_OPTION	Command line parameter –sender.
RECEIVER_OPTION	Command line parameter –receiver.
RANGE_OPTION	Command line parameter –range.
REFILL_FIFO_OPTION	Command line parameter –refill.
LOW_THRESHOLD_OPTION	Command line parameter –low.
PORT_OPTION	Command line parameter –port.
BAUD_OPTION	Command line parameter –baud.
EXTERNAL_OPTION	Command line parameter –external.
SIZE_OPTION	Command line parameter –size.
VERBOSE_OPTION	Command line parameter –verbose.
USER_ID_OPTION	Command line parameter –userid.
COUNT_OPTION	Command line parameter –count.
NUM_OPTION	Command line parameter –num.
SYNC_TERM_OPTION	Command line parameter –syncterm.
BIN2TXT_OPTION	Command line parameter –bin2txt.
STORE_OPTION	Command line parameter –store.
TERM_OPTION	Command line parameter –term (Termination).
REFCLK_OPTION	Command line parameter –refclk (Reference clock selection).
OFIELD_OPTION	Command line parameter –ofile (Output file selection).
PACKED_OPTION	Command line parameter –packed (Packed, 16-bit samples selection).
MASTER_OPTION	Master minor option for synchronization.
SLAVE_OPTION	Slave minor option for synchronization.
SYNC_CONN_OPTION	Syncbus connector option.

Definition at line 43 of file [dm35424_examples.h](#).

4.14 DM35424 Board Macros

Macros

- `#define CLK_40MHZ 40000000`

4.14.1 Detailed Description

4.14.2 Macro Definition Documentation

4.14.2.1 CLK_40MHZ

```
#define CLK_40MHZ 40000000
```

This is the standard clock of the DM35x18 boards

Definition at line 52 of file [dm35424_gbc_library.h](#).

4.15 DM35424 Board Library Public Functions

Functions

- [DM35424LIB_API](#) `int DM35424_Gbc_Board_Reset` (struct [DM35424_Board_Descriptor](#) *handle)
Write the reset value to the correct register to initiate a board-level reset.
- [DM35424LIB_API](#) `int DM35424_Gbc_Ack_Interrupt` (struct [DM35424_Board_Descriptor](#) *handle)
Send an End-Of-Interrupt acknowledgement to the board. This will cause any pending interrupts to re-issue. This is a protection against missing interrupts while in the interrupt handler.
- [DM35424LIB_API](#) `int DM35424_Function_Block_Open` (struct [DM35424_Board_Descriptor](#) *handle, unsigned int number, struct [DM35424_Function_Block](#) *func_block)
Open a specific function block. Nothing is opened in a file sense, but the memory location for the function block is read and certain important values are read. A function block descriptor is allocated to hold the data that will be used every time this function block is accessed.
- [DM35424LIB_API](#) `int DM35424_Function_Block_Open_Module` (struct [DM35424_Board_Descriptor](#) *handle, uint32_t fb_type, unsigned int number_of_type, struct [DM35424_Function_Block](#) *func_block)
Open a specific function block module. This is the same as opening a function block, except we are looking for a function block with a specific type. This is the method you would use to open the 2nd ADC, for example.
- [DM35424LIB_API](#) `int DM35424_Gbc_Get_Format` (struct [DM35424_Board_Descriptor](#) *handle, uint8_t *format_id)
Get the format ID of the board.
- [DM35424LIB_API](#) `int DM35424_Gbc_Get_Revision` (struct [DM35424_Board_Descriptor](#) *handle, uint8_t *rev)
Get the PDP revision number of the board.
- [DM35424LIB_API](#) `int DM35424_Gbc_Get_Pdp_Number` (struct [DM35424_Board_Descriptor](#) *handle, uint32_t *pdp_num)
Get PDP Number of the board.
- [DM35424LIB_API](#) `int DM35424_Gbc_Get_Fpga_Build` (struct [DM35424_Board_Descriptor](#) *handle, uint32_t *fpga_build)
Get the FPGA Build number of the board.
- [DM35424LIB_API](#) `int DM35424_Gbc_Get_Sys_Clock_Freq` (struct [DM35424_Board_Descriptor](#) *handle, uint32_t *clock_freq, int *is_std_clk)
Get the measured frequency of the system clock of the board.

4.15.1 Detailed Description

DM35424_Board_Macros

4.15.2 Function Documentation

4.15.2.1 DM35424_Function_Block_Open()

```
DM35424LIB_API int DM35424_Function_Block_Open (
    struct DM35424_Board_Descriptor * handle,
    unsigned int number,
    struct DM35424_Function_Block * func_block)
```

Open a specific function block. Nothing is opened in a file sense, but the memory location for the function block is read and certain important values are read. A function block descriptor is allocated to hold the data that will be used every time this function block is accessed.

Parameters

<i>handle</i>	
---------------	--

Pointer to the device descriptor, which contains the open file id.

Parameters

<i>number</i>	
---------------	--

Which function block to open. The first function block on the board is at number 0.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. When the function block info is successfully read from the device, then this descriptor will be allocated to hold the data.

Return values

0	
---	--

Success.

Return values

Non-Zero	
----------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#).

4.15.2.2 DM35424_Function_Block_Open_Module()

```
DM35424LIB_API int DM35424_Function_Block_Open_Module (  
    struct DM35424_Board_Descriptor * handle,  
    uint32_t fb_type,  
    unsigned int number_of_type,  
    struct DM35424_Function_Block * func_block)
```

Open a specific function block module. This is the same as opening a function block, except we are looking for a function block with a specific type. This is the method you would use to open the 2nd ADC, for example.

Parameters

<i>handle</i>	
---------------	--

Pointer to the device descriptor, which contains the open file id.

Parameters

<i>fb_type</i>	
----------------	--

Type of function block you want to open. ADC, DAC, DIO, etc. The constant values are in the [dm35424_types.h](#) file.

Parameters

<i>number_of_type</i>	
-----------------------	--

Ordinal number of that particular type of function block that you wish to access. The first instance of that type is 0th.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. When the function block info is successfully read from the device, then this descriptor will be allocated to hold the data.

Return values

0	
---	--

Success.

Return values

Non-Zero	
----------	--

Failure.

References [DM35424LIB_API](#).

4.15.2.3 DM35424_Gbc_Ack_Interrupt()

```
DM35424LIB_API int DM35424_Gbc_Ack_Interrupt (  
    struct DM35424_Board_Descriptor * handle)
```

Send an End-Of-Interrupt acknowledgement to the board. This will cause any pending interrupts to re-issue. This is a protection against missing interrupts while in the interrupt handler.

Parameters

<i>handle</i>	
---------------	--

Pointer to the device descriptor, which contains the open file id.

Return values

0	
---	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [ISR\(\)](#).

4.15.2.4 DM35424_Gbc_Board_Reset()

```
DM35424LIB_API int DM35424_Gbc_Board_Reset (  
    struct DM35424_Board_Descriptor * handle)
```

Write the reset value to the correct register to initiate a board-level reset.

Parameters

<i>handle</i>	
---------------	--

Pointer to the device descriptor, which contains the open file id.

Return values

0	
---	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#), and [run_test_9\(\)](#).

4.15.2.5 DM35424_Gbc_Get_Format()

```
DM35424LIB_API int DM35424_Gbc_Get_Format (  
    struct DM35424_Board_Descriptor * handle,  
    uint8_t * format_id)
```

Get the format ID of the board.

Parameters

<i>handle</i>	
---------------	--

Pointer to the device descriptor, which contains the open file id.

Parameters

<i>format_id</i>	
------------------	--

Pointer to the returned format ID value.

Return values

<i>0</i>	
----------	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

References [DM35424LIB_API](#).

4.15.2.6 DM35424_Gbc_Get_Fpga_Build()

```
DM35424LIB_API int DM35424_Gbc_Get_Fpga_Build (
    struct DM35424_Board_Descriptor * handle,
    uint32_t * fpga_build)
```

Get the FPGA Build number of the board.

Parameters

<i>handle</i>	
---------------	--

Pointer to the device descriptor, which contains the open file id.

Parameters

<i>fpga_build</i>	
-------------------	--

Pointer to the returned FPGA Build number.

Return values

0	
---	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#).

4.15.2.7 DM35424_Gbc_Get_Pdp_Number()

```
DM35424LIB_API int DM35424_Gbc_Get_Pdp_Number (
    struct DM35424_Board_Descriptor * handle,
    uint32_t * pdp_num)
```

Get PDP Number of the board.

Parameters

<i>handle</i>	
---------------	--

Pointer to the device descriptor, which contains the open file id.

Parameters

<i>pdp_num</i>	
----------------	--

Pointer to the returned PDP Number.

Return values

0	
---	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#).

4.15.2.8 DM35424_Gbc_Get_Revision()

```
DM35424LIB_API int DM35424_Gbc_Get_Revision (  
    struct DM35424_Board_Descriptor * handle,  
    uint8_t * rev)
```

Get the PDP revision number of the board.

Parameters

<i>handle</i>	
---------------	--

Pointer to the device descriptor, which contains the open file id.

Parameters

<i>rev</i>	
------------	--

Pointer to the returned revision value.

Return values

0	
---	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#).

4.15.2.9 DM35424_Gbc_Get_Sys_Clock_Freq()

```
DM35424LIB_API int DM35424_Gbc_Get_Sys_Clock_Freq (  
    struct DM35424_Board_Descriptor * handle,  
    uint32_t * clock_freq,  
    int * is_std_clk)
```

Get the measured frequency of the system clock of the board.

Parameters

<i>handle</i>	
---------------	--

Pointer to the device descriptor, which contains the open file id.

Parameters

<i>clock_freq</i>	
-------------------	--

Pointer to the returned system clock frequency (in Hz)

Parameters

<i>is_std_clk</i>	
-------------------	--

Boolean value indicating if the clock read is a standard value. If true, then this function will always return the same value upon every call. If false, then this function will return the clock frequency actually read from the register. Note: If the value read from the GBC register is not a standard clock, then the clock frequency returned can change from read to read by slight variations.

Return values

<i>0</i>	
----------	--

Success.

Return values

Non-Zero	
----------	--

Failure.

4.16 DM35424 ioctl macros

Macros

- **#define DM35424_IOCTL_MAGIC 'D'**
Unique 8-bit value used to generate unique ioctl() request codes.
- **#define DM35424_IOCTL_REQUEST_BASE 0x00**
First ioctl() request number.
- **#define DM35424_IOCTL_REGION_READ**
ioctl() request code for reading from a PCI region
- **#define DM35424_IOCTL_REGION_WRITE**
ioctl() request code for writing to a PCI region
- **#define DM35424_IOCTL_REGION_MODIFY**
ioctl() request code for PCI region read/modify/write
- **#define DM35424_IOCTL_DMA_FUNCTION**
ioctl() request code for DMA function
- **#define DM35424_IOCTL_WAKEUP**
ioctl() request code for User ISR thread wake up
- **#define DM35424_IOCTL_INTERRUPT_GET**
ioctl() request code to retrieve interrupt status information
- **#define DM35424_IOCTL_GET_DEVICE_ID**
ioctl() request code to retrieve PCIe device ID

4.16.1 Detailed Description

4.17 DM35424 Board Access Public Library Functions

Data Structures

- struct [DM35424_Board_Descriptor](#)
DM35424 board descriptor. This structure holds information about the board as a whole. It holds the file descriptor and ISR callback function, if applicable.

Functions

- int [DM35424_Dma_Initialize](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, unsigned int num_buffers, uint32_t buffer_size)
Initialize the DMA channel and prepare it for data. Interrupts are disabled, error conditions are cleared, buffers are allocated in kernel space and their status and controls are cleared.
- int [DM35424_Dma_Read](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, unsigned int buffer_to_read_from, uint32_t buffer_size, void *local_buffer↵
buffer_ptr)
Read data from the DMA buffer. Data is copied from kernel buffers to local user-space buffers.
- int [DM35424_Dma_Write](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, unsigned int buffer_to_write_to, uint32_t buffer_size, void *local_buffer↵
_ptr)
Write data to the DMA buffer. Data is copied from local user buffers to kernel buffers.
- int [DM35424_General_RemoveISR](#) (struct [DM35424_Board_Descriptor](#) *handle)
Remove the ISR from the system interrupt.
- void * [DM35424_General_WaitForInterrupt](#) (void *ptr)
Loop/Poll and wait for an interrupt to happen, then take action.
- int [DM35424_General_InstallISR](#) (struct [DM35424_Board_Descriptor](#) *handle, void(*isr_fnct))
Start a thread that will sit and wait for an interrupt from the board, and call the user ISR when it happens.
- int [DM35424_General_SetISRPriority](#) (struct [DM35424_Board_Descriptor](#) *handle, int priority)
Set the priority of the user ISR thread.

4.17.1 Detailed Description

4.17.2 Function Documentation

4.17.2.1 DM35424_Dma_Initialize()

```
int DM35424_Dma_Initialize (
    struct DM35424\_Board\_Descriptor * handle,
    const struct DM35424\_Function\_Block * func_block,
    unsigned int channel,
    unsigned int num_buffers,
    uint32_t buffer_size)
```

Initialize the DMA channel and prepare it for data. Interrupts are disabled, error conditions are cleared, buffers are allocated in kernel space and their status and controls are cleared.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>channel</i>	
----------------	--

DMA Channel to get state for.

Parameters

<i>num_buffers</i>	
--------------------	--

Number of DMA buffers to allocate and initialize.

Parameters

<i>buffer_size</i>	
--------------------	--

The size in bytes to allocate for each buffer.

Return values

0	
---	--

Success.

Return values

-1	
----	--

Failure. `errno` may be set as follows:

- `EINVAL` Invalid channel or buffer requested. `ENOMEM` Memory could not be allocated for the DMA buffers.

Referenced by [main\(\)](#), [setup_adc\(\)](#), and [setup_dacs_and_start\(\)](#).

4.17.2.2 DM35424_Dma_Read()

```
int DM35424_Dma_Read (
    struct DM35424_Board_Descriptor * handle,
    const struct DM35424_Function_Block * func_block,
    unsigned int channel,
    unsigned int buffer_to_read_from,
    uint32_t buffer_size,
    void * local_buffer_ptr)
```

Read data from the DMA buffer. Data is copied from kernel buffers to local user-space buffers.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>channel</i>	
----------------	--

DMA channel containing the buffer to be read.

Parameters

<i>buffer_to_read_from</i>	
----------------------------	--

Buffer in channel to read.

Parameters

<i>buffer_size</i>	
--------------------	--

Number of bytes to read from the DMA buffer. In most cases, this should be equal to the allocated size of the buffer.

Parameters

<i>local_buffer_ptr</i>	
-------------------------	--

Pointer to local memory buffer already allocated that data will be copied into.

Return values

0	
---	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure. `errno` may be set as follows:

- `EINVAL` Invalid channel or buffer requested.

Referenced by [ISR\(\)](#).

4.17.2.3 DM35424_Dma_Write()

```
int DM35424_Dma_Write (
    struct DM35424_Board_Descriptor * handle,
    const struct DM35424_Function_Block * func_block,
    unsigned int channel,
    unsigned int buffer_to_write_to,
    uint32_t buffer_size,
    void * local_buffer_ptr)
```

Write data to the DMA buffer. Data is copied from local user buffers to kernel buffers.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

<i>channel</i>	
----------------	--

DMA channel containing the buffer to be written to.

Parameters

<i>buffer_to_write↔ _to</i>	
---------------------------------	--

Buffer in channel to write to.

Parameters

<i>buffer_size</i>	
--------------------	--

Number of bytes to write to the DMA buffer. In most cases, this should be equal to the allocated size of the buffer.

Parameters

<i>local_buffer_ptr</i>	
-------------------------	--

Pointer to local memory buffer already allocated where data will come from.

Return values

0	
---	--

Success.

Return values

Non-Zero	
----------	--

Failure. errno may be set as follows:

- `EINVAL` Invalid channel or buffer requested.

Referenced by [main\(\)](#), and [setup_dacs_and_start\(\)](#).

4.17.2.4 DM35424_General_InstallISR()

```
int DM35424_General_InstallISR (  
    struct DM35424_Board_Descriptor * handle,  
    void * isr_fnct)
```

Start a thread that will sit and wait for an interrupt from the board, and call the user ISR when it happens.

Parameters

<i>handle</i>	
---------------	--

Pointer to the device descriptor, which contains the open file id.

Parameters

<i>isr_fnct</i>	
-----------------	--

Pointer to the user ISR function that will be executed when an interrupt happens.

Return values

0	
---	--

Success.

Return values

-1	
----	--

Failure. errno may be set as follows:

- EFAULT Could not create thread.

Referenced by [main\(\)](#).

4.17.2.5 DM35424_General_RemoveISR()

```
int DM35424_General_RemoveISR (  
    struct DM35424_Board_Descriptor * handle)
```

Remove the ISR from the system interrupt.

Parameters

<i>handle</i>	
---------------	--

Pointer to the device descriptor, which contains the open file id.

Return values

0	
---	--

Success.

Return values

-1	
----	--

Failure. errno may be set as follows:

- EFAULT User ISR was already removed.

Referenced by [main\(\)](#).

4.17.2.6 DM35424_General_SetISRPriority()

```
int DM35424_General_SetISRPriority (
    struct DM35424_Board_Descriptor * handle,
    int priority)
```

Set the priority of the user ISR thread.

Parameters

<i>handle</i>	
---------------	--

Pointer to the device descriptor, which contains the open file id.

Parameters

<i>priority</i>	
-----------------	--

Attempt to set the priority of the user ISR thread.

Note

This may require root priviledges

Return values

<i>0</i>	
----------	--

Success.

Return values

<i>-1</i>	
-----------	--

Failure. errno may be set as follows:

- EFAULT User ISR did not exist.

4.17.2.7 DM35424_General_WaitForInterrupt()

```
void * DM35424_General_WaitForInterrupt (
    void * ptr)
```

Loop/Poll and wait for an interrupt to happen, then take action.

Parameters

<i>ptr</i>	
------------	--

A void pointer for the board descriptor.

Return values

0	
---	--

Success.

Return values

-1	
----	--

Failure. errno may be set as follows:

- `ENODATA` File descriptor is missing or unreadable. `EIO` There was no interrupt allocated to this device.

4.18 DM35424 Reference Adjustment Library Constants

Macros

- `#define DM35424_REF_ADJUST_SPI_BUSY 0x00`
Register value for SPI Interface is busy.
- `#define DM35424_REF_ADJUST_SPI_READY 0x01`
Register value for SPI Interface is ready.
- `#define DM35424_REF_ADJUST_START_TRANS 0x01`
Register value for starting the SPI transaction.
- `#define DM35424_REF_ADJUST_WRITE_ADC_VOLATILE 0x0100`
Register value for writing to the ADC Volatile memory.
- `#define DM35424_REF_ADJUST_WRITE_DAC_VOLATILE 0x0200`
Register value for writing to the DAC Volatile memory.
- `#define DM35424_REF_ADJUST_WRITE_ADC_NON_VOLATILE 0x1100`
Register value for writing to the ADC Non-Volatile memory.
- `#define DM35424_REF_ADJUST_WRITE_DAC_NON_VOLATILE 0x1200`
Register value for writing to the DAC Non-Volatile memory.
- `#define DM35424_REF_ADJUST_COPY_ADC_VOL_TO_NON 0x2100`
Register value for copying ADC data from Volatile to Non-Volatile.
- `#define DM35424_REF_ADJUST_COPY_DAC_VOL_TO_NON 0x2200`
Register value for copying DAC data from Volatile to Non-Volatile.
- `#define DM35424_REF_ADJUST_COPY_BOTH_VOL_TO_NON 0x2300`
Register value for copying ADC and DAC data from Volatile to Non-Volatile.
- `#define DM35424_REF_ADJUST_COPY_ADC_NON_TO_VOL 0x3100`
Register value for copying ADC data from Non-Volatile to Volatile.
- `#define DM35424_REF_ADJUST_COPY_DAC_NON_TO_VOL 0x3200`
Register value for copying DAC data from Non-Volatile to Volatile.
- `#define DM35424_REF_ADJUST_COPY_BOTH_NON_TO_VOL 0x3300`
Register value for copying ADC and DAC data from Non-Volatile to Volatile.

Enumerations

- enum [DM35424_Copy_Directions](#) {
[DM35424_ADC_VOL_TO_NON_VOL](#) , [DM35424_DAC_VOL_TO_NON_VOL](#) , [DM35424_BOTH_VOL_TO_NON_VOL](#)
[DM35424_ADC_NON_VOL_TO_VOL](#) ,
[DM35424_DAC_NON_VOL_TO_VOL](#) , [DM35424_BOTH_NON_VOL_TO_VOL](#) }

Direction of Reference Adjustment data copy action.

4.18.1 Detailed Description

4.18.2 Enumeration Type Documentation

4.18.2.1 DM35424_Copy_Directions

enum [DM35424_Copy_Directions](#)

Direction of Reference Adjustment data copy action.

Enumerator

DM35424_ADC_VOL_TO_NON_VOL	ADC Volatile to Non-Volatile
DM35424_DAC_VOL_TO_NON_VOL	DAC Volatile to Non-Volatile
DM35424_BOTH_VOL_TO_NON_VOL	ADC and DAC Volatile to Non-Volatile
DM35424_ADC_NON_VOL_TO_VOL	ADC Non-Volatile to Volatile
DM35424_DAC_NON_VOL_TO_VOL	DAC Non-Volatile to Volatile
DM35424_BOTH_NON_VOL_TO_VOL	ADC and DAC Non-Volatile to Volatile

Definition at line 129 of file [dm35424_ref_adjust_library.h](#).

4.19 DM35424 Reference Adjustment Public Library Functions

Functions

- [DM35424LIB_API](#) int [DM35424_Ref_Adjust_Open](#) (struct [DM35424_Board_Descriptor](#) *handle, unsigned int ordinal_to_open, struct [DM35424_Function_Block](#) *fb_temp)
Open the reference adjustment function block, getting address values that will be used later by other library functions.
- [DM35424LIB_API](#) int [DM35424_Ref_Adjust_Write_Adc_To_Volatile](#) (struct [DM35424_Board_Descriptor](#) *handle, struct [DM35424_Function_Block](#) *fb, uint8_t adjustment)
Write the ADC Reference Adjustment value to volatile memory.
- [DM35424LIB_API](#) int [DM35424_Ref_Adjust_Write_Adc_To_NonVolatile](#) (struct [DM35424_Board_Descriptor](#) *handle, struct [DM35424_Function_Block](#) *fb, uint8_t adjustment)
Write the ADC Reference Adjustment value to non-volatile memory.
- [DM35424LIB_API](#) int [DM35424_Ref_Adjust_Write_Dac_To_Volatile](#) (struct [DM35424_Board_Descriptor](#) *handle, struct [DM35424_Function_Block](#) *fb, uint8_t adjustment)
Write the DAC Reference Adjustment value to volatile memory.
- [DM35424LIB_API](#) int [DM35424_Ref_Adjust_Write_Dac_To_NonVolatile](#) (struct [DM35424_Board_Descriptor](#) *handle, struct [DM35424_Function_Block](#) *fb, uint8_t adjustment)
Write the DAC Reference Adjustment value to non-volatile memory.
- [DM35424LIB_API](#) int [DM35424_Ref_Adjust_Copy_Data](#) (struct [DM35424_Board_Descriptor](#) *handle, struct [DM35424_Function_Block](#) *fb, enum [DM35424_Copy_Directions](#) direction)
Copy the reference adjustment data from volatile to non-volatile, or vice versa.

4.19.1 Detailed Description

DM35424_Ref_Adjust_Library_Constants

4.19.2 Function Documentation

4.19.2.1 DM35424_Ref_Adjust_Copy_Data()

```
DM35424LIB_API int DM35424_Ref_Adjust_Copy_Data (  
    struct DM35424_Board_Descriptor * handle,  
    struct DM35424_Function_Block * fb,  
    enum DM35424_Copy_Directions direction)
```

Copy the reference adjustment data from volatile to non-volatile, or vice versa.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>fb</i>	
-----------	--

Address of the function block that contains the reference adjustment we're using.

Parameters

<i>direction</i>	
------------------	--

Direction of the copy, including whether it is ADC, DAC or both.

Return values

<i>0</i>	
----------	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

4.19.2.2 DM35424_Ref_Adjust_Open()

```
DM35424LIB_API int DM35424_Ref_Adjust_Open (  
    struct DM35424_Board_Descriptor * handle,  
    unsigned int ordinal_to_open,  
    struct DM35424_Function_Block * fb_temp)
```

Open the reference adjustment function block, getting address values that will be used later by other library functions.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>ordinal_to_open</i>	
------------------------	--

Which function block on the board to open (0th, 1st, 2nd, etc)

Parameters

<i>fb_temp</i>	
----------------	--

Pointer to function block structure that will hold register offset values.

Return values

0	
---	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#).

4.19.2.3 DM35424_Ref_Adjust_Write_Adc_To_NonVolatile()

```
DM35424LIB_API int DM35424_Ref_Adjust_Write_Adc_To_NonVolatile (  
    struct DM35424_Board_Descriptor * handle,  
    struct DM35424_Function_Block * fb,  
    uint8_t adjustment)
```

Write the ADC Reference Adjustment value to non-volatile memory.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>fb</i>	
-----------	--

Address of the function block that contains the reference adjustment we're using.

Parameters

<i>adjustment</i>	
-------------------	--

Reference adjustment value.

Return values

<i>0</i>	
----------	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#).

4.19.2.4 DM35424_Ref_Adjust_Write_Adc_To_Volatile()

```
DM35424LIB_API int DM35424_Ref_Adjust_Write_Adc_To_Volatile (  
    struct DM35424_Board_Descriptor * handle,  
    struct DM35424_Function_Block * fb,  
    uint8_t adjustment)
```

Write the ADC Reference Adjustment value to volatile memory.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>fb</i>	
-----------	--

Address of the function block that contains the reference adjustment we're using.

Parameters

<i>adjustment</i>	
-------------------	--

Reference adjustment value.

Return values

0	
---	--

Success.

Return values

Non-Zero	
----------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#).

4.19.2.5 DM35424_Ref_Adjust_Write_Dac_To_NonVolatile()

```
DM35424LIB_API int DM35424_Ref_Adjust_Write_Dac_To_NonVolatile (  
    struct DM35424_Board_Descriptor * handle,  
    struct DM35424_Function_Block * fb,  
    uint8_t adjustment)
```

Write the DAC Reference Adjustment value to non-volatile memory.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>fb</i>	
-----------	--

Address of the function block that contains the reference adjustment we're using.

Parameters

<i>adjustment</i>	
-------------------	--

Reference adjustment value.

Return values

0	
---	--

Success.

Return values

Non-Zero	
----------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#).

4.19.2.6 DM35424_Ref_Adjust_Write_Dac_To_Volatile()

```
DM35424LIB_API int DM35424_Ref_Adjust_Write_Dac_To_Volatile (  
    struct DM35424_Board_Descriptor * handle,  
    struct DM35424_Function_Block * fb,  
    uint8_t adjustment)
```

Write the DAC Reference Adjustment value to volatile memory.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>fb</i>	
-----------	--

Address of the function block that contains the reference adjustment we're using.

Parameters

<i>adjustment</i>	
-------------------	--

Reference adjustment value.

Return values

<i>0</i>	
----------	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#).

4.20 DM35424 Register Offsets

Macros

- #define **DM35424_OFFSET_GBC_FORMAT** 0x00
Offset to General Board Control (BAR0) Format ID register.
- #define **DM35424_OFFSET_GBC_REV** 0x01
Offset to General Board Control (BAR0) Format ID register.
- #define **DM35424_OFFSET_GBC_END_INTERRUPT** 0x02
Offset to General Board Control (BAR0) EOI (End of Interrupt) register.
- #define **DM35424_OFFSET_GBC_BOARD_RESET** 0x03
Offset to General Board Control (BAR0) Board Reset register.
- #define **DM35424_OFFSET_GBC_PDP_NUMBER** 0x04
Offset to General Board Control (BAR0) PDP Number register.
- #define **DM35424_OFFSET_GBC_FPGA_BUILD** 0x08
Offset to General Board Control (BAR0) FPGA Build register.
- #define **DM35424_OFFSET_GBC_SYS_CLK_FREQ** 0x0c
Offset to General Board Control (BAR0) System Clock register.
- #define **DM35424_OFFSET_GBC_IRQ_STATUS** 0x10
Offset to General Board Control (BAR0) IRQ Status register. Each bit corresponds to a function block.
- #define **DM35424_OFFSET_GBC_DMA_IRQ_STATUS** 0x18
Offset to General Board Control (BAR0) DMA IRQ Status register. Each bit corresponds to a function block.
- #define **DM35424_OFFSET_GBC_FB_START** 0x20
Offset to the beginning of the Function Blocks section of the GBC.
- #define **DM35424_GBC_FB_BLK_SIZE** 0x10
Size of the function block entries in the GBC.
- #define **DM35424_OFFSET_GBC_FB_ID** 0x00
Offset to Function Block ID, from the start of the function block section.
- #define **DM35424_FB_ID_TYPE_MASK** 0x0000FFFF
Bit mask for TYPE portion of FB ID.
- #define **DM35424_FB_ID_SUBTYPE_MASK** 0x00FF0000
Bit mask for SUBTYPE portion of FB ID.
- #define **DM35424_FB_ID_TYPE_REV_MASK** 0xFF000000
Bit mask for TYPE REV portion of FB ID.
- #define **DM35424_OFFSET_GBC_FB_OFFSET** 0x04
Offset to the FB Offset in the GBC, from the start of the FB data block.
- #define **DM35424_OFFSET_GBC_FB_DMA_OFFSET** 0x08
Offset to the FB DMA Offset in the GBC, from the start of the FB data block.
- #define **DM35424_OFFSET_DMA_ACTION** 0x00
Offset to the DMA Action Register (BAR2).
- #define **DM35424_OFFSET_DMA_SETUP** 0x01
Offset to the DMA Setup Register (BAR2).
- #define **DM35424_OFFSET_DMA_STAT_OVERFLOW** 0x02
Offset to the DMA Status (Overflow) Register (BAR2).
- #define **DM35424_OFFSET_DMA_STAT_UNDERFLOW** 0x03
Offset to the DMA Status (Underflow) Register (BAR2).
- #define **DM35424_OFFSET_DMA_CURRENT_COUNT** 0x04
Offset to the DMA Current Count Register (BAR2).
- #define **DM35424_OFFSET_DMA_CURRENT_BUFFER** 0x07
Offset to the DMA Current Buffer Register (BAR2).

- **#define DM35424_OFFSET_DMA_WR_FIFO_CNT** 0x08
Offset to the DMA Write FIFO Count Register (BAR2).
- **#define DM35424_OFFSET_DMA_RD_FIFO_CNT** 0x0A
Offset to the DMA Read FIFO Count Register (BAR2).
- **#define DM35424_OFFSET_DMA_STAT_USED** 0x0C
Offset to the DMA Status (Used) Register (BAR2).
- **#define DM35424_OFFSET_DMA_STAT_INVALID** 0x0D
Offset to the DMA Status (Invalid) Register (BAR2).
- **#define DM35424_OFFSET_DMA_STAT_COMPLETE** 0x0E
Offset to the DMA Status (Complete) Register (BAR2).
- **#define DM35424_OFFSET_DMA_LAST_ACTION** 0x0F
Offset to the DMA Last Action Register (BAR2).
- **#define DM35424_OFFSET_DMA_BUFF_START** 0x10
Offset to the start of the buffer control section (BAR2).
- **#define DM35424_OFFSET_DMA_BUFFER_STAT** 0x02
Offset to the buffer status register, from the start of the buffer control section (BAR2).
- **#define DM35424_OFFSET_DMA_BUFFER_CTRL** 0x03
Offset to the buffer control register, from the start of the buffer control section (BAR2).
- **#define DM35424_OFFSET_DMA_BUFFER_SIZE** 0x04
Offset to the buffer size register, from the start of the buffer control section (BAR2).
- **#define DM35424_OFFSET_DMA_BUFFER_ADDRESS** 0x08
Offset to the buffer address register, from the start of the buffer control section (BAR2).
- **#define DM35424_OFFSET_FB_DMA_CHANNELS** 0x06
Offset to the DMA Channels count of the function block (BAR2).
- **#define DM35424_OFFSET_FB_DMA_BUFFERS** 0x07
Offset to the DMA buffers count of the function block (BAR2).
- **#define DM35424_OFFSET_FB_CTRL_START** 0x08
Offset to the beginning of the Function Block control section in BAR2.
- **#define DM35424_OFFSET_ADC_MODE_STATUS** 0x00
Offset to the ADC Mode-Status register, from the start of the ADC control section.
- **#define DM35424_OFFSET_ADC_CLK_SRC** 0x01
Offset to the ADC Clock Source register, from the start of the ADC control section.
- **#define DM35424_OFFSET_ADC_START_TRIG** 0x02
Offset to the ADC Start Trigger register, from the start of the ADC control section.
- **#define DM35424_OFFSET_ADC_STOP_TRIG** 0x03
Offset to the ADC Stop Trigger register, from the start of the ADC control section.
- **#define DM35424_OFFSET_ADC_CLK_DIV** 0x04
Offset to the ADC Clock Divider register, from the start of the ADC control section.
- **#define DM35424_OFFSET_ADC_CLK_DIV_COUNTER** 0x08
Offset to the ADC Clock Divider Counter register, from the start of the ADC control section.
- **#define DM35424_OFFSET_ADC_PRE_CAPT_COUNT** 0x0c
Offset to the ADC Pre-Start Capture Count register, from the start of the ADC control section.
- **#define DM35424_OFFSET_ADC_POST_CAPT_COUNT** 0x10
Offset to the ADC Post-Stop Capture Count register, from the start of the ADC control section.
- **#define DM35424_OFFSET_ADC_SAMPLE_COUNT** 0x14
Offset to the ADC Sample Count register, from the start of the ADC control section.
- **#define DM35424_OFFSET_ADC_INT_ENABLE** 0x18
Offset to the ADC Interrupt Enable register, from the start of the ADC control section.
- **#define DM35424_OFFSET_ADC_INT_STAT** 0x1e
Offset to the ADC Interrupt Status register, from the start of the ADC control section.
- **#define DM35424_OFFSET_ADC_CLK_BUS2** 0x22

- Offset to the ADC Clock Bus 2, from the start of the ADC control section.*

 - #define **DM35424_OFFSET_ADC_CLK_BUS3** 0x23
- Offset to the ADC Clock Bus 3 register, from the start of the ADC control section.*

 - #define **DM35424_OFFSET_ADC_CLK_BUS4** 0x24
- Offset to the ADC Clock Bus 4 register, from the start of the ADC control section.*

 - #define **DM35424_OFFSET_ADC_CLK_BUS5** 0x25
- Offset to the ADC Clock Bus 5 register, from the start of the ADC control section.*

 - #define **DM35424_OFFSET_ADC_CLK_BUS6** 0x26
- Offset to the ADC Clock Bus 6 register, from the start of the ADC control section.*

 - #define **DM35424_OFFSET_ADC_CLK_BUS7** 0x27
- Offset to the ADC Clock Bus 7 register, from the start of the ADC control section.*

 - #define **DM35424_OFFSET_ADC_AD_CONFIG** 0x28
- Offset to the ADC AD Config register, from the start of the ADC control section.*

 - #define **DM35424_OFFSET_ADC_CHAN_CTRL_BLK_START** 0x2c
- Offset to the start of the Channel Control Section, from the start of the ADC control section.*

 - #define **DM35424_ADC_CHAN_CTRL_BLK_SIZE** 0x18
- Constant size of ADC channel section in function block.*

 - #define **DM35424_OFFSET_ADC_CHAN_FRONT_END_CONFIG** 0x00
- Offset to the Channel Front End Config register, from the start of the ADC channel control section.*

 - #define **DM35424_OFFSET_ADC_CHAN_DATA_COUNT** 0x04
- Offset to the Channel FIFO Data count register, from the start of the ADC channel control section.*

 - #define **DM35424_OFFSET_ADC_CHAN_FILTER** 0x09
- Offset to the Channel Filter register, from the start of the ADC channel control section.*

 - #define **DM35424_OFFSET_ADC_CHAN_INTR_STAT** 0x0a
- Offset to the Channel Interrupt Status register, from the start of the ADC channel control section.*

 - #define **DM35424_OFFSET_ADC_CHAN_INTR_ENABLE** 0x0b
- Offset to the Channel Interrupt Enable register, from the start of the ADC channel control section.*

 - #define **DM35424_OFFSET_ADC_CHAN_LOW_THRESHOLD** 0x0c
- Offset to the Channel Low Threshold register, from the start of the ADC channel control section.*

 - #define **DM35424_OFFSET_ADC_CHAN_HIGH_THRESHOLD** 0x10
- Offset to the Channel High Threshold register, from the start of the ADC channel control section.*

 - #define **DM35424_OFFSET_ADC_CHAN_LAST_SAMPLE** 0x14
- Offset to the Channel Last Sample register, from the start of the ADC channel control section.*

 - #define **DM35424_OFFSET_ADC_FIFO_CTRL_BLK_START** 0x334
- Offset to the start of the FIFO Control Section, from the start of the ADC control section.*

 - #define **DM35424_ADC_FIFO_CTRL_BLK_SIZE** 0x4
- Constant size of ADC FIFO section in function block.*

 - #define **DM35424_OFFSET_FB_ADC_FIFO** 0x0334
- Offset to the FIFO for non-DMA read and write operations.*

 - #define **DM35424_OFFSET_DAC_MODE_STATUS** 0x00
- Offset to the Mode/Status register, from the start of the DAC control section.*

 - #define **DM35424_OFFSET_DAC_CLK_SRC** 0x01
- Offset to the Clock Source register, from the start of the DAC control section.*

 - #define **DM35424_OFFSET_DAC_START_TRIG** 0x02
- Offset to the Start Trigger register, from the start of the DAC control section.*

 - #define **DM35424_OFFSET_DAC_STOP_TRIG** 0x03
- Offset to the Stop Trigger register, from the start of the DAC control section.*

 - #define **DM35424_OFFSET_DAC_CLK_DIV** 0x04
- Offset to the Clock Divider register, from the start of the DAC control section.*

 - #define **DM35424_OFFSET_DAC_CLK_DIV_COUNT** 0x08
- Offset to the Clock Divider Counter register, from the start of the DAC control section.*

- **#define DM35424_OFFSET_DAC_POST_STOP_CONV** 0x10
Offset to the Post-Stop Conversion Count register, from the start of the DAC control section.
- **#define DM35424_OFFSET_DAC_CONV_COUNT** 0x14
Offset to the Conversion Count register, from the start of the DAC control section.
- **#define DM35424_OFFSET_DAC_INT_ENABLE** 0x18
Offset to the Interrupt Enable register, from the start of the DAC control section.
- **#define DM35424_OFFSET_DAC_INT_STAT** 0x1e
Offset to the Interrupt Status register, from the start of the DAC control section.
- **#define DM35424_OFFSET_DAC_CLK_BUS2** 0x22
Offset to the Clock Bus 2 register, from the start of the DAC control section.
- **#define DM35424_OFFSET_DAC_CLK_BUS3** 0x23
Offset to the Clock Bus 3 register, from the start of the DAC control section.
- **#define DM35424_OFFSET_DAC_CLK_BUS4** 0x24
Offset to the Clock Bus 4 register, from the start of the DAC control section.
- **#define DM35424_OFFSET_DAC_CLK_BUS5** 0x25
Offset to the Clock Bus 5 register, from the start of the DAC control section.
- **#define DM35424_OFFSET_DAC_CLK_BUS6** 0x26
Offset to the Clock Bus 6 register, from the start of the DAC control section.
- **#define DM35424_OFFSET_DAC_CLK_BUS7** 0x27
Offset to the Clock Bus 7 register, from the start of the DAC control section.
- **#define DM35424_OFFSET_DAC_DA_CONFIG** 0x28
Offset to the DA Config register, from the start of the DAC control section.
- **#define DM35424_OFFSET_DAC_CHAN_CTRL_BLK_START** 0x2c
Offset to the start of the DAC channel control section, from the start of the DAC control section.
- **#define DM35424_DAC_CHAN_CTRL_BLK_SIZE** 0x14
Constant size of channel control section in function block.
- **#define DM35424_OFFSET_DAC_CHAN_FRONT_END_CONFIG** 0x00
Offset to the Front-End Config register, from the start of the DAC channel control section.
- **#define DM35424_OFFSET_DAC_CHAN_MARKER_STATUS** 0x0a
Offset to the Channel marker Interrupt Status register, from the start of the DAC channel control section.
- **#define DM35424_OFFSET_DAC_CHAN_MARKER_ENABLE** 0x0b
Offset to the Channel marker Interrupt Enable register, from the start of the DAC channel control section.
- **#define DM35424_OFFSET_DAC_CHAN_LAST_CONVERSION** 0x10
Offset to the Channel Last Conversion register, from the start of the DAC channel control section.
- **#define DM35424_OFFSET_DAC_FIFO_CTRL_BLK_START** 0x84
Offset to the start of the DAC FIFO control section, from the start of the DAC control section.
- **#define DM35424_OFFSET_DAC_FIFO_CTRL_BLK_SIZE** 0x4
Constant size of FIFO control section in function block.
- **#define DM35424_OFFSET_DIO_INPUT_VAL** 0x00
Offset to the Input Value register, from the start of the DIO control section.
- **#define DM35424_OFFSET_DIO_OUTPUT_VAL** 0x04
Offset to the Output Value register, from the start of the DIO control section.
- **#define DM35424_OFFSET_DIO_DIRECTION** 0x08
Offset to the Direction register, from the start of the DIO control section.
- **#define DM35424_OFFSET_TEMPERATURE** 0x00
Offset to the Temperature register, from the start of the Temperature control section.
- **#define DM35424_OFFSET_REF_ADJUST_GO_BUSY** 0x00
Offset to the Go/Busy register, from the start of the Reference Adjustment control section.
- **#define DM35424_OFFSET_REF_OUTPUT_LATCH** 0x04
Offset to the output latch register, from the start of the Reference Adjustment control section.

4.20.1 Detailed Description

4.20.2 Macro Definition Documentation

4.20.2.1 DM35424_OFFSET_FB_ADC_FIFO

```
#define DM35424_OFFSET_FB_ADC_FIFO 0x0334
```

Offset to the FIFO for non-DMA read and write operations.

Note

This value should be used directly. It is used in conjunction with a channel number.

Definition at line 495 of file [dm35424_registers.h](#).

4.21 DM35424 Temperature

Functions

- [DM35424LIB_API](#) int [DM35424_Temperature_Open](#) (struct [DM35424_Board_Descriptor](#) *handle, unsigned int ordinal_to_open, struct [DM35424_Function_Block](#) *fb_temp)
Open the temperature function block, getting address values that will be used later by other library functions.
- [DM35424LIB_API](#) int [DM35424_Temperature_Read](#) (struct [DM35424_Board_Descriptor](#) *handle, struct [DM35424_Function_Block](#) *temp_fb, float *temperature)
Read the temperature of the board, in Celsius degrees.

4.21.1 Detailed Description

Public Library Functions

4.21.2 Function Documentation

4.21.2.1 DM35424_Temperature_Open()

```
DM35424LIB_API int DM35424_Temperature_Open (
    struct DM35424_Board_Descriptor * handle,
    unsigned int ordinal_to_open,
    struct DM35424_Function_Block * fb_temp)
```

Open the temperature function block, getting address values that will be used later by other library functions.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>ordinal_to_open</i>	
------------------------	--

Which function block on the board to open (0th, 1st, 2nd, etc)

Parameters

<i>fb_temp</i>	
----------------	--

Pointer to function block structure that will hold register offset values.

Return values

0	
---	--

Success.

Return values

Non-Zero	
----------	--

Failure.

References [DM35424LIB_API](#).

Referenced by [main\(\)](#).

4.21.2.2 DM35424_Temperature_Read()

```
DM35424LIB_API int DM35424_Temperature_Read (  
    struct DM35424_Board_Descriptor * handle,  
    struct DM35424_Function_Block * temp_fb,  
    float * temperature)
```

Read the temperature of the board, in Celsius degrees.

Parameters

<i>handle</i>	
---------------	--

Address of the handle pointer, which will contain the device descriptor.

Parameters

<i>temp</i> ↔	
<i>_fb</i>	

Pointer to function block we want to read.

Parameters

<i>temperature</i>	
--------------------	--

Pointer to a preallocated float. On output holds the temperature in Celsius degree.

Return values

<i>0</i>	
----------	--

Success.

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

Referenced by [main\(\)](#).

4.22 DM35424 Board Types

Macros

- **#define DM35424_SUBTYPE_00 0**
Constant for FB subtype 0.
- **#define DM35424_SUBTYPE_01 1**
Constant for FB subtype 1.
- **#define DM35424_SUBTYPE_02 2**
Constant for FB subtype 2.
- **#define DM35424_SUBTYPE_03 3**
Constant for FB subtype 3.
- **#define DM35424_SUBTYPE_INVALID 0xFF**
Constant value indicating an invalid subtype.
- **#define DM35424_FUNC_BLOCK_INVALID 0x0000**
Constant value indicating an invalid function block.
- **#define DM35424_FUNC_BLOCK_INVALID2 0xFFFF**
Constant value indicating an invalid function block.
- **#define DM35424_FUNC_BLOCK_SYNCBUS 0x0001**
Function Block Constant for SyncBus.
- **#define DM35424_FUNC_BLOCK_EXT_CLOCKING 0x0002**

- Function Block Constant for Global Clocking.*

 - **#define DM35424_FUNC_BLOCK_CLK0003** 0x0003
- Function Block Constant for External Clocking (0003).*

 - **#define DM35424_FUNC_BLOCK_CAPTWIN** 0x0005
- Function Block Constant for Capture Window.*

 - **#define DM35424_FUNC_BLOCK_ADC** 0x1000
- Function Block Constant for ADC.*

 - **#define DM35424_FUNC_BLOCK_ADC1001** 0x1001
- Function Block Constant for 10 MHz ADC (1001).*

 - **#define DM35424_FUNC_BLOCK_ADC1002** 0x1002
- Function Block Constant for SDM35541-based ADC (1002).*

 - **#define DM35424_FUNC_BLOCK_DAC** 0x2000
- Function Block Constant for DAC.*

 - **#define DM35424_FUNC_BLOCK_DAC2001** 0x2001
- Function Block Constant for High Speed DAC (2001).*

 - **#define DM35424_FUNC_BLOCK_DIO** 0x3000
- Function Block Constant for DIO.*

 - **#define DM35424_FUNC_BLOCK_ADIO** 0x3001
- Function Block Constant for ADIO.*

 - **#define DM35424_FUNC_BLOCK_ADIO3010** 0x3010
- Function Block Constant for ADIO3010.*

 - **#define DM35424_FUNC_BLOCK_USART** 0x4000
- Function Block Constant for Synchronous/Asynchronous Serial Port.*

 - **#define DM35424_FUNC_BLOCK_REF_ADJUST** 0xF000
- Function Block Constant for Reference Adjustment.*

 - **#define DM35424_FUNC_BLOCK_TEMPERATURE_SENSOR** 0xF001
- Function Block Constant for Temperature Sensor.*

 - **#define DM35424_FUNC_BLOCK_FLASH_PROGRAMMER** 0xF002
- Function Block Constant for Module Firmware Programmer.*

 - **#define DM35424_FUNC_BLOCK_CLK_GEN** 0xF003
- Function Block Constant for Clock Generator.*

 - **#define DM35424_FUNC_BLOCK_DIN3011** 0x3011
- Function Block Constant for Digital Input (3011).*

 - **#define DM35424_FUNC_BLOCK_DOT3012** 0x3012
- Function Block Constant for Digital Output (3012).*

 - **#define DM35424_FUNC_BLOCK_INC3200** 0x3200
- Function Block Constant for Incremental Encoder (3200).*

 - **#define DM35424_FUNC_BLOCK_PWM3100** 0x3100
- Function Block Constant for PWM (3100).*

 - **#define DM35424_FUNC_BLOCK_CLK0004** 0x0004
- Function Block Constant for Programmable Clock (0004).*

 - **#define DM35424_FUNC_BLOCK_I2C3300** 0x3300
- Function Block Constant for I2C Bus Master (3300).*

 - **#define DM35424_MAX_FB** 62
- Maximum possible number of function blocks on a board.*

 - **#define MAX_DMA_BUFFERS** 16
- Maximum possible number of DMA buffers for any function block.*

 - **#define MAX_DMA_CHANNELS** 32
- Maximum possible number of DMA channels for any function block.*

 - **#define DM35424_DMA_MAX_BUFFER_SIZE** 0xFFFFFC
- Maximum possible DMA buffer size.*

- `#define DM35424_BOARD_ACK_INTERRUPT 0x1`
Value to write to the EOI register to acknowledge interrupts.
- `#define DM35424_BOARD_RESET_VALUE 0xAA`
Value to write to the Reset register in order to reset the board.
- `#define DM35424_FIFO_ACCESS_FB_REVISION 0x01`
Minimum function block revision that supports direct FIFO read/write access.

4.22.1 Detailed Description

4.23 DM35424 Utility Library Functions

Enumerations

- enum `DM35424_Waveforms` { `DM35424_SINE_WAVE`, `DM35424_SQUARE_WAVE`, `DM35424_SAWTOOTH_WAVE` }
List of possible waveforms that can be generated for DAC purposes.

Functions

- `uint32_t DM35424_Get_Maskable` (`uint16_t data`, `uint16_t mask`)
Return a 32-bit maskable register value from the data and mask.
- `void DM35424_Micro_Sleep` (`unsigned long microseconds`)
Sleep for a specified number of microseconds.
- `long DM35424_Get_Time_Diff` (`struct timeval last`, `struct timeval first`)
Calculate the time difference between the two timeval structs, in microseconds.
- `int DM35424_Generate_Signal_Data` (`enum DM35424_Waveforms waveform`, `int32_t *data`, `uint32_t data_count`, `int32_t max`, `int32_t minimum`, `int32_t offset`, `uint32_t mask`)
Generate data with a specific wave pattern. This is useful for producing recognizable waves for DAC output.
- `void DM35424_Check_Result` (`int return_val`, `char *message`)
Check the result of an operation, usually a library call. If the result is non-zero, then it is an error and output the passed message.

4.23.1 Detailed Description

4.23.2 Enumeration Type Documentation

4.23.2.1 DM35424_Waveforms

enum `DM35424_Waveforms`

List of possible waveforms that can be generated for DAC purposes.

Enumerator

<code>DM35424_SINE_WAVE</code>	A simple sine wave.
--------------------------------	---------------------

DM35424_SQUARE_WAVE	A square wave starting at max value
DM35424_SAWTOOTH_WAVE	A sawtooth wave going for min to max

Definition at line 46 of file [dm35424_util_library.h](#).

4.23.3 Function Documentation

4.23.3.1 DM35424_Check_Result()

```
void DM35424_Check_Result (
    int return_val,
    char * message)
```

Check the result of an operation, usually a library call. If the result is non-zero, then it is an error and output the passed message.

Parameters

<i>return_val</i>	
-------------------	--

Value to be evaluated. Non-zero values will be considered an error.

Parameters

<i>message</i>	
----------------	--

Pointer to string that will be output if an error condition exists.

Return values

<i>None</i>	
-------------	--

Referenced by [ISR\(\)](#), [main\(\)](#), [output_channel_status\(\)](#), [setup_adc\(\)](#), and [setup_dacs_and_start\(\)](#).

4.23.3.2 DM35424_Generate_Signal_Data()

```
int DM35424_Generate_Signal_Data (
    enum DM35424_Waveforms waveform,
    int32_t * data,
    uint32_t data_count,
    int32_t max,
    int32_t minimum,
    int32_t offset,
    uint32_t mask)
```

Generate data with a specific wave pattern. This is useful for producing recognizeable waves for DAC output.

Parameters

<i>waveform</i>	
-----------------	--

Enumerated value indicating what waveform to produce.

Parameters

<i>data</i>	
-------------	--

Pointer to pre-allocated memory to hold resulting data values.

Parameters

<i>data_count</i>	
-------------------	--

Number of data samples to produce

Parameters

<i>max</i>	
------------	--

The maximum value in this generated data.

Parameters

<i>minimum</i>	
----------------	--

The minimum value in this generated data.

Parameters

<i>offset</i>	
---------------	--

Offset from 0 that will be the median value of this wave.

Parameters

<i>mask</i>	
-------------	--

Bitmask that is applied to every calculated value. This allows for handling of generated data that is less than 32 bits. To use all 32-bits, the mask would be 0xFFFFFFFF.

Note

No matter the data count, the returned data will only contain 1 period of the waveform. A higher data count will result in a "finer" set of data.

Return values

<i>0</i>	
----------	--

Success

Return values

<i>Non-Zero</i>	
-----------------	--

Failure

Referenced by [main\(\)](#), and [setup_dacs_and_start\(\)](#).

4.23.3.3 DM35424_Get_Maskable()

```
uint32_t DM35424_Get_Maskable (  
    uint16_t data,  
    uint16_t mask)
```

Return a 32-bit maskable register value from the data and mask.

Parameters

<i>data</i>	
-------------	--

Data portion (upper 16-bits) of the maskable.

Parameters

<i>mask</i>	
-------------	--

Mask portion (lower 16-bits) of the maskable

Return values

<i>maskable</i>	
-----------------	--

Maskable register value.

4.23.3.4 DM35424_Get_Time_Diff()

```
long DM35424_Get_Time_Diff (
    struct timeval last,
    struct timeval first)
```

Calculate the time difference between the two timeval structs, in microseconds.

Parameters

<i>last</i>	
-------------	--

The last (most recent) timeval to compare.

Parameters

<i>first</i>	
--------------	--

The first (least recent) timeval to compare.

Return values

<i>difference</i>	
-------------------	--

The difference between the two timevals, in microseconds.

Referenced by [main\(\)](#).

4.23.3.5 DM35424_Micro_Sleep()

```
void DM35424_Micro_Sleep (
    unsigned long microsecs)
```

Sleep for a specified number of microseconds.

Parameters

<i>microsecs</i>	
------------------	--

Length of sleep (microseconds)

Return values

<i>None</i>	
-------------	--

Referenced by [main\(\)](#), and [run_test_9\(\)](#).

Chapter 5

Data Structure Documentation

5.1 DM35424_Board_Descriptor Struct Reference

DM35424 board descriptor. This structure holds information about the board as a whole. It holds the file descriptor and ISR callback function, if applicable.

```
#include <dm35424_os.h>
```

Data Fields

- int [file_descriptor](#)
- void(* [isr](#))(struct [dm35424_ioctl_interrupt_info_request](#) info_req)
- pthread_t [pid](#)

5.1.1 Detailed Description

DM35424 board descriptor. This structure holds information about the board as a whole. It holds the file descriptor and ISR callback function, if applicable.

Definition at line [50](#) of file [dm35424_os.h](#).

5.1.2 Field Documentation

5.1.2.1 file_descriptor

```
int file_descriptor
```

File descriptor for device returned from open()

Definition at line [55](#) of file [dm35424_os.h](#).

5.1.2.2 isr

```
void(* isr) (struct dm35424_ioctl_interrupt_info_request info_req)
```

Function pointer to the user ISR callback function.

Definition at line 60 of file [dm35424_os.h](#).

5.1.2.3 pid

```
pthread_t pid
```

Process ID of the child process which will monitor DMA done interrupts.

Definition at line 65 of file [dm35424_os.h](#).

The documentation for this struct was generated from the following file:

- [include/dm35424_os.h](#)

5.2 dm35424_device_descriptor Struct Reference

DM35424 Device Descriptor. The identifying info for this particular board.

```
#include <dm35424_driver.h>
```

Data Fields

- char [name](#) [DM35424_NAME_LENGTH]
- int [device_index](#)
- struct [dm35424_pci_region](#) [pci](#) [PCI_ROM_RESOURCE]
- struct pci_dev * [pdev](#)
- spinlock_t [device_lock](#)
- struct cdev [cdev](#)
- uint8_t [reference_count](#)
- unsigned int [irq_number](#)
- uint8_t [remove_isr_flag](#)
- wait_queue_head_t [int_wait_queue](#)
- wait_queue_head_t [dma_wait_queue](#)
- int [interrupt_fb](#) [DM35424_INT_QUEUE_SIZE]
- unsigned int [int_queue_missed](#)
- unsigned int [int_queue_count](#)
- unsigned int [int_queue_in_marker](#)
- unsigned int [int_queue_out_marker](#)
- struct list_head [dma_descr_list](#)
- unsigned int [device_id](#)

5.2.1 Detailed Description

DM35424 Device Descriptor. The identifying info for this particular board.

Definition at line 222 of file [dm35424_driver.h](#).

5.2.2 Field Documentation

5.2.2.1 cdev

```
struct cdev cdev
```

Character device

Definition at line 256 of file [dm35424_driver.h](#).

5.2.2.2 device_id

```
unsigned int device_id
```

16-bit PCIe device ID

Definition at line 329 of file [dm35424_driver.h](#).

5.2.2.3 device_index

```
int device_index
```

Device index

Definition at line 234 of file [dm35424_driver.h](#).

5.2.2.4 device_lock

```
spinlock_t device_lock
```

Concurrency control

Definition at line 251 of file [dm35424_driver.h](#).

5.2.2.5 dma_descr_list

```
struct list_head dma_descr_list
```

A list of all allocated DMA buffers

Definition at line 324 of file [dm35424_driver.h](#).

5.2.2.6 dma_wait_queue

```
wait_queue_head_t dma_wait_queue
```

Queue of processes waiting to be woken up when an interrupt occurs

Definition at line 288 of file [dm35424_driver.h](#).

5.2.2.7 int_queue_count

```
unsigned int int_queue_count
```

Number of interrupts currently in the queue

Definition at line 307 of file [dm35424_driver.h](#).

5.2.2.8 int_queue_in_marker

```
unsigned int int_queue_in_marker
```

Where in the queue new entries are put

Definition at line 313 of file [dm35424_driver.h](#).

5.2.2.9 int_queue_missed

```
unsigned int int_queue_missed
```

Number of interrupts missed because of a full queue

Definition at line 301 of file [dm35424_driver.h](#).

5.2.2.10 int_queue_out_marker

```
unsigned int int_queue_out_marker
```

Where in the queue entries are pulled from

Definition at line 319 of file [dm35424_driver.h](#).

5.2.2.11 int_wait_queue

```
wait_queue_head_t int_wait_queue
```

Queue of processes waiting to be woken up when an interrupt occurs

Definition at line 282 of file [dm35424_driver.h](#).

5.2.2.12 interrupt_fb

```
int interrupt_fb[DM35424_INT_QUEUE_SIZE]
```

Interrupt queue containing which functional blocks caused interrupts

Definition at line 294 of file [dm35424_driver.h](#).

5.2.2.13 irq_number

```
unsigned int irq_number
```

IRQ line number

Definition at line 269 of file [dm35424_driver.h](#).

5.2.2.14 name

```
char name[DM35424_NAME_LENGTH]
```

Device name used when requesting resources; a NUL terminated string of the form rtd-dm35424-x where x is the device minor number.

Definition at line 229 of file [dm35424_driver.h](#).

5.2.2.15 pci

```
struct dm35424_pci_region pci[PCI_ROM_RESOURCE]
```

Information about each of the standard PCI regions

Definition at line 240 of file [dm35424_driver.h](#).

5.2.2.16 pdev

```
struct pci_dev* pdev
```

PCI device pointer

Definition at line 245 of file [dm35424_driver.h](#).

5.2.2.17 reference_count

```
uint8_t reference_count
```

Number of entities which have the device file open. Used to enforce single open semantics.

Definition at line 263 of file [dm35424_driver.h](#).

5.2.2.18 remove_isr_flag

```
uint8_t remove_isr_flag
```

Used to assist poll in shutting down the thread waiting for interrupts

Definition at line 276 of file [dm35424_driver.h](#).

The documentation for this struct was generated from the following file:

- [include/dm35424_driver.h](#)

5.3 DM35424_DMA_Descriptor Struct Reference

Descriptor for the DMA on this board.

```
#include <dm35424_board_access.h>
```

Data Fields

- `uint32_t` [control_offset](#)
- `uint8_t` [num_buffers](#)
- `uint32_t` [buffer_start_offset](#) [[MAX_DMA_BUFFERS](#)]

5.3.1 Detailed Description

Descriptor for the DMA on this board.

Definition at line 65 of file [dm35424_board_access.h](#).

5.3.2 Field Documentation

5.3.2.1 buffer_start_offset

```
uint32_t buffer_start_offset [MAX\_DMA\_BUFFERS]
```

Offset to the beginning of the buffer control section.

Definition at line 80 of file [dm35424_board_access.h](#).

5.3.2.2 control_offset

```
uint32_t control_offset
```

Offset to the DMA control register section

Definition at line 70 of file [dm35424_board_access.h](#).

5.3.2.3 num_buffers

```
uint8_t num_buffers
```

Number of buffers for this DMA channel.

Definition at line 75 of file [dm35424_board_access.h](#).

The documentation for this struct was generated from the following file:

- [include/dm35424_board_access.h](#)

5.4 dm35424_dma_descriptor Struct Reference

DM35424 DMA descriptor. This structure holds information about a single DMA buffer.

```
#include <dm35424_driver.h>
```

Data Fields

- uint32_t [fb_num](#)
- int [channel](#)
- int [buffer](#)
- void * [virt_addr](#)
- dma_addr_t [bus_addr](#)
- unsigned int [buffer_size](#)
- struct list_head [list](#)

5.4.1 Detailed Description

DM35424 DMA descriptor. This structure holds information about a single DMA buffer.

Definition at line 172 of file [dm35424_driver.h](#).

5.4.2 Field Documentation

5.4.2.1 buffer

```
int buffer
```

DMA buffer number this descriptor represents.

Definition at line 187 of file [dm35424_driver.h](#).

5.4.2.2 buffer_size

```
unsigned int buffer_size
```

Size of this allocated buffer

Definition at line 204 of file [dm35424_driver.h](#).

5.4.2.3 bus_addr

```
dma_addr_t bus_addr
```

Bus memory address for buffer.

Definition at line 198 of file [dm35424_driver.h](#).

5.4.2.4 channel

```
int channel
```

DMA channel this buffer is in.

Definition at line 182 of file [dm35424_driver.h](#).

5.4.2.5 fb_num

```
uint32_t fb_num
```

Function block number this DMA is associated with.

Definition at line 177 of file [dm35424_driver.h](#).

5.4.2.6 list

```
struct list_head list
```

List head so that descriptors can be kept in a linked list.

Definition at line 210 of file [dm35424_driver.h](#).

5.4.2.7 virt_addr

```
void* virt_addr
```

System memory address for buffer

Definition at line 192 of file [dm35424_driver.h](#).

The documentation for this struct was generated from the following file:

- [include/dm35424_driver.h](#)

5.5 DM35424_Function_Block Struct Reference

DM35424 function block descriptor. This structure holds information about a function block, including type, number of DMA channels and buffers, descriptors for each DMA channel, and memory offsets to various control locations.

```
#include <dm35424_board_access.h>
```

Data Fields

- [uint16_t type](#)
- [uint16_t sub_type](#)
- [uint16_t type_revision](#)
- [uint32_t fb_offset](#)
- [uint32_t dma_offset](#)
- [int fb_num](#)
- [int ordinal_fb_type_num](#)
- [uint8_t num_dma_buffers](#)
- [uint8_t num_dma_channels](#)
- [uint32_t control_offset](#)
- [struct DM35424_DMA_Descriptor dma_channel](#) [[MAX_DMA_CHANNELS](#)]

5.5.1 Detailed Description

DM35424 function block descriptor. This structure holds information about a function block, including type, number of DMA channels and buffers, descriptors for each DMA channel, and memory offsets to various control locations.

Definition at line 93 of file [dm35424_board_access.h](#).

5.5.2 Field Documentation

5.5.2.1 control_offset

```
uint32_t control_offset
```

Offset to the beginning of the control registers for this function block

Definition at line 145 of file [dm35424_board_access.h](#).

5.5.2.2 dma_channel

```
struct DM35424_DMA_Descriptor dma_channel[MAX_DMA_CHANNELS]
```

Array of descriptors for each DMA channel

Definition at line 154 of file [dm35424_board_access.h](#).

5.5.2.3 dma_offset

```
uint32_t dma_offset
```

Offset to the beginning of the DMA registers for this function block

Definition at line 119 of file [dm35424_board_access.h](#).

5.5.2.4 fb_num

```
int fb_num
```

Function block num (as identified in GBC)

Definition at line 124 of file [dm35424_board_access.h](#).

Referenced by [ISR\(\)](#), [main\(\)](#), and [output_channel_status\(\)](#).

5.5.2.5 fb_offset

```
uint32_t fb_offset
```

Offset to the beginning of the function block registers

Definition at line 114 of file [dm35424_board_access.h](#).

5.5.2.6 num_dma_buffers

```
uint8_t num_dma_buffers
```

Number of DMA buffers in this function block

Definition at line 135 of file [dm35424_board_access.h](#).

Referenced by [main\(\)](#), and [setup_dacs_and_start\(\)](#).

5.5.2.7 num_dma_channels

```
uint8_t num_dma_channels
```

Number of DMA channels in this function block

Definition at line 140 of file [dm35424_board_access.h](#).

Referenced by [main\(\)](#), and [setup_dacs_and_start\(\)](#).

5.5.2.8 ordinal_fb_type_num

```
int ordinal_fb_type_num
```

The ordinal number of this particular function block type (0th, 1st, etc)

Definition at line 130 of file [dm35424_board_access.h](#).

5.5.2.9 sub_type

```
uint16_t sub_type
```

Type of specific function block (ADC1, ADC2, ADC3, etc)

Definition at line 103 of file [dm35424_board_access.h](#).

Referenced by [main\(\)](#).

5.5.2.10 type

```
uint16_t type
```

Type of function block (ADC, DAC, DIO, etc)

Definition at line 98 of file [dm35424_board_access.h](#).

Referenced by [main\(\)](#).

5.5.2.11 type_revision

```
uint16_t type_revision
```

Revision of subtype (internal use only)

Definition at line 109 of file [dm35424_board_access.h](#).

The documentation for this struct was generated from the following file:

- [include/dm35424_board_access.h](#)

5.6 dm35424_iocli_argument Union Reference

`iocli()` request structure encapsulating all possible requests. This is what gets passed into the kernel from user space on the `iocli()` call.

```
#include <dm35424_board_access_structs.h>
```

Data Fields

- struct [dm35424_ioctl_region_readwrite](#) readwrite
- struct [dm35424_ioctl_region_modify](#) modify
- struct [dm35424_ioctl_interrupt_info_request](#) interrupt
- struct [dm35424_ioctl_dma](#) dma

5.6.1 Detailed Description

ioctl() request structure encapsulating all possible requests. This is what gets passed into the kernel from user space on the ioctl() call.

Definition at line 320 of file [dm35424_board_access_structs.h](#).

5.6.2 Field Documentation

5.6.2.1 dma

```
struct dm35424_ioctl_dma dma
```

DMA Configuration and Control

Definition at line 343 of file [dm35424_board_access_structs.h](#).

5.6.2.2 interrupt

```
struct dm35424_ioctl_interrupt_info_request interrupt
```

Interrupt request structure

Definition at line 338 of file [dm35424_board_access_structs.h](#).

5.6.2.3 modify

```
struct dm35424_ioctl_region_modify modify
```

PCI region read/modify/write

Definition at line 332 of file [dm35424_board_access_structs.h](#).

5.6.2.4 readwrite

```
struct dm35424_ioctl_region_readwrite readwrite
```

PCI region read and write

Definition at line 326 of file [dm35424_board_access_structs.h](#).

The documentation for this union was generated from the following file:

- include/[dm35424_board_access_structs.h](#)

5.7 dm35424_ioctl_dma Struct Reference

ioctl() request structure for DMA

```
#include <dm35424_board_access_structs.h>
```

Data Fields

- enum [DM35424_DMA_FUNCTIONS](#) function
- int [num_buffers](#)
- uint32_t [buffer_size](#)
- uint32_t [fb_num](#)
- int [channel](#)
- int [buffer](#)
- struct [dm35424_pci_access_request](#) pci
- void * [buffer_ptr](#)

5.7.1 Detailed Description

ioctl() request structure for DMA

Definition at line [269](#) of file [dm35424_board_access_structs.h](#).

5.7.2 Field Documentation

5.7.2.1 buffer

```
int buffer
```

Buffer in DMA channel that DMA is meant for.

Definition at line [299](#) of file [dm35424_board_access_structs.h](#).

5.7.2.2 buffer_ptr

```
void* buffer_ptr
```

Pointer to user-space buffer for read or write.

Definition at line [309](#) of file [dm35424_board_access_structs.h](#).

5.7.2.3 buffer_size

```
uint32_t buffer_size
```

Size (in bytes) to allocate for buffers

Definition at line [284](#) of file [dm35424_board_access_structs.h](#).

5.7.2.4 channel

```
int channel
```

Channel in function with DMA operation is for.

Definition at line 294 of file [dm35424_board_access_structs.h](#).

5.7.2.5 fb_num

```
uint32_t fb_num
```

Function Block DMA is for.

Definition at line 289 of file [dm35424_board_access_structs.h](#).

5.7.2.6 function

```
enum DM35424_DMA_FUNCTIONS function
```

Requested DMA function to perform.

Definition at line 274 of file [dm35424_board_access_structs.h](#).

5.7.2.7 num_buffers

```
int num_buffers
```

Number of buffers to initialize for DMA

Definition at line 279 of file [dm35424_board_access_structs.h](#).

5.7.2.8 pci

```
struct dm35424_pci_access_request pci
```

PCI Address of DMA registers for this operation

Definition at line 304 of file [dm35424_board_access_structs.h](#).

The documentation for this struct was generated from the following file:

- [include/dm35424_board_access_structs.h](#)

5.8 dm35424_ioctl_interrupt_info_request Struct Reference

ioctl() request structure for interrupt

```
#include <dm35424_board_access_structs.h>
```

Data Fields

- int [interrupts_remaining](#)
- int [valid_interrupt](#)
- int [error_occurred](#)
- int [interrupt_fb](#)

5.8.1 Detailed Description

ioctl() request structure for interrupt

Definition at line 238 of file [dm35424_board_access_structs.h](#).

5.8.2 Field Documentation

5.8.2.1 error_occurred

```
int error_occurred
```

Boolean if error occurred during interrupt

Definition at line 254 of file [dm35424_board_access_structs.h](#).

Referenced by [ISR\(\)](#).

5.8.2.2 interrupt_fb

```
int interrupt_fb
```

Function block that had interrupt. The MSB indicates if this was a DMA interrupt or not. (0 = Not DMA, 1 = DMA)

Definition at line 260 of file [dm35424_board_access_structs.h](#).

Referenced by [ISR\(\)](#).

5.8.2.3 interrupts_remaining

```
int interrupts_remaining
```

Count of interrupts remaining in the driver queue.

Definition at line 244 of file [dm35424_board_access_structs.h](#).

5.8.2.4 valid_interrupt

```
int valid_interrupt
```

Boolean of if interrupt is valid or not.

Definition at line 249 of file [dm35424_board_access_structs.h](#).

Referenced by [ISR\(\)](#).

The documentation for this struct was generated from the following file:

- [include/dm35424_board_access_structs.h](#)

5.9 dm35424_ioctl_region_modify Struct Reference

ioctl() request structure for PCI region read/modify/write

```
#include <dm35424_board_access_structs.h>
```

Data Fields

- struct [dm35424_pci_access_request](#) access
- union {
 - uint8_t [mask8](#)
 - uint16_t [mask16](#)
 - uint32_t [mask32](#)
- } [mask](#)

5.9.1 Detailed Description

ioctl() request structure for PCI region read/modify/write

Definition at line 189 of file [dm35424_board_access_structs.h](#).

5.9.2 Field Documentation

5.9.2.1 access

```
struct dm35424_pci_access_request access
```

PCI region access request

Definition at line 194 of file [dm35424_board_access_structs.h](#).

5.9.2.2 [union]

```
union { ... } mask
```

Bit mask that controls which bits can be modified. A zero in a bit position means that the corresponding register bit should not be modified. A one in a bit position means that the corresponding register bit should be modified.

Note that it's possible to set bits outside of the mask depending upon the register value before modification. When processing the associated request code, the driver will silently prevent this from happening but will not return an indication that the mask or new value was incorrect.

5.9.2.3 mask16

```
uint16_t mask16
```

Mask for 16-bit operations

Definition at line 220 of file [dm35424_board_access_structs.h](#).

5.9.2.4 mask32

```
uint32_t mask32
```

Mask for 32-bit operations

Definition at line 226 of file [dm35424_board_access_structs.h](#).

5.9.2.5 mask8

```
uint8_t mask8
```

Mask for 8-bit operations

Definition at line 214 of file [dm35424_board_access_structs.h](#).

The documentation for this struct was generated from the following file:

- [include/dm35424_board_access_structs.h](#)

5.10 dm35424_ioctl_region_readwrite Struct Reference

ioctl() request structure for read from or write to PCI region

```
#include <dm35424_board_access_structs.h>
```

Data Fields

- struct [dm35424_pci_access_request](#) access

5.10.1 Detailed Description

ioctl() request structure for read from or write to PCI region

Definition at line 174 of file [dm35424_board_access_structs.h](#).

5.10.2 Field Documentation

5.10.2.1 access

```
struct dm35424_pci_access_request access
```

PCI region access request

Definition at line 180 of file [dm35424_board_access_structs.h](#).

The documentation for this struct was generated from the following file:

- [include/dm35424_board_access_structs.h](#)

5.11 dm35424_pci_access_request Struct Reference

PCI region access request descriptor. This structure holds information about a request to read data from or write data to one of a device's PCI regions.

```
#include <dm35424_board_access_structs.h>
```

Data Fields

- enum [dm35424_pci_region_access_size](#) size
- enum [dm35424_pci_region_num](#) region
- uint16_t offset
- union {
 - uint8_t [data8](#)
 - uint16_t [data16](#)
 - uint32_t [data32](#)

5.11.1 Detailed Description

PCI region access request descriptor. This structure holds information about a request to read data from or write data to one of a device's PCI regions.

Definition at line 122 of file [dm35424_board_access_structs.h](#).

5.11.2 Field Documentation

5.11.2.1 [union]

```
union { ... } data
```

Data to write or the data read

5.11.2.2 data16

```
uint16_t data16
```

16-bit value

Definition at line 158 of file [dm35424_board_access_structs.h](#).

5.11.2.3 data32

```
uint32_t data32
```

32-bit value

Definition at line 164 of file [dm35424_board_access_structs.h](#).

5.11.2.4 data8

```
uint8_t data8
```

8-bit value

Definition at line 152 of file [dm35424_board_access_structs.h](#).

5.11.2.5 offset

```
uint16_t offset
```

Offset within region to access

Definition at line 140 of file [dm35424_board_access_structs.h](#).

5.11.2.6 region

```
enum dm35424_pci_region_num region
```

The PCI region to access

Definition at line 134 of file [dm35424_board_access_structs.h](#).

5.11.2.7 size

```
enum dm35424_pci_region_access_size size
```

Size of access in bits

Definition at line 128 of file [dm35424_board_access_structs.h](#).

The documentation for this struct was generated from the following file:

- [include/dm35424_board_access_structs.h](#)

5.12 dm35424_pci_region Struct Reference

DM35424 PCI region descriptor. This structure holds information about one of a device's PCI memory regions.

```
#include <dm35424_driver.h>
```

Data Fields

- unsigned long [io_addr](#)
- unsigned long [length](#)
- unsigned long [phys_addr](#)
- void __iomem * [virt_addr](#)
- uint8_t [allocated](#)

5.12.1 Detailed Description

DM35424 PCI region descriptor. This structure holds information about one of a device's PCI memory regions.

Definition at line 129 of file [dm35424_driver.h](#).

5.12.2 Field Documentation

5.12.2.1 allocated

```
uint8_t allocated
```

Flag indicating whether or not the I/O-mapped memory ranged was allocated. A value of zero means the memory range was not allocated. Any other value means the memory range was allocated.

Definition at line 163 of file [dm35424_driver.h](#).

5.12.2.2 io_addr

```
unsigned long io_addr
```

I/O port number if I/O mapped

Definition at line 135 of file [dm35424_driver.h](#).

5.12.2.3 length

```
unsigned long length
```

Length of region in bytes

Definition at line 141 of file [dm35424_driver.h](#).

5.12.2.4 phys_addr

```
unsigned long phys_addr
```

Region's physical address if memory mapped or I/O port number if I/O mapped

Definition at line 148 of file [dm35424_driver.h](#).

5.12.2.5 virt_addr

```
void __iomem* virt_addr
```

Address at which region is mapped in kernel virtual address space if memory mapped

Definition at line 155 of file [dm35424_driver.h](#).

The documentation for this struct was generated from the following file:

- [include/dm35424_driver.h](#)

Chapter 6

File Documentation

6.1 examples/_non_public/dm35424_adc_test.c File Reference

Example program which demonstrates the use of the ADC and DMA.

```
#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <fcntl.h>
#include <unistd.h>
#include <signal.h>
#include <limits.h>
#include <getopt.h>
#include <string.h>
#include "dm35424_gbc_library.h"
#include "dm35424_dac_library.h"
#include "dm35424_adc_library.h"
#include "dm35424_ioctl.h"
#include "dm35424_examples.h"
#include "dm35424_dma_library.h"
#include "dm35424.h"
#include "dm35424_util_library.h"
```

Macros

- `#define DEFAULT_RATE 10000`
- `#define BUFFER_SIZE_SAMPLES 100`
- `#define BUFFER_SIZE_BYTES (BUFFER_SIZE_SAMPLES * sizeof(int))`
- `#define DAT_FILE_NAME_PREFIX "./adc"`
- `#define DAT_FILE_NAME_SUFFIX "_dma_data.dat"`

Functions

- static void **usage** (void)
Print information on stderr about how the program is to be used. After doing so, the program is exited.
- static void **sigint_handler** (int signal_number)
Signal handler for SIGINT Control-C keyboard interrupt.
- void **output_channel_status** (struct **DM35424_Board_Descriptor** *handle, const struct **DM35424_Function_Block** *func_block, unsigned int channel)
Output the status of a DMA channel. This is a helper function to determine the cause of an error when it occurs.
- void **ISR** (struct **dm35424_ioctl_interrupt_info_request** int_info)
The interrupt subroutine that will execute when a DMA interrupt occurs. This function will read from the DMA, copying data from the kernel buffers to the user buffers so that we can access the data.
- int **main** (int argument_count, char **arguments)
The main program.

Variables

- static char * **program_name**
- static int **dma_has_error** = 0
- static struct **DM35424_Board_Descriptor** * **board**
- static struct **DM35424_Function_Block** my_adc [**DM35424_NUM_ADC_ON_BOARD**]
- static unsigned long **buffer_count** [**DM35424_NUM_ADC_ON_BOARD**][**DM35424_NUM_ADC_DMA_CHANNELS**]
- static int ** **local_buffer** [**DM35424_NUM_ADC_ON_BOARD**][**DM35424_NUM_ADC_DMA_CHANNELS**]
- static volatile int **exit_program** = 0
- static unsigned long **buffer_size_bytes** = 0

6.1.1 Detailed Description

Example program which demonstrates the use of the ADC and DMA.

This example program will collect data from the ADC(s) specified by the user, at the rate specified by the user, and will write the data to a file. It will do this continuously until the user hits CTRL-C (or the filesystem becomes full).

This is a very intensive operation for the PC, working CPU, memory, and file I/O fairly hard. Thus, there is no way to pre-determine for sure what the highest sustainable rate of collecting data is.

Maximum sustainable throughput is HIGHLY system dependent. Higher sample rates might be achievable through better buffer size selection or use of an operating system with realtime features.

This file and its contents are copyright (C) RTD Embedded Technologies, Inc. All Rights Reserved.

This software is licensed as described in the RTD End-User Software License Agreement. For a copy of this agreement, refer to the file LICENSE.TXT (which should be included with this software) or contact RTD Embedded Technologies, Inc.

Id

[dm35424_adc_test.c](#) 142659 2024-04-26 19:32:32Z lfrankenfield

Definition in file [dm35424_adc_test.c](#).

6.1.2 Macro Definition Documentation

6.1.2.1 BUFFER_SIZE_BYTES

```
#define BUFFER_SIZE_BYTES (BUFFER_SIZE_SAMPLES * sizeof(int))
```

Size of DAC DMA buffer, in bytes

Definition at line 75 of file [dm35424_adc_test.c](#).

Referenced by [main\(\)](#), and [setup_dacs_and_start\(\)](#).

6.1.2.2 BUFFER_SIZE_SAMPLES

```
#define BUFFER_SIZE_SAMPLES 100
```

Number of samples in the DAC buffer (to form the wave pattern)

Definition at line 70 of file [dm35424_adc_test.c](#).

Referenced by [main\(\)](#), and [setup_dacs_and_start\(\)](#).

6.1.2.3 DAT_FILE_NAME_PREFIX

```
#define DAT_FILE_NAME_PREFIX "./adc"
```

Prefix for files that will be output during example

Definition at line 80 of file [dm35424_adc_test.c](#).

Referenced by [main\(\)](#).

6.1.2.4 DAT_FILE_NAME_SUFFIX

```
#define DAT_FILE_NAME_SUFFIX "_dma_data.dat"
```

Suffix for files that will be output during example

Definition at line 85 of file [dm35424_adc_test.c](#).

Referenced by [main\(\)](#).

6.1.2.5 DEFAULT_RATE

```
#define DEFAULT_RATE 10000
```

Default rate to use, if user does not enter one.

Definition at line 65 of file [dm35424_adc_test.c](#).

Referenced by [main\(\)](#), [usage\(\)](#), [usage\(\)](#), and [usage\(\)](#).

6.1.3 Function Documentation

6.1.3.1 ISR()

```
void ISR (
    struct dm35424_ioctl_interrupt_info_request int_info)
```

The interrupt subroutine that will execute when a DMA interrupt occurs. This function will read from the DMA, copying data from the kernel buffers to the user buffers so that we can access the data.

Parameters

<i>int_info</i>	
-----------------	--

A structure containing information about the interrupt.

Return values

<i>None.</i>	
--------------	--

Definition at line 266 of file [dm35424_adc_test.c](#).

References [board](#), [buffer_count](#), [buffer_size_bytes](#), [CLEAR_INTERRUPT](#), [DM35424_Check_Result\(\)](#), [DM35424_Dma_Clear_Interru](#), [DM35424_Dma_Find_Interrupt\(\)](#), [DM35424_Dma_Read\(\)](#), [DM35424_Dma_Reset_Buffer\(\)](#), [DM35424_Gbc_Ack_Interrupt\(\)](#), [DM35424_NUM_ADC_ON_BOARD](#), [dma_has_error](#), [dm35424_ioctl_interrupt_info_request::error_occurred](#), [exit_program](#), [DM35424_Function_Block::fb_num](#), [dm35424_ioctl_interrupt_info_request::interrupt_fb](#), [local_buffer](#), [my_adc](#), [NO_CLEAR_INTERRUPT](#), and [dm35424_ioctl_interrupt_info_request::valid_interrupt](#).

Referenced by [main\(\)](#).

6.1.3.2 main()

```
int main (
    int argument_count,
    char ** arguments)
```

The main program.

Parameters

<i>argument_count</i>	
-----------------------	--

Number of args passed on the command line, including the executable name

Parameters

<i>arguments</i>	
------------------	--

Pointer to array of character strings, which are the args themselves.

Return values

0	
---	--

Success

Return values

Non-Zero	
----------	--

Failure. First, setup the DACS. They will produce a sine wave that needs to be looped back to the ADC inputs. This will cause the ADC to see what looks like a max-value sine wave.

We load the even channels with the wave pattern, and the odd channels with the same pattern, but offset by half its length. Doing this gives us an opposing pattern between the even and odd channels, which helps when using DAC for ADC input.

Check to see if any channel has not yet been copied from DMA.

Definition at line 427 of file [dm35424_adc_test.c](#).

References [AD_MODE_OPTION](#), [ADC_OPTION](#), [board](#), [BUFFER_0](#), [buffer_count](#), [BUFFER_INTERRUPT](#), [BUFFER_LOOP](#), [BUFFER_NO_HALT](#), [BUFFER_NO_INTERRUPT](#), [BUFFER_NO_LOOP](#), [BUFFER_SIZE_BYTES](#), [buffer_size_bytes](#), [BUFFER_SIZE_SAMPLES](#), [BUFFER_VALID](#), [DAT_FILE_NAME_PREFIX](#), [DAT_FILE_NAME_SUFFIX](#), [DEFAULT_RATE](#), [DM35424_Adc_Ad_Config_Set_Mode\(\)](#), [DM35424_Adc_Channel_Setup\(\)](#), [DM35424_Adc_Initialize\(\)](#), [DM35424_ADC_INPUT_DAC_LOOPBACK](#), [DM35424_ADC_INPUT_DIFFERENTIAL](#), [DM35424_ADC_INPUT_SINGLE_ENDED_NEG](#), [DM35424_ADC_INPUT_SINGLE_ENDED_POS](#), [DM35424_ADC_MODE_CONFIG_HIGH_RES](#), [DM35424_ADC_MODE_CONFIG_LOW_POWER](#), [DM35424_ADC_MODE_CONFIG_LOW_SPEED](#), [DM35424_Adc_Open\(\)](#), [DM35424_ADC_RNG_BIPOLAR_19mV](#), [DM35424_Adc_Set_Clock_Src\(\)](#), [DM35424_Adc_Set_Sample_Rate\(\)](#), [DM35424_Adc_Set_Start_Trigger\(\)](#), [DM35424_Adc_Set_Stop_Trigger\(\)](#), [DM35424_Adc_Start\(\)](#), [DM35424_Board_Close\(\)](#), [DM35424_Board_Open\(\)](#), [DM35424_Check_Result\(\)](#), [DM35424_CLK_SRC_IMMEDIATE](#), [DM35424_CLK_SRC_NEVER](#), [DM35424_Dac_Open\(\)](#), [DM35424_Dac_Set_Clock_Src\(\)](#), [DM35424_Dac_Set_Conversion_Rate\(\)](#), [DM35424_Dac_Set_Start_Trigger\(\)](#), [DM35424_Dac_Set_Stop_Trigger\(\)](#), [DM35424_Dac_Start\(\)](#), [DM35424_Dac_Volts_To_Conv\(\)](#), [DM35424_Dma_Buffer_Setup\(\)](#), [DM35424_Dma_Buffer_Status\(\)](#), [DM35424_Dma_Configure_Interrupts\(\)](#), [DM35424_Dma_Initialize\(\)](#), [DM35424_Dma_Setup\(\)](#), [DM35424_DMA_SETUP_DIRECTION_READ](#), [DM35424_DMA_SETUP_DIRECTION_WRITE](#), [DM35424_Dma_Start\(\)](#), [DM35424_Dma_Write\(\)](#), [DM35424_Gbc_Board_Reset\(\)](#), [DM35424_General_InstallISR\(\)](#), [DM35424_General_RemoveISR\(\)](#), [DM35424_Generate_Signal_Data\(\)](#), [DM35424_Micro_Sleep\(\)](#), [DM35424_NUM_ADC_DMA_BUFFERS](#), [DM35424_NUM_ADC_DMA_CHANNELS](#), [DM35424_NUM_ADC_ON_BOARD](#), [DM35424_NUM_DAC_ON_BOARD](#), [DM35424_SINE_WAVE](#), [dma_has_error](#), [ERROR_INTR_DISABLE](#), [ERROR_INTR_ENABLE](#), [exit_program](#), [HELP_OPTION](#), [IGNORE_USED](#), [INTERRUPT_DISABLE](#), [INTERRUPT_ENABLE](#), [ISR\(\)](#), [local_buffer](#), [MINOR_OPTION](#), [MODE_OPTION](#), [my_adc](#), [NOT_IGNORE_USED](#), [output_channel_status\(\)](#), [program_name](#), [RATE_OPTION](#), [SAMPLES_OPTION](#), [sigint_handler\(\)](#), and [usage\(\)](#).

6.1.3.3 output_channel_status()

```
void output_channel_status (
    struct DM35424_Board_Descriptor * handle,
    const struct DM35424_Function_Block * func_block,
    unsigned int channel)
```

Output the status of a DMA channel. This is a helper function to determine the cause of an error when it occurs.

Parameters

<i>handle</i>	
---------------	--

Pointer to the board handle.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block containing the DMA channel

Parameters

<i>channel</i>	
----------------	--

The DMA channel we want the status of.

Return values

<i>None</i>	
-------------	--

Definition at line 215 of file [dm35424_adc_test.c](#).

References [DM35424_Check_Result\(\)](#), [DM35424_Dma_Status\(\)](#), and [DM35424_Function_Block::fb_num](#).

Referenced by [main\(\)](#).

6.1.3.4 sigint_handler()

```
void sigint_handler (
    int signal_number) [static]
```

Signal handler for SIGINT Control-C keyboard interrupt.

Parameters

<i>signal_number</i>	
----------------------	--

Signal number passed in from the kernel.

Warning

One must be extremely careful about what functions are called from a signal handler.

Definition at line 184 of file [dm35424_adc_test.c](#).

References [exit_program](#).

Referenced by [main\(\)](#), and [setup_ctrlc_handler\(\)](#).

6.1.4 Variable Documentation

6.1.4.1 board

```
struct DM35424_Board_Descriptor* board [static]
```

Pointer to board descriptor

Definition at line 101 of file [dm35424_adc_test.c](#).

Referenced by [ISR\(\)](#), [main\(\)](#), [run_test_9\(\)](#), [setup_adc\(\)](#), and [setup_dacs_and_start\(\)](#).

6.1.4.2 buffer_count

```
unsigned long buffer_count[DM35424_NUM_ADC_ON_BOARD][DM35424_NUM_ADC_DMA_CHANNELS] [static]
```

Array of buffer counts, used to track progress of each ADC as data is copied.

Definition at line 112 of file [dm35424_adc_test.c](#).

Referenced by [ISR\(\)](#), and [main\(\)](#).

6.1.4.3 buffer_size_bytes

```
unsigned long buffer_size_bytes = 0 [static]
```

Size of the buffer allocated, in bytes.

Definition at line 128 of file [dm35424_adc_test.c](#).

Referenced by [ISR\(\)](#), [main\(\)](#), and [setup_adc\(\)](#).

6.1.4.4 dma_has_error

```
int dma_has_error = 0 [static]
```

Boolean flag indicating if there was a DMA error.

Definition at line 96 of file [dm35424_adc_test.c](#).

Referenced by [ISR\(\)](#), and [main\(\)](#).

6.1.4.5 exit_program

```
volatile int exit_program = 0 [static]
```

Boolean indicating the program should exit.

Definition at line 123 of file [dm35424_adc_test.c](#).

Referenced by [ISR\(\)](#), [main\(\)](#), [run_test_9\(\)](#), [sigint_handler\(\)](#), [sigint_handler\(\)](#), [sigint_handler\(\)](#), [sigint_handler\(\)](#), [sigint_handler\(\)](#), [sigint_handler\(\)](#), [sigint_handler\(\)](#), [sigint_handler\(\)](#), and [sigint_handler\(\)](#).

6.1.4.6 local_buffer

```
int** local_buffer[DM35424_NUM_ADC_ON_BOARD][DM35424_NUM_ADC_DMA_CHANNELS] [static]
```

Pointer to local memory buffer where data is copied from the kernel buffers when a DMA buffer becomes full.

Definition at line 118 of file [dm35424_adc_test.c](#).

Referenced by [ISR\(\)](#), and [main\(\)](#).

6.1.4.7 my_adc

```
struct DM35424_Function_Block my_adc[DM35424_NUM_ADC_ON_BOARD] [static]
```

Pointer to array of function blocks that will hold the ADC descriptors

Definition at line 106 of file [dm35424_adc_test.c](#).

Referenced by [ISR\(\)](#), [main\(\)](#), [run_test_9\(\)](#), and [setup_adc\(\)](#).

6.1.4.8 program_name

```
char* program_name [static]
```

Name of the program as invoked on the command line

Definition at line 91 of file [dm35424_adc_test.c](#).

Referenced by [main\(\)](#), [usage\(\)](#), [usage\(\)](#), [usage\(\)](#), [usage\(\)](#), [usage\(\)](#), [usage\(\)](#), [usage\(\)](#), [usage\(\)](#), [usage\(\)](#), [usage\(\)](#), [usage\(\)](#), [usage\(\)](#), and [usage\(\)](#).

6.2 dm35424_adc_test.c

[Go to the documentation of this file.](#)

```
00001
00039
00040 #include <stdio.h>
00041 #include <stddef.h>
00042 #include <stdlib.h>
00043 #include <errno.h>
00044 #include <error.h>
00045 #include <fcntl.h>
00046 #include <unistd.h>
00047 #include <signal.h>
00048 #include <limits.h>
00049 #include <getopt.h>
00050 #include <string.h>
00051
00052 #include "dm35424_gbc_library.h"
00053 #include "dm35424_dac_library.h"
00054 #include "dm35424_adc_library.h"
00055 #include "dm35424_ioctl.h"
00056 #include "dm35424_examples.h"
00057 #include "dm35424_dma_library.h"
00058 #include "dm35424.h"
00059 #include "dm35424_util_library.h"
00060
00061
00065 #define DEFAULT_RATE          10000
00066
00070 #define BUFFER_SIZE_SAMPLES   100
```

```

00071
00075 #define BUFFER_SIZE_BYTES      (BUFFER_SIZE_SAMPLES * sizeof(int))
00076
00080 #define DAT_FILE_NAME_PREFIX    "./adc"
00081
00085 #define DAT_FILE_NAME_SUFFIX    "_dma_data.dat"
00086
00090
00091 static char *program_name;
00092
00096 static int dma_has_error = 0;
00097
00101 static struct DM35424_Board_Descriptor *board;
00102
00106 static struct DM35424_Function_Block my_adc[DM35424_NUM_ADC_ON_BOARD];
00107
00112 static unsigned long buffer_count[DM35424_NUM_ADC_ON_BOARD][DM35424_NUM_ADC_DMA_CHANNELS];
00113
00118 static int **local_buffer[DM35424_NUM_ADC_ON_BOARD][DM35424_NUM_ADC_DMA_CHANNELS];
00119
00123 static volatile int exit_program = 0;
00124
00128 static unsigned long buffer_size_bytes = 0;
00129
00137
00138 static void usage(void)
00139 {
00140     fprintf(stderr, "\n");
00141     fprintf(stderr, "%s\n", program_name);
00142     fprintf(stderr, "\n");
00143     fprintf(stderr,
00144         "Usage: %s [--help] --minor MINOR [--adc ADC_NUM] [--rate RATE] [--samples SAMPLES]
--mode [ADC_MODE]\n",
00145         program_name);
00146     fprintf(stderr, "\n");
00147     fprintf(stderr,
00148         "    --help:      Display usage information and exit.\n");
00149     fprintf(stderr, "\n");
00150     fprintf(stderr, "    --minor:    Use specified minor device.\n");
00151     fprintf(stderr, "        MINOR:      Device minor number (>= 0).\n");
00152     fprintf(stderr,
00153         "    --adc:      ADC to collect from (default = 0).\n");
00154     fprintf(stderr,
00155         "        ADC_NUM:   Which ADC to use (0, 1, or 2 for both).\n");
00156     fprintf(stderr, "    --rate:    Use specified rate for all.\n");
00157     fprintf(stderr, "        RATE:      Sample rate (in Hz) (> 0).\n");
00158     fprintf(stderr, "    --samples: Specify the numbers of samples to collect.\n");
00159     fprintf(stderr, "        SAMPLES:   Number of samples to collect before stopping.\n");
00160     fprintf(stderr, "    --mode:    Specify the mode to use with all ADCs.\n");
00161     fprintf(stderr, "        ADC_MODE:   diff, se_pos, se_neg, or se_dac\n");
00162     fprintf(stderr, "    --ad_mode: Specify the AD Config mode to use with all ADCs.\n");
00163     fprintf(stderr, "        AD_MODE:    high_spd, high_res, low_power, low_spd\n");
00164     fprintf(stderr, "\n");
00165     exit(EXIT_FAILURE);
00166 }
00167
00183
00184 static void sigint_handler(int signal_number)
00185 {
00186     exit_program = 0xff;
00187 }
00188
00214
00215 void output_channel_status(struct DM35424_Board_Descriptor *handle,
00216     const struct DM35424_Function_Block *func_block,
00217     unsigned int channel)
00218 {
00219     int result;
00220     unsigned int current_buffer;
00221     uint32_t current_count;
00222     int current_action;
00223     int status_overflow;
00224     int status_underflow;
00225     int status_used;
00226     int status_invalid;
00227     int status_complete;
00228
00229     result = DM35424_Dma_Status(handle,
00230         func_block,
00231         channel,
00232         &current_buffer,
00233         &current_count,
00234         &current_action,
00235         &status_overflow,
00236         &status_underflow,
00237         &status_used,
00238         &status_invalid, &status_complete);

```

```

00239
00240     DM35424_Check_Result(result, "Error getting DMA status");
00241
00242     printf
00243     ("FB%d Ch%d DMA Status: Current Buffer: %u Count: %ul Action: 0x%x Status: "
00244      "Ov: %d Un: %d Used: %d Inv: %d Comp: %d\n",
00245      func_block->fb_num, channel, current_buffer, current_count,
00246      current_action, status_overflow, status_underflow, status_used,
00247      status_invalid, status_complete);
00248 }
00249
00266 void ISR(struct dm35424_ioctl_interrupt_info_request int_info)
00267 {
00268     char message[200];
00269
00270     int result = 0;
00271     unsigned int channel = 0;
00272     unsigned int buffer_to_get = 0;
00273     int channel_complete = 0, channel_error = 0;
00274     uint32_t function_block;
00275     int buffer_full = 0;
00276     int adc_num = -1;
00277     int adc_index = 0;
00278
00279     function_block = int_info.interrupt_fb & 0x7FFFFFFF;
00280
00281     DM35424_Check_Result(int_info.error_occurred, "Error occurred during interrupt.");
00282
00283     if (int_info.valid_interrupt) {
00284
00285         while (adc_num < 0 && adc_index < DM35424_NUM_ADC_ON_BOARD) {
00286             if (my_adc[adc_index].fb_num == function_block) {
00287                 adc_num = adc_index;
00288             }
00289
00290             adc_index ++;
00291         }
00292
00293         sprintf(message, "ISR: Could not match function block number (%d) to an ADC.",
00294                 function_block);
00295
00296         DM35424_Check_Result(adc_num < 0, message);
00297
00298         // It's a DMA interrupt
00299         if (int_info.interrupt_fb < 0) {
00300
00301             result = DM35424_Dma_Find_Interrupt(board,
00302                                                  &my_adc[adc_num],
00303                                                  &channel,
00304                                                  &channel_complete,
00305                                                  &channel_error);
00306
00307             DM35424_Check_Result(result, "Error checking for DMA error.");
00308
00309             if (channel_error) {
00310                 dma_has_error = 1;
00311                 exit_program = 1;
00312                 return;
00313             }
00314
00315             while (channel_complete) {
00316
00317                 result = DM35424_Dma_Find_Used_Buffer(board,
00318                                                         &my_adc[adc_num],
00319                                                         channel,
00320                                                         &buffer_to_get,
00321                                                         &buffer_full);
00322
00323                 DM35424_Check_Result(result, "Error finding used buffer.");
00324
00325                 while (buffer_full) {
00326
00327                     result = DM35424_Dma_Read(board,
00328                                                  &my_adc[adc_num],
00329                                                  channel,
00330                                                  buffer_to_get,
00331                                                  buffer_size_bytes,
00332                                                  local_buffer[adc_num][channel]
00333                                                  [buffer_to_get]);
00334
00335                     buffer_count[adc_num][channel]++;
00336                     DM35424_Check_Result(result,
00337                                           "Error getting DMA buffer");
00338
00339                     result = DM35424_Dma_Reset_Buffer(board,
00340                                                         &my_adc[adc_num],

```



```

00342                                     channel,
00343                                     buffer_to_get);
00344
00345         DM35424_Check_Result(result, "Error resetting buffer");
00346         //printf("» Copied chan %d buffer %d.\n", channel,
        buffer_to_get);
00347
00348         result = DM35424_Dma_Find_Used_Buffer(board,
00349                                             &my_adc[adc_num],
00350                                             channel,
00351                                             &buffer_to_get,
00352                                             &buffer_full);
00353
00354         DM35424_Check_Result(result,
00355                             "Error finding used buffer.");
00356
00357     }
00358
00359     result = DM35424_Dma_Clear_Interrupt(board,
00360                                         &my_adc[adc_num],
00361                                         channel,
00362                                         NO_CLEAR_INTERRUPT,
00363                                         NO_CLEAR_INTERRUPT,
00364                                         NO_CLEAR_INTERRUPT,
00365                                         NO_CLEAR_INTERRUPT,
00366                                         CLEAR_INTERRUPT);
00367
00368     result = DM35424_Dma_Find_Interrupt(board,
00369                                         &my_adc[adc_num],
00370                                         &channel,
00371                                         &channel_complete,
00372                                         &channel_error);
00373
00374     DM35424_Check_Result(result, "Error checking for DMA error.");
00375
00376     if (channel_error) {
00377         dma_has_error = 1;
00378         exit_program = 1;
00379         return;
00380     }
00381
00382     }
00383
00384     } else {
00385         printf("*** Process non-DMA interrupt for FB 0x%x.\n",
00386               int_info.interrupt_fb);
00387     }
00388
00389     result = DM35424_Gbc_Ack_Interrupt(board);
00390
00391     DM35424_Check_Result(result, "Error calling ACK interrupt.");
00392
00393 }
00394
00395
00396 }
00397
00398
00399
00400
00401
00402
00403
00404
00405
00406
00407
00408
00409
00410
00411
00412
00413
00414
00415
00416
00417
00418
00419
00420
00421
00422
00423
00424
00425
00426
00427 int main(int argument_count, char **arguments)
00428 {
00429     unsigned long int minor = 0;
00430     int result;
00431     int index;
00432     char filename[100];
00433     FILE *fp[DM35424_NUM_ADC_ON_BOARD];
00434
00435     struct DM35424_Function_Block my_dac[DM35424_NUM_DAC_ON_BOARD];
00436
00437     int buff;
00438     uint8_t buff_status;
00439     uint8_t buff_control;
00440     uint32_t buff_size;
00441     unsigned int buffer_to_get;
00442     unsigned int buffers_copied;
00443     unsigned long local_buffer_count[DM35424_NUM_ADC_ON_BOARD][DM35424_NUM_ADC_DMA_CHANNELS];
00444     unsigned long output_index[DM35424_NUM_ADC_ON_BOARD];
00445     unsigned long samples_to_collect = -1;
00446     int loop_val;
00447     struct sigaction signal_action;
00448
00449     unsigned int dac_num = 0;
00450     unsigned int adc_num = 0;
00451     unsigned int channel = 0;
00452     unsigned int num_adc_to_use = 1;
00453     unsigned int adc_index = 0;
00454

```

```

00455     uint32_t rate = DEFAULT_RATE, actual_rate = 0;
00456     uint32_t dac_rate = DEFAULT_RATE;
00457
00458     int16_t max_value, min_value, offset;
00459     int32_t *buffer, *offset_buffer;
00460
00461     int help_option_given = 0;
00462     int minor_option_given = 0;
00463     int rate_option_given = 0;
00464     int mode_option_given = 0;
00465     int ad_mode_option_given = 0;
00466
00467     int adc_mode_to_use = DM35424_ADC_INPUT_DIFFERENTIAL;
00468     int ad_mode_to_use = DM35424_ADC_MODE_CONFIG_HIGH_SPEED;
00469     int status;
00470     char *invalid_char_p;
00471     struct option options[] = {
00472         {"help", 0, 0, HELP_OPTION},
00473         {"minor", 1, 0, MINOR_OPTION},
00474         {"adc", 1, 0, ADC_OPTION},
00475         {"rate", 1, 0, RATE_OPTION},
00476         {"samples", 1, 0, SAMPLES_OPTION},
00477         {"mode", 1, 0, MODE_OPTION},
00478         {"ad_mode", 1, 0, AD_MODE_OPTION},
00479         {0, 0, 0, 0}
00480     };
00481
00482     char adc_mode_option[30];
00483     char ad_mode_option[30];
00484
00485     int all_channels_copied = 0;
00486
00487     program_name = arguments[0];
00488
00489     // Show usage, parse arguments
00490     while (1) {
00491
00492         /*
00493          * Parse the next command line option and any arguments it may require
00494          */
00495
00496         status = getopt_long(argument_count,
00497                             arguments, "", options, NULL);
00498
00499         /*
00500          * If getopt_long() returned -1, then all options have been processed
00501          */
00502
00503         if (status == -1) {
00504             break;
00505         }
00506
00507         /*
00508          * Figure out what getopt_long() found
00509          */
00510
00511         switch (status) {
00512
00513             /*#####
00514              User entered '--help'
00515              ##### */
00516             case HELP_OPTION:
00517
00518                 /*
00519                  * Refuse to accept duplicate '--help' options
00520                  */
00521
00522                 if (help_option_given) {
00523                     error(0, 0, "ERROR: Duplicate option '--help'");
00524                     usage();
00525                 }
00526
00527                 /*
00528                  * '--help' option has been seen
00529                  */
00530
00531                 help_option_given = 0xFF;
00532                 break;
00533
00534             /*#####
00535              User entered '--minor'
00536              ##### */
00537             case MINOR_OPTION:
00538
00539                 /*
00540                  * Refuse to accept duplicate '--minor' options
00541                  */

```

```

00542
00543         if (minor_option_given) {
00544             error(0, 0,
00545                 "ERROR: Duplicate option '--minor'");
00546             usage();
00547         }
00548
00549         /*
00550          * Convert option argument string to unsigned long integer
00551          */
00552
00553         errno = 0;
00554
00555         minor = strtoul(optarg, &invalid_char_p, 10);
00556
00557         /*
00558          * Catch unsigned long int overflow
00559          */
00560
00561         if ((minor == ULONG_MAX)
00562             && (errno == ERANGE)) {
00563             error(0, 0,
00564                 "ERROR: Device minor number caused numeric overflow");
00565             usage();
00566         }
00567
00568         /*
00569          * Catch argument strings with valid decimal prefixes, for
00570          * example "1q", and argument strings which cannot be converted,
00571          * for example "abc1"
00572          */
00573
00574         if ((*invalid_char_p != '\0')
00575             || (invalid_char_p == optarg)) {
00576             error(0, 0,
00577                 "ERROR: Non-decimal device minor number");
00578             usage();
00579         }
00580
00581         /*
00582          * '--minor' option has been seen
00583          */
00584
00585         minor_option_given = 0xFF;
00586         break;
00587
00588         /*#####
00589          User entered '--adc'
00590          ##### */
00591     case ADC_OPTION:
00592
00593         /*
00594          * Convert option argument string to unsigned long integer
00595          */
00596
00597         errno = 0;
00598
00599         adc_num = strtoul(optarg, &invalid_char_p, 10);
00600
00601         /*
00602          * Catch unsigned long int overflow
00603          */
00604
00605         if ((adc_num == ULONG_MAX)
00606             && (errno == ERANGE)) {
00607             error(0, 0,
00608                 "ERROR: ADC caused numeric overflow");
00609             usage();
00610         }
00611
00612         /*
00613          * Catch argument strings with valid decimal prefixes, for
00614          * example "1q", and argument strings which cannot be converted,
00615          * for example "abc1"
00616          */
00617
00618         if ((*invalid_char_p != '\0')
00619             || (invalid_char_p == optarg)) {
00620             error(0, 0, "ERROR: Non-decimal ADC");
00621             usage();
00622         }
00623
00624         if (adc_num < 0 || adc_num > 2) {
00625             error(0, 0, "ERROR: ADC must be 0, 1, or 2.");
00626             usage();
00627         }
00628

```

```

00629         break;
00630
00631         /*#####
00632         User entered rate
00633         ##### */
00634     case RATE_OPTION:
00635
00636         /*
00637          * Refuse to accept duplicate '--rate' options
00638          */
00639
00640         if (rate_option_given) {
00641             error(0, 0, "ERROR: Duplicate option '--rate'");
00642             usage();
00643         }
00644
00645         /*
00646          * Convert option argument string to unsigned long integer
00647          */
00648
00649         errno = 0;
00650
00651         rate = strtoul(optarg, &invalid_char_p, 10);
00652
00653         /*
00654          * Catch unsigned long int overflow
00655          */
00656
00657         if ((rate == ULONG_MAX)
00658             && (errno == ERANGE)) {
00659             error(0, 0,
00660                 "ERROR: Rate number caused numeric overflow");
00661             usage();
00662         }
00663
00664         /*
00665          * Catch argument strings with valid decimal prefixes, for
00666          * example "1q", and argument strings which cannot be converted,
00667          * for example "abc1"
00668          */
00669
00670         if ((*invalid_char_p != '\0')
00671             || (invalid_char_p == optarg)) {
00672             error(0, 0,
00673                 "ERROR: Non-decimal rate value entered");
00674             usage();
00675         }
00676
00677         /*
00678          * '--rate' option has been seen
00679          */
00680
00681         rate_option_given = 0xFF;
00682         break;
00683
00684         /*#####
00685         User entered number of samples
00686         ##### */
00687     case SAMPLES_OPTION:
00688
00689         /*
00690          * Convert option argument string to unsigned long integer
00691          */
00692
00693         errno = 0;
00694
00695         samples_to_collect = strtoul(optarg, &invalid_char_p, 10);
00696
00697         /*
00698          * Catch unsigned long int overflow
00699          */
00700
00701         if ((samples_to_collect == ULONG_MAX)
00702             && (errno == ERANGE)) {
00703             error(0, 0,
00704                 "ERROR: Samples number caused numeric overflow");
00705             usage();
00706         }
00707
00708         /*
00709          * Catch argument strings with valid decimal prefixes, for
00710          * example "1q", and argument strings which cannot be converted,
00711          * for example "abc1"
00712          */
00713
00714         if ((*invalid_char_p != '\0')
00715

```

```

00716             || (invalid_char_p == optarg)) {
00717                 error(0, 0,
00718                     "ERROR: Non-decimal samples value entered");
00719                 usage();
00720             }
00721
00722             break;
00723
00724
00725         case MODE_OPTION:
00726             strcpy(adc_mode_option, optarg);
00727             mode_option_given = 1;
00728             break;
00729
00730         case AD_MODE_OPTION:
00731             strcpy(ad_mode_option, optarg);
00732             ad_mode_option_given = 1;
00733             break;
00734
00735             /*#####
00736             User entered unsupported option
00737             ##### */
00738         case '?':
00739             usage();
00740             break;
00741
00742             /*#####
00743             getopt_long() returned unexpected value
00744             ##### */
00745         default:
00746             error(EXIT_FAILURE,
00747                 0,
00748                 "ERROR: getopt_long() returned unexpected value %x",
00749                 status);
00750             break;
00751     }
00752 }
00753
00754 /*
00755  * Recognize '--help' option before any others
00756  */
00757
00758 if (help_option_given) {
00759     usage();
00760 }
00761
00762 /*
00763  * '--minor' option must be given
00764  */
00765
00766 if (minor_option_given == 0x00) {
00767     error(0, 0, "ERROR: Option '--minor' is required");
00768     usage();
00769 }
00770
00771 if (rate < 1 || rate > 105468) {
00772     error(0, 0, "Error: Rate given not within range of board.");
00773     usage();
00774 }
00775
00776 if (mode_option_given) {
00777     if (strcmp(adc_mode_option, "diff") == 0) {
00778         adc_mode_to_use = DM35424_ADC_INPUT_DIFFERENTIAL;
00779     }
00780     else if (strcmp(adc_mode_option, "se_pos") == 0) {
00781         adc_mode_to_use = DM35424_ADC_INPUT_SINGLE_ENDED_POS;
00782     }
00783     else if (strcmp(adc_mode_option, "se_neg") == 0) {
00784         adc_mode_to_use = DM35424_ADC_INPUT_SINGLE_ENDED_NEG;
00785     }
00786     else if (strcmp(adc_mode_option, "se_dac") == 0) {
00787         adc_mode_to_use = DM35424_ADC_INPUT_DAC_LOOPBACK;
00788     }
00789     else {
00790         error(0, 0, "Error: Unrecognizable value for ADC mode.");
00791         usage();
00792     }
00793 }
00794
00795
00796 if (ad_mode_option_given) {
00797     if (strcmp(ad_mode_option, "high_spd") == 0) {
00798         ad_mode_to_use = DM35424_ADC_MODE_CONFIG_HIGH_SPEED;
00799     }
00800     else if (strcmp(ad_mode_option, "high_res") == 0) {
00801         ad_mode_to_use = DM35424_ADC_MODE_CONFIG_HIGH_RES;
00802     }

```

```

00803         }
00804         else if (strcmp(ad_mode_option, "low_power") == 0) {
00805             ad_mode_to_use = DM35424_ADC_MODE_CONFIG_LOW_POWER;
00806         }
00807         else if (strcmp(ad_mode_option, "low_spd") == 0) {
00808             ad_mode_to_use = DM35424_ADC_MODE_CONFIG_LOW_SPEED;
00809         }
00810         else {
00811             error(0, 0, "Error: Unrecognizable value for AD mode.");
00812             usage();
00813         }
00814     }
00815
00816     if (adc_num == 2) {
00817         adc_num = 0;
00818         num_adc_to_use = 2;
00819     }
00820
00821     dac_rate = rate;
00822
00823     /*
00824     * Trying to come up with a reasonable size for buffers that doesn't
00825     * take forever to fill up with slower rates, but also makes it possible
00826     * to run at higher rates.
00827     */
00828     buffer_size_bytes = ((rate * 8) / 50) * num_adc_to_use;
00829     if (buffer_size_bytes < (20 * sizeof(int))) {
00830         buffer_size_bytes = 20 * sizeof(int);
00831     }
00832     buffer_size_bytes &= ~0x3;
00833
00834     signal_action.sa_handler = sigint_handler;
00835     sigfillset(&(signal_action.sa_mask));
00836     signal_action.sa_flags = 0;
00837
00838     if (sigaction(SIGINT, &signal_action, NULL) < 0) {
00839         error(EXIT_FAILURE, errno, "ERROR: sigaction() FAILED");
00840     }
00841
00842     printf("Opening board.....");
00843     result = DM35424_Board_Open(minor, &board);
00844
00845     DM35424_Check_Result(result, "Could not open board");
00846     printf("success.\nResetting board.....");
00847     result = DM35424_Gbc_Board_Reset(board);
00848
00849     DM35424_Check_Result(result, "Could not reset board");
00850     printf("success.\n");
00851
00852     printf("success.\nOpening DAC.....");
00853
00854     for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD; dac_num++) {
00855         result = DM35424_Dac_Open(board, dac_num, &my_dac[dac_num]);
00856
00857         DM35424_Check_Result(result, "Could not open DAC");
00858
00859         printf("Found DAC%u, with %d DMA channels (%d buffers each)\n",
00860             dac_num, my_dac[dac_num].num_dma_channels, my_dac[dac_num].num_dma_buffers);
00861
00862         result = DM35424_Dac_Set_Clock_Src(board,
00863             &my_dac[dac_num],
00864             DM35424_CLK_SRC_IMMEDIATE);
00865
00866         DM35424_Check_Result(result, "Error setting DAC clock");
00867
00868         result = DM35424_Dac_Set_Conversion_Rate(board,
00869             &my_dac[dac_num], dac_rate, &actual_rate);
00870
00871         fprintf(stdout, "Rate requested: %d Actual Rate Achieved: %d\n", dac_rate,
00872             actual_rate);
00873         DM35424_Check_Result(result, "Error setting sample rate");
00874
00875         buffer = (int *)malloc(BUFFER_SIZE_BYTES);
00876         offset_buffer = (int *)malloc(BUFFER_SIZE_BYTES);
00877
00878         DM35424_Check_Result(buffer == NULL || offset_buffer == NULL, "Error allocating space
00879         for buffer.");
00880
00881         result = DM35424_Dac_Volts_To_Conv(1.25f,
00882             &max_value);
00883
00884         DM35424_Check_Result(result, "Error converting value to conversion counts.");
00885
00886         result = DM35424_Dac_Volts_To_Conv(-1.25f,
00887             &min_value);
00888
00889         DM35424_Check_Result(result, "Error converting value to conversion counts.");
00890
00891     }
00892
00893

```

```

00894
00895     result = DM35424_Dac_Volts_To_Conv(2.5f,
00896                                         &offset);
00897
00898     DM35424_Check_Result(result, "Error converting value to conversion counts.");
00899
00900
00901     result = DM35424_Generate_Signal_Data(DM35424_SINE_WAVE,
00902                                           buffer,
00903                                           BUFFER_SIZE_SAMPLES,
00904                                           max_value,
00905                                           min_value,
00906                                           offset,
00907                                           0x0000FFFF);
00908
00909     DM35424_Check_Result(result, "Error trying to generate data for the DAC.");
00910
00911     for (index = 0; index < BUFFER_SIZE_SAMPLES; index++) {
00912         offset_buffer[index] = buffer[(index + (BUFFER_SIZE_SAMPLES / 2)) %
BUFFER_SIZE_SAMPLES];
00913     }
00914
00915     for (channel = 0; channel < my_dac[dac_num].num_dma_channels; channel++) {
00916         fprintf(stdout, "Initializing and configuring DMA Channel %d...",
00917                 channel);
00918         result =
00919             DM35424_Dma_Initialize(board, &my_dac[dac_num], channel,
00920                                     1, BUFFER_SIZE_BYTES);
00921
00922         DM35424_Check_Result(result, "Error initializing DMA");
00923
00924         result = DM35424_Dma_Setup(board,
00925                                     &my_dac[dac_num],
00926                                     channel,
00927                                     DM35424_DMA_SETUP_DIRECTION_WRITE,
00928                                     IGNORE_USED);
00929
00930         DM35424_Check_Result(result, "Error configuring DMA");
00931
00932         fprintf(stdout, "success!\n");
00933
00934         result = DM35424_Dma_Buffer_Setup(board,
00935                                             &my_dac[dac_num],
00936                                             channel,
00937                                             BUFFER_0,
00938                                             BUFFER_VALID,
00939                                             BUFFER_NO_HALT,
00940                                             BUFFER_LOOP, BUFFER_NO_INTERRUPT);
00941
00942         DM35424_Check_Result(result, "Error setting up buffer control.");
00943
00944         if (channel % 2 == 0) {
00945             result = DM35424_Dma_Write(board,
00946                                         &my_dac[dac_num],
00947                                         channel,
00948                                         BUFFER_0, BUFFER_SIZE_BYTES, buffer);
00949
00950             DM35424_Check_Result(result, "Writing to DMA buffer failed");
00951         }
00952         else {
00953             result = DM35424_Dma_Write(board,
00954                                         &my_dac[dac_num],
00955                                         channel,
00956                                         BUFFER_0, BUFFER_SIZE_BYTES,
00957                                         offset_buffer);
00958
00959             DM35424_Check_Result(result, "Writing to DMA buffer failed");
00960         }
00961
00962         fprintf(stdout, "Starting DMA Channel %d.....", channel);
00963         result = DM35424_Dma_Start(board, &my_dac[dac_num], channel);
00964
00965         DM35424_Check_Result(result, "Error starting DMA");
00966
00967         printf("success.\n");
00968     }
00969
00970     free(buffer);
00971     free(offset_buffer);
00972
00973     fprintf(stdout, "Starting DAC.\n");
00974
00975     result = DM35424_Dac_Set_Start_Trigger(board,

```

```

00985                                     &my_dac[dac_num],
00986                                     DM35424_CLK_SRC_IMMEDIATE);
00987
00988     DM35424_Check_Result(result, "Error setting start trigger for DAC.");
00989
00990     result = DM35424_Dac_Set_Stop_Trigger(board,
00991                                     &my_dac[dac_num],
00992                                     DM35424_CLK_SRC_NEVER);
00993
00994     DM35424_Check_Result(result, "Error setting stop trigger for DAC.");
00995
00996
00997     result = DM35424_Dac_Start(board, &my_dac[dac_num]);
00998
00999     DM35424_Check_Result(result, "Error starting DAC");
01000
01001
01002 }
01003
01004
01005 for (adc_index = 0; adc_index < num_adc_to_use; adc_index++) {
01006     output_index[adc_num + adc_index] = 0;
01007     result = DM35424_Adc_Open(board, adc_num + adc_index, &my_adc[adc_num + adc_index]);
01008
01009     DM35424_Check_Result(result, "Could not open ADC");
01010
01011     printf("Found ADC%d, with %d DMA channels (%d buffers each)\n",
01012           adc_num + adc_index, my_adc[adc_num + adc_index].num_dma_channels,
01013           my_adc[adc_num + adc_index].num_dma_buffers);
01014
01015     result = DM35424_Adc_Set_Clock_Src(board,
01016                                     &my_adc[adc_num + adc_index],
01017                                     DM35424_CLK_SRC_IMMEDIATE);
01018
01019     DM35424_Check_Result(result, "Error setting ADC clock");
01020
01021     sprintf(filename, "%s%s", DAT_FILE_NAME_PREFIX, adc_num + adc_index,
01022            DAT_FILE_NAME_SUFFIX);
01023
01024     remove(filename);
01025     fp[adc_num + adc_index] = fopen(filename, "w");
01026
01027     if (fp[adc_num + adc_index] == NULL) {
01028         error(EXIT_FAILURE, errno,
01029            "open() FAILED on output data file.\n");
01030     }
01031
01032     for (channel = 0; channel < my_adc[adc_num + adc_index].num_dma_channels; channel++)
01033     {
01034         buffer_count[adc_num + adc_index][channel] = 0;
01035         local_buffer_count[adc_num + adc_index][channel] = 0;
01036
01037         fprintf(stdout, "Initializing DMA Channel %u...", channel);
01038         result = DM35424_Dma_Initialize(board,
01039                                     &my_adc[adc_num + adc_index],
01040                                     channel,
01041                                     my_adc[adc_num + adc_index].num_dma_buffers,
01042                                     buffer_size_bytes);
01043
01044         DM35424_Check_Result(result, "Error initializing DMA");
01045
01046         result = DM35424_Dma_Setup(board,
01047                                     &my_adc[adc_num + adc_index],
01048                                     channel,
01049                                     DM35424_DMA_SETUP_DIRECTION_READ,
01050                                     NOT_IGNORE_USED);
01051
01052         DM35424_Check_Result(result, "Error configuring DMA");
01053
01054         fprintf(stdout, "Setting DMA Interrupts.....");
01055         result = DM35424_Dma_Configure_Interrupts(board,
01056                                     &my_adc[adc_num + adc_index],
01057                                     channel,
01058                                     INTERRUPT_ENABLE,
01059                                     ERROR_INTR_ENABLE);
01060
01061         DM35424_Check_Result(result, "Error setting DMA Interrupts");
01062         fprintf(stdout, "success!\n");
01063
01064         for (buff = 0; buff < my_adc[adc_num + adc_index].num_dma_buffers; buff++) {
01065             loop_val = BUFFER_NO_LOOP;
01066             if (buff == (DM35424_NUM_ADC_DMA_BUFFERS - 1)) {
01067                 loop_val = BUFFER_LOOP;
01068             }
01069         }
01070     }

```



```

01071
01072         result = DM35424_Dma_Buffer_Setup(board,
01073                                           &my_adc[adc_num + adc_index],
01074                                           channel,
01075                                           buff,
01076                                           BUFFER_VALID,
01077                                           BUFFER_NO_HALT,
01078                                           loop_val,
01079                                           BUFFER_INTERRUPT);
01080
01081         DM35424_Check_Result(result, "Error setting buffer control.");
01082
01083         result = DM35424_Dma_Buffer_Status(board,
01084                                           &my_adc[adc_num + adc_index],
01085                                           channel,
01086                                           buff,
01087                                           &buff_status,
01088                                           &buff_control,
01089                                           &buff_size);
01090
01091         DM35424_Check_Result(result, "Error getting buffer status.");
01092
01093         fprintf(stdout,
01094                "    Buffer %d: Stat: 0x%x Ctrl: 0x%x Size: %d\n",
01095                buff, buff_status, buff_control, buff_size);
01096     }
01097
01098     result = DM35424_Adc_Channel_Setup(board,
01099                                       &my_adc[adc_num + adc_index],
01100                                       channel,
01101                                       DM35424_ADC_RNG_BIPOLAR_19mV,
01102                                       adc_mode_to_use);
01103
01104     DM35424_Check_Result(result, "Error setting up channel.");
01105
01106 }
01107
01108
01109 result = DM35424_Adc_Ad_Config_Set_Mode(board,
01110                                         &my_adc[adc_num + adc_index],
01111                                         ad_mode_to_use);
01112
01113 DM35424_Check_Result(result, "Error setting AD config.");
01114
01115 // Allocate local memory for data.
01116 for (channel = 0; channel < my_adc[adc_num + adc_index].num_dma_channels; channel++) {
01117     local_buffer[adc_num + adc_index][channel] =
01118         malloc(sizeof(int *) * my_adc[adc_num + adc_index].num_dma_buffers);
01119     DM35424_Check_Result(local_buffer[adc_num + adc_index][channel] == NULL,
01120                         "Could not allocate for local buffer");
01121
01122     for (buff = 0; buff < my_adc[adc_num + adc_index].num_dma_buffers; buff++) {
01123         local_buffer[adc_num + adc_index][channel][buff] =
01124             malloc(buffer_size_bytes);
01125
01126         DM35424_Check_Result(local_buffer[adc_num + adc_index][channel][buff]
01127                             == NULL,
01128                             "Could not allocate for local buffer");
01129     }
01130 }
01131
01132 }
01133
01134 printf("success.\nInitializing ADC.....");
01135 result = DM35424_Adc_Set_Start_Trigger(board,
01136                                       &my_adc[adc_num + adc_index],
01137                                       DM35424_CLK_SRC_IMMEDIATE);
01138 DM35424_Check_Result(result, "Error setting start trigger.");
01139
01140 result = DM35424_Adc_Set_Stop_Trigger(board,
01141                                       &my_adc[adc_num + adc_index],
01142                                       DM35424_CLK_SRC_NEVER);
01143 DM35424_Check_Result(result, "Error setting stop trigger.");
01144
01145
01146 result = DM35424_Adc_Set_Sample_Rate(board,
01147                                       &my_adc[adc_num + adc_index],
01148                                       rate, &actual_rate);
01149
01150 DM35424_Check_Result(result, "Failed to set sample rate for ADC.");
01151 fprintf(stdout,
01152        "success.\nADC:%d Rate requested: %d Actual Rate Achieved: %d\n",
01153        adc_num + adc_index, rate, actual_rate);
01154
01155 result = DM35424_Adc_Initialize(board, &my_adc[adc_num + adc_index]);

```

```

01156
01157         DM35424_Check_Result(result, "Failed or timed out initializing ADC.");
01158
01159         for (channel = 0; channel < my_adc[adc_num + adc_index].num_dma_channels; channel++) {
01160
01161             fprintf(stdout, "Starting ADC %d DMA %d .....", adc_num, channel);
01162             result = DM35424_Dma_Start(board, &my_adc[adc_num + adc_index], channel);
01163
01164             DM35424_Check_Result(result, "Error starting DMA");
01165
01166             printf("success.\n");
01167
01168         }
01169     }
01170 }
01171
01172 fprintf(stdout, "Installing user ISR .....");
01173 result = DM35424_General_InstallISR(board, ISR);
01174 DM35424_Check_Result(result, "DM35424_General_InstallISR()");
01175
01176
01177 for (adc_index = 0; adc_index < num_adc_to_use; adc_index++) {
01178
01179     printf("Starting ADC %d\n", adc_num + adc_index);
01180
01181     result = DM35424_Adc_Start(board, &my_adc[adc_num + adc_index]);
01182
01183     DM35424_Check_Result(result, "Error starting ADC");
01184 }
01185
01186 buffers_copied = 0;
01187
01188 /*
01189  * Loop here until an error occurs, or the user hits CTRL-C. Loop through the ADCs
01190  * and see if a buffer has been copied from kernel space. If so, then write it out
01191  * to disk.
01192  */
01193 while (!exit_program && output_index[adc_num] < samples_to_collect) {
01194
01195     for (adc_index = 0; adc_index < num_adc_to_use; adc_index++) {
01196
01197         all_channels_copied = 1;
01198
01199         for (channel = 0; channel < my_adc[adc_num + adc_index].num_dma_channels;
01200             channel++) {
01201
01202             if ((buffer_count[adc_num + adc_index][channel] -
01203                 local_buffer_count[adc_num + adc_index][channel]) >
01204                 my_adc[adc_num + adc_index].num_dma_buffers) {
01205                 fprintf(stdout,
01206                     "Local buffer for ADC Chan %d was overrun.\n",
01207                     channel);
01208                 exit_program = 1;
01209             }
01210
01211             if (local_buffer_count[adc_num + adc_index][channel] ==
01212                 buffer_count[adc_num + adc_index][channel]) {
01213                 all_channels_copied = 0;
01214             }
01215         }
01216
01217         if (all_channels_copied) {
01218
01219             buffer_to_get = local_buffer_count[adc_num + adc_index][0] %
01220                 my_adc[adc_num + adc_index].num_dma_buffers;
01221
01222             for (index = 0;
01223                 index < (buffer_size_bytes / sizeof(int));
01224                 index++) {
01225                 fprintf(fp[adc_num + adc_index], "%lu", output_index[adc_num +
01226                     adc_index]);
01227                 for (channel = 0; channel < my_adc[adc_num +
01228                     adc_index].num_dma_channels; channel++) {
01229
01230                     fprintf(fp[adc_num + adc_index], "\t%d",
01231                         local_buffer[adc_num + adc_index][channel]
01232                         [buffer_to_get][index]);
01233
01234                     }
01235                     fprintf(fp[adc_num + adc_index], "\n");
01236                     output_index[adc_num + adc_index]++;
01237                 }
01238                 buffers_copied++;
01239

```

```

01240             if (buffers_copied % 10 == 0) {
01241                 fprintf(stdout, "Copied %d buffers.\n",
01242                     buffers_copied);
01243             }
01244
01245             for (channel = 0; channel < my_adc[adc_num +
adc_index].num_dma_channels; channel++) {
01246
01247                 local_buffer_count[adc_num + adc_index][channel]++;
01248             }
01249
01250         }
01251     }
01252 }
01253
01254 /*
01255  * Even though we've stopped, there might actually still be
01256  * interrupts in the driver queue. As a result, we need to
01257  * give them time to finish use of the ISR before removing it
01258  */
01259 DM35424_Micro_Sleep(1000000);
01260
01261 for (adc_index = 0; adc_index < num_adc_to_use; adc_index++) {
01262     for (channel = 0; channel < my_adc[adc_num + adc_index].num_dma_channels; channel++) {
01263         if (dma_has_error) {
01264             output_channel_status(board,
01265                 &my_adc[adc_num + adc_index], channel);
01266         }
01267
01268         result = DM35424_Dma_Configure_Interrupts(board,
01269             &my_adc[adc_num + adc_index],
01270             channel,
01271             INTERRUPT_DISABLE,
01272             ERROR_INTR_DISABLE);
01273
01274         DM35424_Check_Result(result, "Error setting DMA Interrupts");
01275
01276         for (buff = 0; buff < my_adc[adc_num + adc_index].num_dma_buffers; buff++) {
01277             free(local_buffer[adc_num + adc_index][channel][buff]);
01278         }
01279     }
01280
01281     if (output_index[adc_num] >= samples_to_collect) {
01282         fprintf(stdout, "Reached number of samples (%lu)\n", samples_to_collect);
01283     }
01284
01285     fclose(fp[adc_num + adc_index]);
01286 }
01287
01288 printf("Removing ISR\n");
01289 result = DM35424_General_RemoveISR(board);
01290
01291 DM35424_Check_Result(result, "Error removing ISR.");
01292
01293 printf("Closing Board\n");
01294 result = DM35424_Board_Close(board);
01295
01296 DM35424_Check_Result(result, "Error closing board.");
01297
01298 printf("Example program successfully completed.\n");
01299 return 0;
01300 }

```

6.3 examples/_non_public/dm35424_board_checkout.c File Reference

Example program which checks out the functionality of all of the basic parts of the board.

```

#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>

```

```
#include <error.h>
#include <unistd.h>
#include <limits.h>
#include <getopt.h>
#include <signal.h>
#include <string.h>
#include <time.h>
#include <sys/time.h>
#include "dm35424_gbc_library.h"
#include "dm35424_dac_library.h"
#include "dm35424_adc_library.h"
#include "dm35424_board_access_struct.h"
#include "dm35424_examples.h"
#include "dm35424.h"
#include "dm35424_dma_library.h"
#include "dm35424_util_library.h"
```

Functions

- static void **usage** (void)
Print information on stderr about how the program is to be used. After doing so, the program is exited.
- static void [sigint_handler](#) (int signal_number)
Signal handler for SIGINT Control-C keyboard interrupt.
- void [run_test_9](#) (unsigned int minor)

Variables

- static char * [program_name](#)

6.3.1 Detailed Description

Example program which checks out the functionality of all of the basic parts of the board.

This example program is meant to be a basic checkout of board functionality. It will make use of a loop-back connector so that we can use the DACs and ADCs together.

The program is meant to have limited user input during execution.

This file and its contents are copyright (C) RTD Embedded Technologies, Inc. All Rights Reserved.

This software is licensed as described in the RTD End-User Software License Agreement. For a copy of this agreement, refer to the file LICENSE.TXT (which should be included with this software) or contact RTD Embedded Technologies, Inc.

Id

[dm35424_board_checkout.c](#) 142659 2024-04-26 19:32:32Z lfrankenfield

Definition in file [dm35424_board_checkout.c](#).

6.3.2 Function Documentation

6.3.2.1 run_test_9()

```
void run_test_9 (
    unsigned int minor)
```

We let the thresh-inv clock run first, then the thresh clock. This is important, as it lets the inv and normal clocks be close to the same. It does NOT work if we put in the low threshold here, and then switch to the high threshold. It works this way for reasons that only Andy understands.

For better or worse, we're not going to investigate why the counts are sometimes off by 1 here, when in theory they should be identical. So, we'll allow a difference of 1 either way between the two threshold clocks and the system clock.

Setting threshold to a value it should be crossed at.

Check that the ADC is still running

Setting threshold to a value it should be crossed at.

Check that the ADC has stopped

Definition at line 5375 of file [dm35424_board_checkout.c](#).

References [board](#), [CHANNEL_0](#), [DM35424_ADC_CHAN_INTR_HIGH_THRESHOLD_MASK](#), [DM35424_ADC_CHAN_INTR_LOW_THRESHOLD_MASK](#), [DM35424_Adc_Channel_Get_Last_Sample\(\)](#), [DM35424_Adc_Channel_Interrupt_Clear_Status\(\)](#), [DM35424_Adc_Channel_Interrupt_Get_Status\(\)](#), [DM35424_Adc_Channel_Set_High_Threshold\(\)](#), [DM35424_Adc_Channel_Set_Low_Threshold\(\)](#), [DM35424_Adc_Get_Mode_Status\(\)](#), [DM35424_Adc_Get_Sample_Count\(\)](#), [DM35424_ADC_MODE_GO_SINGLE_SHOT](#), [DM35424_Adc_Reset\(\)](#), [DM35424_Adc_Set_Clock_Source_Global\(\)](#), [DM35424_Adc_Set_Post_Stop_Samples\(\)](#), [DM35424_Adc_Set_Pre_Trigger_Samples\(\)](#), [DM35424_Adc_Set_Start_Trigger\(\)](#), [DM35424_Adc_Set_Stop_Trigger\(\)](#), [DM35424_Adc_Start\(\)](#), [DM35424_ADC_STAT_DONE](#), [DM35424_ADC_STAT_SAMPLING](#), [DM35424_ADC_STAT_WAITING_START_TRIG](#), [DM35424_Board_Close\(\)](#), [DM35424_Board_Open\(\)](#), [DM35424_CLK_SRC_CHAN_THRESH](#), [DM35424_CLK_SRC_CHAN_THRESH_INV](#), [DM35424_CLK_SRC_IMMEDIATE](#), [DM35424_CLK_SRC_NEVER](#), [DM35424_Dac_Channel_Set_Marker_Config\(\)](#), [DM35424_Dac_Get_Conversion_Count\(\)](#), [DM35424_Dac_Get_Mode_Status\(\)](#), [DM35424_Dac_Reset\(\)](#), [DM35424_Dac_Set_Clock_Source_Global\(\)](#), [DM35424_Dac_Set_Clock_Src\(\)](#), [DM35424_Dac_Set_Last_Conversion\(\)](#), [DM35424_Dac_Set_Post_Stop_Conversion_Count\(\)](#), [DM35424_Dac_Set_Start_Trigger\(\)](#), [DM35424_Dac_Set_Stop_Trigger\(\)](#), [DM35424_Dac_Start\(\)](#), [DM35424_DMA_SETUP_DIRECTION_READ](#), [DM35424_Gbc_Board_Reset\(\)](#), [DM35424_Micro_Sleep\(\)](#), [DM35424_NUM_ADC_ON_BOARD](#), [DM35424_NUM_DAC_ON_BOARD](#), [exit_program](#), [INTERRUPT_DISABLE](#), [INTERRUPT_ENABLE](#), and [my_adc](#).

6.3.2.2 sigint_handler()

```
void sigint_handler (
    int signal_number) [static]
```

Signal handler for SIGINT Control-C keyboard interrupt.

Parameters

<i>signal_number</i>

Signal number passed in from the kernel.

Warning

One must be extremely careful about what functions are called from a signal handler.

Definition at line 226 of file [dm35424_board_checkout.c](#).

References [exit_program](#).

6.3.3 Variable Documentation

6.3.3.1 program_name

```
char* program_name [static]
```

Name of the program as invoked on the command line

Definition at line 166 of file [dm35424_board_checkout.c](#).

6.4 dm35424_board_checkout.c

[Go to the documentation of this file.](#)

```
00001
00032
00033 #include <stdio.h>
00034 #include <stddef.h>
00035 #include <stdlib.h>
00036 #include <errno.h>
00037 #include <error.h>
00038 #include <unistd.h>
00039 #include <limits.h>
00040 #include <getopt.h>
00041 #include <signal.h>
00042 #include <string.h>
00043 #include <time.h>
00044 #include <sys/time.h>
00045
00046
00047 #include "dm35424_gbc_library.h"
00048 #include "dm35424_dac_library.h"
00049 #include "dm35424_adc_library.h"
00050 #include "dm35424_board_access_struct.h"
00051 #include "dm35424_examples.h"
00052 #include "dm35424.h"
00053 #include "dm35424_dma_library.h"
00054 #include "dm35424_util_library.h"
00055
00056 #define TEST_0_DAC_CONVERSION_RATE 100
00057
00058 #define TEST_1_ADC_SAMPLE_RATE 10000
00059 #define TEST_1_ITERATIONS 1
00060
00061 #define TEST_2_ADC_SAMPLE_RATE 1000
00062 #define TEST_2_ITERATIONS 1000
00063
00064 #define TEST_3_ADC_SAMPLE_RATE 1000
00065 #define TEST_3_ITERATIONS 1000
00066
00067
00068 #define TEST_4_ADC_SAMPLE_RATE 1000
00069 #define TEST_4_DAC_CONVERSION_RATE 50
00070 #define TEST_4_ITERATIONS (TEST_4_ADC_SAMPLE_RATE * 10)
00071 #define TEST_4_ADC_LOW_VOLTAGE -2.4f
00072 #define TEST_4_ADC_HIGH_VOLTAGE 2.4f
00073 #define TEST_4_ADC_HIGH_THRESH 2.45f
00074 #define TEST_5_DAC_CONVERSION_RATE 100
00075
00076 #define TEST_6_ADC_SAMPLE_RATE 2000
00077 #define TEST_6_DAC_CONVERSION_RATE 200
00078 #define TEST_6_ADC_LOW_THRESH_VOLTS (-2.3f)
00079 #define TEST_6_ADC_HIGH_THRESH_VOLTS (2.3f)
00080
00081 #define TEST_7_ADC_SAMPLE_RATE 1000
00082 #define TEST_7_DAC_CONVERSION_RATE DM35424_FIFO_SAMPLE_SIZE
00083 #define TEST_8_ADC_SAMPLE_RATE DM35424_FIFO_SAMPLE_SIZE
00084
00085 #define TEST_9_ADC_SAMPLE_RATE 10000
00086 #define TEST_9_DAC_CONVERSION_RATE 100
00087 #define TEST_9_ADC_LOW_THRESH 4000000
00088 #define TEST_9_ADC_HIGH_THRESH -4000000
00089 #define TEST_9_DAC_POST_STOP_CONV 1000
00090
00091
00092 #define TEST_10_DAC_CONVERSION_RATE 100
```

```

00093 #define TEST_10_ADC_CONVERSION_RATE      5
00094
00095
00096 #define TEST6_ADC_PRE_TRIGGER_SAMP_SIZE 412
00097 #define TEST6_ADC_POST_STOP_SAMP_SIZE    212
00098
00099 #define BUFFER_SAMPLE_SIZE                0x100
00100 #define BUFFER_BYTES_SIZE                 (BUFFER_SAMPLE_SIZE * sizeof(int))
00101
00102 #define DAC_SAMPLE_SIZE                   100
00103 #define DAC_BUFFER_SIZE_BYTES             (DAC_SAMPLE_SIZE * sizeof(int))
00104
00105 #define TEST_7_DAC_BUFFER_BYTES           1000
00106
00107
00108 #define NUM_CHANNELS_TO_USE               4
00109 #define NUM_BUFFERS_TO_USE                1
00110
00111 #define NUM_BUFFERS_1                     1
00112 #define DMA_BUFFER0                       0
00113 #define DMA_BUFFER1                       1
00114 #define DMA_BUFFER2                       2
00115 #define DMA_BUFFER3                       3
00116 #define DMA_CHAN0                         0
00117
00118 #define DAC0                              0
00119 #define DAC1                              1
00120 #define DAC2                              2
00121 #define DAC3                              3
00122
00123 #define ADC0                              0
00124 #define ADC1                              1
00125
00126 #define INIT_BUFFER_SIZE                  100
00127 #define BUFFER_SIZE_INCREMENT              100
00128
00129 #define DAC_VALUE_FOR_ADC_MAX              23919
00130 #define DAC_VALUE_FOR_ADC_ZERO             16383
00131 #define DAC_VALUE_FOR_ADC_MIN              8847
00132
00133
00134 #define ADC_LOW_VALUE                      8000
00135 #define ADC_HIGH_VALUE                     8200000
00136 #define ADC_RANGE_COMPARISON               8000000
00137
00138 #define TEN_MICRO_SEC_IN_NANO              10000
00139 #define ONE_SEC_IN_MILLI                    1000
00140 #define FIVE_SEC_IN_MILLI                   5000
00141
00142 #define SAMPLES_TO_DISCARD                 1000
00143
00144 #define DAC0_SIN_FILE_FOR_ADC              "sin_dac0_1k_for_adc.txt"
00145 #define DAC1_SIN_FILE_FOR_ADC              "sin_dac1_1k_for_adc.txt"
00146 #define DAC0_SIN_FILE_1K_FOR_ADC           "sin_dac0_1k_for_adc.txt"
00147 #define DAC1_SIN_FILE_1K_FOR_ADC           "sin_dac1_1k_for_adc.txt"
00148 #define DAC_SQUARE_WAVE                    "square_wave.txt"
00149
00150 #define PROC_FILE                          "/proc/rtd-dm35424-log"
00151
00152 #define RUN_TEST(t)                        (((test_num == -1) || (test_num == (t))) && (!exit_program))
00153 #define MODE_STATUS(m,s)                   (((s) << 4) | (m))
00154 #define GET_MODE(ms)                       ((ms) & 0xF)
00155 #define GET_STATUS(ms)                     ((ms) >> 4)
00156
00157 struct timeval start, end;
00158
00159 int timeout_msec;
00160
00161
00162
00163 static char *program_name;
00164
00165 volatile int exit_program = 0;
00166
00167 volatile int interrupt_count = 0;
00168 volatile int interrupt_fb = -1;
00169 volatile int interrupt_dma_fb = -1;
00170
00171 static int interrupts_expected = 0;
00172
00173 int stop_on_error = 1;
00174
00175 struct DM35424_Board_Descriptor *board;
00176 struct DM35424_Function_Block my_dac[DM35424_NUM_DAC_ON_BOARD];
00177 struct DM35424_Function_Block my_adc[DM35424_NUM_ADC_ON_BOARD];
00178
00179
00180
00181
00182

```

```
00190
00191 static void usage(void)
00192 {
00193     fprintf(stderr, "\n");
00194     fprintf(stderr, "%s\n", program_name);
00195     fprintf(stderr, "\n");
00196     fprintf(stderr, "Usage: %s [--help] --minor MINOR --test TESTNUM --nostop\n", program_name);
00197     fprintf(stderr, "\n");
00198     fprintf(stderr,
00199         "    --help:      Display usage information and exit.\n");
00200     fprintf(stderr, "\n");
00201     fprintf(stderr, "    --minor:    Use specified minor device.\n");
00202     fprintf(stderr, "        MINOR:    Device minor number (>= 0).\n");
00203     fprintf(stderr, "    --test:     Execute a specific test \n");
00204     fprintf(stderr, "        TESTNUM:    Number of the test to run.\n");
00205     fprintf(stderr, "    --nostop:   Do not stop the test if an error occurs.\n");
00206     fprintf(stderr, "\n");
00207     exit(EXIT_FAILURE);
00208 }
00209
00225
00226 static void sigint_handler(int signal_number)
00227 {
00228     exit_program = 0xff;
00229 }
00230
00231
00232
00233 void DM35424_Check_Result_test(int return_val, char *message) {
00234
00235     if (return_val != 0) {
00236
00237         if (stop_on_error) {
00238             fprintf(stderr, "\n");
00239             error(EXIT_FAILURE, errno, "\nERROR: %s", message);
00240         }
00241         else {
00242             fprintf(stderr, "\nERROR: %s\n", message);
00243         }
00244     }
00245 }
00246
00247 }
00248
00249
00250 void load_dacs_with_wave(enum DM35424_Waveforms wave)
00251 {
00252
00253     unsigned int dac_num;
00254     unsigned int channel;
00255     int result;
00256     unsigned int index;
00257     int32_t *buffer, *offset_buffer;
00258     int16_t max_dac_value, min_dac_value, dac_offset;
00259
00260
00261     buffer = (int *)malloc(DAC_BUFFER_SIZE_BYTES);
00262     offset_buffer = (int *)malloc(DAC_BUFFER_SIZE_BYTES);
00263
00264     DM35424_Check_Result_test(buffer == NULL || offset_buffer == NULL, "Error allocating space for
buffer.");
00265
00266
00267     result = DM35424_Dac_Volts_To_Conv(1.25f,
00268                                         &max_dac_value);
00269
00270     DM35424_Check_Result_test(result, "Error converting value to conversion counts.");
00271
00272     result = DM35424_Dac_Volts_To_Conv(-1.25f,
00273                                         &min_dac_value);
00274
00275     DM35424_Check_Result_test(result, "Error converting value to conversion counts.");
00276
00277     result = DM35424_Dac_Volts_To_Conv(2.5f,
00278                                         &dac_offset);
00279
00280     DM35424_Check_Result_test(result, "Error converting value to conversion counts.");
00281
00282
00283     result = DM35424_Generate_Signal_Data(wave,
00284                                           buffer,
00285                                           DAC_SAMPLE_SIZE,
00286                                           max_dac_value,
00287                                           min_dac_value,
00288                                           dac_offset,
00289                                           0x0000FFFF);
00290
```



```

00291     DM35424_Check_Result_test(result, "Error trying to generate data for the DAC.");
00292
00293     for (index = 0; index < DAC_SAMPLE_SIZE; index++) {
00294         offset_buffer[index] =
00295             buffer[(index +
00296                 (DAC_SAMPLE_SIZE / 2)) % DAC_SAMPLE_SIZE];
00297     }
00298
00299     DM35424_Check_Result_test(result, "Error trying to generate data for the DAC.");
00300
00301     for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD; dac_num++) {
00302         for (channel = 0; channel < my_dac[dac_num].num_dma_channels; channel++) {
00303             result = DM35424_Dma_Stop(board,
00304                                     &my_dac[dac_num],
00305                                     channel);
00306
00307             DM35424_Check_Result_test(result, "Error stopping DMA.");
00308
00309             result = DM35424_Dma_Initialize(board,
00310                                             &my_dac[dac_num],
00311                                             channel,
00312                                             NUM_BUFFERS_1,
00313                                             DAC_BUFFER_SIZE_BYTES);
00314
00315             DM35424_Check_Result_test(result, "Error initializing DMA");
00316
00317             result = DM35424_Dma_Setup(board,
00318                                       &my_dac[dac_num],
00319                                       channel,
00320                                       DM35424_DMA_SETUP_DIRECTION_WRITE,
00321                                       IGNORE_USED);
00322
00323             DM35424_Check_Result_test(result, "Error configuring DMA");
00324
00325             result = DM35424_Dma_Buffer_Setup(board,
00326                                               &my_dac[dac_num],
00327                                               channel,
00328                                               DMA_BUFFER0,
00329                                               BUFFER_VALID,
00330                                               BUFFER_NO_HALT,
00331                                               BUFFER_LOOP,
00332                                               BUFFER_NO_INTERRUPT);
00333
00334             DM35424_Check_Result_test(result, "Error setting up buffer control.");
00335
00336             if (channel % 2 == 0) {
00337                 result = DM35424_Dma_Write(board,
00338                                             &(my_dac[dac_num]),
00339                                             channel,
00340                                             0, DAC_BUFFER_SIZE_BYTES, buffer);
00341
00342                 DM35424_Check_Result_test(result, "Writing to DMA buffer failed");
00343             }
00344             else {
00345                 result = DM35424_Dma_Write(board,
00346                                             &(my_dac[dac_num]),
00347                                             channel,
00348                                             0, DAC_BUFFER_SIZE_BYTES, offset_buffer);
00349
00350                 DM35424_Check_Result_test(result, "Writing to DMA buffer failed");
00351             }
00352         }
00353     }
00354
00355     free(buffer);
00356     free(offset_buffer);
00357 }
00358
00359 void start_dacs_with_wave(enum DM35424_Waveforms wave)
00360 {
00361     unsigned int dac_num;
00362     unsigned int channel;

```

```

00378     int result;
00379
00380     load_dacs_with_wave(wave);
00381
00382     for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD; dac_num++) {
00383
00384         result = DM35424_Dac_Set_Start_Trigger(board,
00385                                                 &(my_dac[dac_num]),
00386                                                 DM35424_CLK_SRC_GBL2);
00387
00388         DM35424_Check_Result_test(result, "Error setting start trigger for DAC.");
00389
00390         result = DM35424_Dac_Set_Stop_Trigger(board,
00391                                                 &(my_dac[dac_num]),
00392                                                 DM35424_CLK_SRC_NEVER);
00393
00394         DM35424_Check_Result_test(result, "Error setting stop trigger for DAC.");
00395         for (channel = 0; channel < my_dac[dac_num].num_dma_channels; channel++) {
00396             result = DM35424_Dma_Start(board,
00397                                         &(my_dac[dac_num]),
00398                                         channel);
00399             DM35424_Check_Result_test(result, "Error starting DMA channel.");
00400         }
00401     }
00402
00403     // Set DAC0 to start them all via Clock Src Global 2
00404     result = DM35424_Dac_Set_Start_Trigger(board,
00405                                             &(my_dac[DAC0]),
00406                                             DM35424_CLK_SRC_IMMEDIATE);
00407
00408     DM35424_Check_Result_test(result, "Error setting start trigger for DAC.");
00409
00410     result = DM35424_Dac_Set_Clock_Source_Global(board,
00411                                                  &my_dac[DAC0],
00412                                                  DM35424_CLK_SRC_GBL2,
00413                                                  DM35424_DAC_CLK_EVENT_START_TRIG);
00414
00415     DM35424_Check_Result_test(result, "Error setting clock global source for DAC 0");
00416
00417
00418     // Start them in reverse order so that they are all waiting for the
00419     // start trigger from DAC 0
00420     for (dac_num = DM35424_NUM_DAC_ON_BOARD; dac_num > 0; dac_num--) {
00421
00422         result = DM35424_Dac_Start(board, &(my_dac[dac_num - 1]));
00423
00424         DM35424_Check_Result_test(result, "Error starting DAC");
00425     }
00426
00427 }
00428
00429
00430
00431
00432 /*****
00433  *          UTILITY FUNCTIONS
00434  *****/
00435
00436 void start_timeout_timer(long timeout_val) {
00437
00438     gettimeofday(&start, NULL);
00439     gettimeofday(&end, NULL);
00440
00441     timeout_msec = timeout_val;
00442
00443 }
00444
00445 int timeout_has_expired() {
00446
00447     long time_diff;
00448
00449
00450     gettimeofday(&end, NULL);
00451     time_diff = DM35424_Get_Time_Diff(end, start) / 1.0e3;
00452
00453     return (time_diff > timeout_msec);
00454 }
00455
00456
00457 int interrupt_matches_expected(uint16_t status, uint16_t desired_status,
00458                               char *message)
00459 {
00460
00461     if (status == desired_status) {
00462         sprintf(message, "Interrupt status matches expected.");
00463         return 1;
00464     }

```

```

00465     else {
00466         sprintf(message, "Interrupt status (0x%x) does not match expected (0x%x)",
00467             status,
00468             desired_status);
00469         return 0;
00470     }
00471 }
00472 }
00473 }
00474 }
00475 }
00476 void output_dma_channel_status(struct DM35424_Board_Descriptor *handle,
00477     const struct DM35424_Function_Block *func_block,
00478     unsigned int channel)
00479 {
00480     int result;
00481     unsigned int current_buffer;
00482     uint32_t current_count;
00483     int current_action;
00484     int status_overflow;
00485     int status_underflow;
00486     int status_used;
00487     int status_invalid;
00488     int status_complete;
00489
00490     result = DM35424_Dma_Status(handle,
00491         func_block,
00492         channel,
00493         &current_buffer,
00494         &current_count,
00495         &current_action,
00496         &status_overflow,
00497         &status_underflow,
00498         &status_used,
00499         &status_invalid,
00500         &status_complete);
00501
00502
00503     DM35424_Check_Result_test(result, "Error getting DMA status");
00504
00505     fprintf(stdout, "\nFB%d DMA%d Status: Current Buffer: %u Count: %ul Action: 0x%x Status: "
00506         "Ov: %d Un: %d Used: %d Inv: %d Comp: %d\n", func_block->fb_num, channel,
00507         current_buffer, current_count, current_action,
00508         status_overflow, status_underflow, status_used,
00509         status_invalid, status_complete);
00510
00511 }
00512
00513
00514 /* This initializes the DMA buffers, with direction and buffer size. But
00515    they will be setup to not do anything, as in no interrupts will
00516    be thrown no matter what. It's just a convenient way to setup
00517    the DMA for use without having to worry about them again. */
00518 void initialize_dma_buffers(struct DM35424_Function_Block *fb,
00519     int num_bytes,
00520     int direction)
00521 {
00522     unsigned int channel = 0;
00523     int result = 0;
00524     int loop_val = 0;
00525     int buff = 0;
00526
00527     for (channel = 0; channel < fb->num_dma_channels; channel++) {
00528
00529         result = DM35424_Dma_Initialize(board,
00530             fb,
00531             channel,
00532             fb->num_dma_buffers,
00533             num_bytes);
00534
00535         DM35424_Check_Result_test(result, "Error initializing DMA");
00536
00537         result = DM35424_Dma_Setup(board,
00538             fb,
00539             channel,
00540             direction,
00541             IGNORE_USED);
00542
00543         DM35424_Check_Result_test(result, "Error setting up DMA");
00544
00545         result = DM35424_Dma_Configure_Interrupts(board,
00546             fb,
00547             channel,
00548             INTERRUPT_DISABLE,
00549             ERROR_INTR_DISABLE);
00550
00551

```

```

00552         DM35424_Check_Result_test(result, "Error setting DMA Interrupts");
00553
00554
00555         for (buff = 0; buff < fb->num_dma_buffers; buff++) {
00556
00557             loop_val = BUFFER_NO_LOOP;
00558             if (buff == (fb->num_dma_buffers - 1)) {
00559                 loop_val = BUFFER_LOOP;
00560
00561             }
00562
00563             result = DM35424_Dma_Buffer_Setup(board,
00564                                               fb,
00565                                               channel,
00566                                               buff,
00567                                               BUFFER_VALID,
00568                                               BUFFER_NO_HALT,
00569                                               loop_val,
00570                                               BUFFER_NO_INTERRUPT);
00571
00572             DM35424_Check_Result_test(result, "Error setting buffer control.");
00573
00574         }
00575
00576     }
00577 }
00578 }
00579
00580
00581
00582 void start_all_dacs_with_dma() {
00583
00584     int dac_num;
00585     unsigned int channel;
00586     int result;
00587
00588     for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD; dac_num++) {
00589
00590         for (channel = 0; channel < my_dac[dac_num].num_dma_channels; channel++) {
00591             result = DM35424_Dma_Start(board,
00592                                       &my_dac[dac_num],
00593                                       channel);
00594
00595             DM35424_Check_Result_test(result, "Error starting DMA.");
00596
00597         }
00598
00599         result = DM35424_Dac_Set_Start_Trigger(board,
00600                                               &my_dac[dac_num],
00601                                               DM35424_CLK_SRC_IMMEDIATE);
00602
00603         DM35424_Check_Result_test(result, "Error setting start trigger for DAC.");
00604
00605         result = DM35424_Dac_Set_Stop_Trigger(board,
00606                                               &my_dac[dac_num],
00607                                               DM35424_CLK_SRC_NEVER);
00608
00609         DM35424_Check_Result_test(result, "Error setting stop trigger for DAC.");
00610
00611         result = DM35424_Dac_Start(board,
00612                                   &my_dac[dac_num]);
00613
00614         DM35424_Check_Result_test(result, "Error starting DAC");
00615
00616         fprintf(stdout, "DAC%d: Started.\n", dac_num);
00617
00618     }
00619 }
00620 }
00621
00622
00623 void stop_all_dacs_with_dma() {
00624
00625     int dac_num;
00626     unsigned int channel;
00627     int result;
00628
00629     for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD; dac_num++) {
00630
00631         for (channel = 0; channel < my_dac[dac_num].num_dma_channels; channel++) {
00632
00633             result = DM35424_Dma_Stop(board,
00634                                       &my_dac[dac_num],
00635                                       channel);
00636
00637             DM35424_Check_Result_test(result, "Error stopping DAC DMA.");
00638
00639         }
00640
00641     }
00642 }

```

```

00639
00640         result = DM35424_Dac_Reset(board,
00641                                     &my_dac[dac_num]);
00642
00643         DM35424_Check_Result_test(result, "Error stopping DAC.");
00644     }
00645 }
00646
00647 }
00648
00649 void stop_all_adcs_with_dma() {
00650     int adc_num;
00651     unsigned int channel;
00652     int result;
00653
00654     for (adc_num = 0; adc_num < DM35424_NUM_ADC_ON_BOARD; adc_num++) {
00655         for (channel = 0; channel < my_adc[adc_num].num_dma_channels; channel++) {
00656             result = DM35424_Dma_Stop(board,
00657                                         &my_adc[adc_num],
00658                                         channel);
00659
00660             DM35424_Check_Result_test(result, "Error stopping ADC DMA.");
00661         }
00662         result = DM35424_Adc_Reset(board,
00663                                     &my_adc[adc_num]);
00664
00665         DM35424_Check_Result_test(result, "Error stopping ADC.");
00666     }
00667 }
00668
00669 void reset_all_dma(struct DM35424_Function_Block *fb)
00670 {
00671     unsigned int channel;
00672     int result = 0;
00673     int buff = 0;
00674
00675     for (channel = 0; channel < fb->num_dma_channels; channel++) {
00676         result = DM35424_Dma_Clear(board,
00677                                     fb,
00678                                     channel);
00679
00680         DM35424_Check_Result_test(result, "Error clearing DMA.");
00681
00682         result = DM35424_Dma_Configure_Interrupts(board,
00683                                                     fb,
00684                                                     channel,
00685                                                     INTERRUPT_DISABLE,
00686                                                     ERROR_INTR_DISABLE);
00687
00688         DM35424_Check_Result_test(result, "Error setting DMA Interrupts");
00689
00690         for (buff = 0; buff < fb->num_dma_buffers; buff++) {
00691             result = DM35424_Dma_Reset_Buffer(board,
00692                                                 fb,
00693                                                 channel,
00694                                                 buff);
00695
00696             DM35424_Check_Result_test(result, "Error resetting buffer.");
00697
00698             result = DM35424_Dma_Buffer_Setup(board,
00699                                                 fb,
00700                                                 channel,
00701                                                 buff,
00702                                                 BUFFER_VALID,
00703                                                 BUFFER_NO_HALT,
00704                                                 BUFFER_NO_LOOP,
00705                                                 BUFFER_NO_INTERRUPT);
00706
00707             DM35424_Check_Result_test(result, "Error setting buffer control.");
00708         }
00709     }
00710
00711     result = DM35424_Dma_Clear_Interrupt(board,
00712                                           fb,
00713                                           channel,
00714                                           CLEAR_INTERRUPT,
00715                                           );
00716 }
00717
00718
00719
00720
00721
00722
00723
00724
00725

```

```

00726                                     CLEAR_INTERRUPT,
00727                                     CLEAR_INTERRUPT,
00728                                     CLEAR_INTERRUPT,
00729                                     CLEAR_INTERRUPT);
00730
00731             DM35424_Check_Result_test(result, "Error clearing DMA interrupts.");
00732     }
00733 }
00734
00735 }
00736
00737
00738
00739 void test7_reset_dac_and_dma(struct DM35424_Function_Block *fb)
00740 {
00741     {
00742
00743         stop_all_dacs_with_dma();
00744
00745         reset_all_dma(fb);
00746     }
00747 }
00748
00749 void test8_reset_adc_and_dma(struct DM35424_Function_Block *fb)
00750 {
00751     {
00752         stop_all_adcs_with_dma();
00753         reset_all_dma(fb);
00754     }
00755 }
00756
00757 void open_dacs()
00758 {
00759     int dac_num;
00760     int result;
00761     for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD; dac_num++) {
00762
00763         result = DM35424_Dac_Open(board,
00764                                   dac_num,
00765                                   &my_dac[dac_num]);
00766         DM35424_Check_Result_test(result, "Could not open DAC");
00767
00768         fprintf(stdout, "Opening DAC%d, with %d DMA channels (%d buffers each)\n",
00769                 dac_num, my_dac[dac_num].num_dma_channels, my_dac[dac_num].num_dma_buffers);
00770         DM35424_Check_Result_test(my_dac[dac_num].num_dma_channels !=
00771                                   DM35424_NUM_DAC_DMA_CHANNELS,
00772                                   "Number of DAC DMA channels does not match expected.");
00773         DM35424_Check_Result_test(my_dac[dac_num].num_dma_buffers !=
00774                                   DM35424_NUM_DAC_DMA_BUFFERS,
00775                                   "Number of DAC DMA buffers does not match expected.");
00776
00777         result = DM35424_Dac_Reset(board,
00778                                     &my_dac[dac_num]);
00779         DM35424_Check_Result_test(result, "Error stopping DAC.");
00780     }
00781 }
00782
00783 void open_adcs()
00784 {
00785     int adc_num;
00786     int result;
00787     for (adc_num = 0; adc_num < DM35424_NUM_ADC_ON_BOARD; adc_num++) {
00788
00789         result = DM35424_Adc_Open(board,
00790                                   adc_num,
00791                                   &my_adc[adc_num]);
00792     }
00793 }
00794
00795
00796
00797
00798
00799
00800
00801
00802
00803
00804
00805
00806
00807
00808
00809
00810

```

```

00811
00812         DM35424_Check_Result_test(result, "Could not open ADC");
00813
00814
00815         fprintf(stdout, "Opening ADC%d, with %d DMA channels (%d buffers each)\n",
00816                 adc_num, my_adc[adc_num].num_dma_channels, my_adc[adc_num].num_dma_buffers);
00817
00818         DM35424_Check_Result_test(my_adc[adc_num].num_dma_channels !=
DM35424_NUM_ADC_DMA_CHANNELS,
00819                                   "Number of ADC DMA channels does not match expected.");
00820         DM35424_Check_Result_test(my_adc[adc_num].num_dma_buffers !=
DM35424_NUM_ADC_DMA_BUFFERS,
00821                                   "Number of ADC DMA buffers does not match expected.");
00822
00823     }
00824 }
00825
00826
00827 void initialize_dacs(uint32_t rate)
00828 {
00829     int dac_num;
00830     int result;
00831     uint32_t actual_rate;
00832     unsigned int channel;
00833
00834     for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD; dac_num++) {
00835
00836         result = DM35424_Dac_Reset(board,
                                &my_dac[dac_num]);
00837
00838
00839         DM35424_Check_Result_test(result, "Error stopping DAC.");
00840
00841         // Disable and clear all interrupt registers
00842         result = DM35424_Dac_Interrupt_Set_Config(board,
                                &my_dac[dac_num],
00843                                DM35424_DAC_INT_ALL_MASK,
00844                                INTERRUPT_DISABLE);
00845
00846
00847         DM35424_Check_Result_test(result, "Error disabling interrupts.");
00848
00849
00850         result = DM35424_Dac_Interrupt_Clear_Status(board,
                                &my_dac[dac_num],
00851                                DM35424_DAC_INT_ALL_MASK);
00852
00853         DM35424_Check_Result_test(result, "Error clearing interrupt status.");
00854
00855         if (rate > 0) {
00856
00857             result = DM35424_Dac_Set_Clock_Src(board,
                                &my_dac[dac_num],
00858                                DM35424_CLK_SRC_IMMEDIATE);
00859
00860
00861             DM35424_Check_Result_test(result, "Error setting DAC clock");
00862
00863             result = DM35424_Dac_Set_Conversion_Rate(board,
                                &my_dac[dac_num],
00864                                rate,
00865                                &actual_rate);
00866
00867
00868             DM35424_Check_Result_test(result, "Error setting sample rate");
00869         }
00870
00871
00872         for (channel = 0; channel < my_dac[dac_num].num_dma_channels; channel++) {
00873
00874             result = DM35424_Dma_Clear_Interrupt(board,
                                &my_dac[dac_num],
00875                                channel,
00876                                CLEAR_INTERRUPT,
00877                                CLEAR_INTERRUPT,
00878                                CLEAR_INTERRUPT,
00879                                CLEAR_INTERRUPT,
00880                                CLEAR_INTERRUPT);
00881
00882
00883             DM35424_Check_Result_test(result, "Error clearing channel interrupts.");
00884         }
00885     }
00886 }
00887 }
00888
00889
00890
00891
00892 void initialize_adcs(uint32_t rate)
00893 {
00894     int adc_num;
00895     int result;

```

```
00896     unsigned int channel;
00897     uint32_t actual_rate;
00898
00899     for (adc_num = 0; adc_num < DM35424_NUM_ADC_ON_BOARD; adc_num++) {
00900
00901         result = DM35424_Adc_Reset(board,
00902                                     &my_adc[adc_num]);
00903
00904         DM35424_Check_Result_test(result, "Error resetting ADC.");
00905
00906         result = DM35424_Adc_Interrupt_Set_Config(board,
00907                                                     &my_adc[adc_num],
00908                                                     DM35424_ADC_INT_ALL_MASK,
00909                                                     INTERRUPT_DISABLE);
00910
00911         DM35424_Check_Result_test(result, "Error disabling interrupts.");
00912
00913         result = DM35424_Adc_Interrupt_Clear_Status(board,
00914                                                       &my_adc[adc_num],
00915                                                       DM35424_ADC_INT_ALL_MASK);
00916
00917         DM35424_Check_Result_test(result, "Error clearing interrupt status.");
00918
00919         result = DM35424_Adc_Set_Clock_Src(board,
00920                                             &my_adc[adc_num],
00921                                             DM35424_CLK_SRC_IMMEDIATE);
00922
00923         DM35424_Check_Result_test(result, "Error setting ADC clock");
00924
00925         result = DM35424_Adc_Set_Pre_Trigger_Samples(board,
00926                                                       &my_adc[adc_num],
00927                                                       0);
00928
00929         DM35424_Check_Result_test(result, "Error setting Pre capture values.");
00930
00931         result = DM35424_Adc_Set_Post_Stop_Samples(board,
00932                                                     &my_adc[adc_num],
00933                                                     0);
00934
00935         DM35424_Check_Result_test(result, "Error setting Post capture values.");
00936
00937         for (channel = 0; channel < my_adc[adc_num].num_dma_channels; channel++) {
00938
00939             result = DM35424_Adc_Channel_Setup(board,
00940                                                 &my_adc[adc_num],
00941                                                 channel,
00942                                                 DM35424_ADC_RNG_BIPOLAR_2_5V,
00943                                                 DM35424_ADC_INPUT_DIFFERENTIAL);
00944
00945             DM35424_Check_Result_test(result, "Error setting up channel.");
00946
00947             result = DM35424_Dma_Clear_Interrupt(board,
00948                                                  &my_adc[adc_num],
00949                                                  channel,
00950                                                  CLEAR_INTERRUPT,
00951                                                  CLEAR_INTERRUPT,
00952                                                  CLEAR_INTERRUPT,
00953                                                  CLEAR_INTERRUPT,
00954                                                  CLEAR_INTERRUPT);
00955
00956             DM35424_Check_Result_test(result, "Error clearing channel interrupts.");
00957
00958         }
00959
00960         result = DM35424_Adc_Ad_Config_Set_Mode(board,
00961                                                  &my_adc[adc_num],
00962                                                  DM35424_ADC_MODE_CONFIG_HIGH_SPEED);
00963
00964         DM35424_Check_Result_test(result, "Error setting AD config.");
00965
00966         result = DM35424_Adc_Set_Start_Trigger(board,
00967                                                 &my_adc[adc_num],
00968                                                 DM35424_CLK_SRC_IMMEDIATE);
00969
00970         DM35424_Check_Result_test(result, "Error setting start trigger.");
00971
00972         result = DM35424_Adc_Set_Stop_Trigger(board,
00973                                                &my_adc[adc_num],
00974                                                DM35424_CLK_SRC_NEVER);
00975
00976         DM35424_Check_Result_test(result, "Error setting stop trigger.");
00977
00978         result = DM35424_Adc_Set_Sample_Rate(board,
00979                                              &my_adc[adc_num],
```



```

00983         rate, &actual_rate);
00984
00985     DM35424_Check_Result_test(result, "Failed to set sample rate for ADC.");
00986
00987     fprintf(stdout, "ADC%d: Initializing...", adc_num);
00988     result = DM35424_Adc_Initialize(board,
00989                                     &my_adc[adc_num]);
00990
00991     DM35424_Check_Result_test(result, "Failed or timed out initializing ADC.");
00992     fprintf(stdout, "success.\n");
00993
00994 }
00995 }
00996
00997
00998 void write_to_kernel_log(char *str)
00999 {
01000
01001     FILE *fp;
01002
01003
01004     fp = fopen(PROC_FILE, "w");
01005
01006     if (fp == NULL) {
01007         fprintf(stderr, "**Was unable to write to kernel log.\n");
01008         return;
01009     }
01010
01011     fprintf(fp, "» %s", str);
01012
01013     fclose(fp);
01014
01015 }
01016
01017
01018
01019
01020 void output_all_interrupts(void)
01021 {
01022
01023     unsigned int adc_num;
01024     unsigned int dac_num;
01025     int result;
01026     unsigned int channel;
01027     uint16_t adc_status = 0;
01028     uint16_t adc_enable = 0;
01029     uint16_t dac_status = 0;
01030     uint16_t dac_enable = 0;
01031     int interrupts_found = 0;
01032     int int_enabled = 0;
01033     int error_enabled = 0;
01034     unsigned int current_buffer;
01035     uint32_t current_count;
01036     int current_action;
01037     int status_overflow;
01038     int status_underflow;
01039     int status_used;
01040     int status_invalid;
01041     int status_complete;
01042
01043     fprintf(stdout, "\n");
01044     for (adc_num = 0; adc_num < DM35424_NUM_ADC_ON_BOARD; adc_num++) {
01045         result = DM35424_Adc_Interrupt_Get_Status(board,
01046                                                     &my_adc[adc_num],
01047                                                     &adc_status);
01048
01049         DM35424_Check_Result_test(result, "Error getting ADC interrupt status.");
01050
01051         result = DM35424_Adc_Interrupt_Get_Config(board,
01052                                                     &my_adc[adc_num],
01053                                                     &adc_enable);
01054
01055         DM35424_Check_Result_test(result, "Error getting ADC interrupt enable.");
01056
01057         if (adc_status & adc_enable) {
01058             fprintf(stdout, "ADC%d: Status (0x%x) and Enable (0x%x): 0x%x\n",
01059                     adc_num, adc_status, adc_enable, (adc_status & adc_enable));
01060             interrupts_found++;
01061         }
01062         for (channel = 0; channel < my_adc[adc_num].num_dma_channels; channel++) {
01063             result = DM35424_Dma_Get_Interrupt_Configuration(board,
01064                                                             &my_adc[adc_num],
01065                                                             channel,
01066                                                             &int_enabled,
01067                                                             &error_enabled);
01068             DM35424_Check_Result_test(result, "Error getting interrupt configuration.");
01069

```

```

01070         if (int_enabled || error_enabled) {
01071
01072             result = DM35424_Dma_Status(board,
01073                                         &my_adc[adc_num],
01074                                         channel,
01075                                         &current_buffer,
01076                                         &current_count,
01077                                         &current_action,
01078                                         &status_overflow,
01079                                         &status_underflow,
01080                                         &status_used,
01081                                         &status_invalid,
01082                                         &status_complete);
01083
01084
01085             DM35424_Check_Result_test(result, "Error getting DMA status");
01086
01087             if (int_enabled && status_complete) {
01088
01089                 interrupts_found++;
01090
01091                 output_dma_channel_status(board,
01092                                           &my_adc[adc_num],
01093                                           channel);
01094             }
01095             else if (error_enabled && (status_overflow ||
01096                                       status_underflow ||
01097                                       status_used ||
01098                                       status_invalid)) {
01099
01100                 interrupts_found++;
01101
01102                 output_dma_channel_status(board,
01103                                           &my_adc[adc_num],
01104                                           channel);
01105             }
01106         }
01107     }
01108 }
01109
01110 }
01111
01112
01113 for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD; dac_num++) {
01114
01115     result = DM35424_Dac_Interrupt_Get_Status(board,
01116                                               &my_dac[dac_num],
01117                                               &dac_status);
01118
01119     DM35424_Check_Result_test(result, "Error getting DAC interrupt status.");
01120
01121     result = DM35424_Dac_Interrupt_Get_Config(board,
01122                                               &my_dac[dac_num],
01123                                               &dac_enable);
01124
01125     DM35424_Check_Result_test(result, "Error getting DAC interrupt enable.");
01126
01127     if (dac_status & dac_enable) {
01128         fprintf(stdout, "DAC%d: Status (0x%x) and Enable (0x%x): 0x%x\n",
01129                dac_num, dac_status, dac_enable, (dac_status & dac_enable));
01130         interrupts_found++;
01131     }
01132     for (channel = 0; channel < my_dac[dac_num].num_dma_channels; channel++) {
01133         result = DM35424_Dma_Get_Interrupt_Configuration(board,
01134                                                         &my_dac[dac_num],
01135                                                         channel,
01136                                                         &int_enabled,
01137                                                         &error_enabled);
01138
01139         DM35424_Check_Result_test(result, "Error getting interrupt configuration.");
01140
01141         if (int_enabled || error_enabled) {
01142
01143             result = DM35424_Dma_Status(board,
01144                                         &my_dac[dac_num],
01145                                         channel,
01146                                         &current_buffer,
01147                                         &current_count,
01148                                         &current_action,
01149                                         &status_overflow,
01150                                         &status_underflow,
01151                                         &status_used,
01152                                         &status_invalid,
01153                                         &status_complete);
01154
01155
01156             DM35424_Check_Result_test(result, "Error getting DMA status");

```

```

01157
01158         if (int_enabled && status_complete) {
01159             interrupts_found ++;
01160             output_dma_channel_status(board,
01161                                     &my_dac[dac_num],
01162                                     channel);
01163         }
01164     }
01165     else if (error_enabled && (status_overflow ||
01166                             status_underflow ||
01167                             status_used ||
01168                             status_invalid)) {
01169         interrupts_found ++;
01170         output_dma_channel_status(board,
01171                                 &my_dac[dac_num],
01172                                 channel);
01173     }
01174 }
01175 }
01176 }
01177 }
01178 }
01179 }
01180 }
01181 }
01182 }
01183 if (!interrupts_found) {
01184     fprintf(stdout, "No interrupts were found.\n");
01185 }
01186 }
01187 }
01188
01189 void list_all_enabled_channels(struct DM35424_Function_Block *fb)
01190 {
01191     int result;
01192     unsigned int channel;
01193     int enable;
01194     int error_enable;
01195
01196     for (channel = 0; channel < fb->num_dma_channels; channel++) {
01197         result = DM35424_Dma_Get_Interrupt_Configuration(board,
01198                                                         fb,
01199                                                         channel,
01200                                                         &enable,
01201                                                         &error_enable);
01202
01203         DM35424_Check_Result_test(result, "Error setting DMA Interrupts");
01204
01205         if (enable || error_enable) {
01206             fprintf(stdout, "FB%d DMA%d Interrupts: %d Error Ints: %d\n",
01207                     fb->fb_num,
01208                     channel,
01209                     enable,
01210                     error_enable);
01211         }
01212     }
01213 }
01214 }
01215 }
01216 }
01217 }
01218
01219 void output_dma_buffer_status(struct DM35424_Function_Block *fb,
01220                             unsigned int channel,
01221                             unsigned int buffer)
01222 {
01223     int result;
01224     uint32_t buff_size;
01225     uint8_t buff_status;
01226     uint8_t buff_control;
01227
01228     result = DM35424_Dma_Buffer_Status(board,
01229                                         fb,
01230                                         channel,
01231                                         buffer,
01232                                         &buff_status,
01233                                         &buff_control,
01234                                         &buff_size);
01235
01236     DM35424_Check_Result_test(result, "Error getting buffer status.");
01237
01238     fprintf(stdout, "    Buffer %d: Stat: 0x%x Ctrl: 0x%x Size: %u\n",
01239             buffer, buff_status, buff_control, buff_size);
01240 }
01241 }
01242
01243 void output_all_buffers(struct DM35424_Function_Block *fb,

```

```

01244             unsigned int channel)
01245 {
01246     unsigned int buffer;
01247     fprintf(stdout, "\n");
01248     for (buffer = 0; buffer < fb->num_dma_buffers; buffer++) {
01249
01250         output_dma_buffer_status(fb, channel, buffer);
01251     }
01252 }
01253 }
01254
01255 void Check_Interrupts(int num_expected, int fb_expected, int dma_expected)
01256 {
01257     char message[200];
01258     sprintf(message, "Expected %d interrupt, but received %d.",
01259             num_expected,
01260             interrupt_count);
01261
01262     if (interrupt_count != num_expected) {
01263         output_all_interrupts();
01264         DM35424_Check_Result_test(1, message);
01265     }
01266
01267     if (dma_expected) {
01268         sprintf(message, "Expected DMA interrupt for FB %d, but received from FB %d.",
01269                 fb_expected, interrupt_fb);
01270
01271         if (fb_expected != interrupt_dma_fb) {
01272             output_all_interrupts();
01273             DM35424_Check_Result_test(1, message);
01274         }
01275     }
01276     else {
01277         sprintf(message, "Expected interrupt for FB %d, but received from FB %d.",
01278                 fb_expected, interrupt_fb);
01279
01280         if (fb_expected != interrupt_fb) {
01281             output_all_interrupts();
01282             DM35424_Check_Result_test(1, message);
01283         }
01284     }
01285
01286     interrupt_fb = -1;
01287     interrupt_dma_fb = -1;
01288 }
01289
01290 void eat_adc_samples(struct DM35424_Function_Block *fb)
01291 {
01292     uint32_t num_samples = 0;
01293     int result;
01294
01295     result = DM35424_Adc_Get_Sample_Count(board,
01296             fb,
01297             &num_samples);
01298
01299     DM35424_Check_Result_test(result, "Error getting sample count");
01300
01301     while (num_samples < SAMPLES_TO_DISCARD) {
01302         result = DM35424_Adc_Get_Sample_Count(board,
01303                 fb,
01304                 &num_samples);
01305
01306         DM35424_Check_Result_test(result, "Error getting sample count");
01307     }
01308 }
01309
01310
01311
01312
01313
01314 /*=====
01315     ISR
01316     =====*/
01317 void ISR(struct dm35424_ioctl_interrupt_info_request int_info)
01318 {
01319     char message[200];
01320     int dma = (int_info.interrupt_fb < 0);
01321
01322     if (dma) {
01323         interrupt_dma_fb = int_info.interrupt_fb & 0x7FFFFFFF;
01324         //fprintf(stdout, "\n*** DMA Interrupt received from %d.\n", interrupt_dma_fb);
01325     }
01326     else {
01327         interrupt_fb = int_info.interrupt_fb;
01328         //fprintf(stdout, "\n*** Interrupt received from %d.\n", interrupt_fb);
01329     }
01330 }

```

```

01331     }
01332
01333
01334     if (!interrupts_expected) {
01335         output_all_interrupts();
01336         if (dma) {
01337             sprintf(message, "Received unexpected DMA interrupt from FB %d.\n",
01338                 interrupt_dma_fb);
01339         }
01340         else {
01341             sprintf(message, "Received unexpected interrupt from FB %d.\n",
01342                 interrupt_fb);
01343         }
01344
01345         DM35424_Check_Result_test(1, message);
01346     }
01347
01348     if (int_info.error_occurred) {
01349
01350         fprintf(stdout, "ISR: Error received (%d).\n", int_info.error_occurred);
01351         return;
01352     }
01353
01354
01355
01356     if (int_info.valid_interrupt ) {
01357
01358         interrupt_count++;
01359     }
01360     else {
01361         fprintf(stdout, "WARNING: A non-valid interrupt was received.\n");
01362     }
01363 }
01364
01365
01366
01367 void set_isr()
01368 {
01369     int result;
01370
01371     result = DM35424_General_InstallISR(board, ISR);
01372     DM35424_Check_Result_test(result, "DM35424_General_InstallISR()");
01373 }
01374
01375
01376
01377
01378
01379
01380
01381
01382 void run_test_0(unsigned int minor)
01383 {
01384     int result;
01385     uint8_t uint8_value;
01386     uint32_t uint32_value;
01387     char message[200];
01388
01389     interrupts_expected = 0;
01390     fprintf(stdout, "\n\n-----\n");
01391     fprintf(stdout, "Test 0: General Board Control Registers check\n");
01392
01393     printf("Opening board....");
01394     result = DM35424_Board_Open(minor, &board);
01395
01396     DM35424_Check_Result_test(result, "Could not open board");
01397     printf("success.\nResetting board....");
01398     result = DM35424_Gbc_Board_Reset(board);
01399
01400     DM35424_Check_Result_test(result, "Could not reset board");
01401
01402     open_dacs();
01403
01404     initialize_dacs(TEST_0_DAC_CONVERSION_RATE);
01405
01406     /* Read the CLK DIV register and make sure it is not zero */
01407     result = DM35424_Dac_Get_Clock_Div(board,
01408         &my_dac[DAC0],
01409         &uint32_value);
01410
01411     DM35424_Check_Result_test(result, "Error reading DAC clock div.");
01412
01413     DM35424_Check_Result_test(uint32_value == 0, "Expected a non-zero clock div, but got zero.");

```

```

01414
01415     fprintf(stdout, "Testing Reset...\n");
01416     result = DM35424_Gbc_Board_Reset(board);
01417
01418     DM35424_Check_Result_test(result, "Could not reset board");
01419
01420     /* Read the CLK DIV register and make sure it is zero */
01421     result = DM35424_Dac_Get_Clock_Div(board,
01422                                         &my_dac[DAC0],
01423                                         &uint32_value);
01424
01425     DM35424_Check_Result_test(result, "Error reading DAC clock div.");
01426
01427     sprintf(message, "Expected DAC clock div to be reset to zero, but got %u.",
01428                 uint32_value);
01429
01430     DM35424_Check_Result_test(uint32_value != 0, message);
01431
01432     fprintf(stdout, "success.\n");
01433
01434     result = DM35424_Gbc_Get_Format(board,
01435                                     &uint8_value);
01436
01437     DM35424_Check_Result_test(result, "Error reading board format.");
01438     DM35424_Check_Result_test(uint8_value == 0, "Format is zero, and that was not expected.");
01439
01440     fprintf(stdout, "Board format: %u\n", uint8_value);
01441
01442
01443     result = DM35424_Gbc_Get_Revision(board,
01444                                       &uint8_value);
01445
01446     DM35424_Check_Result_test(result, "Error reading board revision.");
01447     DM35424_Check_Result_test(uint8_value == 0, "Revision is zero, and that was not expected.");
01448
01449     fprintf(stdout, "Board revision: %u\n", uint8_value);
01450
01451     result = DM35424_Gbc_Get_Pdp_Number(board,
01452                                         &uint32_value);
01453
01454     DM35424_Check_Result_test(result, "Error reading PDP Number.");
01455     DM35424_Check_Result_test(uint32_value == 0, "PDP Number is zero, and that was not
01456 expected.");
01457
01458     fprintf(stdout, "Board PDP Number: %u\n", uint32_value);
01459
01460     result = DM35424_Gbc_Get_Fpga_Build(board,
01461                                         &uint32_value);
01462
01463     DM35424_Check_Result_test(result, "Error reading FPGA Build.");
01464     DM35424_Check_Result_test(uint32_value == 0, "PDP Number is zero, and that was not
01465 expected.");
01466
01467     fprintf(stdout, "Board FPGA Build: %u\n", uint32_value);
01468
01469     printf("\nClosing Board...\n");
01470     result = DM35424_Board_Close(board);
01471
01472     DM35424_Check_Result_test(result, "Error closing board.");
01473
01474
01475 }
01476
01477
01478
01479
01480
01481
01482
01483
01484
01485 void run_test_1(unsigned int minor)
01486 {
01487     int result;
01488     unsigned int dac_num, adc_num;
01489     unsigned int channel;
01490     unsigned long local_int_count = 0;
01491     uint16_t interrupt_status;
01492     uint32_t start_sample_count = 0;
01493     int num_ints_processed = 0;
01494     char message[200];

```

```

01495     int32_t adc_value;
01496     uint32_t sample_counts = 0;
01497     int16_t dac_value;
01498
01499     int32_t max_adc_value, min_adc_value;
01500
01501     interrupts_expected = 0;
01502     fprintf(stdout, "\n\n-----\n");
01503     fprintf(stdout, "Test 1: Get Minimum signal out of ADCs. All DACs at 2.5 V\n");
01504
01505     printf("Opening board....");
01506     result = DM35424_Board_Open(minor, &board);
01507
01508     DM35424_Check_Result_test(result, "Could not open board");
01509     printf("success.\nResetting board....\n");
01510     result = DM35424_Gbc_Board_Reset(board);
01511
01512     DM35424_Check_Result_test(result, "Could not reset board");
01513
01514     open_dacs();
01515
01516     initialize_dacs(0);
01517
01518     open_adcs();
01519
01520     initialize_adcs(TEST_1_ADC_SAMPLE_RATE);
01521
01522     set_isr();
01523
01524     DM35424_Dac_Volts_To_Conv(2.5f,
01525                               &dac_value);
01526
01527
01528     // A simple initial test. Set all DACs to the same value, which should
01529     // read about zero on the attached ADCs
01530     for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD; dac_num++) {
01531
01532         for (channel = 0; channel < my_dac[dac_num].num_dma_channels; channel++) {
01533             result = DM35424_Dac_Set_Last_Conversion(board,
01534                                                       &my_dac[dac_num],
01535                                                       channel,
01536                                                       0,
01537                                                       dac_value);
01538
01539         }
01540     }
01541
01542     result = DM35424_Adc_Volts_To_Sample(.01f,
01543                                           &max_adc_value);
01544
01545     result = DM35424_Adc_Volts_To_Sample(-.01f,
01546                                           &min_adc_value);
01547
01548
01549
01550     // Test 1
01551     // We're going to start 1 ADC at a time, and verify we're receiving interrupts
01552     // before we stop it and then move to the next. We'll include a timeout to
01553     // keep from locking up.
01554     for (adc_num = 0; adc_num < DM35424_NUM_ADC_ON_BOARD; adc_num++) {
01555
01556         interrupt_count = 0;
01557         local_int_count = 0;
01558
01559         result = DM35424_Adc_Interrupt_Set_Config(board,
01560                                                     &my_adc[adc_num],
01561                                                     DM35424_ADC_INT_SAMPLE_TAKEN_MASK,
01562                                                     INTERRUPT_ENABLE);
01563
01564         DM35424_Check_Result_test(result, "Error setting interrupt.");
01565         fprintf(stdout, "ADC%d: Setting interrupt enable.\n", adc_num);
01566
01567         // Confirm before start that the status is clear
01568         result = DM35424_Adc_Interrupt_Get_Status(board,
01569                                                     &my_adc[adc_num],
01570                                                     &interrupt_status);
01571
01572         DM35424_Check_Result_test(result, "Error getting interrupt status.");
01573
01574         DM35424_Check_Result_test(!interrupt_matches_expected(interrupt_status,
01575                                                                DM35424_ADC_INT_PACER_TICK_MASK, message),
01576                                   message);
01577
01578         interrupts_expected = 1;
01579         result = DM35424_Adc_Start(board,
01580                                    &my_adc[adc_num]);

```

```

01581
01582     DM35424_Check_Result_test(result, "Error starting ADC");
01583     fprintf(stdout, "ADC%d: Started.\n", adc_num);
01584
01585
01586     start_timeout_timer(5000);
01587
01588     while (interrupt_count < 1 && !timeout_has_expired()) {}
01589
01590     DM35424_Check_Result_test(interrupt_count == 0, "Timed out waiting for interrupt from
ADC.");
01591
01592     eat_adc_samples(&my_adc[adc_num]);
01593
01594     for (channel = 0; channel < my_adc[adc_num].num_dma_channels; channel++) {
01595
01596         start_sample_count = -1;
01597
01598         num_ints_processed = 0;
01599         while (num_ints_processed <= TEST_1_ITERATIONS && !exit_program) {
01600
01601             if (local_int_count < interrupt_count) {
01602
01603                 // An interrupt has arrived. Make sure it's from this ADC
01604                 result = DM35424_Adc_Interrupt_Get_Status(board,
01605
01606                     &my_adc[adc_num],
01607
01608                     &interrupt_status);
01609
01610                 DM35424_Check_Result_test(result, "Error getting channel with
interrupt.");
01611
01612                 // The first interrupt will be because of sample taken
01613                 // and start trigger (which was immediate), so clear
01614                 // that and then check the next interrupts for just
01615                 // sample taken.
01616
01617                 if (local_int_count == 0) {
01618
01619                     DM35424_Check_Result_test(interrupt_matches_expected(interrupt_status,
01620
01621                         (DM35424_ADC_INT_SAMPLE_TAKEN_MASK |
01622
01623                         DM35424_ADC_INT_START_TRIG_MASK |
01624
01625                         DM35424_ADC_INT_PRE_BUFF_FULL_MASK |
01626
01627                         DM35424_ADC_INT_PACER_TICK_MASK),
01628
01629                         message) == 0,
01630                         message);
01631
01632                     }
01633                     else {
01634
01635                         DM35424_Check_Result_test(interrupt_matches_expected(interrupt_status,
01636
01637                         (DM35424_ADC_INT_SAMPLE_TAKEN_MASK |
01638
01639                         DM35424_ADC_INT_PACER_TICK_MASK),
01640
01641                         message) == 0,
01642                         message);
01643
01644                     }
01645
01646                     local_int_count = interrupt_count;
01647
01648                     result = DM35424_Adc_Channel_Get_Last_Sample(board,
01649
01650                         &my_adc[adc_num],
01651                         channel,
01652                         &adc_value);
01653
01654                     DM35424_Check_Result_test(result, "Error getting ADC value.");
01655
01656                     // Test 1
01657
01658                     result = DM35424_Adc_Get_Sample_Count(board,
01659
01660                         &my_adc[adc_num],
01661                         &sample_counts);
01662
01663                     DM35424_Check_Result_test(result, "Error getting sample
counts.");
01664
01665                     if (start_sample_count < 0) {
01666                         start_sample_count = sample_counts;
01667                     }
01668
01669
01670
01671
01672
01673
01674
01675
01676
01677
01678
01679
01680
01681
01682
01683
01684
01685
01686
01687
01688
01689
01690
01691
01692
01693
01694
01695
01696
01697
01698
01699
01700
01701
01702
01703
01704
01705
01706
01707
01708
01709
01710
01711
01712
01713
01714
01715
01716
01717
01718
01719
01720
01721
01722
01723
01724
01725
01726
01727
01728
01729
01730
01731
01732
01733
01734
01735
01736
01737
01738
01739
01740
01741
01742
01743
01744
01745
01746
01747
01748
01749
01750
01751
01752
01753
01754
01755
01756
01757
01758
01759
01760
01761
01762
01763
01764
01765
01766
01767
01768
01769
01770
01771
01772
01773
01774
01775
01776
01777
01778
01779
01780
01781
01782
01783
01784
01785
01786
01787
01788
01789
01790
01791
01792
01793
01794
01795
01796
01797
01798
01799
01800
01801
01802
01803
01804
01805
01806
01807
01808
01809
01810
01811
01812
01813
01814
01815
01816
01817
01818
01819
01820
01821
01822
01823
01824
01825
01826
01827
01828
01829
01830
01831
01832
01833
01834
01835
01836
01837
01838
01839
01840
01841
01842
01843
01844
01845
01846
01847
01848
01849
01850
01851
01852
01853
01854
01855
01856
01857
01858
01859
01860
01861
01862
01863
01864
01865
01866
01867
01868
01869
01870
01871
01872
01873
01874
01875
01876
01877
01878
01879
01880
01881
01882
01883
01884
01885
01886
01887
01888
01889
01890
01891
01892
01893
01894
01895
01896
01897
01898
01899
01900
01901
01902
01903
01904
01905
01906
01907
01908
01909
01910
01911
01912
01913
01914
01915
01916
01917
01918
01919
01920
01921
01922
01923
01924
01925
01926
01927
01928
01929
01930
01931
01932
01933
01934
01935
01936
01937
01938
01939
01940
01941
01942
01943
01944
01945
01946
01947
01948
01949
01950
01951
01952
01953
01954
01955
01956
01957
01958
01959
01960
01961
01962
01963
01964
01965
01966
01967
01968
01969
01970
01971
01972
01973
01974
01975
01976
01977
01978
01979
01980
01981
01982
01983
01984
01985
01986
01987
01988
01989
01990
01991
01992
01993
01994
01995
01996
01997
01998
01999

```



```

        count: %d Ints: %d                \r",                adc_num, channel, adc_value, sample_counts,
01654    num_ints_processed);
01655
01656        if (adc_value > max_adc_value || adc_value < min_adc_value) {
01657            sprintf(message, "Value (%d) is outside expected range
                                adc_value, max_adc_value,
01658            min_adc_value);
                                DM35424_Check_Result_test(1, message);
01659        }
01660
01661        result = DM35424_Adc_Interrupt_Clear_Status(board,
01662        &my_adc[adc_num],
01663        interrupt_status);
01664
01665        DM35424_Check_Result_test(result, "Error clearing interrupt
01666        status.");
01667
01668        num_ints_processed++;
01669
01670    }
01671
01672    }
01673
01674
01675    if (sample_counts - start_sample_count < TEST_1_ITERATIONS) {
01676        sprintf(message, "Sample count (%d) is less than expected (%d).",
01677        (sample_counts - start_sample_count),
01678        TEST_1_ITERATIONS);
01679    }
01680
01681    DM35424_Check_Result_test(sample_counts - start_sample_count <
01682    TEST_1_ITERATIONS, message);
01683
01684    fprintf(stdout, "ADC%d Chan %d: Last Sample = %d Sample count: %d Ints:
    %d\t\tPASS\n",
01685    adc_num, channel, adc_value, sample_counts,
01686    num_ints_processed);
01687
01688    DM35424_Check_Result_test(exit_program, "User aborted with CTRL-C");
01689
01690    }
01691
01692    result = DM35424_Adc_Interrupt_Set_Config(board,
01693    &my_adc[adc_num],
01694    DM35424_ADC_INT_SAMPLE_TAKEN_MASK,
01695    INTERRUPT_DISABLE);
01696
01697    // Test 1
01698
01699    DM35424_Check_Result_test(result, "Error setting interrupt.");
01700    fprintf(stdout, "ADC%d: Disabling interrupt.\n", adc_num);
01701    result = DM35424_Adc_Reset(board,
01702    &my_adc[adc_num]);
01703
01704    DM35424_Check_Result_test(result, "Error stopping ADC");
01705
01706    interrupts_expected = 0;
01707    fprintf(stdout, "ADC%d: Stopped.\n", adc_num);
01708
01709    fprintf(stdout, "\n");
01710
01711    }
01712
01713    result = DM35424_General_RemoveISR(board);
01714    interrupts_expected = 0;
01715    DM35424_Check_Result_test(result, "Error removing ISR.");
01716    printf("\nClosng Board....");
01717    result = DM35424_Board_Close(board);
01718
01719    DM35424_Check_Result_test(result, "Error closing board.");
01720
01721    }
01722
01723    /*****
01724    TEST 2
01725    *****/
01726

```

```

*****/
01727
01728 void run_test_2(unsigned int minor)
01729 {
01730     int result;
01731     unsigned int dac_num, adc_num;
01732     uint16_t interrupt_status;
01733     unsigned int channel;
01734     unsigned long local_int_count = 0;
01735     int start_sample_count = 0;
01736     char message[200];
01737     int32_t adc_value;
01738     unsigned int num_ints_processed = 0;
01739     unsigned int sample_counts = 0;
01740
01741     int16_t max_dac_value, min_dac_value;
01742     int32_t max_adc_value, min_adc_value;
01743
01744     interrupts_expected = 0;
01745     fprintf(stdout, "\n\n-----\n");
01746     fprintf(stdout, "Test 2: Get Maximum/Minimum Signal out of ADCs. DACs at 1.25 v and 3.75,
alternating\n");
01747
01748     printf("Opening board....");
01749     result = DM35424_Board_Open(minor, &board);
01750
01751     DM35424_Check_Result_test(result, "Could not open board");
01752     printf("success.\nResetting board....");
01753     result = DM35424_Gbc_Board_Reset(board);
01754
01755     DM35424_Check_Result_test(result, "Could not reset board");
01756
01757     open_dacs();
01758
01759     initialize_dacs(0);
01760
01761     open_adcs();
01762
01763     initialize_adcs(TEST_2_ADC_SAMPLE_RATE);
01764
01765     set_isr();
01766
01767
01768     // Test 2
01769     DM35424_Dac_Volts_To_Conv(3.75f,
01770                               &max_dac_value);
01771
01772     DM35424_Dac_Volts_To_Conv(1.25f,
01773                               &min_dac_value);
01774
01775
01776     result = DM35424_Adc_Volts_To_Sample(2.4f,
01777                                           &max_adc_value);
01778
01779     result = DM35424_Adc_Volts_To_Sample(-2.4f,
01780                                           &min_adc_value);
01781
01782     for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD; dac_num++) {
01783
01784
01785         result = DM35424_Dac_Interrupt_Get_Config(board,
01786                                                    &my_dac[dac_num],
01787                                                    &interrupt_status);
01788
01789         DM35424_Check_Result_test(result, "Error getting DAC interrupt status.");
01790
01791         DM35424_Check_Result_test(interrupt_status != 0, "DAC has an interrupt set, and
shouldn't");
01792
01793         for (channel = 0; channel < my_dac[dac_num].num_dma_channels; channel++) {
01794
01795             if (channel % 2 == 0) {
01796                 result = DM35424_Dac_Set_Last_Conversion(board,
01797                                                            &my_dac[dac_num],
01798                                                            channel,
01799                                                            0,
01800                                                            max_dac_value);
01801
01802             }
01803             else {
01804                 result = DM35424_Dac_Set_Last_Conversion(board,
01805                                                            &my_dac[dac_num],
01806                                                            channel,
01807                                                            0,
01808                                                            min_dac_value);
01809

```

```

01810
01811         }
01812
01813         // The DAC requires a short amount of time to propagate last samples,
01814         // so sleep for 10 microsec.
01815         DM35424_Micro_Sleep(10);
01816     }
01817 }
01818 }
01819
01820 // Test 2
01821 for (adc_num = 0; adc_num < DM35424_NUM_ADC_ON_BOARD; adc_num++) {
01822
01823     interrupt_count = 0;
01824     local_int_count = 0;
01825
01826     result = DM35424_Adc_Interrupt_Set_Config(board,
01827                                               &my_adc[adc_num],
01828                                               DM35424_ADC_INT_SAMPLE_TAKEN_MASK,
01829                                               INTERRUPT_ENABLE);
01830
01831     fprintf(stdout, "ADC%d: Setting interrupt enable for sample taken.\n", adc_num);
01832     DM35424_Check_Result_test(result, "Error setting interrupt.");
01833
01834     interrupts_expected = 1;
01835     result = DM35424_Adc_Start(board,
01836                               &my_adc[adc_num]);
01837
01838     DM35424_Check_Result_test(result, "Error starting ADC");
01839     fprintf(stdout, "ADC%d: Started\n", adc_num);
01840
01841
01842     start_timeout_timer(5000);
01843
01844     while (interrupt_count < 1 && !timeout_has_expired()) {}
01845
01846     DM35424_Check_Result_test(interrupt_count == 0, "Timed out waiting for interrupt from
ADC.");
01847
01848     eat_adc_samples(&my_adc[adc_num]);
01849
01850     for (channel = 0; channel < my_adc[adc_num].num_dma_channels; channel++) {
01851         start_sample_count = -1;
01852         num_ints_processed = 0;
01853         while (num_ints_processed <= TEST_2_ITERATIONS && !exit_program) {
01854
01855             if (local_int_count < interrupt_count) {
01856
01857                 // An interrupt has arrived. Make sure it's from this ADC
01858                 result = DM35424_Adc_Interrupt_Get_Status(board,
01859                 &my_adc[adc_num],
01860                 &interrupt_status);
01861
01862                 DM35424_Check_Result_test(result, "Error getting channel with
interrupt.");
01863
01864                 // The first interrupt will be because of sample taken
01865                 // and start trigger (which was immediate), so clear
01866                 // that and then check the next interrupts for just
01867                 // sample taken.
01868
01869                 if (local_int_count == 0) {
01870
01871                     DM35424_Check_Result_test(interrupt_matches_expected(interrupt_status,
01872                     (DM35424_ADC_INT_SAMPLE_TAKEN_MASK |
01873                     DM35424_ADC_INT_START_TRIG_MASK |
01874                     DM35424_ADC_INT_PRE_BUFF_FULL_MASK |
01875                     DM35424_ADC_INT_PACER_TICK_MASK),
01876                     message) == 0,
01877                     message);
01878                 }
01879                 else {
01880
01881                     DM35424_Check_Result_test(interrupt_matches_expected(interrupt_status,
01882                     DM35424_ADC_INT_SAMPLE_TAKEN_MASK |
01883                     DM35424_ADC_INT_PACER_TICK_MASK,
01884                     message) == 0,

```

```

01884                                     message);
01885     }
01886
01887     local_int_count = interrupt_count;
01888
01889     // Test 2
01890
01891     result = DM35424_Adc_Channel_Get_Last_Sample(board,
01892                                     &my_adc[adc_num],
01893                                     channel,
01894                                     &adc_value);
01895
01896     DM35424_Check_Result_test(result, "Error getting ADC value.");
01897
01898     result = DM35424_Adc_Get_Sample_Count(board,
01899                                     &my_adc[adc_num],
01900                                     &sample_counts);
01901
01902     DM35424_Check_Result_test(result, "Error getting sample
01903 counts.");
01904
01905     if (start_sample_count < 0) {
01906         start_sample_count = sample_counts;
01907     }
01908
01909     fprintf(stdout, "ADC%d Chan %d: Last Sample = %d Sample
01910 count: %d Int: %d \r",
01911         adc_num, channel, adc_value, sample_counts,
01912         num_ints_processed);
01913
01914     if (channel % 2 == 0) {
01915         if (adc_value < max_adc_value) {
01916             sprintf(message, "Value (%d) is less than
01917 expected value of %d",
01918                 adc_value,
01919                 max_adc_value);
01920             DM35424_Check_Result_test(1, message);
01921         }
01922     }
01923     else {
01924         if (adc_value > min_adc_value) {
01925             sprintf(message, "Value (%d) is greater than
01926 expected value of %d",
01927                 adc_value,
01928                 min_adc_value);
01929             DM35424_Check_Result_test(1, message);
01930         }
01931     }
01932
01933     result = DM35424_Adc_Interrupt_Clear_Status(board,
01934         &my_adc[adc_num],
01935         interrupt_status);
01936
01937     DM35424_Check_Result_test(result, "Error clearing interrupt
01938 status.");
01939
01940     num_ints_processed ++;
01941 }
01942 }
01943
01944 // Test 2
01945
01946 if (sample_counts - start_sample_count < TEST_2_ITERATIONS) {
01947     sprintf(message, "Sample count (%d) is less than expected (%d).",
01948         (sample_counts - start_sample_count),
01949         TEST_2_ITERATIONS);
01950     DM35424_Check_Result_test(1, message);
01951 }
01952
01953 fprintf(stdout, "ADC%d Chan %d: Last Sample = %d Sample count: %d Int:
01954 %d\t\tPASS\n",
01955     adc_num, channel, adc_value, sample_counts,
01956     num_ints_processed);
01957
01958 DM35424_Check_Result_test(exit_program, "User aborted with CTRL-C");
01959 }
01960 }

```

```

01957         result = DM35424_Adc_Interrupt_Set_Config(board,
01958                                                     &my_adc[adc_num],
01959                                                     DM35424_ADC_INT_SAMPLE_TAKEN_MASK,
01960                                                     INTERRUPT_DISABLE);
01961
01962
01963         DM35424_Check_Result_test(result, "Error setting interrupt.");
01964         fprintf(stdout, "ADC%d: Disabling interrupt for sample taken.\n", adc_num);
01965
01966         result = DM35424_Adc_Reset(board,
01967                                     &my_adc[adc_num]);
01968
01969         DM35424_Check_Result_test(result, "Error starting ADC");
01970         interrupts_expected = 0;
01971         fprintf(stdout, "ADC%d: Stopped.\n", adc_num);
01972
01973
01974         fprintf(stdout, "\n");
01975
01976         DM35424_Check_Result_test(exit_program, "User aborted with CTRL-C");
01977     }
01978
01979     result = DM35424_General_RemoveISR(board);
01980     interrupts_expected = 0;
01981     DM35424_Check_Result_test(result, "Error removing ISR.");
01982
01983     printf("\nClosing Board....");
01984     result = DM35424_Board_Close(board);
01985
01986     DM35424_Check_Result_test(result, "Error closing board.");
01987 }
01988
01989
01990
01991
01992 /*****
01993
01994 TEST 3
01995
01996 *****/
01997 void run_test_3(unsigned int minor)
01998 {
01999     int result;
02000     unsigned int dac_num, adc_num;
02001     uint16_t interrupt_status;
02002     unsigned int channel;
02003     unsigned long local_int_count = 0;
02004     int start_sample_count = 0;
02005     char message[200];
02006     int32_t adc_value;
02007     unsigned int num_ints_processed = 0;
02008     unsigned int sample_counts = 0;
02009
02010     int16_t max_dac_value, min_dac_value;
02011     int32_t max_adc_value, min_adc_value;
02012
02013     interrupts_expected = 0;
02014     fprintf(stdout, "\n\n-----\n");
02015     fprintf(stdout, "Test 2: Get Maximum/Minimum Signal out of ADCs. DACs at 1.25 v and 3.75,
alternating\n");
02016
02017     printf("Opening board....");
02018     result = DM35424_Board_Open(minor, &board);
02019
02020     DM35424_Check_Result_test(result, "Could not open board");
02021     printf("success.\nResetting board....");
02022     result = DM35424_Gbc_Board_Reset(board);
02023
02024     DM35424_Check_Result_test(result, "Could not reset board");
02025
02026     open_dacs();
02027
02028     initialize_dacs(0);
02029
02030     open_adcs();
02031
02032     initialize_adcs(TEST_2_ADC_SAMPLE_RATE);
02033
02034 // Test 3
02035     set_isr();
02036
02037

```

```

02038     DM35424_Dac_Volts_To_Conv(3.75f,
02039                             &max_dac_value);
02040
02041     DM35424_Dac_Volts_To_Conv(1.25f,
02042                             &min_dac_value);
02043
02044
02045     result = DM35424_Adc_Volts_To_Sample(2.4f,
02046                                         &max_adc_value);
02047
02048     result = DM35424_Adc_Volts_To_Sample(-2.4f,
02049                                         &min_adc_value);
02050
02051
02052     for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD; dac_num++) {
02053
02054
02055         result = DM35424_Dac_Interrupt_Get_Config(board,
02056                                                    &my_dac[dac_num],
02057                                                    &interrupt_status);
02058
02059         DM35424_Check_Result_test(result, "Error getting DAC interrupt status.");
02060
02061         DM35424_Check_Result_test(interrupt_status != 0, "DAC has an interrupt set, and
02062 shouldn't");
02063
02064         for (channel = 0; channel < my_dac[dac_num].num_dma_channels; channel++) {
02065
02066             if (channel % 2 == 0) {
02067                 result = DM35424_Dac_Set_Last_Conversion(board,
02068                                                         &my_dac[dac_num],
02069                                                         channel,
02070                                                         0,
02071                                                         min_dac_value);
02072
02073             }
02074             else {
02075                 result = DM35424_Dac_Set_Last_Conversion(board,
02076                                                         &my_dac[dac_num],
02077                                                         channel,
02078                                                         0,
02079                                                         max_dac_value);
02080
02081             }
02082
02083             // The DAC requires a short amount of time to propagate last samples,
02084             // so sleep for 10 microsec.
02085             DM35424_Micro_Sleep(10);
02086         }
02087     }
02088
02089     // Test 3
02090     for (adc_num = 0; adc_num < DM35424_NUM_ADC_ON_BOARD; adc_num++) {
02091
02092
02093         interrupt_count = 0;
02094         local_int_count = 0;
02095
02096         result = DM35424_Adc_Interrupt_Set_Config(board,
02097                                                    &my_adc[adc_num],
02098                                                    DM35424_ADC_INT_SAMPLE_TAKEN_MASK,
02099                                                    INTERRUPT_ENABLE);
02100
02101         fprintf(stdout, "ADC%d: Setting interrupt enable for sample taken.\n", adc_num);
02102         DM35424_Check_Result_test(result, "Error setting interrupt.");
02103
02104         interrupts_expected = 1;
02105         result = DM35424_Adc_Start(board,
02106                                   &my_adc[adc_num]);
02107
02108         DM35424_Check_Result_test(result, "Error starting ADC");
02109         fprintf(stdout, "ADC%d: Started\n", adc_num);
02110
02111
02112         start_timeout_timer(5000);
02113
02114         while (interrupt_count < 1 && !timeout_has_expired()) {}
02115
02116         DM35424_Check_Result_test(interrupt_count == 0, "Timed out waiting for interrupt from
02117 ADC.");
02118
02119         eat_adc_samples(&my_adc[adc_num]);
02120
02121         for (channel = 0; channel < my_adc[adc_num].num_dma_channels; channel++) {
02122             start_sample_count = -1;

```

```

02122         num_ints_processed = 0;
02123         while (num_ints_processed <= TEST_2_ITERATIONS && !exit_program) {
02124
02125             if (local_int_count < interrupt_count) {
02126
02127                 // An interrupt has arrived. Make sure it's from this ADC
02128                 result = DM35424_Adc_Interrupt_Get_Status(board,
02129
02130                     &my_adc[adc_num],
02131                     &interrupt_status);
02132
02133                 DM35424_Check_Result_test(result, "Error getting channel with
02134 interrupt.");
02135
02136                 // The first interrupt will be because of sample taken
02137                 // and start trigger (which was immediate), so clear
02138                 // that and then check the next interrupts for just
02139                 // sample taken.
02140
02141                 // Test 3
02142                 if (local_int_count == 0) {
02143                     DM35424_Check_Result_test(interrupt_matches_expected(interrupt_status,
02144
02145                         (DM35424_ADC_INT_SAMPLE_TAKEN_MASK |
02146
02147                         DM35424_ADC_INT_START_TRIG_MASK |
02148
02149                         DM35424_ADC_INT_PRE_BUFF_FULL_MASK |
02150
02151                         DM35424_ADC_INT_PACER_TICK_MASK),
02152
02153                         message) == 0,
02154                         message);
02155                     }
02156                     else {
02157                         DM35424_Check_Result_test(interrupt_matches_expected(interrupt_status,
02158
02159                         DM35424_ADC_INT_SAMPLE_TAKEN_MASK |
02160
02161                         DM35424_ADC_INT_PACER_TICK_MASK,
02162
02163                         message) == 0,
02164                         message);
02165                     }
02166                     local_int_count = interrupt_count;
02167                     result = DM35424_Adc_Channel_Get_Last_Sample(board,
02168
02169                         &my_adc[adc_num],
02170                         channel,
02171                         &adc_value);
02172                     DM35424_Check_Result_test(result, "Error getting ADC value.");
02173                     result = DM35424_Adc_Get_Sample_Count(board,
02174
02175                         &my_adc[adc_num],
02176                         &sample_counts);
02177                     DM35424_Check_Result_test(result, "Error getting sample
02178 counts.");
02179                     if (start_sample_count < 0) {
02180                         start_sample_count = sample_counts;
02181                     }
02182                     fprintf(stdout, "ADC%d Chan %d: Last Sample = %d Sample
02183 count: %d Int: %d \r",
02184                         adc_num, channel, adc_value, sample_counts,
02185                         num_ints_processed);
02186                     if (channel % 2 == 0) {
02187                         if (adc_value > min_adc_value) {
02188                             sprintf(message, "Value (%d) is greater than
02189                                 expected value of %d",
02190                                     adc_value,
02191                                     min_adc_value);
02192                             DM35424_Check_Result_test(1, message);
02193                         }
02194                     }
02195                     else {
02196                         if (adc_value < max_adc_value) {
02197                             sprintf(message, "Value (%d) is less than
02198                                 expected value of %d",

```

```

02191                                     adc_value,
max_adc_value);
02192                                     DM35424_Check_Result_test(1, message);
02193                                     }
02194                                     }
02195                                     }
02196                                     }
// Test 3
02197                                     result = DM35424_Adc_Interrupt_Clear_Status(board,
02198                                     &my_adc[adc_num],
02199                                     interrupt_status);
02200
02201                                     DM35424_Check_Result_test(result, "Error clearing interrupt
02202                                     status.");
02203                                     num_ints_processed++;
02204                                     }
02205                                     }
02206                                     }
02207                                     }
02208                                     }
02209                                     }
02210                                     }
02211                                     if (sample_counts - start_sample_count < TEST_2_ITERATIONS) {
02212                                     sprintf(message, "Sample count (%d) is less than expected (%d).",
02213                                     (sample_counts - start_sample_count),
02214                                     TEST_2_ITERATIONS);
02215                                     DM35424_Check_Result_test(1, message);
02216                                     }
02217                                     }
02218                                     fprintf(stdout, "ADC%d Chan %d: Last Sample = %d Sample count: %d Int:
02219                                     %d\t\tPASS\n",
02220                                     num_ints_processed);
02221                                     }
02222                                     DM35424_Check_Result_test(exit_program, "User aborted with CTRL-C");
02223                                     }
02224                                     }
02225                                     }
02226                                     }
02227                                     result = DM35424_Adc_Interrupt_Set_Config(board,
02228                                     &my_adc[adc_num],
02229                                     DM35424_ADC_INT_SAMPLE_TAKEN_MASK,
02230                                     INTERRUPT_DISABLE);
02231                                     }
02232                                     DM35424_Check_Result_test(result, "Error setting interrupt.");
02233                                     fprintf(stdout, "ADC%d: Disabling interrupt for sample taken.\n", adc_num);
02234                                     }
// Test 3
02235                                     result = DM35424_Adc_Reset(board,
02236                                     &my_adc[adc_num]);
02237                                     }
02238                                     DM35424_Check_Result_test(result, "Error starting ADC");
02239                                     }
02240                                     }
02241                                     interrupts_expected = 0;
02242                                     fprintf(stdout, "ADC%d: Stopped.\n", adc_num);
02243                                     }
02244                                     }
02245                                     }
02246                                     }
02247                                     DM35424_Check_Result_test(exit_program, "User aborted with CTRL-C");
02248                                     }
02249                                     }
02250                                     result = DM35424_General_RemoveISR(board);
02251                                     interrupts_expected = 0;
02252                                     DM35424_Check_Result_test(result, "Error removing ISR.");
02253                                     }
02254                                     printf("\nClosing Board....");
02255                                     result = DM35424_Board_Close(board);
02256                                     }
02257                                     DM35424_Check_Result_test(result, "Error closing board.");
02258                                     }
02259                                     }
02260                                     }
02261                                     }
02262                                     }
02263                                     }
02264                                     }
/*****
02265                                     *****/
02266                                     TEST 4
02267                                     */

```



```

02268 *****
02269 *****/
02270 void run_test_4(unsigned int minor)
02271 {
02272     int result;
02273     unsigned int adc_num, channel;
02274     int32_t adc_high_value, adc_low_value, adc_high_threshold;
02275     interrupts_expected = 1;
02276     unsigned int current_count = 0;
02277     unsigned int local_int_count = 0;
02278     unsigned int num_ints_processed = 0;
02279     uint16_t interrupt_status;
02280     char message[200];
02281     int32_t adc_value;
02282     int32_t chan0_value;
02283
02284     fprintf(stdout, "\n\n-----\n");
02285     fprintf(stdout, "Test 4: Get sine wave out of ADCs. DACs executing sine wave data via
DMA\n");
02286
02287     printf("Opening board....");
02288     result = DM35424_Board_Open(minor, &board);
02289
02290     DM35424_Check_Result_test(result, "Could not open board");
02291     printf("success.\nResetting board....");
02292     result = DM35424_Gbc_Board_Reset(board);
02293
02294     DM35424_Check_Result_test(result, "Could not reset board");
02295
02296     // Test 4
02297     open_adcs();
02298
02299     initialize_adcs(TEST_4_ADC_SAMPLE_RATE);
02300
02301     set_isr();
02302
02303     open_dacs();
02304
02305     initialize_dacs(TEST_4_DAC_CONVERSION_RATE);
02306
02307     start_dacs_with_wave(DM35424_SINE_WAVE);
02308
02309     for (adc_num = 0; adc_num < DM35424_NUM_ADC_ON_BOARD; adc_num++) {
02310
02311         result = DM35424_Adc_Start(board,
                                &my_adc[adc_num]);
02312
02313         DM35424_Check_Result_test(result, "Error starting ADC");
02314
02315         eat_adc_samples(&my_adc[adc_num]);
02316
02317         result = DM35424_Adc_Pause(board,
                                &my_adc[adc_num]);
02318
02319         DM35424_Check_Result_test(result, "Error pausing ADC");
02320
02321     }
02322
02323     // Test 4
02324
02325     DM35424_Adc_Volts_To_Sample(TEST_4_ADC_LOW_VOLTAGE,
                                &adc_low_value);
02326
02327     DM35424_Adc_Volts_To_Sample(TEST_4_ADC_HIGH_VOLTAGE,
                                &adc_high_value);
02328
02329     DM35424_Adc_Volts_To_Sample(TEST_4_ADC_HIGH_THRESH,
                                &adc_high_threshold);
02330
02331     fprintf(stdout, "\nADC      Chan0      Chan1      Chan2      Chan3      Chan4      Chan5      Chan6
Chan7\n");
02332     fprintf(stdout, "---      ---      ---      ---      ---      ---      ---      ---
\n");
02333
02334     for (adc_num = 0; adc_num < DM35424_NUM_ADC_ON_BOARD; adc_num++) {
02335         current_count = 0;
02336         interrupt_count = 0;
02337         local_int_count = 0;
02338
02339         result = DM35424_Adc_Interrupt_Set_Config(board,
                                &my_adc[adc_num],

```

```

02348                                     DM35424_ADC_INT_SAMPLE_TAKEN_MASK,
02349                                     INTERRUPT_ENABLE);
02350
02351     DM35424_Check_Result_test(result, "Error setting interrupt.");
02352
02353     interrupts_expected = 1;
02354     result = DM35424_Adc_Start(board,
02355                                &my_adc[adc_num]);
02356
02357 // Test 4
02358
02359     DM35424_Check_Result_test(result, "Error starting ADC");
02360
02361     start_timeout_timer(5000);
02362
02363     while (interrupt_count < 1 && !timeout_has_expired()) {}
02364
02365     //DM35424_Check_Result_test(interrupt_count == 0, "Timed out waiting for interrupt
02366 from ADC.");
02367
02368     num_ints_processed = 0;
02369     while (num_ints_processed < TEST_4_ITERATIONS && !exit_program) {
02370         if (local_int_count < interrupt_count) {
02371             fprintf(stdout, "%3d ", adc_num);
02372
02373             // An interrupt has arrived. Make sure it's from this ADC
02374             result = DM35424_Adc_Interrupt_Get_Status(board,
02375                                                         &my_adc[adc_num],
02376                                                         &interrupt_status);
02377
02378             DM35424_Check_Result_test(result, "Error getting channel with
02379 interrupt.");
02380
02381             // The first interrupt will be because of sample taken
02382             // and start trigger (which was immediate), so clear
02383             // that and then check the next interrupts for just
02384             // sample taken.
02385
02386 // Test 4
02387
02388             if (local_int_count == 0) {
02389                 DM35424_Check_Result_test(interrupt_matches_expected(interrupt_status,
02390 (DM35424_ADC_INT_SAMPLE_TAKEN_MASK |
02391 DM35424_ADC_INT_START_TRIG_MASK |
02392 DM35424_ADC_INT_PRE_BUFF_FULL_MASK |
02393 DM35424_ADC_INT_PACER_TICK_MASK),
02394                                     message) == 0,
02395                                     message);
02396             }
02397             else {
02398                 DM35424_Check_Result_test(interrupt_matches_expected(interrupt_status,
02399 DM35424_ADC_INT_SAMPLE_TAKEN_MASK |
02400 DM35424_ADC_INT_PACER_TICK_MASK,
02401                                     message) == 0,
02402                                     message);
02403             }
02404
02405             local_int_count = interrupt_count;
02406
02407             result = DM35424_Adc_Interrupt_Clear_Status(board,
02408                                                         &my_adc[adc_num],
02409                                                         interrupt_status);
02410
02411             DM35424_Check_Result_test(result, "Error clearing interrupt status.");
02412
02413 // Test 4
02414
02415             for (channel = 0; channel < my_adc[adc_num].num_dma_channels; channel
02416 +++) {
02417                 result = DM35424_Adc_Channel_Get_Last_Sample(board,
02418                                                         &my_adc[adc_num],
02419                                                         channel,
02420                                                         &adc_value);
02421
02422                 DM35424_Check_Result_test(result, "Error getting ADC value.");
02423

```

```

02421         fprintf(stdout, "%8d ", adc_value);
02422
02423         if (channel == 0) {
02424             chan0_value = adc_value;
02425         }
02426         else {
02427             if (channel % 2 == 0) {
02428                 if (chan0_value > adc_high_threshold) {
02429                     sprintf(message, "\nValue (%d) is less
than expected %d.",
02430                             adc_value, adc_high_value);
02431                     DM35424_Check_Result_test(adc_value <
adc_high_value, message);
02432                     current_count ++;
02433                 }
02434                 if (chan0_value < -1 * adc_high_threshold) {
02435                     sprintf(message, "\nValue (%d) is more
than expected %d.",
02436                             adc_value, adc_low_value);
02437                     DM35424_Check_Result_test(adc_value >
adc_low_value, message);
02438                     current_count ++;
02439                 }
02440             }
02441             else {
02442                 if (chan0_value > adc_high_threshold) {
02443                     // We'd expect these channel values to
be opposite the chan 0
02444                     sprintf(message, "\nValue (%d) is more
than expected %d.",
02445                             adc_value, adc_low_value);
02446                     DM35424_Check_Result_test(adc_value >
adc_low_value, message);
02447                     current_count ++;
02448                 }
02449                 if (chan0_value < -1 * adc_high_threshold) {
02450                     sprintf(message, "\nValue (%d) is less
than expected %d.",
02451                             adc_value, adc_high_value);
02452                     DM35424_Check_Result_test(adc_value <
adc_high_value, message);
02453                     current_count ++;
02454                 }
02455             }
02456         }
02457     }
02458 }
02459 }
02460
02461 // Test 4
02462         num_ints_processed ++;
02463     }
02464     fprintf(stdout, "\r");
02465     DM35424_Check_Result_test(exit_program, "User aborted with CTRL-C");
02466 }
02467
02468 result = DM35424_Adc_Interrupt_Set_Config(board,
02469                                           &my_adc[adc_num],
02470                                           DM35424_ADC_INT_SAMPLE_TAKEN_MASK,
02471                                           INTERRUPT_DISABLE);
02472
02473 DM35424_Check_Result_test(result, "Error setting interrupt.");
02474
02475 result = DM35424_Adc_Reset(board,
02476                             &my_adc[adc_num]);
02477 DM35424_Micro_Sleep(1000);
02478
02479 interrupts_expected = 0;
02480 DM35424_Check_Result_test(result, "Error starting ADC");
02481 fprintf(stdout, "\n");
02482 DM35424_Check_Result_test(exit_program, "User aborted with CTRL-C");
02483
02484 // DM35424_Check_Result_test(current_count < my_adc[adc_num].num_dma_channels, "Values
did not achieve their expected range.");
02485 }
02486
02487 fprintf(stdout, "PASSED!\n");
02488 stop_all_dacs_with_dma();
02489
02490 result = DM35424_General_RemoveISR(board);
02491 interrupts_expected = 0;
02492
02493 DM35424_Check_Result_test(result, "Error removing ISR.");

```

```

02497
02498     printf("\nClosing Board...");
02499     result = DM35424_Board_Close(board);
02500
02501 // Test 4
02502     DM35424_Check_Result_test(result, "Error closing board.");
02503
02504
02505 }
02506
02507
02508
02509 /*****
02510
02511
02512
02513
02514 void run_test_5(unsigned int minor)
02515 {
02516     int result;
02517     unsigned int dac_num;
02518     unsigned int local_int_count = 0;
02519     uint16_t interrupt_status;
02520     char message[200];
02521
02522     interrupts_expected = 0;
02523     fprintf(stdout, "\n\n-----\n");
02524     fprintf(stdout, "Test 5: Testing DAC Interrupts.\n\n");
02525
02526     printf("Opening board....");
02527     result = DM35424_Board_Open(minor, &board);
02528
02529     DM35424_Check_Result_test(result, "Could not open board");
02530     printf("success.\nResetting board....");
02531     result = DM35424_Gbc_Board_Reset(board);
02532
02533     DM35424_Check_Result_test(result, "Could not reset board");
02534
02535     fprintf(stdout, "success.\n");
02536
02537     open_dacs();
02538
02539     initialize_dacs(TEST_5_DAC_CONVERSION_RATE);
02540
02541     set_isr();
02542
02543     interrupt_count = 0;
02544
02545     load_dacs_with_wave(DM35424_SQUARE_WAVE);
02546
02547 // Test 5
02548     for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD; dac_num++) {
02549         interrupt_count = 0;
02550         local_int_count = 0;
02551         /*****
02552         *   START TRIGGER INTERRUPT
02553         *****/
02554         fprintf(stdout, "\nDAC%d: Testing Start Trigger Interrupt...", dac_num);
02555         write_to_kernel_log("Test 5: Start trigger interrupt test.");
02556         result = DM35424_Dac_Interrupt_Set_Config(board,
02557             &my_dac[dac_num],
02558             DM35424_DAC_INT_START_TRIG_MASK,
02559             INTERRUPT_ENABLE);
02560
02561         DM35424_Check_Result_test(result, "Error enabling interrupts.");
02562
02563         result = DM35424_Dma_Start(board,
02564             &my_dac[dac_num],
02565             DMA_CHAN0);
02566
02567         DM35424_Check_Result_test(result, "Error starting DMA.");
02568
02569         result = DM35424_Dac_Set_Start_Trigger(board,
02570             &my_dac[dac_num],
02571             DM35424_CLK_SRC_IMMEDIATE);
02572
02573         DM35424_Check_Result_test(result, "Error setting start trigger for DAC.");
02574
02575         result = DM35424_Dac_Set_Stop_Trigger(board,
02576             &my_dac[dac_num],

```

```

02578                                     DM35424_CLK_SRC_NEVER);
02579
02580         DM35424_Check_Result_test(result, "Error setting stop trigger for DAC.");
02581
02582     // Test 5
02583
02584         interrupts_expected = 1;
02585
02586         write_to_kernel_log("Test 5: Starting DAC.");
02587         result = DM35424_Dac_Start(board,
02588                                     &my_dac[dac_num]);
02589
02590         DM35424_Check_Result_test(result, "Error starting DAC");
02591
02592         start_timeout_timer(ONE_SEC_IN_MILLI);
02593
02594         while (!timeout_has_expired() && !exit_program) {
02595             if (local_int_count < interrupt_count) {
02596                 result = DM35424_Dac_Interrupt_Get_Status(board,
02597                                                             &my_dac[dac_num],
02598                                                             &interrupt_status);
02599
02600                 DM35424_Check_Result_test(result, "Error getting interrupt status");
02601
02602                 if (local_int_count == 0) {
02603                     DM35424_Check_Result_test(interrupt_matches_expected(interrupt_status,
02604                                     (DM35424_DAC_INT_CONVERSION_SENT_MASK |
02605                                     DM35424_DAC_INT_START_TRIG_MASK |
02606                                     DM35424_DAC_INT_PACER_TICK_MASK),
02607                                     message) == 0,
02608                                     message);
02609
02610                     /*****
02611                      * CONVERSION SENT INTERRUPT
02612                      *****/
02613                     /*****
02614                     fprintf(stdout, "success.\nDAC%d: Testing Conversion Sent
02615                     Interrupt...", dac_num);
02616                     result = DM35424_Dac_Interrupt_Set_Config(board,
02617                                                             &my_dac[dac_num],
02618                                                             DM35424_DAC_INT_START_TRIG_MASK,
02619                                                             INTERRUPT_DISABLE);
02620
02621                     DM35424_Check_Result_test(result, "Error enabling
02622                     interrupts.");
02623
02624                     result = DM35424_Dac_Interrupt_Clear_Status(board,
02625                                                             &my_dac[dac_num],
02626                                                             interrupt_status);
02627
02628                     DM35424_Check_Result_test(result, "Error clearing interrupt
02629                     status.");
02630
02631                     // Test 5
02632
02633                     local_int_count = interrupt_count;
02634                     result = DM35424_Dac_Interrupt_Set_Config(board,
02635                                                             &my_dac[dac_num],
02636                                                             DM35424_DAC_INT_CONVERSION_SENT_MASK,
02637                                                             INTERRUPT_ENABLE);
02638
02639                     DM35424_Check_Result_test(result, "Error enabling
02640                     interrupts.");
02641
02642                     }
02643                     else {
02644                         DM35424_Check_Result_test(interrupt_matches_expected(interrupt_status,
02645                                     (DM35424_DAC_INT_CONVERSION_SENT_MASK |
02646                                     DM35424_DAC_INT_PACER_TICK_MASK,
02647                                     message) == 0,
02648                                     message);
02649
02650                         local_int_count = interrupt_count;

```

```

02646
02647         result = DM35424_Dac_Interrupt_Clear_Status(board,
02648         &my_dac[dac_num],
02649         interrupt_status);
02650
02651         DM35424_Check_Result_test(result, "Error clearing interrupt
status.");
02652     }
02653 }
02654 }
02655 }
02656 }
02657 }
02658
// Test 5
02659
02660     // Check we received the number of interrupts we'd expect. At 100 Hz rate,
02661     // we should get close to 100 in 1 second of waiting
02662     DM35424_Check_Result_test(interrupt_count == 0, "Timed out waiting for interrupt from
DAC.");
02663     sprintf(message, "Did not receive expected number of interrupts. Expected 100,
received %d.", interrupt_count);
02664     DM35424_Check_Result_test(!((interrupt_count > 95) && (interrupt_count < 105)),
message);
02665
02666     fprintf(stdout, "success.\n");
02667
02668     result = DM35424_Dac_Interrupt_Set_Config(board,
02669         &my_dac[dac_num],
02670         DM35424_DAC_INT_CONVERSION_SENT_MASK,
02671         INTERRUPT_DISABLE);
02672
02673     DM35424_Check_Result_test(result, "Error disabling interrupts.");
02674
02675     /* Setting up for Stop trigger and post stop conversions interrupts. We're */
02676     /* going to set the stop trigger to be off a global clock tied to conversions */
02677     /* made. That will remove some of the ambiguity on what the final interrupt */
02678     /* status register might be. */
02679     result = DM35424_Dac_Set_Clock_Source_Global(board,
02680         &my_dac[dac_num],
02681         DM35424_CLK_SRC_GBL2,
02682         DM35424_DAC_CLK_EVENT_CONVERSION_SENT);
02683
02684     DM35424_Check_Result_test(result, "Error setting clock global source");
02685
// Test 5
02686
02687     fprintf(stdout, "DAC%d: Testing Stop Trigger Interrupt...", dac_num);
02688
02689     /* *****
02690     * STOP TRIGGER INTERRUPT
02691     * ***** */
02692     result = DM35424_Dac_Interrupt_Set_Config(board,
02693         &my_dac[dac_num],
02694         DM35424_DAC_INT_STOP_TRIG_MASK,
02695         INTERRUPT_ENABLE);
02696
02697     DM35424_Check_Result_test(result, "Error enabling stop trigger interrupt.");
02698
02699     /* Set the post conversion count to be completed before 1 second */
02700     result = DM35424_Dac_Set_Post_Stop_Conversion_Count(board,
02701         &my_dac[dac_num],
02702         (TEST_5_DAC_CONVERSION_RATE *
9) / 10);
02703
02704     DM35424_Check_Result_test(result, "Error setting post conversion count.");
02705
02706     local_int_count = 0;
02707     interrupt_count = 0;
02708     interrupt_status = 0;
02709
02710     result = DM35424_Dac_Set_Stop_Trigger(board,
02711         &my_dac[dac_num],
02712         DM35424_CLK_SRC_GBL2);
02713
02714     DM35424_Check_Result_test(result, "Error enabling stop trigger.");
02715
// Test 5
02716
02717     start_timeout_timer(ONE_SEC_IN_MILLI);
02718
02719

```

```

02723         while (!timeout_has_expired() && !exit_program) {
02724
02725             if (local_int_count < interrupt_count) {
02726
02727                 result = DM35424_Dac_Interrupt_Get_Status(board,
02728                                                             &my_dac[dac_num],
02729                                                             &interrupt_status);
02730
02731                 DM35424_Check_Result_test(result, "Error getting interrupt status");
02732
02733                 if (local_int_count == 0) {
02734
02735                     DM35424_Check_Result_test(interrupt_matches_expected(interrupt_status,
02736                                     (DM35424_DAC_INT_CONVERSION_SENT_MASK |
02737                                     DM35424_DAC_INT_STOP_TRIG_MASK |
02738                                     DM35424_DAC_INT_PACER_TICK_MASK),
02739                                     message) == 0,
02740                                     message);
02741
02742                     fprintf(stdout, "success.\nDAC%d: Testing Post Stop
02743 Conversions Interrupt....", dac_num);
02744                     result = DM35424_Dac_Interrupt_Set_Config(board,
02745                                                             &my_dac[dac_num],
02746                                                             DM35424_DAC_INT_STOP_TRIG_MASK,
02747                                                             INTERRUPT_DISABLE);
02748                     DM35424_Check_Result_test(result, "Error enabling
02749 interrupts.");
02750                     result = DM35424_Dac_Interrupt_Clear_Status(board,
02751                                                             &my_dac[dac_num],
02752                                                             interrupt_status);
02753                     DM35424_Check_Result_test(result, "Error clearing interrupt
02754 status.");
02755                     local_int_count = interrupt_count;
02756
02757                     // Test 5
02758                     result = DM35424_Dac_Interrupt_Set_Config(board,
02759                                                             &my_dac[dac_num],
02760                                                             DM35424_DAC_INT_POST_STOP_DONE_MASK,
02761                                                             INTERRUPT_ENABLE);
02762                     DM35424_Check_Result_test(result, "Error disabling
02763 interrupts.");
02764                     }
02765                     else {
02766
02767                         /*****
02768                          * POST STOP CONVERSIONS COMPLETED INTERRUPT
02769                          *****/
02770
02771                         DM35424_Check_Result_test(interrupt_matches_expected(interrupt_status,
02772                                     (DM35424_DAC_INT_CONVERSION_SENT_MASK |
02773                                     DM35424_DAC_INT_POST_STOP_DONE_MASK |
02774                                     DM35424_DAC_INT_PACER_TICK_MASK,
02775                                     message) == 0,
02776                                     message);
02777
02778                         fprintf(stdout, "success.\n");
02779                         local_int_count = interrupt_count;
02780                         result = DM35424_Dac_Interrupt_Set_Config(board,
02781                                                             &my_dac[dac_num],
02782                                                             DM35424_DAC_INT_POST_STOP_DONE_MASK,
02783                                                             INTERRUPT_DISABLE);
02784                         DM35424_Check_Result_test(result, "Error disabling
02785 interrupts.");
02786
02787                         // Test 5
02788                         result = DM35424_Dac_Interrupt_Clear_Status(board,

```

```

02788     &my_dac[dac_num],
02789     interrupt_status);
02790
02791     DM35424_Check_Result_test(result, "Error clearing interrupt
status.");
02792     }
02793
02794     }
02795     }
02796
02797     }
02798
02799
02800     interrupts_expected = 0;
02801
02802     DM35424_Check_Result_test(result, "Error getting interrupt status");
02803     // Check that we received no interrupts, since DMA was not running
02804     sprintf(message, "Expected 2 interrupts (Stop and Post Stop), but received %d
interrupts instead.",
02805             interrupt_count);
02806
02807     DM35424_Check_Result_test(interrupt_count != 2, message);
02808
02809     fprintf(stdout, "DAC%d: Verifying no interrupts received when none enabled....",
dac_num);
02810     interrupt_count = 0;
02811     start_timeout_timer(FIVE_SEC_IN_MILLI);
02812
02813     while (interrupt_count < 1 && !timeout_has_expired()) {}
02814
02815     DM35424_Check_Result_test(interrupt_count != 0, "Expected no interrupts, but received
some.");
02816
02817
02818     // Test 5
02819     result = DM35424_Dac_Set_Clock_Source_Global(board,
02820             &my_dac[dac_num],
02821             DM35424_CLK_SRC_GBL2,
02822             DM35424_DAC_CLK_EVENT_DISABLE);
02823
02824     DM35424_Check_Result_test(result, "Error setting clock global source");
02825
02826     result = DM35424_Dac_Reset(board,
02827             &my_dac[dac_num]);
02828
02829     DM35424_Check_Result_test(result, "Error stopping DAC");
02830     fprintf(stdout, "success.\nDAC%d: Interrupt testing complete.\n", dac_num);
02831     DM35424_Check_Result_test(exit_program, "User aborted with CTRL-C");
02832
02833     }
02834
02835     result = DM35424_General_RemoveISR(board);
02836     interrupts_expected = 0;
02837
02838     DM35424_Check_Result_test(result, "Error removing ISR.");
02839
02840     printf("\n\nClosing Board...");
02841     result = DM35424_Board_Close(board);
02842
02843     DM35424_Check_Result_test(result, "Error closing board.");
02844 }
02845
02846
02847
02848
02849
02850     TEST 6
02851
02852
02853
02854 void run_test_6(unsigned int minor)
02855 {
02856     int result;
02857     unsigned int adc_num, channel;
02858     unsigned int local_int_count = 0;
02859     uint16_t interrupt_status;
02860     char message[200];
02861     int32_t adc_value, adc_low_thresh, adc_high_thresh;
02862     uint8_t mode_status;
02863     unsigned int low_thresh_count, high_thresh_count;

```



```

02864     unsigned int channel_with_interrupt;
02865     int channel_has_interrupt = 0;
02866     uint8_t uint8_value;
02867     uint8_t chan_interrupt_status;
02868
02869     interrupts_expected = 0;
02870
02871     fprintf(stdout, "\n\n-----\n");
02872     fprintf(stdout, "Test 6: Testing ADC Interrupts and Mode/Status.\n\n");
02873
02874     printf("Opening board....");
02875     result = DM35424_Board_Open(minor, &board);
02876
02877     DM35424_Check_Result_test(result, "Could not open board");
02878     printf("success.\nResetting board....");
02879     result = DM35424_Gbc_Board_Reset(board);
02880
02881     DM35424_Check_Result_test(result, "Could not reset board");
02882
02883     fprintf(stdout, "success.\n");
02884
02885     open_dacs();
02886
02887     initialize_dacs(TEST_6_DAC_CONVERSION_RATE);
02888     start_dacs_with_wave(DM35424_SINE_WAVE);
02889
02890     open_adcs();
02891
02892 // Test 6
02893 for (adc_num = 0; adc_num < DM35424_NUM_ADC_ON_BOARD; adc_num++) {
02894     fprintf(stdout, "ADC%d: Checking mode is Uninitialized....", adc_num);
02895
02896     result = DM35424_Adc_Get_Mode_Status(board,
02897                                         &my_adc[adc_num],
02898                                         &mode_status);
02899
02900     DM35424_Check_Result_test(result, "Error getting mode status");
02901
02902     sprintf(message, "Expected status:mode of UNINITIALIZED/UNINITIALIZED, but received
02903 0x%x.",
02904             mode_status);
02905     DM35424_Check_Result_test(mode_status !=
02906 MODE_STATUS(DM35424_ADC_MODE_UNINITIALIZED,DM35424_ADC_STAT_UNINITIALIZED),
02907 message);
02908
02909     fprintf(stdout, "success.\n");
02910 }
02911 initialize_adcs(TEST_6_ADC_SAMPLE_RATE);
02912 for (adc_num = 0; adc_num < DM35424_NUM_ADC_ON_BOARD; adc_num++) {
02913     fprintf(stdout, "ADC%d: Checking status:mode is Stopped:reset....", adc_num);
02914
02915     result = DM35424_Adc_Get_Mode_Status(board,
02916                                         &my_adc[adc_num],
02917                                         &mode_status);
02918
02919     DM35424_Check_Result_test(result, "Error getting mode status");
02920
02921     sprintf(message, "Expected status:mode of STOPPED:RESET, but received 0x%x.",
02922             mode_status);
02923     DM35424_Check_Result_test(mode_status !=
02924 MODE_STATUS(DM35424_ADC_MODE_RESET,DM35424_ADC_STAT_STOPPED),
02925 message);
02926
02927     fprintf(stdout, "success.\n");
02928 }
02929 set_isr();
02930 interrupt_count = 0;
02931
02932 // Test 6
02933 /* We're going to start the ADC based on the clock global source 2,
02934  * which will be set to the conversion sent event on DAC0 */
02935
02936 result = DM35424_Dac_Set_Clock_Source_Global(board,
02937                                             &my_dac[DAC0],
02938                                             DM35424_CLK_SRC_GBL2,
02939                                             DM35424_DAC_CLK_EVENT_CONVERSION_SENT);
02940
02941 DM35424_Check_Result_test(result, "Error setting clock global source");
02942
02943 write_to_kernel_log("Test 6: Initialization Complete.");
02944 for (adc_num = 0; adc_num < DM35424_NUM_ADC_ON_BOARD; adc_num++) {
02945

```

```

02946
02947     fprintf(stdout, "\nADC%d: Starting interrupt/mode test.\n", adc_num);
02948
02949     initialize_dma_buffers(&my_adc[adc_num],
02950                           BUFFER_BYTES_SIZE,
02951                           DM35424_DMA_SETUP_DIRECTION_READ);
02952
02953     /* Set the start trigger to never so we can capture
02954      * the pre-start buffer filled interrupt
02955      */
02956     result = DM35424_Adc_Set_Start_Trigger(board,
02957                                             &my_adc[adc_num],
02958                                             DM35424_CLK_SRC_NEVER);
02959
02960     DM35424_Check_Result_test(result, "Error setting ADC start trigger.");
02961
02962     /* Give the pre-trigger capture size */
02963     result = DM35424_Adc_Set_Pre_Trigger_Samples(board,
02964                                                  &my_adc[adc_num],
02965                                                  TEST6_ADC_PRE_TRIGGER_SAMP_SIZE);
02966
02967     DM35424_Check_Result_test(result, "Error setting pre-capture sample size.");
02968
02969     /* Give the post-trigger capture size */
02970     result = DM35424_Adc_Set_Post_Stop_Samples(board,
02971                                                &my_adc[adc_num],
02972                                                TEST6_ADC_POST_STOP_SAMP_SIZE);
02973
02974     DM35424_Check_Result_test(result, "Error setting pre-capture sample size.");
02975
02976 // Test 6
02977
02978     interrupt_count = 0;
02979     local_int_count = 0;
02980     fprintf(stdout, "ADC%d: Testing Pre-Trigger Buffer Filled Interrupt...", adc_num);
02981
02982     result = DM35424_Adc_Interrupt_Set_Config(board,
02983                                              &my_adc[adc_num],
02984                                              DM35424_ADC_INT_PRE_BUFF_FULL_MASK,
02985                                              INTERRUPT_ENABLE);
02986
02987     DM35424_Check_Result_test(result, "Error enabling interrupts.");
02988
02989     for (channel = 0; channel < my_adc[adc_num].num_dma_channels; channel++) {
02990         result = DM35424_Dma_Start(board,
02991                                   &my_adc[adc_num],
02992                                   channel);
02993
02994         DM35424_Check_Result_test(result, "Error starting ADC DMA.");
02995     }
02996
02997     write_to_kernel_log("Test 6: Checking for Pre-Trigger Buffer Filled interrupt.");
02998
02999     interrupts_expected = 1;
03000     /* Start the ADC (but not Start Trigger), which should give us the
03001      * buffer filled interrupt once it has pre-trigger amount. That should
03002      * happen within 1 sec.
03003      */
03004     result = DM35424_Adc_Start(board,
03005                               &my_adc[adc_num]);
03006
03007     DM35424_Check_Result_test(result, "Error starting ADC.");
03008
03009     result = DM35424_Adc_Get_Mode_Status(board,
03010                                         &my_adc[adc_num],
03011                                         &mode_status);
03012
03013     DM35424_Check_Result_test(result, "Error getting mode status");
03014
03015     sprintf(message, "Expected status:mode of PRE-TRIGGER:GO, but received 0x%x.",
03016            mode_status);
03017     DM35424_Check_Result_test(mode_status !=
03018 MODE_STATUS(DM35424_ADC_MODE_GO_SINGLE_SHOT, DM35424_ADC_STAT_FILLING_PRE_TRIG_BUFF),
03019 message);
03020
03021     start_timeout_timer(ONE_SEC_IN_MILLI);
03022
03023 // Test 6
03024
03025     while (!timeout_has_expired() && !exit_program) {
03026         if (local_int_count < interrupt_count) {
03027             result = DM35424_Adc_Interrupt_Get_Status(board,
03028                                                       &my_adc[adc_num],
03029                                                       &interrupt_status);

```

```

03030
03031             DM35424_Check_Result_test(result, "Error getting interrupt status");
03032
03033             /*****
03034              * Pre-Trigger Buffer Full Interrupt
03035              *****/
03036             DM35424_Check_Result_test(interrupt_matches_expected(interrupt_status,
03037
03038             DM35424_ADC_INT_PRE_BUFF_FULL_MASK |
03039
03040             DM35424_ADC_INT_SAMPLE_TAKEN_MASK |
03041
03042             DM35424_ADC_INT_PACER_TICK_MASK,
03043
03044             message) == 0,
03045             message);
03046
03047             result = DM35424_Adc_Interrupt_Clear_Status(board,
03048             &my_adc[adc_num],
03049             interrupt_status);
03050
03051             DM35424_Check_Result_test(result, "Error clearing interrupt status.");
03052
03053             local_int_count = interrupt_count;
03054
03055             }
03056
03057             /* We should have only received 1 interrupt, for the pre-trigger
03058              * buffer being full. */
03059             sprintf(message, "ADC%d: Expected 1 interrupt (Pre-Trig buff full), but received %d.",
03060             adc_num, interrupt_count);
03061             DM35424_Check_Result_test(interrupt_count != 1, message);
03062
03063             write_to_kernel_log("Test 6: Checking for Start Trigger interrupt.");
03064
03065             fprintf(stdout, "success\nADC%d: Testing Start Trigger Interrupt...", adc_num);
03066             result = DM35424_Adc_Interrupt_Set_Config(board,
03067             &my_adc[adc_num],
03068             DM35424_ADC_INT_PRE_BUFF_FULL_MASK,
03069             INTERRUPT_DISABLE);
03070
03071             DM35424_Check_Result_test(result, "Error disabling interrupts.");
03072
03073             // Test 6
03074
03075             result = DM35424_Adc_Interrupt_Set_Config(board,
03076             &my_adc[adc_num],
03077             DM35424_ADC_INT_START_TRIG_MASK,
03078             INTERRUPT_ENABLE);
03079
03080             DM35424_Check_Result_test(result, "Error enabling interrupts.");
03081
03082             /* Checking Status:Mode*/
03083             result = DM35424_Adc_Get_Mode_Status(board,
03084             &my_adc[adc_num],
03085             &mode_status);
03086
03087             DM35424_Check_Result_test(result, "Error getting mode status");
03088             sprintf(message, "Expected status:mode of WAIT_FOR_START:GO, but received 0x%x.",
03089             mode_status);
03090             DM35424_Check_Result_test(mode_status !=
03091             MODE_STATUS(DM35424_ADC_MODE_GO_SINGLE_SHOT, DM35424_ADC_STAT_WAITING_START_TRIG),
03092             message);
03093
03094             interrupt_count = 0;
03095             local_int_count = 0;
03096
03097             /* Set the start trigger to never so we can capture
03098              * the pre-start buffer filled interrupt
03099              */
03100             result = DM35424_Adc_Set_Start_Trigger(board,
03101             &my_adc[adc_num],
03102             DM35424_CLK_SRC_GBL2);
03103
03104             DM35424_Check_Result_test(result, "Error setting ADC start trigger.");
03105             start_timeout_timer(ONE_SEC_IN_MILLI);
03106
03107             while (!timeout_has_expired() && !exit_program) {
03108
03109                 if (local_int_count < interrupt_count) {
03110                     result = DM35424_Adc_Interrupt_Get_Status(board,
03111                     &my_adc[adc_num],
03112                     &interrupt_status);
03113
03114                     DM35424_Check_Result_test(result, "Error getting interrupt status");
03115
03116                     // Test 6

```

```

03111
03112         if (local_int_count == 0) {
03113
03114             /* Start Trigger Interrupt
03115             * Start Trigger Interrupt
03116             * Start Trigger Interrupt
03117             */
03118             DM35424_Check_Result_test(interrupt_matches_expected(interrupt_status,
03119 DM35424_ADC_INT_START_TRIG_MASK |
03119 DM35424_ADC_INT_SAMPLE_TAKEN_MASK |
03120 DM35424_ADC_INT_PACER_TICK_MASK,
03121                                     message) == 0,
03122                                     message);
03123
03124             result = DM35424_Adc_Interrupt_Clear_Status(board,
03125 &my_adc[adc_num],
03126 interrupt_status);
03127
03128             DM35424_Check_Result_test(result, "Error clearing interrupt
03129 status.");
03130
03131             result = DM35424_Adc_Interrupt_Set_Config(board,
03132 &my_adc[adc_num],
03133 DM35424_ADC_INT_START_TRIG_MASK,
03134 INTERRUPT_DISABLE);
03135
03136             DM35424_Check_Result_test(result, "Error disabling
03137 interrupts.");
03138
03139             /* Checking Status:Mode*/
03140             result = DM35424_Adc_Get_Mode_Status(board,
03141                                     &my_adc[adc_num],
03142                                     &mode_status);
03143             DM35424_Check_Result_test(result, "Error getting mode
03144 status");
03145             sprintf(message, "Expected status:mode of SAMPLING:GO, but
03146 received 0x%x.",
03147 mode_status);
03148             DM35424_Check_Result_test(mode_status !=
03149 MODE_STATUS(DM35424_ADC_MODE_GO_SINGLE_SHOT, DM35424_ADC_STAT_SAMPLING),
03150 message);
03151
03152             write_to_kernel_log("Test 6: Checking for Sample Taken
03153 interrupt.");
03154             fprintf(stdout, "success\nADC%d: Testing Sample Taken
03155 Interrupt...", adc_num);
03156
03157             result = DM35424_Adc_Interrupt_Set_Config(board,
03158 &my_adc[adc_num],
03159 DM35424_ADC_INT_SAMPLE_TAKEN_MASK,
03160 INTERRUPT_ENABLE);
03161
03162             DM35424_Check_Result_test(result, "Error enabling
03163 interrupts.");
03164
03165             // Test 6
03166
03167             //Restart the timer
03168             start_timeout_timer(ONE_SEC_IN_MILLI);
03169
03170             }
03171             else {
03172
03173             /* Sample Taken Interrupt
03174             * Sample Taken Interrupt
03175             * Sample Taken Interrupt
03176             */
03177             DM35424_Check_Result_test(interrupt_matches_expected(interrupt_status,
03178 DM35424_ADC_INT_SAMPLE_TAKEN_MASK |
03179 DM35424_ADC_INT_PACER_TICK_MASK,

```

```

03174                                     message) == 0,
03175                                     message);
03176
03177                                     result = DM35424_Adc_Interrupt_Clear_Status(board,
03178                                     &my_adc[adc_num],
03179                                     interrupt_status);
03180
03181                                     DM35424_Check_Result_test(result, "Error clearing interrupt
03182                                     status.");
03183                                     }
03184
03185                                     local_int_count = interrupt_count;
03186                                     }
03187
03188                                     }
03189
03190
03191 // Test 6
03192
03193                                     result = DM35424_Adc_Interrupt_Set_Config(board,
03194                                     &my_adc[adc_num],
03195                                     DM35424_ADC_INT_SAMPLE_TAKEN_MASK,
03196                                     INTERRUPT_DISABLE);
03197
03198                                     DM35424_Check_Result_test(result, "Error disabling interrupts.");
03199
03200                                     /* We waited 1 second, so there should be close to 1 seconds
03201                                     worth of interrupts that happened */
03202                                     sprintf(message, "Expected %d interrupts (approx), but received %d.",
03203                                     TEST_6_ADC_SAMPLE_RATE,
03204                                     interrupt_count);
03205                                     DM35424_Check_Result_test(!(interrupt_count > ((float) TEST_6_ADC_SAMPLE_RATE) * 0.9)
03206                                     &&
03207                                     interrupt_count < ((float) TEST_6_ADC_SAMPLE_RATE) * 1.1), message);
03208
03209                                     fprintf(stdout, "success.\nADC%d: Testing Stop Trigger Interrupt....", adc_num);
03210                                     write_to_kernel_log("Test 6: Checking for Stop Trigger interrupt.");
03211
03212                                     result = DM35424_Adc_Interrupt_Set_Config(board,
03213                                     &my_adc[adc_num],
03214                                     DM35424_ADC_INT_STOP_TRIG_MASK,
03215                                     INTERRUPT_ENABLE);
03216
03217                                     DM35424_Check_Result_test(result, "Error enabling interrupts.");
03218
03219                                     interrupt_count = 0;
03220                                     local_int_count = 0;
03221
03222                                     /* Set the stop trigger to the clock src global 2, which is
03223                                     still tied to the sample taken.
03224                                     */
03225                                     result = DM35424_Adc_Set_Stop_Trigger(board,
03226                                     &my_adc[adc_num],
03227                                     DM35424_CLK_SRC_GBL2);
03228
03229                                     DM35424_Check_Result_test(result, "Error setting ADC start trigger.");
03230
03231 // Test 6
03232
03233                                     start_timeout_timer(ONE_SEC_IN_MILLI * 4);
03234
03235                                     while (!timeout_has_expired() && !exit_program) {
03236                                     if (local_int_count < interrupt_count) {
03237                                     result = DM35424_Adc_Interrupt_Get_Status(board,
03238                                     &my_adc[adc_num],
03239                                     &interrupt_status);
03240
03241                                     DM35424_Check_Result_test(result, "Error getting interrupt status");
03242
03243                                     if (local_int_count == 0) {
03244                                     local_int_count = interrupt_count;
03245
03246                                     /* *****
03247                                     * Stop Trigger Interrupt
03248                                     * *****
03249
03250                                     DM35424_Check_Result_test(interrupt_matches_expected(interrupt_status,
03251                                     DM35424_ADC_INT_STOP_TRIG_MASK |
03252                                     DM35424_ADC_INT_SAMPLE_TAKEN_MASK |

```

```

DM35424_ADC_INT_PACER_TICK_MASK,
03252                                     message) == 0,
03253                                     message);
03254
03255                                     /* Checking Status:Mode*/
03256                                     result = DM35424_Adc_Get_Mode_Status(board,
03257                                     &my_adc[adc_num],
03258                                     &mode_status);
03259                                     DM35424_Check_Result_test(result, "Error getting mode
status");
03260                                     sprintf(message, "Expected status:mode of POST-STOP:GO, but
received 0x%x.",
03261                                     mode_status);
03262                                     DM35424_Check_Result_test(mode_status !=
03263                                     MODE_STATUS(DM35424_ADC_MODE_GO_SINGLE_SHOT,DM35424_ADC_STAT_FILLING_POST_TRIG_BUFF),
03264                                     message);
03265                                     result = DM35424_Adc_Interrupt_Set_Config(board,
03266                                     &my_adc[adc_num],
03267                                     DM35424_ADC_INT_STOP_TRIG_MASK,
03268                                     INTERRUPT_DISABLE);
03269
03270                                     DM35424_Check_Result_test(result, "Error disabling
interrupts.");
03271                                     // Test 6
03272
03273                                     result = DM35424_Adc_Interrupt_Clear_Status(board,
03274                                     &my_adc[adc_num],
03275                                     interrupt_status);
03276
03277                                     DM35424_Check_Result_test(result, "Error clearing interrupt
status.");
03278                                     write_to_kernel_log("Test 6: Checking for Post Stop Buffer
Filled interrupt.");
03279                                     fprintf(stdout, "success.\nADC%d: Testing Post Stop Buffer
03280                                     Full Interrupt....", adc_num);
03281
03282                                     result = DM35424_Adc_Interrupt_Set_Config(board,
03283                                     &my_adc[adc_num],
03284                                     DM35424_ADC_INT_POST_BUFF_FULL_MASK |
03285                                     DM35424_ADC_INT_SAMP_COMPL_MASK,
03286                                     INTERRUPT_ENABLE);
03287
03288                                     DM35424_Check_Result_test(result, "Error enabling
03289                                     interrupts.");
03290
03291                                     /* Pause the DMA, because we want to capture the difference
03292                                     between
03293                                     post stop buffer being filled, and when it is actually
03294                                     transferred via DMA
03295                                     */
03296                                     for (channel = 0; channel < my_adc[adc_num].num_dma_channels;
03297                                     channel++) {
03298                                     result = DM35424_Dma_Pause(board,
03299                                     &my_adc[adc_num],
03300                                     channel);
03301                                     DM35424_Check_Result_test(result, "Error pausing ADC
DMA.");
03302                                     }
03303                                     else if (local_int_count == 1) {
03304                                     local_int_count = interrupt_count;
03305
03306                                     // Test 6
03307
03308                                     /*****
03309                                     * Post-Trigger Buffer Full Interrupt
03310                                     *****/
03311                                     DM35424_Check_Result_test(interrupt_matches_expected(interrupt_status,
03312                                     DM35424_ADC_INT_POST_BUFF_FULL_MASK |
03313

```

```

DM35424_ADC_INT_SAMPLE_TAKEN_MASK |
03314 DM35424_ADC_INT_PACER_TICK_MASK,
03315 message) == 0,
03316 message);
03317
03318 result = DM35424_Adc_Interrupt_Set_Config(board,
03319 &my_adc[adc_num],
03320 DM35424_ADC_INT_POST_BUFF_FULL_MASK,
03321 INTERRUPT_DISABLE);
03322
03323 DM35424_Check_Result_test(result, "Error disabling
interrupts.");
03324
03325 result = DM35424_Adc_Interrupt_Clear_Status(board,
03326 &my_adc[adc_num],
03327 interrupt_status);
03328
03329 DM35424_Check_Result_test(result, "Error clearing interrupt
status.");
03330 write_to_kernel_log("Test 6: Checking for Transfer Complete
interrupt.");
03331
03332 fprintf(stdout, "success.\nADC%d: Testing Transfer Complete
Interrupt...", adc_num);
03333
03334 /* Restart the DMA so we can get the "all done" interrupt */
03335 for (channel = 0; channel < my_adc[adc_num].num_dma_channels;
channel++) {
03336 result = DM35424_Dma_Start(board,
03337 &my_adc[adc_num],
03338 channel);
03339
03340 DM35424_Check_Result_test(result, "Error starting ADC
DMA.");
03341 }
03342
// Test 6
03343 }
03344 else {
03345 /******
03346 * Sampling Complete Interrupt
03347 * *****/
03348
03349 DM35424_Check_Result_test(interrupt_matches_expected(interrupt_status,
03350 DM35424_ADC_INT_SAMP_COMPL_MASK |
03351 DM35424_ADC_INT_PACER_TICK_MASK,
03352 message) == 0,
03353 message);
03354
03355 result = DM35424_Adc_Interrupt_Clear_Status(board,
03356 &my_adc[adc_num],
03357 interrupt_status);
03358
03359 DM35424_Check_Result_test(result, "Error clearing interrupt
status.");
03360
03361 local_int_count = interrupt_count;
03362 }
03363 }
03364 }
03365 }
03366 }
03367 }
03368 /* Checking Status:Mode*/
03369 result = DM35424_Adc_Get_Mode_Status(board,
03370 &my_adc[adc_num],
03371 &mode_status);
03372 DM35424_Check_Result_test(result, "Error getting mode status");
03373 sprintf(message, "Expected status:mode of COMPLETE:GO, but received 0x%x.",
mode_status);
03374 DM35424_Check_Result_test(mode_status !=
MODE_STATUS(DM35424_ADC_MODE_GO_SINGLE_SHOT, DM35424_ADC_STAT_DONE),
03375 message);
03376
03377 sprintf(message, "Expected 3 interrupts (Stop, Post Stop, and Complete), but received
%d.",

```

```

03379             interrupt_count);
03380     DM35424_Check_Result_test(interrupt_count != 3, message);
03381
03382     // Test 6
03383
03384     interrupts_expected = 0;
03385     fprintf(stdout, "success.\nADC%d: Making sure no interrupts are still enabled...",
03386             adc_num);
03387     write_to_kernel_log("Test 6: Checking for no interrupts.");
03388     interrupt_count = 0;
03389     local_int_count = 0;
03390
03391     start_timeout_timer(ONE_SEC_IN_MILLI * 5);
03392
03393     while (!timeout_has_expired() && !exit_program && interrupt_count == 0) {
03394         if (interrupt_count > local_int_count) {
03395             output_all_interrupts();
03396         }
03397     }
03398
03399     sprintf(message, "Expected 0 interrupts (none enabled), but received %d.",
03400             interrupt_count);
03401     DM35424_Check_Result_test(interrupt_count != 0, message);
03402     write_to_kernel_log("Test 6: Checking for Pacer Clock Tick Interrupt.");
03403     fprintf(stdout, "success.\nADC%d: Testing Pacer Clock Tick interrupt...",
03404             adc_num);
03405     interrupts_expected = 1;
03406     result = DM35424_Adc_Interrupt_Set_Config(board,
03407                                               &my_adc[adc_num],
03408                                               DM35424_ADC_INT_PACER_TICK_MASK,
03409                                               INTERRUPT_ENABLE);
03410
03411     DM35424_Check_Result_test(result, "Error enabling interrupts.");
03412
03413     local_int_count = 0;
03414     start_timeout_timer(ONE_SEC_IN_MILLI);
03415
03416     // Test 6
03417
03418     while (!timeout_has_expired() && !exit_program && local_int_count == 0) {
03419         if (local_int_count < interrupt_count) {
03420             result = DM35424_Adc_Interrupt_Get_Status(board,
03421                                                       &my_adc[adc_num],
03422                                                       &interrupt_status);
03423
03424             DM35424_Check_Result_test(result, "Error getting interrupt status");
03425
03426             local_int_count = interrupt_count;
03427
03428             /* *****
03429              * Pacer Tick Interrupt
03430              * ***** */
03431             DM35424_Check_Result_test(interrupt_matches_expected(interrupt_status,
03432                                                                     DM35424_ADC_INT_PACER_TICK_MASK,
03433                                                                     message) == 0,
03434                                     message);
03435
03436             result = DM35424_Adc_Interrupt_Clear_Status(board,
03437                                                         &my_adc[adc_num],
03438                                                         interrupt_status);
03439
03440             }
03441         }
03442
03443     DM35424_Check_Result_test(local_int_count == 0, "Expected Pacer Tick Interrupt, but
03444     none received.");
03445
03446     fprintf(stdout, "success.\nADC%d: Testing low/high threshold interrupt.\n", adc_num);
03447
03448     // Test 6
03449
03450     result = DM35424_Adc_Interrupt_Set_Config(board,
03451                                               &my_adc[adc_num],
03452                                               DM35424_ADC_INT_ALL_MASK,
03453                                               INTERRUPT_DISABLE);
03454
03455     DM35424_Check_Result_test(result, "Error disabling interrupts.");
03456
03457
03458
03459

```



```

03460         result = DM35424_Adc_Interrupt_Clear_Status(board,
03461                                                         &my_adc[adc_num],
03462                                                         DM35424_ADC_INT_ALL_MASK);
03463
03464         DM35424_Check_Result_test(result, "Error clearing interrupt status.");
03465         DM35424_Micro_Sleep(10000);
03466
03467         interrupts_expected = 0;
03468
03469         /* Wait 1 second for any interrupts that were still in the system to
03470            have percolated through. Then reset to 0.
03471         */
03472         sleep(1);
03473         interrupt_count = 0;
03474         local_int_count = 0;
03475         write_to_kernel_log("Test 6: Checking for No Interrupts.");
03476         fprintf(stdout, "ADC%d: Checking no interrupts enabled....", adc_num);
03477
03478         start_timeout_timer(ONE_SEC_IN_MILLI * 2);
03479         while (!timeout_has_expired() && !exit_program && (interrupt_count == 0)) {
03480             if (interrupt_count > local_int_count) {
03481                 fprintf(stdout, "Received %d interrupts.\n", interrupt_count);
03482                 output_all_interrupts();
03483             }
03484         }
03485
03486         // Test 6
03487         DM35424_Check_Result_test(interrupt_count != 0, "Interrupts were received when we
weren't expecting them.");
03488
03489         fprintf(stdout, "success.\n");
03490
03491         /*****
03492          * Mode: Pause
03493          *****/
03494         result = DM35424_Adc_Pause(board,
03495                                     &my_adc[adc_num]);
03496
03497         DM35424_Check_Result_test(result, "Error Pausing ADC");
03498
03499         /* Checking Status:Mode*/
03500         result = DM35424_Adc_Get_Mode_Status(board,
03501                                               &my_adc[adc_num],
03502                                               &mode_status);
03503         DM35424_Check_Result_test(result, "Error getting mode status");
03504         sprintf(message, "Expected status:mode of COMPLETE:PAUSE, but received 0x%x.",
03505                 mode_status);
03506         DM35424_Check_Result_test(mode_status !=
MODE_STATUS(DM35424_ADC_MODE_PAUSE, DM35424_ADC_STAT_DONE),
message);
03507
03508
03509
03510         result = DM35424_Adc_Reset(board,
03511                                     &my_adc[adc_num]);
03512
03513         DM35424_Check_Result_test(result, "Error resetting ADC.");
03514
03515         result = DM35424_Adc_Interrupt_Set_Config(board,
03516                                                     &my_adc[adc_num],
03517                                                     DM35424_ADC_INT_CHAN_THRESHOLD_MASK,
03518                                                     INTERRUPT_ENABLE);
03519
03520         DM35424_Check_Result_test(result, "Error enabling interrupts.");
03521
03522         // Test 6
03523
03524         result = DM35424_Adc_Set_Start_Trigger(board,
03525                                                 &my_adc[adc_num],
03526                                                 DM35424_CLK_SRC_IMMEDIATE);
03527
03528         DM35424_Check_Result_test(result, "Error setting ADC start trigger.");
03529
03530         result = DM35424_Adc_Set_Stop_Trigger(board,
03531                                                &my_adc[adc_num],
03532                                                DM35424_CLK_SRC_NEVER);
03533
03534         DM35424_Check_Result_test(result, "Error setting ADC stop trigger.");
03535
03536         /* Give the pre-trigger capture size */
03537         result = DM35424_Adc_Set_Pre_Trigger_Samples(board,
03538                                                       &my_adc[adc_num],
03539                                                       0);
03540
03541         DM35424_Check_Result_test(result, "Error setting pre-trigger sample size.");
03542

```

```

03543         interrupts_expected = 1;
03544         result = DM35424_Adc_Start_Rearm(board,
03545                                         &my_adc[adc_num]);
03546
03547         DM35424_Check_Result_test(result, "Error starting ADC.");
03548
03549         result = DM35424_Adc_Volts_To_Sample(TEST_6_ADC_LOW_THRESH_VOLTS,
03550                                             &adc_low_thresh);
03551
03552         DM35424_Check_Result_test(result, "Error converting volts to samples.");
03553
03554     // Test 6
03555
03556         result = DM35424_Adc_Volts_To_Sample(TEST_6_ADC_HIGH_THRESH_VOLTS,
03557                                             &adc_high_thresh);
03558
03559         DM35424_Check_Result_test(result, "Error converting volts to samples.");
03560
03561         /* Since the DACs are giving us a sine wave, we'll be hitting the max
03562            and min repeatedly. So, for each ADC channel, we'll enable the
03563            high and low threshold interrupts, and then when they happen, we'll check
03564            the last sample value to make sure it is correct for the interrupt
03565            we received. After approximately a second of doing this, we *should*
03566            have roughly the same number of interrupts for high as low.
03567            */
03568         for (channel = 0; channel < my_adc[adc_num].num_dma_channels; channel++) {
03569
03570             result = DM35424_Adc_Channel_Set_Low_Threshold(board,
03571                                                         &my_adc[adc_num],
03572                                                         channel,
03573                                                         adc_low_thresh);
03574
03575             DM35424_Check_Result_test(result, "Error setting low threshold.");
03576
03577             result = DM35424_Adc_Channel_Set_High_Threshold(board,
03578                                                         &my_adc[adc_num],
03579                                                         channel,
03580                                                         adc_high_thresh);
03581
03582             DM35424_Check_Result_test(result, "Error setting high threshold.");
03583
03584             // Test 6
03585
03586             result = DM35424_Adc_Channel_Set_Filter(board,
03587                                                         &my_adc[adc_num],
03588                                                         channel,
03589                                                         DM35424_ADC_CHAN_FILTER_ORDER5);
03590
03591             DM35424_Check_Result_test(result, "Error setting filter order.");
03592
03593             /* Clear the channel interrupt status reg */
03594             result = DM35424_Adc_Channel_Interrupt_Clear_Status(board,
03595                                                         &my_adc[adc_num],
03596                                                         channel,
03597                                                         DM35424_ADC_CHAN_INTR_HIGH_THRESHOLD_MASK);
03598
03599             DM35424_Check_Result_test(result, "Error getting channel interrupt status.");
03600
03601             result = DM35424_Adc_Channel_Interrupt_Clear_Status(board,
03602                                                         &my_adc[adc_num],
03603                                                         DM35424_ADC_INT_ALL_MASK);
03604
03605             DM35424_Check_Result_test(result, "Error clearing interrupt status.");
03606
03607             /* start by enabling the high threshold interrupt. */
03608             result = DM35424_Adc_Channel_Interrupt_Set_Config(board,
03609                                                         &my_adc[adc_num],
03610                                                         channel,
03611                                                         DM35424_ADC_CHAN_INTR_HIGH_THRESHOLD_MASK,
03612                                                         INTERRUPT_ENABLE);
03613
03614             DM35424_Check_Result_test(result, "Error setting high threshold interrupt
03615 enable.");
03616
03617             // Test 6
03618
03619             sprintf(message, "Test 6: ADC%d Chan %d: Checking for High/Low Threshold
03620 Interrupt.",
03621                     adc_num,
03622                     channel);
03623             write_to_kernel_log(message);

```

```

03622         start_timeout_timer(ONE_SEC_IN_MILLI * 5);
03623
03624         interrupt_count = 0;
03625         local_int_count = 0;
03626         low_thresh_count = 0;
03627         high_thresh_count = 0;
03628         while (!timeout_has_expired() && !exit_program) {
03629
03630             if (local_int_count < interrupt_count) {
03631
03632                 result = DM35424_Adc_Channel_Find_Interrupt(board,
03633
03634                     &my_adc[adc_num],
03635                     &channel_with_interrupt,
03636                     &channel_has_interrupt,
03637                     &chan_interrupt_status,
03638
03639                     &uint8_value);
03640
03641                 DM35424_Check_Result_test(result, "Error looking for channel
03642                 with interrupt");
03643
03644                 result = DM35424_Adc_Channel_Get_Last_Sample(board,
03645                     &my_adc[adc_num],
03646                     channel_with_interrupt,
03647
03648                     &adc_value);
03649
03650                 DM35424_Check_Result_test(result, "Error getting last sample
03651                 value from channel.");
03652
03653                 // Test 6
03654
03655                 sprintf(message, "Was expecting interrupt from channel %d, but
03656                 received one from "
03657                     "channel %d, or no interrupt at all
03658                     (has_interrupt=%d).\n",
03659                     channel, channel_with_interrupt,
03660                     channel_has_interrupt);
03661
03662                 DM35424_Check_Result_test((channel_has_interrupt == 0) ||
03663                     (channel_with_interrupt != channel), message);
03664
03665                 /* High/Low Threshold Interrupt
03666                 *****/
03667                 DM35424_Check_Result_test((chan_interrupt_status &
03668                     (DM35424_ADC_CHAN_INTR_HIGH_THRESHOLD_MASK |
03669                     DM35424_ADC_CHAN_INTR_LOW_THRESHOLD_MASK)) ==
03670                     0, "Channel does not have "
03671                     "high or low threshold interrupts");
03672
03673                 if (chan_interrupt_status &
03674                     DM35424_ADC_CHAN_INTR_LOW_THRESHOLD_MASK &
03675                     uint8_value) {
03676
03677                     /* Check that the value which triggered this interrupt
03678                     below the threshold value.
03679
03680                     */
03681
03682                     sprintf(message, "Value %d is not less than threshold
03683                     of %d.",
03684                         adc_value,
03685                         adc_low_thresh);
03686
03687                     DM35424_Check_Result_test(adc_value > adc_low_thresh,
03688                         message);
03689
03690                     low_thresh_count++;
03691
03692                     /* Disable the current interrupt enable */
03693                     result =
03694                         DM35424_Adc_Channel_Interrupt_Set_Config(board,
03695                             &my_adc[adc_num],
03696                             channel,
03697                             DM35424_ADC_CHAN_INTR_LOW_THRESHOLD_MASK,
03698                             INTERRUPT_DISABLE);
03699
03700                     DM35424_Check_Result_test(result, "Error setting low

```

```

        threshold interrupt disable.");
03688
03689 // Test 6
03690
03691 /* Clear the channel interrupt status reg */
03692 result =
DM35424_Adc_Channel_Interrupt_Clear_Status(board,
03693 &my_adc[adc_num],
03694 channel_with_interrupt,
03695 chan_interrupt_status);
03696
03697 DM35424_Check_Result_test(result, "Error getting
03698 channel interrupt status.");
03699
03700 result = DM35424_Adc_Channel_Interrupt_Clear_Status(board,
03701 &my_adc[adc_num],
03702 DM35424_ADC_INT_CHAN_THRESHOLD_MASK);
03703
03704 DM35424_Check_Result_test(result, "Error clearing ADC
03705 interrupt status.");
03706
03707 /* Enable the HIGH threshold interrupt */
03708 result =
DM35424_Adc_Channel_Interrupt_Set_Config(board,
03709 &my_adc[adc_num],
03710 channel,
03711 DM35424_ADC_CHAN_INTR_HIGH_THRESHOLD_MASK,
03712 INTERRUPT_ENABLE);
03713
03714 DM35424_Check_Result_test(result, "Error setting high
03715 threshold interrupt enable.");
03716
03717 }
03718 else if (chan_interrupt_status &
03719 DM35424_ADC_CHAN_INTR_HIGH_THRESHOLD_MASK &
03720 uint8_value) {
03721 // Test 6
03722
03723 /* Check that the value which triggered this interrupt
03724 actually is
03725 below the threshold value.
03726 */
03727 sprintf(message, "Value %d is not more than threshold
03728 of %d.",
03729 adc_value,
03730 adc_high_thresh);
03731
03732 DM35424_Check_Result_test(adc_value < adc_high_thresh,
03733 message);
03734
03735 high_thresh_count++;
03736
03737 /* Disable the current interrupt enable */
03738 result =
DM35424_Adc_Channel_Interrupt_Set_Config(board,
03739 &my_adc[adc_num],
03740 channel,
03741 DM35424_ADC_CHAN_INTR_HIGH_THRESHOLD_MASK,
03742 INTERRUPT_DISABLE);
03743
03744 DM35424_Check_Result_test(result, "Error setting high
03745 threshold interrupt disable.");
03746
03747 /* Clear the channel interrupt status reg */
03748 result =
DM35424_Adc_Channel_Interrupt_Clear_Status(board,
03749 &my_adc[adc_num],
03750 channel_with_interrupt,
03751 chan_interrupt_status);

```

```

        chan_interrupt_status);
03746
03747                                DM35424_Check_Result_test(result, "Error getting
        channel interrupt status.");
03748
03749                                result = DM35424_Adc_Interrupt_Clear_Status(board,
03750                                &my_adc[adc_num],
03751                                DM35424_ADC_INT_CHAN_THRESHOLD_MASK);
03752
03753                                DM35424_Check_Result_test(result, "Error clearing ADC
        interrupt status.");
03754
03755                                // Test 6
03756
03757                                /* Enable the LOW threshold interrupt */
03758                                result =
        DM35424_Adc_Channel_Interrupt_Set_Config(board,
03759                                &my_adc[adc_num],
03760                                channel,
03761                                DM35424_ADC_CHAN_INTR_LOW_THRESHOLD_MASK,
        INTERRUPT_ENABLE);
03762
03763                                DM35424_Check_Result_test(result, "Error setting low
        threshold interrupt enable.");
03764
03765                                }
03766                                else {
03767
03768                                DM35424_Check_Result_test(1, "Not a high or low
        threshold interrupt.");
03769                                }
03770
03771                                local_int_count = interrupt_count;
03772                                fprintf(stdout, "ADC%d Chan %d: Low = %d, High = %d\r",
03773                                adc_num, channel, low_thresh_count,
03774                                high_thresh_count);
03775                                fflush(stdout);
03776                                }
03777                                }
03778
03779                                /* Check that there were ANY interrupts */
03780                                DM35424_Check_Result_test(interrupt_count == 0, "No interrupts occurred.");
03781
03782                                // Test 6
03783
03784                                /* There should be more than 9 interrupts that happened, and a roughly equal
03785                                amount of low and high interrupts */
03786                                sprintf(message, "Did not receive the amount of high (%d) or low (%d)
03787                                interrupts expected.",
03788                                high_thresh_count, low_thresh_count);
03789                                DM35424_Check_Result_test((low_thresh_count < 9) ||
03790                                (abs(high_thresh_count - low_thresh_count) > 1),
03791                                message);
03792
03793                                fprintf(stdout, "ADC%d Chan %d: Low = %d, High =
03794                                adc_num, channel, low_thresh_count,
        high_thresh_count);
03795                                /* Finish this channel test by disabling interrupts. */
03796                                result = DM35424_Adc_Channel_Interrupt_Set_Config(board,
03797                                &my_adc[adc_num],
03798                                channel,
03799                                DM35424_ADC_CHAN_INTR_HIGH_THRESHOLD_MASK |
03800                                DM35424_ADC_CHAN_INTR_LOW_THRESHOLD_MASK,
        INTERRUPT_DISABLE);
03801
03802                                DM35424_Check_Result_test(result, "Error disabling channel threshold
03803                                interrupts.");
03804
03805                                /* Clear the channel interrupt status reg */
03806                                result = DM35424_Adc_Channel_Interrupt_Clear_Status(board,
03807                                &my_adc[adc_num],
03808                                channel,
03809                                DM35424_ADC_CHAN_INTR_HIGH_THRESHOLD_MASK |

```

```

03811     DM35424_ADC_CHAN_INTR_LOW_THRESHOLD_MASK);
03812
03813         DM35424_Check_Result_test(result, "Error clearing channel interrupt status.");
03814         DM35424_Check_Result_test(exit_program, "User aborted with CTRL-C");
03815     }
03816 }
03817
03818 // Test 6
03819
03820 result = DM35424_Adc_Interrupt_Set_Config(board,
03821     &my_adc[adc_num],
03822     DM35424_ADC_INT_ALL_MASK,
03823     INTERRUPT_DISABLE);
03824
03825 DM35424_Check_Result_test(result, "Error disabling interrupts.");
03826
03827 interrupts_expected = 0;
03828
03829 result = DM35424_Adc_Interrupt_Clear_Status(board,
03830     &my_adc[adc_num],
03831     DM35424_ADC_INT_ALL_MASK);
03832
03833 DM35424_Check_Result_test(result, "Error clearing interrupt status.");
03834
03835 for (channel = 0; channel < my_adc[adc_num].num_dma_channels; channel++) {
03836     result = DM35424_Dma_Stop(board,
03837         &my_adc[adc_num],
03838         channel);
03839
03840     DM35424_Check_Result_test(result, "Error stopping ADC DMA.");
03841 }
03842
03843 start_timeout_timer(ONE_SEC_IN_MILLI * 1);
03844
03845 // Test 6
03846
03847 result = DM35424_Adc_Set_Post_Stop_Samples(board,
03848     &my_adc[adc_num],
03849     TEST6_ADC_POST_STOP_SAMP_SIZE);
03850
03851 DM35424_Check_Result_test(result, "Error setting pre-capture sample size.");
03852
03853 result = DM35424_Adc_Set_Stop_Trigger(board,
03854     &my_adc[adc_num],
03855     DM35424_CLK_SRC_IMMEDIATE);
03856
03857 DM35424_Check_Result_test(result, "Error setting ADC stop trigger.");
03858
03859 result = DM35424_Adc_Get_Mode_Status(board,
03860     &my_adc[adc_num],
03861     &mode_status);
03862 DM35424_Check_Result_test(result, "Error getting mode status");
03863
03864 // Test 6
03865
03866 while (!timeout_has_expired() && !exit_program &&
03867     mode_status != MODE_STATUS(DM35424_ADC_MODE_GO_REARM,
03868     DM35424_ADC_STAT_WAIT_REARM)) {
03869     /* Checking Status:Mode*/
03870     result = DM35424_Adc_Get_Mode_Status(board,
03871         &my_adc[adc_num],
03872         &mode_status);
03873     DM35424_Check_Result_test(result, "Error getting mode status");
03874 }
03875
03876 sprintf(message, "Expected status:mode of WAIT_FOR_REARM:GO_REARM, but received 0x%x.",
03877     mode_status);
03878 DM35424_Check_Result_test(mode_status != MODE_STATUS(DM35424_ADC_MODE_GO_REARM,
03879     DM35424_ADC_STAT_WAIT_REARM),
03880     message);
03881
03882 result = DM35424_Adc_Reset(board,
03883     &my_adc[adc_num]);
03884
03885 DM35424_Check_Result_test(result, "Error stopping ADC.");
03886
03887 /* Finally, test the last status of Initializing. To do that, we need
03888    to slow down the ADC and then set it to uninitialized. */
03889 result = DM35424_Adc_Initialize(board,
03890     &my_adc[adc_num]);
03891
03892 DM35424_Check_Result_test(result, "Error initializing.");
03893
03894 result = DM35424_Adc_Uninitialize(board,
03895     &my_adc[adc_num]);

```

```

03892
03893 // Test 6
03894 DM35424_Check_Result_test(result, "Error setting ADC to uninitialized.");
03895
03896 result = DM35424_Adc_Reset(board,
03897                             &my_adc[adc_num]);
03898
03899 DM35424_Check_Result_test(result, "Error stopping ADC");
03900
03901 result = DM35424_Adc_Get_Mode_Status(board,
03902                                     &my_adc[adc_num],
03903                                     &mode_status);
03904 DM35424_Check_Result_test(result, "Error getting mode status");
03905
03906 while (!timeout_has_expired() && !exit_program &&
03907        mode_status != MODE_STATUS(DM35424_ADC_MODE_RESET,
DM35424_ADC_STAT_INITIALIZING)) {
03908     /* Checking Status:Mode*/
03909     result = DM35424_Adc_Get_Mode_Status(board,
03910                                         &my_adc[adc_num],
03911                                         &mode_status);
03912     DM35424_Check_Result_test(result, "Error getting mode status");
03913 }
03914
03915 sprintf(message, "Expected status:mode of INITIALIZING:RESET, but received 0x%x.",
03916          mode_status);
03917 DM35424_Check_Result_test(mode_status != MODE_STATUS(DM35424_ADC_MODE_RESET,
DM35424_ADC_STAT_INITIALIZING),
03918                          message);
03919
03920
03921 fprintf(stdout, "ADC%d: Interrupt testing complete.\n", adc_num);
03922 DM35424_Check_Result_test(exit_program, "User aborted with CTRL-C");
03923
03924 // Test 6
03925 }
03926
03927 stop_all_dacs_with_dma();
03928
03929 result = DM35424_Dac_Set_Clock_Source_Global(board,
03930                                              &my_dac[DAC0],
03931                                              DM35424_CLK_SRC_GBL2,
03932                                              DM35424_DAC_CLK_EVENT_DISABLE);
03933
03934 DM35424_Check_Result_test(result, "Error setting clock global source");
03935
03936
03937 result = DM35424_General_RemoveISR(board);
03938 interrupts_expected = 0;
03939
03940 DM35424_Check_Result_test(result, "Error removing ISR.");
03941
03942 printf("\n\nClosing Board...");
03943 result = DM35424_Board_Close(board);
03944
03945 DM35424_Check_Result_test(result, "Error closing board.");
03946 }
03947
03948
03949
03950
03951
03952 /*****
03953 TEST 7
03954 *****/
03955 /*****
03956 void run_test_7(unsigned int minor)
03957 {
03958     int result;
03959     unsigned int dac_num, channel, buff;
03960     char message[200];
03961     long fifo_time = 0;
03962     int status_overflow;
03963     int status_underflow;
03964     int status_used;
03965     int status_invalid;
03966     int status_complete;
03967     struct DM35424_Function_Block *fb;
03968     unsigned int num_buffers = 0;
03969     uint16_t fifo_write_count = 0;

```

```

03971     unsigned int current_buffer, current_count;
03972     int buffer = 0;
03973     int expected_count = 0;
03974     uint32_t dummy_int;
03975
03976
03977     interrupts_expected = 0;
03978
03979     fprintf(stdout, "\n\n-----\n");
03980     fprintf(stdout, "Test 7: Testing DAC DMA Interrupts.\n\n");
03981
03982     printf("Opening board....");
03983     result = DM35424_Board_Open(minor, &board);
03984
03985     DM35424_Check_Result_test(result, "Could not open board");
03986     printf("success.\nResetting board....");
03987     result = DM35424_Gbc_Board_Reset(board);
03988
03989     DM35424_Check_Result_test(result, "Could not reset board");
03990
03991     fprintf(stdout, "success.\n");
03992
03993     open_dacs();
03994
03995     initialize_dacs(TEST_7_DAC_CONVERSION_RATE * 100);
03996     fifo_time = 1000000 / 100;
03997
03998 // Test 7
03999     set_isr();
04000
04001     for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD; dac_num++) {
04002         fb = &my_dac[dac_num];
04003
04004         fprintf(stdout, "TESTING DAC%d (FB%d)....\n", dac_num, fb->fb_num);
04005         interrupts_expected = 0;
04006
04007         result = DM35424_Dac_Set_Start_Trigger(board,
04008                                                 fb,
04009                                                 DM35424_CLK_SRC_IMMEDIATE);
04010
04011         DM35424_Check_Result_test(result, "Error setting start trigger for DAC");
04012
04013         result = DM35424_Dac_Set_Stop_Trigger(board,
04014                                                 fb,
04015                                                 DM35424_CLK_SRC_NEVER);
04016
04017         DM35424_Check_Result_test(result, "Error setting stop trigger for DAC");
04018
04019         initialize_dma_buffers(fb,
04020                                TEST_7_DAC_CONVERSION_RATE * sizeof(int) * 2,
04021                                DM35424_DMA_SETUP_DIRECTION_WRITE);
04022
04023         for (channel = 0; channel < fb->num_dma_channels; channel++) {
04024
04025             test7_reset_dac_and_dma(fb);
04026             interrupts_expected = 0;
04027             fprintf(stdout, " Channel %d....\n", channel);
04028
04029             result = DM35424_Dma_Configure_Interrupts(board,
04030                                                         fb,
04031                                                         channel,
04032                                                         INTERRUPT_ENABLE,
04033                                                         ERROR_INTR_ENABLE);
04034
04035             DM35424_Check_Result_test(result, "Error setting DMA Interrupts");
04036
04037 // Test 7
04038
04039         num_buffers = 0;
04040
04041         /* Setup buffers so that after 1 buffer is complete, we will HALT,
04042            throw a completed interrupt, and move the current buffer index
04043            to the next buffer and the current count will be 0. */
04044         for (buff = 0; buff < 2; buff++) {
04045
04046             if (buff == 0) {
04047                 result = DM35424_Dma_Buffer_Setup(board,
04048                                                     fb,
04049                                                     channel,
04050                                                     buff,
04051                                                     BUFFER_VALID,
04052                                                     BUFFER_HALT,
04053                                                     BUFFER_NO_LOOP,
04054                                                     BUFFER_INTERRUPT);
04055
04056                 DM35424_Check_Result_test(result, "Error setting buffer
control.");

```



```

04055             num_buffers ++;
04056         }
04057         else {
04058             result = DM35424_Dma_Buffer_Setup(board,
04059                                                 fb,
04060                                                 channel,
04061                                                 buff,
04062                                                 BUFFER_VALID,
04063                                                 BUFFER_NO_HALT,
04064                                                 BUFFER_NO_LOOP,
04065                                                 BUFFER_INTERRUPT);
04066
04067             DM35424_Check_Result_test(result, "Error setting buffer
control.");
04068
04069         }
04070
04071         // Test 7
04072     }
04073
04074     /* Check that current buffer and count are unchanged */
04075     result = DM35424_Dma_Get_Current_Buffer_Count(board,
04076                                                 fb,
04077                                                 channel,
04078                                                 &current_buffer,
04079                                                 &current_count);
04080
04081     DM35424_Check_Result_test(result, "Error getting current buffer count.");
04082
04083     DM35424_Check_Result_test(current_buffer != 0 || current_count != 0, "Expected
current "
                                "buffer and count to be zero, but they were not.");
04084
04085
04086     interrupt_count = 0;
04087     /* Start DMA without starting DAC */
04088     result = DM35424_Dma_Start(board,
04089                                fb,
04090                                channel);
04091
04092     DM35424_Check_Result_test(result, "Error starting DMA.");
04093     DM35424_Micro_Sleep(fifo_time);
04094
04095     /****** CHECKING FIFO COUNT *****/
04096     fprintf(stdout, "    Checking FIFO Write count....");
04097
04098     /* The FIFO should fill immediately, so do a check on FIFO write count */
04099     result = DM35424_Dma_Get_Fifo_Counts(board,
04100                                         fb,
04101                                         channel,
04102                                         &fifo_write_count,
04103                                         NULL);
04104
04105     DM35424_Check_Result_test(result, "Error getting FIFO counts.");
04106
04107     // Test 7
04108
04109     sprintf(message, "Expected FIFO available space of 0 (MSB set), but got
0x%x.",
04110             fifo_write_count);
04111     DM35424_Check_Result_test((fifo_write_count & 0x8000) == 0, message);
04112
04113     /****** CHECKING CURRENT COUNT *****/
04114     fprintf(stdout, "success.\n    Checking current count....");
04115
04116     /* Before starting the DAC, the current count into the buffer
should equal the FIFO size, since the FIFO fills up */
04117     result = DM35424_Dma_Get_Current_Buffer_Count(board,
04118                                                 fb,
04119                                                 channel,
04120                                                 &current_buffer,
04121                                                 &current_count);
04122
04123     DM35424_Check_Result_test(result, "Error getting current buffer count.");
04124
04125     expected_count = DM35424_FIFO_SAMPLE_SIZE * sizeof(int);
04126
04127     sprintf(message, "Expected current buffer (%d) to be 0, current count (%d) to
be %d.",
04128             current_buffer, current_count, expected_count);
04129
04130     DM35424_Check_Result_test(current_buffer != 0 || current_count !=
expected_count, message);
04131
04132     /* Interrupt expected because we set the buffer to interrupt on complete */
04133     interrupts_expected = 1;
04134

```

```

04135         result = DM35424_Dac_Start(board,
04136                                     fb);
04137
04138     DM35424_Check_Result_test(result, "Error starting DAC");
04139
04140
04141     /* With the DACs and DMA started, after running for 1 second (minimum),
04142        the first buffer will have been half output through the DAC, and
04143        the other half will have been loaded into the FIFO.*/
04144
04145     // Test 7
04146
04147     DM35424_Micro_Sleep(fifo_time);
04148     DM35424_Micro_Sleep(fifo_time / 10);
04149
04150     result = DM35424_Dac_Pause(board,
04151                                 fb);
04152
04153     DM35424_Check_Result_test(result, "Error pausing DAC");
04154
04155     /* Give the FIFO plenty of time to load the rest of the buffer into
04156        the FIFO. The DMA will halt and throw a completed interrupt. */
04157     DM35424_Micro_Sleep(fifo_time);
04158
04159     /******
04160      * BUFFER COMPLETE INTERRUPT
04161      * *****/
04162     Check_Interrupts(1, fb->fb_num, 1);
04163
04164     result = DM35424_Dma_Status(board,
04165                                 fb,
04166                                 channel,
04167                                 NULL,
04168                                 NULL,
04169                                 &status_overflow,
04170                                 &status_underflow,
04171                                 &status_used,
04172                                 &status_invalid,
04173                                 &status_complete);
04174
04175     DM35424_Check_Result_test(result, "Error getting DMA status");
04176
04177     if (status_overflow || status_underflow || status_invalid || !status_complete
04178         || status_used) {
04179
04180         output_dma_channel_status(board,
04181                                   fb,
04182                                   channel);
04183
04184         DM35424_Check_Result_test(1, "Expected Complete to be 1, all other
04185                                     status to be 0");
04186     }
04187
04188     // Test 7
04189
04190     /* After halting, the current buffer index will be the 2nd buffer,
04191        and the count should be zero */
04192     result = DM35424_Dma_Get_Current_Buffer_Count(board,
04193                                                    fb,
04194                                                    channel,
04195                                                    &current_buffer,
04196                                                    &current_count);
04197
04198     DM35424_Check_Result_test(result, "Error getting current buffer count.");
04199
04200     sprintf(message, "\nDAC%d (FB%d) Chan %d: Expected current buffer (%d) to be 1
04201                                     and current "
04202                                     "count (%d) to be 0.",
04203             dac_num,
04204             fb->fb_num,
04205             channel,
04206             current_buffer,
04207             current_count);
04208
04209     DM35424_Check_Result_test(current_buffer != 1 ||
04210                               current_count != 0, message);
04211     fprintf(stdout, "success.\n    Checking current buffer and current
04212 count...success.\n");
04213
04214     result = DM35424_Dac_Start(board,
04215                                 fb);
04216
04217     DM35424_Check_Result_test(result, "Error starting DAC");
04218
04219     /* DAC has restarted, and DMA is halted, so the dac will run out
04220        of data. This will generate an underflow interrupt, but also

```

```

04216         set the FIFO count to its max value */
04217         fprintf(stdout, "    Checking FIFO Write count...\n");
04218
04219         DM35424_Micro_Sleep(fifo_time * 3);
04220
04221         /* The FIFO should be empty, so do a check on FIFO write count */
04222         result = DM35424_Dma_Get_Fifo_Counts(board,
04223                                             fb,
04224                                             channel,
04225                                             &fifo_write_count,
04226                                             NULL);
04227
04228         DM35424_Check_Result_test(result, "Error getting FIFO counts.");
04229
04230     // Test 7
04231
04232     expected_count = DM35424_FIFO_SAMPLE_SIZE * sizeof(int);
04233     if (0x3FC < expected_count) {
04234         expected_count = 0x3FC;
04235     }
04236
04237     sprintf(message, "Expected FIFO space of %d, but got 0x%x.",
04238             expected_count,
04239             fifo_write_count);
04240     DM35424_Check_Result_test(fifo_write_count != expected_count, message);
04241
04242     fprintf(stdout, "success.\n");
04243
04244     /* Testing individual DMA channel interrupts */
04245     for (buffer = 0; buffer < fb->num_dma_buffers; buffer++) {
04246         DM35424_Check_Result_test(exit_program, "User aborted with CTRL-C");
04247         fprintf(stdout, "    Checking buffer %d interrupts: ", buffer);
04248         fflush(stdout);
04249
04250         /*****
04251          * USED INTERRUPT
04252          *****/
04253         test7_reset_dac_and_dma(fb);
04254         result = DM35424_Dma_Configure_Interrupts(board,
04255                                                  fb,
04256                                                  channel,
04257                                                  INTERRUPT_ENABLE,
04258                                                  ERROR_INTR_ENABLE);
04259
04260         DM35424_Check_Result_test(result, "Error setting DMA Interrupts");
04261
04262         result = DM35424_Dma_Setup_Set_Used(board,
04263                                             fb,
04264                                             channel,
04265                                             NOT_IGNORE_USED);
04266
04267         DM35424_Check_Result_test(result, "Error configuring DMA Setup");
04268
04269         result = DM35424_Dma_Buffer_Setup(board,
04270                                           fb,
04271                                           channel,
04272                                           buffer,
04273                                           BUFFER_VALID,
04274                                           BUFFER_NO_HALT,
04275                                           BUFFER_LOOP,
04276                                           BUFFER_NO_INTERRUPT);
04277
04278     // Test 7
04279
04280     DM35424_Check_Result_test(result, "Error configuring buffer Setup");
04281     interrupt_count = 0;
04282     result = DM35424_Dma_Start(board,
04283                               fb,
04284                               channel);
04285
04286     DM35424_Check_Result_test(result, "Error starting DMA.");
04287
04288     /* Sleep for half the buffer to go into FIFO */
04289     DM35424_Micro_Sleep(fifo_time);
04290
04291     result = DM35424_Dac_Start(board,
04292                               fb);
04293
04294     DM35424_Check_Result_test(result, "Error starting DAC");
04295
04296     /* Half of the buffer is now in the FIFO. We have to sleep
04297        long enough for that half of the buffer to go out the DAC
04298        so that the 2nd half is loaded into the FIFO and thus gives
04299        the interrupt.*/
04300     DM35424_Micro_Sleep(fifo_time * ((buffer * 2) + 1));
04301     DM35424_Micro_Sleep(fifo_time / 10);

```

```

04301      /* Pause so we don't underflow */
04302      result = DM35424_Dac_Pause(board,
04303                                fb);
04304
04305      DM35424_Check_Result_test(result, "Error pausing DAC");
04306
04307      result = DM35424_Dac_Get_Conversion_Count(board,
04308                                                fb,
04309                                                &dummy_int);
04310      DM35424_Check_Result_test(result, "Error getting conversion count.");
04311
04312      sprintf(message, "Expected conversion count (%d) to be less than
04313                  %d.\n",
04314                  dummy_int,
04315                  TEST_7_DAC_CONVERSION_RATE * 2 * (buffer + 1));
04316      DM35424_Check_Result_test(dummy_int > (TEST_7_DAC_CONVERSION_RATE * 2
04317                  * (buffer + 1)), message);
04318      // Test 7
04319
04320      Check_Interrupts(1, fb->fb_num, 1);
04321
04322      result = DM35424_Dma_Status(board,
04323                                  fb,
04324                                  channel,
04325                                  NULL,
04326                                  NULL,
04327                                  NULL,
04328                                  &status_overflow,
04329                                  &status_underflow,
04330                                  &status_used,
04331                                  &status_invalid,
04332                                  &status_complete);
04333
04334      DM35424_Check_Result_test(result, "Error getting DMA status");
04335
04336      if (status_overflow || status_underflow || status_invalid ||
04337          status_complete || !status_used) {
04338          output_dma_channel_status(board,
04339                                   fb,
04340                                   channel);
04341          DM35424_Check_Result_test(1, "Expected Used to be 1, all other
04342          status to be 0");
04343      }
04344
04345      fprintf(stdout, "USED   ");
04346      fflush(stdout);
04347      DM35424_Check_Result_test(exit_program, "User aborted with CTRL-C");
04348      /***** INVALID INTERRUPT *****/
04349      /***** *****/
04350      test7_reset_dac_and_dma(fb);
04351      result = DM35424_Dma_Configure_Interrupts(board,
04352                                                  fb,
04353                                                  channel,
04354                                                  INTERRUPT_ENABLE,
04355                                                  ERROR_INTR_ENABLE);
04356
04357      DM35424_Check_Result_test(result, "Error setting DMA Interrupts");
04358
04359      // Test 7
04360
04361      result = DM35424_Dma_Status(board,
04362                                  fb,
04363                                  channel,
04364                                  NULL,
04365                                  NULL,
04366                                  NULL,
04367                                  &status_overflow,
04368                                  &status_underflow,
04369                                  &status_used,
04370                                  &status_invalid,
04371                                  &status_complete);
04372
04373      DM35424_Check_Result_test(result, "Error getting DMA status");
04374
04375      DM35424_Check_Result_test(status_used != 0, "Expected USED to be
04376      cleared, but it was not.");
04377
04378      interrupt_count = 0;
04379
04380      result = DM35424_Dma_Buffer_Setup(board,
04381                                          fb,
04382                                          channel,
04383                                          buffer,

```

```

04381                                     BUFFER_NO_VALID,
04382                                     BUFFER_NO_HALT,
04383                                     BUFFER_NO_LOOP,
04384                                     BUFFER_NO_INTERRUPT);
04385
04386 DM35424_Check_Result_test(result, "Error configuring buffer Setup");
04387
04388
04389 result = DM35424_Dma_Start(board,
04390                             fb,
04391                             channel);
04392
04393 DM35424_Check_Result_test(result, "Error starting DMA.");
04394
04395 /* Sleep long enough for half the buffer (if available) to
04396    copy into the FIFO */
04397 DM35424_Micro_Sleep(fifo_time);
04398
04399 /* If this is buffer 0, then the interrupt will have already happened.
04400    Otherwise, we have to delay until the buffer can load into
04401    the FIFO */
04402 if (buffer != DMA_BUFFER0) {
04403     result = DM35424_Dac_Start(board,
04404                                 fb);
04405 }
04406
04407 // Test 7
04408
04409 DM35424_Check_Result_test(result, "Error starting DAC");
04410
04411 DM35424_Micro_Sleep(fifo_time * ((buffer - 1) * 2) + 1));
04412 DM35424_Micro_Sleep(fifo_time / 10);
04413
04414 result = DM35424_Dac_Pause(board,
04415                             fb);
04416
04417 DM35424_Check_Result_test(result, "Error pausing DAC");
04418 }
04419
04420 Check_Interrupts(1, fb->fb_num, 1);
04421
04422 result = DM35424_Dma_Status(board,
04423                             fb,
04424                             channel,
04425                             NULL,
04426                             NULL,
04427                             NULL,
04428                             &status_overflow,
04429                             &status_underflow,
04430                             &status_used,
04431                             &status_invalid,
04432                             &status_complete);
04433
04434 DM35424_Check_Result_test(result, "Error getting DMA status");
04435
04436 if (status_overflow || status_underflow || !status_invalid ||
    status_complete || status_used) {
04437
04438     output_dma_channel_status(board,
04439                               fb,
04440                               channel);
04441
04442     DM35424_Check_Result_test(1, "Expected Invalid to be 1, all
    other status to be 0");
04443 }
04444
04445 fprintf(stdout, "INVALID    ");
04446 fflush(stdout);
04447 DM35424_Check_Result_test(exit_program, "User aborted with CTRL-C");
04448 /*****
04449  * COMPLETE INTERRUPT
04450  *****/
04451 test7_reset_dac_and_dma(fb);
04452 result = DM35424_Dma_Configure_Interrupts(board,
04453                                             fb,
04454                                             channel,
04455                                             INTERRUPT_ENABLE,
04456                                             ERROR_INTR_ENABLE);
04457
04458 DM35424_Check_Result_test(result, "Error setting DMA Interrupts");
04459
04460 // Test 7
04461
04462 interrupt_count = 0;
04463 result = DM35424_Dma_Setup_Set_Used(board,
    fb,

```

```

04464                                     channel,
04465                                     IGNORE_USED);
04466
04467 DM35424_Check_Result_test(result, "Error configuring DMA Setup");
04468
04469 result = DM35424_Dma_Buffer_Setup(board,
04470                                     fb,
04471                                     channel,
04472                                     buffer,
04473                                     BUFFER_VALID,
04474                                     BUFFER_NO_HALT,
04475                                     BUFFER_NO_LOOP,
04476                                     BUFFER_INTERRUPT);
04477
04478 DM35424_Check_Result_test(result, "Error configuring buffer Setup");
04479
04480
04481 result = DM35424_Dma_Start(board,
04482                             fb,
04483                             channel);
04484
04485 DM35424_Check_Result_test(result, "Error starting DMA.");
04486
04487 /* Sleep for half the buffer to go into FIFO */
04488 DM35424_Micro_Sleep(fifo_time);
04489
04490 result = DM35424_Dac_Start(board,
04491                             fb);
04492
04493 // Test 7
04494
04495 DM35424_Check_Result_test(result, "Error starting DAC");
04496
04497 /* Sleep long enough for the buffer of interest to have
04498    filled into the FIFO and then loop, which will
04499    cause the USED interrupt */
04500 DM35424_Micro_Sleep(fifo_time * ((buffer * 2) + 1));
04501 DM35424_Micro_Sleep(fifo_time / 10);
04502
04503 /* Pause so we don't underflow */
04504 result = DM35424_Dac_Pause(board,
04505                             fb);
04506
04507 result = DM35424_Dma_Status(board,
04508                             fb,
04509                             channel,
04510                             NULL,
04511                             NULL,
04512                             NULL,
04513                             &status_overflow,
04514                             &status_underflow,
04515                             &status_used,
04516                             &status_invalid,
04517                             &status_complete);
04518
04519 DM35424_Check_Result_test(result, "Error getting DMA status");
04520
04521 /* Special condition...if we're at the last buffer, then when it
04522    completes,
04523    it will also set the Used bit because it will be looping back to
04524    the first,
04525    used buffer.
04526    */
04527 if (buffer == (fb->num_dma_buffers - 1)) {
04528     if (status_overflow || status_underflow || status_invalid ||
04529         !status_complete || !status_used) {
04530         output_dma_channel_status(board,
04531                                     fb,
04532                                     channel);
04533         DM35424_Check_Result_test(1, "Expected Complete and
04534             Used to be 1, all other status to be 0");
04535     }
04536 }
04537 else {
04538     if (status_overflow || status_underflow || status_invalid ||
04539         !status_complete || status_used) {
04540         // Test 7
04541         output_dma_channel_status(board,
04542                                     fb,
04543                                     channel);
04544         DM35424_Check_Result_test(1, "Expected Complete to be
04545             1, all other status to be 0");

```

```

04543         }
04544     }
04545
04546     Check_Interrupts(1, fb->fb_num, 1);
04547
04548     fprintf(stdout, "COMPLETE....success.\n");
04549     fflush(stdout);
04550     DM35424_Check_Result_test(exit_program, "User aborted with CTRL-C");
04551 }
04552
04553 /*****
04554  * UNDERFLOW INTERRUPT
04555  *****/
04556 test7_reset_dac_and_dma(fb);
04557 result = DM35424_Dma_Configure_Interrupts(board,
04558                                           fb,
04559                                           channel,
04560                                           INTERRUPT_ENABLE,
04561                                           ERROR_INTR_ENABLE);
04562
04563 DM35424_Check_Result_test(result, "Error setting DMA Interrupts");
04564
04565 interrupt_count = 0;
04566
04567 /*
04568  * Dma won't sent interrupts unless it's out of reset/halt
04569  */
04570 result = DM35424_Dma_Pause(board, fb, channel);
04571 DM35424_Check_Result_test(result, "Error setting Dma to Pause.");
04572
04573 fprintf(stdout, "    Testing UNDERFLOW interrupt....");
04574 result = DM35424_Dac_Start(board,
04575                             fb);
04576
04577 DM35424_Check_Result_test(result, "Error starting DAC");
04578
04579
04580 DM35424_Micro_Sleep(2 * fifo_time);
04581
04582 sprintf(message, "Expected 1 interrupt, but received %d.",
04583           interrupt_count);
04584 DM35424_Check_Result_test(interrupt_count != 1, message);
04585
04586 // Test 7
04587
04588 result = DM35424_Dma_Status(board,
04589                             fb,
04590                             channel,
04591                             NULL,
04592                             NULL,
04593                             NULL,
04594                             &status_overflow,
04595                             &status_underflow,
04596                             &status_used,
04597                             &status_invalid,
04598                             &status_complete);
04599
04600 DM35424_Check_Result_test(result, "Error getting DMA status");
04601
04602 if (status_overflow || !status_underflow || status_invalid || status_complete
04603     || status_used) {
04604     output_dma_channel_status(board,
04605                               fb,
04606                               channel);
04607     DM35424_Check_Result_test(1, "Expected Underflow to be 1, all other
04608 status to be 0");
04609 }
04610
04611 result = DM35424_Dac_Reset(board,
04612                             fb);
04613
04614 DM35424_Check_Result_test(result, "Error resetting DAC");
04615
04616 fprintf(stdout, "success.\n");
04617 DM35424_Check_Result_test(exit_program, "User aborted with CTRL-C");
04618 }
04619 DM35424_Check_Result_test(exit_program, "User aborted with CTRL-C");
04620 }
04621
04622 stop_all_dacs_with_dma();
04623
04624 result = DM35424_General_RemoveISR(board);
04625 interrupts_expected = 0;
04626

```

```

04627     DM35424_Check_Result_test(result, "Error removing ISR.");
04628
04629     printf("\n\nClosing Board....");
04630     result = DM35424_Board_Close(board);
04631
04632     DM35424_Check_Result_test(result, "Error closing board.");
04633
04634 }
04635
04636
04637
04638 /*****
04639
04640
04641
04642
04643 void run_test_8(unsigned int minor)
04644 {
04645     int result;
04646     unsigned int adc_num, channel, buff;
04647     char message[200];
04648     long fifo_time = 0;
04649     int status_overflow;
04650     int status_underflow;
04651     int status_used;
04652     int status_invalid;
04653     int status_complete;
04654     struct DM35424_Function_Block *fb;
04655     unsigned int num_buffers = 0;
04656     uint16_t fifo_read_count = 0;
04657     unsigned int current_buffer, current_count;
04658     int buffer = 0;
04659     int expected_count = 0;
04660     uint32_t dummy_int;
04661
04662
04663     interrupts_expected = 0;
04664
04665     fprintf(stdout, "\n\n-----\n");
04666     fprintf(stdout, "Test 8: Testing ADC DMA Interrupts.\n\n");
04667
04668     printf("Opening board....");
04669     result = DM35424_Board_Open(minor, &board);
04670
04671     DM35424_Check_Result_test(result, "Could not open board");
04672     printf("success.\nResetting board....");
04673     result = DM35424_Gbc_Board_Reset(board);
04674
04675     DM35424_Check_Result_test(result, "Could not reset board");
04676
04677     fprintf(stdout, "success.\n");
04678
04679     open_adcs();
04680
04681     initialize_adcs(TEST_8_ADC_SAMPLE_RATE * 100);
04682     fifo_time = 1000000 / 100;
04683
04684     set_isr();
04685
04686 // Test 8
04687 for (adc_num = 0; adc_num < DM35424_NUM_ADC_ON_BOARD; adc_num++) {
04688     fb = &my_adc[adc_num];
04689     fprintf(stdout, "TESTING ADC%d (FB%d)....\n", adc_num, fb->fb_num);
04690     interrupts_expected = 0;
04691
04692     result = DM35424_Adc_Set_Start_Trigger(board,
04693                                           fb,
04694                                           DM35424_CLK_SRC_IMMEDIATE);
04695
04696     DM35424_Check_Result_test(result, "Error setting start trigger for DAC");
04697
04698     result = DM35424_Adc_Set_Stop_Trigger(board,
04699                                           fb,
04700                                           DM35424_CLK_SRC_NEVER);
04701
04702     DM35424_Check_Result_test(result, "Error setting stop trigger for DAC");
04703
04704     initialize_dma_buffers(fb,
04705                           TEST_8_ADC_SAMPLE_RATE * sizeof(int) * 2,
04706                           DM35424_DMA_SETUP_DIRECTION_READ);
04707
04708     for (channel = 0; channel < fb->num_dma_channels; channel++) {

```



```

04709
04710     test8_reset_adc_and_dma(fb);
04711     interrupts_expected = 0;
04712     fprintf(stdout, " Channel %d....\n", channel);
04713
04714     result = DM35424_Dma_Configure_Interrupts(board,
04715                                               fb,
04716                                               channel,
04717                                               INTERRUPT_ENABLE,
04718                                               ERROR_INTR_ENABLE);
04719
04720     DM35424_Check_Result_test(result, "Error setting DMA Interrupts");
04721
04722 // Test 8
04723
04724     num_buffers = 0;
04725
04726     /* Setup buffers so that after 1 buffer is complete, we will HALT,
04727        throw a completed interrupt, and move the current buffer index
04728        to the next buffer and the current count will be 0. */
04729     for (buff = 0; buff < 2; buff++) {
04730         if (buff == 0) {
04731             result = DM35424_Dma_Buffer_Setup(board,
04732                                               fb,
04733                                               channel,
04734                                               buff,
04735                                               BUFFER_VALID,
04736                                               BUFFER_HALT,
04737                                               BUFFER_NO_LOOP,
04738                                               BUFFER_INTERRUPT);
04739
04740             DM35424_Check_Result_test(result, "Error setting buffer
04741 control.");
04742             num_buffers ++;
04743         }
04744         else {
04745             result = DM35424_Dma_Buffer_Setup(board,
04746                                               fb,
04747                                               channel,
04748                                               buff,
04749                                               BUFFER_VALID,
04750                                               BUFFER_NO_HALT,
04751                                               BUFFER_NO_LOOP,
04752                                               BUFFER_INTERRUPT);
04753
04754             DM35424_Check_Result_test(result, "Error setting buffer
04755 control.");
04756         }
04757     }
04758
04759     /* Check that current buffer and count are unchanged */
04760     result = DM35424_Dma_Get_Current_Buffer_Count(board,
04761                                                  fb,
04762                                                  channel,
04763                                                  &current_buffer,
04764                                                  &current_count);
04765     DM35424_Check_Result_test(result, "Error getting current buffer count.");
04766
04767 // Test 8
04768
04769     DM35424_Check_Result_test(current_buffer != 0 || current_count != 0, "Expected
04770 current "
04771                                "buffer and count to be zero, but they were not.");
04772
04773     interrupt_count = 0;
04774     /* Start DMA without starting ADC */
04775     result = DM35424_Dma_Start(board,
04776                               fb,
04777                               channel);
04778
04779     DM35424_Check_Result_test(result, "Error starting DMA.");
04780     DM35424_Micro_Sleep(fifo_time);
04781
04782     /* ***** CHECKING FIFO COUNT ***** */
04783     fprintf(stdout, "    Checking FIFO Read count....");
04784
04785     /* The FIFO should be empty, as the ADC is not yet started */
04786     result = DM35424_Dma_Get_Fifo_Counts(board,
04787                                         fb,
04788                                         channel,
04789                                         NULL,
04790                                         &fifo_read_count);
04791
04792     DM35424_Check_Result_test(result, "Error getting FIFO counts.");

```

```

04791
04792     sprintf(message, "Expected FIFO available data of 0 (MSB set), but got 0x%x.",
04793         fifo_read_count);
04794     DM35424_Check_Result_test((fifo_read_count & 0x8000) == 0, message);
04795
04796
04797     /***** CHECKING CURRENT COUNT *****/
04798     fprintf(stdout, "success.\n    Checking current count....");
04799
04800     /* Start the ADC and let it run long enough for 1 FIFO worth of data
04801        to be passed into the buffer. */
04802     result = DM35424_Adc_Start(board,
04803         fb);
04804
04805     DM35424_Check_Result_test(result, "Error starting ADC");
04806
04807 // Test 8
04808     DM35424_Micro_Sleep(fifo_time);
04809
04810
04811     result = DM35424_Adc_Pause(board,
04812         fb);
04813
04814     DM35424_Check_Result_test(result, "Error pausing ADC");
04815
04816
04817     result = DM35424_Dma_Get_Current_Buffer_Count(board,
04818         fb,
04819         channel,
04820         &current_buffer,
04821         &current_count);
04822     DM35424_Check_Result_test(result, "Error getting current buffer count.");
04823
04824     expected_count = DM35424_FIFO_SAMPLE_SIZE * sizeof(int);
04825
04826     sprintf(message, "Expected current buffer (%d) to be 0, current count (%d) to
04827         be "
04828         "close to %d.",
04829         current_buffer, current_count, expected_count);
04830     DM35424_Check_Result_test(current_buffer != 0 || (current_count <
04831         (expected_count * .9)) ||
04832         (current_count > (expected_count *
04833         1.1)), message);
04834
04835     /* Interrupt expected because we set the buffer to interrupt on complete */
04836     interrupts_expected = 1;
04837
04838     result = DM35424_Adc_Start(board,
04839         fb);
04840
04841     DM35424_Check_Result_test(result, "Error starting DAC");
04842
04843     /* With the ADC and DMA started, after running for a full FIFO time and
04844        more, the first buffer will fill up and it will move to the next buffer.*/
04845
04846     DM35424_Micro_Sleep(fifo_time);
04847     DM35424_Micro_Sleep(fifo_time / 10);
04848
04849     result = DM35424_Adc_Pause(board,
04850         fb);
04851
04852 // Test 8
04853     DM35424_Check_Result_test(result, "Error pausing DAC");
04854
04855     /* Give the FIFO plenty of time to write the rest of the data into
04856        the buffer and throw the Completed interrupt. */
04857     DM35424_Micro_Sleep(fifo_time);
04858
04859     /***** Checking for buffer complete interrupt *****/
04860     Check_Interrupts(1, fb->fb_num, 1);
04861
04862     result = DM35424_Dma_Status(board,
04863         fb,
04864         channel,
04865         NULL,
04866         NULL,
04867         NULL,
04868         &status_overflow,
04869         &status_underflow,
04870         &status_used,
04871         &status_invalid,
04872         &status_complete);

```

```

04873         DM35424_Check_Result_test(result, "Error getting DMA status");
04874
04875         if (status_overflow || status_underflow || status_invalid || !status_complete
|| status_used) {
04876
04877             output_dma_channel_status(board,
04878                                     fb,
04879                                     channel);
04880
04881             DM35424_Check_Result_test(1, "Expected Complete to be 1, all other
status to be 0");
04882         }
04883
04884         /****** CHECKING CURRENT COUNT AND BUFFER *****/
04885         /* After halting, the current buffer index will be the 2nd buffer,
and the count should be zero */
04886         result = DM35424_Dma_Get_Current_Buffer_Count(board,
04887                                                         fb,
04888                                                         channel,
04889                                                         &current_buffer,
04890                                                         &current_count);
04891         DM35424_Check_Result_test(result, "Error getting current buffer count.");
04892
04893         // Test 8
04894
04895         sprintf(message, "\nDAC%d (FB%d) Chan %d: Expected current buffer (%d) to be 1
and current "
04896                 "count (%d) to be 0.",
04897                 adc_num,
04898                 fb->fb_num,
04899                 channel,
04900                 current_buffer,
04901                 current_count);
04902
04903         DM35424_Check_Result_test(current_buffer != 1 ||
current_count != 0, message);
04904         fprintf(stdout, "success.\n    Checking current buffer and current
count...success.\n");
04905
04906         result = DM35424_Adc_Start(board,
04907                                     fb);
04908
04909         DM35424_Check_Result_test(result, "Error starting DAC");
04910
04911         /* ADC has restarted, and DMA is halted, so the adc will continue
to put data into the FIFO, causing an overflow interrupt, but also
set the FIFO count to its max value */
04912         fprintf(stdout, "    Checking FIFO Write count...");
04913
04914         DM35424_Micro_Sleep(fifo_time * 3);
04915
04916         /****** CHECKING FIFO COUNT *****/
04917
04918         /* The FIFO should be full, so do a check on FIFO read count */
04919         result = DM35424_Dma_Get_Fifo_Counts(board,
04920                                                         fb,
04921                                                         channel,
04922                                                         NULL,
04923                                                         &fifo_read_count);
04924
04925         DM35424_Check_Result_test(result, "Error getting FIFO counts.");
04926
04927         expected_count = DM35424_FIFO_SAMPLE_SIZE * sizeof(int);
04928         if (0x3FC < expected_count) {
04929             expected_count = 0x3FC;
04930         }
04931
04932         // Test 8
04933
04934         sprintf(message, "Expected FIFO space of %d, but got 0x%x.",
04935                 expected_count,
04936                 fifo_read_count);
04937         DM35424_Check_Result_test(fifo_read_count != expected_count, message);
04938         fprintf(stdout, "success.\n");
04939
04940         /******
Testing individual DMA channel interrupts
***** */
04941
04942         for (buffer = 0; buffer < fb->num_dma_buffers; buffer++) {
04943             DM35424_Check_Result_test(exit_program, "User aborted with CTRL-C");
04944             fprintf(stdout, "    Checking buffer %d interrupts: ", buffer);
04945             fflush(stdout);
04946             test8_reset_adc_and_dma(fb);
04947             result = DM35424_Dma_Configure_Interrupts(board,

```

```

04954                                     fb,
04955                                     channel,
04956                                     INTERRUPT_ENABLE,
04957                                     ERROR_INTR_ENABLE);
04958
04959 DM35424_Check_Result_test(result, "Error setting DMA Interrupts");
04960
04961 result = DM35424_Dma_Setup_Set_Used(board,
04962                                     fb,
04963                                     channel,
04964                                     NOT_IGNORE_USED);
04965
04966 DM35424_Check_Result_test(result, "Error configuring DMA Setup");
04967
04968 /***** Testing USED Interrupt and LOOPING *****/
04969 result = DM35424_Dma_Buffer_Setup(board,
04970                                     fb,
04971                                     channel,
04972                                     buffer,
04973                                     BUFFER_VALID,
04974                                     BUFFER_NO_HALT,
04975                                     BUFFER_LOOP,
04976                                     BUFFER_NO_INTERRUPT);
04977
04978 DM35424_Check_Result_test(result, "Error configuring buffer Setup");
04979 interrupt_count = 0;
04980 interrupts_expected = 0;
04981
04982 // Test 8
04983
04984 /* Pause the DMA so that the FIFO comes out of RESET */
04985 result = DM35424_Dma_Pause(board,
04986                             fb,
04987                             channel);
04988
04989 DM35424_Check_Result_test(result, "Error pausing DMA.");
04990
04991 result = DM35424_Adc_Start(board,
04992                             fb);
04993
04994 DM35424_Check_Result_test(result, "Error starting DAC");
04995
04996 /* Sleep for a small FIFO's worth of data to build up */
04997 DM35424_Micro_Sleep(fifo_time / 10);
04998
04999 result = DM35424_Dma_Start(board,
05000                             fb,
05001                             channel);
05002
05003 DM35424_Check_Result_test(result, "Error starting DMA.");
05004
05005 /* The buffer now has a half FIFO's worth of data. We have to sleep
05006    long enough for the ADC to generate a full buffer and thus gives
05007    the interrupt.*/
05008 interrupts_expected = 1;
05009 DM35424_Micro_Sleep(fifo_time * 2 * (buffer + 1));
05010
05011 /* Pause so we don't overflow */
05012 result = DM35424_Adc_Pause(board,
05013                             fb);
05014
05015 DM35424_Check_Result_test(result, "Error pausing DAC");
05016
05017 result = DM35424_Adc_Get_Sample_Count(board,
05018                                       fb,
05019                                       &dummy_int);
05020 DM35424_Check_Result_test(result, "Error getting sample count.");
05021
05022 sprintf(message, "Expected sample count (%u) to be more than %d.\n",
05023          dummy_int,
05024          TEST_8_ADC_SAMPLE_RATE * 2 * (buffer + 1));
05025
05026 DM35424_Check_Result_test(dummy_int < (TEST_8_ADC_SAMPLE_RATE * 2 *
05027 (buffer + 1)), message);
05028
05029 // Test 8
05030
05031 Check_Interrupts(1, fb->fb_num, 1);
05032
05033 result = DM35424_Dma_Status(board,
05034                             fb,
05035                             channel,
05036                             NULL,
05037                             NULL,
05038                             NULL,
05039                             &status_overflow,

```

```

05038                                     &status_underflow,
05039                                     &status_used,
05040                                     &status_invalid,
05041                                     &status_complete);
05042
05043         DM35424_Check_Result_test(result, "Error getting DMA status");
05044
05045         if (status_overflow || status_underflow || status_invalid ||
05046             status_complete || !status_used) {
05047
05048             output_dma_channel_status(board,
05049                                     fb,
05050                                     channel);
05051
05052             DM35424_Check_Result_test(1, "Expected Used to be 1, all other
05053             status to be 0");
05054         }
05055
05056         fprintf(stdout, "USED    ");
05057         fflush(stdout);
05058         DM35424_Check_Result_test(exit_program, "User aborted with CTRL-C");
05059         /***** Testing INVALID Interrupt *****/
05060         test8_reset_adc_and_dma(fb);
05061         result = DM35424_Dma_Configure_Interrupts(board,
05062             fb,
05063             channel,
05064             INTERRUPT_ENABLE,
05065             ERROR_INTR_ENABLE);
05066
05067         DM35424_Check_Result_test(result, "Error setting DMA Interrupts");
05068
05069         // Test 8
05070
05071         result = DM35424_Dma_Status(board,
05072             fb,
05073             channel,
05074             NULL,
05075             NULL,
05076             NULL,
05077             &status_overflow,
05078             &status_underflow,
05079             &status_used,
05080             &status_invalid,
05081             &status_complete);
05082
05083         DM35424_Check_Result_test(result, "Error getting DMA status");
05084         DM35424_Check_Result_test(status_used != 0, "Expected USED to be
05085         cleared, but it was not.");
05086
05087         interrupt_count = 0;
05088
05089         result = DM35424_Dma_Buffer_Setup(board,
05090             fb,
05091             channel,
05092             buffer,
05093             BUFFER_NO_VALID,
05094             BUFFER_NO_HALT,
05095             BUFFER_NO_LOOP,
05096             BUFFER_NO_INTERRUPT);
05097
05098         DM35424_Check_Result_test(result, "Error configuring buffer Setup");
05099
05100         /* Pause the DMA so that the FIFO comes out of RESET */
05101         result = DM35424_Dma_Pause(board,
05102             fb,
05103             channel);
05104
05105         DM35424_Check_Result_test(result, "Error pausing DMA.");
05106
05107         result = DM35424_Adc_Start(board,
05108             fb);
05109
05110         DM35424_Check_Result_test(result, "Error starting ADC");
05111
05112         // Test 8
05113
05114         /* Sleep long enough for half a FIFO */
05115         DM35424_Micro_Sleep(fifo_time / 10);
05116
05117         result = DM35424_Dma_Start(board,
05118             fb,
05119             channel);
05120
05121         DM35424_Check_Result_test(result, "Error starting DMA.");
05122
05123         /* If this is buffer 0, then the interrupt will have already happened.

```

```

05120         Otherwise, we have to delay until the buffer can load into
05121         the FIFO */
05122         if (buffer != DMA_BUFFER0) {
05123             DM35424_Micro_Sleep(fifo_time * 2 * buffer);
05124         }
05125     }
05126     result = DM35424_Adc_Pause(board,
05127                                fb);
05128
05129     DM35424_Check_Result_test(result, "Error pausing DAC");
05130     /* Sleep long enough for DMA to finish processing */
05131     DM35424_Micro_Sleep(fifo_time / 4);
05132     Check_Interrupts(1, fb->fb_num, 1);
05133
05134 // Test 8
05135
05136     result = DM35424_Dma_Status(board,
05137                                  fb,
05138                                  channel,
05139                                  NULL,
05140                                  NULL,
05141                                  NULL,
05142                                  &status_overflow,
05143                                  &status_underflow,
05144                                  &status_used,
05145                                  &status_invalid,
05146                                  &status_complete);
05147
05148     DM35424_Check_Result_test(result, "Error getting DMA status");
05149
05150     if (status_overflow || status_underflow || !status_invalid ||
05151         status_complete || status_used) {
05152         output_dma_channel_status(board,
05153                                   fb,
05154                                   channel);
05155
05156         DM35424_Check_Result_test(1, "Expected Invalid to be 1, all
05157 other status to be 0");
05158     }
05159
05160     fprintf(stdout, "INVALID    ");
05161     fflush(stdout);
05162     DM35424_Check_Result_test(exit_program, "User aborted with CTRL-C");
05163     /****** Testing COMPLETE Interrupt *****/
05164     test8_reset_adc_and_dma(fb);
05165     interrupts_expected = 0;
05166     result = DM35424_Dma_Configure_Interrupts(board,
05167                                                 fb,
05168                                                 channel,
05169                                                 INTERRUPT_ENABLE,
05170                                                 ERROR_INTR_ENABLE);
05171
05172     DM35424_Check_Result_test(result, "Error setting DMA Interrupts");
05173
05174     interrupt_count = 0;
05175     result = DM35424_Dma_Setup_Set_Used(board,
05176                                         fb,
05177                                         channel,
05178                                         IGNORE_USED);
05179
05180     DM35424_Check_Result_test(result, "Error configuring DMA Setup");
05181
05182 // Test 8
05183
05184     result = DM35424_Dma_Buffer_Setup(board,
05185                                         fb,
05186                                         channel,
05187                                         buffer,
05188                                         BUFFER_VALID,
05189                                         BUFFER_NO_HALT,
05190                                         BUFFER_NO_LOOP,
05191                                         BUFFER_INTERRUPT);
05192
05193     DM35424_Check_Result_test(result, "Error configuring buffer Setup");
05194
05195     /* Pause the DMA so that the FIFO comes out of RESET */
05196     result = DM35424_Dma_Pause(board,
05197                                 fb,
05198                                 channel);
05199
05200     DM35424_Check_Result_test(result, "Error pausing DMA.");
05201
05202

```

```

05203             result = DM35424_Adc_Start(board,
05204                                     fb);
05205
05206             DM35424_Check_Result_test(result, "Error starting DAC");
05207
05208             /* Sleep for half the FIFO to fill */
05209             DM35424_Micro_Sleep(fifo_time / 10);
05210
05211             result = DM35424_Dma_Start(board,
05212                                     fb,
05213                                     channel);
05214
05215             DM35424_Check_Result_test(result, "Error starting DMA.");
05216
05217             interrupts_expected = 1;
05218             /* Sleep long enough for the buffer of interest to have
05219              * filled from the FIFO */
05220             DM35424_Micro_Sleep(fifo_time * 2 * (buffer + 1));
05221
05222             /* Pause so we don't underflow */
05223             result = DM35424_Adc_Pause(board,
05224                                     fb);
05225
05226             // Test 8
05227
05228             result = DM35424_Dma_Status(board,
05229                                     fb,
05230                                     channel,
05231                                     NULL,
05232                                     NULL,
05233                                     NULL,
05234                                     &status_overflow,
05235                                     &status_underflow,
05236                                     &status_used,
05237                                     &status_invalid,
05238                                     &status_complete);
05239
05240             DM35424_Check_Result_test(result, "Error getting DMA status");
05241             Check_Interrupts(1, fb->fb_num, 1);
05242             /* Special condition...if we're at the last buffer, then when it
05243              * completes,
05244              * it will also set the Used bit because it will be looping back to
05245              * the first,
05246              * used buffer.
05247              */
05248             if (buffer == (fb->num_dma_buffers - 1)) {
05249                 if (status_overflow || status_underflow || status_invalid ||
05250                     !status_complete || !status_used) {
05251                     output_dma_channel_status(board,
05252                                             fb,
05253                                             channel);
05254                     DM35424_Check_Result_test(1, "Expected Complete and
05255                     Used to be 1, all other status to be 0");
05256                 }
05257             } else {
05258                 if (status_overflow || status_underflow || status_invalid ||
05259                     !status_complete || status_used) {
05260                     output_dma_channel_status(board,
05261                                             fb,
05262                                             channel);
05263                     DM35424_Check_Result_test(1, "Expected Complete to be
05264                     1, all other status to be 0");
05265                 }
05266             }
05267             // Test 8
05268
05269             fprintf(stdout, "COMPLETE.....success.\n");
05270             fflush(stdout);
05271             DM35424_Check_Result_test(exit_program, "User aborted with CTRL-C");
05272         }
05273         DM35424_Check_Result_test(exit_program, "User aborted with CTRL-C");
05274         /****** Testing OVERFLOW Interrupt *****/
05275         test8_reset_adc_and_dma(fb);
05276         interrupts_expected = 0;
05277         result = DM35424_Dma_Configure_Interrupts(board,
05278                                                 fb,
05279                                                 channel,
05280                                                 INTERRUPT_ENABLE,
05281                                                 ERROR_INTR_ENABLE);

```

```

05282         DM35424_Check_Result_test(result, "Error setting DMA Interrupts");
05283         result = DM35424_Dma_Buffer_Setup(board,
05284                                           fb,
05285                                           channel,
05286                                           0,
05287                                           BUFFER_VALID,
05288                                           BUFFER_HALT,
05289                                           BUFFER_NO_LOOP,
05290                                           BUFFER_NO_INTERRUPT);
05291
05292         DM35424_Check_Result_test(result, "Error configuring buffer Setup");
05293         interrupt_count = 0;
05294
05295         /* Pause the DMA so that the FIFO comes out of RESET */
05296         result = DM35424_Dma_Pause(board,
05297                                    fb,
05298                                    channel);
05299
05300         DM35424_Check_Result_test(result, "Error pausing DMA.");
05301
05302         fprintf(stdout, "    Testing OVERFLOW interrupt....");
05303         interrupts_expected = 1;
05304         result = DM35424_Adc_Start(board,
05305                                    fb);
05306
05307         // Test 8
05308
05309         DM35424_Check_Result_test(result, "Error starting ADC");
05310
05311         DM35424_Micro_Sleep(fifo_time * 3);
05312
05313         Check_Interrupts(1, fb->fb_num, 1);
05314
05315         result = DM35424_Dma_Status(board,
05316                                     fb,
05317                                     channel,
05318                                     NULL,
05319                                     NULL,
05320                                     NULL,
05321                                     &status_overflow,
05322                                     &status_underflow,
05323                                     &status_used,
05324                                     &status_invalid,
05325                                     &status_complete);
05326
05327         DM35424_Check_Result_test(result, "Error getting DMA status");
05328
05329         if (!status_overflow || status_underflow || status_invalid || status_complete
05330             || status_used) {
05331
05332             output_dma_channel_status(board,
05333                                      fb,
05334                                      channel);
05335
05336             DM35424_Check_Result_test(1, "Expected Overflow to be 1, all other
05337 status to be 0");
05338         }
05339
05340         result = DM35424_Adc_Reset(board,
05341                                    fb);
05342
05343         // Test 8
05344
05345         DM35424_Check_Result_test(result, "Error resetting DAC");
05346
05347         fprintf(stdout, "success.\n");
05348         DM35424_Check_Result_test(exit_program, "User aborted with CTRL-C");
05349     }
05350     DM35424_Check_Result_test(exit_program, "User aborted with CTRL-C");
05351 }
05352
05353 stop_all_adcs_with_dma();
05354
05355 result = DM35424_General_RemoveISR(board);
05356 interrupts_expected = 0;
05357
05358 DM35424_Check_Result_test(result, "Error removing ISR.");
05359
05360 printf("\n\nClosing Board....");
05361 result = DM35424_Board_Close(board);
05362
05363 DM35424_Check_Result_test(result, "Error closing board.");
05364
05365
05366
05367
05368
05369
05370
05371
05372
05373
05374
05375
05376
05377
05378
05379
05380
05381
05382
05383
05384
05385
05386
05387
05388
05389
05390
05391
05392
05393
05394
05395
05396
05397
05398
05399
05400
05401
05402
05403
05404
05405
05406
05407
05408
05409
05410
05411
05412
05413
05414
05415
05416
05417
05418
05419
05420
05421
05422
05423
05424
05425
05426
05427
05428
05429
05430
05431
05432
05433
05434
05435
05436
05437
05438
05439
05440
05441
05442
05443
05444
05445
05446
05447
05448
05449
05450
05451
05452
05453
05454
05455
05456
05457
05458
05459
05460
05461
05462
05463
05464
05465
05466
05467
05468
05469
05470
05471
05472
05473
05474
05475
05476
05477
05478
05479
05480
05481
05482
05483
05484
05485
05486
05487
05488
05489
05490
05491
05492
05493
05494
05495
05496
05497
05498
05499
05500
05501
05502
05503
05504
05505
05506
05507
05508
05509
05510
05511
05512
05513
05514
05515
05516
05517
05518
05519
05520
05521
05522
05523
05524
05525
05526
05527
05528
05529
05530
05531
05532
05533
05534
05535
05536
05537
05538
05539
05540
05541
05542
05543
05544
05545
05546
05547
05548
05549
05550
05551
05552
05553
05554
05555
05556
05557
05558
05559
05560
05561
05562
05563
05564
05565
05566
05567
05568
05569
05570
05571
05572
05573
05574
05575
05576
05577
05578
05579
05580
05581
05582
05583
05584
05585
05586
05587
05588
05589
05590
05591
05592
05593
05594
05595
05596
05597
05598
05599
05600
05601
05602
05603
05604
05605
05606
05607
05608
05609
05610
05611
05612
05613
05614
05615
05616
05617
05618
05619
05620
05621
05622
05623
05624
05625
05626
05627
05628
05629
05630
05631
05632
05633
05634
05635
05636
05637
05638
05639
05640
05641
05642
05643
05644
05645
05646
05647
05648
05649
05650
05651
05652
05653
05654
05655
05656
05657
05658
05659
05660
05661
05662
05663
05664
05665
05666
05667
05668
05669
05670
05671
05672
05673
05674
05675
05676
05677
05678
05679
05680
05681
05682
05683
05684
05685
05686
05687
05688
05689
05690
05691
05692
05693
05694
05695
05696
05697
05698
05699
05700
05701
05702
05703
05704
05705
05706
05707
05708
05709
05710
05711
05712
05713
05714
05715
05716
05717
05718
05719
05720
05721
05722
05723
05724
05725
05726
05727
05728
05729
05730
05731
05732
05733
05734
05735
05736
05737
05738
05739
05740
05741
05742
05743
05744
05745
05746
05747
05748
05749
05750
05751
05752
05753
05754
05755
05756
05757
05758
05759
05760
05761
05762
05763
05764
05765
05766
05767
05768
05769
05770
05771
05772
05773
05774
05775
05776
05777
05778
05779
05780
05781
05782
05783
05784
05785
05786
05787
05788
05789
05790
05791
05792
05793
05794
05795
05796
05797
05798
05799
05800
05801
05802
05803
05804
05805
05806
05807
05808
05809
05810
05811
05812
05813
05814
05815
05816
05817
05818
05819
05820
05821
05822
05823
05824
05825
05826
05827
05828
05829
05830
05831
05832
05833
05834
05835
05836
05837
05838
05839
05840
05841
05842
05843
05844
05845
05846
05847
05848
05849
05850
05851
05852
05853
05854
05855
05856
05857
05858
05859
05860
05861
05862
05863
05864
05865
05866
05867
05868
05869
05870
05871
05872
05873
05874
05875
05876
05877
05878
05879
05880
05881
05882
05883
05884
05885
05886
05887
05888
05889
05890
05891
05892
05893
05894
05895
05896
05897
05898
05899
05900
05901
05902
05903
05904
05905
05906
05907
05908
05909
05910
05911
05912
05913
05914
05915
05916
05917
05918
05919
05920
05921
05922
05923
05924
05925
05926
05927
05928
05929
05930
05931
05932
05933
05934
05935
05936
05937
05938
05939
05940
05941
05942
05943
05944
05945
05946
05947
05948
05949
05950
05951
05952
05953
05954
05955
05956
05957
05958
05959
05960
05961
05962
05963
05964
05965
05966
05967
05968
05969
05970
05971
05972
05973
05974
05975
05976
05977
05978
05979
05980
05981
05982
05983
05984
05985
05986
05987
05988
05989
05990
05991
05992
05993
05994
05995
05996
05997
05998
05999
06000

```



```

05365 }
05366
05367
05368
05369
05370 /*****
05371
05372
05373
05374
05375 void run_test_9(unsigned int minor)
05376 {
05377     int result;
05378     unsigned int dac_num, adc_num, channel;
05379     unsigned int clock;
05380     char message[200];
05381     uint32_t sample_counts = 0;
05382     uint8_t mode_status = 0;
05383     unsigned int total_count, normal_count, inverted_count;
05384     int my_value;
05385     interrupts_expected = 0;
05386
05387
05388     fprintf(stdout, "\n\n-----\n");
05389     fprintf(stdout, "Test 9: Testing Clock Sources.\n\n");
05390
05391     printf("Opening board....");
05392     result = DM35424_Board_Open(minor, &board);
05393
05394     DM35424_Check_Result_test(result, "Could not open board");
05395     printf("success.\nResetting board....");
05396     result = DM35424_Gbc_Board_Reset(board);
05397
05398     DM35424_Check_Result_test(result, "Could not reset board");
05399
05400     fprintf(stdout, "success.\n");
05401
05402     open_dacs();
05403
05404     initialize_dacs(TEST_9_DAC_CONVERSION_RATE);
05405
05406     open_adcs();
05407
05408     // Test 9
05409     initialize_adcs(TEST_9_ADC_SAMPLE_RATE);
05410
05411     for (adc_num = 0; adc_num < DM35424_NUM_ADC_ON_BOARD; adc_num++) {
05412         fprintf(stdout, "Testing clocks (driving) with ADC %d =====\n", adc_num);
05413         initialize_dma_buffers(&my_adc[adc_num],
05414                                BUFFER_BYTES_SIZE,
05415                                DM35424_DMA_SETUP_DIRECTION_READ);
05416
05417         result = DM35424_Adc_Reset(board,
05418                                    &my_adc[adc_num]);
05419
05420         DM35424_Check_Result_test(result, "Error resetting ADC");
05421
05422         result = DM35424_Adc_Set_Pre_Trigger_Samples(board,
05423                                                       &my_adc[adc_num],
05424                                                       10);
05425         DM35424_Check_Result_test(result, "Error setting pre-trigger sample count for ADC.");
05426
05427         result = DM35424_Adc_Set_Post_Stop_Samples(board,
05428                                                       &my_adc[adc_num],
05429                                                       10);
05430         DM35424_Check_Result_test(result, "Error setting post-stop sample count for ADC.");
05431
05432         result = DM35424_Adc_Channel_Interrupt_Set_Config(board,
05433                                                            &my_adc[adc_num],
05434                                                            DMA_CHAN0,
05435                                                            DM35424_ADC_CHAN_INTR_LOW_THRESHOLD_MASK |
05436                                                            DM35424_ADC_CHAN_INTR_HIGH_THRESHOLD_MASK,
05437                                                            INTERRUPT_DISABLE);
05438
05439         DM35424_Check_Result_test(result, "Error enabling threshold interrupts.");
05440
05441         fprintf(stdout, "    Setting up DACs.\n");
05442         for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD; dac_num++) {
05443             fprintf(stdout, "        DAC%d: Setting initial configuration.\n", dac_num);
05444

```

```

05445         result = DM35424_Dac_Reset(board,
05446                                     &my_dac[dac_num]);
05447
05448     // Test 9
05449
05450     DM35424_Check_Result_test(result, "Error resetting DAC.");
05451
05452     result = DM35424_Dac_Get_Conversion_Count(board,
05453                                               &my_dac[dac_num],
05454                                               &sample_counts);
05455
05456     DM35424_Check_Result_test(result, "Error getting sample counts.");
05457     sprintf(message, "DAC%d: EXpected conversion count of 0, but got %d.",
05458               dac_num,
05459               sample_counts);
05460     DM35424_Check_Result_test(sample_counts != 0, message);
05461
05462     for (clock = DM35424_CLK_SRC_GBL2; clock <= DM35424_CLK_SRC_GBL7; clock++) {
05463         result = DM35424_Dac_Set_Clock_Source_Global(board,
05464                                                       &my_dac[dac_num],
05465                                                       clock,
05466
05467         DM35424_DAC_CLK_EVENT_DISABLE);
05468         sprintf(message, "DAC%d: Could not set clock source GBL%d to
05469         DISABLE.",
05470                   dac_num, clock);
05471         DM35424_Check_Result_test(result, message);
05472     }
05473
05474     result = DM35424_Dac_Set_Start_Trigger(board,
05475                                             &my_dac[dac_num],
05476                                             DM35424_CLK_SRC_IMMEDIATE);
05477
05478     DM35424_Check_Result_test(result, "Error setting start trigger for DAC");
05479
05480     result = DM35424_Dac_Set_Stop_Trigger(board,
05481                                           &my_dac[dac_num],
05482                                           DM35424_CLK_SRC_NEVER);
05483
05484     DM35424_Check_Result_test(result, "Error setting stop trigger for DAC");
05485
05486     // Test 9
05487
05488     result = DM35424_Dac_Set_Clock_Div(board,
05489                                       &my_dac[dac_num],
05490                                       0);
05491     DM35424_Check_Result_test(result, "Error setting clock div.");
05492
05493     result = DM35424_Dac_Get_Conversion_Count(board,
05494                                               &my_dac[dac_num],
05495                                               &sample_counts);
05496
05497     DM35424_Check_Result_test(result, "Error getting sample counts.");
05498     sprintf(message, "DAC%d: Expected conversion count of 0, but got %d.",
05499               dac_num,
05500               sample_counts);
05501     DM35424_Check_Result_test(sample_counts != 0, message);
05502     DM35424_Check_Result_test(exit_program, "User aborted with CTRL-C");
05503 }
05504
05505 // Setting DAC0 clock to be GBL_SRC2
05506 result = DM35424_Dac_Set_Clock_Src(board,
05507                                   &my_dac[DAC0],
05508                                   DM35424_CLK_SRC_GBL2);
05509
05510 DM35424_Check_Result_test(result, "Error setting clock of DAC0");
05511
05512 // Setting DAC1 clock to be GBL_SRC3
05513 result = DM35424_Dac_Set_Clock_Src(board,
05514                                   &my_dac[DAC1],
05515                                   DM35424_CLK_SRC_GBL3);
05516
05517 DM35424_Check_Result_test(result, "Error setting clock of DAC1");
05518
05519 // Setting DAC2 clock to be GBL_SRC4
05520 result = DM35424_Dac_Set_Clock_Src(board,
05521                                   &my_dac[DAC2],
05522                                   DM35424_CLK_SRC_GBL4);
05523
05524 DM35424_Check_Result_test(result, "Error setting clock of DAC2");
05525
05526 // Test 9
05527
05528 // Setting DAC3 clock to be GBL_SRC5
05529 result = DM35424_Dac_Set_Clock_Src(board,

```

```

05527                                     &my_dac[DAC3],
05528                                     DM35424_CLK_SRC_GBL5);
05529
05530     DM35424_Check_Result_test(result, "Error setting clock of DAC3");
05531
05532
05533     for (clock = DM35424_CLK_SRC_GBL2; clock <= DM35424_CLK_SRC_GBL7; clock++) {
05534         result = DM35424_Adc_Set_Clock_Source_Global(board,
05535                                                     &my_adc[adc_num],
05536                                                     clock,
05537
DM35424_ADC_CLK_EVENT_DISABLE);
05538         sprintf(message, "ADC%d: Could not set clock src GBL%d to DISABLE.",
05539                     adc_num, clock);
05540         DM35424_Check_Result_test(result, message);
05541     }
05542
05543     fprintf(stdout, "    Setting ADC clock events.\n");
05544     // Set all of the global flags to various events of the adc_num
05545     result = DM35424_Adc_Set_Clock_Source_Global(board,
05546                                                 &my_adc[adc_num],
05547                                                 DM35424_CLK_SRC_GBL2,
05548
DM35424_ADC_CLK_EVENT_PRE_START_BUFF_FULL);
05549
05550     DM35424_Check_Result_test(result, "Error setting Clock Global Source");
05551
05552     result = DM35424_Adc_Set_Clock_Source_Global(board,
05553                                                 &my_adc[adc_num],
05554                                                 DM35424_CLK_SRC_GBL3,
05555                                                 DM35424_ADC_CLK_EVENT_START_TRIG);
05556
05557     DM35424_Check_Result_test(result, "Error setting Clock Global Source");
05558
05559     // Test 9
05560
05561     result = DM35424_Adc_Set_Clock_Source_Global(board,
05562                                                 &my_adc[adc_num],
05563                                                 DM35424_CLK_SRC_GBL4,
05564                                                 DM35424_ADC_CLK_EVENT_STOP_TRIG);
05565
05566     DM35424_Check_Result_test(result, "Error setting Clock Global Source");
05567
05568     result = DM35424_Adc_Set_Clock_Source_Global(board,
05569                                                 &my_adc[adc_num],
05570                                                 DM35424_CLK_SRC_GBL5,
05571
DM35424_ADC_CLK_EVENT_POST_STOP_BUFF_FULL);
05572
05573     DM35424_Check_Result_test(result, "Error setting Clock Global Source");
05574
05575     result = DM35424_Adc_Get_Sample_Count(board,
05576                                         &my_adc[adc_num],
05577                                         &sample_counts);
05578
05579     DM35424_Check_Result_test(result, "Error getting sample counts.");
05580     sprintf(message, "adc_num: Expected sample count of 0, but got %d.",
05581             sample_counts);
05582     DM35424_Check_Result_test(sample_counts != 0, message);
05583     fprintf(stdout, "    Starting DACs.\n");
05584     for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD; dac_num++) {
05585         result = DM35424_Dac_Start(board,
05586                                   &my_dac[dac_num]);
05587
05588         DM35424_Check_Result_test(result, "Error starting DAC");
05589
05590         result = DM35424_Dac_Get_Conversion_Count(board,
05591                                                   &my_dac[dac_num],
05592                                                   &sample_counts);
05593
05594     // Test 9
05595
05596     DM35424_Check_Result_test(result, "Error getting sample counts.");
05597     sprintf(message, "DAC%d: Expected conversion count of 0, but got %d.",
05598             dac_num,
05599             sample_counts);
05600     DM35424_Check_Result_test(sample_counts != 0, message);
05601 }
05602
05603 // Now we're ready to start adc_num. Once we do that, then everything else should
05604 // move forward. Before starting it, we're going to set the start trigger
05605 result = DM35424_Adc_Set_Start_Trigger(board,
05606                                       &my_adc[adc_num],
05607                                       DM35424_CLK_SRC_IMMEDIATE);
05608

```

```
05609         DM35424_Check_Result_test(result, "Error setting start trigger adc_num clock source");
05610
05611         result = DM35424_Adc_Set_Stop_Trigger(board,
05612                                             &my_adc[adc_num],
05613                                             DM35424_CLK_SRC_NEVER);
05614
05615         DM35424_Check_Result_test(result, "Error setting stop trigger adc_num clock source");
05616
05617         // Before starting adc_num, check that all of the DAC counts are zero.
05618         for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD; dac_num++) {
05619             result = DM35424_Dac_Get_Conversion_Count(board,
05620                                                         &my_dac[dac_num],
05621                                                         &sample_counts);
05622
05623             DM35424_Check_Result_test(result, "Error getting sample counts.");
05624
05625             sprintf(message, "Expected conversion count of 0 for DAC%d, but received
05626 %d.\n",
05627                             dac_num, sample_counts);
05628             DM35424_Check_Result_test(sample_counts != 0, message);
05629         }
05630
05631         // Test 9
05632
05633         result = DM35424_Adc_Start(board,
05634                                   &my_adc[adc_num]);
05635
05636         DM35424_Check_Result_test(result, "Error starting adc_num");
05637
05638         // Give everything a chance to complete.
05639         sleep(1);
05640
05641         result = DM35424_Adc_Set_Stop_Trigger(board,
05642                                             &my_adc[adc_num],
05643                                             DM35424_CLK_SRC_IMMEDIATE);
05644
05645         DM35424_Check_Result_test(result, "Error setting stop trigger adc_num clock source");
05646
05647         // Give everything a chance to complete.
05648         sleep(1);
05649
05650         // Now then, we should find the following:
05651         // 1) All DACs have had 1 conversion
05652         // 2) The ADC should be stopped, and have non-zero
05653         //     sample counts.
05654         fprintf(stdout, "    Checking clock triggers: PRE-START ");
05655         result = DM35424_Dac_Get_Conversion_Count(board,
05656                                                     &my_dac[DAC0],
05657                                                     &sample_counts);
05658
05659         DM35424_Check_Result_test(result, "Error getting sample counts.");
05660
05661         sprintf(message, "Expected count of 1 for DAC%d, but received %d.\n",
05662                 0, sample_counts);
05663         DM35424_Check_Result_test(sample_counts != 1, message);
05664
05665         fprintf(stdout, "    START ");
05666
05667         result = DM35424_Dac_Get_Conversion_Count(board,
05668                                                     &my_dac[DAC1],
05669                                                     &sample_counts);
05670
05671         // Test 9
05672
05673         DM35424_Check_Result_test(result, "Error getting sample counts.");
05674
05675         sprintf(message, "Expected count of 1 for DAC%d, but received %d.\n",
05676                 1, sample_counts);
05677         DM35424_Check_Result_test(sample_counts != 1, message);
05678
05679         fprintf(stdout, "    STOP ");
05680
05681         result = DM35424_Dac_Get_Conversion_Count(board,
05682                                                     &my_dac[DAC2],
05683                                                     &sample_counts);
05684
05685         DM35424_Check_Result_test(result, "Error getting sample counts.");
05686
05687         sprintf(message, "Expected count of 1 for DAC%d, but received %d.\n",
05688                 2, sample_counts);
05689         DM35424_Check_Result_test(sample_counts != 1, message);
05690
05691         fprintf(stdout, "    POST-STOP ");
05692
05693         result = DM35424_Dac_Get_Conversion_Count(board,
```

```

05693                                     &my_dac[DAC3],
05694                                     &sample_counts);
05695
05696     DM35424_Check_Result_test(result, "Error getting sample counts.");
05697
05698     sprintf(message, "Expected count of 1 for DAC%d, but received %d.\n",
05699                   1, sample_counts);
05700     DM35424_Check_Result_test(sample_counts != 1, message);
05701
05702     fprintf(stdout, "    ....success!\n");
05703
05704     result = DM35424_Adc_Get_Mode_Status(board,
05705                                         &my_adc[adc_num],
05706                                         &mode_status);
05707
05708     DM35424_Check_Result_test(result, "Error getting mode status from ADC.");
05709
05710 // Test 9
05711     sprintf(message, "ADC%d: Expected status of DONE, but got 0x%x.\n",
05712           adc_num,
05713           GET_STATUS(mode_status));
05714
05715     DM35424_Check_Result_test(GET_STATUS(mode_status) != DM35424_ADC_STAT_DONE, message);
05716
05717     result = DM35424_Adc_Get_Sample_Count(board,
05718                                         &my_adc[adc_num],
05719                                         &sample_counts);
05720
05721     DM35424_Check_Result_test(result, "Error getting sample counts.");
05722     sprintf(message, "ADC%d: Expected non-zero sample count, but got 0",
05723           adc_num);
05724     DM35424_Check_Result_test(sample_counts == 0, message);
05725     DM35424_Check_Result_test(exit_program, "User aborted with CTRL-C");
05726
05727     result = DM35424_Adc_Reset(board,
05728                               &my_adc[adc_num]);
05729
05730     DM35424_Check_Result_test(result, "Error resetting ADC");
05731
05732     /*****
05733     Setup for 2nd round
05734     *****/
05735     for (clock = DM35424_CLK_SRC_GBL2; clock <= DM35424_CLK_SRC_GBL7; clock++) {
05736         result = DM35424_Adc_Set_Clock_Source_Global(board,
05737                                                     &my_adc[adc_num],
05738                                                     clock,
05739                                                     DM35424_ADC_CLK_EVENT_DISABLE);
05740         sprintf(message, "ADC%d: Could not set clock src GBL%d to DISABLE.",
05741               adc_num, clock);
05742         DM35424_Check_Result_test(result, message);
05743     }
05744
05745 // Test 9
05746
05747 // Set all of the global flags to various events of the adc_num
05748     result = DM35424_Adc_Set_Clock_Source_Global(board,
05749                                         &my_adc[adc_num],
05750                                         DM35424_CLK_SRC_GBL2,
05751                                         DM35424_ADC_CLK_EVENT_SAMPLE_TAKEN);
05752
05753     DM35424_Check_Result_test(result, "Error setting Clock Global Source");
05754
05755     result = DM35424_Adc_Set_Clock_Source_Global(board,
05756                                         &my_adc[adc_num],
05757                                         DM35424_CLK_SRC_GBL3,
05758                                         DM35424_ADC_CLK_EVENT_CHAN_THRESH);
05759
05760     DM35424_Check_Result_test(result, "Error setting Clock Global Source");
05761
05762     result = DM35424_Adc_Set_Clock_Source_Global(board,
05763                                         &my_adc[adc_num],
05764                                         DM35424_CLK_SRC_GBL6,
05765                                         DM35424_ADC_CLK_EVENT_SAMPLING_COMPLETE);
05766
05767     DM35424_Check_Result_test(result, "Error setting Clock Global Source");
05768
05769     result = DM35424_Adc_Set_Clock_Source_Global(board,
05770                                         &my_adc[adc_num],
05771                                         DM35424_CLK_SRC_GBL7,
05772                                         DM35424_ADC_CLK_EVENT_PACER_TICK);
05773
05774
05775

```

```

05776
05777     DM35424_Check_Result_test(result, "Error setting Clock Global Source");
05778
05779     // Setting DAC0 clock to be GBL_SRC2
05780     result = DM35424_Dac_Set_Clock_Src(board,
05781                                         &my_dac[DAC0],
05782                                         DM35424_CLK_SRC_GBL2);
05783
05784     DM35424_Check_Result_test(result, "Error setting clock of DAC0");
05785
05786     // Test 9
05787
05788     // Setting DAC1 clock to be GBL_SRC3
05789     result = DM35424_Dac_Set_Clock_Src(board,
05790                                         &my_dac[DAC1],
05791                                         DM35424_CLK_SRC_GBL3);
05792
05793     DM35424_Check_Result_test(result, "Error setting clock of DAC1");
05794
05795     // Setting DAC2 clock to be GBL_SRC4
05796     result = DM35424_Dac_Set_Clock_Src(board,
05797                                         &my_dac[DAC2],
05798                                         DM35424_CLK_SRC_GBL6);
05799
05800     DM35424_Check_Result_test(result, "Error setting clock of DAC2");
05801
05802     // Setting DAC3 clock to be GBL_SRC5
05803     result = DM35424_Dac_Set_Clock_Src(board,
05804                                         &my_dac[DAC3],
05805                                         DM35424_CLK_SRC_GBL7);
05806
05807     DM35424_Check_Result_test(result, "Error setting clock of DAC3");
05808
05809     for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD; dac_num++) {
05810
05811         result = DM35424_Dac_Reset(board,
05812                                     &my_dac[dac_num]);
05813
05814         DM35424_Check_Result_test(result, "Error resetting DAC");
05815
05816         result = DM35424_Dac_Start(board,
05817                                     &my_dac[dac_num]);
05818
05819         DM35424_Check_Result_test(result, "Error starting DAC");
05820     }
05821
05822     // Test 9
05823
05824     result = DM35424_Adc_Set_Stop_Trigger(board,
05825                                             &my_adc[adc_num],
05826                                             DM35424_CLK_SRC_NEVER);
05827
05828     DM35424_Check_Result_test(result, "Error setting stop trigger adc_num clock source");
05829
05830
05831
05832     if (adc_num % 2 == 0) {
05833         result = DM35424_Adc_Channel_Set_Low_Threshold(board,
05834                                                         &my_adc[adc_num],
05835                                                         DMA_CHAN0,
05836                                                         TEST_9_ADC_LOW_THRESH);
05837         DM35424_Check_Result_test(result, "Error setting low threshold.");
05838
05839         result = DM35424_Adc_Channel_Interrupt_Set_Config(board,
05840                                                            &my_adc[adc_num],
05841                                                            DMA_CHAN0,
05842                                                            DM35424_ADC_CHAN_INTR_LOW_THRESHOLD_MASK,
05843                                                            INTERRUPT_ENABLE);
05844
05845         DM35424_Check_Result_test(result, "Error enabling threshold interrupts.");
05846     }
05847     else {
05848
05849         result = DM35424_Adc_Channel_Interrupt_Set_Config(board,
05850                                                            &my_adc[adc_num],
05851                                                            DMA_CHAN0,
05852                                                            DM35424_ADC_CHAN_INTR_HIGH_THRESHOLD_MASK,
05853                                                            INTERRUPT_ENABLE);
05854
05855         DM35424_Check_Result_test(result, "Error enabling threshold interrupts.");
05856
05857     }
05858     // Test 9

```

```

05858
05859         result = DM35424_Adc_Channel_Set_High_Threshold(board,
05860                                                         &my_adc[adc_num],
05861                                                         DMA_CHAN0,
05862                                                         TEST_9_ADC_HIGH_THRESH);
05863         DM35424_Check_Result_test(result, "Error setting high threshold.");
05864     }
05865 }
05866
05867 // Before starting adc_num, check that all of the DAC counts are zero.
05868 // We exclude DAC3, though, because it is the PACER clock tick, which
05869 // happens regardless of ADC start.
05870 for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD - 1; dac_num++) {
05871     result = DM35424_Dac_Get_Conversion_Count(board,
05872                                               &my_dac[dac_num],
05873                                               &sample_counts);
05874
05875     DM35424_Check_Result_test(result, "Error getting sample counts.");
05876
05877     sprintf(message, "Expected conversion count of 0 for DAC%d, but received
05878 %d.\n",
05879             dac_num, sample_counts);
05880     DM35424_Check_Result_test(sample_counts != 0, message);
05881 }
05882
05883 result = DM35424_Adc_Start(board,
05884                             &my_adc[adc_num]);
05885
05886 DM35424_Check_Result_test(result, "Error starting adc_num");
05887
05888 // Give everything a chance to run.
05889 sleep(2);
05890 result = DM35424_Adc_Set_Stop_Trigger(board,
05891                                       &my_adc[adc_num],
05892                                       DM35424_CLK_SRC_IMMEDIATE);
05893
05894 DM35424_Check_Result_test(result, "Error setting stop trigger adc_num clock source");
05895
05896 // Test 9
05897
05898 // Give everything a chance to complete.
05899 sleep(1);
05900 fprintf(stdout, "    Checking clock triggers: SAMPLETAKEN ");
05901 result = DM35424_Dac_Get_Conversion_Count(board,
05902                                           &my_dac[DAC0],
05903                                           &sample_counts);
05904
05905 DM35424_Check_Result_test(result, "Error getting sample counts.");
05906
05907 sprintf(message, "Expected sample count > 100 DAC%d, but received %d.\n",
05908           0, sample_counts);
05909 DM35424_Check_Result_test(sample_counts < 10, message);
05910
05911 fprintf(stdout, "    THRESH ");
05912
05913 result = DM35424_Dac_Get_Conversion_Count(board,
05914                                           &my_dac[DAC1],
05915                                           &sample_counts);
05916
05917 DM35424_Check_Result_test(result, "Error getting sample counts.");
05918
05919 sprintf(message, "Expected sample count > 100 for DAC%d, but received %d.\n",
05920           1, sample_counts);
05921 DM35424_Check_Result_test(sample_counts < 10, message);
05922
05923 fprintf(stdout, "    COMPLETE ");
05924
05925 result = DM35424_Dac_Get_Conversion_Count(board,
05926                                           &my_dac[DAC2],
05927                                           &sample_counts);
05928
05929 DM35424_Check_Result_test(result, "Error getting sample counts.");
05930
05931 sprintf(message, "Expected sample count of 1 for DAC%d, but received %d.\n",
05932           0, sample_counts);
05933 DM35424_Check_Result_test(sample_counts != 1, message);
05934
05935 fprintf(stdout, "    PACER ");
05936
05937 // Test 9
05938
05939 result = DM35424_Dac_Get_Conversion_Count(board,
05940                                           &my_dac[DAC3],
05941                                           &sample_counts);

```

```

05942         DM35424_Check_Result_test(result, "Error getting sample counts.");
05943
05944         sprintf(message, "Expected sample count > 100 DAC%d, but received %d.\n",
05945                        0, sample_counts);
05946         DM35424_Check_Result_test(sample_counts < 10, message);
05947         for (clock = DM35424_CLK_SRC_GBL2; clock <= DM35424_CLK_SRC_GBL7; clock++) {
05948             result = DM35424_Adc_Set_Clock_Source_Global(board,
05949                                                         &my_adc[adc_num],
05950                                                         clock,
05951                                                         DM35424_ADC_CLK_EVENT_DISABLE);
05952             sprintf(message, "ADC%d: Could not set clock src GBL%d to DISABLE.",
05953                          adc_num, clock);
05954             DM35424_Check_Result_test(result, message);
05955         }
05956         fprintf(stdout, "\n");
05957
05958         /*****
05959         Testing INV clocks
05960         *****/
05961         result = DM35424_Adc_Channel_Interrupt_Set_Config(board,
05962                                                         &my_adc[adc_num],
05963                                                         DMA_CHAN0,
05964                                                         DM35424_ADC_CHAN_INTR_LOW_THRESHOLD_MASK |
05965                                                         DM35424_ADC_CHAN_INTR_HIGH_THRESHOLD_MASK,
05966                                                         INTERRUPT_DISABLE);
05967
05968         DM35424_Check_Result_test(result, "Error enabling threshold interrupts.");
05969
05970         // Test 9
05971         result = DM35424_Adc_Channel_Interrupt_Set_Config(board,
05972                                                         &my_adc[adc_num],
05973                                                         DMA_CHAN0,
05974                                                         DM35424_ADC_CHAN_INTR_LOW_THRESHOLD_MASK,
05975                                                         INTERRUPT_ENABLE);
05976
05977         DM35424_Check_Result_test(result, "Error enabling threshold interrupts.");
05978
05979         result = DM35424_Adc_Set_Pre_Trigger_Samples(board,
05980                                                         &my_adc[adc_num],
05981                                                         0);
05982         DM35424_Check_Result_test(result, "Error setting pre-trigger sample count for ADC.");
05983
05984         result = DM35424_Adc_Set_Post_Stop_Samples(board,
05985                                                         &my_adc[adc_num],
05986                                                         0);
05987         DM35424_Check_Result_test(result, "Error setting post-stop sample count for ADC.");
05988
05989         for (clock = 0; clock <= (DM35424_CLK_SRC_GBL7 - DM35424_CLK_SRC_GBL2); clock++) {
05990             fprintf(stdout, "    Testing CLK SRC GBL%d\n", clock + 2);
05991
05992             result = DM35424_Adc_Reset(board,
05993                                       &my_adc[adc_num]);
05994
05995             DM35424_Check_Result_test(result, "Error resetting ADC");
05996
05997             result = DM35424_Adc_Channel_Interrupt_Clear_Status(board,
06000                                                         &my_adc[adc_num],
06001                                                         CHANNEL_0,
06002                                                         0x3);
06003             DM35424_Check_Result_test(result, "doh");
06004
06005             result = DM35424_Adc_Set_Stop_Trigger(board,
06006                                                         &my_adc[adc_num],
06007                                                         DM35424_CLK_SRC_NEVER);
06008
06009             // Test 9
06010             DM35424_Check_Result_test(result, "Error setting stop trigger adc_num clock
06011 source");
06012
06013             for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD; dac_num++) {
06014                 result = DM35424_Dac_Reset(board,
06015                                             &my_dac[dac_num]);
06016
06017                 DM35424_Check_Result_test(result, "Error resetting DAC");
06018
06019                 if (clock < 2) {

```



```

06022         result = DM35424_Adc_Set_Start_Trigger(board,
06023                                                     &my_adc[adc_num],
06024                                                     DM35424_CLK_SRC_GBL4);
06025
06026         DM35424_Check_Result_test(result, "Error setting start trigger
adc_num clock source");
06027
06028         result = DM35424_Dac_Set_Start_Trigger(board,
06029                                                     &my_dac[dac_num],
06030                                                     DM35424_CLK_SRC_GBL4);
06031
06032         DM35424_Check_Result_test(result, "Error setting start trigger
for DAC");
06033
06034         result = DM35424_Dac_Set_Stop_Trigger(board,
06035                                                     &my_dac[dac_num],
06036                                                     DM35424_CLK_SRC_GBL5);
06037
06038         DM35424_Check_Result_test(result, "Error setting stop trigger
for DAC");
06039     }
06040     else {
06041         result = DM35424_Adc_Set_Start_Trigger(board,
06042                                                     &my_adc[adc_num],
06043                                                     DM35424_CLK_SRC_GBL2);
06044
06045         DM35424_Check_Result_test(result, "Error setting start trigger
adc_num clock source");
06046
06047         // Test 9
06048
06049         result = DM35424_Dac_Set_Start_Trigger(board,
06050                                                     &my_dac[dac_num],
06051                                                     DM35424_CLK_SRC_GBL2);
06052
06053         DM35424_Check_Result_test(result, "Error setting start trigger
for DAC");
06054
06055         result = DM35424_Dac_Set_Stop_Trigger(board,
06056                                                     &my_dac[dac_num],
06057                                                     DM35424_CLK_SRC_GBL3);
06058
06059         DM35424_Check_Result_test(result, "Error setting stop trigger
for DAC");
06060     }
06061 }
06062 }
06063
06064 // Set all of the global flags to various events of the adc_num
06065 result = DM35424_Adc_Set_Clock_Source_Global(board,
06066                                               &my_adc[adc_num],
06067                                               DM35424_CLK_SRC_GBL2 + clock,
06068                                               DM35424_ADC_CLK_EVENT_CHAN_THRESH);
06069
06070
06071
06072         DM35424_Check_Result_test(result, "Error setting Clock Global Source");
06073
06074         // Setting DAC0 clock to be GBL_SRC2
06075         result = DM35424_Dac_Set_Clock_Src(board,
06076                                             &my_dac[DAC0],
06077                                             DM35424_CLK_SRC_IMMEDIATE);
06078
06079         DM35424_Check_Result_test(result, "Error setting clock of DAC0");
06080
06081         // Setting DAC1 clock to be GBL_SRC3
06082         result = DM35424_Dac_Set_Clock_Src(board,
06083                                             &my_dac[DAC1],
06084                                             DM35424_CLK_SRC_GBL2 + clock);
06085
06086         DM35424_Check_Result_test(result, "Error setting clock of DAC1");
06087
06088         // Test 9
06089
06090         // Setting DAC2 clock to be GBL_SRC4
06091         result = DM35424_Dac_Set_Clock_Src(board,
06092                                             &my_dac[DAC2],
06093                                             DM35424_CLK_SRC_GBL2_INV + clock);
06094
06095         DM35424_Check_Result_test(result, "Error setting clock of DAC2");
06096
06097         // Setting DAC2 clock to be GBL_SRC4
06098         result = DM35424_Dac_Set_Clock_Src(board,
06099                                             &my_dac[DAC3],
06100                                             DM35424_CLK_SRC_IMMEDIATE);

```

```

06100
06101                 DM35424_Check_Result_test(result, "Error setting clock of DAC2");
06102
06103
06104                 if (clock < 2) {
06105                     // Set all of the global flags to various events of the adc_num
06106                     result = DM35424_Dac_Set_Clock_Source_Global(board,
06107                                                                    &my_dac[DAC3],
06108                                                                    DM35424_CLK_SRC_GBL4,
06109
06110 DM35424_DAC_CLK_EVENT_START_TRIG);
06110
06111                     // Set all of the global flags to various events of the adc_num
06112                     result = DM35424_Dac_Set_Clock_Source_Global(board,
06113                                                                    &my_dac[DAC3],
06114                                                                    DM35424_CLK_SRC_GBL5,
06115
06116 DM35424_DAC_CLK_EVENT_STOP_TRIG);
06116                 }
06117                 else {
06118
06119                     // Set all of the global flags to various events of the adc_num
06120                     result = DM35424_Dac_Set_Clock_Source_Global(board,
06121                                                                    &my_dac[DAC3],
06122                                                                    DM35424_CLK_SRC_GBL2,
06123
06124 DM35424_DAC_CLK_EVENT_START_TRIG);
06124
06125                     // Set all of the global flags to various events of the adc_num
06126                     result = DM35424_Dac_Set_Clock_Source_Global(board,
06127                                                                    &my_dac[DAC3],
06128                                                                    DM35424_CLK_SRC_GBL3,
06129
06130 DM35424_DAC_CLK_EVENT_STOP_TRIG);
06130
06131                 }
06132
06133 // Test 9
06133
06134                 for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD - 1; dac_num++) {
06135
06136
06137
06138                     result = DM35424_Dac_Start(board,
06139                                                                    &my_dac[dac_num]);
06140
06141                     DM35424_Check_Result_test(result, "Error starting DAC");
06142
06143                     result = DM35424_Dac_Get_Conversion_Count(board,
06144                                                                    &my_dac[dac_num],
06145                                                                    &sample_counts);
06146
06147                     DM35424_Check_Result_test(result, "Error getting sample counts.");
06148
06149                     sprintf(message, "INV Clock testing: Expected conversion count of 0
06150 for DAC%d, but received %d.\n",
06151                                dac_num, sample_counts);
06151                     DM35424_Check_Result_test(sample_counts != 0, message);
06152
06153                 }
06154
06155                 result = DM35424_Adc_Channel_Set_Low_Threshold(board,
06156                                                                    &my_adc[adc_num],
06157                                                                    DMA_CHAN0,
06158                                                                    TEST_9_ADC_HIGH_THRESH);
06159                 DM35424_Check_Result_test(result, "Error setting low threshold.");
06160
06161 // Test 9
06161
06162                 result = DM35424_Adc_Start(board,
06163                                                                    &my_adc[adc_num]);
06164
06165                 DM35424_Check_Result_test(result, "Error starting adc_num");
06166
06167                 result = DM35424_Dac_Set_Start_Trigger(board,
06168                                                                    &my_dac[DAC3],
06169                                                                    DM35424_CLK_SRC_IMMEDIATE);
06170
06171                 DM35424_Check_Result_test(result, "Error setting start trigger for DAC");
06172
06173                 result = DM35424_Dac_Set_Stop_Trigger(board,
06174                                                                    &my_dac[DAC3],
06175                                                                    DM35424_CLK_SRC_NEVER);
06176
06177                 DM35424_Check_Result_test(result, "Error setting stop trigger for DAC");
06178
06179                 result = DM35424_Dac_Start(board,
06180                                                                    &my_dac[DAC3],
06181                                                                    DM35424_CLK_SRC_NEVER);
06182
06183                 DM35424_Check_Result_test(result, "Error setting stop trigger for DAC");
06184
06185                 result = DM35424_Dac_Start(board,

```

```

06186                                     &my_dac[DAC3]);
06187     DM35424_Check_Result_test(result, "Error starting DAC3");
06188
06189
06190     // Give everything a chance to run.
06191     sleep(1);
06192
06193     result = DM35424_Adc_Channel_Set_Low_Threshold(board,
06194                                                     &my_adc[adc_num],
06195                                                     CHANNEL_0,
06196                                                     TEST_9_ADC_LOW_THRESH);
06197     DM35424_Check_Result_test(result, "Error setting low threshold.");
06198
06199     sleep(1);
06200
06201     // Test 9
06202
06203     result = DM35424_Dac_Set_Stop_Trigger(board,
06204                                             &my_dac[DAC3],
06205                                             DM35424_CLK_SRC_IMMEDIATE);
06206
06207     DM35424_Check_Result_test(result, "Error setting stop trigger for DAC");
06208
06209     result = DM35424_Dac_Reset(board,
06210                                &my_dac[DAC3]);
06211     DM35424_Check_Result_test(result, "Error resetting DAC3");
06212
06213     // now check the counts of DAC0, 1, and 2.
06214     result = DM35424_Dac_Get_Conversion_Count(board,
06215                                                &my_dac[DAC0],
06216                                                &total_count);
06217
06218     DM35424_Check_Result_test(result, "Error getting count for DAC0.");
06219
06220     result = DM35424_Dac_Get_Conversion_Count(board,
06221                                                &my_dac[DAC1],
06222                                                &normal_count);
06223
06224     DM35424_Check_Result_test(result, "Error getting count for DAC1.");
06225
06226     result = DM35424_Dac_Get_Conversion_Count(board,
06227                                                &my_dac[DAC2],
06228                                                &inverted_count);
06229
06230     DM35424_Check_Result_test(result, "Error getting count for DAC2.");
06231
06232
06233
06234     // Test 9
06235
06236     sprintf(message, "Counts not as expected: Total: %u Normal: %u Inverted: %u\n",
06237              (total: %u, diff: %d)\n",
06238              total_count, normal_count, inverted_count,
06239              (normal_count + inverted_count),
06240              total_count - (normal_count + inverted_count));
06241     DM35424_Check_Result_test((total_count > (normal_count + inverted_count + 1))
06242                               ||
06243                               (total_count < (normal_count + inverted_count - 1)) ||
06244                               (total_count < 100) ||
06245                               (normal_count < (total_count * .3)) ||
06246                               (inverted_count < (total_count * .3)),
06247                               message);
06248
06249     fprintf(stdout, "    Total: %u    Normal: %u    Inverted: %u    PASS\n",
06250             total_count, normal_count, inverted_count);
06251
06252
06253
06254     result = DM35424_Adc_Set_Clock_Source_Global(board,
06255                                                    &my_adc[adc_num],
06256                                                    DM35424_CLK_SRC_GBL2 + clock,
06257                                                    DM35424_ADC_CLK_EVENT_DISABLE);
06258     sprintf(message, "ADC%d: Could not set clock src GBL%d to DISABLE.",
06259             adc_num, clock);
06260     DM35424_Check_Result_test(result, message);
06261
06262
06263     /*****
06264     * Testing Thresh/Inv Thresh clock source
06265     *****/
06266     fprintf(stdout, "\n    Testing Channel Threshold/Inv Clock...\n");
06267     result = DM35424_Adc_Reset(board,
06268                                &my_adc[adc_num]);
06269
06270     DM35424_Check_Result_test(result, "Error resetting ADC");
06271

```

```

06272         result = DM35424_Adc_Set_Pre_Trigger_Samples(board,
06273                                                         &my_adc[adc_num],
06274                                                         2);
06275         DM35424_Check_Result_test(result, "Error setting pre-trigger sample count for ADC.");
06276
06277         // Test 9
06278         result = DM35424_Adc_Channel_Interrupt_Set_Config(board,
06279                                                         &my_adc[adc_num],
06280                                                         CHANNEL_0,
06281                                                         DM35424_ADC_CHAN_INTR_LOW_THRESHOLD_MASK,
06282                                                         INTERRUPT_ENABLE);
06283
06284         DM35424_Check_Result_test(result, "Error enabling threshold interrupts.");
06285         result = DM35424_Adc_Set_Stop_Trigger(board,
06286                                                         &my_adc[adc_num],
06287                                                         DM35424_CLK_SRC_NEVER);
06288
06289         DM35424_Check_Result_test(result, "Error setting stop trigger adc_num clock source");
06290
06291         result = DM35424_Adc_Set_Start_Trigger(board,
06292                                                         &my_adc[adc_num],
06293                                                         DM35424_CLK_SRC_CHAN_THRESH);
06294
06295         DM35424_Check_Result_test(result, "Error setting start trigger adc_num clock source");
06296
06297         result = DM35424_Adc_Channel_Set_Low_Threshold(board,
06298                                                         &my_adc[adc_num],
06299                                                         CHANNEL_0,
06300                                                         TEST_9_ADC_HIGH_THRESH);
06301
06302         DM35424_Check_Result_test(result, "Error setting low threshold.");
06303
06304         result = DM35424_Adc_Start(board,
06305                                                         &my_adc[adc_num]);
06306
06307         DM35424_Check_Result_test(result, "Error starting ADC.");
06308
06309         printf("        Starting ADC, checking mode/status....");
06310         sleep(1);
06311
06312         //There should be no counts, the ADC should not be running because
06313         // the threshold should not have crossed.
06314
06315         // Test 9
06316
06317         result = DM35424_Adc_Get_Mode_Status(board,
06318                                                         &my_adc[adc_num],
06319                                                         &mode_status);
06320
06321         DM35424_Check_Result_test(result, "Error getting mode/status from ADC.");
06322
06323         sprintf(message, "ADC%d Expected mode status of 0x%x, but got 0x%x.",
06324                 adc_num, MODE_STATUS(DM35424_ADC_MODE_GO_SINGLE_SHOT,
06325                                     DM35424_ADC_STAT_WAITING_START_TRIG),
06326                 mode_status);
06327
06328         DM35424_Check_Result_test(mode_status != MODE_STATUS(DM35424_ADC_MODE_GO_SINGLE_SHOT,
06329                                                             DM35424_ADC_STAT_WAITING_START_TRIG),
06330         message);
06331
06332         result = DM35424_Adc_Channel_Set_Low_Threshold(board,
06333                                                         &my_adc[adc_num],
06334                                                         CHANNEL_0,
06335                                                         TEST_9_ADC_LOW_THRESH);
06336
06337         DM35424_Check_Result_test(result, "Error setting low threshold.");
06338         printf("success.\n        Adjusting threshold so that ADC%d should start.\n",
06339         adc_num);
06340
06341         sleep(1);
06342         printf("        Checking that ADC%d has non-zero count (it started)....", adc_num);
06343         result = DM35424_Adc_Get_Sample_Count(board,
06344                                                         &my_adc[adc_num],
06345                                                         &total_count);
06346
06347         DM35424_Check_Result_test(result, "Error getting count for ADC.");
06348         result = DM35424_Adc_Get_Mode_Status(board,
06349                                                         &my_adc[adc_num],
06350                                                         &mode_status);
06351
06352         DM35424_Check_Result_test(result, "Error getting mode/status from ADC.");
06353
06354         result = DM35424_Adc_Channel_Get_Last_Sample(board,
06355                                                         &my_adc[adc_num],
06356                                                         CHANNEL_0,
06357                                                         &my_value);

```

```

06355
06356         DM35424_Check_Result_test(result, "Error getting last sample from ADC.");
06357
06358     // Test 9
06359     sprintf(message, "Expected non-zero count for ADC%d after start trigger. Mode/Status:
0x%x Last Sample: %d",
06360                adc_num, mode_status, my_value);
06361
06362     DM35424_Check_Result_test(total_count == 0, message);
06363
06364     result = DM35424_Adc_Set_Stop_Trigger(board,
06365                                           &my_adc[adc_num],
06366                                           DM35424_CLK_SRC_CHAN_THRESH_INV);
06367
06368     DM35424_Check_Result_test(result, "Error setting stop trigger adc_num clock source");
06369     printf("success.\n        Checking that ADC%d is still running....", adc_num);
06370     sleep(1);
06371
06372     result = DM35424_Adc_Get_Mode_Status(board,
06373                                           &my_adc[adc_num],
06374                                           &mode_status);
06375
06376     DM35424_Check_Result_test(result, "Error getting mode/status from ADC.");
06377
06378     sprintf(message, "Expected ADC%d mode:status of 0x%x, but got 0x%x.",
06379                adc_num,
06380                MODE_STATUS(DM35424_ADC_MODE_GO_SINGLE_SHOT, DM35424_ADC_STAT_SAMPLING),
06381                mode_status);
06382
06383     DM35424_Check_Result_test(mode_status != MODE_STATUS(DM35424_ADC_MODE_GO_SINGLE_SHOT,
06384 DM35424_ADC_STAT_SAMPLING),
06385                               message);
06386
06387     for (channel = 0; channel < my_adc[adc_num].num_dma_channels; channel++) {
06388         result = DM35424_Adc_Channel_Set_Low_Threshold(board,
06389                                                         &my_adc[adc_num],
06390                                                         channel,
06391                                                         TEST_9_ADC_HIGH_THRESH);
06392         DM35424_Check_Result_test(result, "Error setting low threshold.");
06393     }
06394
06395     // Test 9
06396     result = DM35424_Adc_Channel_Set_High_Threshold(board,
06397                                                       &my_adc[adc_num],
06398                                                       channel,
06399                                                       TEST_9_ADC_LOW_THRESH);
06400     DM35424_Check_Result_test(result, "Error setting high threshold.");
06401
06402     result = DM35424_Adc_Channel_Interrupt_Clear_Status(board,
06403                                                         &my_adc[adc_num],
06404                                                         channel,
06405                                                         0x3);
06406     DM35424_Check_Result_test(result, "doh");
06407
06408     }
06409     sleep(1);
06410     printf("success.\n        Changed threshold value, checking that ADC has
06411 stopped....");
06412     result = DM35424_Adc_Get_Mode_Status(board,
06413                                           &my_adc[adc_num],
06414                                           &mode_status);
06415
06416     DM35424_Check_Result_test(result, "Error getting mode/status from ADC.");
06417
06418     sprintf(message, "Expected ADC%d mode:status of 0x%x, but got 0x%x.",
06419                adc_num,
06420                MODE_STATUS(DM35424_ADC_MODE_GO_SINGLE_SHOT, DM35424_ADC_STAT_DONE),
06421                mode_status);
06422
06423     DM35424_Check_Result_test(mode_status != MODE_STATUS(DM35424_ADC_MODE_GO_SINGLE_SHOT,
06424 DM35424_ADC_STAT_DONE),
06425                               message);
06426
06427     printf("success!\n\n");
06428
06429     }
06430
06431     // Test 9
06432     for (adc_num = 0; adc_num < DM35424_NUM_ADC_ON_BOARD; adc_num++) {
06433         result = DM35424_Adc_Reset(board,
06434                                     &my_adc[adc_num]);
06435     }

```

```

06438
06439         DM35424_Check_Result_test(result, "Error starting adc_num");
06440
06441         result = DM35424_Adc_Set_Pre_Trigger_Samples(board,
06442                                                         &my_adc[adc_num],
06443                                                         0);
06444         DM35424_Check_Result_test(result, "Error setting pre-trigger sample count for ADC.");
06445
06446         result = DM35424_Adc_Set_Post_Stop_Samples(board,
06447                                                         &my_adc[adc_num],
06448                                                         0);
06449         DM35424_Check_Result_test(result, "Error setting post-stop sample count for ADC.");
06450
06451         for (clock = DM35424_CLK_SRC_GBL2; clock <= DM35424_CLK_SRC_GBL7; clock++) {
06452             result = DM35424_Adc_Set_Clock_Source_Global(board,
06453                                                         &my_adc[adc_num],
06454                                                         clock,
06455
DM35424_ADC_CLK_EVENT_DISABLE);
06456             sprintf(message, "ADC%d: Could not set clock source GBL%d to DISABLE.",
06457                             adc_num, clock);
06458             DM35424_Check_Result_test(result, message);
06459         }
06460     }
06461
06462     for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD; dac_num++) {
06463         fprintf(stdout, "Testing clocks (driving) with DAC %d =====\n", dac_num);
06464
06465         fprintf(stdout, "    Setting up....");
06466         result = DM35424_Dac_Reset(board,
06467                                     &my_dac[dac_num]);
06468
06469         DM35424_Check_Result_test(result, "Error resetting ADC0");
06470
06471         result = DM35424_Adc_Reset(board,
06472                                     &my_adc[ADC0]);
06473
06474         DM35424_Check_Result_test(result, "Error resetting ADC0");
06475
06476         result = DM35424_Adc_Reset(board,
06477                                     &my_adc[ADC1]);
06478
06479         DM35424_Check_Result_test(result, "Error resetting ADC1");
06480
06481         for (clock = DM35424_CLK_SRC_GBL2; clock <= DM35424_CLK_SRC_GBL7; clock++) {
06482             result = DM35424_Dac_Set_Clock_Source_Global(board,
06483                                                         &my_dac[dac_num],
06484                                                         clock,
06485
DM35424_DAC_CLK_EVENT_DISABLE);
06486             sprintf(message, "DAC%d: Could not set clock source GBL%d to DISABLE.",
06487                             dac_num, clock);
06488             DM35424_Check_Result_test(result, message);
06489         }
06490
06491         result = DM35424_Dac_Set_Clock_Src(board,
06492                                             &my_dac[dac_num],
06493                                             DM35424_CLK_SRC_IMMEDIATE);
06494
06495         DM35424_Check_Result_test(result, "Error setting clock of DAC");
06496
06497         result = DM35424_Adc_Set_Start_Trigger(board,
06498                                                 &my_adc[ADC0],
06499                                                 DM35424_CLK_SRC_GBL2);
06500
06501         DM35424_Check_Result_test(result, "Error setting start trigger adc_num clock source");
06502
06503         result = DM35424_Adc_Set_Stop_Trigger(board,
06504                                                 &my_adc[ADC0],
06505                                                 DM35424_CLK_SRC_GBL3);
06506
06507         DM35424_Check_Result_test(result, "Error setting stop trigger adc_num clock source");
06508
06509         result = DM35424_Adc_Set_Start_Trigger(board,
06510                                                 &my_adc[ADC1],
06511                                                 DM35424_CLK_SRC_GBL4);
06512
06513         DM35424_Check_Result_test(result, "Error setting start trigger adc_num clock source");
06514
06515         result = DM35424_Adc_Set_Stop_Trigger(board,
06516                                                 &my_adc[ADC1],
06517                                                 DM35424_CLK_SRC_GBL5);
06518
06519         DM35424_Check_Result_test(result, "Error setting stop trigger adc_num clock source");
06520
06521         DM35424_Check_Result_test(result, "Error setting stop trigger adc_num clock source");
06522

```

```

06523         result = DM35424_Dac_Set_Start_Trigger(board,
06524             &my_dac[dac_num],
06525             DM35424_CLK_SRC_IMMEDIATE);
06526
06527         DM35424_Check_Result_test(result, "Error setting start trigger DAC clock source");
06528
06529         result = DM35424_Dac_Set_Stop_Trigger(board,
06530             &my_dac[dac_num],
06531             DM35424_CLK_SRC_NEVER);
06532
06533         DM35424_Check_Result_test(result, "Error setting stop trigger DAC clock source");
06534
06535         result = DM35424_Dac_Set_Clock_Source_Global(board,
06536             &my_dac[dac_num],
06537             DM35424_CLK_SRC_GBL2,
06538             DM35424_DAC_CLK_EVENT_START_TRIG);
06539
06540         DM35424_Check_Result_test(result, "Error setting DAC clock source global.");
06541
06542         result = DM35424_Dac_Set_Clock_Source_Global(board,
06543             &my_dac[dac_num],
06544             DM35424_CLK_SRC_GBL3,
06545             DM35424_DAC_CLK_EVENT_CONVERSION_SENT);
06546
06547         DM35424_Check_Result_test(result, "Error setting DAC clock source global.");
06548
06549         result = DM35424_Dac_Set_Clock_Source_Global(board,
06550             &my_dac[dac_num],
06551             DM35424_CLK_SRC_GBL4,
06552             DM35424_DAC_CLK_EVENT_STOP_TRIG);
06553
06554         DM35424_Check_Result_test(result, "Error setting DAC clock source global.");
06555
06556         result = DM35424_Dac_Set_Clock_Source_Global(board,
06557             &my_dac[dac_num],
06558             DM35424_CLK_SRC_GBL5,
06559             DM35424_DAC_CLK_EVENT_CONV_COMPL);
06560
06561         DM35424_Check_Result_test(result, "Error setting DAC clock source global.");
06562
06563         result = DM35424_Dac_Set_Post_Stop_Conversion_Count(board,
06564             &my_dac[dac_num],
06565             TEST_9_DAC_POST_STOP_CONV);
06566
06567         DM35424_Check_Result_test(result, "Error setting DAC post stop conversion count");
06568         fprintf(stdout, "success.\n    Starting ADCs....");
06569
06570         result = DM35424_Adc_Start(board,
06571             &my_adc[ADC0]);
06572
06573         DM35424_Check_Result_test(result, "Error starting adc_num");
06574
06575         result = DM35424_Adc_Start(board,
06576             &my_adc[ADC1]);
06577
06578         DM35424_Check_Result_test(result, "Error starting adc_num");
06579
06580         DM35424_Micro_Sleep(100000);
06581
06582         // Check that ADC is waiting for the start trigger.
06583         DM35424_Adc_Get_Mode_Status(board,
06584             &my_adc[ADC0],
06585             &mode_status);
06586
06587         sprintf(message, "Expected mode/status of 0x22 for ADC0, but found 0x%x\n",
mode_status);
06588
06589         DM35424_Check_Result_test(mode_status != 0x22, message);
06590
06591         fprintf(stdout, "success.\n    Checking ADC0 has not started yet.....");
06592         DM35424_Adc_Get_Mode_Status(board,
06593             &my_adc[ADC1],
06594             &mode_status);
06595
06596         sprintf(message, "Expected mode/status of 0x22 for ADC1, but found 0x%x\n",
mode_status);
06597
06598         DM35424_Check_Result_test(mode_status != 0x22, message);
06599         fprintf(stdout, "success.\n    Checking ADC1 has not started yet.....");
06600
06601         // Start the DAC and confirm the clock events were seen.
06602         result = DM35424_Dac_Start(board,
06603             &my_dac[dac_num]);
06604
06605         DM35424_Check_Result_test(result, "Error starting DAC");
06606         fprintf(stdout, "success.\n    Starting DAC.....");

```

```

06607
06608
06609 // Check that DAC has started.
06610 DM35424_Dac_Get_Mode_Status(board,
06611                             &my_dac[dac_num],
06612                             &mode_status);
06613
06614 sprintf(message, "Expected mode/status of 0x32 for DAC%d, but found 0x%x\n", dac_num,
mode_status);
06615
06616 DM35424_Check_Result_test(mode_status != 0x32, message);
06617
06618
06619 fprintf(stdout, "success.\n    Checking ADC0 has had both triggers: ");
06620
06621 DM35424_Micro_Sleep(200000);
06622
06623 // Check that ADC completed
06624 DM35424_Adc_Get_Mode_Status(board,
06625                             &my_adc[ADC0],
06626                             &mode_status);
06627
06628 sprintf(message, "Expected mode/status of 0x72 for ADC0, but found 0x%x\n",
mode_status);
06629
06630
06631 DM35424_Check_Result_test(mode_status != 0x72, message);
06632
06633 fprintf(stdout, "    [START]    [CONVERSION]");
06634
06635 result = DM35424_Dac_Set_Stop_Trigger(board,
06636                                       &my_dac[dac_num],
06637                                       DM35424_CLK_SRC_IMMEDIATE);
06638
06639 DM35424_Check_Result_test(result, "Error setting stop trigger DAC clock source");
06640
06641 DM35424_Micro_Sleep(200000);
06642
06643 // Check that ADC completed
06644 DM35424_Adc_Get_Mode_Status(board,
06645                             &my_adc[ADC0],
06646                             &mode_status);
06647
06648 sprintf(message, "Expected mode/status of 0x72 for ADC1, but found 0x%x\n",
mode_status);
06649
06650
06651 DM35424_Check_Result_test(mode_status != 0x72, message);
06652 fprintf(stdout, "..... success.\n    Checking ADC1 has had both triggers: ");
06653
06654 fprintf(stdout, "    [STOP]    [POST-CONV]..... success.\n");
06655
06656 result = DM35424_Adc_Reset(board,
06657                             &my_adc[ADC0]);
06658
06659 DM35424_Check_Result_test(result, "Error resetting ADC0");
06660
06661 result = DM35424_Dac_Reset(board,
06662                             &my_dac[dac_num]);
06663
06664 DM35424_Check_Result_test(result, "Error resetting DAC");
06665
06666 result = DM35424_Dac_Set_Clock_Source_Global(board,
06667                                             &my_dac[dac_num],
06668                                             DM35424_CLK_SRC_GBL2,
06669                                             DM35424_DAC_CLK_EVENT_CHAN_MARKER);
06670
06671 DM35424_Check_Result_test(result, "Error setting DAC clock source global.");
06672
06673 result = DM35424_Adc_Start(board,
06674                             &my_adc[ADC0]);
06675
06676 DM35424_Check_Result_test(result, "Error starting adc_num");
06677
06678 result = DM35424_Dac_Channel_Set_Marker_Config(board,
06679                                             &my_dac[dac_num],
06680                                             CHANNEL_0,
06681                                             0x01);
06682
06683 DM35424_Check_Result_test(result, "Error setting marker configuration.");
06684 fprintf(stdout, "    Resetting ADC0 for next test....");
06685
06686 DM35424_Micro_Sleep(200000);
06687
06688 // Check that ADC is waiting for the start trigger.
06689 DM35424_Adc_Get_Mode_Status(board,
06690                             &my_adc[ADC0],
06691                             &mode_status);

```



```

06691
06692         sprintf(message, "Expected mode/status of 0x22 for ADC0, but found 0x%x\n",
mode_status);
06693
06694         DM35424_Check_Result_test(mode_status != 0x22, message);
06695         fprintf(stdout, "success.\n Writing marker value to DAC%d\n Checking that ADC0
started....", dac_num);
06696
06697         result = DM35424_Dac_Set_Last_Conversion(board,
06698                                                     &my_dac[dac_num],
06699                                                     CHANNEL_0,
06700                                                     0,
06701                                                     0x1000);
06702
06703         DM35424_Micro_Sleep(100000);
06704
06705         // Check that ADC is waiting for the start trigger.
06706         DM35424_Adc_Get_Mode_Status(board,
06707                                     &my_adc[ADC0],
06708                                     &mode_status);
06709
06710         sprintf(message, "Expected mode/status of 0x32 for ADC0, but found 0x%x\n",
mode_status);
06711
06712         DM35424_Check_Result_test(mode_status != 0x32, message);
06713
06714         fprintf(stdout, " [CHAN MARKER] .....success.\n");
06715
06716         result = DM35424_Dac_Channel_Set_Marker_Config(board,
06717                                                         &my_dac[dac_num],
06718                                                         CHANNEL_0,
06719                                                         0x00);
06720         DM35424_Check_Result_test(result, "Error setting marker configuration.");
06721     }
06722
06723
06724     fprintf(stdout, "\n");
06725
06726     printf("\n\nClosing Board....");
06727     result = DM35424_Board_Close(board);
06728
06729     DM35424_Check_Result_test(result, "Error closing board.");
06730 }
06731
06732
06733
06734
06735
06736 /*****
06737
06738 TEST 10
06739
06740 *****/
06740
06741 void run_test_10(unsigned int minor)
06742 {
06743     int result;
06744     unsigned int dac_num, channel;
06745     unsigned int index, local_int_count = 0;
06746     char message[200];
06747     uint32_t sample_counts = 0;
06748     uint8_t marker_status;
06749     uint8_t marker_config;
06750     uint16_t interrupt_status;
06751     int32_t *buffer;
06752     uint8_t mode_status;
06753
06754     interrupts_expected = 0;
06755
06756
06757     fprintf(stdout, "\n\n-----\n");
06758     fprintf(stdout, "Test 10: Testing Channel Markers.\n\n");
06759
06760     printf("Opening board....");
06761     result = DM35424_Board_Open(minor, &board);
06762
06763     DM35424_Check_Result_test(result, "Could not open board");
06764     printf("success.\nResetting board....");
06765     result = DM35424_Gbc_Board_Reset(board);
06766
06767     DM35424_Check_Result_test(result, "Could not reset board");
06768
06769     fprintf(stdout, "success.\n");
06770

```

```

06771     open_dacs();
06772
06773     initialize_dacs(20);
06774
06775     open_adcs();
06776
06777     initialize_adcs(TEST_10_ADC_CONVERSION_RATE);
06778
06779     set_isr();
06780     interrupt_count = 0;
06781
06782     fprintf(stdout, "Setup complete.\n");
06783
// Test 10
06784     result = DM35424_Adc_Set_Start_Trigger(board,
06785                                           &my_adc[ADC0],
06786                                           DM35424_CLK_SRC_GBL2);
06787
06788     DM35424_Check_Result_test(result, "Error setting start trigger adc_num clock source");
06789
06790
06791     result = DM35424_Adc_Set_Stop_Trigger(board,
06792                                           &my_adc[ADC0],
06793                                           DM35424_CLK_SRC_NEVER);
06794
06795     DM35424_Check_Result_test(result, "Error setting stop trigger adc_num clock source");
06796     result = DM35424_Adc_Set_Pre_Trigger_Samples(board,
06797                                                  &my_adc[ADC0],
06798                                                  0);
06799     DM35424_Check_Result_test(result, "Error setting pre-trigger sample count for ADC.");
06800
06801     result = DM35424_Adc_Set_Post_Stop_Samples(board,
06802                                                  &my_adc[ADC0],
06803                                                  0);
06804     DM35424_Check_Result_test(result, "Error setting post-stop sample count for ADC.");
06805
06806
06807     buffer = (int *)malloc(DAC_BUFFER_SIZE_BYTES);
06808
06809     DM35424_Check_Result_test(buffer == NULL, "Error allocating space for buffer.");
06810
06811
06812     result = DM35424_Generate_Signal_Data(DM35424_SINE_WAVE,
06813                                           buffer,
06814                                           DAC_SAMPLE_SIZE,
06815                                           28000,
06816                                           -28000,
06817                                           0,
06818                                           0x0000FFFF);
06819
06820     DM35424_Check_Result_test(result, "Error trying to generate data for the DAC.");
06821
// Test 10
06822
06823     for (index = 0; index < 8; index++) {
06824         buffer[index * (DAC_SAMPLE_SIZE / 8)] |= (0x01000000 « index);
06825     }
06826
06827
06828     for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD; dac_num++) {
06829
06830         result = DM35424_Dac_Set_Start_Trigger(board,
06831                                                  &my_dac[dac_num],
06832                                                  DM35424_CLK_SRC_IMMEDIATE);
06833
06834         DM35424_Check_Result_test(result, "Error setting start trigger for DAC");
06835
06836
06837         result = DM35424_Dac_Set_Stop_Trigger(board,
06838                                                  &my_dac[dac_num],
06839                                                  DM35424_CLK_SRC_NEVER);
06840
06841         DM35424_Check_Result_test(result, "Error setting stop trigger ");
06842
06843         for (channel = 0; channel < my_dac[dac_num].num_dma_channels; channel++) {
06844
06845             result = DM35424_Dma_Stop(board,
06846                                       &my_dac[dac_num],
06847                                       channel);
06848
06849             DM35424_Check_Result_test(result, "Error stopping DMA.");
06850
06851
06852
// Test 10
06853
06854     result = DM35424_Dma_Initialize(board,

```

```

06855                                     &my_dac[dac_num],
06856                                     channel,
06857                                     NUM_BUFFERS_1,
06858                                     DAC_BUFFER_SIZE_BYTES);
06859
06860     DM35424_Check_Result_test(result, "Error initializing DMA");
06861
06862     result = DM35424_Dma_Setup(board,
06863                                &my_dac[dac_num],
06864                                channel,
06865                                DM35424_DMA_SETUP_DIRECTION_WRITE,
06866                                IGNORE_USED);
06867
06868     DM35424_Check_Result_test(result, "Error configuring DMA");
06869
06870     result = DM35424_Dma_Buffer_Setup(board,
06871                                       &my_dac[dac_num],
06872                                       channel,
06873                                       DMA_BUFFER0,
06874                                       BUFFER_VALID,
06875                                       BUFFER_HALT,
06876                                       BUFFER_NO_LOOP,
06877                                       BUFFER_NO_INTERRUPT);
06878
06879     result = DM35424_Dma_Write(board,
06880                                &my_dac[dac_num],
06881                                channel,
06882                                BUFFER_0,
06883                                DAC_BUFFER_SIZE_BYTES,
06884                                buffer);
06885
06886     DM35424_Check_Result_test(result, "Writing to DMA buffer failed");
06887
06888 }
06889
06890 // Test 10
06891 free(buffer);
06892
06893 for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD; dac_num++) {
06894     result = DM35424_Dac_Set_Clock_Source_Global(board,
06895                                                    &my_dac[dac_num],
06896                                                    DM35424_CLK_SRC_GBL2,
06897                                                    DM35424_DAC_CLK_EVENT_CHAN_MARKER);
06898     DM35424_Check_Result_test(result, "Error setting clock source global 2 to DAC channel
06899 marker.");
06900
06901     sample_counts = 0;
06902
06903     result = DM35424_Dac_Interrupt_Set_Config(board,
06904                                                &my_dac[dac_num],
06905                                                DM35424_DAC_INT_CHAN_MARKER_MASK,
06906                                                INTERRUPT_ENABLE);
06907
06908     DM35424_Check_Result_test(result, "Error setting interrupt enable for channel
06909 marker");
06910
06911     for (channel = 0; channel < my_dac[dac_num].num_dma_channels; channel++) {
06912         fprintf(stdout, "Testing DAC%d Chan %d.....\n", dac_num, channel);
06913
06914         // Test 10
06915
06916         result = DM35424_Dac_Channel_Set_Marker_Config(board,
06917                                                         &my_dac[dac_num],
06918                                                         channel,
06919                                                         0x00);
06920         DM35424_Check_Result_test(result, "Error setting marker configuration.");
06921
06922         result = DM35424_Dac_Channel_Clear_Marker_Status(board,
06923                                                           &my_dac[dac_num],
06924                                                           channel,
06925                                                           0xFF);
06926         DM35424_Check_Result_test(result, "Error clearing channel marker status.");
06927
06928         result = DM35424_Dma_Start(board,
06929                                    &my_dac[dac_num],
06930                                    channel);
06931
06932     }
06933 }
06934
06935
06936
06937

```

```

06938         DM35424_Check_Result_test(result, "Error starting DMA.");
06939
06940         // Check ADC counts. There should be none yet.
06941         result = DM35424_Adc_Get_Sample_Count(board,
06942                                             &my_adc[ADC0],
06943                                             &sample_counts);
06944         DM35424_Check_Result_test(result, "Error getting sample count.");
06945
06946         DM35424_Check_Result_test(sample_counts != 0, "Expected ADC sample counts to
be zero, but it was not.");
06947
06948         interrupts_expected = 1;
06949
06950         for (index = 0; index < 8; index++) {
06951             fprintf(stdout, "    Checking for marker %d...", index);
06952
06953             result = DM35424_Adc_Start(board,
06954                                     &my_adc[ADC0]);
06955
06956             DM35424_Check_Result_test(result, "Error starting ADC.");
06957
06958             // Check that ADC is waiting for the start trigger.
06959             DM35424_Adc_Get_Mode_Status(board,
06960                                     &my_adc[ADC0],
06961                                     &mode_status);
06962
06963             sprintf(message, "Expected mode/status of 0x22, but found 0x%x\n",
mode_status);
06964
06965             DM35424_Check_Result_test(mode_status != 0x22, message);
06966
06967             result = DM35424_Dac_Channel_Set_Marker_Config(board,
06968                                                         &my_dac[dac_num],
06969                                                         channel,
06970                                                         0x01 << index);
06971             DM35424_Check_Result_test(result, "Error setting marker
configuration.");
06972
06973             result = DM35424_Dac_Channel_Get_Marker_Config(board,
06974                                                         &my_dac[dac_num],
06975                                                         channel,
06976                                                         &marker_config);
06977             DM35424_Check_Result_test(result, "Error getting marker
configuration.");
06978
06979             // Test 10
06980
06981             if (index == 0) {
06982                 result = DM35424_Dac_Start(board,
06983                                             &my_dac[dac_num]);
06984
06985                 DM35424_Check_Result_test(result, "Error starting DAC.");
06986             }
06987             start_timeout_timer(ONE_SEC_IN_MILLI * 5);
06988
06989             while (!timeout_has_expired() && local_int_count == interrupt_count) {
06990                 DM35424_Micro_Sleep(100);
06991             }
06992
06993             fprintf(stdout, " (INTERRUPT) ");
06994
06995             result = DM35424_Dac_Channel_Get_Marker_Status(board,
06996                                                         &my_dac[dac_num],
06997                                                         channel,
06998                                                         &marker_status);
06999
07000             DM35424_Check_Result_test(result, "Error getting channel marker
status.");
07001
07002             sprintf(message, "Expected marker status of 0x%x, but got 0x%x.",
07003                         0x01 << index, marker_status);
07004             DM35424_Check_Result_test((marker_status & (0x01 << index)) == 0,
message);
07005
07006             result = DM35424_Dac_Interrupt_Get_Status(board,
07007                                                         &my_dac[dac_num],
07008                                                         &interrupt_status);
07009
07010             DM35424_Check_Result_test(result, "Error getting DAC interrupt
status.");
07011
07012             sprintf(message, "Expected DAC marker interrupt set, but got 0x%x.",
07013                         interrupt_status);
07014             DM35424_Check_Result_test((interrupt_status &
DM35424_DAC_INT_CHAN_MARKER_MASK) == 0, message);

```



```

07092 *****
07093 *****
07094 *****          MAIN PROGRAM          *****
07095 *****
07096 *****
07097 *****
07098 *****
07099 *****
07100 *****
07101 *****/
07102
07103
07104 int main(int argument_count, char **arguments)
07105 {
07106     unsigned long int minor = 0;
07107     struct sigaction signal_action;
07108
07109     int help_option_given = 0;
07110     int minor_option_given = 0;
07111
07112     int status;
07113     int test_num = -1;
07114
07115     char *invalid_char_p;
07116     struct option options[] = {
07117         {"help", 0, 0, HELP_OPTION},
07118         {"minor", 1, 0, MINOR_OPTION},
07119         {"test", 1, 0, TEST_OPTION},
07120         {"nstop", 0, 0, NOSTOP_OPTION},
07121         {0, 0, 0, 0}
07122     };
07123
07124     program_name = arguments[0];
07125
07126     // Show usage, parse arguments
07127     while (1) {
07128         /*
07129          * Parse the next command line option and any arguments it may require
07130          */
07131
07132         status = getopt_long(argument_count,
07133                             arguments, "", options, NULL);
07134
07135         /*
07136          * If getopt_long() returned -1, then all options have been processed
07137          */
07138
07139         if (status == -1) {
07140             break;
07141         }
07142
07143         /*
07144          * Figure out what getopt_long() found
07145          */
07146
07147         switch (status) {
07148
07149             /*****
07150              * User entered '--help'
07151              *****/
07152             case HELP_OPTION:
07153
07154                 /*
07155                  * Refuse to accept duplicate '--help' options
07156                  */
07157
07158                 if (help_option_given) {
07159                     error(0, 0, "ERROR: Duplicate option '--help'");
07160                     usage();
07161                 }
07162
07163                 /*
07164                  * '--help' option has been seen
07165                  */

```

```

07169
07170         help_option_given = 0xFF;
07171         break;
07172
07173         /*#####
07174         User entered '--minor'
07175         ##### */
07176         case MINOR_OPTION:
07177
07178             /*
07179              * Refuse to accept duplicate '--minor' options
07180              */
07181
07182             if (minor_option_given) {
07183                 error(0, 0,
07184                     "ERROR: Duplicate option '--minor'");
07185                 usage();
07186             }
07187
07188             /*
07189              * Convert option argument string to unsigned long integer
07190              */
07191
07192             errno = 0;
07193
07194             minor = strtoul(optarg, &invalid_char_p, 10);
07195
07196             /*
07197              * Catch unsigned long int overflow
07198              */
07199
07200             if ((minor == ULONG_MAX)
07201                 && (errno == ERANGE)) {
07202                 error(0, 0,
07203                     "ERROR: Device minor number caused numeric overflow");
07204                 usage();
07205             }
07206
07207             /*
07208              * Catch argument strings with valid decimal prefixes, for
07209              * example "1q", and argument strings which cannot be converted,
07210              * for example "abc1"
07211              */
07212
07213             if ((*invalid_char_p != '\0')
07214                 || (invalid_char_p == optarg)) {
07215                 error(0, 0,
07216                     "ERROR: Non-decimal device minor number");
07217                 usage();
07218             }
07219
07220             /*
07221              * '--minor' option has been seen
07222              */
07223
07224             minor_option_given = 0xFF;
07225             break;
07226         /*#####
07227         User entered '--test'
07228         ##### */
07229         case TEST_OPTION:
07230
07231
07232             /*
07233              * Convert option argument string to unsigned long integer
07234              */
07235
07236             errno = 0;
07237
07238             test_num = strtoul(optarg, &invalid_char_p, 10);
07239
07240             /*
07241              * Catch unsigned long int overflow
07242              */
07243
07244             if ((test_num == ULONG_MAX)
07245                 && (errno == ERANGE)) {
07246                 error(0, 0,
07247                     "ERROR: Device minor number caused numeric overflow");
07248                 usage();
07249             }
07250
07251             /*
07252              * Catch argument strings with valid decimal prefixes, for
07253              * example "1q", and argument strings which cannot be converted,
07254              * for example "abc1"
07255              */

```

```

07256
07257         if ((*invalid_char_p != '\0')
07258             || (invalid_char_p == optarg)) {
07259             error(0, 0,
07260                  "ERROR: Non-decimal device minor number");
07261             usage();
07262         }
07263         break;
07264     /*#####
07265      * User entered '--nostop'
07266      *##### */
07267     case NOSTOP_OPTION:
07268         stop_on_error = 0;
07269
07270         break;
07271     case '?':
07272         usage();
07273         break;
07274
07275     /*#####
07276      * getopt_long() returned unexpected value
07277      *##### */
07278     default:
07279         error(EXIT_FAILURE,
07280              0,
07281              "ERROR: getopt_long() returned unexpected value %#x",
07282              status);
07283         break;
07284     }
07285 }
07286
07287 /*
07288  * Recognize '--help' option before any others
07289  */
07290
07291 if (help_option_given) {
07292     usage();
07293 }
07294
07295 /*
07296  * '--minor' option must be given
07297  */
07298
07299 if (minor_option_given == 0x00) {
07300     error(0, 0, "ERROR: Option '--minor' is required");
07301     usage();
07302 }
07303
07304 interrupts_expected = 0;
07305
07306 signal_action.sa_handler = sigint_handler;
07307 sigfillset(&(signal_action.sa_mask));
07308 signal_action.sa_flags = 0;
07309
07310 if (sigaction(SIGINT, &signal_action, NULL) < 0) {
07311     error(EXIT_FAILURE, errno, "ERROR: sigaction() FAILED");
07312 }
07313
07314 /*#####
07315  * TEST 0
07316  *##### */
07317 if (RUN_TEST(0)) {
07318
07319     run_test_0(minor);
07320
07321 }
07322
07323 /*#####
07324  * TEST 1
07325  *##### */
07326 if (RUN_TEST(1)) {
07327
07328     run_test_1(minor);
07329
07330 }
07331
07332 /*#####
07333  * TEST 2
07334  *##### */
07335 if (RUN_TEST(2)) {
07336
07337     run_test_2(minor);
07338
07339 }

```



```
07343
07344
07345     /*****
07346     *           TEST 3
07347     *****/
07348     if (RUN_TEST(3)) {
07349
07350         run_test_3(minor);
07351     }
07352
07353     /*****
07354     *           TEST 4
07355     *****/
07356     if (RUN_TEST(4)) {
07357
07358         run_test_4(minor);
07359     }
07360
07361     /*****
07362     *           TEST 5
07363     *****/
07364     if (RUN_TEST(5)) {
07365
07366         run_test_5(minor);
07367     }
07368
07369     /*****
07370     *           TEST 6
07371     *****/
07372     if (RUN_TEST(6)) {
07373
07374         run_test_6(minor);
07375     }
07376
07377     /*****
07378     *           TEST 7
07379     *****/
07380     if (RUN_TEST(7)) {
07381
07382         run_test_7(minor);
07383     }
07384
07385     /*****
07386     *           TEST 8
07387     *****/
07388     if (RUN_TEST(8)) {
07389
07390         run_test_8(minor);
07391     }
07392
07393     /*****
07394     *           TEST 9
07395     *****/
07396     if (RUN_TEST(9)) {
07397
07398         run_test_9(minor);
07399     }
07400
07401     /*****
07402     *           TEST 10
07403     *****/
07404     if (RUN_TEST(10)) {
07405
07406         run_test_10(minor);
07407     }
07408
07409     printf("success.\nExample program successfully completed.\n\n");
07410
07411     return 0;
07412 }
07413
07414
```

6.5 examples/_non_public/dm35424_calibrate.c File Reference

Example program which demonstrates the reference voltage adjustment function blocks.

```
#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <unistd.h>
#include <limits.h>
#include <signal.h>
#include <string.h>
#include <getopt.h>
#include <termios.h>
#include <time.h>
#include <sys/time.h>
#include "dm35424_gbc_library.h"
#include "dm35424_adc_library.h"
#include "dm35424_dac_library.h"
#include "dm35424_dma_library.h"
#include "dm35424_ioctl.h"
#include "dm35424_examples.h"
#include "dm35424_util_library.h"
#include "dm35424_board_access.h"
#include "dm35424_types.h"
#include "dm35424.h"
#include "dm35424_ref_adjust_library.h"
```

Functions

- static void **usage** (void)
Print information on stderr about how the program is to be used. After doing so, the program is exited.
- static void [sigint_handler](#) (int signal_number)
Signal handler for SIGINT Control-C keyboard interrupt.
- int [main](#) (int argument_count, char **arguments)
The main program.

Variables

- static char * [program_name](#)
- volatile int [exit_program](#) = 0

6.5.1 Detailed Description

Example program which demonstrates the reference voltage adjustment function blocks.

This example program demonstrates using the reference adjustment value to adjust the value reported by the ADC or sent by the DAC. The example allows for setting an adjustment value, and then writing that value to the appropriate memory location.

The reference voltage source should be attached to the differential inputs of ADC0 or ADC1 (depending on which reference adjustment you are using). The voltage reading from the ADC will be displayed on the screen, allowing you to view the effect reference value changes has on the measured voltage.

For purposes of running the example, and for convenience, the onboard DACs will be set to supply a value of 2 volts. You can then loopback the DAC outputs into the ADC inputs, as described below.

Note that the supplied DAC outputs should not be considered a reference voltage source for the purposes of calibrating the board.

WARNING: This example will allow you to change the preset calibration values on the board. Do not do so unless you understand the risks of permanently changing that value.

Setup: Attach a reference voltage source to the ADC Channel 0- and Channel 0+ pins. If using the onboard DACs, connect DAC Channel 0 to ADC Channel 0+ and DAC Channel 1 to ADC Channel 0-.

This file and its contents are copyright (C) RTD Embedded Technologies, Inc. All Rights Reserved.

This software is licensed as described in the RTD End-User Software License Agreement. For a copy of this agreement, refer to the file LICENSE.TXT (which should be included with this software) or contact RTD Embedded Technologies, Inc.

Id

[dm35424_calibrate.c](#) 142659 2024-04-26 19:32:32Z lfrankenfield

Definition in file [dm35424_calibrate.c](#).

6.5.2 Function Documentation

6.5.2.1 main()

```
int main (
    int argument_count,
    char ** arguments)
```

The main program.

Parameters

<i>argument_count</i>	
-----------------------	--

Number of args passed on the command line, including the executable name

Parameters

<i>arguments</i>	
------------------	--

Pointer to array of character strings, which are the args themselves.

Return values

0	
---	--

Success

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

Definition at line 179 of file [dm35424_calibrate.c](#).

References [ADC_0](#), [ADC_1](#), [board](#), [CHANNEL_0](#), [CHANNEL_1](#), [DAC_0](#), [DAC_2](#), [DEFAULT_RATE](#), [DM35424_Adc_Ad_Config_Set_I](#), [DM35424_Adc_Channel_Get_Last_Sample\(\)](#), [DM35424_Adc_Channel_Setup\(\)](#), [DM35424_Adc_Initialize\(\)](#), [DM35424_ADC_INPUT_DIFFERENTIAL](#), [DM35424_ADC_MODE_CONFIG_HIGH_SPEED](#), [DM35424_Adc_Open\(\)](#), [DM35424_Adc_Reset\(\)](#), [DM35424_ADC_RNG_BIPOLAR_2_5V](#), [DM35424_Adc_Sample_To_Volts\(\)](#), [DM35424_Adc_Set_Clock_Sr](#), [DM35424_Adc_Set_Post_Stop_Samples\(\)](#), [DM35424_Adc_Set_Pre_Trigger_Samples\(\)](#), [DM35424_Adc_Set_Sample_Rate\(\)](#), [DM35424_Adc_Set_Start_Trigger\(\)](#), [DM35424_Adc_Set_Stop_Trigger\(\)](#), [DM35424_Adc_Start\(\)](#), [DM35424_Adc_Uninitialize\(\)](#), [DM35424_Board_Close\(\)](#), [DM35424_Board_Open\(\)](#), [DM35424_Check_Result\(\)](#), [DM35424_CLK_SRC_IMMEDIATE](#), [DM35424_CLK_SRC_NEVER](#), [DM35424_Dac_Open\(\)](#), [DM35424_Dac_Reset\(\)](#), [DM35424_Dac_Set_Last_Conversion\(\)](#), [DM35424_Dac_Volts_To_Conv\(\)](#), [DM35424_Gbc_Board_Reset\(\)](#), [DM35424_Micro_Sleep\(\)](#), [DM35424_Ref_Adjust_Open\(\)](#), [DM35424_Ref_Adjust_Write_Adc_To_NonVolatile\(\)](#), [DM35424_Ref_Adjust_Write_Adc_To_Volatile\(\)](#), [DM35424_Ref_Adjust_Write_D](#), [DM35424_Ref_Adjust_Write_Dac_To_Volatile\(\)](#), [exit_program](#), [HELP_OPTION](#), [MINOR_OPTION](#), [my_adc](#), [program_name](#), [REF_0](#), [REF_1](#), [REF_NUM_OPTION](#), [sigint_handler\(\)](#), and [usage\(\)](#).

6.5.2.2 sigint_handler()

```
void sigint_handler (
    int signal_number) [static]
```

Signal handler for SIGINT Control-C keyboard interrupt.

Parameters

<i>signal_number</i>	
----------------------	--

Signal number passed in from the kernel.

Warning

One must be extremely careful about what functions are called from a signal handler.

Definition at line 146 of file [dm35424_calibrate.c](#).

References [exit_program](#).

6.5.3 Variable Documentation

6.5.3.1 exit_program

```
volatile int exit_program = 0
```

Boolean indicating whether or not to exit the program.

Definition at line 95 of file [dm35424_calibrate.c](#).

6.5.3.2 program_name

```
char* program_name [static]
```

Name of the program as invoked on the command line

Definition at line 89 of file [dm35424_calibrate.c](#).

6.6 dm35424_calibrate.c

[Go to the documentation of this file.](#)

```
00001
00053
00054 #include <stdio.h>
00055 #include <stddef.h>
00056 #include <stdlib.h>
00057 #include <errno.h>
00058 #include <error.h>
00059 #include <unistd.h>
00060 #include <limits.h>
00061 #include <signal.h>
00062 #include <string.h>
00063 #include <getopt.h>
00064 #include <signal.h>
00065 #include <termios.h>
00066 #include <time.h>
00067 #include <sys/time.h>
00068
00069
00070 #include "dm35424_gbc_library.h"
00071 #include "dm35424_adc_library.h"
00072 #include "dm35424_dac_library.h"
00073 #include "dm35424_dma_library.h"
00074 #include "dm35424_ioctl.h"
00075 #include "dm35424_examples.h"
00076 #include "dm35424_util_library.h"
00077 #include "dm35424_board_access.h"
00078 #include "dm35424_types.h"
00079 #include "dm35424.h"
00080 #include "dm35424_ref_adjust_library.h"
00081
00082 #define DEFAULT_RATE      80000
00083
00084 #define MAX_REF_ADJUST    0xFF
00085
00089 static char *program_name;
00090
00091
00095 volatile int exit_program = 0;
00096
00104
00105 static void usage(void)
00106 {
00107     fprintf(stderr, "\n");
00108     fprintf(stderr, "%s\n", program_name);
00109     fprintf(stderr, "\n");
00110     fprintf(stderr,
00111         "Usage: %s [--help] --minor MINOR [--ref REF_NUM] [--avg NUM_TO_AVG]\n",
00112         program_name);
```

```

00113     fprintf(stderr, "\n");
00114     fprintf(stderr,
00115         "        --help:      Display usage information and exit.\n");
00116     fprintf(stderr, "\n");
00117     fprintf(stderr, "        --minor:      Use specified minor device.\n");
00118     fprintf(stderr, "        MINOR:        Device minor number (>= 0).\n");
00119     fprintf(stderr, "        --ref:        Use specified reference adjust.\n");
00120     fprintf(stderr, "        REF_NUM:      Ref Adjust to use (0 or 1).\n");
00121     fprintf(stderr, "        --avg:        Average readings.\n");
00122     fprintf(stderr, "        NUM_TO_AVG:    Number of readings to average. \n");
00123
00124     fprintf(stderr, "\n");
00125     exit(EXIT_FAILURE);
00126 }
00127
00128
00129
00145
00146 static void sigint_handler(int signal_number)
00147 {
00148     exit_program = 0xff;
00149 }
00150
00178
00179 int main(int argument_count, char **arguments)
00180 {
00181     struct DM35424_Board_Descriptor *board;
00182     struct DM35424_Function_Block my_adc;
00183     struct DM35424_Function_Block alt_adc;
00184     struct DM35424_Function_Block my_dac;
00185     struct DM35424_Function_Block my_ref;
00186
00187     unsigned long int minor = 0;
00188     int result;
00189     int my_value;
00190     char last_action[100];
00191
00192     int *samples_to_average;
00193     unsigned int sample_index = 0;
00194
00195     unsigned int num_to_average = 2000;
00196
00197     unsigned int last_index = 0;
00198     int wrap_around = 0;
00199
00200     long ground_average = 0;
00201     long sample_sum = 0;
00202     unsigned int add_index = 0;
00203     unsigned int adc_num = ADC_0;
00204     unsigned int alt_adc_num = ADC_1;
00205
00206     unsigned int ref_num = REF_0;
00207     unsigned int rate = DEFAULT_RATE;
00208     unsigned int actual_rate;
00209     unsigned int dac_num = DAC_0;
00210
00211     unsigned int dac_positive = 1;
00212
00213     int seconds_count = 15 * 60;
00214
00215     int help_option_given = 0;
00216     int minor_option_given = 0;
00217     int status;
00218     struct sigaction signal_action;
00219
00220     uint8_t ref_value = 0;
00221     float volts = 0;
00222
00223     int16_t max_value, min_value;
00224     struct termios old_tio, new_tio;
00225     unsigned char keypress = '0';
00226
00227     char *invalid_char_p;
00228     struct option options[] = {
00229         {"help", 0, 0, HELP_OPTION},
00230         {"minor", 1, 0, MINOR_OPTION},
00231         {"ref", 1, 0, REF_NUM_OPTION},
00232         {"avg", 1, 0, 500},
00233         {0, 0, 0, 0}
00234     };
00235
00236     program_name = arguments[0];
00237
00238     // Show usage, parse arguments
00239     while (1) {
00240
00241         /*

```

```

00242         * Parse the next command line option and any arguments it may require
00243         */
00244
00245     status = getopt_long(argument_count,
00246                         arguments, "", options, NULL);
00247
00248     /*
00249     * If getopt_long() returned -1, then all options have been processed
00250     */
00251
00252     if (status == -1) {
00253         break;
00254     }
00255
00256     /*
00257     * Figure out what getopt_long() found
00258     */
00259
00260     switch (status) {
00261
00262         /*#####
00263         User entered '--help'
00264         ##### */
00265     case HELP_OPTION:
00266
00267         /*
00268         * Refuse to accept duplicate '--help' options
00269         */
00270
00271         if (help_option_given) {
00272             error(0, 0, "ERROR: Duplicate option '--help'");
00273             usage();
00274         }
00275
00276         /*
00277         * '--help' option has been seen
00278         */
00279
00280         help_option_given = 0xFF;
00281         break;
00282
00283         /*#####
00284         User entered '--minor'
00285         ##### */
00286     case MINOR_OPTION:
00287
00288         /*
00289         * Refuse to accept duplicate '--minor' options
00290         */
00291
00292         if (minor_option_given) {
00293             error(0, 0,
00294                 "ERROR: Duplicate option '--minor'");
00295             usage();
00296         }
00297
00298         /*
00299         * Convert option argument string to unsigned long integer
00300         */
00301
00302         errno = 0;
00303
00304         minor = strtoul(optarg, &invalid_char_p, 10);
00305
00306         /*
00307         * Catch unsigned long int overflow
00308         */
00309
00310         if ((minor == ULONG_MAX)
00311             && (errno == ERANGE)) {
00312             error(0, 0,
00313                 "ERROR: Device minor number caused numeric overflow");
00314             usage();
00315         }
00316
00317         /*
00318         * Catch argument strings with valid decimal prefixes, for
00319         * example "1q", and argument strings which cannot be converted,
00320         * for example "abc1"
00321         */
00322
00323         if ((*invalid_char_p != '\0')
00324             || (invalid_char_p == optarg)) {
00325             error(0, 0,
00326                 "ERROR: Non-decimal device minor number");
00327             usage();
00328         }

```

```
00329
00330
00331     /*
00332     * '--minor' option has been seen
00333     */
00334     minor_option_given = 0xFF;
00335     break;
00336
00337
00338
00339
00340     /*#####
00341     User entered '--ref'
00342     ##### */
00343 case REF_NUM_OPTION:
00344
00345     /*
00346     * Convert option argument string to unsigned long integer
00347     */
00348     errno = 0;
00349
00350     ref_num = strtoul(optarg, &invalid_char_p, 10);
00351
00352     /*
00353     * Catch unsigned long int overflow
00354     */
00355
00356     if ((ref_num == ULONG_MAX)
00357         && (errno == ERANGE)) {
00358         error(0, 0,
00359             "ERROR: Ref number caused numeric overflow");
00360         usage();
00361     }
00362
00363     /*
00364     * Catch argument strings with valid decimal prefixes, for
00365     * example "1q", and argument strings which cannot be converted,
00366     * for example "abcl"
00367     */
00368
00369     if ((*invalid_char_p != '\0')
00370         || (invalid_char_p == optarg)) {
00371         error(0, 0, "ERROR: Non-decimal Ref number");
00372         usage();
00373     }
00374
00375     if (ref_num != REF_0 && ref_num != REF_1) {
00376         error(0, 0,
00377             "ERROR: Reference Adjust number must be 0 or 1.");
00378         usage();
00379     }
00380     break;
00381 case 500:
00382
00383     /*
00384     * Convert option argument string to unsigned long integer
00385     */
00386
00387     errno = 0;
00388
00389     num_to_average = strtoul(optarg, &invalid_char_p, 10);
00390
00391     /*
00392     * Catch unsigned long int overflow
00393     */
00394
00395     if ((num_to_average == ULONG_MAX)
00396         && (errno == ERANGE)) {
00397         error(0, 0,
00398             "ERROR: Avg number caused numeric overflow");
00399         usage();
00400     }
00401
00402     /*
00403     * Catch argument strings with valid decimal prefixes, for
00404     * example "1q", and argument strings which cannot be converted,
00405     * for example "abcl"
00406     */
00407
00408     if ((*invalid_char_p != '\0')
00409         || (invalid_char_p == optarg)) {
00410         error(0, 0, "ERROR: Non-decimal avg number");
00411         usage();
00412     }
00413
00414     if (num_to_average < 1) {
00415         error(0, 0, "ERROR: Invalid number for averaging.\n");
```



```

00416                                     usage();
00417                                     }
00418
00419                                     break;
00420
00421                                     /*#####
00422                                     User entered unsupported option
00423                                     ##### */
00424                                     case '?':
00425                                         usage();
00426                                         break;
00427
00428                                     /*#####
00429                                     getopt_long() returned unexpected value
00430                                     ##### */
00431                                     default:
00432                                         error(EXIT_FAILURE,
00433                                                0,
00434                                                "ERROR: getopt_long() returned unexpected value %x",
00435                                                status);
00436                                         break;
00437                                     }
00438                                 }
00439
00440                                /*
00441                                * Recognize '--help' option before any others
00442                                */
00443
00444                                if (help_option_given) {
00445                                    usage();
00446                                }
00447
00448                                /*
00449                                * '--minor' option must be given
00450                                */
00451
00452                                if (minor_option_given == 0x00) {
00453                                    error(0, 0, "ERROR: Option '--minor' is required");
00454                                    usage();
00455                                }
00456
00457                                signal_action.sa_handler = sigint_handler;
00458                                sigfillset(&(signal_action.sa_mask));
00459                                signal_action.sa_flags = 0;
00460
00461                                if (sigaction(SIGINT, &signal_action, NULL) < 0) {
00462                                    error(EXIT_FAILURE, errno, "ERROR: sigaction() FAILED");
00463                                }
00464
00465                                printf("\n\n");
00466                                printf("*****\n");
00467                                printf("*                               WARNING                               *\n");
00468                                printf("*****\n");
00469                                printf("* This example program demonstrates how to change the calibration *\n");
00470                                printf("* values of the board. Specifically, writing to volatile memory will *\n");
00471                                printf("* temporarily change it, and writing to non-volatile will permanently *\n");
00472                                printf("* change it. Do not proceed unless you understand the risks of *\n");
00473                                printf("* changing the calibration values. *\n");
00474                                printf("*****\n");
00475                                printf("\n\nPress Enter to continue, or Ctrl-C to abort.\n\n");
00476
00477                                keypress = getchar();
00478
00479                                if (exit_program) {
00480                                    return 0;
00481                                }
00482
00483                                samples_to_average = (int *) malloc(num_to_average * sizeof(int));
00484
00485                                if (samples_to_average == NULL) {
00486                                    error(0, 0, "ERROR: Could not allocate for samples.\n");
00487                                    usage();
00488                                }
00489
00490                                for (sample_index = 0; sample_index < num_to_average; sample_index++) {
00491                                    samples_to_average[sample_index] = 0;
00492                                }
00493
00494                                tcgetattr(STDIN_FILENO, &old_tio);
00495
00496                                new_tio = old_tio;
00497
00498                                new_tio.c_lflag &= ~ICANON;
00499                                new_tio.c_lflag &= ~ECHO;
00500                                new_tio.c_lflag &= ~ISIG;
00501                                new_tio.c_cc[VMIN] = 0;
00502                                new_tio.c_cc[VTIME] = 0;

```

```
00503
00504     tcsetattr(STDIN_FILENO, TCSANOW, &new_tio);
00505
00506     printf("Opening board.....");
00507     result = DM35424_Board_Open(minor, &board);
00508
00509     DM35424_Check_Result(result, "Could not open board");
00510     printf("success. (%d)\nResetting board.....", board->file_descriptor);
00511     result = DM35424_Gbc_Board_Reset(board);
00512
00513     DM35424_Check_Result(result, "Could not reset board");
00514     printf("success.\n");
00515
00516     if (ref_num == REF_0) {
00517         adc_num = ADC_0;
00518         dac_num = DAC_0;
00519         alt_adc_num = ADC_1;
00520     }
00521     else {
00522         adc_num = ADC_1;
00523         dac_num = DAC_2;
00524         alt_adc_num = ADC_0;
00525     }
00526
00527     printf("success.\nOpening DAC.....");
00528
00529
00530     result = DM35424_Dac_Open(board, dac_num, &my_dac);
00531
00532     DM35424_Check_Result(result, "Could not open DAC");
00533
00534     printf("Found DAC%u, with %d DMA channels (%d buffers each)\n",
00535           dac_num, my_dac.num_dma_channels,
00536           my_dac.num_dma_buffers);
00537
00538     result = DM35424_Dac_Reset(board,
00539                                &my_dac);
00540
00541     DM35424_Check_Result(result, "Error resetting DAC.");
00542
00543
00544
00545     result = DM35424_Dac_Volts_To_Conv(3.5f, &max_value);
00546
00547     DM35424_Check_Result(result,
00548                           "Error converting value to conversion counts.");
00549
00550     result = DM35424_Dac_Set_Last_Conversion(board,
00551                                               &my_dac,
00552                                               CHANNEL_0,
00553                                               0,
00554                                               max_value);
00555
00556     DM35424_Check_Result(result, "Error setting last conversion for DAC0 Chan 0.");
00557
00558     result = DM35424_Dac_Volts_To_Conv(1.5f, &min_value);
00559
00560     DM35424_Check_Result(result,
00561                           "Error converting value to conversion counts.");
00562
00563
00564     result = DM35424_Dac_Set_Last_Conversion(board,
00565                                               &my_dac,
00566                                               CHANNEL_1,
00567                                               0,
00568                                               min_value);
00569
00570     DM35424_Check_Result(result, "Error setting last conversion for DAC0 Chan 1.");
00571
00572     printf("Opening Reference Adjustment....\n");
00573
00574     result = DM35424_Ref_Adjust_Open(board,
00575                                       ref_num,
00576                                       &my_ref);
00577
00578     DM35424_Check_Result(result, "Error opening reference adjustment.");
00579
00580     printf("Opening ADC.....\n");
00581
00582     result = DM35424_Adc_Open(board, adc_num, &my_adc);
00583
00584     DM35424_Check_Result(result, "Could not open ADC");
00585
00586     printf("Found ADC%d, with %d DMA channels (%d buffers each)\n",
00587           adc_num, my_adc.num_dma_channels, my_adc.num_dma_buffers);
00588
00589     result = DM35424_Adc_Uninitialize(board, &my_adc);
```

```
00590
00591     DM35424_Check_Result(result, "Error setting ADC to Uninitialized.");
00592
00593     result = DM35424_Adc_Channel_Setup(board,
00594                                         &my_adc,
00595                                         CHANNEL_0,
00596                                         DM35424_ADC_RNG_BIPOLAR_2_5V,
00597                                         DM35424_ADC_INPUT_DIFFERENTIAL);
00598
00599     DM35424_Check_Result(result, "Error setting up channel.");
00600
00601
00602     result = DM35424_Adc_Ad_Config_Set_Mode(board,
00603                                               &my_adc,
00604                                               DM35424_ADC_MODE_CONFIG_HIGH_SPEED);
00605
00606     DM35424_Check_Result(result, "Error setting AD config.");
00607
00608     result = DM35424_Adc_Set_Start_Trigger(board,
00609                                              &my_adc,
00610                                              DM35424_CLK_SRC_IMMEDIATE);
00611     DM35424_Check_Result(result, "Error setting start trigger.");
00612
00613     result = DM35424_Adc_Set_Stop_Trigger(board,
00614                                             &my_adc,
00615                                             DM35424_CLK_SRC_NEVER);
00616     DM35424_Check_Result(result, "Error setting stop trigger.");
00617
00618     result = DM35424_Adc_Set_Clock_Src(board,
00619                                         &my_adc,
00620                                         DM35424_CLK_SRC_IMMEDIATE);
00621
00622     DM35424_Check_Result(result, "Error setting ADC clock");
00623
00624     result = DM35424_Adc_Set_Sample_Rate(board,
00625                                           &my_adc, rate, &actual_rate);
00626
00627     DM35424_Check_Result(result, "Failed to set sample rate for ADC.");
00628     fprintf(stdout,
00629             "ADC:%d Rate requested: %d Actual Rate Achieved: %d\n",
00630             adc_num, rate, actual_rate);
00631
00632     result = DM35424_Adc_Set_Pre_Trigger_Samples(board, &my_adc, 0);
00633     DM35424_Check_Result(result, "Error setting pre-capture samples.");
00634
00635     result = DM35424_Adc_Set_Post_Stop_Samples(board, &my_adc, 0);
00636     DM35424_Check_Result(result, "Error setting post-capture samples.");
00637
00638     result = DM35424_Adc_Initialize(board, &my_adc);
00639
00640     DM35424_Check_Result(result, "Failed or timed out initializing ADC.");
00641
00642     printf("Starting ADC.....\n");
00643
00644     result = DM35424_Adc_Start(board, &my_adc);
00645
00646
00647
00648
00649     /*****
00650     * Also start the ADC not in use and let it run to warm
00651     * up also, so that after this test is concluded on
00652     * one ADC, the next can be immediately done as well.
00653     *****/
00654     result = DM35424_Adc_Open(board, alt_adc_num, &alt_adc);
00655
00656     DM35424_Check_Result(result, "Could not open ADC");
00657
00658     result = DM35424_Adc_Uninitialize(board, &alt_adc);
00659
00660     DM35424_Check_Result(result, "Error setting ADC to Uninitialized.");
00661
00662     result = DM35424_Adc_Channel_Setup(board,
00663                                         &alt_adc,
00664                                         CHANNEL_0,
00665                                         DM35424_ADC_RNG_BIPOLAR_2_5V,
00666                                         DM35424_ADC_INPUT_DIFFERENTIAL);
00667
00668     DM35424_Check_Result(result, "Error setting up channel.");
00669
00670
00671     result = DM35424_Adc_Ad_Config_Set_Mode(board,
00672                                               &alt_adc,
00673                                               DM35424_ADC_MODE_CONFIG_HIGH_SPEED);
00674
00675     DM35424_Check_Result(result, "Error setting AD config.");
00676
```

```

00677     result = DM35424_Adc_Set_Start_Trigger(board,
00678                                             &alt_adc,
00679                                             DM35424_CLK_SRC_IMMEDIATE);
00680     DM35424_Check_Result(result, "Error setting start trigger.");
00681
00682     result = DM35424_Adc_Set_Stop_Trigger(board,
00683                                             &alt_adc,
00684                                             DM35424_CLK_SRC_NEVER);
00685     DM35424_Check_Result(result, "Error setting stop trigger.");
00686
00687     result = DM35424_Adc_Set_Clock_Src(board,
00688                                         &alt_adc,
00689                                         DM35424_CLK_SRC_IMMEDIATE);
00690
00691     DM35424_Check_Result(result, "Error setting ADC clock");
00692
00693     result = DM35424_Adc_Set_Sample_Rate(board,
00694                                         &alt_adc, rate, &actual_rate);
00695
00696     DM35424_Check_Result(result, "Failed to set sample rate for ADC.");
00697
00698
00699     result = DM35424_Adc_Initialize(board, &alt_adc);
00700
00701
00702     printf("***** BOTH ADCs ARE NOW ACTIVE *****\n");
00703     printf("Beginning warmup waiting period...(15 minutes) \n");
00704     printf("Hit Enter to skip.\n");
00705
00706     keypress = '0';
00707
00708     while (keypress != '\n' && seconds_count > 0) {
00709         sleep(1);
00710         fprintf(stdout, "Remaining seconds: %d    \r", seconds_count);
00711         fflush(stdout);
00712         keypress = getchar();
00713         seconds_count --;
00714     }
00715
00716     exit_program = 0;
00717
00718     printf("\n\n** Attach ADC positive to ADC negative and hit Enter. **\n");
00719     keypress = '0';
00720
00721     while (keypress != '\n') {
00722         sleep(1);
00723         keypress = getchar();
00724     }
00725
00726     sample_sum = 0;
00727     for (sample_index = 0; sample_index < num_to_average; sample_index++) {
00728         result =
00729             DM35424_Adc_Channel_Get_Last_Sample(board,
00730                                                 &my_adc,
00731                                                 CHANNEL_0,
00732                                                 &my_value);
00733
00734         DM35424_Check_Result(result,
00735                             "Error getting ADC value.");
00736
00737         sample_sum += my_value;
00738
00739         DM35424_Micro_Sleep(1000);
00740     }
00741
00742     ground_average = (sample_sum / num_to_average);
00743     result =
00744         DM35424_Adc_Sample_To_Volts(ground_average,
00745                                     &volts);
00746
00747     DM35424_Check_Result(result,
00748                         "Error converting ADC sample to volts.");
00749
00750     printf("\nThe ground average is %ld (%2.8f volts) and will be subtracted from each
00751 reading.\n",
00752           ground_average, volts);
00753
00754     ref_value = 128;
00755
00756     printf("\n\n\n===== \n");
00757     printf("                                CALIBRATING ADC\n");
00758     printf("===== \n");
00759     printf("Press i to increase the ref adjust value.\n");
00760     printf("Press d to decrease the ref adjust value.\n");
00761     printf("Press n to write the ref adjust value to ADC non-volatile memory.\n");
00762     printf("Press q to continue to DAC calibration.\n\n");

```

```

00763     printf("Ref Value      ADC%d Chan 0      Last Action\n", adc_num);
00764     printf("-----      -----      -----\n");
00765
00766     DM35424_Check_Result(result, "Error starting ADC");
00767
00768     strcpy(last_action, "Waiting for input.");
00769
00770     sample_index = 0;
00771     while (!exit_program && keypress != 'q') {
00772         keypress = '0';
00773         keypress = getchar();
00774
00775         if (keypress == 'i') {
00776
00777             if (ref_value < MAX_REF_ADJUST) {
00778                 ref_value ++;
00779                 strcpy(last_action, "Reference value increased.");
00780             }
00781             result = DM35424_Ref_Adjust_Write_Adc_To_Volatile(
00782                 board,
00783                 &my_ref,
00784                 ref_value);
00785
00786             DM35424_Check_Result(result, "Error writing reference value to ADC volatile
00787 memory.");
00788
00789             wrap_around = 0;
00790             last_index = sample_index;
00791         }
00792
00793         if (keypress == 'd') {
00794             if (ref_value > 0) {
00795                 ref_value --;
00796                 strcpy(last_action, "Reference value decreased.");
00797             }
00798             result = DM35424_Ref_Adjust_Write_Adc_To_Volatile(
00799                 board,
00800                 &my_ref,
00801                 ref_value);
00802
00803             DM35424_Check_Result(result, "Error writing reference value to ADC volatile
00804 memory.");
00805             wrap_around = 0;
00806             last_index = sample_index;
00807         }
00808
00809         if (keypress == 'n') {
00810             result = DM35424_Ref_Adjust_Write_Adc_To_NonVolatile(
00811                 board,
00812                 &my_ref,
00813                 ref_value);
00814
00815             DM35424_Check_Result(result, "Error writing reference value to ADC
00816 non-volatile memory.");
00817             strcpy(last_action, "Ref value written to ADC non-volatile memory.");
00818         }
00819         result =
00820             DM35424_Adc_Channel_Get_Last_Sample(board,
00821                 &my_adc,
00822                 CHANNEL_0,
00823                 &my_value);
00824
00825
00826         if (result) {
00827             printf("\n\nFile descriptor: %d\n\n", board->file_descriptor);
00828
00829             tcsetattr(STDIN_FILENO, TCSANOW, &old_tio);
00830         }
00831         DM35424_Check_Result(result,
00832             "Error getting ADC value.");
00833
00834         samples_to_average[sample_index] = my_value;
00835         sample_index ++;
00836
00837         if (sample_index == num_to_average) {
00838             sample_index = 0;
00839         }
00840
00841         if (sample_index == last_index) {
00842             wrap_around = 1;
00843         }
00844
00845         sample_sum = 0;
00846         for (add_index = 0; add_index < num_to_average; add_index ++) {

```



```

00931             ref_value --;
00932             result = DM35424_Ref_Adjust_Write_Dac_To_Volatile(
00933                 board,
00934                 &my_ref,
00935                 ref_value);
00936
00937             DM35424_Check_Result(result, "Error writing reference value to DAC
volatile memory.");
00938             strcpy(last_action, "Reference value written to DAC volatile
memory.");
00939         }
00940     }
00941 }
00942
00943     if (keypress == 'f') {
00944         if (dac_positive) {
00945             DM35424_Check_Result(result,
00946                 "Error converting value to conversion counts.");
00947             result = DM35424_Dac_Set_Last_Conversion(board,
00948                 &my_dac,
00949                 CHANNEL_0,
00950                 0,
00951                 min_value);
00952
00953             DM35424_Check_Result(result, "Error setting DAC to -5.0");
00954             strcpy(last_action, "DAC value changed to -5.0.");
00955             dac_positive = 0;
00956         }
00957         else {
00958             DM35424_Check_Result(result,
00959                 "Error converting value to conversion counts.");
00960             result = DM35424_Dac_Set_Last_Conversion(board,
00961                 &my_dac,
00962                 CHANNEL_0,
00963                 0,
00964                 max_value);
00965
00966             DM35424_Check_Result(result, "Error setting DAC to 5.0");
00967             strcpy(last_action, "DAC value changed to 5.0.");
00968             dac_positive = 1;
00969         }
00970     }
00971
00972     if (keypress == 'n') {
00973         result = DM35424_Ref_Adjust_Write_Dac_To_NonVolatile(
00974             board,
00975             &my_ref,
00976             ref_value);
00977
00978         DM35424_Check_Result(result, "Error writing reference value to DAC
non-volatile memory.");
00979         strcpy(last_action, "Ref value written to DAC non-volatile memory.");
00980     }
00981
00982     printf("    %2u          %s          \r",
00983         ref_value, last_action);
00984
00985     DM35424_Micro_Sleep(10000);
00986 }
00987
00988     tcsetattr(STDIN_FILENO, TCSANOW, &old_tio);
00989
00990     free(samples_to_average);
00991     printf("\n\nStopping Adc %d.....", adc_num);
00992
00993     result = DM35424_Adc_Reset(board, &my_adc);
00994
00995     DM35424_Check_Result(result, "Error starting ADC");
00996
00997
00998
00999
01000
01001
01002
01003
01004
01005
01006
01007
01008
01009
01010
01011
01012
01013
01014

```

```

01015     printf("success.\n");
01016
01017     printf("Closing Board\n");
01018     result = DM35424_Board_Close(board);
01019
01020     DM35424_Check_Result(result, "Error closing board.");
01021     printf("Example program successfully completed.\n");
01022     return 0;
01023
01024 }
```

6.7 examples/_non_public/dm35424_oven_test.c File Reference

```

#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <unistd.h>
#include <limits.h>
#include <getopt.h>
#include <signal.h>
#include <string.h>
#include <math.h>
#include <sys/time.h>
#include <time.h>
#include "dm35424_gbc_library.h"
#include "dm35424_dac_library.h"
#include "dm35424_adc_library.h"
#include "dm35424_ioctl.h"
#include "dm35424_examples.h"
#include "dm35424_dma_library.h"
#include "dm35424.h"
#include "dm35424_util_library.h"
```

Functions

- static void **usage** (void)
Print information on stderr about how the program is to be used. After doing so, the program is exited.
- static void **sigint_handler** (int signal_number)
Signal handler for SIGINT Control-C keyboard interrupt.

Variables

- static char * **program_name**

6.7.1 Detailed Description

This file and its contents are copyright (C) RTD Embedded Technologies, Inc. All Rights Reserved.

This software is licensed as described in the RTD End-User Software License Agreement. For a copy of this agreement, refer to the file LICENSE.TXT (which should be included with this software) or contact RTD Embedded Technologies, Inc.

Id

[dm35424_oven_test.c](#) 68203 2013-03-19 19:00:51Z rgroner

Definition in file [dm35424_oven_test.c](#).

6.7.2 Function Documentation

6.7.2.1 sigint_handler()

```
void sigint_handler (  
    int signal_number) [static]
```

Signal handler for SIGINT Control-C keyboard interrupt.

Parameters

<i>signal_number</i>	
----------------------	--

Signal number passed in from the kernel.

Warning

One must be extremely careful about what functions are called from a signal handler.

Definition at line [195](#) of file [dm35424_oven_test.c](#).

References [exit_program](#).

6.7.3 Variable Documentation

6.7.3.1 program_name

```
char* program_name [static]
```

Name of the program as invoked on the command line

Definition at line [90](#) of file [dm35424_oven_test.c](#).

6.8 dm35424_oven_test.c

[Go to the documentation of this file.](#)

```

00001
00027
00028 #include <stdio.h>
00029 #include <stddef.h>
00030 #include <stdlib.h>
00031 #include <errno.h>
00032 #include <error.h>
00033 #include <unistd.h>
00034 #include <limits.h>
00035 #include <getopt.h>
00036 #include <signal.h>
00037 #include <string.h>
00038 #include <math.h>
00039 #include <sys/time.h>
00040 #include <time.h>
00041
00042 #include "dm35424_gbc_library.h"
00043 #include "dm35424_dac_library.h"
00044 #include "dm35424_adc_library.h"
00045 #include "dm35424_ioctl.h"
00046 #include "dm35424_examples.h"
00047 #include "dm35424_dma_library.h"
00048 #include "dm35424.h"
00049 #include "dm35424_util_library.h"
00050
00051 #define DAC0_SIN_FILE_FOR_ADC "sin_dac0_for_adc.txt"
00052 #define DAC1_SIN_FILE_FOR_ADC "sin_dac1_for_adc.txt"
00053
00054 #define NUM_CYCLES 150
00055 #define DATAPOINTS_TO_CYCLE 100
00056
00057 #define BUFFER_SAMPLE_SIZE (NUM_CYCLES * DATAPOINTS_TO_CYCLE) // 100 datapoints to a cycle, 100
cycles
00058 #define BUFFER_BYTES_SIZE (BUFFER_SAMPLE_SIZE * sizeof(int))
00059
00060 #define INIT_BUFFER_SIZE 100
00061 #define BUFFER_SIZE_INCREMENT 100
00062
00063 #define ADC_SAMPLE_RATE 106000
00064 #define DAC_CONVERSION_RATE 105468
00065
00066 #define DMA_BUFFER0 0
00067 #define DAC0 0
00068 #define ADC0 0
00069 #define ADC1 1
00070 #define NUM_BUFFERS_1 1
00071
00072 #define GET_MODE(ms) ((ms) & 0xF)
00073 #define GET_STATUS(ms) ((ms) >> 4)
00074
00075 #define SAMPLES_TO_DISCARD 50
00076
00077 #define MAX_ERROR_COUNT 1000
00078
00079 #define ONE_MINUTE_MSEC (60 * 1000)
00080 #define ONE_SECOND_MSEC 1000
00081
00082
00083 #define RUN_TIMER 0
00084 #define ADC_COLLECTION 1
00085
00089
00090 static char *program_name;
00091
00092 volatile int exit_program = 0;
00093 volatile int interrupt_count = 0;
00094 volatile int function_block = 0;
00095
00096 struct DM35424_Board_Descriptor *board;
00097 struct DM35424_Function_Block my_dac[DM35424_NUM_DAC_ON_BOARD];
00098 struct DM35424_Function_Block my_adc[DM35424_NUM_ADC_ON_BOARD];
00099
00100 static int **local_data_buffer[DM35424_NUM_ADC_ON_BOARD];
00101
00102 double initial_rms_value[DM35424_NUM_ADC_ON_BOARD][10];
00103 double rms_delta[DM35424_NUM_ADC_ON_BOARD][10];
00104
00105 int adc_num_converted = 0;
00106
00107 static int interrupts_expected = 0;
00108 volatile int buffers_copied = 0;
00109

```

```

00110 struct timeval start[5], end[5];
00111
00112 long timeout_msec[5];
00113
00114
00115 void load_dacs_with_sine_wave();
00116 void start_all_dacs_with_dma();
00117 void write_buffers_to_file(int adc_num, char *filename);
00118 void write_adc_to_file(int adc_num, char *filename);
00119 void output_all_interrupts(void);
00120 void calc_rms(int adc_num, char *file_name);
00128
00129 static void usage(void)
00130 {
00131     fprintf(stderr, "\n");
00132     fprintf(stderr, "%s\n", program_name);
00133     fprintf(stderr, "\n");
00134     fprintf(stderr, "Usage: %s [--help] --minor MINOR --dump --hours HOURS --adc ADC --output_rms
--output_adc\n", program_name);
00135     fprintf(stderr, "\n");
00136     fprintf(stderr, "    --help:      Display usage information and exit.\n");
00137     fprintf(stderr, "\n");
00138     fprintf(stderr, "    --minor:    Use specified minor device.\n");
00139     fprintf(stderr, "        MINOR:      Device minor number (>= 0).\n");
00140     fprintf(stderr, "\n");
00141     fprintf(stderr, "    --dump:     Have ADC output written to file (1 pass only).\n");
00142     fprintf(stderr, "\n");
00143     fprintf(stderr, "    --hours:    Run for specific number of hours\n");
00144     fprintf(stderr, "        HOURS:      Number of hours to run.\n");
00145     fprintf(stderr, "\n");
00146     fprintf(stderr, "    --adc:      Collect data from a specific ADC\n");
00147     fprintf(stderr, "        ADC:        ADC to collect data from (0 or 1)\n");
00148     fprintf(stderr, "\n");
00149     fprintf(stderr, "    --output_rms Collect RMS data. Must be used with --hours.\n");
00150     fprintf(stderr, "\n");
00151     fprintf(stderr, "    --output_adc Collect unprocessed ADC data every minute, for channel
0. Must be used with --hours.\n");
00152     fprintf(stderr, "\n");
00153
00154     fprintf(stderr, "\n");
00155     exit(EXIT_FAILURE);
00156 }
00157
00158
00159 void start_timeout_timer(long timeout_val, unsigned int which_timer) {
00160
00161     gettimeofday(&start[which_timer], NULL);
00162     gettimeofday(&end[which_timer], NULL);
00163
00164     timeout_msec[which_timer] = timeout_val;
00165
00166 }
00167
00168 int timeout_has_expired(unsigned int which_timer) {
00169
00170     long time_diff;
00171
00172     gettimeofday(&end[which_timer], NULL);
00173     time_diff = DM35424_Get_Time_Diff(end[which_timer], start[which_timer]) / 1.0e3;
00174
00175     return (time_diff > timeout_msec[which_timer]);
00176 }
00177
00178
00194
00195 static void sigint_handler(int signal_number)
00196 {
00197     exit_program = 0xff;
00198 }
00199
00200
00201 void check_lib_result(int returnVal, char *message) {
00202
00203     if (returnVal != 0) {
00204
00205         error(EXIT_FAILURE, errno, "\nERROR: %s", message);
00206
00207     }
00208 }
00209
00210
00211
00212 void check_condition(int cond, char *message) {
00213     if (cond != 0) {
00214         printf("\nERROR: %s\n\n", message);
00215         exit(1);
00216     }

```

```

00217 }
00218
00219
00220 void Output_Channel_Status(struct DM35424_Board_Descriptor *handle,
00221                          const struct DM35424_Function_Block *func_block,
00222                          unsigned int channel)
00223 {
00224     int result;
00225     unsigned int current_buffer;
00226     uint32_t current_count;
00227     int current_action;
00228     int status_overflow;
00229     int status_underflow;
00230     int status_used;
00231     int status_invalid;
00232     int status_complete;
00233
00234     result = DM35424_Dma_Status(handle,
00235                                func_block,
00236                                channel,
00237                                &current_buffer,
00238                                &current_count,
00239                                &current_action,
00240                                &status_overflow,
00241                                &status_underflow,
00242                                &status_used,
00243                                &status_invalid,
00244                                &status_complete);
00245
00246
00247     check_lib_result(result, "Error getting DMA status");
00248
00249
00250     printf("FB%d Ch%d DMA Status: Current Buffer: %u Count: %ul Action: 0x%x Status: "
00251           "Ov: %d Un: %d Used: %d Inv: %d Comp: %d\n", func_block->fb_num, channel,
00252           current_buffer, current_count, current_action,
00253           status_overflow, status_underflow, status_used,
00254           status_invalid, status_complete);
00255 }
00256
00257
00258 void ISR(struct dm35424_ioctl_interrupt_info_request int_info)
00259 {
00260
00261     char message[200];
00262     int channel_has_interrupt = 0;
00263     unsigned int channel = 0;
00264     int result = 0;
00265     int adc_num = -1;
00266     int adc_index = 0;
00267     int error_interrupt;
00268     unsigned int buffer_to_get = 0;
00269
00270     function_block = int_info.interrupt_fb & 0x7FFFFFFF;
00271     sprintf(message, "Interrupt error occured in FB%u.\n", function_block);
00272
00273     if (!interrupts_expected) {
00274         if (int_info.interrupt_fb < 0) {
00275             fprintf(stdout, "### DMA Interrupt received when not expected from function
00276 block %d.\n", function_block);
00277         }
00278         else {
00279             fprintf(stdout, "### Non-DMA Interrupt received when not expected from
00280 function block %d.\n", function_block);
00281         }
00282         output_all_interrupts();
00283     }
00284
00285     check_condition(int_info.error_occurred, message);
00286
00287     if (int_info.valid_interrupt ) {
00288
00289         while (adc_num < 0 && adc_index < DM35424_NUM_ADC_ON_BOARD) {
00290             if (my_adc[adc_index].fb_num == function_block) {
00291                 adc_num = adc_index;
00292             }
00293
00294             adc_index ++;
00295         }
00296
00297         sprintf(message, "ISR: Could not match function block number (%d) to an ADC.",
00298                 function_block);
00299
00300         check_condition(adc_num < 0, message);
00301

```

```

00302         // It's a DMA interrupt
00303         if (int_info.interrupt_fb < 0) {
00304
00305             // These should only be giving an interrupt if they're
00306             // done, which means their DMA has halted, so pause them
00307             // so they don't overrun.
00308             result = DM35424_Adc_Pause(board,
00309                                     &my_adc[adc_num]);
00310             check_lib_result(result, "Error pausing ADC.");
00311
00312
00313             result = DM35424_Dma_Find_Interrupt(board,
00314                                                &my_adc[adc_num],
00315                                                &channel,
00316                                                &channel_has_interrupt,
00317                                                &error_interrupt);
00318             check_lib_result(result, "Error finding channel with interrupt");
00319             //fprintf(stdout, "[Interrupt] ADC%d.\n", adc_num);
00320             while (channel_has_interrupt && (channel < (my_adc[adc_num].num_dma_channels -
00321 1))) {
00322
00323                 if (error_interrupt) {
00324                     printf("DMA channel %d error!\n",
00325                           channel);
00326
00327                     Output_Channel_Status(board,
00328                                          &my_adc[adc_num],
00329                                          channel);
00330
00331                     check_condition(1, "Error with DMA.");
00332                 }
00333                 //fprintf(stdout, "%d ", channel);
00334                 buffer_to_get = 0;
00335                 while (buffer_to_get < my_adc[adc_num].num_dma_buffers) {
00336                     result = DM35424_Dma_Read(board,
00337                                              &my_adc[adc_num],
00338                                              channel,
00339                                              buffer_to_get,
00340                                              BUFFER_BYTES_SIZE,
00341                                              local_data_buffer[adc_num][channel][buffer_to_get]);
00342
00343                     buffers_copied++;
00344                     check_lib_result(result, "Error getting DMA buffer");
00345
00346                     result = DM35424_Dma_Reset_Buffer(board,
00347                                                       &my_adc[adc_num],
00348                                                       channel,
00349                                                       buffer_to_get);
00350
00351                     check_lib_result(result, "Error resetting buffer");
00352
00353                     buffer_to_get++;
00354                 }
00355
00356                 result = DM35424_Dma_Clear_Interrupt(board,
00357                                                       &my_adc[adc_num],
00358                                                       channel,
00359                                                       NO_CLEAR_INTERRUPT,
00360                                                       NO_CLEAR_INTERRUPT,
00361                                                       NO_CLEAR_INTERRUPT,
00362                                                       NO_CLEAR_INTERRUPT,
00363                                                       CLEAR_INTERRUPT);
00364
00365
00366                 result = DM35424_Dma_Find_Interrupt(board,
00367                                                       &my_adc[adc_num],
00368                                                       &channel,
00369                                                       &channel_has_interrupt,
00370                                                       &error_interrupt);
00371
00372                 check_lib_result(result, "Error finding channel with interrupt");
00373                 interrupt_count++;
00374             }
00375         }
00376         if (error_interrupt) {
00377             printf("DMA channel %d error!\n",
00378                   channel);
00379
00380             Output_Channel_Status(board,
00381                                  &my_adc[adc_num],
00382                                  channel);
00383
00384             check_condition(1, "Error with DMA.");
00385         }
00386         //fprintf(stdout, "\n");

```

```

00387
00388
00389     }
00390     else {
00391         printf("*** Process non-DMA interrupt for FB 0x%x.\n",
00392             int_info.interrupt_fb);
00393     }
00394     if (interrupt_count < 8) {
00395         result = DM35424_Adc_Start(board,
00396             &my_adc[adc_num]);
00397         fprintf(stdout, "Restarting ADC%d.\n", adc_num);
00398         check_lib_result(result, "Error pausing ADC.");
00399     }
00400     result = DM35424_Gbc_Ack_Interrupt(board);
00401
00402     check_lib_result(result, "Error calling ACK interrupt.");
00403
00404 }
00405
00406 }
00407
00408
00409 int main(int argument_count, char **arguments)
00410 {
00411
00412     unsigned long int minor = 0;
00413     int result;
00414     char message[200];
00415
00416     struct sigaction signal_action;
00417
00418     uint8_t buff_status;
00419     uint8_t buff_control;
00420     uint32_t buff_size;
00421     int buffer_count = 0;
00422     int buff = 0;
00423
00424     int adc_num = 0;
00425     int dac_num = 0;
00426     int channel = 0;
00427     uint32_t actual_rate;
00428     uint32_t adc_actual_rate = 0;
00429     int clock = 0;
00430
00431     int help_option_given = 0;
00432     int minor_option_given = 0;
00433     uint32_t num_samples = 0;
00434
00435     unsigned int num_adc_captured = 0;
00436     float hours_to_run = 0.0f;
00437     int dump_to_file = 0;
00438
00439     int timed_run = 0;
00440     int status;
00441     int output_rms = 0;
00442     int output_adc = 0;
00443
00444     int adc_to_use = 0;
00445     unsigned int start_adc = 0;
00446     unsigned int end_adc = DM35424_NUM_ADC_ON_BOARD;
00447     unsigned int error_count = 0;
00448
00449     char *invalid_char_p;
00450     struct option options[] = {
00451         {"help", 0, 0, HELP_OPTION},
00452         {"minor", 1, 0, MINOR_OPTION},
00453         {"dump", 0, 0, DUMP_OPTION},
00454         {"hours", 1, 0, HOURS_OPTION},
00455         {"output_rms", 0, 0, OUTPUT_RMS_OPTION},
00456         {"output_adc", 0, 0, OUTPUT_ADC_OPTION},
00457         {"adc", 1, 0, ADC_OPTION},
00458         {0, 0, 0, 0}
00459     };
00460     time_t timer;
00461     uint32_t adc_counters[DM35424_NUM_ADC_ON_BOARD];
00462     uint32_t dac_counters[DM35424_NUM_DAC_ON_BOARD];
00463     uint32_t count_value;
00464     char dump_file_name[DM35424_NUM_ADC_ON_BOARD][200];
00465     char rms_file_name[DM35424_NUM_ADC_ON_BOARD][200];
00466     char adc_file_name[DM35424_NUM_ADC_ON_BOARD][200];
00467     char time_str[200];
00468     char file_prefix[200];
00469
00470     FILE *data_fp;
00471     program_name = arguments[0];
00472
00473     // Show usage, parse arguments

```

```

00474     while (1) {
00475
00476         /*
00477          * Parse the next command line option and any arguments it may require
00478          */
00479
00480         status = getopt_long(argument_count,
00481                             arguments, "", options, NULL);
00482
00483         /*
00484          * If getopt_long() returned -1, then all options have been processed
00485          */
00486
00487         if (status == -1) {
00488             break;
00489         }
00490
00491         /*
00492          * Figure out what getopt_long() found
00493          */
00494
00495         switch (status) {
00496
00497             /*#####
00498              User entered '--help'
00499              ##### */
00500             case HELP_OPTION:
00501
00502                 /*
00503                  * Refuse to accept duplicate '--help' options
00504                  */
00505
00506                 if (help_option_given) {
00507                     error(0, 0, "ERROR: Duplicate option '--help'");
00508                     usage();
00509                 }
00510
00511                 /*
00512                  * '--help' option has been seen
00513                  */
00514
00515                 help_option_given = 0xFF;
00516                 break;
00517
00518             /*#####
00519              User entered '--minor'
00520              ##### */
00521             case MINOR_OPTION:
00522
00523                 /*
00524                  * Refuse to accept duplicate '--minor' options
00525                  */
00526
00527                 if (minor_option_given) {
00528                     error(0, 0,
00529                          "ERROR: Duplicate option '--minor'");
00530                     usage();
00531                 }
00532
00533                 /*
00534                  * Convert option argument string to unsigned long integer
00535                  */
00536
00537                 errno = 0;
00538
00539                 minor = strtoul(optarg, &invalid_char_p, 10);
00540
00541                 /*
00542                  * Catch unsigned long int overflow
00543                  */
00544
00545                 if ((minor == ULONG_MAX)
00546                     && (errno == ERANGE)) {
00547                     error(0, 0,
00548                          "ERROR: Device minor number caused numeric overflow");
00549                     usage();
00550                 }
00551
00552                 /*
00553                  * Catch argument strings with valid decimal prefixes, for
00554                  * example "1q", and argument strings which cannot be converted,
00555                  * for example "abc1"
00556                  */
00557
00558                 if ((*invalid_char_p != '\0')
00559                     || (invalid_char_p == optarg)) {
00560                     error(0, 0,

```

```

00561         "ERROR: Non-decimal device minor number");
00562         usage();
00563     }
00564
00565     /*
00566     * '--minor' option has been seen
00567     */
00568
00569     minor_option_given = 0xFF;
00570     break;
00571
00572     /*#####
00573     User entered '--file'
00574     ##### */
00575     case DUMP_OPTION:
00576         dump_to_file = 1;
00577
00578         break;
00579     /*#####
00580     User entered '--rms'
00581     ##### */
00582     case OUTPUT_RMS_OPTION:
00583         output_rms = 1;
00584
00585         break;
00586     /*#####
00587     User entered '--adc'
00588     ##### */
00589     case OUTPUT_ADC_OPTION:
00590         output_adc = 1;
00591
00592         break;
00593     /*#####
00594     User entered '--hours'
00595     ##### */
00596     case HOURS_OPTION:
00597         /*
00598         * Convert option argument string to unsigned long integer
00599         */
00600
00601         errno = 0;
00602
00603         hours_to_run = strtouf(optarg, &invalid_char_p);
00604
00605         if (hours_to_run <= 0 || hours_to_run > 12.0f) {
00606             error(0, 0, "ERROR: Hours must be positive float, and "
00607                    "no greater than 12.");
00608             usage();
00609         }
00610         break;
00611
00612     /*#####
00613     User entered '--adc'
00614     ##### */
00615     case ADC_OPTION:
00616
00617         /*
00618         * Convert option argument string to unsigned long integer
00619         */
00620
00621         errno = 0;
00622
00623         adc_to_use = strtoul(optarg, &invalid_char_p, 10);
00624
00625         /*
00626         * Catch unsigned long int overflow
00627         */
00628
00629         if ((adc_to_use == ULONG_MAX)
00630             && (errno == ERANGE)) {
00631             error(0, 0,
00632                    "ERROR: ADC number caused numeric overflow");
00633             usage();
00634         }
00635
00636         /*
00637         * Catch argument strings with valid decimal prefixes, for
00638         * example "1k", and argument strings which cannot be converted,
00639         * for example "abc1"
00640         */
00641
00642         if ((*invalid_char_p != '\0')
00643             || (invalid_char_p == optarg)) {
00644             error(0, 0,
00645                    "ERROR: Non-decimal ADC number");
00646             usage();
00647         }

```



```

00648
00649         start_adc = adc_to_use;
00650         end_adc = start_adc + 1;
00651         break;
00652
00653
00654         case '?':
00655             usage();
00656             break;
00657
00658         /*#####
00659            getopt_long() returned unexpected value
00660            ##### */
00661         default:
00662             error(EXIT_FAILURE,
00663                   0,
00664                   "ERROR: getopt_long() returned unexpected value %#x",
00665                   status);
00666             break;
00667     }
00668 }
00669
00670 /*
00671  * Recognize '--help' option before any others
00672  */
00673
00674 if (help_option_given) {
00675     usage();
00676 }
00677
00678 /*
00679  * '--minor' option must be given
00680  */
00681
00682 if (minor_option_given == 0x00) {
00683     error(0, 0, "ERROR: Option '--minor' is required");
00684     usage();
00685 }
00686
00687 timed_run = hours_to_run > 0.001f;
00688
00689 if (dump_to_file && timed_run) {
00690     error(0, 0, "ERROR: Dump option cannot be used with other options.");
00691     usage();
00692 }
00693
00694
00695 if (!dump_to_file && !timed_run) {
00696     error(0, 0, "ERROR: Must choose either --dump or --hours.");
00697     usage();
00698 }
00699
00700 if (timed_run && !(output_adc || output_rms)) {
00701     error(0, 0, "ERROR: Must include --adc and/or --rms when using --hours.");
00702     usage();
00703 }
00704
00705
00706 interrupts_expected = 0;
00707
00708 signal_action.sa_handler = sigint_handler;
00709 sigfillset(&(signal_action.sa_mask));
00710 signal_action.sa_flags = 0;
00711
00712 if (sigaction(SIGINT, &signal_action, NULL) < 0) {
00713     error(EXIT_FAILURE, errno, "ERROR: sigaction() FAILED");
00714 }
00715
00716 adc_num_converted = 0;
00717 printf("Opening board....");
00718 result = DM35424_Board_Open(minor, &board);
00719
00720 check_lib_result(result, "Could not open board");
00721
00722 printf("success.\nResetting board....");
00723 result = DM35424_Gbc_Board_Reset(board);
00724
00725 check_lib_result(result, "Could not reset board");
00726 printf("success.\n");
00727
00728 timer = time(NULL);
00729 strftime(file_prefix, 200, "%m_%d_%y__%H_%M_%S", localtime(&timer));
00730 strftime(time_str, 200, "%a, %b %d, %Y @ %H:%M:%S", localtime(&timer));
00731 for (adc_num = start_adc; adc_num < end_adc; adc_num++) {
00732     if (dump_to_file) {
00733
00734         sprintf(dump_file_name[adc_num], "%s_dump_adc%d.csv", file_prefix, adc_num);

```

```

00735         data_fp = fopen(dump_file_name[adc_num], "w");
00736
00737         if (data_fp == NULL) {
00738             error(0, 0, "ERROR: Could not create data file.");
00739         }
00740
00741         fprintf(data_fp, "Started %s\n", time_str);
00742         fclose(data_fp);
00743     }
00744     if (timed_run) {
00745         if (output_adc) {
00746             sprintf(adc_file_name[adc_num], "%s_data_adc%d.csv", file_prefix,
adc_num);
00747             data_fp = fopen(adc_file_name[adc_num], "w");
00748
00749             if (data_fp == NULL) {
00750                 error(0, 0, "ERROR: Could not create data file.");
00751             }
00752
00753             fprintf(data_fp, "Started %s\n", time_str);
00754             fclose(data_fp);
00755
00756         }
00757         if (output_rms) {
00758             sprintf(rms_file_name[adc_num], "%s_data_rms%d.csv", file_prefix,
adc_num);
00759             data_fp = fopen(rms_file_name[adc_num], "w");
00760
00761             if (data_fp == NULL) {
00762                 error(0, 0, "ERROR: Could not create data file.");
00763             }
00764
00765             fprintf(data_fp, "Started %s\n", time_str);
00766             fclose(data_fp);
00767
00768         }
00769     }
00770 }
00771
00772 }
00773
00774
00775
00776 for (adc_num = start_adc; adc_num < end_adc; adc_num++) {
00777
00778
00779     result = DM35424_Adc_Open(board,
00780                               adc_num,
00781                               &my_adc[adc_num]);
00782
00783     check_lib_result(result, "Could not open ADC");
00784
00785
00786     fprintf(stdout, "Opening ADC%d, with %d DMA channels (%d buffers each)\n",
00787            adc_num, my_adc[adc_num].num_dma_channels, my_adc[adc_num].num_dma_buffers);
00788
00789     check_condition(my_adc[adc_num].num_dma_channels != DM35424_NUM_ADC_DMA_CHANNELS,
00790                    "Number of ADC DMA channels does not match expected.");
00791     check_condition(my_adc[adc_num].num_dma_buffers != DM35424_NUM_ADC_DMA_BUFFERS,
00792                    "Number of ADC DMA buffers does not match expected.");
00793
00794     result = DM35424_Adc_Reset(board,
00795                                &my_adc[adc_num]);
00796
00797     check_lib_result(result, "Error resetting ADC.");
00798
00799     for (clock = DM35424_CLK_SRC_GBL2; clock <= DM35424_CLK_SRC_GBL7; clock++) {
00800         result = DM35424_Adc_Set_Clock_Source_Global(board,
00801                                                       &my_adc[adc_num],
00802                                                       clock,
DM35424_ADC_CLK_EVENT_DISABLE);
00803
00804         sprintf(message, "ADC%d: Could not set clock src GBL%d to DISABLE.",
00805                adc_num, clock);
00806         check_lib_result(result, message);
00807     }
00808
00809
00810
00811     result = DM35424_Adc_Interrupt_Set_Config(board,
00812                                                &my_adc[adc_num],
00813                                                DM35424_ADC_INT_ALL_MASK,
00814                                                INTERRUPT_DISABLE);
00815
00816     check_lib_result(result, "Error disabling interrupts.");
00817
00818     result = DM35424_Adc_Set_Clock_Src(board,

```

```

00819             &my_adc[adc_num],
00820             DM35424_CLK_SRC_IMMEDIATE);
00821
00822     check_lib_result(result, "Error setting ADC clock");
00823
00824     result = DM35424_Adc_Set_Pre_Trigger_Samples(board,
00825             &my_adc[adc_num],
00826             0);
00827
00828     check_lib_result(result, "Error setting Pre capture values.");
00829
00830     result = DM35424_Adc_Set_Post_Stop_Samples(board,
00831             &my_adc[adc_num],
00832             0);
00833
00834     check_lib_result(result, "Error setting Post capture values.");
00835
00836
00837     for (channel = 0; channel < my_adc[adc_num].num_dma_channels; channel++) {
00838
00839         result = DM35424_Adc_Channel_Setup(board,
00840             &my_adc[adc_num],
00841             channel,
00842             DM35424_ADC_RNG_BIPOLAR_2_5V,
00843             DM35424_ADC_INPUT_DIFFERENTIAL);
00844
00845         DM35424_Check_Result(result, "Error setting up channel.");
00846
00847     }
00848
00849     result = DM35424_Adc_Interrupt_Clear_Status(board,
00850             &my_adc[adc_num],
00851             DM35424_ADC_INT_ALL_MASK);
00852
00853     check_lib_result(result, "Error clearing interrupt status.");
00854
00855
00856     result = DM35424_Adc_Initialize(board,
00857             &my_adc[adc_num]);
00858
00859     check_lib_result(result, "Failed or timed out initializing ADC.");
00860
00861     fprintf(stdout, "ADC%d: Requested rate: %d Actual rate: %d\n", adc_num,
00862             ADC_SAMPLE_RATE,
00863             adc_actual_rate);
00864
00865     for (channel = 0; channel < my_adc[adc_num].num_dma_channels - 1; channel++) {
00866
00867         fprintf(stdout, "Initializing ADC DMA Channel %d....", channel);
00868         result = DM35424_Dma_Initialize(board,
00869             &my_adc[adc_num],
00870             channel,
00871             my_adc[adc_num].num_dma_buffers,
00872             BUFFER_BYTES_SIZE);
00873
00874         check_lib_result(result, "Error initializing DMA");
00875
00876
00877         result = DM35424_Dma_Setup(board,
00878             &my_adc[adc_num],
00879             channel,
00880             DM35424_DMA_SETUP_DIRECTION_READ,
00881             NOT_IGNORE_USED);
00882
00883         check_lib_result(result, "Error configuring DMA");
00884
00885         fprintf(stdout, "success.\nSetting ADC DMA Interrupts.....");
00886         result = DM35424_Dma_Configure_Interrupts(board,
00887             &my_adc[adc_num],
00888             channel,
00889             INTERRUPT_ENABLE,
00890             ERROR_INTR_ENABLE);
00891
00892         check_lib_result(result, "Error setting DMA Interrupts");
00893         fprintf(stdout, "success!\nADC%d Channel %d buffer setup:\n",
00894             adc_num, channel);
00895
00896
00897         for (buff = 0; buff < my_adc[adc_num].num_dma_buffers; buff++) {
00898
00899             if (buff == (my_adc[adc_num].num_dma_buffers - 1)) {
00900
00901                 result = DM35424_Dma_Buffer_Setup(board,
00902                     &my_adc[adc_num],
00903                     channel,
00904                     buff,
00905

```

```

00906                                     BUFFER_VALID,
00907                                     BUFFER_HALT,
00908                                     BUFFER_NO_LOOP,
00909                                     BUFFER_INTERRUPT);
00910
00911         check_lib_result(result, "Error setting buffer control.");
00912     }
00913     else {
00914         result = DM35424_Dma_Buffer_Setup(board,
00915                                     &my_adc[adc_num],
00916                                     channel,
00917                                     buff,
00918                                     BUFFER_VALID,
00919                                     BUFFER_NO_HALT,
00920                                     BUFFER_NO_LOOP,
00921                                     BUFFER_NO_INTERRUPT);
00922
00923         check_lib_result(result, "Error setting buffer control.");
00924     }
00925
00926     result = DM35424_Dma_Buffer_Status(board,
00927                                     &my_adc[adc_num],
00928                                     channel,
00929                                     buff,
00930                                     &buff_status,
00931                                     &buff_control,
00932                                     &buff_size);
00933
00934     check_lib_result(result, "Error getting buffer status.");
00935
00936     fprintf(stdout, "    Buffer %d: Stat: 0x%x Ctrl: 0x%x Size: %d\n",
00937            buff, buff_status, buff_control, buff_size);
00938 }
00939
00940
00941     result = DM35424_Adc_Channel_Setup(board,
00942                                     &my_adc[adc_num],
00943                                     channel,
00944                                     DM35424_ADC_RNG_BIPOLAR_2_5V,
00945                                     DM35424_ADC_INPUT_DIFFERENTIAL);
00946
00947     DM35424_Check_Result(result, "Error setting up channel.");
00948
00949
00950 }
00951
00952 // Grab the sample counter, because we're going to compare to
00953 // this later to make sure sampling hasn't started yet.
00954 result = DM35424_Adc_Get_Sample_Count(board,
00955                                     &my_adc[adc_num],
00956                                     &(adc_counters[adc_num]));
00957
00958 check_lib_result(result, "Error getting ADC sample count.");
00959
00960 }
00961
00962 }
00963
00964 for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD; dac_num++) {
00965
00966     result = DM35424_Dac_Open(board,
00967                             dac_num,
00968                             &my_dac[dac_num]);
00969
00970     check_lib_result(result, "Could not open DAC");
00971
00972     fprintf(stdout, "Opening DAC%d, with %d DMA channels (%d buffers each)\n",
00973            dac_num, my_dac[dac_num].num_dma_channels, my_dac[dac_num].num_dma_buffers);
00974
00975     check_condition(my_dac[dac_num].num_dma_channels != DM35424_NUM_DAC_DMA_CHANNELS,
00976                    "Number of DAC DMA channels does not match expected.");
00977     check_condition(my_dac[dac_num].num_dma_buffers != DM35424_NUM_DAC_DMA_BUFFERS,
00978                    "Number of DAC DMA buffers does not match expected.");
00979
00980     result = DM35424_Dac_Reset(board,
00981                             &my_dac[dac_num]);
00982
00983     check_lib_result(result, "Error stopping DAC.");
00984
00985     for (clock = DM35424_CLK_SRC_GBL2; clock <= DM35424_CLK_SRC_GBL7; clock++) {
00986         result = DM35424_Dac_Set_Clock_Source_Global(board,
00987                                     &my_dac[dac_num],
00988                                     clock,

```

```

DM35424_DAC_CLK_EVENT_DISABLE);
00993         sprintf(message, "DAC%d: Could not set clock source GBL%d to DISABLE.",
00994                     dac_num, clock);
00995         check_lib_result(result, message);
00996     }
00997 }
00998
00999 // Disable and clear all interrupt registers
01000 result = DM35424_Dac_Interrupt_Set_Config(board,
01001                                           &my_dac[dac_num],
01002                                           DM35424_DAC_INT_ALL_MASK,
01003                                           INTERRUPT_DISABLE);
01004
01005 check_lib_result(result, "Error disabling interrupts.");
01006
01007 result = DM35424_Dac_Interrupt_Clear_Status(board,
01008                                           &my_dac[dac_num],
01009                                           DM35424_DAC_INT_ALL_MASK);
01010
01011 check_lib_result(result, "Error clearing interrupt status.");
01012
01013 result = DM35424_Dac_Set_Clock_Src(board,
01014                                     &my_dac[dac_num],
01015                                     DM35424_CLK_SRC_IMMEDIATE);
01016
01017 check_lib_result(result, "Error setting DAC clock");
01018
01019 fprintf(stdout, "DAC%d: Requested rate: %d ", dac_num,
01020         adc_actual_rate);
01021
01022 result = DM35424_Dac_Set_Conversion_Rate(board,
01023                                           &my_dac[dac_num],
01024                                           adc_actual_rate,
01025                                           &actual_rate);
01026
01027 check_lib_result(result, "Error setting sample rate");
01028
01029 fprintf(stdout, "Actual rate: %d\n", actual_rate);
01030
01031 // Disable and clear all interrupt registers
01032 result = DM35424_Dac_Interrupt_Set_Config(board,
01033                                           &my_dac[dac_num],
01034                                           DM35424_DAC_INT_ALL_MASK,
01035                                           INTERRUPT_DISABLE);
01036
01037 check_lib_result(result, "Error disabling interrupts.");
01038
01039 result = DM35424_Dac_Interrupt_Clear_Status(board,
01040                                           &my_dac[dac_num],
01041                                           DM35424_DAC_INT_ALL_MASK);
01042
01043 check_lib_result(result, "Error clearing interrupt status.");
01044
01045 for (channel = 0; channel < my_dac[dac_num].num_dma_channels - 1; channel++) {
01046     result = DM35424_Dma_Clear_Interrupt(board,
01047                                           &my_dac[dac_num],
01048                                           channel,
01049                                           CLEAR_INTERRUPT,
01050                                           CLEAR_INTERRUPT,
01051                                           CLEAR_INTERRUPT,
01052                                           CLEAR_INTERRUPT,
01053                                           CLEAR_INTERRUPT);
01054
01055     check_lib_result(result, "Error clearing channel interrupts.");
01056 }
01057
01058 // Grab the conversion counter, because we're going to compare to
01059 // this later to make sure conversion hasn't started yet.
01060 result = DM35424_Dac_Get_Conversion_Count(board,
01061                                           &my_dac[dac_num],
01062                                           &(dac_counters[dac_num]));
01063
01064 check_lib_result(result, "Error getting DAC conversion count.");
01065 }
01066
01067 load_dacs_with_sine_wave();
01068
01069 // This has to be done before anything else ties into the CLK SRC GBL,
01070 // as its default state is the system clock if nothing is driving it.
01071 result = DM35424_Adc_Set_Clock_Source_Global(board,
01072                                             &my_adc[start_adc],

```

```

01079                                     DM35424_CLK_SRC_GBL2,
01080                                     DM35424_ADC_CLK_EVENT_START_TRIG);
01081     check_lib_result(result, "Error setting ADC start trigger to global clock 2.");
01082
01083
01084     // Allocate local memory for data.
01085     for (adc_num = start_adc; adc_num < end_adc; adc_num++) {
01086         local_data_buffer[adc_num] = malloc(sizeof(int**) * (my_adc[adc_num].num_dma_channels
01087 - 1));
01088
01089         for (channel = 0; channel < (my_adc[adc_num].num_dma_channels - 1); channel++) {
01090             local_data_buffer[adc_num][channel] = malloc(sizeof(int*) *
01091 my_adc[adc_num].num_dma_buffers);
01092             check_condition(local_data_buffer[adc_num][channel] == NULL, "Could not
allocate for local buffer");
01093
01094             for (buff = 0; buff < my_adc[adc_num].num_dma_buffers; buff++) {
01095                 local_data_buffer[adc_num][channel][buff] = malloc(BUFFER_BYTES_SIZE);
01096
01097                 check_condition(local_data_buffer[adc_num][channel][buff] == NULL,
01098 "Could not allocate for local buffer");
01099                 buffer_count++;
01100             }
01101         }
01102     }
01103
01104     fprintf(stdout, "Installing user ISR ....");
01105     result = DM35424_General_InstallISR(board, ISR);
01106     check_lib_result(result, "DM35424_General_InstallISR()");
01107
01108     fprintf(stdout, "success.\n");
01109
01110     start_all_dacs_with_dma();
01111
01112
01113
01114
01115
01116     for (adc_num = start_adc; adc_num < end_adc; adc_num++) {
01117
01118         result = DM35424_Adc_Set_Start_Trigger(board,
01119 &my_adc[adc_num],
01120 DM35424_CLK_SRC_IMMEDIATE);
01121         check_lib_result(result, "Error setting start trigger.");
01122
01123
01124         result = DM35424_Adc_Set_Stop_Trigger(board,
01125 &my_adc[adc_num],
01126 DM35424_CLK_SRC_NEVER);
01127         check_lib_result(result, "Error setting stop trigger.");
01128
01129         fprintf(stdout, "ADC%d ready.\n", adc_num);
01130
01131
01132         result = DM35424_Adc_Get_Sample_Count(board,
01133 &my_adc[adc_num],
01134 &count_value);
01135
01136         check_lib_result(result, "Error getting ADC conversion count.");
01137
01138         sprintf(message, "ADC%d: Expected counters to be same. Start count: %d End count:
01139 %d\n",
01140 adc_num,
01141 adc_counters[adc_num],
01142 count_value);
01143
01144         check_condition(adc_counters[adc_num] != count_value, message);
01145     }
01146     fprintf(stdout, "success.\nStarting ADC%d.\n", start_adc);
01147
01148
01149     /* We need to run a certain number of samples through the ADC before we start
01150 * actually collecting data. These first samples, which are only a problem
01151 * during the first start (when a clock rate has changed) are not actual
01152 * data, so we want to make sure they don't end up in the FIFO or DMA.
01153 */
01154     for (adc_num = start_adc; adc_num < end_adc; adc_num++) {
01155
01156         /* Set DMA to clear so samples will not be collected in the FIFO,
01157         and no overrun interrupt will occur
01158         */
01159         for (channel = 0; channel < my_adc[adc_num].num_dma_channels - 1; channel++) {
01160             result = DM35424_Dma_Clear(board,

```

```

01161                                     &my_adc[adc_num],
01162                                     channel);
01163
01164         check_lib_result(result, "Error starting DMA.");
01165     }
01166
01167     result = DM35424_Adc_Start(board,
01168                                &my_adc[adc_num]);
01169
01170
01171     check_lib_result(result, "Error starting ADC.");
01172
01173     result = DM35424_Adc_Get_Sample_Count(board,
01174                                            &my_adc[adc_num],
01175                                            &num_samples);
01176
01177     check_lib_result(result, "Error getting sample count");
01178
01179     while (num_samples < SAMPLES_TO_DISCARD) {
01180         result = DM35424_Adc_Get_Sample_Count(board,
01181                                                &my_adc[adc_num],
01182                                                &num_samples);
01183
01184         check_lib_result(result, "Error getting sample count");
01185     }
01186
01187     result = DM35424_Adc_Pause(board,
01188                                &my_adc[adc_num]);
01189
01190     check_lib_result(result, "Error pausing ADC.");
01191
01192     for (channel = 0; channel < my_adc[adc_num].num_dma_channels - 1; channel++) {
01193         result = DM35424_Dma_Start(board,
01194                                    &my_adc[adc_num],
01195                                    channel);
01196
01197         check_lib_result(result, "Error starting DMA.");
01198     }
01199 }
01200
01201 interrupt_count = 0;
01202 interrupts_expected = 1;
01203
01204 result = DM35424_Adc_Start(board,
01205                             &my_adc[start_adc]);
01206
01207
01208 check_lib_result(result, "Error starting ADC.");
01209
01210 adc_num = 0;
01211
01212 if (timed_run) {
01213     start_timeout_timer((long) (hours_to_run * 60.0f * 60.0f * 1000.0f), RUN_TIMER);
01214     start_timeout_timer(ONE_MINUTE_MSEC, ADC_COLLECTION);
01215 }
01216
01217 timer = time(NULL);
01218 strftime(time_str, 200, "%a, %b %d, %Y @ %H:%M:%S", localtime(&timer));
01219 fprintf(stdout, "%s Data collection started.\n", time_str);
01220
01221 /*****
01222  * MAIN LOOP
01223  *****/
01224
01225 while (!exit_program && error_count < MAX_ERROR_COUNT) {
01226     if (interrupt_count == 8) {
01227         timer = time(NULL);
01228         strftime(time_str, 200, "%a, %b %d, %Y @ %H:%M:%S", localtime(&timer));
01229         interrupts_expected = 0;
01230         int adc_index = 0;
01231         adc_num = -1;
01232         while (adc_num < 0 && adc_index < DM35424_NUM_ADC_ON_BOARD) {
01233             if (my_adc[adc_index].fb_num == function_block) {
01234                 adc_num = adc_index;
01235             }
01236             adc_index++;
01237         }
01238
01239         sprintf(message, "Could not match function block number (%d) to an ADC.",
01240                 function_block);
01241         check_condition(adc_num < 0, message);
01242     }
01243 }

```

```

01246
01247
01248         if (dump_to_file) {
01249
01250             write_buffers_to_file(adc_num, dump_file_name[adc_num]);
01251
01252             fprintf(stdout, "%s Buffer data for ADC%d dumped to file.\n",
time_str, adc_num);
01253
01254             if (adc_num == (end_adc - 1)) {
01255                 exit_program = 1;
01256             }
01257         else {
01258
01259             if (output_rms) {
01260
01261                 calc_rms(adc_num, rms_file_name[adc_num]);
01262                 fprintf(stdout, "%s RMS data for ADC %d saved to file.\n",
01263                     time_str, adc_num);
01264             }
01265
01266             if (output_adc) {
01267
01268                 if (timeout_has_expired(ADC_COLLECTION)) {
01269                     write_adc_to_file(adc_num, adc_file_name[adc_num]);
01270                     fprintf(stdout, "%s Data for ADC %d (Chan 0, Buff 0)
saved to file.\n",
01271                         time_str, adc_num);
01272                     num_adc_captured ++;
01273                     if (num_adc_captured == (end_adc - start_adc)) {
01274                         start_timeout_timer(ONE_MINUTE_MSEC,
ADC_COLLECTION);
01275                         num_adc_captured = 0;
01276                     }
01277                 }
01278             }
01279
01280             if (timed_run) {
01281                 if (timeout_has_expired(RUN_TIMER)) {
01282                     exit_program = 1;
01283                 }
01284             }
01285         }
01286     buffers_copied = 0;
01287
01288     // Move to the next ADC
01289     if (start_adc == 0) {
01290         adc_num = (adc_num + 1) % end_adc;
01291     }
01292     for (channel = 0; channel < my_adc[adc_num].num_dma_channels - 1; channel++ )
01293     {
01294         result = DM35424_Dma_Clear(board,
01295             &my_adc[adc_num],
01296             channel);
01297
01298         check_lib_result(result, "Error clearing DMA.");
01299
01300         result = DM35424_Dma_Start(board,
01301             &my_adc[adc_num],
01302             channel);
01303
01304         check_lib_result(result, "Error starting DMA.");
01305     }
01306
01307     interrupts_expected = 1;
01308     interrupt_count = 0;
01309     result = DM35424_Adc_Start(board,
01310         &my_adc[adc_num]);
01311     check_lib_result(result, "Error restarting ADC.");
01312
01313
01314
01315
01316     }
01317     else {
01318         DM35424_Micro_Sleep(100);
01319     }
01320 }
01321
01322 printf("success!\nReleasing allocated memory.....");
01323
01324 timer = time(NULL);
01325 strftime(time_str, 200, "%a, %b %d, %Y @ %H:%M:%S", localtime(&timer));
01326
01327 for (adc_num = start_adc; adc_num < end_adc; adc_num ++ ) {
01328

```



```

01329         for (channel = 0; channel < my_adc[adc_num].num_dma_channels - 1; channel++) {
01330             for (buff = 0; buff < my_adc[adc_num].num_dma_buffers; buff++) {
01331                 free(local_data_buffer[adc_num][channel][buff]);
01332             }
01333             free(local_data_buffer[adc_num][channel]);
01334         }
01335         free(local_data_buffer[adc_num]);
01336
01337         if (dump_to_file) {
01338             data_fp = fopen(dump_file_name[adc_num], "a");
01339             check_condition(data_fp == NULL, "Error opening data file to put ending
01340 time.");
01341
01342             fprintf(data_fp, "Stopped %s\n", time_str);
01343             fclose(data_fp);
01344         }
01345         if (output_rms) {
01346             data_fp = fopen(rms_file_name[adc_num], "a");
01347             check_condition(data_fp == NULL, "Error opening data file to put ending
01348 time.");
01349             fprintf(data_fp, "Stopped %s\n", time_str);
01350             fclose(data_fp);
01351         }
01352         if (output_adc) {
01353             data_fp = fopen(adc_file_name[adc_num], "a");
01354             check_condition(data_fp == NULL, "Error opening data file to put ending
01355 time.");
01356             fprintf(data_fp, "Stopped %s\n", time_str);
01357             fclose(data_fp);
01358         }
01359         printf("success.\nClosing Board....");
01360         result = DM35424_Board_Close(board);
01361         check_lib_result(result, "Error closing board.");
01362         printf("success.\nExample program successfully completed.\n\n");
01363         return 0;
01364     }
01365 }
01366
01367 void load_dacs_with_sine_wave() {
01368     int num_samples = 0, max_num_samples = 0;
01369     FILE *input_file_ptr = NULL;
01370     int *dac0_buffer, *dac1_buffer;
01371     char input_str[200];
01372     int dac_num, channel;
01373     int result = 0;
01374
01375     input_file_ptr = fopen(DAC0_SIN_FILE_FOR_ADC, "rt");
01376
01377     check_condition(input_file_ptr == NULL, "Could not open file: "DAC0_SIN_FILE_FOR_ADC);
01378
01379     dac0_buffer = (int *) malloc(INIT_BUFFER_SIZE * sizeof(int));
01380     dac1_buffer = (int *) malloc(INIT_BUFFER_SIZE * sizeof(int));
01381     max_num_samples = INIT_BUFFER_SIZE;
01382
01383     check_condition(dac0_buffer == NULL, "Could not allocate memory for data buffer.");
01384     check_condition(dac1_buffer == NULL, "Could not allocate memory for data buffer.");
01385     while (fgets(input_str, 200, input_file_ptr) != NULL) {
01386         sscanf(input_str, "%d", dac0_buffer + num_samples);
01387         num_samples++;
01388
01389         // Check if we need more memory
01390         if (num_samples == max_num_samples) {
01391             max_num_samples += BUFFER_SIZE_INCREMENT;
01392             dac0_buffer = (int *) realloc(dac0_buffer, max_num_samples * sizeof(int));
01393             check_condition(dac0_buffer == NULL, "Could not re-allocate buffer to larger
01394 size.");
01395         }
01396     }

```

```

01412     }
01413
01414     fclose(input_file_ptr);
01415     num_samples = 0;
01416     input_file_ptr = fopen(DAC1_SIN_FILE_FOR_ADC, "rt");
01417     max_num_samples = INIT_BUFFER_SIZE;
01418
01419     check_condition(input_file_ptr == NULL, "Could not open file: "DAC1_SIN_FILE_FOR_ADC);
01420
01421     while (fgets(input_str, 200, input_file_ptr) != NULL) {
01422
01423         sscanf(input_str, "%d", dac1_buffer + num_samples);
01424         num_samples++;
01425
01426         // Check if we need more memory
01427         if (num_samples == max_num_samples) {
01428             max_num_samples += BUFFER_SIZE_INCREMENT;
01429             dac1_buffer = (int *) realloc(dac1_buffer, max_num_samples * sizeof(int));
01430             check_condition(dac1_buffer == NULL, "Could not re-allocate buffer to larger
size.");
01431         }
01432     }
01433
01434     fclose(input_file_ptr);
01435
01436     for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD; dac_num++) {
01437
01438         for (channel = 0; channel < (my_dac[dac_num].num_dma_channels - 1); channel++) {
01439
01440             result = DM35424_Dma_Stop(board,
01441                                     &my_dac[dac_num],
01442                                     channel);
01443
01444             check_lib_result(result, "Error stopping DMA.");
01445
01446             result = DM35424_Dma_Initialize(board,
01447                                             &my_dac[dac_num],
01448                                             channel,
01449                                             my_dac[dac_num].num_dma_buffers,
01450                                             num_samples * sizeof(int));
01451
01452             check_lib_result(result, "Error initializing DMA");
01453
01454             result = DM35424_Dma_Setup(board,
01455                                       &my_dac[dac_num],
01456                                       channel,
01457                                       DM35424_DMA_SETUP_DIRECTION_WRITE,
01458                                       IGNORE_UNUSED);
01459
01460             check_lib_result(result, "Error configuring DMA");
01461
01462             result = DM35424_Dma_Buffer_Setup(board,
01463                                               &my_dac[dac_num],
01464                                               channel,
01465                                               DMA_BUFFER0,
01466                                               BUFFER_VALID,
01467                                               BUFFER_NO_HALT,
01468                                               BUFFER_LOOP,
01469                                               BUFFER_NO_INTERRUPT);
01470
01471             check_lib_result(result, "Error setting up buffer control.");
01472
01473             if (channel == 0 || channel % 2 == 0) {
01474
01475                 result = DM35424_Dma_Write(board,
01476                                             &my_dac[dac_num],
01477                                             channel,
01478                                             DMA_BUFFER0,
01479                                             num_samples * sizeof(int),
01480                                             dac0_buffer);
01481
01482                 check_lib_result(result, "Writing to DMA buffer failed");
01483             }
01484             else {
01485
01486                 result = DM35424_Dma_Write(board,
01487                                             &my_dac[dac_num],
01488                                             channel,
01489                                             DMA_BUFFER0,
01490                                             num_samples * sizeof(int),
01491                                             dac1_buffer);
01492
01493                 check_lib_result(result, "Writing to DMA buffer failed");
01494             }
01495         }
01496     }
01497

```

```

01498         }
01499     }
01500 }
01501
01502     free(dac0_buffer);
01503     free(dac1_buffer);
01504 }
01505 }
01506
01507
01508
01509
01510
01511 void start_all_dacs_with_dma() {
01512
01513     int dac_num;
01514     int channel;
01515     int result;
01516
01517     for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD; dac_num++) {
01518
01519
01520
01521         result = DM35424_Dac_Set_Start_Trigger(board,
01522                                                 &my_dac[dac_num],
01523                                                 DM35424_CLK_SRC_GBL2);
01524
01525         check_lib_result(result, "Error setting start trigger for DAC.");
01526
01527         result = DM35424_Dac_Set_Stop_Trigger(board,
01528                                                 &my_dac[dac_num],
01529                                                 DM35424_CLK_SRC_NEVER);
01530
01531         check_lib_result(result, "Error setting stop trigger for DAC.");
01532
01533         for (channel = 0; channel < my_dac[dac_num].num_dma_channels - 1; channel++) {
01534             result = DM35424_Dma_Start(board,
01535                                         &my_dac[dac_num],
01536                                         channel);
01537
01538             check_lib_result(result, "Error starting DMA.");
01539         }
01540
01541         result = DM35424_Dac_Start(board,
01542                                     &my_dac[dac_num]);
01543
01544         check_lib_result(result, "Error starting DAC");
01545
01546         fprintf(stdout, "DAC%d: Started.\n", dac_num);
01547
01548
01549     }
01550 }
01551 }
01552
01553
01554 void write_buffers_to_file(int adc_num, char *dump_file_name)
01555 {
01556
01557     FILE *fp;
01558     int channel, buffer;
01559     int index = 0;
01560
01561
01562     fp = fopen(dump_file_name, "a");
01563
01564     if (fp == NULL) {
01565         error(EXIT_FAILURE, errno, "open() FAILED on output data file.\n");
01566     }
01567
01568     for (buffer = 0; buffer < my_adc[adc_num].num_dma_buffers; buffer++) {
01569
01570         for (index = 0; index < BUFFER_BYTES_SIZE / 4; index++) {
01571             for (channel = 0; channel < my_adc[adc_num].num_dma_channels - 1; channel++)
01572             {
01573                 fprintf(fp, "%d,",
01574                         local_data_buffer[adc_num][channel][buffer][index]);
01575             }
01576
01577             fprintf(fp, "\n");
01578         }
01579
01580         fclose(fp);
01581     }
01582 }

```

```

01583
01584 void write_adc_to_file(int adc_num, char *dump_file_name)
01585 {
01586     FILE *fp;
01587     int channel = 0, buffer = 0;
01588     int index = 0;
01589
01590
01591     fp = fopen(dump_file_name, "a");
01592
01593     if (fp == NULL) {
01594         error(EXIT_FAILURE, errno, "open() FAILED on output data file.\n");
01595     }
01596
01597     for (index = 0; index < BUFFER_BYTES_SIZE / 4; index++) {
01598         fprintf(fp, "%d", local_data_buffer[adc_num][channel][buffer][index]);
01599         fprintf(fp, "\n");
01600     }
01601
01602     fclose(fp);
01603 }
01604
01605 void output_all_interrupts(void)
01606 {
01607     unsigned int adc_num;
01608     unsigned int dac_num;
01609     int result;
01610     unsigned int channel;
01611     int has_interrupt = 0;
01612     uint8_t adc_status = 0;
01613     uint8_t adc_enable = 0;
01614     uint16_t dac_status = 0;
01615     uint16_t dac_enable = 0;
01616     int interrupts_found = 0;
01617     int int_enabled = 0;
01618     int error_enabled = 0;
01619
01620     fprintf(stdout, "\n");
01621     for (adc_num = 0; adc_num < DM35424_NUM_ADC_ON_BOARD; adc_num++) {
01622         result = DM35424_Adc_Channel_Find_Interrupt(board,
01623             &my_adc[adc_num],
01624             &channel,
01625             &has_interrupt,
01626             &adc_status,
01627             &adc_enable);
01628         check_lib_result(result, "Error finding interrupts for ADC");
01629
01630         if (has_interrupt) {
01631             interrupts_found++;
01632             fprintf(stdout, "ADC%d Chan %d: Status (0x%x) and Enable (0x%x): 0x%x\n",
01633                 adc_num, channel, adc_status, adc_enable, (adc_status & adc_enable));
01634
01635             for (channel = 0; channel < my_adc[adc_num].num_dma_channels - 1; channel++)
01636             {
01637                 result = DM35424_Dma_Get_Interrupt_Configuration(board,
01638                     &my_adc[adc_num],
01639                     channel,
01640                     &int_enabled,
01641                     &error_enabled);
01642                 check_lib_result(result, "Error getting interrupt configuration.");
01643
01644                 if (int_enabled || error_enabled) {
01645                     interrupts_found++;
01646
01647                     Output_Channel_Status(board,
01648                         &my_adc[adc_num],
01649                         channel);
01650                 }
01651             }
01652         }
01653     }
01654
01655     for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD; dac_num++) {
01656         result = DM35424_Dac_Interrupt_Get_Status(board,
01657             &my_dac[dac_num],

```

```

01667                                     &dac_status);
01668
01669     check_lib_result(result, "Error getting DAC interrupt status.");
01670
01671     result = DM35424_Dac_Interrupt_Get_Config(board,
01672                                               &my_dac[dac_num],
01673                                               &dac_enable);
01674
01675     check_lib_result(result, "Error getting DAC interrupt enable.");
01676
01677     if (dac_status & dac_enable) {
01678         fprintf(stdout, "DAC%d: Status (0x%x) and Enable (0x%x): 0x%x\n",
01679                 dac_num, dac_status, dac_enable, (dac_status & dac_enable));
01680         interrupts_found ++;
01681     }
01682     for (channel = 0; channel < my_dac[dac_num].num_dma_channels - 1; channel ++) {
01683         result = DM35424_Dma_Get_Interrupt_Configuration(board,
01684                                                         &my_dac[dac_num],
01685                                                         channel,
01686                                                         &int_enabled,
01687                                                         &error_enabled);
01688         check_lib_result(result, "Error getting interrupt configuration.");
01689
01690         if (int_enabled || error_enabled) {
01691             interrupts_found ++;
01692             Output_Channel_Status(board,
01693                                  &my_dac[dac_num],
01694                                  channel);
01695         }
01696     }
01697 }
01698
01699 }
01700
01701 if (!interrupts_found) {
01702     fprintf(stdout, "No interrupts were found.\n");
01703 }
01704
01705 }
01706
01707 void calc_rms(int adc_num, char *file_name)
01708 {
01709     int channel;
01710     int buffer;
01711     int index;
01712     int sample_count = 0;
01713     double rms = 0.0;
01714     float rms_volts;
01715     int return_code;
01716     char time_str[200];
01717     time_t timer;
01718     FILE *fp;
01719
01720     timer = time(NULL);
01721
01722     strftime(time_str, 200, "%m/%d/%Y,%H:%M:%S", localtime(&timer));
01723
01724     fp = fopen(file_name, "a");
01725     check_condition(fp == NULL, "Error opening output file.");
01726
01727     fprintf(fp, "%s,", time_str);
01728
01729     for (channel = 0; channel < my_adc[adc_num].num_dma_channels - 1; channel ++) {
01730
01731         rms = 0.0;
01732         sample_count = 0;
01733         for (buffer = 0; buffer < my_adc[adc_num].num_dma_buffers; buffer++) {
01734             for (index = 0; index < BUFFER_BYTES_SIZE / 4; index++) {
01735
01736                 rms += ((double) local_data_buffer[adc_num][channel][buffer][index]) *
01737                        ((double) local_data_buffer[adc_num][channel][buffer][index]);
01738
01739                 sample_count ++;
01740             }
01741         }
01742
01743         rms /= (double) sample_count;
01744
01745         rms = sqrt(rms);
01746
01747         return_code = DM35424_Adc_Sample_To_Volts((int32_t) rms,
01748                                                    &rms_volts);
01749
01750         check_lib_result(return_code, "Error converting sample to volts.");
01751     }
01752 }
01753

```

```

01754             fprintf(fp, "%.8f", rms_volts);
01755
01756
01757
01758
01759     }
01760     //adc_num_converted ++;
01761     fprintf(fp, "\n");
01762     fclose(fp);
01763
01764
01765 }
01766
01767
01768
01769
01770
01771
01772
01773

```

6.9 examples/_non_public/dm35424_ref_adjust_test.c File Reference

Example program which demonstrates the use of the digital potentiometer, to adjust the reference voltage of the DACs and ADCs on the board.

```

#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <unistd.h>
#include <limits.h>
#include <signal.h>
#include <string.h>
#include <getopt.h>
#include <termios.h>
#include <time.h>
#include <sys/time.h>
#include "dm35424_gbc_library.h"
#include "dm35424_adc_library.h"
#include "dm35424_dac_library.h"
#include "dm35424_dma_library.h"
#include "dm35424_ioctl.h"
#include "dm35424_examples.h"
#include "dm35424_util_library.h"
#include "dm35424_board_access.h"
#include "dm35424_types.h"
#include "dm35424.h"
#include "dm35424_ref_adjust_library.h"

```

Functions

- static void **usage** (void)
Print information on stderr about how the program is to be used. After doing so, the program is exited.
- static void **sigint_handler** (int signal_number)
Signal handler for SIGINT Control-C keyboard interrupt.
- int **main** (int argument_count, char **arguments)
The main program.

Variables

- static char * [program_name](#)
- volatile int [exit_program](#) = 0

6.9.1 Detailed Description

Example program which demonstrates the use of the digital potentiometer, to adjust the reference voltage of the DACs and ADCs on the board.

The program will continue to run until CTRL-C is pressed.

```
-----  
This file and its contents are copyright (C) RTD Embedded Technologies,  
Inc. All Rights Reserved.
```

```
This software is licensed as described in the RTD End-User Software License  
Agreement. For a copy of this agreement, refer to the file LICENSE.TXT  
(which should be included with this software) or contact RTD Embedded  
Technologies, Inc.  
-----
```

Id

[dm35424_ref_adjust_test.c](#) 142659 2024-04-26 19:32:32Z lfrankenfield

Definition in file [dm35424_ref_adjust_test.c](#).

6.9.2 Function Documentation

6.9.2.1 main()

```
int main (  
    int argument_count,  
    char ** arguments)
```

The main program.

Parameters

<i>argument_count</i>	
-----------------------	--

Number of args passed on the command line, including the executable name

Parameters

<i>arguments</i>	
------------------	--

Pointer to array of character strings, which are the args themselves.

Return values

0	
---	--

Success

Return values

Non-Zero	
----------	--

Failure.

Definition at line 158 of file [dm35424_ref_adjust_test.c](#).

References [ADC_0](#), [ADC_1](#), [board](#), [CHANNEL_0](#), [CHANNEL_1](#), [DAC_0](#), [DAC_2](#), [DEFAULT_RATE](#), [DM35424_Adc_Ad_Config_Set_I](#), [DM35424_Adc_Channel_Get_Last_Sample\(\)](#), [DM35424_Adc_Channel_Setup\(\)](#), [DM35424_Adc_Initialize\(\)](#), [DM35424_ADC_INPUT_DIFFERENTIAL](#), [DM35424_ADC_MODE_CONFIG_HIGH_SPEED](#), [DM35424_Adc_Open\(\)](#), [DM35424_Adc_Reset\(\)](#), [DM35424_ADC_RNG_BIPOLAR_2_5V](#), [DM35424_Adc_Sample_To_Volts\(\)](#), [DM35424_Adc_Set_Clock_Sr](#), [DM35424_Adc_Set_Post_Stop_Samples\(\)](#), [DM35424_Adc_Set_Pre_Trigger_Samples\(\)](#), [DM35424_Adc_Set_Sample_Rate\(\)](#), [DM35424_Adc_Set_Start_Trigger\(\)](#), [DM35424_Adc_Set_Stop_Trigger\(\)](#), [DM35424_Adc_Start\(\)](#), [DM35424_Adc_Uninitialize\(\)](#), [DM35424_Board_Close\(\)](#), [DM35424_Board_Open\(\)](#), [DM35424_Check_Result\(\)](#), [DM35424_CLK_SRC_IMMEDIATE](#), [DM35424_CLK_SRC_NEVER](#), [DM35424_Dac_Open\(\)](#), [DM35424_Dac_Reset\(\)](#), [DM35424_Dac_Set_Last_Conversion\(\)](#), [DM35424_Dac_Volts_To_Conv\(\)](#), [DM35424_Gbc_Board_Reset\(\)](#), [DM35424_Micro_Sleep\(\)](#), [DM35424_Ref_Adjust_Open\(\)](#), [DM35424_Ref_Adjust_Write_Adc_To_NonVolatile\(\)](#), [DM35424_Ref_Adjust_Write_Adc_To_Volatile\(\)](#), [exit_program](#), [HELP_OPTION](#), [MINOR_OPTION](#), [my_adc](#), [program_name](#), [REF_0](#), [REF_1](#), [REF_NUM_OPTION](#), [sigint_handler\(\)](#), and [usage\(\)](#).

6.9.2.2 sigint_handler()

```
void sigint_handler (
    int signal_number) [static]
```

Signal handler for SIGINT Control-C keyboard interrupt.

Parameters

<i>signal_number</i>	
----------------------	--

Signal number passed in from the kernel.

Warning

One must be extremely careful about what functions are called from a signal handler.

Definition at line 125 of file [dm35424_ref_adjust_test.c](#).

References [exit_program](#).

6.9.3 Variable Documentation

6.9.3.1 exit_program

```
volatile int exit_program = 0
```

Boolean indicating whether or not to exit the program.

Definition at line 76 of file [dm35424_ref_adjust_test.c](#).

6.9.3.2 program_name

```
char* program_name [static]
```

Name of the program as invoked on the command line

Definition at line 70 of file [dm35424_ref_adjust_test.c](#).

6.10 dm35424_ref_adjust_test.c

[Go to the documentation of this file.](#)

```
00001
00031
00032 #include <stdio.h>
00033 #include <stddef.h>
00034 #include <stdlib.h>
00035 #include <errno.h>
00036 #include <error.h>
00037 #include <unistd.h>
00038 #include <limits.h>
00039 #include <signal.h>
00040 #include <string.h>
00041 #include <getopt.h>
00042 #include <signal.h>
00043 #include <termios.h>
00044 #include <time.h>
00045 #include <sys/time.h>
00046
00047
00048 #include "dm35424_gbc_library.h"
00049 #include "dm35424_adc_library.h"
00050 #include "dm35424_dac_library.h"
00051 #include "dm35424_dma_library.h"
00052 #include "dm35424_ioctl.h"
00053 #include "dm35424_examples.h"
00054 #include "dm35424_util_library.h"
00055 #include "dm35424_board_access.h"
00056 #include "dm35424_types.h"
00057 #include "dm35424.h"
00058 #include "dm35424_ref_adjust_library.h"
00059
00060 #define DEFAULT_RATE      80000
00061
00062 #define MAX_REF_ADJUST    0xFF
00063
00064 #define SAMPLES_TO_AVG    100000
00065
00066
00070 static char *program_name;
00071
00072
00076 volatile int exit_program = 0;
00077
00085
00086 static void usage(void)
00087 {
00088     fprintf(stderr, "\n");
00089     fprintf(stderr, "%s\n", program_name);
00090     fprintf(stderr, "\n");
```

```

00091     fprintf(stderr,
00092         "Usage: %s [--help] --minor MINOR [--ref REF_NUM]\n",
00093         program_name);
00094     fprintf(stderr, "\n");
00095     fprintf(stderr,
00096         "    --help:      Display usage information and exit.\n");
00097     fprintf(stderr, "\n");
00098     fprintf(stderr, "    --minor:      Use specified minor device.\n");
00099     fprintf(stderr, "        MINOR:      Device minor number (>= 0).\n");
00100     fprintf(stderr, "    --ref:       Use specified reference adjust.\n");
00101     fprintf(stderr, "        REF_NUM:    Ref Adjust to use (0 or 1).\n");
00102
00103     fprintf(stderr, "\n");
00104     exit(EXIT_FAILURE);
00105 }
00106
00107
00108
00109
00110
00111
00112
00113
00114
00115 static void sigint_handler(int signal_number)
00116 {
00117     exit_program = 0xff;
00118 }
00119
00120
00121
00122
00123 int main(int argument_count, char **arguments)
00124 {
00125     struct DM35424_Board_Descriptor *board;
00126     struct DM35424_Function_Block my_adc;
00127     struct DM35424_Function_Block my_dac;
00128     struct DM35424_Function_Block my_ref;
00129
00130     unsigned long int minor = 0;
00131     int result;
00132     int my_value;
00133     char last_action[100];
00134
00135     unsigned int adc_num = ADC_0;
00136     unsigned int ref_num = REF_0;
00137     unsigned int rate = DEFAULT_RATE;
00138     unsigned int actual_rate;
00139     unsigned int dac_num = DAC_0;
00140
00141     int adc_average = 0;
00142
00143     int adc_sample_count = 0;
00144
00145     int help_option_given = 0;
00146     int minor_option_given = 0;
00147     int status;
00148     struct sigaction signal_action;
00149
00150     uint8_t ref_value = 0;
00151     float volts = 0;
00152
00153     int16_t max_value, min_value;
00154     struct termios old_tio, new_tio;
00155     unsigned char keypress = '0';
00156     int forward = 1;
00157
00158     char *invalid_char_p;
00159     struct option options[] = {
00160         {"help", 0, 0, HELP_OPTION},
00161         {"minor", 1, 0, MINOR_OPTION},
00162         {"ref", 1, 0, REF_NUM_OPTION},
00163         {0, 0, 0, 0}
00164     };
00165
00166     program_name = arguments[0];
00167
00168     // Show usage, parse arguments
00169     while (1) {
00170         /*
00171          * Parse the next command line option and any arguments it may require
00172          */
00173
00174         status = getopt_long(argument_count,
00175                             arguments, "", options, NULL);
00176
00177         /*
00178          * If getopt_long() returned -1, then all options have been processed
00179          */
00180
00181         if (status == -1) {
00182             break;
00183         }
00184     }

```

```

00220
00221      /*
00222      * Figure out what getopt_long() found
00223      */
00224
00225      switch (status) {
00226
00227          /*#####
00228           User entered '--help'
00229           ##### */
00230      case HELP_OPTION:
00231
00232          /*
00233           * Refuse to accept duplicate '--help' options
00234           */
00235
00236          if (help_option_given) {
00237              error(0, 0, "ERROR: Duplicate option '--help'");
00238              usage();
00239          }
00240
00241          /*
00242           * '--help' option has been seen
00243           */
00244
00245          help_option_given = 0xFF;
00246          break;
00247
00248          /*#####
00249           User entered '--minor'
00250           ##### */
00251      case MINOR_OPTION:
00252
00253          /*
00254           * Refuse to accept duplicate '--minor' options
00255           */
00256
00257          if (minor_option_given) {
00258              error(0, 0,
00259                  "ERROR: Duplicate option '--minor'");
00260              usage();
00261          }
00262
00263          /*
00264           * Convert option argument string to unsigned long integer
00265           */
00266
00267          errno = 0;
00268
00269          minor = strtoul(optarg, &invalid_char_p, 10);
00270
00271          /*
00272           * Catch unsigned long int overflow
00273           */
00274
00275          if ((minor == ULONG_MAX)
00276              && (errno == ERANGE)) {
00277              error(0, 0,
00278                  "ERROR: Device minor number caused numeric overflow");
00279              usage();
00280          }
00281
00282          /*
00283           * Catch argument strings with valid decimal prefixes, for
00284           * example "1q", and argument strings which cannot be converted,
00285           * for example "abc1"
00286           */
00287
00288          if ((*invalid_char_p != '\0')
00289              || (invalid_char_p == optarg)) {
00290              error(0, 0,
00291                  "ERROR: Non-decimal device minor number");
00292              usage();
00293          }
00294
00295          /*
00296           * '--minor' option has been seen
00297           */
00298
00299          minor_option_given = 0xFF;
00300          break;
00301
00302
00303          /*#####
00304           User entered '--ref'
00305           ##### */
00306

```

```

00307         case REF_NUM_OPTION:
00308
00309             /*
00310              * Convert option argument string to unsigned long integer
00311              */
00312
00313             errno = 0;
00314
00315             ref_num = strtoul(optarg, &invalid_char_p, 10);
00316
00317             /*
00318              * Catch unsigned long int overflow
00319              */
00320
00321             if ((ref_num == ULONG_MAX)
00322                 && (errno == ERANGE)) {
00323                 error(0, 0,
00324                     "ERROR: Ref number caused numeric overflow");
00325                 usage();
00326             }
00327
00328             /*
00329              * Catch argument strings with valid decimal prefixes, for
00330              * example "1q", and argument strings which cannot be converted,
00331              * for example "abc1"
00332              */
00333
00334             if ((*invalid_char_p != '\0')
00335                 || (invalid_char_p == optarg)) {
00336                 error(0, 0, "ERROR: Non-decimal Ref number");
00337                 usage();
00338             }
00339
00340             if (ref_num != REF_0 && ref_num != REF_1) {
00341                 error(0, 0,
00342                     "ERROR: Reference Adjust number must be 0 or 1.");
00343                 usage();
00344             }
00345             break;
00346
00347             /*#####
00348              User entered unsupported option
00349              ##### */
00350         case '?':
00351             usage();
00352             break;
00353
00354             /*#####
00355              getopt_long() returned unexpected value
00356              ##### */
00357         default:
00358             error(EXIT_FAILURE,
00359                 0,
00360                 "ERROR: getopt_long() returned unexpected value %#x",
00361                 status);
00362             break;
00363     }
00364 }
00365
00366 /*
00367  * Recognize '--help' option before any others
00368  */
00369
00370 if (help_option_given) {
00371     usage();
00372 }
00373
00374 /*
00375  * '--minor' option must be given
00376  */
00377
00378 if (minor_option_given == 0x00) {
00379     error(0, 0, "ERROR: Option '--minor' is required");
00380     usage();
00381 }
00382
00383 signal_action.sa_handler = sigint_handler;
00384 sigfillset(&(signal_action.sa_mask));
00385 signal_action.sa_flags = 0;
00386
00387 if (sigaction(SIGINT, &signal_action, NULL) < 0) {
00388     error(EXIT_FAILURE, errno, "ERROR: sigaction() FAILED");
00389 }
00390
00391 tcgetattr(STDIN_FILENO, &old_tio);
00392
00393

```

```
00394     new_tio = old_tio;
00395
00396     new_tio.c_lflag &= ~ICANON;
00397     new_tio.c_lflag &= ~ECHO;
00398     new_tio.c_lflag &= ~ISIG;
00399     new_tio.c_cc[VMIN] = 0;
00400     new_tio.c_cc[VTIME] = 0;
00401
00402
00403     tcsetattr(STDIN_FILENO, TCSANOW, &new_tio);
00404
00405
00406     printf("Opening board.....");
00407     result = DM35424_Board_Open(minor, &board);
00408
00409     DM35424_Check_Result(result, "Could not open board");
00410     printf("success.\nResetting board.....");
00411     result = DM35424_Gbc_Board_Reset(board);
00412
00413     DM35424_Check_Result(result, "Could not reset board");
00414     printf("success.\n");
00415
00416     if (ref_num == REF_0) {
00417         adc_num = ADC_0;
00418         dac_num = DAC_0;
00419     }
00420     else {
00421         adc_num = ADC_1;
00422         dac_num = DAC_2;
00423     }
00424
00425     printf("success.\nOpening DAC.....");
00426
00427
00428     result = DM35424_Dac_Open(board, dac_num, &my_dac);
00429
00430     DM35424_Check_Result(result, "Could not open DAC");
00431
00432     printf("Found DAC%u, with %d DMA channels (%d buffers each)\n",
00433           dac_num, my_dac.num_dma_channels,
00434           my_dac.num_dma_buffers);
00435
00436     result = DM35424_Dac_Reset(board,
00437                                &my_dac);
00438
00439     DM35424_Check_Result(result, "Error resetting DAC.");
00440
00441     result = DM35424_Dac_Set_Last_Conversion(board,
00442                                              &my_dac,
00443                                              CHANNEL_0,
00444                                              0,
00445                                              0);
00446
00447     DM35424_Check_Result(result, "Error setting last conversion for DAC0 Chan 0.");
00448
00449     result = DM35424_Dac_Set_Last_Conversion(board,
00450                                              &my_dac,
00451                                              CHANNEL_1,
00452                                              0,
00453                                              0);
00454
00455     DM35424_Check_Result(result, "Error setting last conversion for DAC0 Chan 1.");
00456
00457     printf("Opening Reference Adjustment....\n");
00458
00459     result = DM35424_Ref_Adjust_Open(board,
00460                                     ref_num,
00461                                     &my_ref);
00462
00463     DM35424_Check_Result(result, "Error opening reference adjustment.");
00464
00465     printf("Opening ADC.....\n");
00466
00467     result = DM35424_Adc_Open(board, adc_num, &my_adc);
00468
00469     DM35424_Check_Result(result, "Could not open ADC");
00470
00471     printf("Found ADC%d, with %d DMA channels (%d buffers each)\n",
00472           adc_num, my_adc.num_dma_channels, my_adc.num_dma_buffers);
00473
00474     result = DM35424_Adc_Uninitialize(board, &my_adc);
00475
00476     DM35424_Check_Result(result, "Error setting ADC to Uninitialized.");
00477
00478     result = DM35424_Adc_Channel_Setup(board,
00479                                         &my_adc,
00480                                         CHANNEL_0,
```

```
00481                                     DM35424_ADC_RNG_BIPOLAR_2_5V,
00482                                     DM35424_ADC_INPUT_DIFFERENTIAL);
00483
00484     DM35424_Check_Result(result, "Error setting up channel.");
00485
00486
00487     result = DM35424_Adc_Ad_Config_Set_Mode(board,
00488                                             &my_adc,
00489                                             DM35424_ADC_MODE_CONFIG_HIGH_SPEED);
00490
00491     DM35424_Check_Result(result, "Error setting AD config.");
00492
00493     result = DM35424_Adc_Set_Start_Trigger(board,
00494                                             &my_adc,
00495                                             DM35424_CLK_SRC_IMMEDIATE);
00496     DM35424_Check_Result(result, "Error setting start trigger.");
00497
00498     result = DM35424_Adc_Set_Stop_Trigger(board,
00499                                           &my_adc,
00500                                           DM35424_CLK_SRC_NEVER);
00501     DM35424_Check_Result(result, "Error setting stop trigger.");
00502
00503     result = DM35424_Adc_Set_Clock_Src(board,
00504                                        &my_adc,
00505                                        DM35424_CLK_SRC_IMMEDIATE);
00506
00507     DM35424_Check_Result(result, "Error setting ADC clock");
00508
00509     result = DM35424_Adc_Set_Sample_Rate(board,
00510                                         &my_adc, rate, &actual_rate);
00511
00512     DM35424_Check_Result(result, "Failed to set sample rate for ADC.");
00513     fprintf(stdout,
00514            "ADC:%d Rate requested: %d Actual Rate Achieved: %d\n",
00515            adc_num, rate, actual_rate);
00516
00517     result = DM35424_Adc_Set_Pre_Trigger_Samples(board, &my_adc, 0);
00518     DM35424_Check_Result(result, "Error setting pre-capture samples.");
00519
00520     result = DM35424_Adc_Set_Post_Stop_Samples(board, &my_adc, 0);
00521
00522     DM35424_Check_Result(result, "Error setting post-capture samples.");
00523
00524     result = DM35424_Adc_Initialize(board, &my_adc);
00525
00526     DM35424_Check_Result(result, "Failed or timed out initializing ADC.");
00527
00528     printf("Starting ADC.....\n");
00529
00530     result = DM35424_Adc_Start(board, &my_adc);
00531
00532     printf("Getting 100000 sample average....");
00533
00534     for (adc_sample_count = 0; adc_sample_count < SAMPLES_TO_AVG; adc_sample_count++) {
00535         result =
00536             DM35424_Adc_Channel_Get_Last_Sample(board,
00537                                                  &my_adc,
00538                                                  CHANNEL_0,
00539                                                  &my_value);
00540
00541         DM35424_Check_Result(result,
00542                             "Error getting ADC value.");
00543
00544         adc_average += my_value;
00545     }
00546
00547     adc_average /= SAMPLES_TO_AVG;
00548
00549     printf("done.  (%d)\n", adc_average);
00550
00551     result = DM35424_Dac_Volts_To_Conv(3.5f, &max_value);
00552
00553     DM35424_Check_Result(result,
00554                         "Error converting value to conversion counts.");
00555
00556     result = DM35424_Dac_Set_Last_Conversion(board,
00557                                              &my_dac,
00558                                              CHANNEL_0,
00559                                              0,
00560                                              max_value);
00561
00562     DM35424_Check_Result(result, "Error setting last conversion for DAC Chan 0.");
00563
00564     result = DM35424_Dac_Volts_To_Conv(1.5f, &min_value);
```

```

00568
00569     DM35424_Check_Result(result,
00570         "Error converting value to conversion counts.");
00571
00572
00573     result = DM35424_Dac_Set_Last_Conversion(board,
00574         &my_dac,
00575         CHANNEL_1,
00576         0,
00577         min_value);
00578
00579     DM35424_Check_Result(result, "Error setting last conversion for DAC Chan 1.");
00580
00581     printf("\n\n=====\\n");
00582     printf("Press i to increase the ref adjust value.\\n");
00583     printf("Press d to decrease the ref adjust value.\\n");
00584     printf("Press v to write the ref adjust value to volatile memory.\\n");
00585     printf("Press n to write the ref adjust value to non-volatile memory.\\n");
00586     printf("Press s to swap the DAC voltages (when using loopback).\\n");
00587     printf("Press a to re-average the ADC value with DACs set to zero.\\n");
00588     printf("Press q to quit.\\n\\n");
00589     printf("Ref Value      ADC%d Chan 0      ADC Average @ 0      Last Action\\n", adc_num);
00590     printf("-----\\n");
00591
00592     DM35424_Check_Result(result, "Error starting ADC");
00593
00594     strcpy(last_action, "Waiting for input.");
00595
00596     while (!exit_program && keypress != 'q') {
00597
00598         keypress = '0';
00599         keypress = getchar();
00600
00601         if (keypress == 'i') {
00602
00603             if (ref_value < MAX_REF_ADJUST) {
00604                 ref_value ++;
00605                 strcpy(last_action, "Reference value increased.");
00606             }
00607
00608         }
00609
00610         if (keypress == 'd') {
00611             if (ref_value > 0) {
00612                 ref_value --;
00613                 strcpy(last_action, "Reference value decreased.");
00614             }
00615
00616         }
00617
00618         if (keypress == 's') {
00619             if (forward) {
00620                 result = DM35424_Dac_Volts_To_Conv(1.5f, &max_value);
00621
00622                 DM35424_Check_Result(result,
00623                     "Error converting value to conversion counts.");
00624
00625                 result = DM35424_Dac_Set_Last_Conversion(board,
00626                     &my_dac,
00627                     CHANNEL_0,
00628                     0,
00629                     max_value);
00630
00631                 DM35424_Check_Result(result, "Error setting last conversion for DAC
00632 Chan 0.");
00633
00634                 result = DM35424_Dac_Volts_To_Conv(3.5f, &min_value);
00635
00636                 DM35424_Check_Result(result,
00637                     "Error converting value to conversion counts.");
00638
00639                 result = DM35424_Dac_Set_Last_Conversion(board,
00640                     &my_dac,
00641                     CHANNEL_1,
00642                     0,
00643                     min_value);
00644
00645                 DM35424_Check_Result(result, "Error setting last conversion for DAC
00646 Chan 1.");
00647
00648                 forward = 0;
00649                 strcpy(last_action, "Voltages swapped.");
00650             }
00651             else {

```

```

00652         result = DM35424_Dac_Volts_To_Conv(3.5f, &max_value);
00653
00654         DM35424_Check_Result(result,
00655             "Error converting value to conversion counts.");
00656
00657         result = DM35424_Dac_Set_Last_Conversion(board,
00658             &my_dac,
00659             CHANNEL_0,
00660             0,
00661             max_value);
00662
00663         DM35424_Check_Result(result, "Error setting last conversion for DAC
Chan 0.");
00664
00665         result = DM35424_Dac_Volts_To_Conv(1.5f, &min_value);
00666
00667         DM35424_Check_Result(result,
00668             "Error converting value to conversion counts.");
00669
00670         result = DM35424_Dac_Set_Last_Conversion(board,
00671             &my_dac,
00672             CHANNEL_1,
00673             0,
00674             min_value);
00675
00676         DM35424_Check_Result(result, "Error setting last conversion for DAC
Chan 1.");
00677
00678         forward = 1;
00679         strcpy(last_action, "Voltages swapped.");
00680     }
00681 }
00682
00683
00684
00685 if (keypress == 'a') {
00686     result = DM35424_Dac_Set_Last_Conversion(board,
00687         &my_dac,
00688         CHANNEL_0,
00689         0,
00690         0);
00691
00692     DM35424_Check_Result(result, "Error setting last conversion for DAC0 Chan
0.");
00693
00694     result = DM35424_Dac_Set_Last_Conversion(board,
00695         &my_dac,
00696         CHANNEL_1,
00697         0,
00698         0);
00699
00700     DM35424_Check_Result(result, "Error setting last conversion for DAC0 Chan
1.");
00701
00702     adc_average = 0;
00703     for (adc_sample_count = 0; adc_sample_count < SAMPLES_TO_AVG; adc_sample_count
++) {
00704
00705         result =
00706             DM35424_Adc_Channel_Get_Last_Sample(board,
00707                 &my_adc,
00708                 CHANNEL_0,
00709                 &my_value);
00710
00711         DM35424_Check_Result(result,
00712             "Error getting ADC value.");
00713
00714         adc_average += my_value;
00715     }
00716
00717
00718     adc_average /= SAMPLES_TO_AVG;
00719     sprintf(last_action, "ADC value re-averaged: %d", adc_average);
00720
00721     if (!forward) {
00722         result = DM35424_Dac_Volts_To_Conv(1.5f, &max_value);
00723
00724         DM35424_Check_Result(result,
00725             "Error converting value to conversion counts.");
00726
00727         result = DM35424_Dac_Set_Last_Conversion(board,
00728             &my_dac,
00729             CHANNEL_0,
00730             0,
00731             max_value);
00732
00733

```



```

00734         DM35424_Check_Result(result, "Error setting last conversion for DAC
Chan 0.");
00735
00736         result = DM35424_Dac_Volts_To_Conv(3.5f, &min_value);
00737
00738         DM35424_Check_Result(result,
00739             "Error converting value to conversion counts.");
00740
00741
00742         result = DM35424_Dac_Set_Last_Conversion(board,
00743             &my_dac,
00744             CHANNEL_1,
00745             0,
00746             min_value);
00747
00748         DM35424_Check_Result(result, "Error setting last conversion for DAC
Chan 1.");
00749
00750         }
00751     else {
00752         result = DM35424_Dac_Volts_To_Conv(3.5f, &max_value);
00753
00754         DM35424_Check_Result(result,
00755             "Error converting value to conversion counts.");
00756
00757         result = DM35424_Dac_Set_Last_Conversion(board,
00758             &my_dac,
00759             CHANNEL_0,
00760             0,
00761             max_value);
00762
00763         DM35424_Check_Result(result, "Error setting last conversion for DAC
Chan 0.");
00764
00765         result = DM35424_Dac_Volts_To_Conv(1.5f, &min_value);
00766
00767         DM35424_Check_Result(result,
00768             "Error converting value to conversion counts.");
00769
00770
00771         result = DM35424_Dac_Set_Last_Conversion(board,
00772             &my_dac,
00773             CHANNEL_1,
00774             0,
00775             min_value);
00776
00777         DM35424_Check_Result(result, "Error setting last conversion for DAC
Chan 1.");
00778
00779         }
00780     }
00781 }
00782
00783
00784
00785     if (keypress == 'v') {
00786         result = DM35424_Ref_Adjust_Write_Adc_To_Volatile(
00787             board,
00788             &my_ref,
00789             ref_value);
00790
00791         DM35424_Check_Result(result, "Error writing reference value to ADC volatile
memory.");
00792
00793         strcpy(last_action, "Reference value written to ADC volatile memory.");
00794     }
00795
00796     if (keypress == 'n') {
00797         result = DM35424_Ref_Adjust_Write_Adc_To_NonVolatile(
00798             board,
00799             &my_ref,
00800             ref_value);
00801
00802         DM35424_Check_Result(result, "Error writing reference value to ADC
non-volatile memory.");
00803
00804         strcpy(last_action, "Reference value written to ADC non-volatile memory.");
00805     }
00806
00807     result =
00808         DM35424_Adc_Channel_Get_Last_Sample(board,
00809             &my_adc,
00810             CHANNEL_0,
00811             &my_value);
00812
00813     DM35424_Check_Result(result,
00814         "Error getting ADC value.");
00815
00816     result =

```

```

00815             DM35424_Adc_Sample_To_Volts(my_value - adc_average,
00816                                         &volts);
00817
00818             DM35424_Check_Result(result,
00819                                   "Error converting ADC sample to volts.");
00820
00821             printf("    %3u          %+2.8f    %8d          %s
",
00822                   ref_value, volts, adc_average, last_action);
00823
00824             printf("\r");
00825
00826             DM35424_Micro_Sleep(100);
00827
00828         }
00829
00830         tcsetattr(STDIN_FILENO, TCSANOW, &old_tio);
00831
00832
00833         printf("\n\nStopping Adc %d.....", adc_num);
00834
00835         result = DM35424_Adc_Reset(board, &my_adc);
00836
00837         DM35424_Check_Result(result, "Error starting ADC");
00838
00839         printf("success.\n");
00840
00841         printf("Closing Board\n");
00842         result = DM35424_Board_Close(board);
00843
00844         DM35424_Check_Result(result, "Error closing board.");
00845         printf("Example program successfully completed.\n");
00846         return 0;
00847
00848     }

```

6.11 examples/dm35424_adc.c File Reference

Example program which demonstrates the use of the ADC, setting and responding to interrupts.

```

#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <unistd.h>
#include <limits.h>
#include <signal.h>
#include <getopt.h>
#include "dm35424_gbc_library.h"
#include "dm35424_adc_library.h"
#include "dm35424_dac_library.h"
#include "dm35424_dma_library.h"
#include "dm35424_ioctl.h"
#include "dm35424_examples.h"
#include "dm35424_util_library.h"
#include "dm35424_board_access.h"
#include "dm35424_types.h"
#include "dm35424.h"

```

Macros

- #define `DAC_RATE` 25
- #define `DEFAULT_RATE` 300
- #define `BUFFER_SIZE_SAMPLES` 100
- #define `BUFFER_SIZE_BYTES` (`BUFFER_SIZE_SAMPLES` * `sizeof(int)`)

Functions

- static void **usage** (void)
Print information on stderr about how the program is to be used. After doing so, the program is exited.
- void **ISR** (struct [dm35424_iocctl_interrupt_info_request](#) int_info)
The interrupt subroutine that will execute when an interrupt occurs. It will simply increment a count, which the main program will then trigger from.
- static void [sigint_handler](#) (int signal_number)
Signal handler for SIGINT Control-C keyboard interrupt.
- int [main](#) (int argument_count, char **arguments)
The main program.

Variables

- static char * [program_name](#)
- volatile int [interrupt_count](#) = 0
- volatile int [exit_program](#) = 0

6.11.1 Detailed Description

Example program which demonstrates the use of the ADC, setting and responding to interrupts.

This example program uses an ADC to collect data. An interrupt is generated every time data is collected by the ADC. After acknowledging the interrupt, the program queries the value last taken by the ADC, and the sample counter, and prints them to the screen.

You can put any differential signal you want on the ADC input pins. However, for convenience, this example sets up the DACs to provide a signal for the ADC to measure. In order for that to work, you must loopback the DAC outputs to the ADC inputs. Since there are twice as many ADC inputs as DAC outputs, each DAC output must go to 2 different ADC inputs. DAC Channel 0 would go to ADC Channel 0+ and Channel 1-. DAC Channel 1 would go to ADC Channel 0- and Channel 1+, etc. In this way, all ADC even channels will have the same signal, and all odd channels will have the opposite.

The program will demonstrate all input modes of the ADC: DIFFERENTIAL, SINGLE-ENDED POS, SINGLE-ENDED NEG, and DAC INTERNAL LOOPBACK.

The program will also demonstrate the use of filters. They will be applied to the odd-numbered channels after differential mode.

The program will continue to run until CTRL-C is pressed.

This file and its contents are copyright (C) RTD Embedded Technologies, Inc. All Rights Reserved.

This software is licensed as described in the RTD End-User Software License Agreement. For a copy of this agreement, refer to the file LICENSE.TXT (which should be included with this software) or contact RTD Embedded Technologies, Inc.

Id

[dm35424_adc.c](#) 141531 2024-03-06 21:05:25Z lfrankenfield

Definition in file [dm35424_adc.c](#).

6.11.2 Macro Definition Documentation

6.11.2.1 BUFFER_SIZE_BYTES

```
#define BUFFER_SIZE_BYTES (BUFFER_SIZE_SAMPLES * sizeof(int))
```

Number of bytes in DAC sample buffer

Definition at line 91 of file [dm35424_adc.c](#).

6.11.2.2 BUFFER_SIZE_SAMPLES

```
#define BUFFER_SIZE_SAMPLES 100
```

Number of samples to play out DAC pins

Definition at line 86 of file [dm35424_adc.c](#).

6.11.2.3 DAC_RATE

```
#define DAC_RATE 25
```

DAC rate to use.

Definition at line 76 of file [dm35424_adc.c](#).

Referenced by [main\(\)](#).

6.11.2.4 DEFAULT_RATE

```
#define DEFAULT_RATE 300
```

Rate to run at, if the user does not provide one. (Hz)

Definition at line 81 of file [dm35424_adc.c](#).

6.11.3 Function Documentation

6.11.3.1 main()

```
int main (
    int argument_count,
    char ** arguments)
```

The main program.

Parameters

<i>argument_count</i>	
-----------------------	--

Number of args passed on the command line, including the executable name

Parameters

<i>arguments</i>	
------------------	--

Pointer to array of character strings, which are the args themselves.

Return values

0	
---	--

Success

Return values

<i>Non-Zero</i>	
-----------------	--

Failure. First, setup the DACS. They will produce a sine wave that needs to be looped back to the ADC inputs. This will cause the ADC to see what looks like a max-value sine wave.

We load the even channels with the wave pattern, and the odd channels with the same pattern, but offset by half its length. Doing this gives us an opposing pattern between the even and odd channels, which helps when using DAC for ADC input.

Definition at line 213 of file [dm35424_adc.c](#).

References [ADC_0](#), [board](#), [BUFFER_0](#), [BUFFER_SIZE_BYTES](#), [BUFFER_SIZE_SAMPLES](#), [DAC_RATE](#), [DEFAULT_RATE](#), [DM35424_Adc_Ad_Config_Set_Mode\(\)](#), [DM35424_ADC_CHAN_FILTER_ORDER0](#), [DM35424_ADC_CHAN_FILTER_ORDER1](#), [DM35424_Adc_Channel_Get_Last_Sample\(\)](#), [DM35424_Adc_Channel_Set_Filter\(\)](#), [DM35424_Adc_Channel_Setup\(\)](#), [DM35424_Adc_Get_Sample_Count\(\)](#), [DM35424_Adc_Initialize\(\)](#), [DM35424_ADC_INPUT_DAC_LOOPBACK](#), [DM35424_ADC_INPUT_DIFFERENTIAL](#), [DM35424_ADC_INPUT_SINGLE_ENDED_NEG](#), [DM35424_ADC_INPUT_SINGLE_ENDED_POS](#), [DM35424_ADC_INT_ALL_MASK](#), [DM35424_ADC_INT_SAMPLE_TAKEN_MASK](#), [DM35424_Adc_Interrupt_Clear_Status\(\)](#), [DM35424_Adc_Interrupt_Get_Config\(\)](#), [DM35424_Adc_Interrupt_Get_Status\(\)](#), [DM35424_Adc_Interrupt_Set_Config\(\)](#), [DM35424_ADC_MODE_CONFIG_HIGH_SPEED](#), [DM35424_Adc_Open\(\)](#), [DM35424_Adc_Reset\(\)](#), [DM35424_ADC_RNG_BIPOLAR_5V](#), [DM35424_ADC_RNG_UNIPOLAR_5V](#), [DM35424_Adc_Sample_To_Volts\(\)](#), [DM35424_Adc_Set_Clock_Src\(\)](#), [DM35424_Adc_Set_Post_Stop_Samples\(\)](#), [DM35424_Adc_Set_Pre_Trigger_Samples\(\)](#), [DM35424_Adc_Set_Sample_Rate\(\)](#),

DM35424_Adc_Set_Start_Trigger(), DM35424_Adc_Set_Stop_Trigger(), DM35424_Adc_Start(), DM35424_Adc_Uninitialize(), DM35424_Board_Close(), DM35424_Board_Open(), DM35424_Check_Result(), DM35424_CLK_SRC_IMMEDIATE, DM35424_CLK_SRC_NEVER, DM35424_Dac_Open(), DM35424_Dac_Reset(), DM35424_Dac_Set_Clock_Src(), DM35424_Dac_Set_Conversion_Rate(), DM35424_Dac_Set_Start_Trigger(), DM35424_Dac_Set_Stop_Trigger(), DM35424_Dac_Start(), DM35424_Dac_Volts_To_Conv(), DM35424_DMA_BUFFER_CTRL_LOOP, DM35424_DMA_BUFFER_CTRL, DM35424_Dma_Buffer_Setup(), DM35424_Dma_Clear(), DM35424_Dma_Initialize(), DM35424_Dma_Setup(), DM35424_DMA_SETUP_DIRECTION_WRITE, DM35424_Dma_Start(), DM35424_Dma_Write(), DM35424_Gbc_Board_Reset(), DM35424_General_InstallISR(), DM35424_General_RemoveISR(), DM35424_Generate_Signal_Data(), DM35424_Micro_Sleep(), DM35424_NUM_DAC_ON_BOARD, DM35424_SINE_WAVE, exit_program, HELP_OPTION, IGNORE_USED, INTERRUPT_DISABLE, INTERRUPT_ENABLE, ISR(), MINOR_OPTION, my_adc, program_name, RATE_OPTION, sigint_handler(), and usage().

6.11.3.2 sigint_handler()

```
void sigint_handler (
    int signal_number) [static]
```

Signal handler for SIGINT Control-C keyboard interrupt.

Parameters

<i>signal_number</i>	
----------------------	--

Signal number passed in from the kernel.

Warning

One must be extremely careful about what functions are called from a signal handler.

Definition at line 179 of file [dm35424_adc.c](#).

References [exit_program](#).

6.11.4 Variable Documentation

6.11.4.1 exit_program

```
volatile int exit_program = 0
```

Boolean indicating whether or not to exit the program.

Definition at line 106 of file [dm35424_adc.c](#).

6.11.4.2 interrupt_count

```
volatile int interrupt_count = 0
```

Count of interrupts that have happened.

Definition at line 101 of file [dm35424_adc.c](#).

6.11.4.3 program_name

```
char* program_name [static]
```

Name of the program as invoked on the command line

Definition at line 96 of file dm35424_adc.c.

6.12 dm35424_adc.c

[Go to the documentation of this file.](#)

```
00001
00050
00051 #include <stdio.h>
00052 #include <stddef.h>
00053 #include <stdlib.h>
00054 #include <errno.h>
00055 #include <error.h>
00056 #include <unistd.h>
00057 #include <limits.h>
00058 #include <signal.h>
00059 #include <getopt.h>
00060 #include <signal.h>
00061
00062 #include "dm35424_gbc_library.h"
00063 #include "dm35424_adc_library.h"
00064 #include "dm35424_dac_library.h"
00065 #include "dm35424_dma_library.h"
00066 #include "dm35424_ioctl.h"
00067 #include "dm35424_examples.h"
00068 #include "dm35424_util_library.h"
00069 #include "dm35424_board_access.h"
00070 #include "dm35424_types.h"
00071 #include "dm35424.h"
00072
00076 #define DAC_RATE          25
00077
00081 #define DEFAULT_RATE      300
00082
00086 #define BUFFER_SIZE_SAMPLES    100
00087
00091 #define BUFFER_SIZE_BYTES      (BUFFER_SIZE_SAMPLES * sizeof(int))
00092
00096 static char *program_name;
00097
00101 volatile int interrupt_count = 0;
00102
00106 volatile int exit_program = 0;
00107
00115 static void usage(void)
00116 {
00117     fprintf(stderr, "\n");
00118     fprintf(stderr, "NAME\n\t%s\n", program_name);
00119     fprintf(stderr, "USAGE\n\t%s [OPTIONS]\n", program_name);
00120
00121     fprintf(stderr, "OPTIONS\n\n");
00122     fprintf(stderr, "\t--help\n");
00123     fprintf(stderr, "\t\tShow this help screen and exit.\n");
00124
00125     fprintf(stderr, "\t--minor NUM\n");
00126     fprintf(stderr, "\t\tSpecify the minor number (>= 0) of the board to open.  When not
specified,\n");
00127     fprintf(stderr, "\t\tthe device file with minor 0 is opened.\n");
00128
00129     fprintf(stderr, "\t--rate RATE\n");
00130     fprintf(stderr, "\t\tUse the specified rate (Hz).  The default is %d.\n", DEFAULT_RATE);
00131     fprintf(stderr, "\n");
00132     exit(EXIT_FAILURE);
00133 }
00134
00135
00143
00144 void ISR(struct dm35424_ioctl_interrupt_info_request int_info)
00145 {
00146
00147     if (int_info.error_occurred) {
00148
```

```

00149         printf("ISR: Error received.\n");
00150         return;
00151     }
00152 }
00153
00154 if (int_info.valid_interrupt) {
00155     interrupt_count++;
00156 }
00157
00158 }
00159
00160 }
00161
00162
00163
00164
00165
00166
00167
00168
00169 static void sigint_handler(int signal_number)
00170 {
00171     exit_program = 0xff;
00172 }
00173
00174
00175
00176
00177
00178
00179
00180
00181
00182
00183
00184
00185
00186
00187
00188
00189
00190
00191
00192
00193
00194
00195
00196
00197
00198
00199
00200
00201
00202
00203
00204
00205
00206
00207
00208
00209
00210
00211
00212
00213 int main(int argument_count, char **arguments)
00214 {
00215     struct DM35424_Board_Descriptor *board;
00216     struct DM35424_Function_Block my_adc;
00217     struct DM35424_Function_Block my_dac[DM35424_NUM_DAC_ON_BOARD];
00218
00219     unsigned long int minor = 0;
00220     int result;
00221     int my_value;
00222     uint32_t sample_counts;
00223     int last_int_count = 0;
00224     uint16_t int_status;
00225
00226     unsigned int rate = DEFAULT_RATE;
00227     unsigned int actual_rate;
00228     unsigned int channel = 0;
00229     unsigned int dac_num = 0;
00230     unsigned int index = 0;
00231
00232     int help_option_given = 0;
00233     int status;
00234     struct sigaction signal_action;
00235
00236     uint16_t interrupt_ena = 0;
00237     float volts = 0;
00238
00239     int16_t max_value, min_value, offset;
00240     int32_t *buffer, *offset_buffer;
00241
00242     char *invalid_char_p;
00243     struct option options[] = {
00244         {"help", 0, 0, HELP_OPTION},
00245         {"minor", 1, 0, MINOR_OPTION},
00246         {"rate", 1, 0, RATE_OPTION},
00247         {0, 0, 0, 0}
00248     };
00249
00250     program_name = arguments[0];
00251
00252     // Show usage, parse arguments
00253     while (1) {
00254         /*
00255          * Parse the next command line option and any arguments it may require
00256          */
00257         status = getopt_long(argument_count,
00258                             arguments, "", options, NULL);
00259
00260         /*
00261          * If getopt_long() returned -1, then all options have been processed
00262          */
00263         if (status == -1) {
00264             break;
00265         }
00266
00267         /*
00268          * Figure out what getopt_long() found
00269          */
00270         switch (status) {
00271
00272             /*#####
00273              * User entered '--help'
00274              *##### */
00275             case HELP_OPTION:
00276                 help_option_given = 0xFF;
00277                 break;

```



```

00278
00279
00280     User entered '--minor'
00281     ##### */
00282 case MINOR_OPTION:
00283     /*
00284      * Convert option argument string to unsigned long integer
00285      */
00286     errno = 0;
00287     minor = strtoul(optarg, &invalid_char_p, 10);
00288
00289     /*
00290      * Catch unsigned long int overflow
00291      */
00292     if ((minor == ULONG_MAX)
00293         && (errno == ERANGE)) {
00294         error(0, 0,
00295             "ERROR: Device minor number caused numeric overflow");
00296         usage();
00297     }
00298
00299     /*
00300      * Catch argument strings with valid decimal prefixes, for
00301      * example "1q", and argument strings which cannot be converted,
00302      * for example "abc1"
00303      */
00304     if ((*invalid_char_p != '\0')
00305         || (invalid_char_p == optarg)) {
00306         error(0, 0,
00307             "ERROR: Non-decimal device minor number");
00308         usage();
00309     }
00310
00311     break;
00312
00313     User entered rate
00314     ##### */
00315 case RATE_OPTION:
00316     /*
00317      * Convert option argument string to unsigned long integer
00318      */
00319     errno = 0;
00320     rate = strtoul(optarg, &invalid_char_p, 10);
00321
00322     /*
00323      * Catch unsigned long int overflow
00324      */
00325     if ((rate == ULONG_MAX)
00326         && (errno == ERANGE)) {
00327         error(0, 0,
00328             "ERROR: Rate number caused numeric overflow");
00329         usage();
00330     }
00331
00332     /*
00333      * Catch argument strings with valid decimal prefixes, for
00334      * example "1q", and argument strings which cannot be converted,
00335      * for example "abc1"
00336      */
00337     if ((*invalid_char_p != '\0')
00338         || (invalid_char_p == optarg)) {
00339         error(0, 0,
00340             "ERROR: Non-decimal rate value entered");
00341         usage();
00342     }
00343
00344     break;
00345
00346     User entered unsupported option
00347     ##### */
00348 case '?':
00349     usage();
00350     break;
00351
00352     #####
00353     getopt_long() returned unexpected value
00354     ##### */
00355 default:
00356     error(EXIT_FAILURE,
00357         0,
00358         "ERROR: getopt_long() returned unexpected value %x",
00359         status);
00360     break;
00361 }
00362 }
00363 }
00364

```

```

00365      /*
00366      * Recognize '--help' option before any others
00367      */
00368
00369      if (help_option_given) {
00370          usage();
00371      }
00372
00373      signal_action.sa_handler = sigint_handler;
00374      sigfillset(&(signal_action.sa_mask));
00375      signal_action.sa_flags = 0;
00376
00377      if (sigaction(SIGINT, &signal_action, NULL) < 0) {
00378          error(EXIT_FAILURE, errno, "ERROR: sigaction() FAILED");
00379      }
00380
00381      printf("Opening board....");
00382      result = DM35424_Board_Open(minor, &board);
00383
00384      DM35424_Check_Result(result, "Could not open board");
00385      printf("success.\nResetting board....");
00386      result = DM35424_Gbc_Board_Reset(board);
00387
00388      DM35424_Check_Result(result, "Could not reset board");
00389      printf("success.\n");
00390
00391      /*****
00392      * DIFFERENTIAL MODE SETUP
00393      *****/
00394      printf("success.\nOpening DACs.....\n");
00395
00400      for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD; dac_num++) {
00401          result = DM35424_Dac_Open(board, dac_num, &my_dac[dac_num]);
00402
00403          DM35424_Check_Result(result, "Could not open DAC");
00404
00405          printf("Found DAC%u, with %d DMA channels (%d buffers each)\n",
00406              dac_num, my_dac[dac_num].num_dma_channels,
00407              my_dac[dac_num].num_dma_buffers);
00408
00409          result = DM35424_Dac_Set_Clock_Src(board,
00410              &my_dac[dac_num],
00411              DM35424_CLK_SRC_IMMEDIATE);
00412
00413          DM35424_Check_Result(result, "Error setting DAC clock");
00414
00415          result = DM35424_Dac_Set_Conversion_Rate(board,
00416              &my_dac[dac_num],
00417              DAC_RATE,
00418              &actual_rate);
00419
00420          fprintf(stdout,
00421              "Rate requested: %d Actual Rate Achieved: %d\n",
00422              DAC_RATE, actual_rate);
00423          DM35424_Check_Result(result, "Error setting sample rate");
00424
00425          buffer = (int *)malloc(BUFFER_SIZE_BYTES);
00426          offset_buffer = (int *)malloc(BUFFER_SIZE_BYTES);
00427
00428          DM35424_Check_Result(buffer == NULL
00429              || offset_buffer == NULL,
00430              "Error allocating space for buffer.");
00431
00432          result = DM35424_Dac_Volts_To_Conv(1.25f, &max_value);
00433
00434          DM35424_Check_Result(result,
00435              "Error converting value to conversion counts.");
00436
00437          result = DM35424_Dac_Volts_To_Conv(-1.25f, &min_value);
00438
00439          DM35424_Check_Result(result,
00440              "Error converting value to conversion counts.");
00441
00442          result = DM35424_Dac_Volts_To_Conv(2.5f, &offset);
00443
00444          DM35424_Check_Result(result,
00445              "Error converting value to conversion counts.");
00446
00447          result = DM35424_Generate_Signal_Data(DM35424_SINE_WAVE,
00448              buffer,
00449              BUFFER_SIZE_SAMPLES,
00450              max_value,
00451              min_value, offset,
00452              0x0000FFFF);
00453
00454          DM35424_Check_Result(result,
00455              "Error trying to generate data for the DAC.");
00456

```

```

00457
00458     for (index = 0; index < BUFFER_SIZE_SAMPLES; index++) {
00459         offset_buffer[index] =
00460             buffer[(index +
00461                 (BUFFER_SIZE_SAMPLES / 2)) %
00462                 BUFFER_SIZE_SAMPLES];
00463     }
00464
00465     for (channel = 0; channel < my_dac[dac_num].num_dma_channels;
00466         channel++) {
00467         fprintf(stdout,
00468             "Initializing and configuring DMA Channel %d....",
00469             channel);
00470         result =
00471             DM35424_Dma_Initialize(board, &my_dac[dac_num],
00472                                     channel, 1,
00473                                     BUFFER_SIZE_BYTES);
00474
00475         DM35424_Check_Result(result, "Error initializing DMA");
00476
00477         result = DM35424_Dma_Setup(board,
00478                                   &my_dac[dac_num],
00479                                   channel,
00480                                   DM35424_DMA_SETUP_DIRECTION_WRITE,
00481                                   IGNORE_USED);
00482
00483         DM35424_Check_Result(result, "Error configuring DMA");
00484
00485         fprintf(stdout, "success!\n");
00486
00487         result = DM35424_Dma_Buffer_Setup(board,
00488                                           &my_dac[dac_num],
00489                                           channel,
00490                                           BUFFER_0,
00491                                           DM35424_DMA_BUFFER_CTRL_VALID |
00492                                           DM35424_DMA_BUFFER_CTRL_LOOP);
00493
00494         DM35424_Check_Result(result,
00495                               "Error setting up buffer control.");
00496
00503         if (channel % 2 == 0) {
00504             result = DM35424_Dma_Write(board,
00505                                       &my_dac[dac_num],
00506                                       channel,
00507                                       BUFFER_0,
00508                                       BUFFER_SIZE_BYTES,
00509                                       buffer);
00510
00511             DM35424_Check_Result(result,
00512                                   "Writing to DMA buffer failed");
00513         } else {
00514
00515             result = DM35424_Dma_Write(board,
00516                                       &my_dac[dac_num],
00517                                       channel,
00518                                       BUFFER_0,
00519                                       BUFFER_SIZE_BYTES,
00520                                       offset_buffer);
00521
00522             DM35424_Check_Result(result,
00523                                   "Writing to DMA buffer failed");
00524
00525         }
00526
00527         fprintf(stdout, "Starting DMA Channel %d.....",
00528             channel);
00529         result =
00530             DM35424_Dma_Start(board, &my_dac[dac_num], channel);
00531
00532         DM35424_Check_Result(result, "Error starting DMA");
00533
00534         printf("success.\n");
00535     }
00536
00537     free(buffer);
00538     free(offset_buffer);
00539
00540     fprintf(stdout, "Starting DAC.\n");
00541
00542     result = DM35424_Dac_Set_Start_Trigger(board,
00543                                             &my_dac[dac_num],
00544                                             DM35424_CLK_SRC_IMMEDIATE);
00545
00546     DM35424_Check_Result(result, "Error setting start trigger for DAC.");
00547
00548     result = DM35424_Dac_Set_Stop_Trigger(board,

```

```

00550                                     &my_dac[dac_num],
00551                                     DM35424_CLK_SRC_NEVER);
00552
00553     DM35424_Check_Result(result, "Error setting stop trigger for DAC.");
00554
00555     result = DM35424_Dac_Start(board, &my_dac[dac_num]);
00556
00557     DM35424_Check_Result(result, "Error starting DAC");
00558
00559 }
00560
00561 printf("Opening ADC.....\n");
00562
00563 result = DM35424_Adc_Open(board, ADC_0, &my_adc);
00564
00565 DM35424_Check_Result(result, "Could not open ADC");
00566
00567 printf("Found ADC_0, with %d DMA channels (%d buffers each)\n",
00568        my_adc.num_dma_channels, my_adc.num_dma_buffers);
00569
00570 result = DM35424_Adc_Uninitialize(board, &my_adc);
00571
00572 DM35424_Check_Result(result, "Error setting ADC to Uninitialized.");
00573
00574 result = DM35424_Adc_Interrupt_Set_Config(board,
00575                                           &my_adc,
00576                                           DM35424_ADC_INT_ALL_MASK,
00577                                           INTERRUPT_DISABLE);
00578
00579 DM35424_Check_Result(result, "Error disabling interrupts.");
00580
00581 result = DM35424_Adc_Interrupt_Clear_Status(board,
00582                                           &my_adc,
00583                                           DM35424_ADC_INT_ALL_MASK);
00584
00585 DM35424_Check_Result(result, "Error clearing interrupt status.");
00586
00587 for (channel = 0; channel < my_adc.num_dma_channels; channel++) {
00588
00589     result = DM35424_Adc_Channel_Setup(board,
00590                                         &my_adc,
00591                                         channel,
00592                                         DM35424_ADC_RNG_BIPOLAR_2_5V,
00593                                         DM35424_ADC_INPUT_DIFFERENTIAL);
00594
00595     DM35424_Check_Result(result, "Error setting up channel.");
00596
00597     result = DM35424_Adc_Channel_Set_Filter(board,
00598                                             &my_adc,
00599                                             channel,
00600                                             DM35424_ADC_CHAN_FILTER_ORDER0);
00601
00602     DM35424_Check_Result(result, "Error setting filter order to 0.");
00603
00604 }
00605
00606 result = DM35424_Adc_Ad_Config_Set_Mode(board,
00607                                         &my_adc,
00608                                         DM35424_ADC_MODE_CONFIG_HIGH_SPEED);
00609
00610 DM35424_Check_Result(result, "Error setting AD config.");
00611
00612 result = DM35424_Adc_Set_Start_Trigger(board,
00613                                         &my_adc,
00614                                         DM35424_CLK_SRC_IMMEDIATE);
00615
00616 DM35424_Check_Result(result, "Error setting start trigger.");
00617
00618 result = DM35424_Adc_Set_Stop_Trigger(board,
00619                                       &my_adc,
00620                                       DM35424_CLK_SRC_NEVER);
00621
00622 DM35424_Check_Result(result, "Error setting stop trigger.");
00623
00624 result = DM35424_Adc_Set_Clock_Src(board,
00625                                    &my_adc,
00626                                    DM35424_CLK_SRC_IMMEDIATE);
00627
00628 DM35424_Check_Result(result, "Error setting ADC clock");
00629
00630 result = DM35424_Adc_Set_Sample_Rate(board,
00631                                     &my_adc, rate, &actual_rate);
00632
00633 DM35424_Check_Result(result, "Failed to set sample rate for ADC.");
00634 fprintf(stdout,
00635         "ADC:0; Rate requested: %d Actual Rate Achieved: %d\n",
00636         rate, actual_rate);
00637
00638 result = DM35424_Adc_Set_Pre_Trigger_Samples(board, &my_adc, 0);

```

```

00637
00638     DM35424_Check_Result(result, "Error setting pre-capture samples.");
00639
00640     result = DM35424_Adc_Set_Post_Stop_Samples(board, &my_adc, 0);
00641
00642     DM35424_Check_Result(result, "Error setting post-capture samples.");
00643
00644     result = DM35424_Adc_Initialize(board, &my_adc);
00645
00646     DM35424_Check_Result(result, "Failed or timed out initializing ADC.");
00647
00648     fprintf(stdout, "Installing user ISR ...\n");
00649     result = DM35424_General_InstallISR(board, ISR);
00650     DM35424_Check_Result(result, "DM35424_General_InstallISR()");
00651
00652     result = DM35424_Adc_Interrupt_Set_Config(board,
00653                                               &my_adc,
00654                                               DM35424_ADC_INT_SAMPLE_TAKEN_MASK,
00655                                               INTERRUPT_ENABLE);
00656
00657     DM35424_Check_Result(result, "Error setting interrupt.");
00658
00659     printf("Starting ADC.....\nADC in DIFFERENTIAL mode, using DAC outputs via physical
loopback.\n");
00660
00661     result = DM35424_Adc_Start(board, &my_adc);
00662
00663     DM35424_Check_Result(result, "Error starting ADC");
00664
00665     result = DM35424_Adc_Interrupt_Get_Config(board,
00666                                               &my_adc, &interrupt_ena);
00667
00668     DM35424_Check_Result(result, "Error getting interrupt value");
00669
00670     printf("\nPress Ctrl-C for next mode.\n\n");
00671
00672     printf("Sample Count");
00673     for (channel = 0; channel < my_adc.num_dma_channels; channel++) {
00674
00675         printf("\tVoltage %u.", channel);
00676
00677     }
00678     printf("\n");
00679     printf("=====");
00680     for (channel = 0; channel < my_adc.num_dma_channels; channel++) {
00681
00682         printf("\t=====");
00683
00684     }
00685     printf("\n");
00686
00687     while (!exit_program) {
00688         if (last_int_count < interrupt_count) {
00689
00690             result = DM35424_Adc_Interrupt_Get_Status(board,
00691                                                       &my_adc,
00692                                                       &int_status);
00693             DM35424_Check_Result(result, "Error getting interrupt status");
00694
00695             result = DM35424_Adc_Get_Sample_Count(board,
00696                                                  &my_adc,
00697                                                  &sample_counts);
00698
00699             DM35424_Check_Result(result, "Error getting sample count.");
00700
00701             printf("%12d", sample_counts);
00702
00703             for (channel = 0; channel < my_adc.num_dma_channels;
00704                 channel++) {
00705
00706                 result =
00707                     DM35424_Adc_Channel_Get_Last_Sample(board,
00708                                                         &my_adc,
00709                                                         channel,
00710                                                         &my_value);
00711
00712                 DM35424_Check_Result(result,
00713                                     "Error getting ADC value.");
00714
00715                 result =
00716                     DM35424_Adc_Sample_To_Volts(DM35424_ADC_RNG_BIPOLAR_2_5V,
00717                                                  my_value,
00718                                                  &volts);
00719
00720                 DM35424_Check_Result(result,
00721                                     "Error converting ADC sample to volts.");
00722

```

```

00723             printf("    %+2.5f    ", volts);
00724
00725         }
00726         printf("\r");
00727
00728         result = DM35424_Adc_Interrupt_Clear_Status(board,
00729                                                     &my_adc,
00730                                                     int_status);
00731         DM35424_Check_Result(result, "Error clearing interrupt status");
00732
00733         last_int_count++;
00734
00735     }
00736
00737     DM35424_Micro_Sleep(100);
00738 }
00739
00740 exit_program = 0;
00741 printf("\n\nStopping Adc 0.....");
00742
00743 result = DM35424_Adc_Reset(board, &my_adc);
00744
00745 DM35424_Check_Result(result, "Error starting ADC");
00746
00747 printf("success!\n");
00748
00749 /*****
00750 * SINGLE-ENDED POSITIVE MODE SETUP
00751 *****/
00752 for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD; dac_num++) {
00753     result = DM35424_Dac_Reset(board, &my_dac[dac_num]);
00754
00755     DM35424_Check_Result(result, "Could not reset DAC");
00756
00757     printf("Resetting DAC%d.\n", dac_num);
00758
00759     buffer = (int *)malloc(BUFFER_SIZE_BYTES);
00760
00761     DM35424_Check_Result(buffer == NULL, "Error allocating space for buffer.");
00762
00763     result = DM35424_Dac_Volts_To_Conv(2.5f, &max_value);
00764
00765     DM35424_Check_Result(result,
00766                         "Error converting value to conversion counts.");
00767
00768     result = DM35424_Dac_Volts_To_Conv(-2.5f, &min_value);
00769
00770     DM35424_Check_Result(result,
00771                         "Error converting value to conversion counts.");
00772
00773
00774     result = DM35424_Generate_Signal_Data(DM35424_SINE_WAVE,
00775                                           buffer,
00776                                           BUFFER_SIZE_SAMPLES,
00777                                           max_value,
00778                                           min_value,
00779                                           offset,
00780                                           0x0000FFFF);
00781
00782     DM35424_Check_Result(result,
00783                         "Error trying to generate data for the DAC.");
00784
00785     for (channel = 0; channel < my_dac[dac_num].num_dma_channels;
00786         channel++) {
00787         fprintf(stdout,
00788             "Initializing and configuring DMA Channel %d....",
00789             channel);
00790
00791         result =
00792             DM35424_Dma_Clear(board, &my_dac[dac_num], channel);
00793
00794         DM35424_Check_Result(result, "Error clearing DMA");
00795
00796
00797         result = DM35424_Dma_Write(board,
00798                                     &my_dac[dac_num],
00799                                     channel,
00800                                     BUFFER_0,
00801                                     BUFFER_SIZE_BYTES,
00802                                     buffer);
00803
00804         DM35424_Check_Result(result, "Writing to DMA buffer failed");
00805
00806         fprintf(stdout, "Starting DMA Channel %d.....",
00807             channel);
00808     }
00809 }

```

```

00810         result =
00811             DM35424_Dma_Start(board, &my_dac[dac_num], channel);
00812
00813         DM35424_Check_Result(result, "Error starting DMA");
00814
00815         printf("success.\n");
00816
00817     }
00818
00819     free(buffer);
00820
00821     fprintf(stdout, "Starting DAC.\n");
00822
00823     result = DM35424_Dac_Set_Start_Trigger(board,
00824                                             &my_dac[dac_num],
00825                                             DM35424_CLK_SRC_IMMEDIATE);
00826
00827     DM35424_Check_Result(result, "Error setting start trigger for DAC.");
00828
00829     result = DM35424_Dac_Set_Stop_Trigger(board,
00830                                            &my_dac[dac_num],
00831                                            DM35424_CLK_SRC_NEVER);
00832
00833     DM35424_Check_Result(result, "Error setting stop trigger for DAC.");
00834
00835     result = DM35424_Dac_Start(board, &my_dac[dac_num]);
00836
00837     DM35424_Check_Result(result, "Error starting DAC");
00838
00839 }
00840
00841
00842 for (channel = 0; channel < my_adc.num_dma_channels; channel++) {
00843
00844     result = DM35424_Adc_Channel_Setup(board,
00845                                         &my_adc,
00846                                         channel,
00847                                         DM35424_ADC_RNG_UNIPOLAR_5V,
00848                                         DM35424_ADC_INPUT_SINGLE_ENDED_POS);
00849
00850     DM35424_Check_Result(result, "Error setting up channel.");
00851
00852     if (channel % 2 == 1) {
00853
00854         printf("Setting ADC Channel %d Filter to Order 7.\n", channel);
00855         result = DM35424_Adc_Channel_Set_Filter(board,
00856                                                  &my_adc,
00857                                                  channel,
00858                                                  DM35424_ADC_CHAN_FILTER_ORDER7);
00859
00860         DM35424_Check_Result(result, "Error setting filter order to 7.");
00861     }
00862
00863 }
00864
00865
00866 printf("Starting ADC.....\nADC in SINGLE-ENDED POSITIVE mode, using DAC outputs "
00867        "via physical loopback.\n");
00868
00869 result = DM35424_Adc_Start(board, &my_adc);
00870
00871 DM35424_Check_Result(result, "Error starting ADC");
00872
00873
00874 printf("\nPress Ctrl-C for next mode.\n\n");
00875
00876 printf("Sample Count");
00877 for (channel = 0; channel < my_adc.num_dma_channels; channel++) {
00878
00879     printf("\tVoltage %u.", channel);
00880
00881 }
00882 printf("\n");
00883 printf("=====");
00884 for (channel = 0; channel < my_adc.num_dma_channels; channel++) {
00885
00886     printf("\t=====");
00887
00888 }
00889 printf("\n");
00890
00891 while (!exit_program) {
00892     if (last_int_count < interrupt_count) {
00893
00894         result = DM35424_Adc_Interrupt_Get_Status(board,
00895                                                    &my_adc,
00896                                                    &int_status);

```

```

00897         DM35424_Check_Result(result, "Error getting interrupt status");
00898
00899         result = DM35424_Adc_Get_Sample_Count(board,
00900                                             &my_adc,
00901                                             &sample_counts);
00902
00903         DM35424_Check_Result(result, "Error getting sample count.");
00904
00905         printf("%12d", sample_counts);
00906
00907         for (channel = 0; channel < my_adc.num_dma_channels;
00908             channel++) {
00909
00910             result =
00911                 DM35424_Adc_Channel_Get_Last_Sample(board,
00912                                                     &my_adc,
00913                                                     channel,
00914                                                     &my_value);
00915
00916             DM35424_Check_Result(result,
00917                                   "Error getting ADC value.");
00918
00919             result =
00920                 DM35424_Adc_Sample_To_Volts(DM35424_ADC_RNG_BIPOLAR_2_5V,
00921                                             my_value,
00922                                             &volts);
00923
00924             DM35424_Check_Result(result,
00925                                   "Error converting ADC sample to volts.");
00926
00927             printf("    %+2.5f    ", volts);
00928
00929         }
00930         printf("\r");
00931
00932         result = DM35424_Adc_Interrupt_Clear_Status(board,
00933                                                     &my_adc,
00934                                                     int_status);
00935         DM35424_Check_Result(result, "Error clearing interrupt status");
00936
00937         last_int_count++;
00938
00939     }
00940
00941     DM35424_Micro_Sleep(100);
00942 }
00943
00944 exit_program = 0;
00945 printf("\n\nStopping Adc 0.....");
00946
00947 result = DM35424_Adc_Reset(board, &my_adc);
00948
00949 DM35424_Check_Result(result, "Error starting ADC");
00950
00951 printf("success!\n");
00952
00953 /*****
00954  * SINGLE-ENDED NEGATIVE MODE SETUP
00955  *****/
00956 for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD; dac_num++) {
00957     result = DM35424_Dac_Reset(board, &my_dac[dac_num]);
00958
00959     DM35424_Check_Result(result, "Could not reset DAC");
00960
00961     printf("Resetting DAC%d.\n", dac_num);
00962
00963
00964     for (channel = 0; channel < my_dac[dac_num].num_dma_channels;
00965         channel++) {
00966         fprintf(stdout,
00967             "Initializing and configuring DMA Channel %d....",
00968             channel);
00969
00970         fprintf(stdout, "Starting DMA Channel %d.....",
00971             channel);
00972
00973         result =
00974             DM35424_Dma_Start(board, &my_dac[dac_num], channel);
00975
00976         DM35424_Check_Result(result, "Error starting DMA");
00977
00978         printf("success.\n");
00979
00980     }
00981
00982     fprintf(stdout, "Starting DAC.\n");
00983

```



```

00984         result = DM35424_Dac_Set_Start_Trigger(board,
00985                                                 &my_dac[dac_num],
00986                                                 DM35424_CLK_SRC_IMMEDIATE);
00987
00988         DM35424_Check_Result(result, "Error setting start trigger for DAC.");
00989
00990         result = DM35424_Dac_Set_Stop_Trigger(board,
00991                                                 &my_dac[dac_num],
00992                                                 DM35424_CLK_SRC_NEVER);
00993
00994         DM35424_Check_Result(result, "Error setting stop trigger for DAC.");
00995
00996         result = DM35424_Dac_Start(board, &my_dac[dac_num]);
00997
00998         DM35424_Check_Result(result, "Error starting DAC");
00999
01000     }
01001
01002
01003
01004     for (channel = 0; channel < my_adc.num_dma_channels; channel++) {
01005
01006         result = DM35424_Adc_Channel_Setup(board,
01007                                             &my_adc,
01008                                             channel,
01009                                             DM35424_ADC_RNG_UNIPOLAR_5V,
01010                                             DM35424_ADC_INPUT_SINGLE_ENDED_NEG);
01011
01012         DM35424_Check_Result(result, "Error setting up channel.");
01013
01014     }
01015
01016
01017     printf("Starting ADC.....\nADC in SINGLE-ENDED NEGATIVE mode, using DAC outputs "
01018           "via physical loopback.\n");
01019
01020     result = DM35424_Adc_Start(board, &my_adc);
01021
01022     DM35424_Check_Result(result, "Error starting ADC");
01023
01024
01025     printf("\nPress Ctrl-C for next mode.\n\n");
01026
01027     printf("Sample Count");
01028     for (channel = 0; channel < my_adc.num_dma_channels; channel++) {
01029
01030         printf("\tVoltage %u.", channel);
01031
01032     }
01033     printf("\n");
01034     printf("=====");
01035     for (channel = 0; channel < my_adc.num_dma_channels; channel++) {
01036
01037         printf("\t=====");
01038
01039     }
01040     printf("\n");
01041
01042     while (!exit_program) {
01043         if (last_int_count < interrupt_count) {
01044
01045             result = DM35424_Adc_Interrupt_Get_Status(board,
01046                                                         &my_adc,
01047                                                         &int_status);
01048
01049             DM35424_Check_Result(result, "Error getting interrupt status");
01050
01051             result = DM35424_Adc_Get_Sample_Count(board,
01052                                                    &my_adc,
01053                                                    &sample_counts);
01054
01055             DM35424_Check_Result(result, "Error getting sample count.");
01056
01057             printf("%12d", sample_counts);
01058
01059             for (channel = 0; channel < my_adc.num_dma_channels;
01060                  channel++) {
01061
01062                 result =
01063                     DM35424_Adc_Channel_Get_Last_Sample(board,
01064                                                           &my_adc,
01065                                                           channel,
01066                                                           &my_value);
01067
01068                 DM35424_Check_Result(result,
01069                                       "Error getting ADC value.");
01069
01070                 result =

```

```

01071         DM35424_Adc_Sample_To_Volts(DM35424_ADC_RNG_BIPOLAR_2_5V,
01072                                     my_value,
01073                                     &volts);
01074
01075         DM35424_Check_Result(result,
01076                               "Error converting ADC sample to volts.");
01077
01078         printf("    %+2.5f    ", volts);
01079
01080     }
01081     printf("\r");
01082
01083     result = DM35424_Adc_Interrupt_Clear_Status(board,
01084                                                  &my_adc,
01085                                                  int_status);
01086     DM35424_Check_Result(result, "Error clearing interrupt status");
01087
01088     last_int_count++;
01089
01090 }
01091
01092     DM35424_Micro_Sleep(100);
01093 }
01094
01095 exit_program = 0;
01096 printf("\n\nStopping ADC 0.....");
01097
01098 result = DM35424_Adc_Reset(board, &my_adc);
01099
01100 DM35424_Check_Result(result, "Error starting ADC");
01101
01102 printf("success!\n");
01103
01104 /*****
01105  * DAC LOOPBACK MODE SETUP
01106  *****/
01107 for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD; dac_num++) {
01108     result = DM35424_Dac_Reset(board, &my_dac[dac_num]);
01109
01110     DM35424_Check_Result(result, "Could not reset DAC");
01111
01112     printf("Resetting DAC%d.\n", dac_num);
01113
01114     for (channel = 0; channel < my_dac[dac_num].num_dma_channels;
01115          channel++) {
01116         fprintf(stdout,
01117                 "Initializing and configuring DMA Channel %d....",
01118                 channel);
01119
01120         fprintf(stdout, "Starting DMA Channel %d.....",
01121                 channel);
01122
01123         result =
01124             DM35424_Dma_Start(board, &my_dac[dac_num], channel);
01125
01126         DM35424_Check_Result(result, "Error starting DMA");
01127
01128         printf("success.\n");
01129     }
01130
01131     fprintf(stdout, "Starting DAC.\n");
01132
01133     result = DM35424_Dac_Set_Start_Trigger(board,
01134                                             &my_dac[dac_num],
01135                                             DM35424_CLK_SRC_IMMEDIATE);
01136
01137     DM35424_Check_Result(result, "Error setting start trigger for DAC.");
01138
01139     result = DM35424_Dac_Set_Stop_Trigger(board,
01140                                             &my_dac[dac_num],
01141                                             DM35424_CLK_SRC_NEVER);
01142
01143     DM35424_Check_Result(result, "Error setting stop trigger for DAC.");
01144
01145     result = DM35424_Dac_Start(board, &my_dac[dac_num]);
01146
01147     DM35424_Check_Result(result, "Error starting DAC");
01148
01149     for (channel = 0; channel < my_adc.num_dma_channels; channel++) {
01150         result = DM35424_Adc_Channel_Setup(board,
01151

```

```

01158                                     &my_adc,
01159                                     channel,
01160                                     DM35424_ADC_RNG_UNIPOLAR_5V,
01161                                     DM35424_ADC_INPUT_DAC_LOOPBACK);
01162
01163         DM35424_Check_Result(result, "Error setting up channel.");
01164     }
01165
01166
01167
01168     printf("Starting ADC.....\nADC in DAC INTERNAL LOOPBACK mode. Please remove "
01169           "physical loopbacks.\n");
01170
01171     result = DM35424_Adc_Start(board, &my_adc);
01172
01173     DM35424_Check_Result(result, "Error starting ADC");
01174
01175
01176     printf("\nPress Ctrl-C to exit program.\n\n");
01177
01178     printf("Sample Count");
01179     for (channel = 0; channel < my_adc.num_dma_channels; channel++) {
01180
01181         printf("\tVoltage %u.", channel);
01182
01183     }
01184     printf("\n");
01185     printf("=====");
01186     for (channel = 0; channel < my_adc.num_dma_channels; channel++) {
01187
01188         printf("\t=====");
01189
01190     }
01191     printf("\n");
01192
01193     while (!exit_program) {
01194         if (last_int_count < interrupt_count) {
01195
01196             result = DM35424_Adc_Interrupt_Get_Status(board,
01197                                                         &my_adc,
01198                                                         &int_status);
01199             DM35424_Check_Result(result, "Error getting interrupt status");
01200
01201             result = DM35424_Adc_Get_Sample_Count(board,
01202                                                     &my_adc,
01203                                                     &sample_counts);
01204
01205             DM35424_Check_Result(result, "Error getting sample count.");
01206
01207             printf("%12d", sample_counts);
01208
01209             for (channel = 0; channel < my_adc.num_dma_channels;
01210                 channel++) {
01211
01212                 result =
01213                     DM35424_Adc_Channel_Get_Last_Sample(board,
01214                                                            &my_adc,
01215                                                            channel,
01216                                                            &my_value);
01217
01218                 DM35424_Check_Result(result,
01219                                         "Error getting ADC value.");
01220
01221                 result =
01222                     DM35424_Adc_Sample_To_Volts(DM35424_ADC_RNG_BIPOLAR_2_5V,
01223                                                  my_value,
01224                                                  &volts);
01225
01226                 DM35424_Check_Result(result,
01227                                         "Error converting ADC sample to volts.");
01228
01229                 printf("    %+2.5f    ", volts);
01230
01231             }
01232             printf("\r");
01233
01234             result = DM35424_Adc_Interrupt_Clear_Status(board,
01235                                                           &my_adc,
01236                                                           int_status);
01237             DM35424_Check_Result(result, "Error clearing interrupt status");
01238
01239             last_int_count++;
01240
01241         }
01242
01243         DM35424_Micro_Sleep(100);
01244

```

```

01245     }
01246
01247     printf("\n\nStopping ADC 0.....");
01248
01249     result = DM35424_Adc_Reset(board, &my_adc);
01250
01251     DM35424_Check_Result(result, "Error starting ADC");
01252
01253     printf("success!\n");
01254
01255     printf("Disabling interrupt....");
01256
01257     result = DM35424_Adc_Interrupt_Set_Config(board,
01258                                                &my_adc,
01259                                                DM35424_ADC_INT_SAMPLE_TAKEN_MASK,
01260                                                INTERRUPT_DISABLE);
01261
01262     DM35424_Check_Result(result, "Error removing interrupt.");
01263
01264     printf("success!\nRemoving ISR.....");
01265     result = DM35424_General_RemoveISR(board);
01266
01267     DM35424_Check_Result(result, "Error removing ISR.");
01268
01269     printf("success.\n");
01270
01271     result = DM35424_Gbc_Board_Reset(board);
01272     printf("Closing Board\n");
01273     result = DM35424_Board_Close(board);
01274
01275     DM35424_Check_Result(result, "Error closing board.");
01276     printf("Example program successfully completed.\n");
01277     return 0;
01278
01279 }

```

6.13 examples/dm35424_adc_continuous_dma.c File Reference

Example program which demonstrates the use of the ADC and DMA.

```

#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <fcntl.h>
#include <unistd.h>
#include <signal.h>
#include <limits.h>
#include <getopt.h>
#include "dm35424_gbc_library.h"
#include "dm35424_dac_library.h"
#include "dm35424_adc_library.h"
#include "dm35424_ioctl.h"
#include "dm35424_examples.h"
#include "dm35424_dma_library.h"
#include "dm35424.h"
#include "dm35424_util_library.h"

```

Macros

- `#define DEFAULT_RATE 10000`
- `#define BUFFER_SIZE_SAMPLES 1000`
- `#define BUFFER_SIZE_BYTES (BUFFER_SIZE_SAMPLES * sizeof(int))`
- `#define ASCII_FILE_NAME "./adc_dma.txt"`
- `#define BIN_FILE_NAME "./adc_dma.bin"`

Functions

- static void **usage** (void)
Print information on stderr about how the program is to be used. After doing so, the program is exited.
- static void **sigint_handler** (int signal_number)
Signal handler for SIGINT Control-C keyboard interrupt.
- void **output_channel_status** (struct **DM35424_Board_Descriptor** *handle, const struct **DM35424_Function_Block** *func_block, unsigned int channel)
Output the status of a DMA channel. This is a helper function to determine the cause of an error when it occurs.
- void **ISR** (struct **dm35424_ioctl_interrupt_info_request** int_info)
The interrupt subroutine that will execute when a DMA interrupt occurs. This function will read from the DMA, copying data from the kernel buffers to the user buffers so that we can access the data.
- void **setup_dacs_and_start** (struct **DM35424_Function_Block** *my_dac)
Setup the DACs to produce a sine wave as a signal to sample, then start them.
- void **setup_adc** (uint32_t rate)
Setup the ADCs to sample.
- void **setup_ctrlc_handler** ()
Handler to detect when user hits Ctrl-C.
- void **convert_bin_to_txt** (unsigned int samples_in_buff)
Convert a binary data file to ASCII values. The format will be the same as the data file produced without the -binary argument. The example program will exit after finishing.
- int **main** (int argument_count, char **arguments)
The main program.

Variables

- static char * **program_name**
- static int **dma_has_error** = 0
- static struct **DM35424_Board_Descriptor** * **board**
- static struct **DM35424_Function_Block** **my_adc**
- static unsigned long **buffer_count** [**DM35424_NUM_ADC_DMA_CHANNELS**]
- static int ** **local_buffer** [**DM35424_NUM_ADC_DMA_CHANNELS**]
- static volatile int **exit_program** = 0
- static unsigned long **buffer_size_bytes** = 0
- static unsigned int **next_buffer** [**DM35424_NUM_ADC_DMA_CHANNELS**]

6.13.1 Detailed Description

Example program which demonstrates the use of the ADC and DMA.

This example program will collect data from the ADC(s) specified by the user, at the rate specified by the user, and will write the data to a file. It will do this continuously until the user hits CTRL-C (or the filesystem becomes full).

You can put any signal you want on the ADC input pins. However, for convenience, this example sets up the DACs to provide a signal for the ADC to measure. In order for that to work, you must loopback the DAC outputs to the ADC differential inputs. Connect DAC0 Channel 0 to ADC_0 Channel 0+ and ADC_0 Channel 1-, and connect DAC0 Channel 1 to ADC_0 Channel 0- and ADC_0 Channel 1+, etc.

Maximum sustainable throughput is HIGHLY system dependent. Higher sample rates might be achievable through better buffer size selection or use of an operating system with realtime features.

```
-----
This file and its contents are copyright (C) RTD Embedded Technologies,
Inc. All Rights Reserved.

This software is licensed as described in the RTD End-User Software License
Agreement. For a copy of this agreement, refer to the file LICENSE.TXT
(which should be included with this software) or contact RTD Embedded
Technologies, Inc.
-----
```

Id

[dm35424_adc_continuous_dma.c](#) 141531 2024-03-06 21:05:25Z lfrankenfield

Definition in file [dm35424_adc_continuous_dma.c](#).

6.13.2 Macro Definition Documentation

6.13.2.1 ASCII_FILE_NAME

```
#define ASCII_FILE_NAME "./adc_dma.txt"
```

Name of file when saving as ASCII

Definition at line 82 of file [dm35424_adc_continuous_dma.c](#).

Referenced by [convert_bin_to_txt\(\)](#), [main\(\)](#), and [usage\(\)](#).

6.13.2.2 BIN_FILE_NAME

```
#define BIN_FILE_NAME "./adc_dma.bin"
```

Name of file when saving as binary

Definition at line 87 of file [dm35424_adc_continuous_dma.c](#).

Referenced by [convert_bin_to_txt\(\)](#), [main\(\)](#), and [usage\(\)](#).

6.13.2.3 BUFFER_SIZE_BYTES

```
#define BUFFER_SIZE_BYTES (BUFFER_SIZE_SAMPLES * sizeof(int))
```

Size of DAC DMA buffer, in bytes

Definition at line 77 of file [dm35424_adc_continuous_dma.c](#).

6.13.2.4 BUFFER_SIZE_SAMPLES

```
#define BUFFER_SIZE_SAMPLES 1000
```

Number of samples in the DAC buffer (to form the wave pattern)

Definition at line 72 of file [dm35424_adc_continuous_dma.c](#).

6.13.2.5 DEFAULT_RATE

```
#define DEFAULT_RATE 10000
```

Default rate to use, if user does not enter one. (Hz)

Definition at line 67 of file [dm35424_adc_continuous_dma.c](#).

6.13.3 Function Documentation

6.13.3.1 convert_bin_to_txt()

```
void convert_bin_to_txt (
    unsigned int samples_in_buff)
```

Convert a binary data file to ASCII values. The format will be the same as the data file produced without the `-binary` argument. The example program will exit after finishing.

Return values

<i>None.</i>	
--------------	--

Definition at line 716 of file [dm35424_adc_continuous_dma.c](#).

References [ASCII_FILE_NAME](#), [BIN_FILE_NAME](#), and [DM35424_NUM_ADC_DMA_CHANNELS](#).

Referenced by [main\(\)](#).

6.13.3.2 ISR()

```
void ISR (
    struct dm35424_ioctl_interrupt_info_request int_info)
```

The interrupt subroutine that will execute when a DMA interrupt occurs. This function will read from the DMA, copying data from the kernel buffers to the user buffers so that we can access the data.

Parameters

<i>int_info</i>	
-----------------	--

A structure containing information about the interrupt.

Return values

<i>None.</i>	
--------------	--

Definition at line 279 of file [dm35424_adc_continuous_dma.c](#).

References [board](#), [buffer_count](#), [buffer_size_bytes](#), [CHANNEL_0](#), [CLEAR_INTERRUPT](#), [DM35424_Check_Result\(\)](#), [DM35424_Dma_Clear_Interrupt\(\)](#), [DM35424_Dma_Find_Interrupt\(\)](#), [DM35424_Dma_Read\(\)](#), [DM35424_Dma_Reset_Buffer\(\)](#), [DM35424_Gbc_Ack_Interrupt\(\)](#), [DM35424_NUM_ADC_DMA_BUFFERS](#), [DM35424_NUM_ADC_DMA_CHANNELS](#), [dma_has_error](#), [exit_program](#), [dm35424_ioctl_interrupt_info_request::interrupt_fb](#), [local_buffer](#), [my_adc](#), [next_buffer](#), [NO_CLEAR_INTERRUPT](#), and [dm35424_ioctl_interrupt_info_request::valid_interrupt](#).

6.13.3.3 main()

```
int main (
    int argument_count,
    char ** arguments)
```

The main program.

Parameters

<i>argument_count</i>	
-----------------------	--

Number of args passed on the command line, including the executable name

Parameters

<i>arguments</i>	
------------------	--

Pointer to array of character strings, which are the args themselves.

Return values

<i>0</i>	
----------	--

Success

Return values

<i>Non-Zero</i>	
-----------------	--

Failure. First, setup the DACS. They will produce a sine wave that needs to be looped back to the ADC inputs. This will cause the ADC to see what looks like a max-value sine wave.

Check to see if any channel has not yet been copied from DMA.

Definition at line 818 of file [dm35424_adc_continuous_dma.c](#).

References [ASCII_FILE_NAME](#), [BIN2TXT_OPTION](#), [BIN_FILE_NAME](#), [BINARY_OPTION](#), [board](#), [buffer_count](#), [buffer_size_bytes](#), [convert_bin_to_txt\(\)](#), [DEFAULT_RATE](#), [DM35424_Adc_Start\(\)](#), [DM35424_Board_Close\(\)](#), [DM35424_Board_Open\(\)](#), [DM35424_Check_Result\(\)](#), [DM35424_Dma_Configure_Interrupts\(\)](#), [DM35424_Dma_Start\(\)](#), [DM35424_Gbc_Board_Reset\(\)](#), [DM35424_General_InstallISR\(\)](#), [DM35424_General_RemoveISR\(\)](#), [DM35424_Micro_Sleep\(\)](#), [DM35424_NUM_ADC_DMA_CHANNELS](#), [DM35424_NUM_DAC_ON_BOARD](#), [dma_has_error](#), [ERROR_INTR_DISABLE](#), [exit_program](#), [HELP_OPTION](#), [INTERRUPT_DISABLE](#), [ISR\(\)](#), [local_buffer](#), [MINOR_OPTION](#), [my_adc](#), [output_channel_status\(\)](#), [program_name](#), [RATE_OPTION](#), [SAMPLES_OPTION](#), [setup_adc\(\)](#), [setup_ctrlc_handler\(\)](#), [setup_dacs_and_start\(\)](#), and [usage\(\)](#).

6.13.3.4 output_channel_status()

```
void output_channel_status (  
    struct DM35424_Board_Descriptor * handle,  
    const struct DM35424_Function_Block * func_block,  
    unsigned int channel)
```

Output the status of a DMA channel. This is a helper function to determine the cause of an error when it occurs.

Parameters

<i>handle</i>	
---------------	--

Pointer to the board handle.

Parameters

<i>func_block</i>	
-------------------	--

Pointer to the function block containing the DMA channel

Parameters

<i>channel</i>	
----------------	--

The DMA channel we want the status of.

Return values

<i>None</i>	
-------------	--

Definition at line 227 of file [dm35424_adc_continuous_dma.c](#).

References [DM35424_Check_Result\(\)](#), [DM35424_Dma_Status\(\)](#), and [DM35424_Function_Block::fb_num](#).

Referenced by [main\(\)](#).

6.13.3.5 setup_adc()

```
void setup_adc (  
    uint32_t rate)
```

Setup the ADCs to sample.

Parameters

<i>rate</i>	
-------------	--

The sampling rate to set the ADC to.

Return values

<i>None.</i>	
--------------	--

Definition at line 544 of file [dm35424_adc_continuous_dma.c](#).

References [ADC_0](#), [board](#), [buffer_size_bytes](#), [CHANNEL_0](#), [DM35424_Adc_Ad_Config_Set_Mode\(\)](#), [DM35424_Adc_Channel_Setup\(\)](#), [DM35424_Adc_Initialize\(\)](#), [DM35424_ADC_INPUT_DIFFERENTIAL](#), [DM35424_ADC_MODE_CONFIG_HIGH_SPEED](#), [DM35424_Adc_Open\(\)](#), [DM35424_ADC_RNG_BIPOLAR_2_5V](#), [DM35424_Adc_Set_Clock_Src\(\)](#), [DM35424_Adc_Set_Sample_Rate\(\)](#), [DM35424_Adc_Set_Start_Trigger\(\)](#), [DM35424_Adc_Set_Stop_Trigger\(\)](#), [DM35424_Check_Result\(\)](#), [DM35424_CLK_SRC_IMMEDIATE](#), [DM35424_CLK_SRC_NEVER](#), [DM35424_DMA_BUFFER_CTRL_INTR](#), [DM35424_DMA_BUFFER_CTRL_LOOP](#), [DM35424_DMA_BUFFER_CTRL_VALID](#), [DM35424_Dma_Buffer_Setup\(\)](#), [DM35424_Dma_Buffer_Status\(\)](#), [DM35424_Dma_Configure_Interrupts\(\)](#), [DM35424_Dma_Initialize\(\)](#), [DM35424_Dma_Setup\(\)](#), [DM35424_DMA_SETUP_DIRECTION](#), [DM35424_NUM_ADC_DMA_BUFFERS](#), [ERROR_INTR_ENABLE](#), [INTERRUPT_ENABLE](#), [my_adc](#), [next_buffer](#), and [NOT_IGNORE_USED](#).

Referenced by [main\(\)](#).

6.13.3.6 setup_ctrlc_handler()

```
void setup_ctrlc_handler ()
```

Handler to detect when user hits Ctrl-C.

Return values

<i>None.</i>	
--------------	--

Definition at line 689 of file [dm35424_adc_continuous_dma.c](#).

References [sigint_handler\(\)](#).

Referenced by [main\(\)](#).

6.13.3.7 setup_dacs_and_start()

```
void setup_dacs_and_start (
    struct DM35424\_Function\_Block * my_dac)
```

Setup the DACs to produce a sine wave as a signal to sample, then start them.

Parameters

<i>my_dac</i>	
---------------	--

Pointer to the DAC function block structure.

Return values

<i>None.</i>	
--------------	--

We load the even channels with the wave pattern, and the odd channels with the same pattern, but offset by half its length. Doing this gives us an opposing pattern between the even and odd channels, which helps when using DAC for ADC input.

Definition at line 375 of file [dm35424_adc_continuous_dma.c](#).

References [board](#), [BUFFER_0](#), [BUFFER_SIZE_BYTES](#), [BUFFER_SIZE_SAMPLES](#), [DM35424_Check_Result\(\)](#), [DM35424_CLK_SRC_IMMEDIATE](#), [DM35424_CLK_SRC_NEVER](#), [DM35424_Dac_Open\(\)](#), [DM35424_Dac_Set_Clock_Src\(\)](#), [DM35424_Dac_Set_Conversion_Rate\(\)](#), [DM35424_Dac_Set_Start_Trigger\(\)](#), [DM35424_Dac_Set_Stop_Trigger\(\)](#), [DM35424_Dac_Start\(\)](#), [DM35424_Dac_Volts_To_Conv\(\)](#), [DM35424_DMA_BUFFER_CTRL_LOOP](#), [DM35424_DMA_BUFFER_CTRL_STOP](#), [DM35424_Dma_Buffer_Setup\(\)](#), [DM35424_Dma_Initialize\(\)](#), [DM35424_Dma_Setup\(\)](#), [DM35424_DMA_SETUP_DIRECTION_WRITE](#), [DM35424_Dma_Start\(\)](#), [DM35424_Dma_Write\(\)](#), [DM35424_Generate_Signal_Data\(\)](#), [DM35424_NUM_DAC_ON_BOARD](#), [DM35424_SINE_WAVE](#), [IGNORE_USED](#), [DM35424_Function_Block::num_dma_buffers](#), and [DM35424_Function_Block::num_dma_buffers](#).

Referenced by [main\(\)](#).

6.13.3.8 sigint_handler()

```
void sigint_handler (
    int signal_number) [static]
```

Signal handler for SIGINT Control-C keyboard interrupt.

Parameters

<i>signal_number</i>	
----------------------	--

Signal number passed in from the kernel.

Warning

One must be extremely careful about what functions are called from a signal handler.

Definition at line 196 of file [dm35424_adc_continuous_dma.c](#).

References [exit_program](#).

6.13.4 Variable Documentation**6.13.4.1 board**

```
struct DM35424_Board_Descriptor* board [static]
```

Pointer to board descriptor

Definition at line 103 of file [dm35424_adc_continuous_dma.c](#).

6.13.4.2 buffer_count

```
unsigned long buffer_count[DM35424_NUM_ADC_DMA_CHANNELS] [static]
```

Array of buffer counts, used to track progress of each ADC as data is copied.

Definition at line 114 of file [dm35424_adc_continuous_dma.c](#).

6.13.4.3 buffer_size_bytes

```
unsigned long buffer_size_bytes = 0 [static]
```

Size of the buffer allocated, in bytes.

Definition at line 130 of file [dm35424_adc_continuous_dma.c](#).

6.13.4.4 dma_has_error

```
int dma_has_error = 0 [static]
```

Boolean flag indicating if there was a DMA error.

Definition at line 98 of file [dm35424_adc_continuous_dma.c](#).

6.13.4.5 exit_program

```
volatile int exit_program = 0 [static]
```

Boolean indicating the program should exit.

Definition at line 125 of file [dm35424_adc_continuous_dma.c](#).

6.13.4.6 local_buffer

```
int** local_buffer[DM35424_NUM_ADC_DMA_CHANNELS] [static]
```

Pointer to local memory buffer where data is copied from the kernel buffers when a DMA buffer becomes full.

Definition at line 120 of file [dm35424_adc_continuous_dma.c](#).

6.13.4.7 my_adc

```
struct DM35424_Function_Block my_adc [static]
```

Pointer to array of function blocks that will hold the ADC descriptors

Definition at line 108 of file [dm35424_adc_continuous_dma.c](#).

6.13.4.8 next_buffer

```
unsigned int next_buffer[DM35424_NUM_ADC_DMA_CHANNELS] [static]
```

Which buffer is next to be copied from DMA

Definition at line 135 of file [dm35424_adc_continuous_dma.c](#).

Referenced by [ISR\(\)](#), and [setup_adc\(\)](#).

6.13.4.9 program_name

```
char* program_name [static]
```

Name of the program as invoked on the command line

Definition at line 93 of file [dm35424_adc_continuous_dma.c](#).

6.14 dm35424_adc_continuous_dma.c

[Go to the documentation of this file.](#)

```
00001
00043
00044 #include <stdio.h>
00045 #include <stddef.h>
00046 #include <stdlib.h>
00047 #include <errno.h>
00048 #include <error.h>
00049 #include <fcntl.h>
00050 #include <unistd.h>
00051 #include <signal.h>
00052 #include <limits.h>
00053 #include <getopt.h>
00054
00055 #include "dm35424_gbc_library.h"
00056 #include "dm35424_dac_library.h"
00057 #include "dm35424_adc_library.h"
00058 #include "dm35424_ioctl.h"
00059 #include "dm35424_examples.h"
00060 #include "dm35424_dma_library.h"
00061 #include "dm35424.h"
00062 #include "dm35424_util_library.h"
00063
00067 #define DEFAULT_RATE          10000
00068
00072 #define BUFFER_SIZE_SAMPLES    1000
00073
00077 #define BUFFER_SIZE_BYTES      (BUFFER_SIZE_SAMPLES * sizeof(int))
00078
00082 #define ASCII_FILE_NAME " ./adc_dma.txt"
00083
00087 #define BIN_FILE_NAME " ./adc_dma.bin"
00088
00092
00093 static char *program_name;
00094
00098 static int dma_has_error = 0;
00099
00103 static struct DM35424_Board_Descriptor *board;
00104
00108 static struct DM35424_Function_Block my_adc;
00109
00114 static unsigned long buffer_count[DM35424_NUM_ADC_DMA_CHANNELS];
00115
00120 static int **local_buffer[DM35424_NUM_ADC_DMA_CHANNELS];
00121
00125 static volatile int exit_program = 0;
00126
00130 static unsigned long buffer_size_bytes = 0;
00131
```

```

00135 static unsigned int next_buffer[DM35424_NUM_ADC_DMA_CHANNELS];
00136
00144
00145 static void usage(void)
00146 {
00147     fprintf(stderr, "\n");
00148     fprintf(stderr, "NAME\n\n\t%s\n\n", program_name);
00149     fprintf(stderr, "USAGE\n\n\t%s [OPTIONS]\n\n", program_name);
00150
00151     fprintf(stderr, "OPTIONS\n\n");
00152     fprintf(stderr, "\t--help\n");
00153     fprintf(stderr, "\t\tShow this help screen and exit.\n");
00154
00155     fprintf(stderr, "\t--minor NUM\n");
00156     fprintf(stderr, "\t\tSpecify the minor number (>= 0) of the board to open. When not
specified,\n");
00157     fprintf(stderr, "\t\tthe device file with minor 0 is opened.\n");
00158
00159     fprintf(stderr, "\t--rate RATE\n");
00160     fprintf(stderr, "\t\tUse the specified rate (Hz). The default is %d.\n", DEFAULT_RATE);
00161
00162     fprintf(stderr, "\t--samples NUM\n");
00163     fprintf(stderr, "\t\tThe number of samples to collect before stopping. Note that the\n");
00164     fprintf(stderr, "\t\tactual number collected may be higher due to buffer size.\n");
00165
00166     fprintf(stderr, "\t--binary\n");
00167     fprintf(stderr, "\t\tWrite data to file in binary format, instead of default ASCII.\n");
00168     fprintf(stderr, "\t\tData is stored as [ADC_0 Buff0][ADC1 Buff0]...[ADC_0 Buff1][ADC1
Buff1],etc\n");
00169
00170     fprintf(stderr, "\t--bin2txt\n");
00171     fprintf(stderr, "\t\tThe program will convert the %s file to\n", BIN_FILE_NAME);
00172     fprintf(stderr, "\t\t%s and exit.\n", ASCII_FILE_NAME);
00173     fprintf(stderr, "\t\tNote: Because the rate affects the buffer size, you must\n");
00174     fprintf(stderr, "\t\tinclude the --rate argument as well, IF it was used to\n");
00175     fprintf(stderr, "\t\tcreate the binary file in the first place.\n");
00176
00177     fprintf(stderr, "\n");
00178     exit(EXIT_FAILURE);
00179 }
00180
00196 static void sigint_handler(int signal_number)
00197 {
00198     exit_program = 0xff;
00199 }
00200
00201
00227 void output_channel_status(struct DM35424_Board_Descriptor *handle,
00228                          const struct DM35424_Function_Block *func_block,
00229                          unsigned int channel)
00230 {
00231     int result;
00232     unsigned int current_buffer;
00233     uint32_t current_count;
00234     int current_action;
00235     int status_overflow;
00236     int status_underflow;
00237     int status_used;
00238     int status_invalid;
00239     int status_complete;
00240
00241     result = DM35424_Dma_Status(handle,
00242                                func_block,
00243                                channel,
00244                                &current_buffer,
00245                                &current_count,
00246                                &current_action,
00247                                &status_overflow,
00248                                &status_underflow,
00249                                &status_used,
00250                                &status_invalid, &status_complete);
00251
00252     DM35424_Check_Result(result, "Error getting DMA status");
00253
00254     printf
00255     ("FB%d Ch%d DMA Status: Current Buffer: %u Count: %ul Action: 0x%x Status: "
00256      "Ov: %d Un: %d Used: %d Inv: %d Comp: %d\n",
00257      func_block->fb_num, channel, current_buffer, current_count,
00258      current_action, status_overflow, status_underflow, status_used,
00259      status_invalid, status_complete);
00260 }
00261
00262
00279 void ISR(struct dm35424_ioctl_interrupt_info_request int_info)
00280 {
00281
00282     int result = 0;

```

```

00283     unsigned int channel = CHANNEL_0;
00284     int channel_complete = 0, channel_error = 0;
00285     uint32_t function_block;
00286
00287     function_block = int_info.interrupt_fb & 0x7FFFFFFF;
00288
00289     if (int_info.valid_interrupt) {
00290
00291         DM35424_Check_Result(my_adc.fb_num != function_block, "Interrupt from unexpected
function block.");
00292
00293         // It's a DMA interrupt
00294         if (int_info.interrupt_fb < 0) {
00295
00296             result = DM35424_Dma_Find_Interrupt(board,
00297                                                 &my_adc,
00298                                                 &channel,
00299                                                 &channel_complete,
00300                                                 &channel_error);
00301
00302             DM35424_Check_Result(result, "Error checking for DMA error.");
00303
00304             if (channel_error) {
00305                 dma_has_error = 1;
00306                 exit_program = 1;
00307                 return;
00308             }
00309
00310             for (channel = CHANNEL_0; channel < DM35424_NUM_ADC_DMA_CHANNELS; channel++)
00311 {
00312                 result = DM35424_Dma_Read(board,
00313                                           &my_adc,
00314                                           channel,
00315                                           next_buffer[channel],
00316                                           buffer_size_bytes,
00317                                           local_buffer[channel]
00318                                           [next_buffer[channel]]);
00319
00320                 buffer_count[channel]++;
00321                 DM35424_Check_Result(result,
00322                                     "Error getting DMA buffer");
00323
00324                 result = DM35424_Dma_Reset_Buffer(board,
00325                                                  &my_adc,
00326                                                  channel,
00327                                                  next_buffer[channel]);
00328
00329                 DM35424_Check_Result(result, "Error resetting buffer");
00330
00331                 next_buffer[channel] = (next_buffer[channel] + 1) %
DM35424_NUM_ADC_DMA_BUFFERS;
00332
00333             }
00334
00335             result = DM35424_Dma_Clear_Interrupt(board,
00336                                                  &my_adc,
00337                                                  CHANNEL_0,
00338                                                  NO_CLEAR_INTERRUPT,
00339                                                  NO_CLEAR_INTERRUPT,
00340                                                  NO_CLEAR_INTERRUPT,
00341                                                  NO_CLEAR_INTERRUPT,
00342                                                  CLEAR_INTERRUPT);
00343
00344             }
00345         }
00346         else {
00347             exit_program = 1;
00348             printf("Error: Non-DMA interrupt received.");
00349             return;
00350         }
00351
00352         result = DM35424_Gbc_Ack_Interrupt(board);
00353
00354         DM35424_Check_Result(result, "Error calling ACK interrupt.");
00355     }
00356 }
00357 }
00358 }
00359
00360
00375 void setup_dacs_and_start(struct DM35424_Function_Block *my_dac)
00376 {
00377     unsigned int dac_num, channel = 0, index;
00378     int result;
00379     int16_t max_value, min_value, offset;
00380     int32_t *buffer, *offset_buffer;

```

```

00381     uint32_t actual_rate = 0;
00382
00383     printf("Setting up DACs...\n");
00384
00385     buffer = (int *)malloc(BUFFER_SIZE_BYTES);
00386     offset_buffer = (int *)malloc(BUFFER_SIZE_BYTES);
00387
00388     DM35424_Check_Result(buffer == NULL || offset_buffer == NULL, "Error allocating space for
buffer.");
00389
00390     for (dac_num = 0; dac_num < DM35424_NUM_DAC_ON_BOARD; dac_num++) {
00391         result = DM35424_Dac_Open(board, dac_num, &my_dac[dac_num]);
00392
00393         DM35424_Check_Result(result, "Could not open DAC");
00394
00395         printf("    Found DAC%u, with %d DMA channels (%d buffers each)\n",
00396             dac_num, my_dac[dac_num].num_dma_channels, my_dac[dac_num].num_dma_buffers);
00397
00398         result = DM35424_Dac_Set_Clock_Src(board,
00399             &my_dac[dac_num],
00400             DM35424_CLK_SRC_IMMEDIATE);
00401
00402         DM35424_Check_Result(result, "Error setting DAC clock");
00403
00404         result = DM35424_Dac_Set_Conversion_Rate(board,
00405             &my_dac[dac_num], BUFFER_SIZE_SAMPLES,
00406             &actual_rate);
00407         fprintf(stdout, "    Rate requested: %d Actual Rate Achieved: %d\n",
00408             BUFFER_SIZE_SAMPLES,
00409             actual_rate);
00410         DM35424_Check_Result(result, "Error setting sample rate");
00411
00412         result = DM35424_Dac_Volts_To_Conv(1.25f,
00413             &max_value);
00414         DM35424_Check_Result(result, "Error converting value to conversion counts.");
00415
00416         result = DM35424_Dac_Volts_To_Conv(-1.25f,
00417             &min_value);
00418         DM35424_Check_Result(result, "Error converting value to conversion counts.");
00419
00420         result = DM35424_Dac_Volts_To_Conv(2.5f,
00421             &offset);
00422         DM35424_Check_Result(result, "Error converting value to conversion counts.");
00423
00424         result = DM35424_Generate_Signal_Data(DM35424_SINE_WAVE,
00425             buffer,
00426             BUFFER_SIZE_SAMPLES,
00427             max_value,
00428             min_value,
00429             offset,
00430             0x0000FFFF);
00431         DM35424_Check_Result(result, "Error trying to generate data for the DAC.");
00432
00433         for (index = 0; index < BUFFER_SIZE_SAMPLES; index++) {
00434             offset_buffer[index] = buffer[(index + (BUFFER_SIZE_SAMPLES / 2)) %
00435                 BUFFER_SIZE_SAMPLES];
00436         }
00437
00438         for (channel = 0; channel < my_dac[dac_num].num_dma_channels; channel++) {
00439             fprintf(stdout, "    Initializing and configuring DMA Channel %d....",
00440                 channel);
00441             result =
00442                 DM35424_Dma_Initialize(board, &my_dac[dac_num], channel,
00443                     1, BUFFER_SIZE_BYTES);
00444             DM35424_Check_Result(result, "Error initializing DMA");
00445
00446             result = DM35424_Dma_Setup(board,
00447                 &my_dac[dac_num],
00448                 channel,
00449                 DM35424_DMA_SETUP_DIRECTION_WRITE,
00450                 IGNORE_USED);
00451             DM35424_Check_Result(result, "Error configuring DMA");
00452
00453             fprintf(stdout, "success!\n");
00454
00455             result = DM35424_Dma_Buffer_Setup(board,
00456                 &my_dac[dac_num],
00457                 channel,
00458                 BUFFER_0,

```



```

00464                                     DM35424_DMA_BUFFER_CTRL_VALID |
00465                                     DM35424_DMA_BUFFER_CTRL_LOOP);
00466
00467     DM35424_Check_Result(result, "Error setting up buffer control.");
00468
00475     if (channel % 2 == 0) {
00476         result = DM35424_Dma_Write(board,
00477                                     &my_dac[dac_num],
00478                                     channel,
00479                                     BUFFER_0, BUFFER_SIZE_BYTES, buffer);
00480
00481         DM35424_Check_Result(result, "    Writing to DMA buffer failed");
00482     }
00483     else {
00484
00485         result = DM35424_Dma_Write(board,
00486                                     &my_dac[dac_num],
00487                                     channel,
00488                                     BUFFER_0, BUFFER_SIZE_BYTES,
offset_buffer);
00489
00490         DM35424_Check_Result(result, "    Writing to DMA buffer failed");
00491
00492     }
00493
00494     fprintf(stdout, "    Starting DMA Channel %d.....", channel);
00495     result = DM35424_Dma_Start(board, &my_dac[dac_num], channel);
00496
00497     DM35424_Check_Result(result, "Error starting DMA");
00498
00499     printf("success.\n");
00500
00501 }
00502
00503 fprintf(stdout, "Starting DAC.\n");
00504
00505 result = DM35424_Dac_Set_Start_Trigger(board,
00506                                         &my_dac[dac_num],
00507                                         DM35424_CLK_SRC_IMMEDIATE);
00508
00509 DM35424_Check_Result(result, "Error setting start trigger for DAC.");
00510
00511 result = DM35424_Dac_Set_Stop_Trigger(board,
00512                                         &my_dac[dac_num],
00513                                         DM35424_CLK_SRC_NEVER);
00514
00515 DM35424_Check_Result(result, "Error setting stop trigger for DAC.");
00516
00517 result = DM35424_Dac_Start(board, &my_dac[dac_num]);
00518
00519 DM35424_Check_Result(result, "Error starting DAC");
00520
00521 }
00522
00523 free(buffer);
00524 free(offset_buffer);
00525 }
00526
00527 void setup_adc(uint32_t rate)
00528 {
00529     unsigned int channel = 0, buff;
00530     uint8_t buff_status, buff_control;
00531     uint32_t buff_size, actual_rate;
00532     int result;
00533
00534     result = DM35424_Adc_Open(board, ADC_0, &my_adc);
00535
00536     DM35424_Check_Result(result, "Could not open ADC");
00537
00538     printf("Found ADC_0, with %d DMA channels (%d buffers each)\n",
00539           my_adc.num_dma_channels,
00540           my_adc.num_dma_buffers);
00541
00542     result = DM35424_Adc_Set_Clock_Src(board,
00543                                         &my_adc,
00544                                         DM35424_CLK_SRC_IMMEDIATE);
00545
00546     DM35424_Check_Result(result, "Error setting ADC clock");
00547
00548     for (channel = 0; channel < my_adc.num_dma_channels; channel++) {
00549         fprintf(stdout, "Initializing DMA Channel 0....");
00550         result = DM35424_Dma_Initialize(board,

```

```

00570                                     &my_adc,
00571                                     channel,
00572                                     my_adc.num_dma_buffers,
00573                                     buffer_size_bytes);
00574
00575     DM35424_Check_Result(result, "Error initializing DMA");
00576
00577     result = DM35424_Dma_Setup(board,
00578                                &my_adc,
00579                                channel,
00580                                DM35424_DMA_SETUP_DIRECTION_READ,
00581                                NOT_IGNORE_USED);
00582
00583     DM35424_Check_Result(result, "Error configuring DMA");
00584
00585     fprintf(stdout, "Setting DMA Interrupts.....");
00586     result = DM35424_Dma_Configure_Interrupts(board,
00587                                                &my_adc,
00588                                                channel,
00589                                                INTERRUPT_ENABLE,
00590                                                ERROR_INTR_ENABLE);
00591
00592     DM35424_Check_Result(result, "Error setting DMA Interrupts");
00593     fprintf(stdout, "success!\n");
00594
00595     for (buff = 0; buff < my_adc.num_dma_buffers; buff++) {
00596
00597         buff_control = DM35424_DMA_BUFFER_CTRL_VALID;
00598
00599         if (buff == (DM35424_NUM_ADC_DMA_BUFFERS - 1)) {
00600             buff_control |= DM35424_DMA_BUFFER_CTRL_LOOP;
00601         }
00602
00603         // All channels will complete at the same time, so we only
00604         // need 1 interrupt to tell us.
00605         if (channel == CHANNEL_0) {
00606             buff_control |= DM35424_DMA_BUFFER_CTRL_INTR;
00607         }
00608
00609     }
00610
00611     result = DM35424_Dma_Buffer_Setup(board,
00612                                       &my_adc,
00613                                       channel,
00614                                       buff,
00615                                       buff_control);
00616
00617     DM35424_Check_Result(result, "Error setting buffer control.");
00618
00619     result = DM35424_Dma_Buffer_Status(board,
00620                                       &my_adc,
00621                                       channel,
00622                                       buff,
00623                                       &buff_status,
00624                                       &buff_control,
00625                                       &buff_size);
00626
00627     DM35424_Check_Result(result, "Error getting buffer status.");
00628
00629     fprintf(stdout,
00630            "    Buffer %d: Stat: 0x%x  Ctrl: 0x%x  Size: %d\n",
00631            buff, buff_status, buff_control, buff_size);
00632 }
00633
00634 result = DM35424_Adc_Channel_Setup(board,
00635                                    &my_adc,
00636                                    channel,
00637                                    DM35424_ADC_RNG_BIPOLAR_2_5V,
00638                                    DM35424_ADC_INPUT_DIFFERENTIAL);
00639
00640     DM35424_Check_Result(result, "Error setting up channel.");
00641
00642     next_buffer[channel] = 0;
00643
00644 }
00645
00646 result = DM35424_Adc_Ad_Config_Set_Mode(board,
00647                                          &my_adc,
00648                                          DM35424_ADC_MODE_CONFIG_HIGH_SPEED);
00649
00650     DM35424_Check_Result(result, "Error setting AD config.");
00651
00652     printf("success.\nInitializing ADC.....");
00653     result = DM35424_Adc_Set_Start_Trigger(board,
00654                                             &my_adc,
00655                                             DM35424_CLK_SRC_IMMEDIATE);
00656     DM35424_Check_Result(result, "Error setting start trigger.");

```

```

00657
00658     result = DM35424_Adc_Set_Stop_Trigger(board,
00659                                           &my_adc,
00660                                           DM35424_CLK_SRC_NEVER);
00661     DM35424_Check_Result(result, "Error setting stop trigger.");
00662
00663
00664     result = DM35424_Adc_Set_Sample_Rate(board,
00665                                           &my_adc,
00666                                           rate, &actual_rate);
00667
00668     DM35424_Check_Result(result, "Failed to set sample rate for ADC.");
00669     fprintf(stdout,
00670             "success.\nADC:0 Rate requested: %d Actual Rate Achieved: %d\n",
00671             rate, actual_rate);
00672
00673     result = DM35424_Adc_Initialize(board, &my_adc);
00674
00675     DM35424_Check_Result(result, "Failed or timed out initializing ADC.");
00676
00677 }
00678
00679
00689 void setup_ctrlc_handler()
00690 {
00691     struct sigaction signal_action;
00692
00693     signal_action.sa_handler = sigint_handler;
00694     sigfillset(&(signal_action.sa_mask));
00695     signal_action.sa_flags = 0;
00696
00697     if (sigaction(SIGINT, &signal_action, NULL) < 0) {
00698         error(EXIT_FAILURE, errno, "ERROR: sigaction() FAILED");
00699     }
00700 }
00701
00702
00703
00716 void convert_bin_to_txt(unsigned int samples_in_buff)
00717 {
00718
00719     FILE *fp_in, *fp_out;
00720     unsigned long sample_num, output_index = 0;
00721     int num_read = 0;
00722     unsigned int channel;
00723
00724     int **buff;
00725
00726     buff = (int **) malloc(DM35424_NUM_ADC_DMA_CHANNELS * sizeof(int *));
00727     if (buff == NULL) {
00728         error(EXIT_FAILURE, errno,
00729             "Error allocating memory for all channels.\n");
00730     }
00731
00732     for (channel = 0; channel < DM35424_NUM_ADC_DMA_CHANNELS; channel++) {
00733         buff[channel] = (int *) malloc(samples_in_buff * sizeof(int));
00734
00735         if (buff[channel] == NULL) {
00736             error(EXIT_FAILURE, errno,
00737                 "Error allocating memory per channel.\n");
00738         }
00739     }
00740 }
00741
00742
00743     fp_in = fopen(BIN_FILE_NAME, "rb");
00744
00745     if (fp_in == NULL) {
00746         error(EXIT_FAILURE, errno,
00747             "open() FAILED to open binary input file %s.\n", BIN_FILE_NAME);
00748     }
00749
00750     fp_out = fopen(ASCII_FILE_NAME, "w");
00751
00752     if (fp_out == NULL) {
00753         error(EXIT_FAILURE, errno,
00754             "open() FAILED to open ASCII output file %s.\n", ASCII_FILE_NAME);
00755     }
00756
00757     for (channel = 0; channel < DM35424_NUM_ADC_DMA_CHANNELS; channel++) {
00758         num_read = fread(buff[channel], sizeof(int), samples_in_buff, fp_in);
00759     }
00760
00761
00762
00763     while (num_read > 0) {
00764

```

```

00765         for (sample_num = 0; sample_num < samples_in_buff; sample_num++) {
00766             fprintf(fp_out, "%lu", output_index);
00767             for (channel = 0; channel < DM35424_NUM_ADC_DMA_CHANNELS; channel++) {
00768                 fprintf(fp_out, "\t%d", buff[channel][sample_num]);
00769             }
00770             fprintf(fp_out, "\n");
00771             output_index++;
00772         }
00773     }
00774     for (channel = 0; channel < DM35424_NUM_ADC_DMA_CHANNELS; channel++) {
00775         num_read = fread(buff[channel], sizeof(int), samples_in_buff, fp_in);
00776     }
00777 }
00778
00779
00780     for (channel = 0; channel < DM35424_NUM_ADC_DMA_CHANNELS; channel++) {
00781         free(buff[channel]);
00782     }
00783
00784     free(buff);
00785     fclose(fp_in);
00786     fclose(fp_out);
00787 }
00788 }
00789
00790
00818 int main(int argument_count, char **arguments)
00819 {
00820     unsigned long int minor = 0;
00821     int result;
00822     int index;
00823     FILE *fp;
00824
00825     unsigned int buffer_to_get;
00826     unsigned int buffers_copied;
00827     unsigned long local_buffer_count[DM35424_NUM_ADC_DMA_CHANNELS];
00828     unsigned long output_index = 0;
00829     unsigned long samples_to_collect = -1;
00830     unsigned long bytes_written = 0;
00831
00832     unsigned int channel = 0, buff;
00833
00834     uint32_t rate = DEFAULT_RATE;
00835     struct DM35424_Function_Block my_dac[DM35424_NUM_DAC_ON_BOARD];
00836
00837     int store_in_binary = 0, samples_in_buffer;
00838     int help_option_given = 0, convert_bin_file = 0;
00839     int status;
00840     char *invalid_char_p;
00841     struct option options[] = {
00842         {"help", 0, 0, HELP_OPTION},
00843         {"minor", 1, 0, MINOR_OPTION},
00844         {"rate", 1, 0, RATE_OPTION},
00845         {"samples", 1, 0, SAMPLES_OPTION},
00846         {"binary", 0, 0, BINARY_OPTION},
00847         {"bin2txt", 0, 0, BIN2TXT_OPTION},
00848         {0, 0, 0, 0}
00849     };
00850
00851     int all_channels_copied = 0;
00852
00853     program_name = arguments[0];
00854
00855     // Show usage, parse arguments
00856     while (1) {
00857         /*
00858          * Parse the next command line option and any arguments it may require
00859          */
00860         status = getopt_long(argument_count,
00861                             arguments, "", options, NULL);
00862
00863         /*
00864          * If getopt_long() returned -1, then all options have been processed
00865          */
00866         if (status == -1) {
00867             break;
00868         }
00869
00870         /*
00871          * Figure out what getopt_long() found
00872          */
00873         switch (status) {
00874
00875             /*#####
00876              * User entered '--help'
00877              *##### */
00878             case HELP_OPTION:

```

```

00879         help_option_given = 0xFF;
00880         break;
00881
00882     /*#####
00883      * User entered '--minor'
00884      *##### */
00885     case MINOR_OPTION:
00886         /*
00887          * Convert option argument string to unsigned long integer
00888          */
00889         errno = 0;
00890         minor = strtoul(optarg, &invalid_char_p, 10);
00891
00892         /*
00893          * Catch unsigned long int overflow
00894          */
00895         if ((minor == ULONG_MAX)
00896             && (errno == ERANGE)) {
00897             error(0, 0,
00898                  "ERROR: Device minor number caused numeric overflow");
00899             usage();
00900         }
00901
00902         /*
00903          * Catch argument strings with valid decimal prefixes, for
00904          * example "lq", and argument strings which cannot be converted,
00905          * for example "abc1"
00906          */
00907         if ((*invalid_char_p != '\0')
00908             || (invalid_char_p == optarg)) {
00909             error(0, 0,
00910                  "ERROR: Non-decimal device minor number");
00911             usage();
00912         }
00913
00914         break;
00915
00916     /*#####
00917      * User entered rate
00918      *##### */
00919     case RATE_OPTION:
00920         /*
00921          * Convert option argument string to unsigned long integer
00922          */
00923         errno = 0;
00924         rate = strtoul(optarg, &invalid_char_p, 10);
00925
00926         /*
00927          * Catch unsigned long int overflow
00928          */
00929         if ((rate == ULONG_MAX)
00930             && (errno == ERANGE)) {
00931             error(0, 0,
00932                  "ERROR: Rate number caused numeric overflow");
00933             usage();
00934         }
00935
00936         /*
00937          * Catch argument strings with valid decimal prefixes, for
00938          * example "lq", and argument strings which cannot be converted,
00939          * for example "abc1"
00940          */
00941         if ((*invalid_char_p != '\0')
00942             || (invalid_char_p == optarg)) {
00943             error(0, 0,
00944                  "ERROR: Non-decimal rate value entered");
00945             usage();
00946         }
00947         break;
00948
00949     /*#####
00950      * User entered number of samples
00951      *##### */
00952     case SAMPLES_OPTION:
00953         /*
00954          * Convert option argument string to unsigned long integer
00955          */
00956         errno = 0;
00957         samples_to_collect =
00958             strtoul(optarg, &invalid_char_p, 10);
00959
00960         /*
00961          * Catch unsigned long int overflow
00962          */
00963         if ((samples_to_collect == ULONG_MAX)
00964             && (errno == ERANGE)) {

```

```

00966             error(0, 0,
00967                    "ERROR: Samples number caused numeric overflow");
00968             usage();
00969         }
00970
00971         /*
00972          * Catch argument strings with valid decimal prefixes, for
00973          * example "1q", and argument strings which cannot be converted,
00974          * for example "abc1"
00975          */
00976         if ((*invalid_char_p != '\0')
00977             || (invalid_char_p == optarg)) {
00978             error(0, 0,
00979                    "ERROR: Non-decimal samples value entered");
00980             usage();
00981         }
00982
00983         break;
00984
00985         /*#####
00986          User entered '--binary'
00987          ##### */
00988         case BINARY_OPTION:
00989             /*
00990              * '--binary' option has been seen
00991              */
00992             store_in_binary = 0xFF;
00993             break;
00994
00995         /*#####
00996          User entered '--bin2txt'
00997          ##### */
00998         case BIN2TXT_OPTION:
00999             convert_bin_file = 1;
01000             break;
01001
01002         /*#####
01003          User entered unsupported option
01004          ##### */
01005         case '?':
01006             usage();
01007             break;
01008
01009         /*#####
01010          getopt_long() returned unexpected value
01011          ##### */
01012         default:
01013             error(EXIT_FAILURE,
01014                    0,
01015                    "ERROR: getopt_long() returned unexpected value %#x",
01016                    status);
01017             break;
01018     }
01019 }
01020
01021 /*
01022  * Recognize '--help' option before any others
01023  */
01024
01025 if (help_option_given) {
01026     usage();
01027 }
01028
01029 if (rate < 1 || rate > 105468) {
01030     error(0, 0, "Error: Rate given not within range of board.");
01031     usage();
01032 }
01033
01034 /*
01035  * Trying to come up with a reasonable size for buffers that doesn't
01036  * take forever to fill up with slower rates, but also makes it possible
01037  * to run at higher rates.
01038  */
01039 buffer_size_bytes = rate;
01040 if (buffer_size_bytes < (20 * sizeof(int))) {
01041     buffer_size_bytes = 20 * sizeof(int);
01042 }
01043
01044 buffer_size_bytes &= ~0x3;
01045 samples_in_buffer = buffer_size_bytes / sizeof(int);
01046
01047 if (convert_bin_file) {
01048     convert_bin_to_txt(samples_in_buffer);
01049     return 0;
01050 }
01051
01052 setup_ctrlc_handler();

```

```

01053
01054     if (store_in_binary) {
01055         fp = fopen(BIN_FILE_NAME, "wb");
01056     } else {
01057         fp = fopen(ASCII_FILE_NAME, "w");
01058     }
01059
01060     if (fp == NULL) {
01061         error(EXIT_FAILURE, errno,
01062             "open() FAILED on output data file.\n");
01063     }
01064
01065     printf("Opening board.....");
01066     result = DM35424_Board_Open(minor, &board);
01067
01068     DM35424_Check_Result(result, "Could not open board");
01069     printf("success.\nResetting board.....");
01070     result = DM35424_Gbc_Board_Reset(board);
01071
01072     DM35424_Check_Result(result, "Could not reset board");
01073     printf("success.\n");
01074
01080     printf("success.\n");
01081
01082     setup_dacs_and_start(my_dac);
01083
01084     setup_adc(rate);
01085
01086     // Allocate local memory for data.
01087     for (channel = 0; channel < my_adc.num_dma_channels; channel++) {
01088         buffer_count[channel] = 0;
01089         local_buffer_count[channel] = 0;
01090
01091         local_buffer[channel] =
01092             malloc(sizeof(int *) * my_adc.num_dma_buffers);
01093         DM35424_Check_Result(local_buffer[channel] == NULL,
01094             "Could not allocate for local buffer");
01095
01096         for (buff = 0; buff < my_adc.num_dma_buffers; buff++) {
01097
01098             local_buffer[channel][buff] = malloc(buffer_size_bytes);
01099
01100             DM35424_Check_Result(local_buffer[channel][buff] == NULL,
01101                 "Could not allocate for local buffer");
01102
01103         }
01104     }
01105
01106 }
01107
01108 fprintf(stdout, "Installing user ISR .....");
01109 result = DM35424_General_InstallISR(board, ISR);
01110 DM35424_Check_Result(result, "DM35424_General_InstallISR()");
01111
01112
01113 for (channel = 0; channel < my_adc.num_dma_channels; channel++) {
01114
01115     fprintf(stdout, "Starting ADC_0 DMA %d .....", channel);
01116     result = DM35424_Dma_Start(board, &my_adc, channel);
01117
01118     DM35424_Check_Result(result, "Error starting DMA");
01119
01120     printf("success.\n");
01121
01122 }
01123
01124 printf("Starting ADC 0\n");
01125
01126 result = DM35424_Adc_Start(board, &my_adc);
01127
01128 DM35424_Check_Result(result, "Error starting ADC");
01129
01130 buffers_copied = 0;
01131
01132 /*
01133  * Loop here until an error occurs, or the user hits CTRL-C. Loop through the ADCs
01134  * and see if a buffer has been copied from kernel space. If so, then write it out
01135  * to disk.
01136  */
01137 while (!exit_program && output_index < samples_to_collect) {
01138
01139     all_channels_copied = 1;
01140     for (channel = 0; channel < my_adc.num_dma_channels; channel++) {
01141         if ((buffer_count[channel] -
01142             local_buffer_count[channel]) >
01143             my_adc.num_dma_buffers) {
01144             fprintf(stdout,

```

```

01145         "Local buffer for ADC Chan %d was overrun.\n",
01146         channel);
01147         exit_program = 1;
01148     }
01149
01152     if (local_buffer_count[channel] == buffer_count[channel]) {
01153         all_channels_copied = 0;
01154     }
01155 }
01156
01157 if (exit_program) {
01158     // Exit out of the while loop
01159     break;
01160 }
01161
01162 if (all_channels_copied) {
01163     buffer_to_get = local_buffer_count[0] % my_adc.num_dma_buffers;
01164
01165     if (store_in_binary) {
01166         for (channel = 0; channel < my_adc.num_dma_channels; channel++) {
01167             fwrite(local_buffer[channel][buffer_to_get],
01168                 sizeof(int),
01169                 (buffer_size_bytes / sizeof(int)),
01170                 fp);
01171
01172             bytes_written += buffer_size_bytes;
01173         }
01174         output_index += samples_in_buffer;
01175     }
01176
01177     else {
01178         for (index = 0;
01179             index < (buffer_size_bytes / sizeof(int));
01180             index++) {
01181             fprintf(fp, "%lu", output_index);
01182             for (channel = 0; channel < my_adc.num_dma_channels; channel
01183 ++)) {
01184
01185                 fprintf(fp, "\t%d",
01186                     local_buffer[channel]
01187                     [buffer_to_get][index]);
01188
01189             }
01190             fprintf(fp, "\n");
01191             output_index++;
01192         }
01193     }
01194
01195     buffers_copied += my_adc.num_dma_channels;
01196
01197     if (buffers_copied % (10 * my_adc.num_dma_channels) == 0) {
01198         fprintf(stdout, "Copied %d buffers.\n",
01199             buffers_copied);
01200     }
01201
01202     for (channel = 0; channel < my_adc.num_dma_channels; channel++) {
01203         local_buffer_count[channel]++;
01204     }
01205
01206 }
01207
01208
01209 }
01210
01211 DM35424_Micro_Sleep(100);
01212
01213 }
01214
01215
01216
01217 if (dma_has_error) {
01218     printf("\n** DMA Error **\n");
01219 }
01220
01221 for (channel = 0; channel < my_adc.num_dma_channels; channel++) {
01222
01223     if (dma_has_error) {
01224         output_channel_status(board,
01225             &my_adc, channel);
01226     }
01227
01228     result = DM35424_Dma_Configure_Interrupts(board,
01229         &my_adc,
01230         channel,
01231         INTERRUPT_DISABLE,
01232         ERROR_INTR_DISABLE);

```



```

01233
01234         DM35424_Check_Result(result, "Error setting DMA Interrupts");
01235
01236         for (buff = 0; buff < my_adc.num_dma_buffers; buff++) {
01237
01238             free(local_buffer[channel][buff]);
01239
01240         }
01241
01242     }
01243     fprintf(stdout, "\n\n");
01244
01245     fprintf(stdout, "Copied %u buffers.\n", buffers_copied);
01246
01247     if (output_index >= samples_to_collect) {
01248         fprintf(stdout, "Reached number of samples (%lu)\n", samples_to_collect);
01249     }
01250
01251     if (store_in_binary) {
01252         fprintf(stdout, "Wrote %lu bytes to file.\n", bytes_written);
01253     }
01254
01255     fclose(fp);
01256
01257     printf("Removing ISR\n");
01258     result = DM35424_General_RemoveISR(board);
01259
01260     DM35424_Check_Result(result, "Error removing ISR.");
01261
01262     result = DM35424_Gbc_Board_Reset(board);
01263     printf("Closing Board\n");
01264     result = DM35424_Board_Close(board);
01265
01266     DM35424_Check_Result(result, "Error closing board.");
01267
01268     printf("Example program successfully completed.\n");
01269     return 0;
01270
01271 }

```

6.15 examples/dm35424_dac.c File Reference

Example program which demonstrates the use of the DAC.

```

#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <unistd.h>
#include <limits.h>
#include <getopt.h>
#include <termios.h>
#include <time.h>
#include <sys/time.h>
#include "dm35424_gbc_library.h"
#include "dm35424_dac_library.h"
#include "dm35424_ioctl.h"
#include "dm35424_examples.h"
#include "dm35424.h"
#include "dm35424_util_library.h"

```

Macros

- #define `DEFAULT_DAC_NUM` 0

Functions

- static void **usage** (void)
Print information on stderr about how the program is to be used. After doing so, the program is exited.
- int **main** (int argument_count, char **arguments)
The main program.

Variables

- static char * **program_name**

6.15.1 Detailed Description

Example program which demonstrates the use of the DAC.

This example program sends data to the DAC for instant conversion. To see the output data, connect an oscilloscope to the DACx Channel 0 through Channel 3 pins, or appropriate DACx pin if you change the DAC number.

The user can control what value goes out the DAC by using keys to increase or decrease the desired voltage, up to 5 V and down to -5 V. Follow the on-screen instructions for adjusting the voltage.

Press 'q' to quit the program.

This file and its contents are copyright (C) RTD Embedded Technologies, Inc. All Rights Reserved.

This software is licensed as described in the RTD End-User Software License Agreement. For a copy of this agreement, refer to the file LICENSE.TXT (which should be included with this software) or contact RTD Embedded Technologies, Inc.

Id

[dm35424_dac.c](#) 141531 2024-03-06 21:05:25Z lfrankenfield

Definition in file [dm35424_dac.c](#).

6.15.2 Macro Definition Documentation

6.15.2.1 DEFAULT_DAC_NUM

```
#define DEFAULT_DAC_NUM 0
```

DAC to use, if user does not choose one.

Definition at line 59 of file [dm35424_dac.c](#).

Referenced by [main\(\)](#), and [usage\(\)](#).

6.15.3 Function Documentation

6.15.3.1 main()

```
int main (
    int argument_count,
    char ** arguments)
```

The main program.

Parameters

<i>argument_count</i>	
-----------------------	--

Number of args passed on the command line, including the executable name

Parameters

<i>arguments</i>	
------------------	--

Pointer to array of character strings, which are the args themselves.

Return values

0	
---	--

Success

Return values

<i>Non-Zero</i>	
-----------------	--

Failure. The DAC cannot achieve +5.0 volts, but for purposes of the example, we allow them to select 5 as a value. However, we'll have to request the actual max value from the library function.

Definition at line 120 of file [dm35424_dac.c](#).

References [board](#), [DAC_NUM_OPTION](#), [DEFAULT_DAC_NUM](#), [DM35424_Board_Close\(\)](#), [DM35424_Board_Open\(\)](#), [DM35424_Check_Result\(\)](#), [DM35424_Dac_Conv_To_Volts\(\)](#), [DM35424_Dac_Get_Last_Conversion\(\)](#), [DM35424_Dac_Open\(\)](#), [DM35424_Dac_Reset\(\)](#), [DM35424_Dac_Set_Last_Conversion\(\)](#), [DM35424_Dac_Volts_To_Conv\(\)](#), [DM35424_Gbc_Board_Reset\(\)](#), [DM35424_Get_Time_Diff\(\)](#), [HELP_OPTION](#), [MINOR_OPTION](#), [program_name](#), and [usage\(\)](#).

6.15.4 Variable Documentation

6.15.4.1 program_name

```
char* program_name [static]
```

Name of the program as invoked on the command line

Definition at line 64 of file [dm35424_dac.c](#).

6.16 dm35424_dac.c

[Go to the documentation of this file.](#)

```

00001
00036
00037 #include <stdio.h>
00038 #include <stddef.h>
00039 #include <stdlib.h>
00040 #include <errno.h>
00041 #include <error.h>
00042 #include <unistd.h>
00043 #include <limits.h>
00044 #include <getopt.h>
00045 #include <termios.h>
00046 #include <time.h>
00047 #include <sys/time.h>
00048
00049 #include "dm35424_gbc_library.h"
00050 #include "dm35424_dac_library.h"
00051 #include "dm35424_ioctl.h"
00052 #include "dm35424_examples.h"
00053 #include "dm35424.h"
00054 #include "dm35424_util_library.h"
00055
00059 #define DEFAULT_DAC_NUM          0
00060
00064 static char *program_name;
00065
00073
00074 static void usage(void)
00075 {
00076     fprintf(stderr, "\n");
00077     fprintf(stderr, "NAME\n\n\t%s\n\n", program_name);
00078     fprintf(stderr, "USAGE\n\n\t%s [OPTIONS]\n\n", program_name);
00079
00080     fprintf(stderr, "OPTIONS\n\n");
00081     fprintf(stderr, "\t--help\n");
00082     fprintf(stderr, "\t\tShow this help screen and exit.\n");
00083     fprintf(stderr, "\t--minor NUM\n");
00084     fprintf(stderr, "\t\tSpecify the minor number (>= 0) of the board to open.  When not
specified,\n");
00085     fprintf(stderr, "\t\tthe device file with minor 0 is opened.\n");
00086     fprintf(stderr, "\t--dac NUM\n");
00087     fprintf(stderr, "\t\tUse the specified DAC.  The default is %d.\n", DEFAULT_DAC_NUM);
00088     fprintf(stderr, "\n");
00089     exit(EXIT_FAILURE);
00090 }
00091
00119
00120 int main(int argument_count, char **arguments)
00121 {
00122
00123     struct DM35424_Board_Descriptor *board;
00124     struct DM35424_Function_Block my_dac;
00125     unsigned long int minor = 0;
00126     int result;
00127
00128     unsigned int dac_num = DEFAULT_DAC_NUM;
00129     unsigned int channel = 0, channel_to_change = 0;
00130
00131     int help_option_given = 0;
00132     struct termios old_tio, new_tio;
00133
00134     int status;
00135     char *invalid_char_p;
00136     struct option options[] = {
00137         {"help", 0, 0, HELP_OPTION},
00138         {"minor", 1, 0, MINOR_OPTION},
00139         {"dac", 1, 0, DAC_NUM_OPTION},
00140         {0, 0, 0, 0}
00141     };
00142
00143     float voltage = 0.0;
00144     int16_t conv_value = 0;
00145
00146     int16_t register_value;
00147     int increment = 1;
00148     unsigned char keypress = '0';
00149
00150     uint8_t marker;
00151
00152     struct timeval start_pressing;
00153     struct timeval next_press;
00154     struct timeval temp_clock;
00155     float time_difference = 0.0;

```

```

00156
00157     program_name = arguments[0];
00158
00159     // Show usage, parse arguments
00160     while (1) {
00161         /*
00162          * Parse the next command line option and any arguments it may require
00163          */
00164         status = getopt_long(argument_count,
00165                             arguments, "", options, NULL);
00166
00167         /*
00168          * If getopt_long() returned -1, then all options have been processed
00169          */
00170         if (status == -1) {
00171             break;
00172         }
00173
00174         /*
00175          * Figure out what getopt_long() found
00176          */
00177         switch (status) {
00178
00179             /*#####
00180              * User entered '--help'
00181              *##### */
00182             case HELP_OPTION:
00183                 help_option_given = 0xFF;
00184                 break;
00185
00186             /*#####
00187              * User entered '--minor'
00188              *##### */
00189             case MINOR_OPTION:
00190                 /*
00191                  * Convert option argument string to unsigned long integer
00192                  */
00193                 errno = 0;
00194                 minor = strtoul(optarg, &invalid_char_p, 10);
00195
00196                 /*
00197                  * Catch unsigned long int overflow
00198                  */
00199                 if ((minor == ULONG_MAX)
00200                     && (errno == ERANGE)) {
00201                     error(0, 0,
00202                          "ERROR: Device minor number caused numeric overflow");
00203                     usage();
00204                 }
00205
00206                 /*
00207                  * Catch argument strings with valid decimal prefixes, for
00208                  * example "1q", and argument strings which cannot be converted,
00209                  * for example "abc1"
00210                  */
00211                 if ((*invalid_char_p != '\0')
00212                     || (invalid_char_p == optarg)) {
00213                     error(0, 0,
00214                          "ERROR: Non-decimal device minor number");
00215                     usage();
00216                 }
00217
00218                 break;
00219
00220             /*#####
00221              * User entered '--dac'
00222              *##### */
00223             case DAC_NUM_OPTION:
00224                 /*
00225                  * Convert option argument string to unsigned long integer
00226                  */
00227                 errno = 0;
00228                 dac_num = strtoul(optarg, &invalid_char_p, 10);
00229
00230                 /*
00231                  * Catch unsigned long int overflow
00232                  */
00233                 if ((dac_num == ULONG_MAX)
00234                     && (errno == ERANGE)) {
00235                     error(0, 0,
00236                          "ERROR: DAC number caused numeric overflow");
00237                     usage();
00238                 }
00239
00240                 /*
00241                  * Catch argument strings with valid decimal prefixes, for
00242                  * example "1q", and argument strings which cannot be converted,

```

```

00243         * for example "abc1"
00244     */
00245     if ((*invalid_char_p != '\0')
00246         || (invalid_char_p == optarg)) {
00247         error(0, 0, "ERROR: Non-decimal DAC number");
00248         usage();
00249     }
00250
00251     break;
00252
00253     /*#####
00254     User entered unsupported option
00255     ##### */
00256     case '?':
00257         usage();
00258         break;
00259
00260     /*#####
00261     getopt_long() returned unexpected value
00262     ##### */
00263     default:
00264         error(EXIT_FAILURE,
00265             0,
00266             "ERROR: getopt_long() returned unexpected value %x",
00267             status);
00268         break;
00269     }
00270 }
00271
00272 /*
00273  * Recognize '--help' option before any others
00274  */
00275 if (help_option_given) {
00276     usage();
00277 }
00278
00279 printf("Opening board....");
00280 result = DM35424_Board_Open(minor, &board);
00281
00282 DM35424_Check_Result(result, "Could not open board");
00283 printf("success.\nResetting board....");
00284 result = DM35424_Gbc_Board_Reset(board);
00285
00286 DM35424_Check_Result(result, "Could not reset board");
00287 printf("success.\nOpening DAC.....");
00288
00289 result = DM35424_Dac_Open(board, dac_num, &my_dac);
00290
00291 DM35424_Check_Result(result, "Could not open DAC");
00292
00293 printf("Found DAC%d, with %d DMA channels (%d buffers each)\n",
00294     dac_num, my_dac.num_dma_channels, my_dac.num_dma_buffers);
00295
00296 result = DM35424_Dac_Reset(board, &my_dac);
00297
00298 DM35424_Check_Result(result, "Error stopping DAC");
00299
00300 tcgetattr(STDIN_FILENO, &old_tio);
00301
00302 new_tio = old_tio;
00303
00304 new_tio.c_lflag &= ~ICANON;
00305 new_tio.c_lflag &= ~ECHO;
00306
00307 tcsetattr(STDIN_FILENO, TCSANOW, &new_tio);
00308
00309 result = DM35424_Dac_Volts_To_Conv(voltage, &conv_value);
00310
00311 DM35424_Check_Result(result, "Error converting voltage to conversion");
00312
00313 printf
00314     ("\n\nPress 'c' or TAB to cycle through the channels. The active channel will have
brackets around it.\n");
00315 printf("Press 'i' to increase the voltage, and 'd' to decrease it.\n");
00316 printf("Hold down the key to change the voltage more rapidly.\n");
00317 printf("Press '1' for 1.0 V, '2' for 2.0 V, etc.\n");
00318 printf
00319     ("For negative numbers, hold down the Shift key ('Shift-1', 'Shift-2', etc)\n");
00320 printf
00321     ("Press 'q' to quit.\n\n===== Voltages =====\n");
00322 for (channel = 0; channel < my_dac.num_dma_channels; channel++) {
00323     printf("Chan %d ", channel);
00324 }
00325 printf("\n");
00326 gettimeofday(&start_pressing, NULL);
00327
00328 channel_to_change = 0;

```

```

00329
00330     for (channel = 0; channel < my_dac.num_dma_channels; channel++) {
00331
00332         result = DM35424_Dac_Get_Last_Conversion(board,
00333                                                    &my_dac,
00334                                                    channel,
00335                                                    &marker,
00336                                                    &register_value);
00337
00338         if (result != 0) {
00339             printf
00340                 ("ERROR getting last conversion.  Errno = %d.\n",
00341                  errno);
00342             keypress = 'q';
00343         }
00344
00345         result = DM35424_Dac_Conv_To_Volts((int32_t) register_value,
00346                                             &voltage);
00347
00348         if (result != 0) {
00349             printf
00350                 ("ERROR converting conversion to voltage.  Errno = %d.\n",
00351                  errno);
00352             keypress = 'q';
00353         }
00354         if (channel == channel_to_change) {
00355             printf(" [%3.2f] ", voltage);
00356         } else {
00357             printf("  %3.2f ", voltage);
00358         }
00359     }
00360 }
00361
00362 printf("\r");
00363
00364
00365
00366 while (keypress != 'q') {
00367     gettimeofday(&next_press, NULL);
00368     keypress = getchar();
00369
00370     gettimeofday(&temp_clock, NULL);
00371     time_difference = DM35424_Get_Time_Diff(temp_clock, next_press);
00372     if (time_difference < 38000.0f) {
00373         /* They're holding down the key.  The longer it is held down,
00374          * the larger the increment in the value to output the DAC,
00375          * thus the faster the change in voltage.
00376          */
00377         next_press = temp_clock;
00378         increment = (((int)
00379                     DM35424_Get_Time_Diff(next_press,
00380                                             start_pressing)) +
00381                     1) / 50000;
00382     } else {
00383         start_pressing = temp_clock;
00384         next_press = temp_clock;
00385         increment = 1;
00386     }
00387 }
00388
00389 if (keypress == 'c' || keypress == '\t') {
00390     channel_to_change =
00391         (channel_to_change + 1) % my_dac.num_dma_channels;
00392     result =
00393         DM35424_Dac_Get_Last_Conversion(board, &my_dac,
00394                                          channel_to_change,
00395                                          &marker,
00396                                          &conv_value);
00397
00398     if (result != 0) {
00399         printf
00400             ("ERROR getting last conversion.  Errno = %d.\n",
00401              errno);
00402         keypress = 'q';
00403     }
00404 }
00405
00406 if (keypress == 'i') {
00407     if (conv_value <=
00408         0x7FFF - increment) {
00409         conv_value += increment;
00410     }
00411 }
00412
00413 if (keypress == 'd') {
00414
00415     if (conv_value >=

```

```
00416             -32768 + increment) {
00417                 conv_value -= increment;
00418             }
00419         }
00420     }
00421     if (keypress == '0') {
00422         result = DM35424_Dac_Volts_To_Conv(0, &conv_value);
00423         DM35424_Check_Result(result,
00424             "Error converting voltage to conversion");
00425     }
00426 }
00427
00428 if (keypress == '1') {
00429     result = DM35424_Dac_Volts_To_Conv(1, &conv_value);
00430     DM35424_Check_Result(result,
00431         "Error converting voltage to conversion");
00432 }
00433
00434 if (keypress == '2') {
00435     result = DM35424_Dac_Volts_To_Conv(2, &conv_value);
00436     DM35424_Check_Result(result,
00437         "Error converting voltage to conversion");
00438 }
00439
00440 if (keypress == '3') {
00441     result = DM35424_Dac_Volts_To_Conv(3, &conv_value);
00442     DM35424_Check_Result(result,
00443         "Error converting voltage to conversion");
00444 }
00445
00446 if (keypress == '4') {
00447     result = DM35424_Dac_Volts_To_Conv(4, &conv_value);
00448     DM35424_Check_Result(result,
00449         "Error converting voltage to conversion");
00450 }
00451
00452 if (keypress == '5') {
00453     result = DM35424_Dac_Volts_To_Conv(4.999847412f,
00454         &conv_value);
00455     DM35424_Check_Result(result,
00456         "Error converting voltage to conversion");
00457 }
00458
00459 if (keypress == '!') {
00460     result = DM35424_Dac_Volts_To_Conv(-1, &conv_value);
00461     DM35424_Check_Result(result,
00462         "Error converting voltage to conversion");
00463 }
00464
00465 if (keypress == '@') {
00466     result = DM35424_Dac_Volts_To_Conv(-2, &conv_value);
00467     DM35424_Check_Result(result,
00468         "Error converting voltage to conversion");
00469 }
00470
00471 if (keypress == '#') {
00472     result = DM35424_Dac_Volts_To_Conv(-3, &conv_value);
00473     DM35424_Check_Result(result,
00474         "Error converting voltage to conversion");
00475 }
00476
00477 if (keypress == '$') {
00478     result = DM35424_Dac_Volts_To_Conv(-4, &conv_value);
00479     DM35424_Check_Result(result,
00480         "Error converting voltage to conversion");
00481 }
00482
00483 if (keypress == '%') {
00484     result = DM35424_Dac_Volts_To_Conv(-5, &conv_value);
00485     DM35424_Check_Result(result,
00486         "Error converting voltage to conversion");
00487 }
00488
00489 if (keypress != 'c') {
00490     result = DM35424_Dac_Set_Last_Conversion(board,
00491         &my_dac,
```



```

00509                                     channel_to_change,
00510                                     0,
00511                                     conv_value);
00512     DM35424_Check_Result(result, "Error setting last conversion");
00513
00514     if (result != 0) {
00515         printf
00516             ("ERROR setting last conversion.  Errno = %d.\n",
00517              errno);
00518         keypress = 'q';
00519     }
00520 }
00521
00522     for (channel = 0; channel < my_dac.num_dma_channels; channel++) {
00523
00524         result = DM35424_Dac_Get_Last_Conversion(board,
00525                                                  &my_dac,
00526                                                  channel,
00527                                                  &marker,
00528                                                  &register_value);
00529
00530         if (result != 0) {
00531             printf
00532                 ("ERROR getting last conversion.  Errno = %d.\n",
00533                  errno);
00534             keypress = 'q';
00535         }
00536
00537         result = DM35424_Dac_Conv_To_Volts((int32_t) register_value,
00538                                           &voltage);
00539
00540         if (result != 0) {
00541             printf
00542                 ("ERROR converting conversion to voltage.  Errno = %d.\n",
00543                  errno);
00544             keypress = 'q';
00545         }
00546         if (channel == channel_to_change) {
00547             printf(" [%3.2f] ", voltage);
00548         } else {
00549             printf("  %3.2f ", voltage);
00550         }
00551     }
00552
00553     printf("\n");
00554
00555 }
00556
00557     tcsetattr(STDIN_FILENO, TCSANOW, &old_tio);
00558
00559     result = DM35424_Gbc_Board_Reset(board);
00560     printf("\n\nClosing Board\n");
00561     result = DM35424_Board_Close(board);
00562
00563     DM35424_Check_Result(result, "Error closing board.");
00564
00565     printf("Example program successfully completed.\n");
00566     return 0;
00567
00568 }
00569

```

6.17 examples/dm35424_dac_dma.c File Reference

Example program which demonstrates the use of the DAC and DMA.

```

#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <unistd.h>
#include <limits.h>
#include <getopt.h>
#include <signal.h>
#include <string.h>

```

```
#include "dm35424_gbc_library.h"
#include "dm35424_dac_library.h"
#include "dm35424_ioctl.h"
#include "dm35424_examples.h"
#include "dm35424_dma_library.h"
#include "dm35424_util_library.h"
#include "dm35424.h"
```

Macros

- `#define NUM_BUFFERS_TO_USE 1`
- `#define DEFAULT_DAC_TO_USE 0`
- `#define DEFAULT_RATE 100`
- `#define BUFFER_SIZE_SAMPLES 100`
- `#define BUFFER_SIZE_BYTES (BUFFER_SIZE_SAMPLES * sizeof(int))`

Functions

- static void **usage** (void)
Print information on stderr about how the program is to be used. After doing so, the program is exited.
- static void **sigint_handler** (int signal_number)
Signal handler for SIGINT Control-C keyboard interrupt.
- int **main** (int argument_count, char **arguments)
The main program.

Variables

- static char * **program_name**
- volatile int **exit_program** = 0

6.17.1 Detailed Description

Example program which demonstrates the use of the DAC and DMA.

This example program generates wave form data and "plays" it out all channels for the specified DAC. To see the output data, connect an oscilloscope to the DAC0 Channel 0 - 3 pins, or appropriate DAC/Channel pins if you change the DAC number.

The odd-numbered channels will be delayed in their starting so that they are a half-period out of phase.

After the program is running, you can alter the rate of DAC output by entering a new frequency and hitting Enter. Note that the frequency of the waveform seen on an oscilloscope will be different than the frequency of the DAC, depending on the number of samples used in creating the wave.

Use the `--help` command line option to see all possible input values.

Hit Ctrl-C to exit the example.

This file and its contents are copyright (C) RTD Embedded Technologies, Inc. All Rights Reserved.

This software is licensed as described in the RTD End-User Software License Agreement. For a copy of this agreement, refer to the file LICENSE.TXT (which should be included with this software) or contact RTD Embedded Technologies, Inc.

Id

[dm35424_dac_dma.c](#) 141531 2024-03-06 21:05:25Z lfrankenfield

Definition in file [dm35424_dac_dma.c](#).

6.17.2 Macro Definition Documentation

6.17.2.1 BUFFER_SIZE_BYTES

```
#define BUFFER_SIZE_BYTES (BUFFER_SIZE_SAMPLES * sizeof(int))
```

Buffer size to allocate in bytes

Definition at line 86 of file [dm35424_dac_dma.c](#).

6.17.2.2 BUFFER_SIZE_SAMPLES

```
#define BUFFER_SIZE_SAMPLES 100
```

Number of samples to create. Increase this number for a "finer" waveform.

Definition at line 81 of file [dm35424_dac_dma.c](#).

6.17.2.3 DEFAULT_DAC_TO_USE

```
#define DEFAULT_DAC_TO_USE 0
```

DAC to use if user does not enter one on the command line

Definition at line 70 of file [dm35424_dac_dma.c](#).

Referenced by [main\(\)](#), and [usage\(\)](#).

6.17.2.4 DEFAULT_RATE

```
#define DEFAULT_RATE 100
```

Rate to use if user does not enter one on the command line (Hz)

Definition at line 75 of file [dm35424_dac_dma.c](#).

6.17.2.5 NUM_BUFFERS_TO_USE

```
#define NUM_BUFFERS_TO_USE 1
```

We will only use one buffer in this example, and loop it

Definition at line 65 of file [dm35424_dac_dma.c](#).

6.17.3 Function Documentation

6.17.3.1 main()

```
int main (
    int argument_count,
    char ** arguments)
```

The main program.

Parameters

<i>argument_count</i>	
-----------------------	--

Number of args passed on the command line, including the executable name

Parameters

<i>arguments</i>	
------------------	--

Pointer to array of character strings, which are the args themselves.

Return values

<i>0</i>	
----------	--

Success

Return values

<i>Non-Zero</i>	
-----------------	--

Failure. We load the even channels with the wave pattern, and the odd channels with the same pattern, but offset by half its length. Doing this gives us an opposing pattern between the even and odd channels, which helps when using DAC for ADC input, or just viewing on a scope.

Definition at line 186 of file [dm35424_dac_dma.c](#).

References [board](#), [BUFFER_0](#), [BUFFER_SIZE_BYTES](#), [BUFFER_SIZE_SAMPLES](#), [DAC_OPTION](#), [DEFAULT_DAC_TO_USE](#), [DEFAULT_RATE](#), [DM35424_Board_Close\(\)](#), [DM35424_Board_Open\(\)](#), [DM35424_Check_Result\(\)](#), [DM35424_CLK_SRC_IMMEDIATE](#), [DM35424_CLK_SRC_NEVER](#), [DM35424_Dac_Open\(\)](#), [DM35424_Dac_Set_Clock_Src\(\)](#), [DM35424_Dac_Set_Conversion_Rate\(\)](#), [DM35424_Dac_Set_Start_Trigger\(\)](#), [DM35424_Dac_Set_Stop_Trigger\(\)](#), [DM35424_Dac_Start\(\)](#), [DM35424_Dac_Volts_To_Conv\(\)](#), [DM35424_DMA_BUFFER_CTRL_LOOP](#), [DM35424_DMA_BUFFER_CTRL_VALID](#), [DM35424_Dma_Buffer_Setup\(\)](#), [DM35424_Dma_Buffer_Status\(\)](#), [DM35424_Dma_Initialize\(\)](#), [DM35424_Dma_Setup\(\)](#), [DM35424_DMA_SETUP_DIRECTION_WRITE](#), [DM35424_Dma_Start\(\)](#), [DM35424_Dma_Status\(\)](#), [DM35424_Dma_Write\(\)](#), [DM35424_Gbc_Board_Reset\(\)](#), [DM35424_Generate_Signal_Data\(\)](#), [DM35424_NUM_DAC_ON_BOARD](#), [DM35424_SAWTOOTH_WAVE](#), [DM35424_SINE_WAVE](#), [DM35424_SQUARE_WAVE](#), [exit_program](#), [HELP_OPTION](#), [IGNORE_USED](#), [MINOR_OPTION](#), [PATTERN_OPTION](#), [program_name](#), [RATE_OPTION](#), [sigint_handler\(\)](#), [usage\(\)](#), and [WAVE_OPTION](#).

6.17.3.2 sigint_handler()

```
void sigint_handler (  
    int signal_number)    [static]
```

Signal handler for SIGINT Control-C keyboard interrupt.

Parameters

<i>signal_number</i>	
----------------------	--

Signal number passed in from the kernel.

Warning

One must be extremely careful about what functions are called from a signal handler.

Definition at line [154](#) of file [dm35424_dac_dma.c](#).

References [exit_program](#).

6.17.4 Variable Documentation

6.17.4.1 exit_program

```
volatile int exit_program = 0
```

Boolean indicating the program should be exited.

Definition at line [97](#) of file [dm35424_dac_dma.c](#).

6.17.4.2 program_name

```
char* program_name    [static]
```

Name of the program as invoked on the command line

Definition at line [92](#) of file [dm35424_dac_dma.c](#).


```

00193     char input_str[200];
00194
00195     struct sigaction signal_action;
00196
00197     unsigned int current_buffer;
00198     uint32_t current_count, index = 0;
00199     int current_action;
00200     int status_overflow;
00201     int status_underflow;
00202     int status_used;
00203     int status_invalid;
00204     int status_complete;
00205
00206     uint8_t buff_status;
00207     uint8_t buff_control;
00208     uint32_t buff_size;
00209     unsigned int channel = 0;
00210     unsigned int dac_num = DEFAULT_DAC_TO_USE;
00211     uint32_t rate = DEFAULT_RATE, actual_rate;
00212
00213     int help_option_given = 0;
00214     int wave_option_given = 0;
00215
00216     int16_t max_value, min_value, offset;
00217
00218     unsigned int pattern = 0;
00219
00220     float max_voltage, min_voltage, offset_voltage;
00221
00222     int status;
00223     char waveform_str[100];
00224     enum DM35424_Waveforms waveform = DM35424_SINE_WAVE;
00225
00226     int32_t *buffer, *offset_buffer;
00227     char *invalid_char_p;
00228     struct option options[] = {
00229         {"help", 0, 0, HELP_OPTION},
00230         {"minor", 1, 0, MINOR_OPTION},
00231         {"rate", 1, 0, RATE_OPTION},
00232         {"wave", 1, 0, WAVE_OPTION},
00233         {"dac", 1, 0, DAC_OPTION},
00234         {"pattern", 1, 0, PATTERN_OPTION},
00235         {0, 0, 0, 0}
00236     };
00237
00238     program_name = arguments[0];
00239
00240     // Show usage, parse arguments
00241     while (1) {
00242         /*
00243          * Parse the next command line option and any arguments it may require
00244          */
00245         status = getopt_long(argument_count,
00246                             arguments, "", options, NULL);
00247
00248         /*
00249          * If getopt_long() returned -1, then all options have been processed
00250          */
00251         if (status == -1) {
00252             break;
00253         }
00254
00255         /*
00256          * Figure out what getopt_long() found
00257          */
00258         switch (status) {
00259
00260             /*#####
00261              * User entered '--help'
00262              #####*/
00263             case HELP_OPTION:
00264                 help_option_given = 0xFF;
00265                 break;
00266
00267             /*#####
00268              * User entered '--minor'
00269              #####*/
00270             case MINOR_OPTION:
00271                 /*
00272                  * Convert option argument string to unsigned long integer
00273                  */
00274                 errno = 0;
00275                 minor = strtoul(optarg, &invalid_char_p, 10);
00276
00277                 /*
00278                  * Catch unsigned long int overflow
00279                  */

```

```

00280         if ((minor == ULONG_MAX)
00281             && (errno == ERANGE)) {
00282             error(0, 0,
00283                 "ERROR: Device minor number caused numeric overflow");
00284             usage();
00285         }
00286
00287         /*
00288          * Catch argument strings with valid decimal prefixes, for
00289          * example "1q", and argument strings which cannot be converted,
00290          * for example "abc1"
00291          */
00292         if ((*invalid_char_p != '\0')
00293             || (invalid_char_p == optarg)) {
00294             error(0, 0,
00295                 "ERROR: Non-decimal device minor number");
00296             usage();
00297         }
00298
00299         break;
00300
00301     /*#####
00302      User entered rate
00303     ##### */
00304     case RATE_OPTION:
00305         /*
00306          * Convert option argument string to unsigned long integer
00307          */
00308         errno = 0;
00309         rate = strtoul(optarg, &invalid_char_p, 10);
00310
00311         /*
00312          * Catch unsigned long int overflow
00313          */
00314         if ((rate == ULONG_MAX)
00315             && (errno == ERANGE)) {
00316             error(0, 0,
00317                 "ERROR: Rate number caused numeric overflow");
00318             usage();
00319         }
00320
00321         /*
00322          * Catch argument strings with valid decimal prefixes, for
00323          * example "1q", and argument strings which cannot be converted,
00324          * for example "abc1"
00325          */
00326         if ((*invalid_char_p != '\0')
00327             || (invalid_char_p == optarg)) {
00328             error(0, 0,
00329                 "ERROR: Non-decimal rate value entered");
00330             usage();
00331         }
00332         break;
00333
00334     /*#####
00335      User entered a waveform choice
00336     ##### */
00337     case WAVE_OPTION:
00338         strcpy(waveform_str, optarg);
00339         wave_option_given = 1;
00340         break;
00341
00342     /*#####
00343      User entered '--dac'
00344     ##### */
00345     case DAC_OPTION:
00346         errno = 0;
00347         dac_num =
00348             (unsigned int)strtoul(optarg, &invalid_char_p, 10);
00349
00350         /*
00351          * Catch unsigned long int overflow
00352          */
00353         if ((dac_num == ULONG_MAX)
00354             && (errno == ERANGE)) {
00355             error(0, 0,
00356                 "ERROR: DAC number caused numeric overflow");
00357             usage();
00358         }
00359
00360         /*
00361          * Catch argument strings with valid decimal prefixes, for
00362          * example "1q", and argument strings which cannot be converted,
00363          * for example "abc1"
00364          */
00365         if ((*invalid_char_p != '\0')
00366             || (invalid_char_p == optarg)) {

```



```

00367             error(0, 0, "ERROR: Non-decimal DAC number");
00368             usage();
00369         }
00370
00371         break;
00372
00373         /*#####
00374         User entered '--pattern'
00375         #####*/
00376     case PATTERN_OPTION:
00377         errno = 0;
00378         pattern =
00379             (unsigned int)strtoul(optarg, &invalid_char_p, 10);
00380
00381         /*
00382          * Catch unsigned long int overflow
00383          */
00384         if ((pattern == ULONG_MAX)
00385             && (errno == ERANGE)) {
00386             error(0, 0,
00387                 "ERROR: Pattern number caused numeric overflow");
00388             usage();
00389         }
00390
00391         /*
00392          * Catch argument strings with valid decimal prefixes, for
00393          * example "1q", and argument strings which cannot be converted,
00394          * for example "abc1"
00395          */
00396         if ((*invalid_char_p != '\0')
00397             || (invalid_char_p == optarg)) {
00398             error(0, 0,
00399                 "ERROR: Non-decimal pattern number");
00400             usage();
00401         }
00402         break;
00403
00404     case '?':
00405         usage();
00406         break;
00407
00408     /*#####
00409     getopt_long() returned unexpected value
00410     #####*/
00411     default:
00412         error(EXIT_FAILURE,
00413             0,
00414             "ERROR: getopt_long() returned unexpected value %#x",
00415             status);
00416         break;
00417     }
00418 }
00419
00420 /*
00421  * Recognize '--help' option before any others
00422  */
00423
00424 if (help_option_given) {
00425     usage();
00426 }
00427
00428 if (!wave_option_given) {
00429     error(0, 0, "ERROR: Please specify a waveform to display.");
00430     usage();
00431 }
00432
00433 if (dac_num >= DM35424_NUM_DAC_ON_BOARD) {
00434     error(0, 0,
00435         "ERROR: Specified DAC not available on this board.");
00436     usage();
00437 }
00438
00439 if (strcmp(waveform_str, "sine") == 0) {
00440     waveform = DM35424_SINE_WAVE;
00441 } else if (strcmp(waveform_str, "square") == 0) {
00442     waveform = DM35424_SQUARE_WAVE;
00443 } else if (strcmp(waveform_str, "sawtooth") == 0) {
00444     waveform = DM35424_SAWTOOTH_WAVE;
00445 } else {
00446     error(0, 0,
00447         "ERROR: Invalid waveform specified. Please use either sine, square, or
00448         sawtooth.");
00449     usage();
00450 }
00451
00452 switch (pattern) {

```

```

00453     case 1:
00454         max_voltage = 1.25f;
00455         min_voltage = -1.25f;
00456         offset_voltage = 2.5f;
00457         break;
00458     default:
00459         max_voltage = 4.9998474121f;
00460         min_voltage = -5.0f;
00461         offset_voltage = 0.0f;
00462         break;
00463 }
00464
00465 signal_action.sa_handler = sigint_handler;
00466 sigfillset(&(signal_action.sa_mask));
00467 signal_action.sa_flags = 0;
00468
00469 if (sigaction(SIGINT, &signal_action, NULL) < 0) {
00470     error(EXIT_FAILURE, errno, "ERROR: sigaction() FAILED");
00471 }
00472
00473 printf("Opening board....");
00474 result = DM35424_Board_Open(minor, &board);
00475
00476 DM35424_Check_Result(result, "Could not open board");
00477 printf("success.\nResetting board....");
00478 result = DM35424_Gbc_Board_Reset(board);
00479
00480 DM35424_Check_Result(result, "Could not reset board");
00481 printf("success.\nOpening DAC.....");
00482
00483 result = DM35424_Dac_Open(board, dac_num, &my_dac);
00484
00485 DM35424_Check_Result(result, "Could not open DAC");
00486
00487 printf("Found DAC%u, with %d DMA channels (%d buffers each)\n",
00488     dac_num, my_dac.num_dma_channels, my_dac.num_dma_buffers);
00489
00490 result = DM35424_Dac_Set_Clock_Src(board,
00491     &my_dac,
00492     DM35424_CLK_SRC_IMMEDIATE);
00493
00494 DM35424_Check_Result(result, "Error setting DAC clock");
00495
00496 result = DM35424_Dac_Set_Conversion_Rate(board,
00497     &my_dac, rate, &actual_rate);
00498
00499 fprintf(stdout, "Rate requested: %d Actual Rate Achieved: %d\n", rate,
00500     actual_rate);
00501 DM35424_Check_Result(result, "Error setting sample rate");
00502
00503 buffer = (int *)malloc(BUFFER_SIZE_BYTES);
00504 offset_buffer = (int *)malloc(BUFFER_SIZE_BYTES);
00505
00506 DM35424_Check_Result(buffer == NULL
00507     || offset_buffer == NULL,
00508     "Error allocating space for buffer.");
00509
00510 result = DM35424_Dac_Volts_To_Conv(max_voltage, &max_value);
00511
00512 DM35424_Check_Result(result, "Error converting value to conversion counts.");
00513
00514 result = DM35424_Dac_Volts_To_Conv(min_voltage, &min_value);
00515
00516 DM35424_Check_Result(result, "Error converting value to conversion counts.");
00517
00518 result = DM35424_Dac_Volts_To_Conv(offset_voltage, &offset);
00519
00520 DM35424_Check_Result(result, "Error converting value to conversion counts.");
00521
00522 result = DM35424_Generate_Signal_Data(waveform,
00523     buffer,
00524     BUFFER_SIZE_SAMPLES,
00525     max_value,
00526     min_value,
00527     offset,
00528     0x0000FFFF);
00529
00530 DM35424_Check_Result(result, "Error trying to generate data for the DAC.");
00531
00532 for (index = 0; index < BUFFER_SIZE_SAMPLES; index++) {
00533     offset_buffer[index] =
00534         buffer[(index +
00535             (BUFFER_SIZE_SAMPLES / 2)) % BUFFER_SIZE_SAMPLES];
00536 }
00537
00538 for (channel = 0; channel < my_dac.num_dma_channels; channel++) {
00539     fprintf(stdout,

```

```

00540         "Initializing and configuring DMA Channel %d....",
00541         channel);
00542     result =
00543         DM35424_Dma_Initialize(board, &my_dac, channel,
00544                                NUM_BUFFERS_TO_USE,
00545                                BUFFER_SIZE_BYTES);
00546
00547     DM35424_Check_Result(result, "Error initializing DMA");
00548
00549     result = DM35424_Dma_Setup(board,
00550                                &my_dac,
00551                                channel,
00552                                DM35424_DMA_SETUP_DIRECTION_WRITE,
00553                                IGNORE_USED);
00554
00555     DM35424_Check_Result(result, "Error configuring DMA");
00556
00557     fprintf(stdout, "success!\n");
00558
00559     result = DM35424_Dma_Status(board,
00560                                &my_dac,
00561                                channel,
00562                                &current_buffer,
00563                                &current_count,
00564                                &current_action,
00565                                &status_overflow,
00566                                &status_underflow,
00567                                &status_used,
00568                                &status_invalid, &status_complete);
00569
00570     DM35424_Check_Result(result, "Error getting DMA status");
00571
00572     printf
00573         ("DMA Status: Current Buffer: %d Count: %d Action: 0x%x Status: "
00574          "Ov: %d Un: %d Used: %d Inv: %d Comp: %d\n",
00575          current_buffer, current_count, current_action,
00576          status_overflow, status_underflow, status_used,
00577          status_invalid, status_complete);
00578
00579     result = DM35424_Dma_Buffer_Setup(board,
00580                                       &my_dac,
00581                                       channel,
00582                                       BUFFER_0,
00583                                       DM35424_DMA_BUFFER_CTRL_VALID |
00584                                       DM35424_DMA_BUFFER_CTRL_LOOP);
00585
00586     DM35424_Check_Result(result, "Error setting up buffer control.");
00587
00588     result = DM35424_Dma_Buffer_Status(board,
00589                                       &my_dac,
00590                                       channel,
00591                                       BUFFER_0,
00592                                       &buff_status,
00593                                       &buff_control, &buff_size);
00594
00595     fprintf(stdout,
00596             "Buffer 0: Stat: 0x%x Ctrl: 0x%x Size: %u\n",
00597             buff_status, buff_control, buff_size);
00598
00599     DM35424_Check_Result(result, "Error getting buffer status.");
00600
00601     if (channel % 2 == 0) {
00602         result = DM35424_Dma_Write(board,
00603                                    &my_dac,
00604                                    channel,
00605                                    BUFFER_0, BUFFER_SIZE_BYTES,
00606                                    buffer);
00607
00608         DM35424_Check_Result(result, "Writing to DMA buffer failed");
00609     } else {
00610         result = DM35424_Dma_Write(board,
00611                                    &my_dac,
00612                                    channel,
00613                                    BUFFER_0, BUFFER_SIZE_BYTES,
00614                                    offset_buffer);
00615
00616         DM35424_Check_Result(result, "Writing to DMA buffer failed");
00617     }
00618
00619     fprintf(stdout, "Starting DMA Channel %d.....", channel);
00620     result = DM35424_Dma_Start(board, &my_dac, channel);
00621
00622     DM35424_Check_Result(result, "Error starting DMA");
00623
00624     printf("success.\n");
00625 }

```

```

00633
00634     free(buffer);
00635     free(offset_buffer);
00636
00637     fprintf(stdout, "Starting DAC.\n");
00638
00639     result = DM35424_Dac_Set_Start_Trigger(board,
00640                                             &my_dac,
00641                                             DM35424_CLK_SRC_IMMEDIATE);
00642
00643     DM35424_Check_Result(result, "Error setting start trigger for DAC.");
00644
00645     result = DM35424_Dac_Set_Stop_Trigger(board,
00646                                             &my_dac,
00647                                             DM35424_CLK_SRC_NEVER);
00648
00649     DM35424_Check_Result(result, "Error setting stop trigger for DAC.");
00650
00651     result = DM35424_Dac_Start(board, &my_dac);
00652
00653     DM35424_Check_Result(result, "Error starting DAC");
00654
00655     printf("\nPress Ctrl-C to exit.\n\n");
00656     while (!exit_program) {
00657
00658         printf("Current Rate: %d    Enter new rate: ", actual_rate);
00659
00660         if (scanf("%s", input_str) < 0) {
00661             exit_program = 1;
00662             printf("Error reading input\n");
00663             continue;
00664         }
00665
00666         rate = atoi(input_str);
00667
00668         if (rate > 0) {
00669             result = DM35424_Dac_Set_Conversion_Rate(board,
00670                                                         &my_dac,
00671                                                         rate,
00672                                                         &actual_rate);
00673
00674             DM35424_Check_Result(result, "Error setting sample rate");
00675         }
00676     }
00677
00678     result = DM35424_Gbc_Board_Reset(board);
00679     printf("success.\nClosing Board...\n");
00680     result = DM35424_Board_Close(board);
00681
00682     DM35424_Check_Result(result, "Error closing board.");
00683     printf("success.\nExample program successfully completed.\n\n");
00684
00685     return 0;
00686
00687
00688 }

```

6.19 examples/dm35424_dio.c File Reference

Example program which demonstrates the use of the DIO.

```

#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <unistd.h>
#include <limits.h>
#include <getopt.h>
#include "dm35424_gbc_library.h"
#include "dm35424_dio_library.h"
#include "dm35424_ioctl.h"
#include "dm35424_util_library.h"
#include "dm35424_examples.h"

```

Macros

- `#define DM35424_DIO_DIRECTION 0x0000007F`

Functions

- static void **usage** (void)
Print information on stderr about how the program is to be used. After doing so, the program is exited.
- int **main** (int argument_count, char **arguments)
The main program.

Variables

- static char * **program_name**
- struct **DM35424_Board_Descriptor** * **board**
- struct **DM35424_Function_Block** **my_dio**

6.19.1 Detailed Description

Example program which demonstrates the use of the DIO.

This example program sets the lower 7-bits of DIO to output, and the upper 7-bits to input. We'll then write every possible 7-bit value to the output and verify the same value on the input pins.

This example requires a loopback from DIO Port 0:0 to Port 0:7, Port 0:1 to Port 0:8, etc.

Port 0, Pins 0-6: Output
Port 0, Pins 7-13: Input

```
-----
This file and its contents are copyright (C) RTD Embedded Technologies,
Inc. All Rights Reserved.
```

```
This software is licensed as described in the RTD End-User Software License
Agreement. For a copy of this agreement, refer to the file LICENSE.TXT
(which should be included with this software) or contact RTD Embedded
Technologies, Inc.
-----
```

Id

`dm35424_dio.c` 141531 2024-03-06 21:05:25Z lfrankenfield

Definition in file `dm35424_dio.c`.

6.19.2 Macro Definition Documentation

6.19.2.1 DM35424_DIO_DIRECTION

```
#define DM35424_DIO_DIRECTION 0x0000007F
```

Setting for DIO pin direction

Definition at line 55 of file [dm35424_dio.c](#).

Referenced by [main\(\)](#).

6.19.3 Function Documentation

6.19.3.1 main()

```
int main (
    int argument_count,
    char ** arguments)
```

The main program.

Parameters

<i>argument_count</i>	
-----------------------	--

Number of args passed on the command line, including the executable name

Parameters

<i>arguments</i>	
------------------	--

Pointer to array of character strings, which are the args themselves.

Return values

0	
---	--

Success

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

Definition at line 124 of file [dm35424_dio.c](#).

References [board](#), [DM35424_Board_Close\(\)](#), [DM35424_Board_Open\(\)](#), [DM35424_Check_Result\(\)](#), [DM35424_DIO_DIRECTION](#), [DM35424_Dio_Get_Input_Value\(\)](#), [DM35424_Dio_Open\(\)](#), [DM35424_Dio_Set_Direction\(\)](#), [DM35424_Dio_Set_Output_Value\(\)](#), [DM35424_Gbc_Board_Reset\(\)](#), [HELP_OPTION](#), [MINOR_OPTION](#), [my_dio](#), [program_name](#), and [usage\(\)](#).

6.19.4 Variable Documentation

6.19.4.1 board

```
struct DM35424_Board_Descriptor* board
```

Descriptor for board

Definition at line 65 of file [dm35424_dio.c](#).

6.19.4.2 my_dio

```
struct DM35424_Function_Block my_dio
```

Descriptor for DIO function block

Definition at line 70 of file [dm35424_dio.c](#).

Referenced by [main\(\)](#).

6.19.4.3 program_name

```
char* program_name [static]
```

Name of the program as invoked on the command line

Definition at line 60 of file [dm35424_dio.c](#).

6.20 dm35424_dio.c

[Go to the documentation of this file.](#)

```
00001
00036
00037 #include <stdio.h>
00038 #include <stddef.h>
00039 #include <stdlib.h>
00040 #include <errno.h>
00041 #include <error.h>
00042 #include <unistd.h>
00043 #include <limits.h>
00044 #include <getopt.h>
00045
00046 #include "dm35424_gbc_library.h"
00047 #include "dm35424_dio_library.h"
00048 #include "dm35424_ioctl.h"
00049 #include "dm35424_util_library.h"
00050 #include "dm35424_examples.h"
00051
00055 #define DM35424_DIO_DIRECTION          0x0000007F
00056
00060 static char *program_name;
00061
00065 struct DM35424_Board_Descriptor *board;
00066
00070 struct DM35424_Function_Block my_dio;
00071
00079
00080 static void usage(void)
00081 {
00082     fprintf(stderr, "\n");
00083     fprintf(stderr, "NAME\n\n\t%s\n\n", program_name);
```

```

00084         fprintf(stderr, "USAGE\n\n\t%s [OPTIONS]\n\n", program_name);
00085
00086         fprintf(stderr, "OPTIONS\n\n");
00087         fprintf(stderr, "\t--help\n");
00088         fprintf(stderr, "\t\tShow this help screen and exit.\n\n");
00089         fprintf(stderr, "\t--minor NUM\n");
00090         fprintf(stderr, "\t\tSpecify the minor number (>= 0) of the board to open.  When not
specified,\n");
00091         fprintf(stderr, "\t\tthe device file with minor 0 is opened.\n");
00092         fprintf(stderr, "\n");
00093         exit(EXIT_FAILURE);
00094     }
00095
00096
00123
00124 int main(int argument_count, char **arguments)
00125 {
00126
00127     unsigned long int minor = 0;
00128     int result;
00129
00130     int dio_num = 0;
00131     int help_option_given = 0;
00132     int status;
00133     uint32_t input_value;
00134     uint32_t output_value;
00135
00136     char *invalid_char_p;
00137     struct option options[] = {
00138         {"help", 0, 0, 1},
00139         {"minor", 1, 0, 2},
00140         {0, 0, 0, 0}
00141     };
00142
00143     program_name = arguments[0];
00144
00145     // Show usage, parse arguments
00146     while (1) {
00147         /*
00148          * Parse the next command line option and any arguments it may require
00149          */
00150         status = getopt_long(argument_count,
00151                             arguments, "", options, NULL);
00152
00153         /*
00154          * If getopt_long() returned -1, then all options have been processed
00155          */
00156         if (status == -1) {
00157             break;
00158         }
00159
00160         /*
00161          * Figure out what getopt_long() found
00162          */
00163         switch (status) {
00164
00165             /*#####
00166              User entered '--help'
00167             #####*/
00168             case HELP_OPTION:
00169                 help_option_given = 0xFF;
00170                 break;
00171
00172             /*#####
00173              User entered '--minor'
00174             #####*/
00175             case MINOR_OPTION:
00176                 /*
00177                  * Convert option argument string to unsigned long integer
00178                  */
00179                 errno = 0;
00180                 minor = strtoul(optarg, &invalid_char_p, 10);
00181
00182                 /*
00183                  * Catch unsigned long int overflow
00184                  */
00185                 if ((minor == ULONG_MAX)
00186                     && (errno == ERANGE)) {
00187                     error(0, 0,
00188                          "ERROR: Device minor number caused numeric overflow");
00189                     usage();
00190                 }
00191
00192                 /*
00193                  * Catch argument strings with valid decimal prefixes, for
00194                  * example "1k", and argument strings which cannot be converted,
00195                  * for example "abc1"
00196                  */

```



```

00197         if ((*invalid_char_p != '\0')
00198             || (invalid_char_p == optarg)) {
00199             error(0, 0,
00200                  "ERROR: Non-decimal device minor number");
00201             usage();
00202         }
00203     }
00204     break;
00205
00206     /*#####
00207     User entered unsupported option
00208     ##### */
00209     case '?':
00210         usage();
00211         break;
00212
00213     /*#####
00214     getopt_long() returned unexpected value
00215     ##### */
00216     default:
00217         error(EXIT_FAILURE,
00218              0,
00219              "ERROR: getopt_long() returned unexpected value %s",
00220              status);
00221         break;
00222     }
00223 }
00224
00225 /*
00226  * Recognize '--help' option before any others
00227  */
00228
00229 if (help_option_given) {
00230     usage();
00231 }
00232
00233 printf("Opening board....");
00234 result = DM35424_Board_Open(minor, &board);
00235
00236 DM35424_Check_Result(result, "Could not open board");
00237 printf("success.\nResetting board....");
00238 result = DM35424_Gbc_Board_Reset(board);
00239
00240 DM35424_Check_Result(result, "Could not reset board");
00241 printf("success.\nOpening DIO.....");
00242
00243 result = DM35424_Dio_Open(board, dio_num, &my_dio);
00244
00245 DM35424_Check_Result(result, "Could not open DIO");
00246
00247 printf("Found DIO%d\n", dio_num);
00248
00249 /*
00250  * Set the direction of bits 0-15 to output and bits 16-31 to input
00251  */
00252 result = DM35424_Dio_Set_Direction(board,
00253                                     &my_dio, DM35424_DIO_DIRECTION);
00254
00255 DM35424_Check_Result(result, "Could not set direction of DIO pins.");
00256
00257 output_value = 0;
00258 input_value = 0;
00259
00260 for (output_value = 0; output_value <= 0x7F; output_value++) {
00261     result = DM35424_Dio_Set_Output_Value(board,
00262                                             &my_dio, output_value);
00263
00264     DM35424_Check_Result(result, "Could not set output value.");
00265
00266     result = DM35424_Dio_Get_Input_Value(board,
00267                                           &my_dio, &input_value);
00268
00269     input_value >>= 7;
00270
00271     printf("Output: %u\t\tInput: %u\n", output_value, input_value);
00272
00273     DM35424_Check_Result(!(output_value == input_value),
00274                          "Values do not match!");
00275 }
00276
00277 result = DM35424_Gbc_Board_Reset(board);
00278 printf("Closing Board\n");
00279 result = DM35424_Board_Close(board);
00280
00281 DM35424_Check_Result(result, "Error closing board.");

```

```

00284         printf("Example program successfully completed.\n");
00285         return 0;
00286
00287     }

```

6.21 examples/dm35424_list_fb.c File Reference

Example program which demonstrates use of the library to open a function block for use.

```

#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <unistd.h>
#include <limits.h>
#include <getopt.h>
#include "dm35424_gbc_library.h"
#include "dm35424_ioctl.h"
#include "dm35424_examples.h"
#include "dm35424_util_library.h"

```

Functions

- static void **usage** (void)
Print information on stderr about how the program is to be used. After doing so, the program is exited.
- int **main** (int argument_count, char **arguments)
The main program.

Variables

- static char * **program_name**

6.21.1 Detailed Description

Example program which demonstrates use of the library to open a function block for use.

This example program uses the board library to query all function blocks on the board. When a function block is opened that has a valid function type, then the number of DMA channels and buffers is printed to the screen. In this way, the example program shows an inventory of the function blocks on a given board.

This file and its contents are copyright (C) RTD Embedded Technologies, Inc. All Rights Reserved.

This software is licensed as described in the RTD End-User Software License Agreement. For a copy of this agreement, refer to the file LICENSE.TXT (which should be included with this software) or contact RTD Embedded Technologies, Inc.

ld

[dm35424_list_fb.c](#) 141531 2024-03-06 21:05:25Z lfrankenfield

Definition in file [dm35424_list_fb.c](#).

6.21.2 Function Documentation

6.21.2.1 main()

```
int main (
    int argument_count,
    char ** arguments)
```

The main program.

Parameters

<i>argument_count</i>	
-----------------------	--

Number of args passed on the command line, including the executable name

Parameters

<i>arguments</i>	
------------------	--

Pointer to array of character strings, which are the args themselves.

Return values

0	
---	--

Success

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

Definition at line 105 of file [dm35424_list_fb.c](#).

References [board](#), [DM35424_Board_Close\(\)](#), [DM35424_Board_Open\(\)](#), [DM35424_Check_Result\(\)](#), [DM35424_FUNC_BLOCK_ADC](#), [DM35424_FUNC_BLOCK_DAC](#), [DM35424_FUNC_BLOCK_DIO](#), [DM35424_FUNC_BLOCK_INVALID](#), [DM35424_FUNC_BLOCK_R](#), [DM35424_FUNC_BLOCK_TEMPERATURE_SENSOR](#), [DM35424_Function_Block_Open\(\)](#), [DM35424_Gbc_Board_Reset\(\)](#), [DM35424_Gbc_Get_Fpga_Build\(\)](#), [DM35424_Gbc_Get_Pdp_Number\(\)](#), [DM35424_Gbc_Get_Revision\(\)](#), [DM35424_MAX_FB](#), [DM35424_Function_Block::fb_num](#), [HELP_OPTION](#), [MINOR_OPTION](#), [DM35424_Function_Block::num_dma_buffers](#), [DM35424_Function_Block::num_dma_channels](#), [program_name](#), [DM35424_Function_Block::sub_type](#), [DM35424_Function_Block::t](#) and [usage\(\)](#).

6.21.3 Variable Documentation

6.21.3.1 program_name

```
char* program_name [static]
```

Name of the program as invoked on the command line

Definition at line 51 of file [dm35424_list_fb.c](#).

6.22 dm35424_list_fb.c

[Go to the documentation of this file.](#)

```

00001
00033
00034 #include <stdio.h>
00035 #include <stddef.h>
00036 #include <stdlib.h>
00037 #include <errno.h>
00038 #include <error.h>
00039 #include <unistd.h>
00040 #include <limits.h>
00041 #include <getopt.h>
00042
00043 #include "dm35424_gbc_library.h"
00044 #include "dm35424_ioctl.h"
00045 #include "dm35424_examples.h"
00046 #include "dm35424_util_library.h"
00047
00051 static char *program_name;
00052
00060
00061 static void usage(void)
00062 {
00063     fprintf(stderr, "\n");
00064     fprintf(stderr, "NAME\n\t%s\n", program_name);
00065     fprintf(stderr, "USAGE\n\t%s [OPTIONS]\n", program_name);
00066
00067     fprintf(stderr, "OPTIONS\n\n");
00068     fprintf(stderr, "\t--help\n");
00069     fprintf(stderr, "\t\tShow this help screen and exit.\n\n");
00070     fprintf(stderr, "\t--minor NUM\n");
00071     fprintf(stderr, "\t\tSpecify the minor number (>= 0) of the board to open. When not
specified,\n");
00072     fprintf(stderr, "\t\tthe device file with minor 0 is opened.\n");
00073     fprintf(stderr, "\n");
00074     exit(EXIT_FAILURE);
00075 }
00076
00104
00105 int main(int argument_count, char **arguments)
00106 {
00107
00108     unsigned long int minor = 0;
00109     int result;
00110     int fb_num = 0;
00111     struct DM35424_Board_Descriptor *board;
00112     struct DM35424_Function_Block my_func_block;
00113     char errMsg[300];
00114     uint8_t rev_num;
00115     uint32_t pdp_num, fpga_num;
00116     int help_option_given = 0;
00117
00118     int status;
00119
00120     char *invalid_char_p;
00121     struct option options[] = {
00122         {"help", 0, 0, 1},
00123         {"minor", 1, 0, 2},
00124         {0, 0, 0, 0}
00125     };
00126
00127     program_name = arguments[0];
00128
00129     // Show usage, parse arguments
00130     while (1) {
00131         /*
00132          * Parse the next command line option and any arguments it may require
00133          */
00134         status = getopt_long(argument_count,
00135                             arguments, "", options, NULL);
00136
00137         /*
00138          * If getopt_long() returned -1, then all options have been processed
00139          */
00140         if (status == -1) {
00141             break;
00142         }
00143
00144         /*
00145          * Figure out what getopt_long() found
00146          */
00147         switch (status) {
00148
00149             /*****

```

```

00150             User entered '--help'
00151             ##### */
00152             case HELP_OPTION:
00153                 help_option_given = 0xFF;
00154                 break;
00155
00156             /*#####
00157             User entered '--minor'
00158             ##### */
00159             case MINOR_OPTION:
00160                 /*
00161                  * Convert option argument string to unsigned long integer
00162                  */
00163                 errno = 0;
00164                 minor = strtoul(optarg, &invalid_char_p, 10);
00165
00166                 /*
00167                  * Catch unsigned long int overflow
00168                  */
00169                 if ((minor == ULONG_MAX)
00170                     && (errno == ERANGE)) {
00171                     error(0, 0,
00172                         "ERROR: Device minor number caused numeric overflow");
00173                     usage();
00174                 }
00175
00176                 /*
00177                  * Catch argument strings with valid decimal prefixes, for
00178                  * example "1q", and argument strings which cannot be converted,
00179                  * for example "abc1"
00180                  */
00181                 if ((*invalid_char_p != '\0')
00182                     || (invalid_char_p == optarg)) {
00183                     error(0, 0,
00184                         "ERROR: Non-decimal device minor number");
00185                     usage();
00186                 }
00187
00188                 break;
00189
00190             /*#####
00191             User entered unsupported option
00192             ##### */
00193             case '?':
00194                 usage();
00195                 break;
00196
00197             /*#####
00198             getopt_long() returned unexpected value
00199             ##### */
00200             default:
00201                 error(EXIT_FAILURE,
00202                     0,
00203                     "ERROR: getopt_long() returned unexpected value %#x",
00204                     status);
00205                 break;
00206         }
00207     }
00208
00209     /*
00210     * Recognize '--help' option before any others
00211     */
00212
00213     if (help_option_given) {
00214         usage();
00215     }
00216
00217     printf("Opening board....");
00218     result = DM35424_Board_Open(minor, &board);
00219
00220     DM35424_Check_Result(result, "Could not open board");
00221
00222     printf("success.\nResetting board....");
00223     result = DM35424_Gbc_Board_Reset(board);
00224
00225     DM35424_Check_Result(result, "Could not reset board");
00226     printf("success.\n\n");
00227
00228     result = DM35424_Gbc_Get_Revision(board, &rev_num);
00229     DM35424_Check_Result(result, "Error getting board revision number");
00230
00231     result = DM35424_Gbc_Get_Fpga_Build(board,
00232                                         &fpga_num);
00233     DM35424_Check_Result(result, "Error getting FPGA build number.");
00234
00235     result = DM35424_Gbc_Get_Pdp_Number(board,
00236                                         &pdp_num);

```

```

00237     DM35424_Check_Result(result, "Error getting PDP number.");
00238
00239     printf("FPGA Build: %u\n", fpga_num);
00240     printf("PDP Number: %u rev %c\n\n", pdp_num, 'A' + (rev_num - 1));
00241
00242     printf
00243         ("\nListing Function Blocks\n=====\\n");
00244
00245     fb_num = 0;
00246     while ((fb_num < DM35424_MAX_FB) && (errno != ERANGE)) {
00247         result = DM35424_Function_Block_Open(board,
00248                                             fb_num, &my_func_block);
00249
00250         /*
00251          * Because the GBC size is allowed to be less than what it
00252          * would take to define all possible function blocks, we
00253          * need to capture that error case and not report it.
00254          * It just means we're done finding function blocks.
00255          */
00256         if (errno != ERANGE) {
00257
00258             sprintf(errMsg, "Could not open function block %d.",
00259                     fb_num);
00259             DM35424_Check_Result(result, errMsg);
00260
00261             switch (my_func_block.type) {
00262
00263                 case DM35424_FUNC_BLOCK_ADC:
00264                     printf
00265                         (" FB%d: ADC (Type 0x%x Subtype %d), with %d DMA Channels (%d
00266 buffers each)\n",
00267                         my_func_block.fb_num,
00268                         my_func_block.type,
00269                         my_func_block.sub_type,
00270                         my_func_block.num_dma_channels,
00271                         my_func_block.num_dma_buffers);
00272                     break;
00273                 case DM35424_FUNC_BLOCK_DAC:
00274                     printf
00275                         (" FB%d: DAC (Type 0x%x Subtype %d), with %d DMA Channels (%d
00276 buffers each)\n",
00277                         my_func_block.fb_num,
00278                         my_func_block.type,
00279                         my_func_block.sub_type,
00280                         my_func_block.num_dma_channels,
00281                         my_func_block.num_dma_buffers);
00282                     break;
00283                 case DM35424_FUNC_BLOCK_DIO:
00284                     printf
00285                         (" FB%d: DIO (Type 0x%x Subtype %d), with %d DMA Channels (%d
00286 buffers each)\n",
00287                         my_func_block.fb_num,
00288                         my_func_block.type,
00289                         my_func_block.sub_type,
00290                         my_func_block.num_dma_channels,
00291                         my_func_block.num_dma_buffers);
00292                     break;
00293                 case DM35424_FUNC_BLOCK_REF_ADJUST:
00294                     printf
00295                         (" FB%d: Ref. Adj. Module (Type 0x%x Subtype %d), with %d DMA
00296 Channels (%d buffers each)\n",
00297                         my_func_block.fb_num,
00298                         my_func_block.type,
00299                         my_func_block.sub_type,
00300                         my_func_block.num_dma_channels,
00301                         my_func_block.num_dma_buffers);
00302                     break;
00303                 case DM35424_FUNC_BLOCK_TEMPERATURE_SENSOR:
00304                     printf
00305                         (" FB%d: Temp. Sensor Module (Type 0x%x Subtype %d), with %d DMA
00306 Channels (%d buffers each)\n",
00307                         my_func_block.fb_num,
00308                         my_func_block.type,
00309                         my_func_block.sub_type,
00310                         my_func_block.num_dma_channels,
00311                         my_func_block.num_dma_buffers);
00312                     break;
00313                 case DM35424_FUNC_BLOCK_INVALID:
00314                     // Do nothing
00315                     break;
00316                 default:
00317                     printf(" FB%d: **Unknown module type (0x%x)\n",
00318                         my_func_block.fb_num,
00319                         my_func_block.type);
00320                     break;
00321             }
00322         }
00323         fb_num++;
00324     }

```

```

00319             fb_num++;
00320         }
00321     }
00322 }
00323
00324     result = DM35424_Gbc_Board_Reset(board);
00325     printf("\nClosing Board\n");
00326     result = DM35424_Board_Close(board);
00327
00328     DM35424_Check_Result(result, "Error closing board.");
00329     printf("Example program successfully completed.\n");
00330     return 0;
00331 }
00332 }

```

6.23 examples/dm35424_ref_adjust.c File Reference

Example program which demonstrates the reference voltage adjustment function blocks.

```

#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <unistd.h>
#include <limits.h>
#include <signal.h>
#include <string.h>
#include <getopt.h>
#include <termios.h>
#include <time.h>
#include <sys/time.h>
#include "dm35424_gbc_library.h"
#include "dm35424_adc_library.h"
#include "dm35424_dac_library.h"
#include "dm35424_dma_library.h"
#include "dm35424_ioctl.h"
#include "dm35424_examples.h"
#include "dm35424_util_library.h"
#include "dm35424_board_access.h"
#include "dm35424_types.h"
#include "dm35424.h"
#include "dm35424_ref_adjust_library.h"

```

Macros

- #define `DEFAULT_RATE` 80000
- #define `MAX_REF_ADJUST` 0xFF

Functions

- static void `usage` (void)
Print information on stderr about how the program is to be used. After doing so, the program is exited.
- int `keyboard_hit` ()
Detect a keypress by selecting on STDIO with 0 wait time.
- int `getch` ()

Get the key that was pressed.

- static void [sigint_handler](#) (int signal_number)
Signal handler for SIGINT Control-C keyboard interrupt.
- int [main](#) (int argument_count, char **arguments)
The main program.

Variables

- static char * [program_name](#)
- volatile int [exit_program](#) = 0

6.23.1 Detailed Description

Example program which demonstrates the reference voltage adjustment function blocks.

This example program demonstrates using the reference adjustment value to adjust the value reported by the ADC or sent by the DAC. The example allows for setting an adjustment value, and then writing that value to the appropriate memory location.

The reference voltage source should be attached to the differential inputs of ADC0 or ADC1 (depending on which reference adjustment you are using). The voltage reading from the ADC will be displayed on the screen, allowing you to view the effect reference value changes has on the measured voltage.

For purposes of running the example, and for convenience, the onboard DACs will be set to supply a value of 2 volts. You can then loopback the DAC outputs into the ADC inputs, as described below.

Note that the supplied DAC outputs should not be considered a reference voltage source for the purposes of calibrating the board.

WARNING: This example will allow you to change the preset calibration values on the board. Do not do so unless you understand the risks of permanently changing that value.

Setup: Attach a reference voltage source to the ADC Channel 0- and Channel 0+ pins. If using the onboard DACs, connect DAC Channel 0 to ADC Channel 0+ and DAC Channel 1 to ADC Channel 0-.

This file and its contents are copyright (C) RTD Embedded Technologies, Inc. All Rights Reserved.

This software is licensed as described in the RTD End-User Software License Agreement. For a copy of this agreement, refer to the file LICENSE.TXT (which should be included with this software) or contact RTD Embedded Technologies, Inc.

Id

[dm35424_ref_adjust.c](#) 141531 2024-03-06 21:05:25Z lfrankenfield

Definition in file [dm35424_ref_adjust.c](#).

6.23.2 Macro Definition Documentation

6.23.2.1 DEFAULT_RATE

```
#define DEFAULT_RATE 80000
```

Sampling rate to use (Hz)

Definition at line 85 of file [dm35424_ref_adjust.c](#).

6.23.2.2 MAX_REF_ADJUST

```
#define MAX_REF_ADJUST 0xFF
```

Maximum REF adjust to allow

Definition at line 90 of file [dm35424_ref_adjust.c](#).

6.23.3 Function Documentation

6.23.3.1 getch()

```
int getch ()
```

Get the key that was pressed.

Return values

<i>None.</i>	
--------------	--

Definition at line 158 of file [dm35424_ref_adjust.c](#).

Referenced by [main\(\)](#).

6.23.3.2 keyboard_hit()

```
int keyboard_hit ()
```

Detect a keypress by selecting on STDIO with 0 wait time.

Return values

<i>None.</i>	
--------------	--

Definition at line 136 of file [dm35424_ref_adjust.c](#).

Referenced by [main\(\)](#).

6.23.3.3 main()

```
int main (  
    int argument_count,  
    char ** arguments)
```

The main program.

Parameters

<i>argument_count</i>	
-----------------------	--

Number of args passed on the command line, including the executable name

Parameters

<i>arguments</i>	
------------------	--

Pointer to array of character strings, which are the args themselves.

Return values

0	
---	--

Success

Return values

Non-Zero	
----------	--

Failure.

Definition at line 219 of file [dm35424_ref_adjust.c](#).

References [ADC_0](#), [board](#), [CHANNEL_0](#), [CHANNEL_1](#), [DAC_0](#), [DEFAULT_RATE](#), [DM35424_Adc_Ad_Config_Set_Mode\(\)](#), [DM35424_Adc_Channel_Get_Last_Sample\(\)](#), [DM35424_Adc_Channel_Setup\(\)](#), [DM35424_Adc_Initialize\(\)](#), [DM35424_ADC_INPUT_DIFFERENTIAL](#), [DM35424_ADC_MODE_CONFIG_HIGH_SPEED](#), [DM35424_Adc_Open\(\)](#), [DM35424_Adc_Reset\(\)](#), [DM35424_ADC_RNG_BIPOLAR_2_5V](#), [DM35424_Adc_Sample_To_Volts\(\)](#), [DM35424_Adc_Set_Clock_Src\(\)](#), [DM35424_Adc_Set_Post_Stop_Samples\(\)](#), [DM35424_Adc_Set_Pre_Trigger_Samples\(\)](#), [DM35424_Adc_Set_Sample_Rate\(\)](#), [DM35424_Adc_Set_Start_Trigger\(\)](#), [DM35424_Adc_Set_Stop_Trigger\(\)](#), [DM35424_Adc_Start\(\)](#), [DM35424_Adc_Uninitialize\(\)](#), [DM35424_Board_Close\(\)](#), [DM35424_Board_Open\(\)](#), [DM35424_Check_Result\(\)](#), [DM35424_CLK_SRC_IMMEDIATE](#), [DM35424_CLK_SRC_NEVER](#), [DM35424_Dac_Open\(\)](#), [DM35424_Dac_Reset\(\)](#), [DM35424_Dac_Set_Last_Conversion\(\)](#), [DM35424_Dac_Volts_To_Conv\(\)](#), [DM35424_Gbc_Board_Reset\(\)](#), [DM35424_Micro_Sleep\(\)](#), [DM35424_Ref_Adjust_Open\(\)](#), [DM35424_Ref_Adjust_Write_Adc_To_NonVolatile\(\)](#), [DM35424_Ref_Adjust_Write_Adc_To_Volatile\(\)](#), [DM35424_Ref_Adjust_Write_Dac_To_Volatile\(\)](#), [exit_program](#), [getch\(\)](#), [HELP_OPTION](#), [keyboard_hit\(\)](#), [MINOR_OPTION](#), [my_adc](#), [program_name](#), [REF_0](#), [sigint_handler\(\)](#), and [usage\(\)](#).

6.23.3.4 sigint_handler()

```
void sigint_handler (
    int signal_number) [static]
```

Signal handler for SIGINT Control-C keyboard interrupt.

Parameters

<i>signal_number</i>	
----------------------	--

Signal number passed in from the kernel.

Warning

One must be extremely careful about what functions are called from a signal handler.

Definition at line 186 of file [dm35424_ref_adjust.c](#).

References [exit_program](#).

6.23.4 Variable Documentation

6.23.4.1 exit_program

```
volatile int exit_program = 0
```

Boolean indicating whether or not to exit the program.

Definition at line 100 of file [dm35424_ref_adjust.c](#).

6.23.4.2 program_name

```
char* program_name [static]
```

Name of the program as invoked on the command line

Definition at line 95 of file [dm35424_ref_adjust.c](#).

6.24 dm35424_ref_adjust.c

[Go to the documentation of this file.](#)

```

00001
00053
00054 #include <stdio.h>
00055 #include <stddef.h>
00056 #include <stdlib.h>
00057 #include <errno.h>
00058 #include <error.h>
00059 #include <unistd.h>
00060 #include <limits.h>
00061 #include <signal.h>
00062 #include <string.h>
00063 #include <getopt.h>
00064 #include <signal.h>
00065 #include <termios.h>
00066 #include <time.h>
00067 #include <sys/time.h>
00068
00069
00070 #include "dm35424_gbc_library.h"
00071 #include "dm35424_adc_library.h"
00072 #include "dm35424_dac_library.h"
00073 #include "dm35424_dma_library.h"
00074 #include "dm35424_ioctl.h"
00075 #include "dm35424_examples.h"
00076 #include "dm35424_util_library.h"
00077 #include "dm35424_board_access.h"
00078 #include "dm35424_types.h"
00079 #include "dm35424.h"
00080 #include "dm35424_ref_adjust_library.h"
00081
00085 #define DEFAULT_RATE      80000
00086
00090 #define MAX_REF_ADJUST    0xFF
00091
00095 static char *program_name;
00096
00100 volatile int exit_program = 0;
00101
00109
00110 static void usage(void)
00111 {
00112     fprintf(stderr, "\n");
00113     fprintf(stderr, "NAME\n\t%s\n", program_name);
00114     fprintf(stderr, "USAGE\n\t%s [OPTIONS]\n", program_name);
00115
00116     fprintf(stderr, "OPTIONS\n");
00117     fprintf(stderr, "\t--help\n");
00118     fprintf(stderr, "\t\tShow this help screen and exit.\n");
00119     fprintf(stderr, "\t--minor NUM\n");
00120     fprintf(stderr, "\t\tSpecify the minor number (>= 0) of the board to open.  When not
specified,\n");
00121     fprintf(stderr, "\t\tthe device file with minor 0 is opened.\n");
00122
00123     fprintf(stderr, "\n");
00124     exit(EXIT_FAILURE);
00125 }
00126
00136 int keyboard_hit()
00137 {
00138
00139     struct timeval tv = { 0L, 0L };
00140     fd_set fds;
00141
00142     FD_ZERO(&fds);
00143     FD_SET(0, &fds);
00144
00145     return select(1, &fds, NULL, NULL, &tv);
00146 }
00147
00148
00158 int getch()
00159 {
00160     int data;
00161     unsigned char c;
00162
00163     if ((data = read(0, &c, sizeof(c))) < 0) {
00164         return data;
00165     } else {
00166         return c;
00167     }
00168 }
00169

```

```

00185
00186 static void sigint_handler(int signal_number)
00187 {
00188     exit_program = 0xff;
00189 }
00190
00218
00219 int main(int argument_count, char **arguments)
00220 {
00221     struct DM35424_Board_Descriptor *board;
00222     struct DM35424_Function_Block my_adc;
00223     struct DM35424_Function_Block my_dac;
00224     struct DM35424_Function_Block my_ref;
00225
00226     unsigned long int minor = 0;
00227     int result;
00228     int my_value;
00229     char last_action[100];
00230
00231     unsigned int rate = DEFAULT_RATE;
00232     unsigned int actual_rate;
00233
00234     int help_option_given = 0;
00235     int status;
00236     struct sigaction signal_action;
00237
00238     uint8_t ref_value = 0;
00239     float volts = 0;
00240
00241     int16_t max_value, min_value;
00242     struct termios old_tio, new_tio;
00243     unsigned char keypress = '0';
00244
00245     char *invalid_char_p;
00246     struct option options[] = {
00247         {"help", 0, 0, HELP_OPTION},
00248         {"minor", 1, 0, MINOR_OPTION},
00249         {0, 0, 0, 0}
00250     };
00251
00252     program_name = arguments[0];
00253
00254     // Show usage, parse arguments
00255     while (1) {
00256         /*
00257          * Parse the next command line option and any arguments it may require
00258          */
00259         status = getopt_long(argument_count,
00260                             arguments, "", options, NULL);
00261
00262         /*
00263          * If getopt_long() returned -1, then all options have been processed
00264          */
00265         if (status == -1) {
00266             break;
00267         }
00268
00269         /*
00270          * Figure out what getopt_long() found
00271          */
00272         switch (status) {
00273
00274             /*#####
00275              * User entered '--help'
00276              *##### */
00277             case HELP_OPTION:
00278                 help_option_given = 0xFF;
00279                 break;
00280
00281             /*#####
00282              * User entered '--minor'
00283              *##### */
00284             case MINOR_OPTION:
00285                 /*
00286                  * Convert option argument string to unsigned long integer
00287                  */
00288                 errno = 0;
00289                 minor = strtoul(optarg, &invalid_char_p, 10);
00290
00291                 /*
00292                  * Catch unsigned long int overflow
00293                  */
00294                 if ((minor == ULONG_MAX)
00295                     && (errno == ERANGE)) {
00296                     error(0, 0,
00297                          "ERROR: Device minor number caused numeric overflow");
00298                     usage();

```

```

00299         }
00300
00301         /*
00302          * Catch argument strings with valid decimal prefixes, for
00303          * example "1q", and argument strings which cannot be converted,
00304          * for example "abc1"
00305          */
00306         if ((*invalid_char_p != '\0')
00307             || (invalid_char_p == optarg)) {
00308             error(0, 0,
00309                  "ERROR: Non-decimal device minor number");
00310             usage();
00311         }
00312
00313         break;
00314
00315         /*#####
00316          User entered unsupported option
00317          ##### */
00318         case '?':
00319             usage();
00320             break;
00321
00322         /*#####
00323          getopt_long() returned unexpected value
00324          ##### */
00325         default:
00326             error(EXIT_FAILURE,
00327                  0,
00328                  "ERROR: getopt_long() returned unexpected value %x",
00329                  status);
00330             break;
00331     }
00332 }
00333
00334 /*
00335  * Recognize '--help' option before any others
00336  */
00337
00338 if (help_option_given) {
00339     usage();
00340 }
00341
00342 signal_action.sa_handler = sigint_handler;
00343 sigfillset(&(signal_action.sa_mask));
00344 signal_action.sa_flags = 0;
00345
00346 if (sigaction(SIGINT, &signal_action, NULL) < 0) {
00347     error(EXIT_FAILURE, errno, "ERROR: sigaction() FAILED");
00348 }
00349
00350 printf("\n\n");
00351 printf("*****\n");
00352 printf("*                WARNING                *\n");
00353 printf("*****\n");
00354 printf("* This example program demonstrates how to change the calibration *\n");
00355 printf("* values of the board. Specifically, writing to volatile memory will *\n");
00356 printf("* temporarily change it, and writing to non-volatile will permanently *\n");
00357 printf("* change it. Do not proceed unless you understand the risks of *\n");
00358 printf("* changing the calibration values. *\n");
00359 printf("*****\n");
00360 printf("\n\nPress Enter to continue, or Ctrl-C to abort.\n\n");
00361
00362 keypress = getchar();
00363
00364 if (exit_program) {
00365     return 0;
00366 }
00367
00368 printf("\n\nApply reference voltage to ADC differential inputs (or attach \n");
00369 printf("loopbacks) if adjusting ADC reference, or attach voltage measuring\n");
00370 printf("device to DAC output, if adjusting DAC reference. Press Enter to\n");
00371 printf("continue.\n");
00372
00373 keypress = getchar();
00374
00375 tcgetattr(STDIN_FILENO, &old_tio);
00376
00377 new_tio = old_tio;
00378
00379 new_tio.c_lflag &= ~ICANON;
00380 new_tio.c_lflag &= ~ECHO;
00381 new_tio.c_lflag &= ~ISIG;
00382
00383 tcsetattr(STDIN_FILENO, TCSANOW, &new_tio);
00384
00385 printf("Opening board....");

```

```
00386     result = DM35424_Board_Open(minor, &board);
00387
00388     DM35424_Check_Result(result, "Could not open board");
00389     printf("success.\nResetting board.....");
00390     result = DM35424_Gbc_Board_Reset(board);
00391
00392     DM35424_Check_Result(result, "Could not reset board");
00393     printf("success.\n");
00394
00395     printf("success.\nOpening DAC.....");
00396
00397     result = DM35424_Dac_Open(board, DAC_0, &my_dac);
00398
00399     DM35424_Check_Result(result, "Could not open DAC");
00400
00401     printf("Found DAC%u, with %d DMA channels (%d buffers each)\n",
00402           DAC_0, my_dac.num_dma_channels,
00403           my_dac.num_dma_buffers);
00404
00405     result = DM35424_Dac_Reset(board,
00406                               &my_dac);
00407
00408     DM35424_Check_Result(result, "Error resetting DAC.");
00409
00410
00411
00412
00413     result = DM35424_Dac_Volts_To_Conv(3.5f, &max_value);
00414
00415     DM35424_Check_Result(result,
00416                           "Error converting value to conversion counts.");
00417
00418     result = DM35424_Dac_Set_Last_Conversion(board,
00419                                              &my_dac,
00420                                              CHANNEL_0,
00421                                              0,
00422                                              max_value);
00423
00424     DM35424_Check_Result(result, "Error setting last conversion for DAC0 Chan 0.");
00425
00426     result = DM35424_Dac_Volts_To_Conv(1.5f, &min_value);
00427
00428     DM35424_Check_Result(result,
00429                           "Error converting value to conversion counts.");
00430
00431
00432     result = DM35424_Dac_Set_Last_Conversion(board,
00433                                              &my_dac,
00434                                              CHANNEL_1,
00435                                              0,
00436                                              min_value);
00437
00438     DM35424_Check_Result(result, "Error setting last conversion for DAC0 Chan 1.");
00439
00440     printf("Opening Reference Adjustment....\n");
00441
00442     result = DM35424_Ref_Adjust_Open(board,
00443                                     REF_0,
00444                                     &my_ref);
00445
00446     DM35424_Check_Result(result, "Error opening reference adjustment.");
00447
00448     printf("Opening ADC.....\n");
00449
00450     result = DM35424_Adc_Open(board, ADC_0, &my_adc);
00451
00452     DM35424_Check_Result(result, "Could not open ADC");
00453
00454     printf("Found ADC%d, with %d DMA channels (%d buffers each)\n",
00455           ADC_0, my_adc.num_dma_channels, my_adc.num_dma_buffers);
00456
00457     result = DM35424_Adc_Uninitialize(board, &my_adc);
00458
00459     DM35424_Check_Result(result, "Error setting ADC to Uninitialized.");
00460
00461     result = DM35424_Adc_Channel_Setup(board,
00462                                       &my_adc,
00463                                       CHANNEL_0,
00464                                       DM35424_ADC_RNG_BIPOLAR_2_5V,
00465                                       DM35424_ADC_INPUT_DIFFERENTIAL);
00466
00467     DM35424_Check_Result(result, "Error setting up channel.");
00468
00469
00470     result = DM35424_Adc_Ad_Config_Set_Mode(board,
00471                                              &my_adc,
00472                                              DM35424_ADC_MODE_CONFIG_HIGH_SPEED);
```



```

00559
00560         if (keypress == 'i') {
00561
00562             if (ref_value < MAX_REF_ADJUST) {
00563                 ref_value ++;
00564                 strcpy(last_action, "Reference value increased.");
00565             }
00566         }
00567
00568         if (keypress == 'd') {
00569             if (ref_value > 0) {
00570                 ref_value --;
00571                 strcpy(last_action, "Reference value decreased.");
00572             }
00573         }
00574
00575         if (keypress == 'v') {
00576             result = DM35424_Ref_Adjust_Write_Adc_To_Volatile(
00577                 board,
00578                 &my_ref,
00579                 ref_value);
00580
00581             DM35424_Check_Result(result, "Error writing reference value to ADC volatile
00582 memory.");
00583             strcpy(last_action, "Reference value written to ADC volatile memory.");
00584         }
00585
00586         if (keypress == 'n') {
00587             result = DM35424_Ref_Adjust_Write_Adc_To_NonVolatile(
00588                 board,
00589                 &my_ref,
00590                 ref_value);
00591
00592             DM35424_Check_Result(result, "Error writing reference value to ADC
00593 non-volatile memory.");
00594             strcpy(last_action, "Reference value written to ADC non-volatile memory.");
00595         }
00596
00597         if (keypress == 'c') {
00598             result = DM35424_Ref_Adjust_Write_Dac_To_Volatile(
00599                 board,
00600                 &my_ref,
00601                 ref_value);
00602
00603             DM35424_Check_Result(result, "Error writing reference value to DAC volatile
00604 memory.");
00605             strcpy(last_action, "Reference value written to DAC volatile memory.");
00606         }
00607
00608         if (keypress == 'm') {
00609             result = DM35424_Ref_Adjust_Write_Dac_To_NonVolatile(
00610                 board,
00611                 &my_ref,
00612                 ref_value);
00613
00614             DM35424_Check_Result(result, "Error writing reference value to DAC
00615 non-volatile memory.");
00616             strcpy(last_action, "Reference value written to DAC non-volatile memory.");
00617         }
00618     }
00619     tcsetattr(STDIN_FILENO, TCSANOW, &old_tio);
00620
00621     printf("\n\nStopping Adc %d.....", ADC_0);
00622
00623     result = DM35424_Adc_Reset(board, &my_adc);
00624
00625     DM35424_Check_Result(result, "Error starting ADC");
00626
00627     printf("success.\n");
00628
00629     result = DM35424_Gbc_Board_Reset(board);
00630     printf("Closing Board\n");
00631     result = DM35424_Board_Close(board);
00632
00633     DM35424_Check_Result(result, "Error closing board.");
00634     printf("Example program successfully completed.\n");
00635     return 0;
00636
00637
00638 }

```

6.25 examples/dm35424_temperature.c File Reference

Example program for read the temperature sensor.

```
#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <unistd.h>
#include <limits.h>
#include <signal.h>
#include <getopt.h>
#include <string.h>
#include "dm35424_gbc_library.h"
#include "dm35424_temperature_library.h"
#include "dm35424_ioctl.h"
#include "dm35424_examples.h"
#include "dm35424_util_library.h"
```

Macros

- #define `TEMP_FB_TO_OPEN` 0

Functions

- static void `usage` (void)
Print information on stderr about how the program is to be used. After doing so, the program is exited.
- static void `sigint_handler` (int signal_number)
Signal handler for SIGINT Control-C keyboard interrupt.
- int `main` (int argument_count, char **arguments)
The main program.

Variables

- static char * `program_name`
- volatile int `exit_program` = 0

6.25.1 Detailed Description

Example program for read the temperature sensor.

The program will read the temperature value from the temperature function block and print it to the screen. It will also print the maximum and minimum temperature received from the board for the time that the example program has been running.

The program will continue to run until CTRL-C is pressed.

This file and its contents are copyright (C) RTD Embedded Technologies, Inc. All Rights Reserved.

This software is licensed as described in the RTD End-User Software License Agreement. For a copy of this agreement, refer to the file LICENSE.TXT (which should be included with this software) or contact RTD Embedded Technologies, Inc.

Id

[dm35424_temperature.c](#) 141531 2024-03-06 21:05:25Z lfrankenfield

Definition in file [dm35424_temperature.c](#).

6.25.2 Macro Definition Documentation

6.25.2.1 TEMP_FB_TO_OPEN

```
#define TEMP_FB_TO_OPEN 0
```

Which temperature function block to open on this board (there is only one).

Definition at line 57 of file [dm35424_temperature.c](#).

Referenced by [main\(\)](#).

6.25.3 Function Documentation

6.25.3.1 main()

```
int main (  
    int argument_count,  
    char ** arguments)
```

The main program.

Parameters

<i>argument_count</i>	
-----------------------	--

Number of args passed on the command line, including the executable name

Parameters

<i>arguments</i>	
------------------	--

Pointer to array of character strings, which are the args themselves.

Return values

0	
---	--

Success

Return values

<i>Non-Zero</i>	
-----------------	--

Failure.

Definition at line 140 of file [dm35424_temperature.c](#).

References [board](#), [DM35424_Board_Close\(\)](#), [DM35424_Board_Open\(\)](#), [DM35424_Check_Result\(\)](#), [DM35424_Gbc_Board_Reset\(\)](#), [DM35424_Micro_Sleep\(\)](#), [DM35424_Temperature_Open\(\)](#), [DM35424_Temperature_Read\(\)](#), [exit_program](#), [HELP_OPTION](#), [MINOR_OPTION](#), [program_name](#), [sigint_handler\(\)](#), [TEMP_FB_TO_OPEN](#), and [usage\(\)](#).

6.25.3.2 sigint_handler()

```
void sigint_handler (
    int signal_number)    [static]
```

Signal handler for SIGINT Control-C keyboard interrupt.

Parameters

<i>signal_number</i>	
----------------------	--

Signal number passed in from the kernel.

Warning

One must be extremely careful about what functions are called from a signal handler.

Definition at line 107 of file [dm35424_temperature.c](#).

References [exit_program](#).

6.25.4 Variable Documentation**6.25.4.1 exit_program**

```
volatile int exit_program = 0
```

Boolean indicating whether or not to exit the program.

Definition at line 67 of file [dm35424_temperature.c](#).

6.25.4.2 program_name

```
char* program_name    [static]
```

Name of the program as invoked on the command line

Definition at line 62 of file [dm35424_temperature.c](#).

6.26 dm35424_temperature.c

[Go to the documentation of this file.](#)

```

00001
00033
00034 #include <stdio.h>
00035 #include <stddef.h>
00036 #include <stdlib.h>
00037 #include <errno.h>
00038 #include <error.h>
00039 #include <unistd.h>
00040 #include <limits.h>
00041 #include <signal.h>
00042 #include <getopt.h>
00043 #include <signal.h>
00044 #include <string.h>
00045
00046 #include "dm35424_gbc_library.h"
00047 #include "dm35424_temperature_library.h"
00048 #include "dm35424_ioctl.h"
00049 #include "dm35424_examples.h"
00050 #include "dm35424_util_library.h"
00051
00052
00057 #define          TEMP_FB_TO_OPEN          0
00058
00062 static char *program_name;
00063
00067 volatile int exit_program = 0;
00068
00076
00077 static void usage(void)
00078 {
00079     fprintf(stderr, "\n");
00080     fprintf(stderr, "NAME\n\t%s\n", program_name);
00081     fprintf(stderr, "USAGE\n\t%s [OPTIONS]\n", program_name);
00082
00083     fprintf(stderr, "OPTIONS\n\n");
00084     fprintf(stderr, "\t--help\n");
00085     fprintf(stderr, "\t\tShow this help screen and exit.\n\n");
00086     fprintf(stderr, "\t--minor NUM\n");
00087     fprintf(stderr, "\t\tSpecify the minor number (>= 0) of the board to open.  When not
specified,\n");
00088     fprintf(stderr, "\t\tthe device file with minor 0 is opened.\n");
00089     exit(EXIT_FAILURE);
00090 }
00091
00107 static void sigint_handler(int signal_number)
00108 {
00109     exit_program = 0xff;
00110 }
00111
00112
00140 int main(int argument_count, char **arguments)
00141 {
00142     struct DM35424_Board_Descriptor *board;
00143     struct DM35424_Function_Block temp_fb;
00144     unsigned long int minor = 0;
00145     int result;
00146
00147     int help_option_given = 0;
00148
00149     int status;
00150     struct sigaction signal_action;
00151
00152     float temperature;
00153     float max_temp = 0;
00154     float min_temp = 1000.0f;
00155     char *invalid_char_p;
00156     struct option options[] = {
00157         {"help", 0, 0, HELP_OPTION},
00158         {"minor", 1, 0, MINOR_OPTION},
00159         {0, 0, 0, 0}
00160     };
00161
00162     program_name = arguments[0];
00163
00164     // Show usage, parse arguments
00165     while (1) {
00166         /*
00167          * Parse the next command line option and any arguments it may require
00168          */
00169         status = getopt_long(argument_count,
00170                             arguments, "", options, NULL);
00171

```

```

00172         /*
00173          * If getopt_long() returned -1, then all options have been processed
00174          */
00175         if (status == -1) {
00176             break;
00177         }
00178     /*
00179      * Figure out what getopt_long() found
00180      */
00181     switch (status) {
00182     /*#####
00183      * User entered '--help'
00184      *##### */
00185     case HELP_OPTION:
00186         help_option_given = 0xFF;
00187         break;
00188     /*#####
00189      * User entered '--minor'
00190      *##### */
00191     case MINOR_OPTION:
00192         /*
00193          * Convert option argument string to unsigned long integer
00194          */
00195         errno = 0;
00196         minor = strtoul(optarg, &invalid_char_p, 10);
00197
00198         /*
00199          * Catch unsigned long int overflow
00200          */
00201         if ((minor == ULONG_MAX)
00202             && (errno == ERANGE)) {
00203             error(0, 0,
00204                  "ERROR: Device minor number caused numeric overflow");
00205             usage();
00206         }
00207
00208         /*
00209          * Catch argument strings with valid decimal prefixes, for
00210          * example "1k", and argument strings which cannot be converted,
00211          * for example "abcd"
00212          */
00213         if ((*invalid_char_p != '\0')
00214             || (invalid_char_p == optarg)) {
00215             error(0, 0,
00216                  "ERROR: Non-decimal device minor number");
00217             usage();
00218         }
00219         break;
00220     /*#####
00221      * User entered unsupported option
00222      *##### */
00223     case '?':
00224         usage();
00225         break;
00226     /*#####
00227      * getopt_long() returned unexpected value
00228      *##### */
00229     default:
00230         error(EXIT_FAILURE,
00231              0,
00232              "ERROR: getopt_long() returned unexpected value %i",
00233              status);
00234         break;
00235     }
00236 }
00237
00238 /*
00239  * Recognize '--help' option before any others
00240  */
00241
00242 if (help_option_given) {
00243     usage();
00244 }
00245
00246 signal_action.sa_handler = sigint_handler;
00247 sigfillset(&(signal_action.sa_mask));
00248 signal_action.sa_flags = 0;
00249
00250 if (sigaction(SIGINT, &signal_action, NULL) < 0) {
00251     error(EXIT_FAILURE, errno, "ERROR: sigaction() FAILED");
00252 }

```

```

00259
00260     printf("Opening board....");
00261     result = DM35424_Board_Open(minor, &board);
00262
00263     DM35424_Check_Result(result, "Could not open board");
00264     printf("success.\nResetting board....");
00265     result = DM35424_Gbc_Board_Reset(board);
00266
00267     DM35424_Check_Result(result, "Could not reset board");
00268     printf("success.\nOpening Temperature Function Block....");
00269
00270     result = DM35424_Temperature_Open(board,
00271                                         TEMP_FB_TO_OPEN,
00272                                         &temp_fb);
00273
00274     DM35424_Check_Result(result, "Could not open temperature function block.");
00275     printf("success.\n");
00276
00277     printf("\nPress Ctrl-C to exit.\n\n");
00278     printf("Temp (C)\t Min (C)\t Max (C)");
00279     printf("\n");
00280     printf("=====\t=====\t=====");
00281     printf("\n");
00282
00283     while (!exit_program) {
00284         result = DM35424_Temperature_Read(board, &temp_fb, &temperature);
00285         DM35424_Check_Result(result, "Could not read temperature");
00286
00287         if (temperature < min_temp) {
00288             min_temp = temperature;
00289         }
00290
00291         if (temperature > max_temp) {
00292             max_temp = temperature;
00293         }
00294
00295         fprintf(stdout, "    %3.2f\t    %3.2f\t    %3.2f\n", temperature,
00296                     min_temp, max_temp);
00297         fflush(stdout);
00298         DM35424_Micro_Sleep(500000);
00299     }
00300
00301     printf("\n\n");
00302
00303     result = DM35424_Gbc_Board_Reset(board);
00304     printf("Closing Board\n");
00305     result = DM35424_Board_Close(board);
00306
00307     DM35424_Check_Result(result, "Error closing board.");
00308     printf("Example program successfully completed.\n");
00309     return 0;
00310
00311 }

```

6.27 include/dm35424.h File Reference

Defines for the DM35424 (Device-specific values).

Macros

- **#define DM35424_PCI_VENDOR_ID** 0x1435
DM35424 PCI vendor ID.
- **#define DM35424_PCI_DEVICE_ID** 0x5424
DM35424 PCI device ID.
- **#define DM35424_NUM_ADC_ON_BOARD** 2
Number of ADC on the DM35424.
- **#define DM35424_NUM_DAC_ON_BOARD** 4
Number of DAC on the DM35424.
- **#define DM35424_NUM_ADC_DMA_CHANNELS** 8
Number of channels per ADC.

- **#define DM35424_NUM_ADC_DMA_BUFFERS 7**
Number of buffers per ADC DMA channel.
- **#define DM35424_NUM_DAC_DMA_CHANNELS 4**
Number of channels per DAC.
- **#define DM35424_NUM_DAC_DMA_BUFFERS 7**
Number of buffers per DAC DMA channel.
- **#define DM35424_DAC_FE_CONFIG_GAIN_MASK 0**
Bit Mask for the gain bits of the FE Config.
- **#define DM35424_FIFO_SAMPLE_SIZE 511**
Sample size of the FIFO.
- **#define DM35424_DAC_MAX_VALUE 32767**
Max value of the DAC.
- **#define DM35424_DAC_MIN_VALUE -32768**
Min value of the DAC.
- **#define DM35424_DAC_MAX_RATE 106000**
Max rate of the DAC.

6.27.1 Detailed Description

Defines for the DM35424 (Device-specific values).

```
-----
This file and its contents are copyright (C) RTD Embedded Technologies,
Inc. All Rights Reserved.
```

```
This software is licensed as described in the RTD End-User Software License
Agreement. For a copy of this agreement, refer to the file LICENSE.TXT
(which should be included with this software) or contact RTD Embedded
Technologies, Inc.
-----
```

Id

[dm35424.h](#) 90112 2015-07-15 20:05:32Z rgroner

Definition in file [dm35424.h](#).

6.28 dm35424.h

[Go to the documentation of this file.](#)

```
00001
00021
00022 #ifndef __DM35424_H__
00023 #define __DM35424_H__
00024
00029
00030 #define DM35424_PCI_VENDOR_ID    0x1435
00031
00036 #define DM35424_PCI_DEVICE_ID    0x5424
00037
00042 #define DM35424_NUM_ADC_ON_BOARD    2
00043
00048 #define DM35424_NUM_DAC_ON_BOARD    4
00049
00054 #define DM35424_NUM_ADC_DMA_CHANNELS    8
00055
```



```

00060 #define DM35424_NUM_ADC_DMA_BUFFERS          7
00061
00066 #define DM35424_NUM_DAC_DMA_CHANNELS          4
00067
00072 #define DM35424_NUM_DAC_DMA_BUFFERS          7
00073
00078 #define DM35424_DAC_FE_CONFIG_GAIN_MASK      0
00079
00084 #define DM35424_FIFO_SAMPLE_SIZE             511
00085
00090 #define DM35424_DAC_MAX_VALUE                 32767
00091
00096 #define DM35424_DAC_MIN_VALUE                -32768
00097
00102 #define DM35424_DAC_MAX_RATE                 106000
00103
00104 #endif

```

6.29 include/dm35424_adc_library.h File Reference

Definitions for the DM35424 ADC Library.

```
#include "dm35424_gbc_library.h"
```

Macros

- **#define DM35424_ADC_MODE_RESET** 0x00
Register value for ADC Mode Reset.
- **#define DM35424_ADC_MODE_PAUSE** 0x01
Register value for ADC Mode Pause.
- **#define DM35424_ADC_MODE_GO_SINGLE_SHOT** 0x02
Register value for ADC Mode Go (Single Shot).
- **#define DM35424_ADC_MODE_GO_REARM** 0x03
Register value for ADC Mode Go (Rearm after Stop).
- **#define DM35424_ADC_MODE_UNINITIALIZED** 0x04
Register value for ADC Mode Uninitialized.
- **#define DM35424_ADC_STAT_STOPPED** 0x00
Register value for ADC Status - Stopped.
- **#define DM35424_ADC_STAT_FILLING_PRE_TRIG_BUFF** 0x01
Register value for ADC Status - Filling Pre-Start Buffer.
- **#define DM35424_ADC_STAT_WAITING_START_TRIG** 0x02
Register value for ADC Status - Waiting for Start Trigger.
- **#define DM35424_ADC_STAT_SAMPLING** 0x03
Register value for ADC Status - Sampling Data.
- **#define DM35424_ADC_STAT_FILLING_POST_TRIG_BUFF** 0x04
Register value for ADC Status - Filling Post-Stop Buffer.
- **#define DM35424_ADC_STAT_WAIT_REARM** 0x05
Register value for ADC Status - Wait for Rearm.
- **#define DM35424_ADC_STAT_DONE** 0x07
Register value for ADC Status - Done.
- **#define DM35424_ADC_STAT_UNINITIALIZED** 0x08
Register value for ADC Status - Uninitialized.
- **#define DM35424_ADC_STAT_INITIALIZING** 0x09
Register value for ADC Status - Initializing.
- **#define DM35424_ADC_INT_SAMPLE_TAKEN_MASK** 0x01

- Register value for Interrupt Mask - Sample Taken.*

 - **#define DM35424_ADC_INT_CHAN_THRESHOLD_MASK** 0x02
- Register value for Interrupt Mask - Channel Threshold Exceeded.*

 - **#define DM35424_ADC_INT_PRE_BUFF_FULL_MASK** 0x04
- Register value for Interrupt Mask - Pre-Start Buffer Filled.*

 - **#define DM35424_ADC_INT_START_TRIG_MASK** 0x08
- Register value for Interrupt Mask - Start Trigger Occurred.*

 - **#define DM35424_ADC_INT_STOP_TRIG_MASK** 0x10
- Register value for Interrupt Mask - Stop Trigger Occurred.*

 - **#define DM35424_ADC_INT_POST_BUFF_FULL_MASK** 0x20
- Register value for Interrupt Mask - Post-Stop Buffer Filled.*

 - **#define DM35424_ADC_INT_SAMP_COMPL_MASK** 0x40
- Register value for Interrupt Mask - Sampling Complete.*

 - **#define DM35424_ADC_INT_PACER_TICK_MASK** 0x80
- Register value for Interrupt Mask - Pacer Clock Tick Occurred.*

 - **#define DM35424_ADC_INT_ALL_MASK** 0xFF
- Register value for Interrupt Mask - All Bits.*

 - **#define DM35424_ADC_CHAN_INTR_LOW_THRESHOLD_MASK** 0x01
- Register value for Channel Low Threshold Interrupt.*

 - **#define DM35424_ADC_CHAN_INTR_HIGH_THRESHOLD_MASK** 0x02
- Register value for Channel High Threshold Interrupt.*

 - **#define DM35424_ADC_CHAN_FILTER_ORDER0** 0x0
- Register value for Channel Filter Order 0.*

 - **#define DM35424_ADC_CHAN_FILTER_ORDER1** 0x1
- Register value for Channel Filter Order 1.*

 - **#define DM35424_ADC_CHAN_FILTER_ORDER2** 0x2
- Register value for Channel Filter Order 2.*

 - **#define DM35424_ADC_CHAN_FILTER_ORDER3** 0x3
- Register value for Channel Filter Order 3.*

 - **#define DM35424_ADC_CHAN_FILTER_ORDER4** 0x4
- Register value for Channel Filter Order 4.*

 - **#define DM35424_ADC_CHAN_FILTER_ORDER5** 0x5
- Register value for Channel Filter Order 5.*

 - **#define DM35424_ADC_CHAN_FILTER_ORDER6** 0x6
- Register value for Channel Filter Order 6.*

 - **#define DM35424_ADC_CHAN_FILTER_ORDER7** 0x7
- Register value for Channel Filter Order 7.*

 - **#define DM35424_ADC_FE_CONFIG_POWER_ACTIVE** 0x80
- Register value for setting channel power to active.*

 - **#define DM35424_ADC_FE_CONFIG_PGA_ACTIVE** 0x40
- Register value for setting channel PGA to active.*

 - **#define DM35424_ADC_FE_CONFIG_IN_SWITCH_ENABLED** 0x20
- Register value for setting channel input switch to enabled.*

 - **#define DM35424_ADC_FE_CONFIG_DAC_LOOPBACK** 0x00
- Register value for measuring between DACx Chy and VREF (2.5V).*

 - **#define DM35424_ADC_FE_CONFIG_SNGL_END_POS** 0x08
- Register value for measuring InP - VREF (singled ended).*

 - **#define DM35424_ADC_FE_CONFIG_SNGL_END_NEG** 0x10
- Register value for measuring VREF - InN (singled ended).*

 - **#define DM35424_ADC_FE_CONFIG_DIFFERENTIAL** 0x18
- Register value for setting channel to In(Positive) - In(Negative) connection. This is the most common.*

- **#define DM35424_ADC_FE_CONFIG_GAIN_1** 0x00
Register value for setting a Gain of 1.
- **#define DM35424_ADC_FE_CONFIG_GAIN_2** 0x04
Register value for setting a Gain of 2.
- **#define DM35424_ADC_FE_CONFIG_GAIN_4** 0x02
Register value for setting a Gain of 4.
- **#define DM35424_ADC_FE_CONFIG_GAIN_8** 0x06
Register value for setting a Gain of 8.
- **#define DM35424_ADC_FE_CONFIG_GAIN_16** 0x01
Register value for setting a Gain of 16.
- **#define DM35424_ADC_FE_CONFIG_GAIN_32** 0x05
Register value for setting a Gain of 32.
- **#define DM35424_ADC_FE_CONFIG_GAIN_64** 0x03
Register value for setting a Gain of 64.
- **#define DM35424_ADC_FE_CONFIG_GAIN_128** 0x07
Register value for setting a Gain of 128.
- **#define DM35424_ADC_FE_CONFIG_POWER_MASK** 0x80
Bit mask for the channel power bit of the FE Config.
- **#define DM35424_ADC_FE_CONFIG_PGA_MASK** 0x40
Bit mask for the channel PGA bit of the FE Config.
- **#define DM35424_ADC_FE_CONFIG_INPUT_SW_ENABLE_MASK** 0x20
Bit mask for the channel input switch bit of the FE Config.
- **#define DM35424_ADC_FE_CONFIG_INPUT_LINE_MASK** 0x18
Bit mask for the channel input line bits of the FE Config.
- **#define DM35424_ADC_FE_CONFIG_GAIN_MASK** 0x07
Bit mask for the channel gain bits of the FE Config.
- **#define DM35424_ADC_THRESHOLD_MAX** 8388607L
Maximum allowable value to write to the threshold register.
- **#define DM35424_ADC_THRESHOLD_MIN** -8388608L
Minimum allowable value to write to the threshold register.
- **#define DM35424_ADC_HIGH_SPD_MIN_DIV** 2
Minimum divider allowed for HIGH SPEED mode.
- **#define DM35424_ADC_HIGH_RES_MIN_DIV** 2
Minimum divider allowed for HIGH RES mode.
- **#define DM35424_ADC_LOW_POW_MIN_DIV** 4
Minimum divider allowed for LOW POWER mode.
- **#define DM35424_ADC_LOW_SPD_MIN_DIV** 10
Minimum divider allowed for LOW SPEED mode.
- **#define DM35424_ADC_MAX_RATE** 106000
Max rate of the ADC (Hz).
- **#define DM35424_ADC_MAX_VALUE** 8388607
Max possible value for ADC.
- **#define DM35424_ADC_MIN_VALUE** -8388608
Min possible value for ADC.

Enumerations

- enum [DM35424_Adc_Clock_Events](#) {
[DM35424_ADC_CLK_BUS_SRC_DISABLE](#) = 0x00 , [DM35424_ADC_CLK_BUS_SRC_SAMPLE_TAKEN](#) = 0x80 , [DM35424_ADC_CLK_BUS_SRC_CHAN_THRESH](#) = 0x81 , [DM35424_ADC_CLK_BUS_SRC_PRE_START_BUFF_FULL](#) = 0x82 ,
[DM35424_ADC_CLK_BUS_SRC_START_TRIG](#) = 0x83 , [DM35424_ADC_CLK_BUS_SRC_STOP_TRIG](#) = 0x84 , [DM35424_ADC_CLK_BUS_SRC_POST_STOP_BUFF_FULL](#) = 0x85 , [DM35424_ADC_CLK_BUS_SRC_SAMPLING_COMPLETED](#) = 0x86 ,
[DM35424_ADC_CLK_BUS_SRC_PACER_TICK](#) = 0x87 }
Clock events for the global source clocks.
- enum [DM35424_Input_Ranges](#) {
[DM35424_ADC_RNG_BIPOLAR_2_5V](#) , [DM35424_ADC_RNG_BIPOLAR_1_25V](#) , [DM35424_ADC_RNG_BIPOLAR_625mV](#) ,
[DM35424_ADC_RNG_BIPOLAR_312mV](#) ,
[DM35424_ADC_RNG_BIPOLAR_156mV](#) , [DM35424_ADC_RNG_BIPOLAR_78mV](#) , [DM35424_ADC_RNG_BIPOLAR_39mV](#) ,
[DM35424_ADC_RNG_BIPOLAR_19mV](#) ,
[DM35424_ADC_RNG_UNIPOLAR_5V](#) }
Input range of the ADC input pin. This combines polarity and gain into a single enumeration, and is the preferred way of setting polarity and gain.
- enum [DM35424_Input_Mode](#) { [DM35424_ADC_INPUT_DIFFERENTIAL](#) , [DM35424_ADC_INPUT_SINGLE_ENDED_POS](#) , [DM35424_ADC_INPUT_SINGLE_ENDED_NEG](#) , [DM35424_ADC_INPUT_DAC_LOOPBACK](#) }
Input mode of the ADC pin.
- enum [DM35424_Gains](#) {
[DM35424_ADC_GAIN_05](#) , [DM35424_ADC_GAIN_1](#) , [DM35424_ADC_GAIN_2](#) , [DM35424_ADC_GAIN_4](#) ,
[DM35424_ADC_GAIN_8](#) , [DM35424_ADC_GAIN_16](#) , [DM35424_ADC_GAIN_32](#) , [DM35424_ADC_GAIN_64](#) ,
[DM35424_ADC_GAIN_128](#) }
Input gain to apply to the incoming signal. Note that the preferred method of setting the gain is through the input range enumeration.
- enum [DM35424_Sampling_Mode](#) { [DM35424_ADC_MODE_CONFIG_HIGH_SPEED](#) =0x01 , [DM35424_ADC_MODE_CONFIG_HIGH_POWER](#) =0x03 , [DM35424_ADC_MODE_CONFIG_LOW_POWER](#) =0x04 , [DM35424_ADC_MODE_CONFIG_LOW_SPEED](#) =0x06 }
Sampling mode for the AD Config Register.

Functions

- [DM35424LIB_API](#) int [DM35424_Adc_Open](#) (struct [DM35424_Board_Descriptor](#) *handle, unsigned int number_of_type, struct [DM35424_Function_Block](#) *func_block)
Open the ADC indicated, and determine register locations of control blocks needed to control it.
- [DM35424LIB_API](#) int [DM35424_Adc_Get_Start_Trigger](#) (struct [DM35424_Board_Descriptor](#) *handle, struct [DM35424_Function_Block](#) *func_block, uint8_t *start_trigger)
Get the start trigger for data collection.
- [DM35424LIB_API](#) int [DM35424_Adc_Set_Start_Trigger](#) (struct [DM35424_Board_Descriptor](#) *handle, struct [DM35424_Function_Block](#) *func_block, uint8_t start_trigger)
Set the start trigger for data collection.
- [DM35424LIB_API](#) int [DM35424_Adc_Get_Stop_Trigger](#) (struct [DM35424_Board_Descriptor](#) *handle, struct [DM35424_Function_Block](#) *func_block, uint8_t *stop_trigger)
Get the stop trigger for data collection.
- [DM35424LIB_API](#) int [DM35424_Adc_Set_Stop_Trigger](#) (struct [DM35424_Board_Descriptor](#) *handle, struct [DM35424_Function_Block](#) *func_block, uint8_t stop_trigger)
Set the stop trigger for data collection.
- [DM35424LIB_API](#) int [DM35424_Adc_Get_Pre_Trigger_Samples](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t *count)
Get the amount of data to capture prior to start trigger.

- [DM35424LIB_API](#) int [DM35424_Adc_Set_Pre_Trigger_Samples](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t count)
Set the amount of data to capture prior to start trigger.
- [DM35424LIB_API](#) int [DM35424_Adc_Get_Post_Stop_Samples](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t *count)
Get the amount of data to capture after stop trigger.
- [DM35424LIB_API](#) int [DM35424_Adc_Set_Post_Stop_Samples](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t count)
Set the amount of data to capture after stop trigger.
- [DM35424LIB_API](#) int [DM35424_Adc_Get_Clock_Src](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, enum [DM35424_Clock_Sources](#) *source)
Get the clock source for the ADC.
- [DM35424LIB_API](#) int [DM35424_Adc_Set_Clock_Src](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, enum [DM35424_Clock_Sources](#) source)
Set the clock source for the ADC.
- [DM35424LIB_API](#) int [DM35424_Adc_Initialize](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block)
Prepare the ADC for actual data collection. Moves the ADC from uninitialized to stopped.
- [DM35424LIB_API](#) int [DM35424_Adc_Set_Clk_Divider](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t divider)
Set the Clock Divider for the ADC function block.
- [DM35424LIB_API](#) int [DM35424_Adc_Set_Sample_Rate](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t rate, uint32_t *actual_rate)
Set the sampling rate for the ADC.
- [DM35424LIB_API](#) int [DM35424_Adc_Channel_Get_Front_End_Config](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, uint16_t *fe_config)
Get the front-end config register contents.
- [DM35424LIB_API](#) int [DM35424_Adc_Ad_Config_Set_Mode](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, enum [DM35424_Sampling_Mode](#) mode)
Set the Configuration of the AD Mode.
- [DM35424LIB_API](#) int [DM35424_Adc_Ad_Config_Get_Mode](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint16_t *mode)
Get AD Config register.
- [DM35424LIB_API](#) int [DM35424_Adc_Interrupt_Set_Config](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint16_t int_source, int enable)
Configure the interrupts for the ADC.
- [DM35424LIB_API](#) int [DM35424_Adc_Interrupt_Get_Config](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint16_t *interrupt_ena)
Get the interrupt configuration for the ADC.
- [DM35424LIB_API](#) int [DM35424_Adc_Start](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block)
Set the ADC mode to Start.
- [DM35424LIB_API](#) int [DM35424_Adc_Start_Rearm](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block)
Set the ADC mode to Start-Rearm.
- [DM35424LIB_API](#) int [DM35424_Adc_Reset](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block)
Set the ADC mode to Reset.
- [DM35424LIB_API](#) int [DM35424_Adc_Pause](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block)
Set the ADC mode to Pause.
- [DM35424LIB_API](#) int [DM35424_Adc_Uninitialize](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block)

Set the ADC mode to Uninitialized.

- [DM35424LIB_API](#) int [DM35424_Adc_Get_Mode_Status](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint8_t *mode_status)

Get the ADC mode-status value.

- [DM35424LIB_API](#) int [DM35424_Adc_Channel_Get_Last_Sample](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, int32_t *value)

Get the last sample taken from the ADC.

- [DM35424LIB_API](#) int [DM35424_Adc_Get_Sample_Count](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t *value)

Get the count of number of samples taken.

- [DM35424LIB_API](#) int [DM35424_Adc_Interrupt_Get_Status](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint16_t *value)

Get the interrupt status register.

- [DM35424LIB_API](#) int [DM35424_Adc_Interrupt_Clear_Status](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint16_t value)

Clear the interrupt status register.

- [DM35424LIB_API](#) int [DM35424_Adc_Channel_Setup](#) (struct [DM35424_Board_Descriptor](#) *handle, struct [DM35424_Function_Block](#) *func_block, unsigned int channel, enum [DM35424_Input_Ranges](#) input_range, enum [DM35424_Input_Mode](#) input_mode)

Setup the channel input for the ADC.

- [DM35424LIB_API](#) int [DM35424_Adc_Channel_Reset](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel)

Reset the channel front-end config.

- [DM35424LIB_API](#) int [DM35424_Adc_Channel_Interrupt_Set_Config](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, uint8_t interrupts_to_↵ set, int enable)

Setup the channel interrupts.

- [DM35424LIB_API](#) int [DM35424_Adc_Channel_Interrupt_Get_Config](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, uint8_t *chan_intr_↵ enable)

Get the channel interrupt configuration.

- [DM35424LIB_API](#) int [DM35424_Adc_Channel_Interrupt_Get_Status](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, uint8_t *chan_intr_↵ status)

Get the channel interrupt status.

- [DM35424LIB_API](#) int [DM35424_Adc_Channel_Interrupt_Clear_Status](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, uint8_t chan_intr_status)

Clear the interrupt status for this channel.

- [DM35424LIB_API](#) int [DM35424_Adc_Channel_Find_Interrupt](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int *channel_with_interrupt, int *channel_↵ has_interrupt, uint8_t *channel_intr_status, uint8_t *channel_intr_enable)

Find the first channel with an interrupt. Note that this is only useful when looking for a threshold interrupt.

- [DM35424LIB_API](#) int [DM35424_Adc_Channel_Set_Filter](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, uint8_t chan_filter)

Set the filter value for the channel.

- [DM35424LIB_API](#) int [DM35424_Adc_Channel_Get_Filter](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, uint8_t *chan_filter)

Get the filter value for the channel.

- [DM35424LIB_API](#) int [DM35424_Adc_Channel_Set_Low_Threshold](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, int32_t threshold)

Set the lower threshold for this channel.

- [DM35424LIB_API](#) int [DM35424_Adc_Channel_Set_High_Threshold](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, int32_t threshold)

Set the high threshold for this channel.

- [DM35424LIB_API](#) int [DM35424_Adc_Channel_Get_Thresholds](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, int32_t *low_threshold, int32_t *high_threshold)

Get both thresholds for this channel.

- [DM35424LIB_API](#) int [DM35424_Adc_Fifo_Channel_Read](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, int32_t *value)

Read an ADC sample stored in the onboard FIFO.

- [DM35424LIB_API](#) int [DM35424_Adc_Set_Clock_Source_Global](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, enum [DM35424_Clock_Sources](#) clock_select, enum [DM35424_Adc_Clock_Events](#) clock_driver)

Set the global clock source for the ADC.

- [DM35424LIB_API](#) int [DM35424_Adc_Get_Clock_Source_Global](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, int clock_select, int *clock_source)

Get the global clock source for the selected clock.

- [DM35424LIB_API](#) int [DM35424_Adc_Sample_To_Volts](#) (enum [DM35424_Input_Ranges](#) input_range, int32_t adc_sample, float *volts)

Convert an ADC sample to a volts value.

- [DM35424LIB_API](#) int [DM35424_Adc_Volts_To_Sample](#) (enum [DM35424_Input_Ranges](#) input_range, float volts, int32_t *adc_sample)

Convert volts to an ADC value.

6.29.1 Detailed Description

Definitions for the DM35424 ADC Library.

Id

[dm35424_adc_library.h](#) 106898 2017-03-08 13:44:23Z rgroner

Definition in file [dm35424_adc_library.h](#).

6.30 dm35424_adc_library.h

[Go to the documentation of this file.](#)

```
00001
00009
00010
00011 //-----
00012 //  COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC.  ALL RIGHTS RESERVED.
00013 //
00014 //  This software package is dual-licensed.  Source code that is compiled for
00015 //  kernel mode execution is licensed under the GNU General Public License
00016 //  version 2.  For a copy of this license, refer to the file
00017 //  LICENSE GPLv2.TXT (which should be included with this software) or contact
00018 //  the Free Software Foundation.  Source code that is compiled for user mode
00019 //  execution is licensed under the RTD End-User Software License Agreement.
00020 //  For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00021 //  Technologies, Inc.  Using this software indicates agreement with the
00022 //  license terms listed above.
00023 //-----
00024
00025 #ifndef _DM35424_ADC_LIBRARY_H_
00026 #define _DM35424_ADC_LIBRARY_H_
00027
00028 #include "dm35424_gbc_library.h"
00029
00030 #ifdef __cplusplus
00031 extern "C" {
```

```

00032 #endif
00033
00034
00039
00040 /*=====
00041 Constants
00042 =====*/
00043
00048 #define DM35424_ADC_MODE_RESET          0x00
00053 #define DM35424_ADC_MODE_PAUSE          0x01
00054
00059 #define DM35424_ADC_MODE_GO_SINGLE_SHOT  0x02
00060
00065 #define DM35424_ADC_MODE_GO_REARM        0x03
00070 #define DM35424_ADC_MODE_UNINITIALIZED   0x04
00071
00076 #define DM35424_ADC_STAT_STOPPED         0x00
00077
00082 #define DM35424_ADC_STAT_FILLING_PRE_TRIG_BUFF 0x01
00083
00088 #define DM35424_ADC_STAT_WAITING_START_TRIG 0x02
00089
00094 #define DM35424_ADC_STAT_SAMPLING        0x03
00095
00100 #define DM35424_ADC_STAT_FILLING_POST_TRIG_BUFF 0x04
00101
00106 #define DM35424_ADC_STAT_WAIT_REARM      0x05
00107
00112 #define DM35424_ADC_STAT_DONE            0x07
00113
00118 #define DM35424_ADC_STAT_UNINITIALIZED   0x08
00119
00124 #define DM35424_ADC_STAT_INITIALIZING    0x09
00125
00130 #define DM35424_ADC_INT_SAMPLE_TAKEN_MASK 0x01
00131
00136 #define DM35424_ADC_INT_CHAN_THRESHOLD_MASK 0x02
00137
00142 #define DM35424_ADC_INT_PRE_BUFF_FULL_MASK 0x04
00143
00148 #define DM35424_ADC_INT_START_TRIG_MASK 0x08
00149
00154 #define DM35424_ADC_INT_STOP_TRIG_MASK 0x10
00155
00160 #define DM35424_ADC_INT_POST_BUFF_FULL_MASK 0x20
00161
00166 #define DM35424_ADC_INT_SAMP_COMPL_MASK 0x40
00167
00172 #define DM35424_ADC_INT_PACER_TICK_MASK 0x80
00173
00178 #define DM35424_ADC_INT_ALL_MASK         0xFF
00179
00184 #define DM35424_ADC_CHAN_INTR_LOW_THRESHOLD_MASK 0x01
00185
00190 #define DM35424_ADC_CHAN_INTR_HIGH_THRESHOLD_MASK 0x02
00191
00196 #define DM35424_ADC_CHAN_FILTER_ORDER0 0x0
00197
00202 #define DM35424_ADC_CHAN_FILTER_ORDER1 0x1
00203
00208 #define DM35424_ADC_CHAN_FILTER_ORDER2 0x2
00209
00214 #define DM35424_ADC_CHAN_FILTER_ORDER3 0x3
00215
00220 #define DM35424_ADC_CHAN_FILTER_ORDER4 0x4
00221
00226 #define DM35424_ADC_CHAN_FILTER_ORDER5 0x5
00227
00232 #define DM35424_ADC_CHAN_FILTER_ORDER6 0x6
00233
00238 #define DM35424_ADC_CHAN_FILTER_ORDER7 0x7
00239
00244 #define DM35424_ADC_FE_CONFIG_POWER_ACTIVE 0x80
00245
00250 #define DM35424_ADC_FE_CONFIG_PGA_ACTIVE 0x40
00251
00256 #define DM35424_ADC_FE_CONFIG_IN_SWITCH_ENABLED 0x20
00257
00262 #define DM35424_ADC_FE_CONFIG_DAC_LOOPBACK 0x00
00263
00268 #define DM35424_ADC_FE_CONFIG_SNGL_END_POS 0x08
00269
00274 #define DM35424_ADC_FE_CONFIG_SNGL_END_NEG 0x10
00275
00281 #define DM35424_ADC_FE_CONFIG_DIFFERENTIAL 0x18
00282
00287 #define DM35424_ADC_FE_CONFIG_GAIN_1 0x00

```



```

00288
00293 #define DM35424_ADC_FE_CONFIG_GAIN_2          0x04
00294
00299 #define DM35424_ADC_FE_CONFIG_GAIN_4          0x02
00300
00305 #define DM35424_ADC_FE_CONFIG_GAIN_8          0x06
00306
00311 #define DM35424_ADC_FE_CONFIG_GAIN_16         0x01
00312
00317 #define DM35424_ADC_FE_CONFIG_GAIN_32         0x05
00318
00323 #define DM35424_ADC_FE_CONFIG_GAIN_64         0x03
00324
00329 #define DM35424_ADC_FE_CONFIG_GAIN_128        0x07
00330
00335 #define DM35424_ADC_FE_CONFIG_POWER_MASK      0x80
00336
00341 #define DM35424_ADC_FE_CONFIG_PGA_MASK        0x40
00342
00347 #define DM35424_ADC_FE_CONFIG_INPUT_SW_ENABLE_MASK 0x20
00348
00353 #define DM35424_ADC_FE_CONFIG_INPUT_LINE_MASK 0x18
00354
00359 #define DM35424_ADC_FE_CONFIG_GAIN_MASK        0x07
00360
00365 #define DM35424_ADC_THRESHOLD_MAX             8388607L
00366
00371 #define DM35424_ADC_THRESHOLD_MIN            -8388608L
00372
00377 #define DM35424_ADC_HIGH_SPD_MIN_DIV          2
00378
00383 #define DM35424_ADC_HIGH_RES_MIN_DIV          2
00384
00389 #define DM35424_ADC_LOW_POW_MIN_DIV           4
00390
00395 #define DM35424_ADC_LOW_SPD_MIN_DIV           10
00396
00401 #define DM35424_ADC_MAX_RATE                 106000
00402
00407 #define DM35424_ADC_MAX_VALUE                 8388607
00408
00413 #define DM35424_ADC_MIN_VALUE                 -8388608
00414
00415 /*=====
00416 Enumerations
00417 =====*/
00418
00419
00425 enum DM35424_Adc_Clock_Events {
00426
00427
00432     DM35424_ADC_CLK_BUS_SRC_DISABLE = 0x00,
00433
00438     DM35424_ADC_CLK_BUS_SRC_SAMPLE_TAKEN = 0x80,
00439
00444     DM35424_ADC_CLK_BUS_SRC_CHAN_THRESH = 0x81,
00445
00450     DM35424_ADC_CLK_BUS_SRC_PRE_START_BUFF_FULL = 0x82,
00451
00456     DM35424_ADC_CLK_BUS_SRC_START_TRIG = 0x83,
00457
00462     DM35424_ADC_CLK_BUS_SRC_STOP_TRIG = 0x84,
00463
00468     DM35424_ADC_CLK_BUS_SRC_POST_STOP_BUFF_FULL = 0x85,
00469
00474     DM35424_ADC_CLK_BUS_SRC_SAMPLING_COMPLETE = 0x86,
00475
00480     DM35424_ADC_CLK_BUS_SRC_PACER_TICK = 0x87
00481
00482 };
00483
00494 enum DM35424_Input_Ranges {
00495
00499     DM35424_ADC_RNG_BIPOLAR_2_5V,
00500
00504     DM35424_ADC_RNG_BIPOLAR_1_25V,
00505
00509     DM35424_ADC_RNG_BIPOLAR_625mV,
00510
00514     DM35424_ADC_RNG_BIPOLAR_312mV,
00515
00519     DM35424_ADC_RNG_BIPOLAR_156mV,
00520
00524     DM35424_ADC_RNG_BIPOLAR_78mV,
00525
00529     DM35424_ADC_RNG_BIPOLAR_39mV,
00530

```

```

00534         DM35424_ADC_RNG_BIPOLAR_19mV,
00535
00539         DM35424_ADC_RNG_UNIPOLAR_5V,
00540
00541     };
00542
00543
00553     enum DM35424_Input_Mode {
00557         DM35424_ADC_INPUT_DIFFERENTIAL,
00558
00563         DM35424_ADC_INPUT_SINGLE_ENDED_POS,
00564
00569         DM35424_ADC_INPUT_SINGLE_ENDED_NEG,
00570
00575         DM35424_ADC_INPUT_DAC_LOOPBACK,
00576
00577     };
00578
00579
00591     enum DM35424_Gains {
00592
00596         DM35424_ADC_GAIN_05,
00597
00601         DM35424_ADC_GAIN_1,
00602
00606         DM35424_ADC_GAIN_2,
00607
00611         DM35424_ADC_GAIN_4,
00612
00616         DM35424_ADC_GAIN_8,
00617
00621         DM35424_ADC_GAIN_16,
00622
00626         DM35424_ADC_GAIN_32,
00627
00631         DM35424_ADC_GAIN_64,
00632
00636         DM35424_ADC_GAIN_128
00637     };
00638
00639
00640
00645     enum DM35424_Sampling_Mode {
00646
00651         DM35424_ADC_MODE_CONFIG_HIGH_SPEED           =0x01,
00652
00657         DM35424_ADC_MODE_CONFIG_HIGH_RES             =0x03,
00658
00663         DM35424_ADC_MODE_CONFIG_LOW_POWER            =0x04,
00664
00669         DM35424_ADC_MODE_CONFIG_LOW_SPEED            =0x06
00670     };
00671
00672
00673
00674
00675
00676
00677     /*=====
00678     Public library functions
00679     =====*/
00680
00681
00682
00683
00684
00685
00686
00687
00688
00689
00690
00691
00692
00693
00694
00695
00696
00697
00698
00699
00700
00701
00702
00703
00704
00705
00706
00707
00708
00709
00710
00711
00712
00713
00714
00715
00716
00717
00718
00719
00720
00721
00722
00723
00724
00725
00726
00727
00728
00729
00730
00731
00732
00733
00734
00735
00736
00737
00738
00739
00740
00741
00742
00743
00744
00745
00746
00747
00748
00749
00750
00751
00752
00753
00754
00755
00756
00757
00758
00759
00760
00761
00762
00763
00764
00765
00766
00767
00768
00769
00770
00771
00772
00773
00774
00775
00776
00777
00778
00779
00780
00781
00782
00783
00784
00785
00786
00787
00788
00789
00790
00791
00792
00793
00794
00795
00796
00797
00798
00799
00800
00801
00802
00803
00804
00805
00806
00807
00808
00809
00810
00811
00812
00813
00814
00815
00816
00817
00818
00819
00820
00821
00822
00823
00824
00825
00826
00827
00828
00829
00830
00831
00832
00833
00834
00835
00836
00837
00838
00839
00840
00841
00842
00843
00844
00845
00846
00847

```

```
00884 DM35424LIB_API
00885 int DM35424_Adc_Set_Stop_Trigger(struct DM35424_Board_Descriptor *handle,
00886                                  struct DM35424_Function_Block *func_block,
00887                                  uint8_t stop_trigger);
00888
00889
00923 DM35424LIB_API
00924 int DM35424_Adc_Get_Pre_Trigger_Samples(struct DM35424_Board_Descriptor *handle,
00925                                          const struct DM35424_Function_Block *func_block,
00926                                          uint32_t *count);
00927
00928
00969 DM35424LIB_API
00970 int DM35424_Adc_Set_Pre_Trigger_Samples(struct DM35424_Board_Descriptor *handle,
00971                                          const struct DM35424_Function_Block *func_block,
00972                                          uint32_t count);
00973
00974
01008 DM35424LIB_API
01009 int DM35424_Adc_Get_Post_Stop_Samples(struct DM35424_Board_Descriptor *handle,
01010                                       const struct DM35424_Function_Block *func_block,
01011                                       uint32_t *count);
01012
01013
01047 DM35424LIB_API
01048 int DM35424_Adc_Set_Post_Stop_Samples(struct DM35424_Board_Descriptor *handle,
01049                                       const struct DM35424_Function_Block *func_block,
01050                                       uint32_t count);
01051
01052
01086 DM35424LIB_API
01087 int DM35424_Adc_Get_Clock_Src(struct DM35424_Board_Descriptor *handle,
01088                               const struct DM35424_Function_Block *func_block,
01089                               enum DM35424_Clock_Sources *source);
01090
01091
01128 DM35424LIB_API
01129 int DM35424_Adc_Set_Clock_Src(struct DM35424_Board_Descriptor *handle,
01130                               const struct DM35424_Function_Block *func_block,
01131                               enum DM35424_Clock_Sources source);
01132
01133
01134
01173 DM35424LIB_API
01174 int DM35424_Adc_Initialize(struct DM35424_Board_Descriptor *handle,
01175                            const struct DM35424_Function_Block *func_block);
01176
01177
01178
01211 DM35424LIB_API
01212 int DM35424_Adc_Set_Clk_Divider(struct DM35424_Board_Descriptor *handle,
01213                                 const struct DM35424_Function_Block *func_block,
01214                                 uint32_t divider);
01215
01216
01261 DM35424LIB_API
01262 int DM35424_Adc_Set_Sample_Rate(struct DM35424_Board_Descriptor *handle,
01263                                 const struct DM35424_Function_Block *func_block,
01264                                 uint32_t rate,
01265                                 uint32_t *actual_rate);
01266
01267
01268
01309 DM35424LIB_API
01310 int DM35424_Adc_Channel_Get_Front_End_Config(struct DM35424_Board_Descriptor *handle,
01311                                               const struct DM35424_Function_Block *func_block,
01312                                               unsigned int channel,
01313                                               uint16_t *fe_config);
01314
01355 DM35424LIB_API
01356 int DM35424_Adc_Ad_Config_Set_Mode(struct DM35424_Board_Descriptor *handle,
01357                                     const struct DM35424_Function_Block *func_block,
01358                                     enum DM35424_Sampling_Mode mode);
01359
01400 DM35424LIB_API
01401 int DM35424_Adc_Ad_Config_Get_Mode(struct DM35424_Board_Descriptor *handle,
01402                                    const struct DM35424_Function_Block *func_block,
01403                                    uint16_t *mode);
01404
01405
01445 DM35424LIB_API
01446 int DM35424_Adc_Interrupt_Set_Config(struct DM35424_Board_Descriptor *handle,
01447                                       const struct DM35424_Function_Block *func_block,
01448                                       uint16_t int_source,
01449                                       int enable);
01450
01451
```

```
01485 DM35424LIB_API
01486 int DM35424_Adc_Interrupt_Get_Config(struct DM35424_Board_Descriptor *handle,
01487                                     const struct DM35424_Function_Block *func_block,
01488                                     uint16_t *interrupt_ena);
01489
01490
01519 DM35424LIB_API
01520 int DM35424_Adc_Start(struct DM35424_Board_Descriptor *handle,
01521                      const struct DM35424_Function_Block *func_block);
01522
01523
01552 DM35424LIB_API
01553 int DM35424_Adc_Start_Rearm(struct DM35424_Board_Descriptor *handle,
01554                             const struct DM35424_Function_Block *func_block);
01555
01556
01585 DM35424LIB_API
01586 int DM35424_Adc_Reset(struct DM35424_Board_Descriptor *handle,
01587                       const struct DM35424_Function_Block *func_block);
01588
01589
01618 DM35424LIB_API
01619 int DM35424_Adc_Pause(struct DM35424_Board_Descriptor *handle,
01620                       const struct DM35424_Function_Block *func_block);
01621
01622
01651 DM35424LIB_API
01652 int DM35424_Adc_Uninitialize(struct DM35424_Board_Descriptor *handle,
01653                              const struct DM35424_Function_Block *func_block);
01654
01655
01689 DM35424LIB_API
01690 int DM35424_Adc_Get_Mode_Status(struct DM35424_Board_Descriptor *handle,
01691                                 const struct DM35424_Function_Block *func_block,
01692                                 uint8_t *mode_status);
01693
01694
01733 DM35424LIB_API
01734 int DM35424_Adc_Channel_Get_Last_Sample(struct DM35424_Board_Descriptor *handle,
01735                                           const struct DM35424_Function_Block *func_block,
01736                                           unsigned int channel,
01737                                           int32_t *value);
01738
01739
01773 DM35424LIB_API
01774 int DM35424_Adc_Get_Sample_Count(struct DM35424_Board_Descriptor *handle,
01775                                  const struct DM35424_Function_Block *func_block,
01776                                  uint32_t *value);
01777
01778
01812 DM35424LIB_API
01813 int DM35424_Adc_Interrupt_Get_Status(struct DM35424_Board_Descriptor *handle,
01814                                      const struct DM35424_Function_Block *func_block,
01815                                      uint16_t *value);
01816
01817
01851 DM35424LIB_API
01852 int DM35424_Adc_Interrupt_Clear_Status(struct DM35424_Board_Descriptor *handle,
01853                                         const struct DM35424_Function_Block *func_block,
01854                                         uint16_t value);
01855
01856
01909 DM35424LIB_API
01910 int DM35424_Adc_Channel_Setup(struct DM35424_Board_Descriptor *handle,
01911                               struct DM35424_Function_Block *func_block,
01912                               unsigned int channel,
01913                               enum DM35424_Input_Ranges input_range,
01914                               enum DM35424_Input_Mode input_mode);
01915
01952 DM35424LIB_API
01953 int DM35424_Adc_Channel_Reset(struct DM35424_Board_Descriptor *handle,
01954                               const struct DM35424_Function_Block *func_block,
01955                               unsigned int channel);
01956
02005 DM35424LIB_API
02006 int DM35424_Adc_Channel_Interrupt_Set_Config(struct DM35424_Board_Descriptor *handle,
02007                                               const struct DM35424_Function_Block *func_block,
02008                                               unsigned int channel,
02009                                               uint8_t interrupts_to_set,
02010                                               int enable);
02011
02012
02053 DM35424LIB_API
02054 int DM35424_Adc_Channel_Interrupt_Get_Config(struct DM35424_Board_Descriptor *handle,
02055                                               const struct DM35424_Function_Block *func_block,
02056                                               unsigned int channel,
02057                                               uint8_t *chan_intr_enable);
```

```
02058
02059
02101 DM35424LIB_API
02102 int DM35424_Adc_Channel_Interrupt_Get_Status(struct DM35424_Board_Descriptor *handle,
02103                                             const struct DM35424_Function_Block *func_block,
02104                                             unsigned int channel,
02105                                             uint8_t *chan_intr_status);
02106
02107
02149 DM35424LIB_API
02150 int DM35424_Adc_Channel_Interrupt_Clear_Status(struct DM35424_Board_Descriptor *handle,
02151                                                const struct DM35424_Function_Block *func_block,
02152                                                unsigned int channel,
02153                                                uint8_t chan_intr_status);
02154
02155
02205 DM35424LIB_API
02206 int DM35424_Adc_Channel_Find_Interrupt(struct DM35424_Board_Descriptor *handle,
02207                                        const struct DM35424_Function_Block *func_block,
02208                                        unsigned int *channel_with_interrupt,
02209                                        int *channel_has_interrupt,
02210                                        uint8_t *channel_intr_status,
02211                                        uint8_t *channel_intr_enable);
02212
02213
02256 DM35424LIB_API
02257 int DM35424_Adc_Channel_Set_Filter(struct DM35424_Board_Descriptor *handle,
02258                                   const struct DM35424_Function_Block *func_block,
02259                                   unsigned int channel,
02260                                   uint8_t chan_filter);
02261
02262
02306 DM35424LIB_API
02307 int DM35424_Adc_Channel_Get_Filter(struct DM35424_Board_Descriptor *handle,
02308                                   const struct DM35424_Function_Block *func_block,
02309                                   unsigned int channel,
02310                                   uint8_t *chan_filter);
02311
02312
02359 DM35424LIB_API
02360 int DM35424_Adc_Channel_Set_Low_Threshold(struct DM35424_Board_Descriptor *handle,
02361                                           const struct DM35424_Function_Block *func_block,
02362                                           unsigned int channel,
02363                                           int32_t threshold);
02364
02365
02412 DM35424LIB_API
02413 int DM35424_Adc_Channel_Set_High_Threshold(struct DM35424_Board_Descriptor *handle,
02414                                             const struct DM35424_Function_Block *func_block,
02415                                             unsigned int channel,
02416                                             int32_t threshold);
02417
02418
02470 DM35424LIB_API
02471 int DM35424_Adc_Channel_Get_Thresholds(struct DM35424_Board_Descriptor *handle,
02472                                        const struct DM35424_Function_Block *func_block,
02473                                        unsigned int channel,
02474                                        int32_t *low_threshold,
02475                                        int32_t *high_threshold);
02476
02477
02516 DM35424LIB_API
02517 int DM35424_Adc_Fifo_Channel_Read(struct DM35424_Board_Descriptor *handle,
02518                                  const struct DM35424_Function_Block *func_block,
02519                                  unsigned int channel,
02520                                  int32_t *value);
02521
02562 DM35424LIB_API
02563 int DM35424_Adc_Set_Clock_Source_Global(struct DM35424_Board_Descriptor *handle,
02564                                         const struct DM35424_Function_Block *func_block,
02565                                         enum DM35424_Clock_Sources clock_select,
02566                                         enum DM35424_Adc_Clock_Events clock_driver);
02567
02568
02609 DM35424LIB_API
02610 int DM35424_Adc_Get_Clock_Source_Global(struct DM35424_Board_Descriptor *handle,
02611                                         const struct DM35424_Function_Block *func_block,
02612                                         int clock_select,
02613                                         int *clock_source);
02614
02615
02650 DM35424LIB_API
02651 int DM35424_Adc_Sample_To_Volts(
02652     enum DM35424_Input_Ranges input_range,
02653     int32_t adc_sample,
```

```

02654             float *volts);
02655
02656
02690 DM35424LIB_API
02691 int DM35424_Adc_Volts_To_Sample(
02692     enum DM35424_Input_Ranges input_range,
02693     float volts,
02694     int32_t *adc_sample);
02695
02696
02697
02698
02702
02703 #ifdef __cplusplus
02704 }
02705 #endif
02706
02707 #endif

```

6.31 include/dm35424_board_access.h File Reference

Structures for the DM35424 Board Access Library.

```

#include <stdint.h>
#include "dm35424_board_access_structs.h"
#include "dm35424_types.h"
#include "dm35424_os.h"

```

Data Structures

- struct [DM35424_DMA_Descriptor](#)
Descriptor for the DMA on this board.
- struct [DM35424_Function_Block](#)
DM35424 function block descriptor. This structure holds information about a function block, including type, number of DMA channels and buffers, descriptors for each DMA channel, and memory offsets to various control locations.

Macros

- #define [DM35424LIB_API](#)

Functions

- [DM35424LIB_API](#) int [DM35424_Board_Open](#) (uint8_t dev_num, struct [DM35424_Board_Descriptor](#) **handle)
Open the board, providing the file descriptor that all future operations will reference. Also allocate memory for the device descriptor.
- [DM35424LIB_API](#) int [DM35424_Board_Close](#) (struct [DM35424_Board_Descriptor](#) *handle)
Close the board, closing the open handle for the device file, and freeing the memory allocated for the descriptor.
- [DM35424LIB_API](#) int [DM35424_Read](#) (struct [DM35424_Board_Descriptor](#) *handle, union [dm35424_ioctl_argument](#) *ioctl_request)
Read from the board.
- [DM35424LIB_API](#) int [DM35424_Write](#) (struct [DM35424_Board_Descriptor](#) *handle, union [dm35424_ioctl_argument](#) *ioctl_request)
Write to the board.
- [DM35424LIB_API](#) int [DM35424_Modify](#) (struct [DM35424_Board_Descriptor](#) *handle, union [dm35424_ioctl_argument](#) *ioctl_request)
Read/Modify/Write to the board.
- int [DM35424_Dma](#) (struct [DM35424_Board_Descriptor](#) *handle, union [dm35424_ioctl_argument](#) *ioctl_request)
Perform a DMA operation.

6.31.1 Detailed Description

Structures for the DM35424 Board Access Library.

Id

[dm35424_board_access.h](#) 104840 2016-11-30 19:20:54Z rgroner

Definition in file [dm35424_board_access.h](#).

6.31.2 Macro Definition Documentation

6.31.2.1 DM35424LIB_API

```
#define DM35424LIB_API
```

Conditionally set up the library export symbol for the Windows DLL. This will expand to nothing when compiled for Linux.

Definition at line 47 of file [dm35424_board_access.h](#).

Referenced by [DM35424_Adc_Ad_Config_Get_Mode\(\)](#), [DM35424_Adc_Ad_Config_Set_Mode\(\)](#), [DM35424_Adc_Channel_Find_Inte](#), [DM35424_Adc_Channel_Get_Filter\(\)](#), [DM35424_Adc_Channel_Get_Front_End_Config\(\)](#), [DM35424_Adc_Channel_Get_Last_Samp](#), [DM35424_Adc_Channel_Get_Thresholds\(\)](#), [DM35424_Adc_Channel_Interrupt_Clear_Status\(\)](#), [DM35424_Adc_Channel_Interrupt_C](#), [DM35424_Adc_Channel_Interrupt_Get_Status\(\)](#), [DM35424_Adc_Channel_Interrupt_Set_Config\(\)](#), [DM35424_Adc_Channel_Reset\(\)](#), [DM35424_Adc_Channel_Set_Filter\(\)](#), [DM35424_Adc_Channel_Set_High_Threshold\(\)](#), [DM35424_Adc_Channel_Set_Low_Threshold](#), [DM35424_Adc_Channel_Setup\(\)](#), [DM35424_Adc_Fifo_Channel_Read\(\)](#), [DM35424_Adc_Get_Clock_Source_Global\(\)](#), [DM35424_Adc_Get_Clock_Src\(\)](#), [DM35424_Adc_Get_Mode_Status\(\)](#), [DM35424_Adc_Get_Post_Stop_Samples\(\)](#), [DM35424_Adc_Get_Pre_Trigger_Samples\(\)](#), [DM35424_Adc_Get_Sample_Count\(\)](#), [DM35424_Adc_Get_Start_Trigger\(\)](#), [DM35424_Adc_Get_Stop_Trigger\(\)](#), [DM35424_Adc_Initialize\(\)](#), [DM35424_Adc_Interrupt_Clear_Status\(\)](#), [DM35424_Adc_Interrupt_C](#), [DM35424_Adc_Interrupt_Get_Status\(\)](#), [DM35424_Adc_Interrupt_Set_Config\(\)](#), [DM35424_Adc_Open\(\)](#), [DM35424_Adc_Pause\(\)](#), [DM35424_Adc_Reset\(\)](#), [DM35424_Adc_Sample_To_Volts\(\)](#), [DM35424_Adc_Set_Clk_Divider\(\)](#), [DM35424_Adc_Set_Clock_Source](#), [DM35424_Adc_Set_Clock_Src\(\)](#), [DM35424_Adc_Set_Post_Stop_Samples\(\)](#), [DM35424_Adc_Set_Pre_Trigger_Samples\(\)](#), [DM35424_Adc_Set_Sample_Rate\(\)](#), [DM35424_Adc_Set_Start_Trigger\(\)](#), [DM35424_Adc_Set_Stop_Trigger\(\)](#), [DM35424_Adc_Start\(\)](#), [DM35424_Adc_Start_Rearm\(\)](#), [DM35424_Adc_Uninitialize\(\)](#), [DM35424_Board_Close\(\)](#), [DM35424_Board_Open\(\)](#), [DM35424_Dac_Channel_Clear_Marker_Status\(\)](#), [DM35424_Dac_Channel_Get_Marker_Config\(\)](#), [DM35424_Dac_Channel_Get_Marker_Status\(\)](#), [DM35424_Dac_Channel_Set_Marker_Config\(\)](#), [DM35424_Dac_Fifo_Channel_Write](#), [DM35424_Dac_Get_Clock_Div\(\)](#), [DM35424_Dac_Get_Clock_Src\(\)](#), [DM35424_Dac_Get_Conversion_Count\(\)](#), [DM35424_Dac_Get_Last_Conversion\(\)](#), [DM35424_Dac_Get_Mode_Status\(\)](#), [DM35424_Dac_Get_Post_Stop_Conversion_Count\(\)](#), [DM35424_Dac_Get_Start_Trigger\(\)](#), [DM35424_Dac_Get_Stop_Trigger\(\)](#), [DM35424_Dac_Interrupt_Clear_Status\(\)](#), [DM35424_Dac_Interrupt_Get_Config\(\)](#), [DM35424_Dac_Interrupt_Get_Status\(\)](#), [DM35424_Dac_Interrupt_Set_Config\(\)](#), [DM35424_Dac_Open\(\)](#), [DM35424_Dac_Pause\(\)](#), [DM35424_Dac_Reset\(\)](#), [DM35424_Dac_Set_Clock_Div\(\)](#), [DM35424_Dac_Set_Clock_Source_Global\(\)](#), [DM35424_Dac_Set_Clock_Src\(\)](#), [DM35424_Dac_Set_Conversion_Rate\(\)](#), [DM35424_Dac_Set_Last_Conversion\(\)](#), [DM35424_Dac_Set_Post_Stop_Conversion_Count\(\)](#), [DM35424_Dac_Set_Start_Trigger\(\)](#), [DM35424_Dac_Set_Stop_Trigger\(\)](#), [DM35424_Dac_Start\(\)](#), [DM35424_Dac_Volts_To_Conv\(\)](#), [DM35424_Dio_Get_Direction\(\)](#), [DM35424_Dio_Get_Input_Value\(\)](#), [DM35424_Dio_Get_Output_Value\(\)](#), [DM35424_Dio_Open\(\)](#), [DM35424_Dio_Set_Direction\(\)](#), [DM35424_Dma_Buffer_Setup\(\)](#), [DM35424_Dma_Buffer_Status\(\)](#), [DM35424_Dma_Check_For_Error\(\)](#), [DM35424_Dma_Clear\(\)](#), [DM35424_Dma_Configure_Interrupts\(\)](#), [DM35424_Dma_Get_Current_Buffer_Count\(\)](#), [DM35424_Dma_Get_Errors\(\)](#), [DM35424_Dma_Get_Fifo_Counts\(\)](#), [DM35424_Dma_Get_Fifo_State\(\)](#), [DM35424_Dma_Get_Interrupt_Configuration\(\)](#), [DM35424_Dma_Pause\(\)](#), [DM35424_Dma_Reset_Buffer\(\)](#), [DM35424_Dma_Setup\(\)](#), [DM35424_Dma_Setup_Set_Direction\(\)](#), [DM35424_Dma_Setup_Set_Used\(\)](#), [DM35424_Dma_Start\(\)](#), [DM35424_Dma_Status\(\)](#), [DM35424_Dma_Stop\(\)](#), [DM35424_Function_Block_Open\(\)](#), [DM35424_Function_Block_Open_Module\(\)](#), [DM35424_Gbc_Ack_Interrupt\(\)](#), [DM35424_Gbc_Board_Reset\(\)](#), [DM35424_Gbc_Get_Format\(\)](#), [DM35424_Gbc_Get_Fpga_Build\(\)](#), [DM35424_Gbc_Get_Pdp_Numbe](#), [DM35424_Gbc_Get_Revision\(\)](#), [DM35424_Read\(\)](#), [DM35424_Ref_Adjust_Open\(\)](#), [DM35424_Ref_Adjust_Write_Adc_To_NonVolatil](#), [DM35424_Ref_Adjust_Write_Adc_To_Volatile\(\)](#), [DM35424_Ref_Adjust_Write_Dac_To_NonVolatile\(\)](#), [DM35424_Ref_Adjust_Write_D](#), [DM35424_Temperature_Open\(\)](#), and [DM35424_Write\(\)](#).

6.32 dm35424_board_access.h

[Go to the documentation of this file.](#)

```

00001
00009
00010 //-----
00011 // COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC. ALL RIGHTS RESERVED.
00012 //
00013 // This software package is dual-licensed. Source code that is compiled for
00014 // kernel mode execution is licensed under the GNU General Public License
00015 // version 2. For a copy of this license, refer to the file
00016 // LICENSE GPLv2.TXT (which should be included with this software) or contact
00017 // the Free Software Foundation. Source code that is compiled for user mode
00018 // execution is licensed under the RTD End-User Software License Agreement.
00019 // For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00020 // Technologies, Inc. Using this software indicates agreement with the
00021 // license terms listed above.
00022 //-----
00023
00024 #ifndef _DM35424_BOARD_ACCESS_H_
00025 #define _DM35424_BOARD_ACCESS_H_
00026
00027 #ifdef __cplusplus
00028 extern "C" {
00029 #endif
00030
00031 #include <stdint.h>
00032
00033 #include "dm35424_board_access_structs.h"
00034 #include "dm35424_types.h"
00035
00040 #ifdef _WIN32
00041     #ifdef DM35424LIB_EXPORT /* defined in Visual Studio project file */
00042         #define DM35424LIB_API __declspec(dllexport)
00043     #else
00044         #define DM35424LIB_API __declspec(dllimport)
00045     #endif
00046 #else /* Linux, expands to nothing */
00047     #define DM35424LIB_API
00048 #endif
00049
00050 #include "dm35424_os.h"
00051
00052
00053 /*=====
00054 Structures
00055 =====*/
00056
00065 struct DM35424_DMA_Descriptor {
00066
00070     uint32_t control_offset;
00071
00075     uint8_t num_buffers;
00076
00080     uint32_t buffer_start_offset[MAX_DMA_BUFFERS];
00081
00082 };
00083
00084
00085
00092
00093 struct DM35424_Function_Block {
00094
00098     uint16_t type;
00099
00103     uint16_t sub_type;
00104
00108
00109     uint16_t type_revision;
00110
00114     uint32_t fb_offset;
00115
00119     uint32_t dma_offset;
00120
00124     int fb_num;
00125
00130     int ordinal_fb_type_num;
00131
00135     uint8_t num_dma_buffers;
00136
00140     uint8_t num_dma_channels;
00141
00145     uint32_t control_offset;
00146
00147     uint8_t control_size;

```



```

00148
00149     uint8_t channel_size;
00150
00151     struct DM35424_DMA_Descriptor dma_channel[MAX_DMA_CHANNELS];
00152
00153 };
00154
00155
00156
00157
00158
00159
00160
00161 /*****
00162  * Board Level Functions
00163  *****/
00164
00165
00166
00167
00168
00169 DM35424LIB_API
00170 int
00171 DM35424_Board_Open(uint8_t dev_num,
00172                    struct DM35424_Board_Descriptor ** handle);
00173
00174
00175
00176
00177 DM35424LIB_API
00178 int DM35424_Board_Close(struct DM35424_Board_Descriptor *handle);
00179
00180
00181
00182
00183 DM35424LIB_API
00184 int DM35424_Read(struct DM35424_Board_Descriptor *handle,
00185                  union dm35424_ioctl_argument *ioctl_request);
00186
00187
00188
00189
00190 DM35424LIB_API
00191 int DM35424_Write(struct DM35424_Board_Descriptor *handle,
00192                   union dm35424_ioctl_argument *ioctl_request);
00193
00194
00195
00196
00197 DM35424LIB_API
00198 int DM35424_Modify(struct DM35424_Board_Descriptor *handle,
00199                    union dm35424_ioctl_argument *ioctl_request);
00200
00201
00202
00203
00204
00205
00206
00207
00208
00209
00210
00211
00212
00213
00214
00215
00216
00217
00218
00219
00220
00221
00222
00223
00224
00225
00226
00227
00228
00229
00230
00231
00232
00233
00234
00235
00236
00237
00238
00239
00240
00241
00242
00243
00244
00245
00246
00247
00248
00249
00250
00251
00252
00253
00254
00255
00256
00257
00258
00259
00260
00261
00262
00263
00264
00265
00266
00267
00268
00269
00270
00271
00272
00273
00274
00275
00276
00277
00278
00279
00280
00281
00282
00283
00284
00285
00286
00287
00288
00289
00290
00291
00292
00293
00294
00295
00296
00297
00298
00299
00300
00301
00302
00303
00304
00305
00306
00307
00308
00309
00310
00311
00312
00313
00314
00315
00316
00317
00318
00319
00320
00321
00322
00323
00324
00325
00326
00327
00328
00329
00330
00331
00332
00333
00334
00335
00336
00337
00338
00339
00340
00341
00342
00343
00344
00345
00346
00347
00348
00349
00350
00351
00352
00353
00354
00355
00356
00357
00358
00359
00360
00361
00362
00363
00364
00365
00366
00367
00368
00369
00370
00371
00372
00373

```

6.33 include/dm35424_board_access_structs.h File Reference

Structures for the DM35424 Board Access Library.

Data Structures

- struct [dm35424_pci_access_request](#)
PCI region access request descriptor. This structure holds information about a request to read data from or write data to one of a device's PCI regions.
- struct [dm35424_ioctl_region_readwrite](#)
ioctl() request structure for read from or write to PCI region
- struct [dm35424_ioctl_region_modify](#)
ioctl() request structure for PCI region read/modify/write
- struct [dm35424_ioctl_interrupt_info_request](#)
ioctl() request structure for interrupt

- struct [dm35424_ioctl_dma](#)
ioctl() request structure for DMA
- union [dm35424_ioctl_argument](#)
ioctl() request structure encapsulating all possible requests. This is what gets passed into the kernel from user space on the ioctl() call.

Enumerations

- enum [dm35424_pci_region_num](#) { [DM35424_PCI_REGION_GBC](#) = 0 , [DM35424_PCI_REGION_GBC2](#) , [DM35424_PCI_REGION_FB](#) }
Standard PCI region number.
- enum [dm35424_pci_region_access_size](#) { [DM35424_PCI_REGION_ACCESS_8](#) = 0 , [DM35424_PCI_REGION_ACCESS_16](#) , [DM35424_PCI_REGION_ACCESS_32](#) }
Desired size in bits of access to standard PCI region.
- enum [DM35424_DMA_FUNCTIONS](#) { [DM35424_DMA_INITIALIZE](#) , [DM35424_DMA_READ](#) , [DM35424_DMA_WRITE](#) }

6.33.1 Detailed Description

Structures for the DM35424 Board Access Library.

Id

[dm35424_board_access_structs.h](#) 80523 2014-07-17 18:38:59Z rgroner

Definition in file [dm35424_board_access_structs.h](#).

6.34 dm35424_board_access_structs.h

[Go to the documentation of this file.](#)

```

00001
00009
00010 //-----
00011 //  COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC.  ALL RIGHTS RESERVED.
00012 //
00013 //  This software package is dual-licensed.  Source code that is compiled for
00014 //  kernel mode execution is licensed under the GNU General Public License
00015 //  version 2.  For a copy of this license, refer to the file
00016 //  LICENSE GPLv2.TXT (which should be included with this software) or contact
00017 //  the Free Software Foundation.  Source code that is compiled for user mode
00018 //  execution is licensed under the RTD End-User Software License Agreement.
00019 //  For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00020 //  Technologies, Inc.  Using this software indicates agreement with the
00021 //  license terms listed above.
00022 //-----
00023
00024
00025 #ifndef _DM35424_BOARD_ACCESS_STRUCTS_H_
00026 #define _DM35424_BOARD_ACCESS_STRUCTS_H_
00027
00028
00033
00034
00039
00040 enum dm35424_pci_region_num {
00041
00045
00046     DM35424_PCI_REGION_GBC = 0,
00047
00051
00052     DM35424_PCI_REGION_GBC2,

```

```
00053
00057
00058     DM35424_PCI_REGION_FB
00059 };
00060
00061
00062
00063
00068 enum dm35424_pci_region_access_size {
00069
00073
00074     DM35424_PCI_REGION_ACCESS_8 = 0,
00075
00079
00080     DM35424_PCI_REGION_ACCESS_16,
00081
00085
00086     DM35424_PCI_REGION_ACCESS_32,
00087
00088 };
00089
00090
00096 enum DM35424_DMA_FUNCTIONS {
00097
00101     DM35424_DMA_INITIALIZE,
00102
00106     DM35424_DMA_READ,
00107
00111     DM35424_DMA_WRITE
00112 };
00113
00114
00121
00122 struct dm35424_pci_access_request {
00123
00127
00128     enum dm35424_pci_region_access_size size;
00129
00133
00134     enum dm35424_pci_region_num region;
00135
00139
00140     uint16_t offset;
00141
00145
00146     union {
00147
00151
00152         uint8_t data8;
00153
00157
00158         uint16_t data16;
00159
00163
00164         uint32_t data32;
00165     } data;
00166 };
00167
00168
00173
00174 struct dm35424_ioctl_region_readwrite {
00175
00179
00180     struct dm35424_pci_access_request access;
00181 };
00182
00183
00188
00189 struct dm35424_ioctl_region_modify {
00190
00194     struct dm35424_pci_access_request access;
00195
00207
00208     union {
00209
00213
00214         uint8_t mask8;
00215
00219
00220         uint16_t mask16;
00221
00225
00226         uint32_t mask32;
00227     } mask;
00228 };
00229
00230
```

```

00231
00232
00237
00238 struct dm35424_ioctl_interrupt_info_request {
00239
00240
00244     int interrupts_remaining;
00245
00249     int valid_interrupt;
00250
00254     int error_occurred;
00255
00260     int interrupt_fb;
00261
00262 };
00263
00264
00269 struct dm35424_ioctl_dma {
00270
00274     enum DM35424_DMA_FUNCTIONS function;
00275
00279     int num_buffers;
00280
00284     uint32_t buffer_size;
00285
00289     uint32_t fb_num;
00290
00294     int channel;
00295
00299     int buffer;
00300
00304     struct dm35424_pci_access_request pci;
00305
00309     void *buffer_ptr;
00310
00311 };
00312
00313
00319
00320 union dm35424_ioctl_argument {
00321
00325
00326     struct dm35424_ioctl_region_readwrite readwrite;
00327
00331
00332     struct dm35424_ioctl_region_modify modify;
00333
00337
00338     struct dm35424_ioctl_interrupt_info_request interrupt;
00342
00343     struct dm35424_ioctl_dma dma;
00344
00345
00346 };
00347
00348
00352
00353 #endif
00354

```

6.35 include/dm35424_dac_library.h File Reference

Definitions for the DM35424 DAC Library.

Macros

- **#define DM35424_DAC_INT_CONVERSION_SENT_MASK** 0x01
Register value for Interrupt Mask - Conversion Sent.
- **#define DM35424_DAC_INT_CHAN_MARKER_MASK** 0x02
Register value for Interrupt Mask - Channel has enabled marker.
- **#define DM35424_DAC_INT_START_TRIG_MASK** 0x08
Register value for Interrupt Mask - Start Trigger Occurred.
- **#define DM35424_DAC_INT_STOP_TRIG_MASK** 0x10

- Register value for Interrupt Mask - Stop Trigger Occurred.*

 - #define **DM35424_DAC_INT_POST_STOP_DONE_MASK** 0x20
- Register value for Interrupt Mask - Post-Stop Conversions Completed.*

 - #define **DM35424_DAC_INT_PACER_TICK_MASK** 0x80
- Register value for Interrupt Mask - Pacer Clock Tick.*

 - #define **DM35424_DAC_INT_ALL_MASK** 0xBB
- Register value for Interrupt Mask - All Bits.*

 - #define **DM35424_DAC_MODE_RESET** 0x00
- Register value for Mode - Reset.*

 - #define **DM35424_DAC_MODE_PAUSE** 0x01
- Register value for Mode - Pause.*

 - #define **DM35424_DAC_MODE_GO_SINGLE_SHOT** 0x02
- Register value for Mode - Go (Single Shot).*

 - #define **DM35424_DAC_MODE_GO_REARM** 0x03
- Register value for Mode - Go (Re-arm).*

 - #define **DM35424_DAC_STATUS_STOPPED** 0x00
- Register value for DAC Status - Stopped.*

 - #define **DM35424_DAC_STATUS_WAITING_START_TRIG** 0x02
- Register value for DAC Status - Waiting for Start Trigger.*

 - #define **DM35424_DAC_STATUS_CONVERTING** 0x03
- Register value for DAC Status - Converting Data.*

 - #define **DM35424_DAC_STATUS_OUTPUT_POST** 0x04
- Register value for DAC Status - Outputting Post-Stop conversions.*

 - #define **DM35424_DAC_STATUS_WAITING_REARM** 0x05
- Register value for DAC Status - Waiting for Re-Arm.*

 - #define **DM35424_DAC_STATUS_DONE** 0x07
- Register value for DAC Status - Done.*

 - #define **DM35424_DAC_MAX_RATE** 106000
- Max allowable rate for the DAC (Hz).*

 - #define **DM35424_DAC_MAX_VALUE** 32767
- Max value of the DAC.*

 - #define **DM35424_DAC_MIN_VALUE** -32768
- Min value of the DAC.*

 - #define **DM35424_DAC_LSB_AT_MIN_RANGE** 0.000152587890625f
- DAC LSB (at lowest voltage range).*

Enumerations

- enum **DM35424_Dac_Clock_Events** {
DM35424_DAC_CLK_BUS_SRC_DISABLE = 0x00 , **DM35424_DAC_CLK_BUS_SRC_CONVERSION_SENT**
= 0x80 , **DM35424_DAC_CLK_BUS_SRC_CHAN_MARKER** = 0x81 , **DM35424_DAC_CLK_BUS_SRC_START_TRIG**
= 0x83 ,
DM35424_DAC_CLK_BUS_SRC_STOP_TRIG = 0x84 , **DM35424_DAC_CLK_BUS_SRC_CONV_COMPL**
= 0x85 }
Clocking events that can be used as the global clock sources.

Functions

- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Open](#) (struct [DM35424_Board_Descriptor](#) *handle, unsigned int number_of_type, struct [DM35424_Function_Block](#) *func_block)
Open the DAC indicated, and determine register locations of control blocks needed to control it.
- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Set_Clock_Src](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, enum [DM35424_Clock_Sources](#) source)
Set the clock source of the DAC.
- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Get_Clock_Src](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, enum [DM35424_Clock_Sources](#) *source)
Get the clock source of the DAC.
- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Get_Clock_Div](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t *divider)
Get the clock divider value.
- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Set_Clock_Div](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t divider)
Set the clock divider value.
- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Set_Conversion_Rate](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t requested_rate, uint32_t *actual_rate)
Set the conversion rate of this DAC.
- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Interrupt_Set_Config](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint16_t interrupt_src, int enable)
Set the interrupt configuration for this DAC.
- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Interrupt_Get_Config](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint16_t *interrupt_ena)
Get the interrupt configuration for this DAC.
- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Set_Start_Trigger](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint8_t trigger_value)
Set the start trigger.
- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Set_Stop_Trigger](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint8_t trigger_value)
Set the stop trigger.
- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Get_Start_Trigger](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint8_t *trigger_value)
Get the start trigger.
- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Get_Stop_Trigger](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint8_t *trigger_value)
Get the stop trigger.
- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Start](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block)
Set the DAC Mode to Start.
- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Reset](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block)
Set the DAC Mode to Reset.
- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Pause](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block)
Set the DAC Mode to Pause.
- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Get_Mode_Status](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint8_t *mode_status)
Get the Mode and Status of the DAC.
- [DM35424LIB_API](#) [int](#) [DM35424_Dac_Get_Last_Conversion](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, uint8_t *marker, int16_t *value)
Get the value of the last conversion of the DAC.

- [DM35424LIB_API](#) int [DM35424_Dac_Set_Last_Conversion](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, uint8_t marker, int16_t value)
Set a value to be converted by the DAC immediately.
- [DM35424LIB_API](#) int [DM35424_Dac_Get_Conversion_Count](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t *value)
Get a count of the number of conversions that DAC has executed.
- [DM35424LIB_API](#) int [DM35424_Dac_Interrupt_Get_Status](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint16_t *value)
Get a interrupt status register of the DAC.
- [DM35424LIB_API](#) int [DM35424_Dac_Interrupt_Clear_Status](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint16_t value)
Clear the interrupt status register of the DAC.
- [DM35424LIB_API](#) int [DM35424_Dac_Set_Post_Stop_Conversion_Count](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t value)
Set the number of conversions the DAC will make after a stop trigger.
- [DM35424LIB_API](#) int [DM35424_Dac_Get_Post_Stop_Conversion_Count](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t *value)
Get the number of conversions the DAC will make after a stop trigger.
- [DM35424LIB_API](#) int [DM35424_Dac_Set_Clock_Source_Global](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, enum [DM35424_Clock_Sources](#) clock, enum [DM35424_Dac_Clock_Events](#) clock_driver)
Set the source that will drive the global clock.
- [DM35424LIB_API](#) int [DM35424_Dac_Channel_Set_Marker_Config](#) (struct [DM35424_Board_Descriptor](#) *handle, struct [DM35424_Function_Block](#) *func_block, unsigned int channel, uint8_t marker_enable)
Set the configuration of the marker interrupts for this channel.
- [DM35424LIB_API](#) int [DM35424_Dac_Channel_Get_Marker_Config](#) (struct [DM35424_Board_Descriptor](#) *handle, struct [DM35424_Function_Block](#) *func_block, unsigned int channel, uint8_t *marker_enable)
Get the configuration of the marker interrupts for this channel.
- [DM35424LIB_API](#) int [DM35424_Dac_Channel_Get_Marker_Status](#) (struct [DM35424_Board_Descriptor](#) *handle, struct [DM35424_Function_Block](#) *func_block, unsigned int channel, uint8_t *marker_status)
Get the status of the marker interrupts for this channel.
- [DM35424LIB_API](#) int [DM35424_Dac_Channel_Clear_Marker_Status](#) (struct [DM35424_Board_Descriptor](#) *handle, struct [DM35424_Function_Block](#) *func_block, unsigned int channel, uint8_t marker_to_clear)
Clear the marker interrupts for this channel.
- [DM35424LIB_API](#) int [DM35424_Dac_Fifo_Channel_Write](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, int32_t value)
Write a value to the onboard FIFO.
- [DM35424LIB_API](#) int [DM35424_Dac_Volts_To_Conv](#) (float volts, int16_t *dac_conversion)
Convert a value in volts to a DAC equivalent signed value.
- [DM35424LIB_API](#) int [DM35424_Dac_Conv_To_Volts](#) (int16_t conversion, float *volts)
Convert a DAC conversion value to volts.

6.35.1 Detailed Description

Definitions for the DM35424 DAC Library.

Id

[dm35424_dac_library.h](#) 106898 2017-03-08 13:44:23Z rgroner

Definition in file [dm35424_dac_library.h](#).

6.36 dm35424_dac_library.h

[Go to the documentation of this file.](#)

```

00001
00009
00010 //-----
00011 // COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC. ALL RIGHTS RESERVED.
00012 //
00013 // This software package is dual-licensed. Source code that is compiled for
00014 // kernel mode execution is licensed under the GNU General Public License
00015 // version 2. For a copy of this license, refer to the file
00016 // LICENSE GPLv2.TXT (which should be included with this software) or contact
00017 // the Free Software Foundation. Source code that is compiled for user mode
00018 // execution is licensed under the RTD End-User Software License Agreement.
00019 // For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00020 // Technologies, Inc. Using this software indicates agreement with the
00021 // license terms listed above.
00022 //-----
00023
00024
00025 #ifndef _DM35424_DAC_LIBRARY_H_
00026 #define _DM35424_DAC_LIBRARY_H_
00027
00028
00029 #ifdef __cplusplus
00030 extern "C" {
00031 #endif
00032
00033
00039
00040 /*=====
00041 Public library constants
00042 =====*/
00043
00048 #define DM35424_DAC_INT_CONVERSION_SENT_MASK    0x01
00049
00054 #define DM35424_DAC_INT_CHAN_MARKER_MASK        0x02
00055
00060 #define DM35424_DAC_INT_START_TRIG_MASK         0x08
00061
00066 #define DM35424_DAC_INT_STOP_TRIG_MASK          0x10
00067
00072 #define DM35424_DAC_INT_POST_STOP_DONE_MASK     0x20
00073
00078 #define DM35424_DAC_INT_PACER_TICK_MASK         0x80
00079
00084 #define DM35424_DAC_INT_ALL_MASK                0xBB
00085
00090 #define DM35424_DAC_MODE_RESET                  0x00
00091
00096 #define DM35424_DAC_MODE_PAUSE                  0x01
00097
00102 #define DM35424_DAC_MODE_GO_SINGLE_SHOT         0x02
00103
00108 #define DM35424_DAC_MODE_GO_REARM               0x03
00109
00114 #define DM35424_DAC_STATUS_STOPPED              0x00
00115
00120 #define DM35424_DAC_STATUS_WAITING_START_TRIG   0x02
00121
00126 #define DM35424_DAC_STATUS_CONVERTING           0x03
00127
00133 #define DM35424_DAC_STATUS_OUTPUT_POST         0x04
00134
00139 #define DM35424_DAC_STATUS_WAITING_REARM        0x05
00140
00145 #define DM35424_DAC_STATUS_DONE                 0x07
00146
00151 #define DM35424_DAC_MAX_RATE                    106000
00152
00153
00158 #define DM35424_DAC_MAX_VALUE                   32767
00159
00164 #define DM35424_DAC_MIN_VALUE                   -32768
00165
00170 #define DM35424_DAC_LSB_AT_MIN_RANGE            0.000152587890625f
00171
00172
00173 /*=====
00174 Enumerations
00175 =====*/
00176
00181 enum DM35424_Dac_Clock_Events {
00182
00187     DM35424_DAC_CLK_BUS_SRC_DISABLE = 0x00,

```



```

00188
00193     DM35424_DAC_CLK_BUS_SRC_CONVERSION_SENT = 0x80,
00194
00199     DM35424_DAC_CLK_BUS_SRC_CHAN_MARKER = 0x81,
00200
00205     DM35424_DAC_CLK_BUS_SRC_START_TRIG = 0x83,
00206
00211     DM35424_DAC_CLK_BUS_SRC_STOP_TRIG = 0x84,
00212
00217     DM35424_DAC_CLK_BUS_SRC_CONV_COMPL = 0x85,
00218
00219 };
00220
00221
00222
00223
00227
00228
00233
00234
00235 /*=====
00236 Public library functions
00237 =====*/
00238
00272 DM35424LIB_API
00273 int DM35424_Dac_Open(struct DM35424_Board_Descriptor *handle,
00274                     unsigned int number_of_type,
00275                     struct DM35424_Function_Block *func_block);
00276
00277
00310 DM35424LIB_API
00311 int DM35424_Dac_Set_Clock_Src(struct DM35424_Board_Descriptor *handle,
00312                               const struct DM35424_Function_Block *func_block,
00313                               enum DM35424_Clock_Sources source);
00314
00315
00348 DM35424LIB_API
00349 int DM35424_Dac_Get_Clock_Src(struct DM35424_Board_Descriptor *handle,
00350                               const struct DM35424_Function_Block *func_block,
00351                               enum DM35424_Clock_Sources *source);
00352
00353
00386 DM35424LIB_API
00387 int DM35424_Dac_Get_Clock_Div(struct DM35424_Board_Descriptor *handle,
00388                               const struct DM35424_Function_Block *func_block,
00389                               uint32_t *divider);
00390
00391
00424 DM35424LIB_API
00425 int DM35424_Dac_Set_Clock_Div(struct DM35424_Board_Descriptor *handle,
00426                               const struct DM35424_Function_Block *func_block,
00427                               uint32_t divider);
00428
00429
00475 DM35424LIB_API
00476 int DM35424_Dac_Set_Conversion_Rate(struct DM35424_Board_Descriptor *handle,
00477                                       const struct DM35424_Function_Block *func_block,
00478                                       uint32_t requested_rate,
00479                                       uint32_t *actual_rate);
00480
00481
00521 DM35424LIB_API
00522 int DM35424_Dac_Interrupt_Set_Config(struct DM35424_Board_Descriptor *handle,
00523                                       const struct DM35424_Function_Block *func_block,
00524                                       uint16_t interrupt_src,
00525                                       int enable);
00526
00527
00560 DM35424LIB_API
00561 int DM35424_Dac_Interrupt_Get_Config(struct DM35424_Board_Descriptor *handle,
00562                                       const struct DM35424_Function_Block *func_block,
00563                                       uint16_t *interrupt_ena);
00564
00565
00598 DM35424LIB_API
00599 int DM35424_Dac_Set_Start_Trigger(struct DM35424_Board_Descriptor *handle,
00600                                    const struct DM35424_Function_Block *func_block,
00601                                    uint8_t trigger_value);
00602
00603
00636 DM35424LIB_API
00637 int DM35424_Dac_Set_Stop_Trigger(struct DM35424_Board_Descriptor *handle,
00638                                   const struct DM35424_Function_Block *func_block,
00639                                   uint8_t trigger_value);
00640
00641
00675 DM35424LIB_API

```

```
00676 int DM35424_Dac_Get_Start_Trigger(struct DM35424_Board_Descriptor *handle,
00677                                     const struct DM35424_Function_Block *func_block,
00678                                     uint8_t *trigger_value);
00679
00680
00714 DM35424LIB_API
00715 int DM35424_Dac_Get_Stop_Trigger(struct DM35424_Board_Descriptor *handle,
00716                                     const struct DM35424_Function_Block *func_block,
00717                                     uint8_t *trigger_value);
00718
00719
00747 DM35424LIB_API
00748 int DM35424_Dac_Start(struct DM35424_Board_Descriptor *handle,
00749                       const struct DM35424_Function_Block *func_block);
00750
00751
00779 DM35424LIB_API
00780 int DM35424_Dac_Reset(struct DM35424_Board_Descriptor *handle,
00781                       const struct DM35424_Function_Block *func_block);
00782
00783
00811 DM35424LIB_API
00812 int DM35424_Dac_Pause(struct DM35424_Board_Descriptor *handle,
00813                       const struct DM35424_Function_Block *func_block);
00814
00815
00848 DM35424LIB_API
00849 int DM35424_Dac_Get_Mode_Status(struct DM35424_Board_Descriptor *handle,
00850                                  const struct DM35424_Function_Block *func_block,
00851                                  uint8_t *mode_status);
00852
00853
00896 DM35424LIB_API
00897 int DM35424_Dac_Get_Last_Conversion(struct DM35424_Board_Descriptor *handle,
00898                                     const struct DM35424_Function_Block *func_block,
00899                                     unsigned int channel,
00900                                     uint8_t *marker,
00901                                     int16_t *value);
00902
00903
00950 DM35424LIB_API
00951 int DM35424_Dac_Set_Last_Conversion(struct DM35424_Board_Descriptor *handle,
00952                                     const struct DM35424_Function_Block *func_block,
00953                                     unsigned int channel,
00954                                     uint8_t marker,
00955                                     int16_t value);
00956
00957
00990 DM35424LIB_API
00991 int DM35424_Dac_Get_Conversion_Count(struct DM35424_Board_Descriptor *handle,
00992                                     const struct DM35424_Function_Block *func_block,
00993                                     uint32_t *value);
00994
00995
01028 DM35424LIB_API
01029 int DM35424_Dac_Interrupt_Get_Status(struct DM35424_Board_Descriptor *handle,
01030                                     const struct DM35424_Function_Block *func_block,
01031                                     uint16_t *value);
01032
01033
01066 DM35424LIB_API
01067 int DM35424_Dac_Interrupt_Clear_Status(struct DM35424_Board_Descriptor *handle,
01068                                         const struct DM35424_Function_Block *func_block,
01069                                         uint16_t value);
01070
01071
01104 DM35424LIB_API
01105 int DM35424_Dac_Set_Post_Stop_Conversion_Count(struct DM35424_Board_Descriptor *handle,
01106                                                  const struct DM35424_Function_Block *func_block,
01107                                                  uint32_t value);
01108
01109
01142 DM35424LIB_API
01143 int DM35424_Dac_Get_Post_Stop_Conversion_Count(struct DM35424_Board_Descriptor *handle,
01144                                                  const struct DM35424_Function_Block *func_block,
01145                                                  uint32_t *value);
01146
01147
01189 DM35424LIB_API
01190 int DM35424_Dac_Set_Clock_Source_Global(struct DM35424_Board_Descriptor *handle,
01191                                         const struct DM35424_Function_Block *func_block,
01192                                         enum DM35424_Clock_Sources clock,
01193                                         enum DM35424_Dac_Clock_Events clock_driver);
01194
01195
01196
01197
```

```

01232 DM35424LIB_API
01233 int DM35424_Dac_Channel_Set_Marker_Config(struct DM35424_Board_Descriptor *handle,
01234                                           struct DM35424_Function_Block *func_block,
01235                                           unsigned int channel,
01236                                           uint8_t marker_enable);
01237
01238 DM35424LIB_API
01273 int DM35424_Dac_Channel_Get_Marker_Config(struct DM35424_Board_Descriptor *handle,
01274                                           struct DM35424_Function_Block *func_block,
01275                                           unsigned int channel,
01276                                           uint8_t *marker_enable);
01277
01278
01279
01280
01315 DM35424LIB_API
01316 int DM35424_Dac_Channel_Get_Marker_Status(struct DM35424_Board_Descriptor *handle,
01317                                           struct DM35424_Function_Block *func_block,
01318                                           unsigned int channel,
01319                                           uint8_t *marker_status);
01320
01321
01356 DM35424LIB_API
01357 int DM35424_Dac_Channel_Clear_Marker_Status(struct DM35424_Board_Descriptor *handle,
01358                                           struct DM35424_Function_Block *func_block,
01359                                           unsigned int channel,
01360                                           uint8_t marker_to_clear);
01361
01362
01401 DM35424LIB_API
01402 int DM35424_Dac_Fifo_Channel_Write(struct DM35424_Board_Descriptor *handle,
01403                                     const struct DM35424_Function_Block *func_block,
01404                                     unsigned int channel,
01405                                     int32_t value);
01406
01437 DM35424LIB_API
01438 int DM35424_Dac_Volts_To_Conv(
01439     float volts,
01440     int16_t *dac_conversion);
01441
01442
01472 DM35424LIB_API
01473 int DM35424_Dac_Conv_To_Volts(
01474     int16_t conversion,
01475     float *volts);
01476
01477
01478
01482
01483 #ifdef __cplusplus
01484 }
01485 #endif
01486
01487 #endif

```

6.37 include/dm35424_dio_library.h File Reference

Definitions for the DM35424 DIO Library.

```
#include "dm35424_gbc_library.h"
```

Functions

- [DM35424LIB_API int DM35424_Dio_Open](#) (struct [DM35424_Board_Descriptor](#) *handle, unsigned int number_of_type, struct [DM35424_Function_Block](#) *func_block)
Open the DIO indicated, and determine register locations of control blocks needed to control it.
- [DM35424LIB_API int DM35424_Dio_Set_Direction](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t direction)
Set the direction of the DIO pins.
- [DM35424LIB_API int DM35424_Dio_Get_Direction](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t *direction)

Get the direction of the DIO pins.

- [DM35424LIB_API](#) int [DM35424_Dio_Get_Input_Value](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t *value)

Get the input value of the DIO.

- [DM35424LIB_API](#) int [DM35424_Dio_Get_Output_Value](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t *value)

Get the current value of the output register.

- [DM35424LIB_API](#) int [DM35424_Dio_Set_Output_Value](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, uint32_t value)

Set the value to be put on output pins.

6.37.1 Detailed Description

Definitions for the DM35424 DIO Library.

Id

[dm35424_dio_library.h](#) 84663 2014-12-19 22:03:35Z mmcintire

Definition in file [dm35424_dio_library.h](#).

6.38 dm35424_dio_library.h

[Go to the documentation of this file.](#)

```
00001
00009
00010 //-----
00011 //  COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC.  ALL RIGHTS RESERVED.
00012 //
00013 //  This software package is dual-licensed.  Source code that is compiled for
00014 //  kernel mode execution is licensed under the GNU General Public License
00015 //  version 2.  For a copy of this license, refer to the file
00016 //  LICENSE_GPLv2.TXT (which should be included with this software) or contact
00017 //  the Free Software Foundation.  Source code that is compiled for user mode
00018 //  execution is licensed under the RTD End-User Software License Agreement.
00019 //  For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00020 //  Technologies, Inc.  Using this software indicates agreement with the
00021 //  license terms listed above.
00022 //-----
00023
00024
00025 #ifndef _DM35424_DIO_LIBRARY_H_
00026 #define _DM35424_DIO_LIBRARY_H_
00027
00028
00029 #include "dm35424_gbc_library.h"
00030
00031 #ifdef __cplusplus
00032 extern "C" {
00033 #endif
00034
00040
00041
00075 DM35424LIB_API
00076 int DM35424_Dio_Open(struct DM35424_Board_Descriptor *handle,
00077                     unsigned int number_of_type,
00078                     struct DM35424_Function_Block *func_block);
00079
00080
00113 DM35424LIB_API
00114 int DM35424_Dio_Set_Direction(struct DM35424_Board_Descriptor *handle,
00115                               const struct DM35424_Function_Block *func_block,
00116                               uint32_t direction);
00117
00118
00150 DM35424LIB_API
```

```

00151 int DM35424_Dio_Get_Direction(struct DM35424_Board_Descriptor *handle,
00152                               const struct DM35424_Function_Block *func_block,
00153                               uint32_t *direction);
00154
00155
00190 DM35424LIB_API
00191 int DM35424_Dio_Get_Input_Value(struct DM35424_Board_Descriptor *handle,
00192                                 const struct DM35424_Function_Block *func_block,
00193                                 uint32_t *value);
00194
00229 DM35424LIB_API
00230 int DM35424_Dio_Get_Output_Value(struct DM35424_Board_Descriptor *handle,
00231                                  const struct DM35424_Function_Block *func_block,
00232                                  uint32_t *value);
00233
00234
00269 DM35424LIB_API
00270 int DM35424_Dio_Set_Output_Value(struct DM35424_Board_Descriptor *handle,
00271                                  const struct DM35424_Function_Block *func_block,
00272                                  uint32_t value);
00273
00274
00278
00279 #ifdef __cplusplus
00280 }
00281 #endif
00282
00283 #endif

```

6.39 include/dm35424_dma_library.h File Reference

Definitions for the DM35424 DMA Library.

```
#include <stdint.h>
```

Macros

- **#define DM35424_DMA_ACTION_CLEAR 0x00**
Register value for DMA clear action.
- **#define DM35424_DMA_ACTION_GO 0x01**
Register value for DMA go action.
- **#define DM35424_DMA_ACTION_PAUSE 0x02**
Register value for DMA pause action.
- **#define DM35424_DMA_ACTION_HALT 0x03**
Register value for DMA halt action.
- **#define DM35424_DMA_SETUP_DIRECTION_READ 0x04**
Register value to set DMA to READ direction.
- **#define DM35424_DMA_SETUP_DIRECTION_WRITE 0x00**
Register value to set DMA to WRITE direction.
- **#define DM35424_DMA_SETUP_DIRECTION_MASK 0x04**
Register value to set DMA to READ direction.
- **#define DM35424_DMA_SETUP_IGNORE_USED 0x08**
Register value to tell DMA to ignore used buffers.
- **#define DM35424_DMA_SETUP_NOT_IGNORE_USED 0x00**
Register value to tell DMA to not ignore used buffers.
- **#define DM35424_DMA_SETUP_IGNORE_USED_MASK 0x08**
Bit mask for Ignore Used bit in setup register.
- **#define DM35424_DMA_SETUP_INT_ENABLE 0x01**
Register value to enabled interrupts in the setup register.

- **#define DM35424_DMA_SETUP_INT_DISABLE** 0x00
Register value to disable interrupts in the setup register.
- **#define DM35424_DMA_SETUP_INT_MASK** 0x01
Bit mask for the interrupt bit in the setup register.
- **#define DM35424_DMA_SETUP_ERR_INT_ENABLE** 0x02
Register value to enable the error interrupt.
- **#define DM35424_DMA_SETUP_ERR_INT_DISABLE** 0x00
Register value to disable the error interrupt.
- **#define DM35424_DMA_SETUP_ERR_INT_MASK** 0x02
Bit mask for the error interrupt bit in the setup register.
- **#define DM35424_DMA_STATUS_CLEAR** 0x00
Register value to write to status registers to clear them.
- **#define DM35424_DMA_CTRL_CLEAR** 0x00
Register value to write to control register to clear it.
- **#define DM35424_DMA_BUFFER_STATUS_CLEAR** 0x00
Register value to write to the buffer status register to clear it.
- **#define DM35424_DMA_BUFFER_CTRL_CLEAR** 0x00
Register value to write to the buffer control register to clear it.
- **#define DM35424_DMA_BUFFER_STATUS_USED_MASK** 0x01
Bit mask for the used buffer bit in the buffer status register.
- **#define DM35424_DMA_BUFFER_STATUS_TERM_MASK** 0x02
Bit mask for the terminated buffer bit in the buffer status register.
- **#define DM35424_DMA_BUFFER_CTRL_VALID** 0x01
Register value to write to buffer control register to mark it as valid.
- **#define DM35424_DMA_BUFFER_CTRL_HALT** 0x02
Register value to write to buffer control register to tell DMA to halt after processing this buffer.
- **#define DM35424_DMA_BUFFER_CTRL_LOOP** 0x04
Register value to write to buffer control register to tell DMA to loop back to buffer 0 after using this buffer.
- **#define DM35424_DMA_BUFFER_CTRL_INTR** 0x08
Register value to write to buffer control register to tell DMA to issue an interrupt after using this buffer.
- **#define DM35424_DMA_BUFFER_CTRL_PAUSE** 0x10
Register value to write to buffer control register to tell DMA to pause after processing this buffer.
- **#define DM35424_DMA_CTRL_BLOCK_SIZE** 0x10
Constant value indicating DMA control block size.
- **#define DM35424_DMA_BUFFER_CTRL_BLOCK_SIZE** 0x10
Constant value indicating DMA buffer control block size.
- **#define DM35424_BIT_MASK_DMA_BUFFER_SIZE** 0x0FFFFFFF
Bit mask for the DMA buffer size, since it is 24-bits of a 32-bit register.

Enumerations

- enum **DM35424_Fifo_States** { **DM35424_FIFO_UNKNOWN**, **DM35424_FIFO_EMPTY**, **DM35424_FIFO_FULL**, **DM35424_FIFO_HAS_DATA** }
Descriptions of the possible states the FIFO might be in.

Functions

- [DM35424LIB_API](#) int [DM35424_Dma_Start](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel)
Start the DMA.
- [DM35424LIB_API](#) int [DM35424_Dma_Stop](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel)
Stop the DMA.
- [DM35424LIB_API](#) int [DM35424_Dma_Pause](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel)
Pause the DMA.
- [DM35424LIB_API](#) int [DM35424_Dma_Clear](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel)
Clear the DMA.
- [DM35424LIB_API](#) int [DM35424_Dma_Get_Fifo_Counts](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, uint16_t *write_count, uint16_t *read_count)
Get the Read and Write FIFO count values.
- [DM35424LIB_API](#) int [DM35424_Dma_Get_Fifo_State](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, enum [DM35424_Fifo_States](#) *state)
Get the state of the FIFO.
- [DM35424LIB_API](#) int [DM35424_Dma_Configure_Interrupts](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, int enable, int error_enable)
Configure the interrupts for the DMA channel.
- [DM35424LIB_API](#) int [DM35424_Dma_Get_Interrupt_Configuration](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, int *enable, int *error_enable)
Get the configuration of the interrupts for the DMA channel.
- [DM35424LIB_API](#) int [DM35424_Dma_Setup](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, int direction, int ignore_used)
Setup the DMA channel, specifically the direction and if used buffers are ignored.
- [DM35424LIB_API](#) int [DM35424_Dma_Setup_Set_Direction](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, int direction)
Set the direction of the DMA, read or write.
- [DM35424LIB_API](#) int [DM35424_Dma_Setup_Set_Used](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, int ignore_used)
Set the DMA channel to ignore or not ignore a used buffer. Ignoring used buffers is mostly useful when outputting a repeating data cycle.
- [DM35424LIB_API](#) int [DM35424_Dma_Get_Errors](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, int *stat_overflow, int *stat_underflow, int *stat_used, int *stat_invalid)
Get the current value of the DMA channel error registers.
- [DM35424LIB_API](#) int [DM35424_Dma_Status](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, uint32_t *current_buffer, uint32_t *current_count, int *current_action, int *stat_overflow, int *stat_underflow, int *stat_used, int *stat_invalid, int *stat_complete)
Get the current status of the DMA channel. Determine which buffer it is using, what its current action is, and the state of all error conditions and normal interrupt conditions.
- [DM35424LIB_API](#) int [DM35424_Dma_Get_Current_Buffer_Count](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, uint32_t *current_buffer, uint32_t *current_count)
Get the current buffer and buffer count in use by the DMA.
- [DM35424LIB_API](#) int [DM35424_Dma_Check_For_Error](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, int *has_error)

Check the DMA channel for any error conditions. This just returns a simple boolean as quickly as possible. If there is an error condition, you will have to query the DMA again to determine what the error is.

- [DM35424LIB_API](#) [int](#) [DM35424_Dma_Buffer_Setup](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, unsigned int buffer, uint8_t ctrl)

Setup the DMA buffer for use.

- [DM35424LIB_API](#) [int](#) [DM35424_Dma_Buffer_Status](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, unsigned int buffer, uint8_t *status, uint8_t *control, uint32_t *size)

Get the status of the buffer. This gets the status, control, and size registers.

- [DM35424LIB_API](#) [int](#) [DM35424_Dma_Check_Buffer_Used](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, unsigned int buffer_num, int *is_←_used)

Check if the indicated buffer has the "Used" flag set.

- [int](#) [DM35424_Dma_Find_Interrupt](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int *channel, int *channel_complete, int *channel_error)

Find which DMA channel has an interrupt condition, whether from using a buffer with interrupt set, or from an error. DMA channels are evaluated starting at Channel 0.

- [int](#) [DM35424_Dma_Clear_Interrupt](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, int clear_overflow, int clear_underflow, int clear_used, int clear_invalid, int clear_complete)

Clear the interrupt flag from a DMA channel. Clearing the flags will allow another interrupt of the same type to occur again, and is the normal operation after handling the interrupt itself.

- [int](#) [DM35424_Dma_Reset_Buffer](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, unsigned int buffer)

Reset the DMA buffer, preparing it to be used again by the DMA engine.

6.39.1 Detailed Description

Definitions for the DM35424 DMA Library.

Id

[dm35424_dma_library.h](#) 105018 2016-12-07 15:47:31Z rgroner

Definition in file [dm35424_dma_library.h](#).

6.40 dm35424_dma_library.h

[Go to the documentation of this file.](#)

```
00001
00009
00010 //-----
00011 //  COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC.  ALL RIGHTS RESERVED.
00012 //
00013 //  This software package is dual-licensed.  Source code that is compiled for
00014 //  kernel mode execution is licensed under the GNU General Public License
00015 //  version 2.  For a copy of this license, refer to the file
00016 //  LICENSE GPLv2.TXT (which should be included with this software) or contact
00017 //  the Free Software Foundation.  Source code that is compiled for user mode
00018 //  execution is licensed under the RTD End-User Software License Agreement.
00019 //  For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00020 //  Technologies, Inc.  Using this software indicates agreement with the
00021 //  license terms listed above.
00022 //-----
00023
00024
00025 #ifndef __DM35424_DMA_LIBRARY_H__
```



```

00026 #define __DM35424_DMA_LIBRARY_H__
00027
00028
00029 #include <stdint.h>
00030
00031 #ifdef __cplusplus
00032 extern "C" {
00033 #endif
00034
00035
00036 /*=====
00037 Public library constants
00038 =====*/
00039
00040
00041
00042
00043 #define DM35424_DMA_ACTION_CLEAR          0x00
00044
00045
00046 #define DM35424_DMA_ACTION_GO             0x01
00047
00048
00049 #define DM35424_DMA_ACTION_PAUSE          0x02
00050
00051
00052 #define DM35424_DMA_ACTION_HALT           0x03
00053
00054
00055 #define DM35424_DMA_SETUP_DIRECTION_READ  0x04
00056
00057
00058 #define DM35424_DMA_SETUP_DIRECTION_WRITE 0x00
00059
00060
00061 #define DM35424_DMA_SETUP_DIRECTION_MASK  0x04
00062
00063
00064 #define DM35424_DMA_SETUP_IGNORE_USED     0x08
00065
00066
00067 #define DM35424_DMA_SETUP_NOT_IGNORE_USED 0x00
00068
00069
00070 #define DM35424_DMA_SETUP_IGNORE_USED_MASK 0x08
00071
00072
00073 #define DM35424_DMA_SETUP_INT_ENABLE      0x01
00074
00075
00076 #define DM35424_DMA_SETUP_INT_DISABLE     0x00
00077
00078
00079 #define DM35424_DMA_SETUP_INT_MASK        0x01
00080
00081
00082 #define DM35424_DMA_SETUP_ERR_INT_ENABLE   0x02
00083
00084
00085 #define DM35424_DMA_SETUP_ERR_INT_DISABLE 0x00
00086
00087
00088 #define DM35424_DMA_SETUP_ERR_INT_MASK     0x02
00089
00090
00091 #define DM35424_DMA_STATUS_CLEAR          0x00
00092
00093
00094 #define DM35424_DMA_CTRL_CLEAR            0x00
00095
00096
00097 #define DM35424_DMA_BUFFER_STATUS_CLEAR    0x00
00098
00099
00100 #define DM35424_DMA_BUFFER_CTRL_CLEAR     0x00
00101
00102
00103 #define DM35424_DMA_BUFFER_STATUS_USED_MASK 0x01
00104
00105
00106 #define DM35424_DMA_BUFFER_STATUS_TERM_MASK 0x02
00107
00108
00109 #define DM35424_DMA_BUFFER_CTRL_VALID     0x01
00110
00111
00112 #define DM35424_DMA_BUFFER_CTRL_HALT      0x02
00113
00114
00115 #define DM35424_DMA_BUFFER_CTRL_LOOP      0x04
00116
00117
00118 #define DM35424_DMA_BUFFER_CTRL_INTR      0x08
00119
00120
00121 #define DM35424_DMA_BUFFER_CTRL_PAUSE     0x10
00122
00123
00124 #define DM35424_DMA_CTRL_BLOCK_SIZE       0x10
00125
00126
00127 #define DM35424_DMA_BUFFER_CTRL_BLOCK_SIZE 0x10
00128
00129
00130 #define DM35424_BIT_MASK_DMA_BUFFER_SIZE 0x0FFFFFFF
00131
00132
00133 enum DM35424_Fifo_States {
00134     DM35424_FIFO_UNKNOWN,
00135     DM35424_FIFO_EMPTY,
00136     DM35424_FIFO_FULL,
00137     DM35424_FIFO_HAS_DATA
00138 };
00139
00140

```

```
00263
00264 /*=====
00265 Public library functions
00266 =====*/
00267
00272
00273
00310 DM35424LIB_API
00311 int
00312 DM35424_Dma_Start(struct DM35424_Board_Descriptor *handle,
00313                  const struct DM35424_Function_Block *func_block,
00314                  unsigned int channel);
00315
00316
00353 DM35424LIB_API
00354 int
00355 DM35424_Dma_Stop(struct DM35424_Board_Descriptor *handle,
00356                  const struct DM35424_Function_Block *func_block,
00357                  unsigned int channel);
00358
00359
00360
00397 DM35424LIB_API
00398 int
00399 DM35424_Dma_Pause(struct DM35424_Board_Descriptor *handle,
00400                  const struct DM35424_Function_Block *func_block,
00401                  unsigned int channel);
00402
00403
00441 DM35424LIB_API
00442 int
00443 DM35424_Dma_Clear(struct DM35424_Board_Descriptor *handle,
00444                  const struct DM35424_Function_Block *func_block,
00445                  unsigned int channel);
00446
00447
00496 DM35424LIB_API
00497 int DM35424_Dma_Get_Fifo_Counts(struct DM35424_Board_Descriptor *handle,
00498                                const struct DM35424_Function_Block *func_block,
00499                                unsigned int channel,
00500                                uint16_t *write_count,
00501                                uint16_t *read_count);
00502
00503
00548 DM35424LIB_API
00549 int DM35424_Dma_Get_Fifo_State(struct DM35424_Board_Descriptor *handle,
00550                                const struct DM35424_Function_Block *func_block,
00551                                unsigned int channel,
00552                                enum DM35424_Fifo_States *state);
00553
00554
00555
00602 DM35424LIB_API
00603 int DM35424_Dma_Configure_Interrupts(struct DM35424_Board_Descriptor *handle,
00604                                       const struct DM35424_Function_Block *func_block,
00605                                       unsigned int channel,
00606                                       int enable,
00607                                       int error_enable);
00608
00609
00656 DM35424LIB_API
00657 int DM35424_Dma_Get_Interrupt_Configuration(struct DM35424_Board_Descriptor *handle,
00658                                              const struct DM35424_Function_Block *func_block,
00659                                              unsigned int channel,
00660                                              int *enable,
00661                                              int *error_enable);
00662
00663
00716 DM35424LIB_API
00717 int DM35424_Dma_Setup(struct DM35424_Board_Descriptor *handle,
00718                      const struct DM35424_Function_Block *func_block,
00719                      unsigned int channel,
00720                      int direction,
00721                      int ignore_used);
00722
00723
00765 DM35424LIB_API
00766 int DM35424_Dma_Setup_Set_Direction(struct DM35424_Board_Descriptor *handle,
00767                                      const struct DM35424_Function_Block *func_block,
00768                                      unsigned int channel,
00769                                      int direction);
00770
00771
00813 DM35424LIB_API
00814 int DM35424_Dma_Setup_Set_Used(struct DM35424_Board_Descriptor *handle,
00815                                const struct DM35424_Function_Block *func_block,
00816                                unsigned int channel,
```

```
00817             int ignore_used);
00818
00819
00877 DM35424LIB_API
00878 int DM35424_Dma_Get_Errors(struct DM35424_Board_Descriptor *handle,
00879                             const struct DM35424_Function_Block *func_block,
00880                             unsigned int channel,
00881                             int *stat_overflow,
00882                             int *stat_underflow,
00883                             int *stat_used,
00884                             int *stat_invalid);
00885
00967 DM35424LIB_API
00968 int DM35424_Dma_Status(struct DM35424_Board_Descriptor *handle,
00969                         const struct DM35424_Function_Block *func_block,
00970                         unsigned int channel,
00971                         uint32_t *current_buffer,
00972                         uint32_t *current_count,
00973                         int *current_action,
00974                         int *stat_overflow,
00975                         int *stat_underflow,
00976                         int *stat_used,
00977                         int *stat_invalid,
00978                         int *stat_complete);
00979
00980
01024 DM35424LIB_API
01025 int DM35424_Dma_Get_Current_Buffer_Count(struct DM35424_Board_Descriptor *handle,
01026                                           const struct DM35424_Function_Block *func_block,
01027                                           unsigned int channel,
01028                                           uint32_t *current_buffer,
01029                                           uint32_t *current_count);
01030
01031
01075 DM35424LIB_API
01076 int DM35424_Dma_Check_For_Error(struct DM35424_Board_Descriptor *handle,
01077                                  const struct DM35424_Function_Block *func_block,
01078                                  unsigned int channel,
01079                                  int *has_error);
01080
01081
01082
01128 DM35424LIB_API
01129 int DM35424_Dma_Buffer_Setup(struct DM35424_Board_Descriptor *handle,
01130                               const struct DM35424_Function_Block *func_block,
01131                               unsigned int channel,
01132                               unsigned int buffer,
01133                               uint8_t ctrl);
01134
01135
01192 DM35424LIB_API
01193 int DM35424_Dma_Buffer_Status(struct DM35424_Board_Descriptor *handle,
01194                               const struct DM35424_Function_Block *func_block,
01195                               unsigned int channel,
01196                               unsigned int buffer,
01197                               uint8_t *status,
01198                               uint8_t *control,
01199                               uint32_t *size);
01200
01201
01248
01249 DM35424LIB_API
01250 int DM35424_Dma_Check_Buffer_Used(struct DM35424_Board_Descriptor *handle,
01251                                    const struct DM35424_Function_Block *func_block,
01252                                    unsigned int channel,
01253                                    unsigned int buffer_num,
01254                                    int *is_used);
01255
01256
01257
01308 int
01309 DM35424_Dma_Find_Interrupt(struct DM35424_Board_Descriptor *handle,
01310                             const struct DM35424_Function_Block *func_block,
01311                             unsigned int *channel,
01312                             int *channel_complete,
01313                             int *channel_error);
01314
01315
01384 int
01385 DM35424_Dma_Clear_Interrupt(struct DM35424_Board_Descriptor *handle,
01386                              const struct DM35424_Function_Block *func_block,
01387                              unsigned int channel,
01388                              int clear_overflow,
01389                              int clear_underflow,
01390                              int clear_used,
01391                              int clear_invalid,
01392                              int clear_complete);
```

```

01393
01394
01439 int
01440 DM35424_Dma_Reset_Buffer(struct DM35424_Board_Descriptor *handle,
01441                          const struct DM35424_Function_Block *func_block,
01442                          unsigned int channel,
01443                          unsigned int buffer);
01444
01445
01446 DM35424_LIB_API
01447 int DM35424_Dma_Buffer_Get_Size(struct DM35424_Board_Descriptor *handle,
01448                                const struct DM35424_Function_Block *func_block,
01449                                unsigned int channel,
01450                                unsigned int buffer,
01451                                uint32_t *buffer_size);
01452
01453
01454 int DM35424_Dma_Buffer_Set_Size(struct DM35424_Board_Descriptor *handle,
01455                                const struct DM35424_Function_Block *func_block,
01456                                unsigned int channel,
01457                                unsigned int buffer,
01458                                uint32_t buffer_size);
01459
01463
01464
01465 #ifdef __cplusplus
01466 }
01467 #endif
01468
01469 #endif

```

6.41 include/dm35424_driver.h File Reference

Structures and defines for the DM35424 driver module.

```

#include <linux/pci.h>
#include <linux/spinlock.h>
#include <linux/types.h>

```

Data Structures

- struct [dm35424_pci_region](#)
DM35424 PCI region descriptor. This structure holds information about one of a device's PCI memory regions.
- struct [dm35424_dma_descriptor](#)
DM35424 DMA descriptor. This structure holds information about a single DMA buffer.
- struct [dm35424_device_descriptor](#)
DM35424 Device Descriptor. The identifying info for this particular board.

Macros

- #define **DM35424_MAX_NUM_DEVICES** 8
DM35424 Maximum number of devices.
- #define **DM35424_NUM_MINORS_PER_DEVICE** 1
DM35424 Number of minors per device.
- #define **DM35424_NAME_LENGTH** 200
DM35424 Max possible board name length.
- #define **DM35424_PCI_NUM_REGIONS** PCI_ROM_RESOURCE
Number of standard PCI regions.
- #define **DM35424_INT_QUEUE_SIZE** 256
Number of interrupts to hold in a queue for processing.

Enumerations

- enum [dm35424_pci_region_access_dir](#) { [DM35424_PCI_REGION_ACCESS_READ](#) = 0 , [DM35424_PCI_REGION_ACCESS_WRITE](#) }

Direction of access to standard PCI region.

Variables

- static struct file_operations [dm35424_file_ops](#)

Placeholder prototype for file ops struct.

6.41.1 Detailed Description

Structures and defines for the DM35424 driver module.

Id

[dm35424_driver.h](#) 150218 2025-10-13 15:58:55Z bkorpacz

Definition in file [dm35424_driver.h](#).

6.42 dm35424_driver.h

[Go to the documentation of this file.](#)

```

00001
00010
00011 //-----
00012 //  COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC.  ALL RIGHTS RESERVED.
00013 //
00014 //  This software package is dual-licensed.  Source code that is compiled for
00015 //  kernel mode execution is licensed under the GNU General Public License
00016 //  version 2.  For a copy of this license, refer to the file
00017 //  LICENSE GPLv2.TXT (which should be included with this software) or contact
00018 //  the Free Software Foundation.  Source code that is compiled for user mode
00019 //  execution is licensed under the RTD End-User Software License Agreement.
00020 //  For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00021 //  Technologies, Inc.  Using this software indicates agreement with the
00022 //  license terms listed above.
00023 //-----
00024
00025 #ifndef __DM35424_DRIVER_H__
00026 #define __DM35424_DRIVER_H__
00027
00028
00029 #include <linux/pci.h>
00030 #include <linux/spinlock.h>
00031 #include <linux/types.h>
00032
00033 /*=====
00034 Constants
00035 =====*/
00036
00041
00046 #define DM35424_MAX_NUM_DEVICES          8
00047
00052 #define DM35424_NUM_MINORS_PER_DEVICE 1
00053
00058 #define DM35424_NAME_LENGTH 200
00059
00064
00065 #define DM35424_PCI_NUM_REGIONS    PCI_ROM_RESOURCE
00066
00071 #define DM35424_INT_QUEUE_SIZE    256
00072
00076

```

```

00077
00078 /*=====
00079 Enumerations
00080 =====*/
00081
00086
00091
00092 enum dm35424_pci_region_access_dir {
00093
00097
00098     DM35424_PCI_REGION_ACCESS_READ = 0,
00099
00103
00104     DM35424_PCI_REGION_ACCESS_WRITE
00105 };
00106
00107
00111
00112
00113
00114 /*=====
00115 Structures
00116 =====*/
00117
00122
00128
00129 struct dm35424_pci_region {
00130
00134
00135     unsigned long io_addr;
00136
00140
00141     unsigned long length;
00142
00147
00148     unsigned long phys_addr;
00149
00154
00155     void __iomem *virt_addr;
00156
00162
00163     uint8_t allocated;
00164 };
00165
00171
00172 struct dm35424_dma_descriptor {
00173
00177     uint32_t fb_num;
00178
00182     int channel;
00183
00187     int buffer;
00188
00192     void *virt_addr;
00193
00194
00198     dma_addr_t bus_addr;
00199
00200
00204     unsigned int buffer_size;
00205
00206
00210     struct list_head list;
00211
00212
00213 };
00214
00215
00221
00222 struct dm35424_device_descriptor {
00223
00228
00229     char name[DM35424_NAME_LENGTH];
00230
00234     int device_index;
00235
00239
00240     struct dm35424_pci_region pci[PCI_ROM_RESOURCE];
00241
00245     struct pci_dev *pdev;
00246
00250
00251     spinlock_t device_lock;
00252
00256     struct cdev cdev;
00257
00262

```

```

00263         uint8_t reference_count;
00264
00268
00269         unsigned int irq_number;
00270
00271
00275
00276         uint8_t remove_isr_flag;
00277
00281
00282         wait_queue_head_t int_wait_queue;
00283
00287
00288         wait_queue_head_t dma_wait_queue;
00289
00293
00294         int interrupt_fb[DM35424_INT_QUEUE_SIZE];
00295
00296
00300
00301         unsigned int int_queue_missed;
00302
00306
00307         unsigned int int_queue_count;
00308
00312
00313         unsigned int int_queue_in_marker;
00314
00318
00319         unsigned int int_queue_out_marker;
00320
00324         struct list_head dma_descr_list;
00325
00329         unsigned int device_id;
00330     };
00331
00336 static struct file_operations dm35424_file_ops;
00337
00341
00342
00343 #endif
00344

```

6.43 include/dm35424_examples.h File Reference

Defines for the DM35424 Example programs. Commonly used constants for the example programs included with the software package.

Macros

- #define **BUFFER_VALID** 1
Boolean indicating buffer valid.
- #define **BUFFER_NO_VALID** 0
Boolean indicating buffer not valid.
- #define **BUFFER_HALT** 1
Boolean indicating buffer halt set.
- #define **BUFFER_NO_HALT** 0
Boolean indicating buffer halt not set.
- #define **BUFFER_LOOP** 1
Boolean indicating buffer loop set.
- #define **BUFFER_NO_LOOP** 0
Boolean indicating buffer loop not set.
- #define **BUFFER_INTERRUPT** 1
Boolean indicating buffer interrupt.
- #define **BUFFER_NO_INTERRUPT** 0
Boolean indicating no buffer interrupt.

- **#define BUFFER_PAUSE 1**
Boolean indicating buffer should pause when filled.
- **#define BUFFER_NO_PAUSE 0**
Boolean indicating buffer should not pause when filled.
- **#define IGNORE_USED 1**
Boolean indicating ignore used buffers.
- **#define NOT_IGNORE_USED 0**
Boolean indicating not ignore used buffers.
- **#define CLEAR_INTERRUPT 1**
Boolean indicating to clear an interrupt.
- **#define NO_CLEAR_INTERRUPT 0**
Boolean indicating to not clear an interrupt.
- **#define INTERRUPT_ENABLE 1**
Boolean indicating interrupt enable.
- **#define INTERRUPT_DISABLE 0**
Boolean indicating interrupt disable.
- **#define ERROR_INTR_ENABLE 1**
Boolean indicating error interrupt enable.
- **#define ERROR_INTR_DISABLE 0**
Boolean indicating error interrupt disable.
- **#define SYNCBUS_NONE 0**
Value indicating no Syncbus option was chosen.
- **#define SYNCBUS_MASTER 1**
Value indicating Syncbus Master was chosen.
- **#define SYNCBUS_SLAVE 2**
Value indicating Syncbus Slave was chosen.
- **#define CHANNEL_0 0**
Constant for selecting Channel 0.
- **#define CHANNEL_1 1**
Constant for selecting Channel 1.
- **#define CHANNEL_2 2**
Constant for selecting Channel 2.
- **#define CHANNEL_3 3**
Constant for selecting Channel 3.
- **#define BUFFER_0 0**
Constant for selecting Buffer 0.
- **#define BUFFER_1 1**
Constant for selecting Buffer 1.
- **#define ADC_0 0**
Constant for selecting ADC 0.
- **#define ADC_1 1**
Constant for selecting ADC 1.
- **#define DAC_0 0**
Constant for selecting DAC 0.
- **#define DAC_1 1**
Constant for selecting DAC 1.
- **#define DAC_2 2**
Constant for selecting DAC 2.
- **#define DAC_3 3**
Constant for selecting DAC 3.
- **#define REF_0 0**

- Constant for selecting REF 0.*
 - **#define REF_1** 1
- Constant for selecting REF 1.*
 - **#define DIO_0** 0
- Constant for selecting DIO 0.*
 - **#define ADIO_0** 0
- Constant for selecting ADIO 0.*
 - **#define ENABLED** 1
- Constant to indicate an Enabled value.*
 - **#define DISABLED** 0
- Constant to indicate a Disabled value.*

Enumerations

- enum [Help_Options](#) {
[HELP_OPTION](#) = 1 , [MINOR_OPTION](#) , [RATE_OPTION](#) , [CHANNELS_OPTION](#) ,
[FILE_OPTION](#) , [START_OPTION](#) , [WAVE_OPTION](#) , [TEST_OPTION](#) ,
[NOSTOP_OPTION](#) , [SYNCBUS_OPTION](#) , [DUMP_OPTION](#) , [HOURS_OPTION](#) ,
[OUTPUT_RMS_OPTION](#) , [OUTPUT_ADC_OPTION](#) , [ADC_NUM_OPTION](#) , [DAC_NUM_OPTION](#) ,
[ADC_OPTION](#) , [DAC_OPTION](#) , [PATTERN_OPTION](#) , [SAMPLES_OPTION](#) ,
[MODE_OPTION](#) , [AD_MODE_OPTION](#) , [REF_NUM_OPTION](#) , [BINARY_OPTION](#) ,
[SENDER_OPTION](#) , [RECEIVER_OPTION](#) , [RANGE_OPTION](#) , [REFILL_FIFO_OPTION](#) ,
[LOW_THRESHOLD_OPTION](#) , [PORT_OPTION](#) , [BAUD_OPTION](#) , [EXTERNAL_OPTION](#) ,
[SIZE_OPTION](#) , [VERBOSE_OPTION](#) , [USER_ID_OPTION](#) , [COUNT_OPTION](#) ,
[NUM_OPTION](#) , [SYNC_TERM_OPTION](#) , [BIN2TXT_OPTION](#) , [STORE_OPTION](#) ,
[TERM_OPTION](#) , [REFCLK_OPTION](#) , [OFILE_OPTION](#) , [PACKED_OPTION](#) ,
[MASTER_OPTION](#) , [SLAVE_OPTION](#) , [SYNC_CONN_OPTION](#) }
Constants used for parsing command line parameters of example programs.

6.43.1 Detailed Description

Defines for the DM35424 Example programs. Commonly used constants for the example programs included with the software package.

Id

[dm35424_examples.h](#) 114740 2018-07-12 14:41:17Z prucz

Definition in file [dm35424_examples.h](#).

6.44 dm35424_examples.h

[Go to the documentation of this file.](#)

```
00001
00011
00012 //-----
00013 //  COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC.  ALL RIGHTS RESERVED.
00014 //
00015 //  This software package is dual-licensed.  Source code that is compiled for
00016 //  kernel mode execution is licensed under the GNU General Public License
00017 //  version 2.  For a copy of this license, refer to the file
00018 //  LICENSE_GPLv2.TXT (which should be included with this software) or contact
00019 //  the Free Software Foundation.  Source code that is compiled for user mode
```

```
00020 // execution is licensed under the RTD End-User Software License Agreement.
00021 // For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00022 // Technologies, Inc. Using this software indicates agreement with the
00023 // license terms listed above.
00024 //-----
00025
00026
00027 #ifndef __DM35424_EXAMPLES_H__
00028 #define __DM35424_EXAMPLES_H__
00029
00034
00043 enum Help_Options {
00048     HELP_OPTION = 1,
00049
00054     MINOR_OPTION,
00055
00060     RATE_OPTION,
00061
00066     CHANNELS_OPTION,
00067
00072     FILE_OPTION,
00073
00078     START_OPTION,
00079
00084     WAVE_OPTION,
00085
00090     TEST_OPTION,
00091
00096     NOSTOP_OPTION,
00097
00102     SYNCBUS_OPTION,
00103
00108     DUMP_OPTION,
00109
00114     HOURS_OPTION,
00115
00120     OUTPUT_RMS_OPTION,
00121
00126     OUTPUT_ADC_OPTION,
00127
00132     ADC_NUM_OPTION,
00133
00138     DAC_NUM_OPTION,
00139
00144     ADC_OPTION,
00145
00150     DAC_OPTION,
00151
00156     PATTERN_OPTION,
00157
00162     SAMPLES_OPTION,
00163
00168     MODE_OPTION,
00169
00174     AD_MODE_OPTION,
00175
00180     REF_NUM_OPTION,
00181
00186     BINARY_OPTION,
00187
00192     SENDER_OPTION,
00193
00198     RECEIVER_OPTION,
00199
00204     RANGE_OPTION,
00205
00210     REFILL_FIFO_OPTION,
00211
00216     LOW_THRESHOLD_OPTION,
00217
00222     PORT_OPTION,
00223
00228     BAUD_OPTION,
00229
00234     EXTERNAL_OPTION,
00235
00240     SIZE_OPTION,
00241
00246     VERBOSE_OPTION,
00247
00252     USER_ID_OPTION,
00253
00258     COUNT_OPTION,
00259
00264     NUM_OPTION,
00265
00270     SYNC_TERM_OPTION,
```

```

00271
00276     BIN2TXT_OPTION,
00277
00282     STORE_OPTION,
00283
00289     TERM_OPTION,
00290
00296     REFCLK_OPTION,
00297
00303     OFILE_OPTION,
00304
00310     PACKED_OPTION,
00311
00316     MASTER_OPTION,
00317
00322     SLAVE_OPTION,
00323
00328     SYNC_CONN_OPTION,
00329 };
00330
00335 #define BUFFER_VALID          1
00336
00341 #define BUFFER_NO_VALID      0
00342
00347 #define BUFFER_HALT          1
00348
00353 #define BUFFER_NO_HALT       0
00354
00359 #define BUFFER_LOOP          1
00360
00365 #define BUFFER_NO_LOOP       0
00366
00371 #define BUFFER_INTERRUPT      1
00372
00377 #define BUFFER_NO_INTERRUPT   0
00378
00383 #define BUFFER_PAUSE          1
00384
00389 #define BUFFER_NO_PAUSE       0
00390
00395 #define IGNORE_USED           1
00396
00401 #define NOT_IGNORE_USED       0
00402
00407 #define CLEAR_INTERRUPT       1
00408
00413 #define NO_CLEAR_INTERRUPT     0
00414
00419 #define INTERRUPT_ENABLE      1
00420
00425 #define INTERRUPT_DISABLE     0
00426
00431 #define ERROR_INTR_ENABLE     1
00432
00437 #define ERROR_INTR_DISABLE    0
00438
00443 #define SYNCBUS_NONE         0
00444
00449 #define SYNCBUS_MASTER        1
00450
00455 #define SYNCBUS_SLAVE         2
00456
00461 #define CHANNEL_0             0
00462
00467 #define CHANNEL_1             1
00468
00473 #define CHANNEL_2             2
00474
00479 #define CHANNEL_3             3
00480
00485 #define BUFFER_0              0
00486
00491 #define BUFFER_1              1
00492
00497 #define ADC_0                 0
00498
00503 #define ADC_1                 1
00504
00509 #define DAC_0                 0
00510
00515 #define DAC_1                 1
00516
00521 #define DAC_2                 2
00522
00527 #define DAC_3                 3
00528
00533 #define REF_0                 0

```

```

00534
00539 #define REF_1      1
00540
00545 #define DIO_0       0
00546
00551 #define ADIO_0     0
00552
00557 #define ENABLED      1
00558
00563 #define DISABLED     0
00564
00565
00569 #endif

```

6.45 include/dm35424_gbc_library.h File Reference

Definitions for the DM35424 Board Library, a library for accessing the board registers.

```

#include <stdint.h>
#include <time.h>
#include "dm35424_board_access.h"

```

Macros

- #define [CLK_40MHZ](#) 40000000

Functions

- [DM35424LIB_API](#) int [DM35424_Gbc_Board_Reset](#) (struct [DM35424_Board_Descriptor](#) *handle)
Write the reset value to the correct register to initiate a board-level reset.
- [DM35424LIB_API](#) int [DM35424_Gbc_Ack_Interrupt](#) (struct [DM35424_Board_Descriptor](#) *handle)
Send an End-Of-Interrupt acknowledgement to the board. This will cause any pending interrupts to re-issue. This is a protection against missing interrupts while in the interrupt handler.
- [DM35424LIB_API](#) int [DM35424_Function_Block_Open](#) (struct [DM35424_Board_Descriptor](#) *handle, unsigned int number, struct [DM35424_Function_Block](#) *func_block)
Open a specific function block. Nothing is opened in a file sense, but the memory location for the function block is read and certain important values are read. A function block descriptor is allocated to hold the data that will be used every time this function block is accessed.
- [DM35424LIB_API](#) int [DM35424_Function_Block_Open_Module](#) (struct [DM35424_Board_Descriptor](#) *handle, uint32_t fb_type, unsigned int number_of_type, struct [DM35424_Function_Block](#) *func_block)
Open a specific function block module. This is the same as opening a function block, except we are looking for a function block with a specific type. This is the method you would use to open the 2nd ADC, for example.
- [DM35424LIB_API](#) int [DM35424_Gbc_Get_Format](#) (struct [DM35424_Board_Descriptor](#) *handle, uint8_t *format_id)
Get the format ID of the board.
- [DM35424LIB_API](#) int [DM35424_Gbc_Get_Revision](#) (struct [DM35424_Board_Descriptor](#) *handle, uint8_t *rev)
Get the PDP revision number of the board.
- [DM35424LIB_API](#) int [DM35424_Gbc_Get_Pdp_Number](#) (struct [DM35424_Board_Descriptor](#) *handle, uint32_t *pdp_num)
Get PDP Number of the board.
- [DM35424LIB_API](#) int [DM35424_Gbc_Get_Fpga_Build](#) (struct [DM35424_Board_Descriptor](#) *handle, uint32_t *fpga_build)
Get the FPGA Build number of the board.
- [DM35424LIB_API](#) int [DM35424_Gbc_Get_Sys_Clock_Freq](#) (struct [DM35424_Board_Descriptor](#) *handle, uint32_t *clock_freq, int *is_std_clk)
Get the measured frequency of the system clock of the board.

6.45.1 Detailed Description

Definitions for the DM35424 Board Library, a library for accessing the board registers.

Id

[dm35424_gbc_library.h](#) 103741 2016-10-17 20:35:58Z rgroner

Definition in file [dm35424_gbc_library.h](#).

6.46 dm35424_gbc_library.h

[Go to the documentation of this file.](#)

```

00001
00011
00012 //-----
00013 //  COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC.  ALL RIGHTS RESERVED.
00014 //
00015 //  This software package is dual-licensed.  Source code that is compiled for
00016 //  kernel mode execution is licensed under the GNU General Public License
00017 //  version 2.  For a copy of this license, refer to the file
00018 //  LICENSE_GPLv2.TXT (which should be included with this software) or contact
00019 //  the Free Software Foundation.  Source code that is compiled for user mode
00020 //  execution is licensed under the RTD End-User Software License Agreement.
00021 //  For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00022 //  Technologies, Inc.  Using this software indicates agreement with the
00023 //  license terms listed above.
00024 //-----
00025
00026
00027 #ifndef _DM35424_BOARD_LIBRARY__H_
00028 #define _DM35424_BOARD_LIBRARY__H_
00029
00030
00031 #include <stdint.h>
00032 #include <time.h>
00033
00034 #include "dm35424_board_access.h"
00035
00036
00037 #ifdef __cplusplus
00038 extern "C" {
00039 #endif
00040
00041
00046
00047 #define CLOSE_TO_40MHZ(x)      ((x) <= 42000000 && (x) >= 38000000)
00048
00052 #define CLK_40MHZ              40000000
00053
00054 #define CLOSE_TO_54MHZ(x)      ((x) <= 56700000 && (x) >= 51300000)
00055
00056 #define CLK_54MHZ              54000000
00057
00058 #define CLOSE_TO_100MHZ(x)      ((x) <= 105000000 && (x) >= 95000000)
00059
00060 #define CLK_100MHZ             100000000
00061
00062 #define CLOSE_TO_57_6MHZ(x)      ((x) <= 63360000 && (x) >= 51840000)
00063
00064 #define CLK_57_6MHZ            57600000
00065
00066 #define CLK_50MHZ              50000000
00067
00068 #define CLOSE_TO_50MHZ(x)      ((x) <= 52500000 && (x) >= 47500000)
00072
00073
00078
00079 /*****
00080 * Board Level Functions
00081 *****/
00082
00083
00105 DM35424LIB_API
00106 int DM35424_Gbc_Board_Reset(struct DM35424_Board_Descriptor *handle);

```

```

00107
00108
00131 DM35424LIB_API
00132 int DM35424_Gbc_Ack_Interrupt(struct DM35424_Board_Descriptor *handle);
00133
00134
00135
00172 DM35424LIB_API
00173 int DM35424_Function_Block_Open(struct DM35424_Board_Descriptor *handle,
00174                                unsigned int number,
00175                                struct DM35424_Function_Block *func_block);
00176
00177
00218 DM35424LIB_API
00219 int DM35424_Function_Block_Open_Module(struct DM35424_Board_Descriptor *handle,
00220                                       uint32_t fb_type,
00221                                       unsigned int number_of_type,
00222                                       struct DM35424_Function_Block *func_block);
00223
00224
00251 DM35424LIB_API
00252 int DM35424_Gbc_Get_Format(struct DM35424_Board_Descriptor *handle,
00253                            uint8_t *format_id);
00254
00255
00281 DM35424LIB_API
00282 int DM35424_Gbc_Get_Revision(struct DM35424_Board_Descriptor *handle,
00283                              uint8_t *rev);
00284
00285
00311 DM35424LIB_API
00312 int DM35424_Gbc_Get_Pdp_Number(struct DM35424_Board_Descriptor *handle,
00313                                uint32_t *pdp_num);
00314
00315
00341 DM35424LIB_API
00342 int DM35424_Gbc_Get_Fpga_Build(struct DM35424_Board_Descriptor *handle,
00343                                uint32_t *fpga_build);
00344
00345
00381 DM35424LIB_API
00382 int DM35424_Gbc_Get_Sys_Clock_Freq(struct DM35424_Board_Descriptor *handle,
00383                                     uint32_t *clock_freq, int *is_std_clk);
00384
00385
00386
00390
00391 #ifdef __cplusplus
00392 }
00393 #endif
00394
00395 #endif

```

6.47 include/dm35424_ioctl.h File Reference

DM35424 Low level ioctl() request descriptor structure and request code definitions.

```

#include <linux/types.h>
#include <linux/ioctl.h>

```

Macros

- **#define DM35424_IOCTL_MAGIC 'D'**
Unique 8-bit value used to generate unique ioctl() request codes.
- **#define DM35424_IOCTL_REQUEST_BASE 0x00**
First ioctl() request number.
- **#define DM35424_IOCTL_REGION_READ**
ioctl() request code for reading from a PCI region
- **#define DM35424_IOCTL_REGION_WRITE**

- ioctl()* request code for writing to a PCI region
- **#define DM35424_IOCTL_REGION_MODIFY**
ioctl() request code for PCI region read/modify/write
- **#define DM35424_IOCTL_DMA_FUNCTION**
ioctl() request code for DMA function
- **#define DM35424_IOCTL_WAKEUP**
ioctl() request code for User ISR thread wake up
- **#define DM35424_IOCTL_INTERRUPT_GET**
ioctl() request code to retrieve interrupt status information
- **#define DM35424_IOCTL_GET_DEVICE_ID**
ioctl() request code to retrieve PCIe device ID

6.47.1 Detailed Description

DM35424 Low level ioctl() request descriptor structure and request code definitions.

Id

[dm35424_ioctl.h](#) 150217 2025-10-13 15:58:41Z bkorpacz

Definition in file [dm35424_ioctl.h](#).

6.48 dm35424_ioctl.h

[Go to the documentation of this file.](#)

```

00001
00010
00011 //-----
00012 //  COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC.  ALL RIGHTS RESERVED.
00013 //
00014 //  This software package is dual-licensed.  Source code that is compiled for
00015 //  kernel mode execution is licensed under the GNU General Public License
00016 //  version 2.  For a copy of this license, refer to the file
00017 //  LICENSE GPLv2.TXT (which should be included with this software) or contact
00018 //  the Free Software Foundation.  Source code that is compiled for user mode
00019 //  execution is licensed under the RTD End-User Software License Agreement.
00020 //  For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00021 //  Technologies, Inc.  Using this software indicates agreement with the
00022 //  license terms listed above.
00023 //-----
00024
00025 #ifndef __DM35424_IOCTL_H__
00026 #define __DM35424_IOCTL_H__
00027
00028 #include <linux/types.h>
00029 #include <linux/ioctl.h>
00030
00031
00032
00033 /*=====
00034 Macros
00035 =====*/
00036
00041
00046
00047 #define DM35424_IOCTL_MAGIC          'D'
00048
00053
00054 #define DM35424_IOCTL_REQUEST_BASE    0x00
00055
00060
00061 #define DM35424_IOCTL_REGION_READ \
00062     _IOR( \
00063         DM35424_IOCTL_MAGIC, \

```

```

00064         (DM35424_IOCTL_REQUEST_BASE + 1), \
00065         union dm35424_ioctl_argument)
00066
00071
00072 #define DM35424_IOCTL_REGION_WRITE \
00073     _IOW( \
00074         DM35424_IOCTL_MAGIC, \
00075         (DM35424_IOCTL_REQUEST_BASE + 2), \
00076         union dm35424_ioctl_argument)
00077
00082
00083 #define DM35424_IOCTL_REGION_MODIFY \
00084     _IOWR( \
00085         DM35424_IOCTL_MAGIC, \
00086         (DM35424_IOCTL_REQUEST_BASE + 3), \
00087         union dm35424_ioctl_argument)
00088
00089
00094
00095 #define DM35424_IOCTL_DMA_FUNCTION \
00096     _IOW( \
00097         DM35424_IOCTL_MAGIC, \
00098         (DM35424_IOCTL_REQUEST_BASE + 4), \
00099         union dm35424_ioctl_argument)
00100
00105
00106 #define DM35424_IOCTL_WAKEUP \
00107     _IOW( \
00108         DM35424_IOCTL_MAGIC, \
00109         (DM35424_IOCTL_REQUEST_BASE + 5), \
00110         union dm35424_ioctl_argument)
00111
00116
00117 #define DM35424_IOCTL_INTERRUPT_GET \
00118     _IOWR( \
00119         DM35424_IOCTL_MAGIC, \
00120         (DM35424_IOCTL_REQUEST_BASE + 6), \
00121         union dm35424_ioctl_argument)
00122
00127 #define DM35424_IOCTL_GET_DEVICE_ID \
00128     _IOR( \
00129         DM35424_IOCTL_MAGIC, \
00130         (DM35424_IOCTL_REQUEST_BASE + 7), \
00131         uint16_t)
00132
00136
00137
00138 #endif /* __dm35424_ioctl_h__ */

```

6.49 include/dm35424_os.h File Reference

Function declarations for the DM35424 that are Linux specific.

```
#include <pthread.h>
```

Data Structures

- struct [DM35424_Board_Descriptor](#)

DM35424 board descriptor. This structure holds information about the board as a whole. It holds the file descriptor and ISR callback function, if applicable.

Functions

- int [DM35424_Dma_Initialize](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, unsigned int num_buffers, uint32_t buffer_size)

Initialize the DMA channel and prepare it for data. Interrupts are disabled, error conditions are cleared, buffers are allocated in kernel space and their status and controls are cleared.

- int [DM35424_Dma_Read](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, unsigned int buffer_to_read_from, uint32_t buffer_size, void *local_buffer_ptr)
Read data from the DMA buffer. Data is copied from kernel buffers to local user-space buffers.
- int [DM35424_Dma_Write](#) (struct [DM35424_Board_Descriptor](#) *handle, const struct [DM35424_Function_Block](#) *func_block, unsigned int channel, unsigned int buffer_to_write_to, uint32_t buffer_size, void *local_buffer_ptr)
Write data to the DMA buffer. Data is copied from local user buffers to kernel buffers.
- int [DM35424_General_RemoveISR](#) (struct [DM35424_Board_Descriptor](#) *handle)
Remove the ISR from the system interrupt.
- void * [DM35424_General_WaitForInterrupt](#) (void *ptr)
Loop/Poll and wait for an interrupt to happen, then take action.
- int [DM35424_General_InstallISR](#) (struct [DM35424_Board_Descriptor](#) *handle, void(*isr_fnct))
Start a thread that will sit and wait for an interrupt from the board, and call the user ISR when it happens.
- int [DM35424_General_SetISRPriority](#) (struct [DM35424_Board_Descriptor](#) *handle, int priority)
Set the priority of the user ISR thread.

6.49.1 Detailed Description

Function declarations for the DM35424 that are Linux specific.

Id

[dm35424_os.h](#) 154351 2026-05-14 12:16:17Z asutton

Definition in file [dm35424_os.h](#).

6.50 dm35424_os.h

[Go to the documentation of this file.](#)

```

00001
00009
00010 //-----
00011 //  COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC.  ALL RIGHTS RESERVED.
00012 //
00013 //  This software package is dual-licensed.  Source code that is compiled for
00014 //  kernel mode execution is licensed under the GNU General Public License
00015 //  version 2.  For a copy of this license, refer to the file
00016 //  LICENSE GPLv2.TXT (which should be included with this software) or contact
00017 //  the Free Software Foundation.  Source code that is compiled for user mode
00018 //  execution is licensed under the RTD End-User Software License Agreement.
00019 //  For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00020 //  Technologies, Inc.  Using this software indicates agreement with the
00021 //  license terms listed above.
00022 //-----
00023
00024
00025 #ifndef _DM35424_BOARD_OS__H_
00026 #define _DM35424_BOARD_OS__H_
00027
00028 #include <pthread.h>
00029
00030 // This forward declaration is made so that board_access.h
00031 // does not need to be included, which would causes circular dependencies.
00032 struct DM35424_Function_Block;
00033
00034 #ifdef __cplusplus
00035 extern "C" {
00036 #endif
00037
00042
00049

```

```

00050 struct DM35424_Board_Descriptor {
00051
00055     int file_descriptor;
00056
00060     void (*isr) (struct dm35424_ioctl_interrupt_info_request info_req);
00061
00065     pthread_t pid;
00066
00067 };
00068
00069
00119 int
00120 DM35424_Dma_Initialize(struct DM35424_Board_Descriptor *handle,
00121                       const struct DM35424_Function_Block *func_block,
00122                       unsigned int channel,
00123                       unsigned int num_buffers,
00124                       uint32_t buffer_size);
00125
00126
00181 int
00182 DM35424_Dma_Read(struct DM35424_Board_Descriptor *handle,
00183                  const struct DM35424_Function_Block *func_block,
00184                  unsigned int channel,
00185                  unsigned int buffer_to_read_from,
00186                  uint32_t buffer_size,
00187                  void *local_buffer_ptr);
00188
00189
00190
00245 int
00246 DM35424_Dma_Write(struct DM35424_Board_Descriptor *handle,
00247                   const struct DM35424_Function_Block *func_block,
00248                   unsigned int channel,
00249                   unsigned int buffer_to_write_to,
00250                   uint32_t buffer_size,
00251                   void *local_buffer_ptr);
00252
00253
00254
00278 int DM35424_General_RemoveISR(struct DM35424_Board_Descriptor *handle);
00279
00280
00305 void *DM35424_General_WaitForInterrupt(void *ptr);
00306
00307
00337 int
00338 DM35424_General_InstallISR(struct DM35424_Board_Descriptor *handle, void (*isr_fnct));
00339
00340
00372 int DM35424_General_SetISRPriority(struct DM35424_Board_Descriptor
00373                                    *handle, int priority);
00374
00378
00379 #ifdef __cplusplus
00380 }
00381 #endif
00382
00383 #endif

```

6.51 include/dm35424_ref_adjust_library.h File Reference

Definitions for the DM35424 Reference Adjustment Library.

```
#include "dm35424_gbc_library.h"
```

Macros

- **#define DM35424_REF_ADJUST_SPI_BUSY 0x00**
Register value for SPI Interface is busy.
- **#define DM35424_REF_ADJUST_SPI_READY 0x01**
Register value for SPI Interface is ready.
- **#define DM35424_REF_ADJUST_START_TRANS 0x01**

- Register value for starting the SPI transaction.*
- **#define DM35424_REF_ADJUST_WRITE_ADC_VOLATILE** 0x0100
Register value for writing to the ADC Volatile memory.
- **#define DM35424_REF_ADJUST_WRITE_DAC_VOLATILE** 0x0200
Register value for writing to the DAC Volatile memory.
- **#define DM35424_REF_ADJUST_WRITE_ADC_NON_VOLATILE** 0x1100
Register value for writing to the ADC Non-Volatile memory.
- **#define DM35424_REF_ADJUST_WRITE_DAC_NON_VOLATILE** 0x1200
Register value for writing to the DAC Non-Volatile memory.
- **#define DM35424_REF_ADJUST_COPY_ADC_VOL_TO_NON** 0x2100
Register value for copying ADC data from Volatile to Non-Volatile.
- **#define DM35424_REF_ADJUST_COPY_DAC_VOL_TO_NON** 0x2200
Register value for copying DAC data from Volatile to Non-Volatile.
- **#define DM35424_REF_ADJUST_COPY_BOTH_VOL_TO_NON** 0x2300
Register value for copying ADC and DAC data from Volatile to Non-Volatile.
- **#define DM35424_REF_ADJUST_COPY_ADC_NON_TO_VOL** 0x3100
Register value for copying ADC data from Non-Volatile to Volatile.
- **#define DM35424_REF_ADJUST_COPY_DAC_NON_TO_VOL** 0x3200
Register value for copying DAC data from Non-Volatile to Volatile.
- **#define DM35424_REF_ADJUST_COPY_BOTH_NON_TO_VOL** 0x3300
Register value for copying ADC and DAC data from Non-Volatile to Volatile.

Enumerations

- enum [DM35424_Copy_Directions](#) {
[DM35424_ADC_VOL_TO_NON_VOL](#) , [DM35424_DAC_VOL_TO_NON_VOL](#) , [DM35424_BOTH_VOL_TO_NON_VOL](#)
[DM35424_ADC_NON_VOL_TO_VOL](#) ,
[DM35424_DAC_NON_VOL_TO_VOL](#) , [DM35424_BOTH_NON_VOL_TO_VOL](#) }
Direction of Reference Adjustment data copy action.

Functions

- [DM35424LIB_API](#) int [DM35424_Ref_Adjust_Open](#) (struct [DM35424_Board_Descriptor](#) *handle, unsigned int ordinal_to_open, struct [DM35424_Function_Block](#) *fb_temp)
Open the reference adjustment function block, getting address values that will be used later by other library functions.
- [DM35424LIB_API](#) int [DM35424_Ref_Adjust_Write_Adc_To_Volatile](#) (struct [DM35424_Board_Descriptor](#) *handle, struct [DM35424_Function_Block](#) *fb, uint8_t adjustment)
Write the ADC Reference Adjustment value to volatile memory.
- [DM35424LIB_API](#) int [DM35424_Ref_Adjust_Write_Adc_To_NonVolatile](#) (struct [DM35424_Board_Descriptor](#) *handle, struct [DM35424_Function_Block](#) *fb, uint8_t adjustment)
Write the ADC Reference Adjustment value to non-volatile memory.
- [DM35424LIB_API](#) int [DM35424_Ref_Adjust_Write_Dac_To_Volatile](#) (struct [DM35424_Board_Descriptor](#) *handle, struct [DM35424_Function_Block](#) *fb, uint8_t adjustment)
Write the DAC Reference Adjustment value to volatile memory.
- [DM35424LIB_API](#) int [DM35424_Ref_Adjust_Write_Dac_To_NonVolatile](#) (struct [DM35424_Board_Descriptor](#) *handle, struct [DM35424_Function_Block](#) *fb, uint8_t adjustment)
Write the DAC Reference Adjustment value to non-volatile memory.
- [DM35424LIB_API](#) int [DM35424_Ref_Adjust_Copy_Data](#) (struct [DM35424_Board_Descriptor](#) *handle, struct [DM35424_Function_Block](#) *fb, enum [DM35424_Copy_Directions](#) direction)
Copy the reference adjustment data from volatile to non-volatile, or vice versa.

6.51.1 Detailed Description

Definitions for the DM35424 Reference Adjustment Library.

Id

[dm35424_ref_adjust_library.h](#) 60276 2012-06-05 16:04:15Z rgroner

Definition in file [dm35424_ref_adjust_library.h](#).

6.52 dm35424_ref_adjust_library.h

[Go to the documentation of this file.](#)

```

00001
00009
00010 //-----
00011 //  COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC.  ALL RIGHTS RESERVED.
00012 //
00013 //  This software package is dual-licensed.  Source code that is compiled for
00014 //  kernel mode execution is licensed under the GNU General Public License
00015 //  version 2.  For a copy of this license, refer to the file
00016 //  LICENSE_GPLv2.TXT (which should be included with this software) or contact
00017 //  the Free Software Foundation.  Source code that is compiled for user mode
00018 //  execution is licensed under the RTD End-User Software License Agreement.
00019 //  For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00020 //  Technologies, Inc.  Using this software indicates agreement with the
00021 //  license terms listed above.
00022 //-----
00023
00024
00025 #ifndef _DM35424_REF_ADJUST_LIBRARY_H_
00026 #define _DM35424_REF_ADJUST_LIBRARY_H_
00027
00028
00029 #include "dm35424_gbc_library.h"
00030
00031 #ifdef __cplusplus
00032 extern "C" {
00033 #endif
00034
00035
00040
00041 /*=====
00042 Constants
00043 =====*/
00048 #define DM35424_REF_ADJUST_SPI_BUSY          0x00
00049
00054 #define DM35424_REF_ADJUST_SPI_READY        0x01
00055
00060 #define DM35424_REF_ADJUST_START_TRANS      0x01
00061
00066 #define DM35424_REF_ADJUST_WRITE_ADC_VOLATILE  0x0100
00067
00072 #define DM35424_REF_ADJUST_WRITE_DAC_VOLATILE  0x0200
00073
00078 #define DM35424_REF_ADJUST_WRITE_ADC_NON_VOLATILE  0x1100
00079
00084 #define DM35424_REF_ADJUST_WRITE_DAC_NON_VOLATILE  0x1200
00085
00090 #define DM35424_REF_ADJUST_COPY_ADC_VOL_TO_NON  0x2100
00091
00096 #define DM35424_REF_ADJUST_COPY_DAC_VOL_TO_NON  0x2200
00097
00103 #define DM35424_REF_ADJUST_COPY_BOTH_VOL_TO_NON  0x2300
00104
00109 #define DM35424_REF_ADJUST_COPY_ADC_NON_TO_VOL  0x3100
00110
00115 #define DM35424_REF_ADJUST_COPY_DAC_NON_TO_VOL  0x3200
00116
00121 #define DM35424_REF_ADJUST_COPY_BOTH_NON_TO_VOL  0x3300
00122
00123
00124
00129 enum DM35424_Copy_Directions {
00130

```

```

00134         DM35424_ADC_VOL_TO_NON_VOL,
00135
00139         DM35424_DAC_VOL_TO_NON_VOL,
00140
00144         DM35424_BOTH_VOL_TO_NON_VOL,
00145
00149         DM35424_ADC_NON_VOL_TO_VOL,
00150
00154         DM35424_DAC_NON_VOL_TO_VOL,
00155
00159         DM35424_BOTH_NON_VOL_TO_VOL
00160
00161     };
00162
00166
00167
00172
00173
00206 DM35424LIB_API
00207 int DM35424_Ref_Adjust_Open(struct DM35424_Board_Descriptor *handle,
00208                             unsigned int ordinal_to_open,
00209                             struct DM35424_Function_Block *fb_temp);
00210
00211
00212
00245 DM35424LIB_API
00246 int DM35424_Ref_Adjust_Write_Adc_To_Volatile(struct DM35424_Board_Descriptor *handle,
00247                                               struct DM35424_Function_Block *fb,
00248                                               uint8_t adjustment);
00249
00250
00283 DM35424LIB_API
00284 int DM35424_Ref_Adjust_Write_Adc_To_NonVolatile(struct DM35424_Board_Descriptor *handle,
00285                                                  struct DM35424_Function_Block *fb,
00286                                                  uint8_t adjustment);
00287
00288
00321 DM35424LIB_API
00322 int DM35424_Ref_Adjust_Write_Dac_To_Volatile(struct DM35424_Board_Descriptor *handle,
00323                                               struct DM35424_Function_Block *fb,
00324                                               uint8_t adjustment);
00325
00326
00359 DM35424LIB_API
00360 int DM35424_Ref_Adjust_Write_Dac_To_NonVolatile(struct DM35424_Board_Descriptor *handle,
00361                                                  struct DM35424_Function_Block *fb,
00362                                                  uint8_t adjustment);
00363
00364
00398 DM35424LIB_API
00399 int DM35424_Ref_Adjust_Copy_Data(struct DM35424_Board_Descriptor *handle,
00400                                  struct DM35424_Function_Block *fb,
00401                                  enum DM35424_Copy_Directions direction);
00402
00403
00404
00408
00409 #ifdef __cplusplus
00410 }
00411 #endif
00412
00413 #endif

```

6.53 include/dm35424_registers.h File Reference

Defines for the DM35424 Registers (Offsets).

Macros

- **#define DM35424_OFFSET_GBC_FORMAT** 0x00
Offset to General Board Control (BAR0) Format ID register.
- **#define DM35424_OFFSET_GBC_REV** 0x01
Offset to General Board Control (BAR0) Format ID register.
- **#define DM35424_OFFSET_GBC_END_INTERRUPT** 0x02

- Offset to General Board Control (BAR0) EOI (End of Interrupt) register.*

 - #define **DM35424_OFFSET_GBC_BOARD_RESET** 0x03
- Offset to General Board Control (BAR0) Board Reset register.*

 - #define **DM35424_OFFSET_GBC_PDP_NUMBER** 0x04
- Offset to General Board Control (BAR0) PDP Number register.*

 - #define **DM35424_OFFSET_GBC_FPGA_BUILD** 0x08
- Offset to General Board Control (BAR0) FPGA Build register.*

 - #define **DM35424_OFFSET_GBC_SYS_CLK_FREQ** 0x0c
- Offset to General Board Control (BAR0) System Clock register.*

 - #define **DM35424_OFFSET_GBC_IRQ_STATUS** 0x10
- Offset to General Board Control (BAR0) IRQ Status register. Each bit corresponds to a function block.*

 - #define **DM35424_OFFSET_GBC_DMA_IRQ_STATUS** 0x18
- Offset to General Board Control (BAR0) DMA IRQ Status register. Each bit corresponds to a function block.*

 - #define **DM35424_OFFSET_GBC_FB_START** 0x20
- Offset to the beginning of the Function Blocks section of the GBC.*

 - #define **DM35424_GBC_FB_BLK_SIZE** 0x10
- Size of the function block entries in the GBC.*

 - #define **DM35424_OFFSET_GBC_FB_ID** 0x00
- Offset to Function Block ID, from the start of the function block section.*

 - #define **DM35424_FB_ID_TYPE_MASK** 0x0000FFFF
- Bit mask for TYPE portion of FB ID.*

 - #define **DM35424_FB_ID_SUBTYPE_MASK** 0x00FF0000
- Bit mask for SUBTYPE portion of FB ID.*

 - #define **DM35424_FB_ID_TYPE_REV_MASK** 0xFF000000
- Bit mask for TYPE REV portion of FB ID.*

 - #define **DM35424_OFFSET_GBC_FB_OFFSET** 0x04
- Offset to the FB Offset in the GBC, from the start of the FB data block.*

 - #define **DM35424_OFFSET_GBC_FB_DMA_OFFSET** 0x08
- Offset to the FB DMA Offset in the GBC, from the start of the FB data block.*

 - #define **DM35424_OFFSET_DMA_ACTION** 0x00
- Offset to the DMA Action Register (BAR2).*

 - #define **DM35424_OFFSET_DMA_SETUP** 0x01
- Offset to the DMA Setup Register (BAR2).*

 - #define **DM35424_OFFSET_DMA_STAT_OVERFLOW** 0x02
- Offset to the DMA Status (Overflow) Register (BAR2).*

 - #define **DM35424_OFFSET_DMA_STAT_UNDERFLOW** 0x03
- Offset to the DMA Status (Underflow) Register (BAR2).*

 - #define **DM35424_OFFSET_DMA_CURRENT_COUNT** 0x04
- Offset to the DMA Current Count Register (BAR2).*

 - #define **DM35424_OFFSET_DMA_CURRENT_BUFFER** 0x07
- Offset to the DMA Current Buffer Register (BAR2).*

 - #define **DM35424_OFFSET_DMA_WR_FIFO_CNT** 0x08
- Offset to the DMA Write FIFO Count Register (BAR2).*

 - #define **DM35424_OFFSET_DMA_RD_FIFO_CNT** 0x0A
- Offset to the DMA Read FIFO Count Register (BAR2).*

 - #define **DM35424_OFFSET_DMA_STAT_USED** 0x0C
- Offset to the DMA Status (Used) Register (BAR2).*

 - #define **DM35424_OFFSET_DMA_STAT_INVALID** 0x0D
- Offset to the DMA Status (Invalid) Register (BAR2).*

 - #define **DM35424_OFFSET_DMA_STAT_COMPLETE** 0x0E
- Offset to the DMA Status (Complete) Register (BAR2).*

- **#define DM35424_OFFSET_DMA_LAST_ACTION** 0x0F
Offset to the DMA Last Action Register (BAR2).
- **#define DM35424_OFFSET_DMA_BUFF_START** 0x10
Offset to the start of the buffer control section (BAR2).
- **#define DM35424_OFFSET_DMA_BUFFER_STAT** 0x02
Offset to the buffer status register, from the start of the buffer control section (BAR2).
- **#define DM35424_OFFSET_DMA_BUFFER_CTRL** 0x03
Offset to the buffer control register, from the start of the buffer control section (BAR2).
- **#define DM35424_OFFSET_DMA_BUFFER_SIZE** 0x04
Offset to the buffer size register, from the start of the buffer control section (BAR2).
- **#define DM35424_OFFSET_DMA_BUFFER_ADDRESS** 0x08
Offset to the buffer address register, from the start of the buffer control section (BAR2).
- **#define DM35424_OFFSET_FB_DMA_CHANNELS** 0x06
Offset to the DMA Channels count of the function block (BAR2).
- **#define DM35424_OFFSET_FB_DMA_BUFFERS** 0x07
Offset to the DMA buffers count of the function block (BAR2).
- **#define DM35424_OFFSET_FB_CTRL_START** 0x08
Offset to the beginning of the Function Block control section in BAR2.
- **#define DM35424_OFFSET_ADC_MODE_STATUS** 0x00
Offset to the ADC Mode-Status register, from the start of the ADC control section.
- **#define DM35424_OFFSET_ADC_CLK_SRC** 0x01
Offset to the ADC Clock Source register, from the start of the ADC control section.
- **#define DM35424_OFFSET_ADC_START_TRIG** 0x02
Offset to the ADC Start Trigger register, from the start of the ADC control section.
- **#define DM35424_OFFSET_ADC_STOP_TRIG** 0x03
Offset to the ADC Stop Trigger register, from the start of the ADC control section.
- **#define DM35424_OFFSET_ADC_CLK_DIV** 0x04
Offset to the ADC Clock Divider register, from the start of the ADC control section.
- **#define DM35424_OFFSET_ADC_CLK_DIV_COUNTER** 0x08
Offset to the ADC Clock Divider Counter register, from the start of the ADC control section.
- **#define DM35424_OFFSET_ADC_PRE_CAPT_COUNT** 0x0c
Offset to the ADC Pre-Start Capture Count register, from the start of the ADC control section.
- **#define DM35424_OFFSET_ADC_POST_CAPT_COUNT** 0x10
Offset to the ADC Post-Stop Capture Count register, from the start of the ADC control section.
- **#define DM35424_OFFSET_ADC_SAMPLE_COUNT** 0x14
Offset to the ADC Sample Count register, from the start of the ADC control section.
- **#define DM35424_OFFSET_ADC_INT_ENABLE** 0x18
Offset to the ADC Interrupt Enable register, from the start of the ADC control section.
- **#define DM35424_OFFSET_ADC_INT_STAT** 0x1e
Offset to the ADC Interrupt Status register, from the start of the ADC control section.
- **#define DM35424_OFFSET_ADC_CLK_BUS2** 0x22
Offset to the ADC Clock Bus 2, from the start of the ADC control section.
- **#define DM35424_OFFSET_ADC_CLK_BUS3** 0x23
Offset to the ADC Clock Bus 3 register, from the start of the ADC control section.
- **#define DM35424_OFFSET_ADC_CLK_BUS4** 0x24
Offset to the ADC Clock Bus 4 register, from the start of the ADC control section.
- **#define DM35424_OFFSET_ADC_CLK_BUS5** 0x25
Offset to the ADC Clock Bus 5 register, from the start of the ADC control section.
- **#define DM35424_OFFSET_ADC_CLK_BUS6** 0x26
Offset to the ADC Clock Bus 6 register, from the start of the ADC control section.
- **#define DM35424_OFFSET_ADC_CLK_BUS7** 0x27

- Offset to the ADC Clock Bus 7 register, from the start of the ADC control section.*

 - **#define DM35424_OFFSET_ADC_AD_CONFIG** 0x28
- Offset to the ADC AD Config register, from the start of the ADC control section.*

 - **#define DM35424_OFFSET_ADC_CHAN_CTRL_BLK_START** 0x2c
- Offset to the start of the Channel Control Section, from the start of the ADC control section.*

 - **#define DM35424_ADC_CHAN_CTRL_BLK_SIZE** 0x18
- Constant size of ADC channel section in function block.*

 - **#define DM35424_OFFSET_ADC_CHAN_FRONT_END_CONFIG** 0x00
- Offset to the Channel Front End Config register, from the start of the ADC channel control section.*

 - **#define DM35424_OFFSET_ADC_CHAN_DATA_COUNT** 0x04
- Offset to the Channel FIFO Data count register, from the start of the ADC channel control section.*

 - **#define DM35424_OFFSET_ADC_CHAN_FILTER** 0x09
- Offset to the Channel Filter register, from the start of the ADC channel control section.*

 - **#define DM35424_OFFSET_ADC_CHAN_INTR_STAT** 0x0a
- Offset to the Channel Interrupt Status register, from the start of the ADC channel control section.*

 - **#define DM35424_OFFSET_ADC_CHAN_INTR_ENABLE** 0x0b
- Offset to the Channel Interrupt Enable register, from the start of the ADC channel control section.*

 - **#define DM35424_OFFSET_ADC_CHAN_LOW_THRESHOLD** 0x0c
- Offset to the Channel Low Threshold register, from the start of the ADC channel control section.*

 - **#define DM35424_OFFSET_ADC_CHAN_HIGH_THRESHOLD** 0x10
- Offset to the Channel High Threshold register, from the start of the ADC channel control section.*

 - **#define DM35424_OFFSET_ADC_CHAN_LAST_SAMPLE** 0x14
- Offset to the Channel Last Sample register, from the start of the ADC channel control section.*

 - **#define DM35424_OFFSET_ADC_FIFO_CTRL_BLK_START** 0x334
- Offset to the start of the FIFO Control Section, from the start of the ADC control section.*

 - **#define DM35424_ADC_FIFO_CTRL_BLK_SIZE** 0x4
- Constant size of ADC FIFO section in function block.*

 - **#define DM35424_OFFSET_FB_ADC_FIFO** 0x0334
- Offset to the FIFO for non-DMA read and write operations.*

 - **#define DM35424_OFFSET_DAC_MODE_STATUS** 0x00
- Offset to the Mode/Status register, from the start of the DAC control section.*

 - **#define DM35424_OFFSET_DAC_CLK_SRC** 0x01
- Offset to the Clock Source register, from the start of the DAC control section.*

 - **#define DM35424_OFFSET_DAC_START_TRIG** 0x02
- Offset to the Start Trigger register, from the start of the DAC control section.*

 - **#define DM35424_OFFSET_DAC_STOP_TRIG** 0x03
- Offset to the Stop Trigger register, from the start of the DAC control section.*

 - **#define DM35424_OFFSET_DAC_CLK_DIV** 0x04
- Offset to the Clock Divider register, from the start of the DAC control section.*

 - **#define DM35424_OFFSET_DAC_CLK_DIV_COUNT** 0x08
- Offset to the Clock Divider Counter register, from the start of the DAC control section.*

 - **#define DM35424_OFFSET_DAC_POST_STOP_CONV** 0x10
- Offset to the Post-Stop Conversion Count register, from the start of the DAC control section.*

 - **#define DM35424_OFFSET_DAC_CONV_COUNT** 0x14
- Offset to the Conversion Count register, from the start of the DAC control section.*

 - **#define DM35424_OFFSET_DAC_INT_ENABLE** 0x18
- Offset to the Interrupt Enable register, from the start of the DAC control section.*

 - **#define DM35424_OFFSET_DAC_INT_STAT** 0x1e
- Offset to the Interrupt Status register, from the start of the DAC control section.*

 - **#define DM35424_OFFSET_DAC_CLK_BUS2** 0x22
- Offset to the Clock Bus 2 register, from the start of the DAC control section.*

- **#define DM35424_OFFSET_DAC_CLK_BUS3** 0x23
Offset to the Clock Bus 3 register, from the start of the DAC control section.
- **#define DM35424_OFFSET_DAC_CLK_BUS4** 0x24
Offset to the Clock Bus 4 register, from the start of the DAC control section.
- **#define DM35424_OFFSET_DAC_CLK_BUS5** 0x25
Offset to the Clock Bus 5 register, from the start of the DAC control section.
- **#define DM35424_OFFSET_DAC_CLK_BUS6** 0x26
Offset to the Clock Bus 6 register, from the start of the DAC control section.
- **#define DM35424_OFFSET_DAC_CLK_BUS7** 0x27
Offset to the Clock Bus 7 register, from the start of the DAC control section.
- **#define DM35424_OFFSET_DAC_DA_CONFIG** 0x28
Offset to the DA Config register, from the start of the DAC control section.
- **#define DM35424_OFFSET_DAC_CHAN_CTRL_BLK_START** 0x2c
Offset to the start of the DAC channel control section, from the start of the DAC control section.
- **#define DM35424_DAC_CHAN_CTRL_BLK_SIZE** 0x14
Constant size of channel control section in function block.
- **#define DM35424_OFFSET_DAC_CHAN_FRONT_END_CONFIG** 0x00
Offset to the Front-End Config register, from the start of the DAC channel control section.
- **#define DM35424_OFFSET_DAC_CHAN_MARKER_STATUS** 0x0a
Offset to the Channel marker Interrupt Status register, from the start of the DAC channel control section.
- **#define DM35424_OFFSET_DAC_CHAN_MARKER_ENABLE** 0x0b
Offset to the Channel marker Interrupt Enable register, from the start of the DAC channel control section.
- **#define DM35424_OFFSET_DAC_CHAN_LAST_CONVERSION** 0x10
Offset to the Channel Last Conversion register, from the start of the DAC channel control section.
- **#define DM35424_OFFSET_DAC_FIFO_CTRL_BLK_START** 0x84
Offset to the start of the DAC FIFO control section, from the start of the DAC control section.
- **#define DM35424_OFFSET_DAC_FIFO_CTRL_BLK_SIZE** 0x4
Constant size of FIFO control section in function block.
- **#define DM35424_OFFSET_DIO_INPUT_VAL** 0x00
Offset to the Input Value register, from the start of the DIO control section.
- **#define DM35424_OFFSET_DIO_OUTPUT_VAL** 0x04
Offset to the Output Value register, from the start of the DIO control section.
- **#define DM35424_OFFSET_DIO_DIRECTION** 0x08
Offset to the Direction register, from the start of the DIO control section.
- **#define DM35424_OFFSET_TEMPERATURE** 0x00
Offset to the Temperature register, from the start of the Temperature control section.
- **#define DM35424_OFFSET_REF_ADJUST_GO_BUSY** 0x00
Offset to the Go/Busy register, from the start of the Reference Adjustment control section.
- **#define DM35424_OFFSET_REF_OUTPUT_LATCH** 0x04
Offset to the output latch register, from the start of the Reference Adjustment control section.

6.53.1 Detailed Description

Defines for the DM35424 Registers (Offsets).

Id

[dm35424_registers.h](#) 153456 2026-04-01 16:00:08Z bkorpacz

Definition in file [dm35424_registers.h](#).

6.54 dm35424_registers.h

[Go to the documentation of this file.](#)

```

00001
00009
00010 //-----
00011 // COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC. ALL RIGHTS RESERVED.
00012 //
00013 // This software package is dual-licensed. Source code that is compiled for
00014 // kernel mode execution is licensed under the GNU General Public License
00015 // version 2. For a copy of this license, refer to the file
00016 // LICENSE_GPLv2.TXT (which should be included with this software) or contact
00017 // the Free Software Foundation. Source code that is compiled for user mode
00018 // execution is licensed under the RTD End-User Software License Agreement.
00019 // For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00020 // Technologies, Inc. Using this software indicates agreement with the
00021 // license terms listed above.
00022 //-----
00023
00024 #ifndef __DM35424_REGISTERS_H__
00025 #define __DM35424_REGISTERS_H__
00026
00027
00032
00033 /*****
00034  * General Board Control (BAR0)
00035  *****/
00040 #define DM35424_OFFSET_GBC_FORMAT 0x00
00041
00046 #define DM35424_OFFSET_GBC_REV 0x01
00047
00052 #define DM35424_OFFSET_GBC_END_INTERRUPT 0x02
00053
00058 #define DM35424_OFFSET_GBC_BOARD_RESET 0x03
00059
00064 #define DM35424_OFFSET_GBC_PDP_NUMBER 0x04
00065
00070 #define DM35424_OFFSET_GBC_FPGA_BUILD 0x08
00071
00076 #define DM35424_OFFSET_GBC_SYS_CLK_FREQ 0x0c
00077
00078
00084 #define DM35424_OFFSET_GBC_IRQ_STATUS 0x10
00085
00091 #define DM35424_OFFSET_GBC_DMA_IRQ_STATUS 0x18
00092
00097 #define DM35424_OFFSET_GBC_FB_START 0x20
00098
00103 #define DM35424_GBC_FB_BLK_SIZE 0x10
00104
00110 #define DM35424_OFFSET_GBC_FB_ID 0x00
00111
00116 #define DM35424_FB_ID_TYPE_MASK 0x0000FFFF
00117
00122 #define DM35424_FB_ID_SUBTYPE_MASK 0x00FF0000
00123
00128 #define DM35424_FB_ID_TYPE_REV_MASK 0xFF000000
00129
00135 #define DM35424_OFFSET_GBC_FB_OFFSET 0x04
00136
00142 #define DM35424_OFFSET_GBC_FB_DMA_OFFSET 0x08
00143
00144
00145 /*****
00146  * DMA Control (BAR2)
00147  *****/
00152 #define DM35424_OFFSET_DMA_ACTION 0x00
00153
00158 #define DM35424_OFFSET_DMA_SETUP 0x01
00159
00164 #define DM35424_OFFSET_DMA_STAT_OVERFLOW 0x02
00165
00170 #define DM35424_OFFSET_DMA_STAT_UNDERFLOW 0x03
00171
00176 #define DM35424_OFFSET_DMA_CURRENT_COUNT 0x04
00177
00182 #define DM35424_OFFSET_DMA_CURRENT_BUFFER 0x07
00183
00188 #define DM35424_OFFSET_DMA_WR_FIFO_CNT 0x08
00189
00194 #define DM35424_OFFSET_DMA_RD_FIFO_CNT 0x0A
00195
00200 #define DM35424_OFFSET_DMA_STAT_USED 0x0C
00201
00206 #define DM35424_OFFSET_DMA_STAT_INVALID 0x0D

```

```

00207
00212 #define DM35424_OFFSET_DMA_STAT_COMPLETE           0x0E
00213
00218 #define DM35424_OFFSET_DMA_LAST_ACTION             0x0F
00219
00224 #define DM35424_OFFSET_DMA_BUFF_START              0x10
00225
00231 #define DM35424_OFFSET_DMA_BUFFER_STAT              0x02
00232
00238 #define DM35424_OFFSET_DMA_BUFFER_CTRL             0x03
00239
00245 #define DM35424_OFFSET_DMA_BUFFER_SIZE             0x04
00246
00252 #define DM35424_OFFSET_DMA_BUFFER_ADDRESS          0x08
00253
00254
00255 /*****
00256  *   All Function Blocks (BAR2)
00257  *****/
00262 #define DM35424_OFFSET_FB_DMA_CHANNELS              0x06
00263
00268 #define DM35424_OFFSET_FB_DMA_BUFFERS              0x07
00269
00274 #define DM35424_OFFSET_FB_CTRL_START                0x08
00275
00276 /*****
00277  *   ADC Function Blocks (BAR2)
00278  *****/
00284 #define DM35424_OFFSET_ADC_MODE_STATUS              0x00
00285
00291 #define DM35424_OFFSET_ADC_CLK_SRC                  0x01
00292
00298 #define DM35424_OFFSET_ADC_START_TRIG               0x02
00299
00305 #define DM35424_OFFSET_ADC_STOP_TRIG                0x03
00306
00312 #define DM35424_OFFSET_ADC_CLK_DIV                  0x04
00313
00319 #define DM35424_OFFSET_ADC_CLK_DIV_COUNTER          0x08
00320
00326 #define DM35424_OFFSET_ADC_PRE_CAPT_COUNT           0x0c
00327
00333 #define DM35424_OFFSET_ADC_POST_CAPT_COUNT           0x10
00334
00340 #define DM35424_OFFSET_ADC_SAMPLE_COUNT             0x14
00341
00347 #define DM35424_OFFSET_ADC_INT_ENABLE               0x18
00348
00354 #define DM35424_OFFSET_ADC_INT_STAT                 0x1e
00355
00361 #define DM35424_OFFSET_ADC_CLK_BUS2                  0x22
00362
00368 #define DM35424_OFFSET_ADC_CLK_BUS3                  0x23
00369
00375 #define DM35424_OFFSET_ADC_CLK_BUS4                  0x24
00376
00382 #define DM35424_OFFSET_ADC_CLK_BUS5                  0x25
00383
00389 #define DM35424_OFFSET_ADC_CLK_BUS6                  0x26
00390
00396 #define DM35424_OFFSET_ADC_CLK_BUS7                  0x27
00397
00403 #define DM35424_OFFSET_ADC_AD_CONFIG                 0x28
00404
00410 #define DM35424_OFFSET_ADC_CHAN_CTRL_BLK_START      0x2c
00411
00416 #define DM35424_ADC_CHAN_CTRL_BLK_SIZE              0x18
00417
00423 #define DM35424_OFFSET_ADC_CHAN_FRONT_END_CONFIG     0x00
00424
00430 #define DM35424_OFFSET_ADC_CHAN_DATA_COUNT           0x04
00431
00437 #define DM35424_OFFSET_ADC_CHAN_FILTER               0x09
00438
00444 #define DM35424_OFFSET_ADC_CHAN_INTR_STAT            0x0a
00445
00451 #define DM35424_OFFSET_ADC_CHAN_INTR_ENABLE          0x0b
00452
00458 #define DM35424_OFFSET_ADC_CHAN_LOW_THRESHOLD        0x0c
00459
00465 #define DM35424_OFFSET_ADC_CHAN_HIGH_THRESHOLD       0x10
00466
00472 #define DM35424_OFFSET_ADC_CHAN_LAST_SAMPLE          0x14
00473
00479 #define DM35424_OFFSET_ADC_FIFO_CTRL_BLK_START       0x334
00480
00485 #define DM35424_ADC_FIFO_CTRL_BLK_SIZE              0x4

```

```

00486
00495 #define DM35424_OFFSET_FB_ADC_FIFO          0x0334
00496
00497
00498
00499
00500
00501 /*****
00502  *   DAC Function Blocks (BAR2)
00503  *****/
00504
00510 #define DM35424_OFFSET_DAC_MODE_STATUS      0x00
00511
00517 #define DM35424_OFFSET_DAC_CLK_SRC          0x01
00518
00524 #define DM35424_OFFSET_DAC_START_TRIG      0x02
00525
00531 #define DM35424_OFFSET_DAC_STOP_TRIG       0x03
00532
00538 #define DM35424_OFFSET_DAC_CLK_DIV         0x04
00539
00545 #define DM35424_OFFSET_DAC_CLK_DIV_COUNT   0x08
00546
00552 #define DM35424_OFFSET_DAC_POST_STOP_CONV   0x10
00553
00559 #define DM35424_OFFSET_DAC_CONV_COUNT      0x14
00560
00566 #define DM35424_OFFSET_DAC_INT_ENABLE      0x18
00567
00573 #define DM35424_OFFSET_DAC_INT_STAT        0x1e
00574
00580 #define DM35424_OFFSET_DAC_CLK_BUS2        0x22
00581
00587 #define DM35424_OFFSET_DAC_CLK_BUS3        0x23
00588
00594 #define DM35424_OFFSET_DAC_CLK_BUS4        0x24
00595
00601 #define DM35424_OFFSET_DAC_CLK_BUS5        0x25
00602
00608 #define DM35424_OFFSET_DAC_CLK_BUS6        0x26
00609
00615 #define DM35424_OFFSET_DAC_CLK_BUS7        0x27
00616
00622 #define DM35424_OFFSET_DAC_DA_CONFIG       0x28
00623
00629 #define DM35424_OFFSET_DAC_CHAN_CTRL_BLK_START 0x2c
00630
00635 #define DM35424_DAC_CHAN_CTRL_BLK_SIZE     0x14
00636
00642 #define DM35424_OFFSET_DAC_CHAN_FRONT_END_CONFIG 0x00
00643
00649 #define DM35424_OFFSET_DAC_CHAN_MARKER_STATUS 0x0a
00650
00656 #define DM35424_OFFSET_DAC_CHAN_MARKER_ENABLE 0x0b
00657
00663 #define DM35424_OFFSET_DAC_CHAN_LAST_CONVERSION 0x10
00664
00670 #define DM35424_OFFSET_DAC_FIFO_CTRL_BLK_START 0x84
00671
00676 #define DM35424_OFFSET_DAC_FIFO_CTRL_BLK_SIZE 0x4
00677
00678
00679
00680 /*****
00681  *   DIO Function Blocks (BAR2)
00682  *****/
00683
00689 #define DM35424_OFFSET_DIO_INPUT_VAL        0x00
00690
00696 #define DM35424_OFFSET_DIO_OUTPUT_VAL       0x04
00697
00703 #define DM35424_OFFSET_DIO_DIRECTION        0x08
00704
00705
00706
00707
00708
00709 /*****
00710  *   Temperature Function Blocks (BAR2)
00711  *****/
00712
00718 #define DM35424_OFFSET_TEMPERATURE          0x00
00719
00720
00721
00722
00728 #define DM35424_OFFSET_REF_ADJUST_GO_BUSY   0x00

```

```
00729
00735 #define DM35424_OFFSET_REF_OUTPUT_LATCH 0x04
00736
00737
00738
00739
00740
00741
00742
00743
00744
00745
00746
00747
00748
00749
00753
00754 #endif
```

6.55 include/dm35424_temperature_library.h File Reference

Definitions for the DM35424 Temperature Library.

```
#include "dm35424_gbc_library.h"
```

Functions

- [DM35424LIB_API](#) int [DM35424_Temperature_Open](#) (struct [DM35424_Board_Descriptor](#) *handle, unsigned int ordinal_to_open, struct [DM35424_Function_Block](#) *fb_temp)
Open the temperature function block, getting address values that will be used later by other library functions.
- [DM35424LIB_API](#) int [DM35424_Temperature_Read](#) (struct [DM35424_Board_Descriptor](#) *handle, struct [DM35424_Function_Block](#) *temp_fb, float *temperature)
Read the temperature of the board, in Celsius degrees.

6.55.1 Detailed Description

Definitions for the DM35424 Temperature Library.

Id

[dm35424_temperature_library.h](#) 60276 2012-06-05 16:04:15Z rgroner

Definition in file [dm35424_temperature_library.h](#).

6.56 dm35424_temperature_library.h

[Go to the documentation of this file.](#)

```

00001
00009
00010 //-----
00011 //  COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC.  ALL RIGHTS RESERVED.
00012 //
00013 //  This software package is dual-licensed.  Source code that is compiled for
00014 //  kernel mode execution is licensed under the GNU General Public License
00015 //  version 2.  For a copy of this license, refer to the file
00016 //  LICENSE GPLv2.TXT (which should be included with this software) or contact
00017 //  the Free Software Foundation.  Source code that is compiled for user mode
00018 //  execution is licensed under the RTD End-User Software License Agreement.
00019 //  For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00020 //  Technologies, Inc.  Using this software indicates agreement with the
00021 //  license terms listed above.
00022 //-----
00023
00024
00025 #ifndef _DM35424_TEMPERATURE_LIBRARY__H_
00026 #define _DM35424_TEMPERATURE_LIBRARY__H_
00027
00028
00029 #include "dm35424_gbc_library.h"
00030
00031 #ifdef __cplusplus
00032 extern "C" {
00033 #endif
00034
00040
00073 DM35424LIB_API
00074 int DM35424_Temperature_Open(struct DM35424_Board_Descriptor *handle,
00075                             unsigned int ordinal_to_open,
00076                             struct DM35424_Function_Block *fb_temp);
00077
00078
00110 DM35424LIB_API
00111 int DM35424_Temperature_Read(struct DM35424_Board_Descriptor *handle,
00112                             struct DM35424_Function_Block *temp_fb,
00113                             float *temperature);
00114
00118
00119 #ifdef __cplusplus
00120 }
00121 #endif
00122
00123 #endif

```

6.57 include/dm35424_types.h File Reference

Defines for the DM35424. Values for the general board, not specific to a particular function block.

Macros

- **#define DM35424_SUBTYPE_00 0**
Constant for FB subtype 0.
- **#define DM35424_SUBTYPE_01 1**
Constant for FB subtype 1.
- **#define DM35424_SUBTYPE_02 2**
Constant for FB subtype 2.
- **#define DM35424_SUBTYPE_03 3**
Constant for FB subtype 3.
- **#define DM35424_SUBTYPE_INVALID 0xFF**
Constant value indicating an invalid subtype.
- **#define DM35424_FUNC_BLOCK_INVALID 0x0000**

- Constant value indicating an invalid function block.*

 - **#define DM35424_FUNC_BLOCK_INVALID2** 0xFFFF
- Constant value indicating an invalid function block.*

 - **#define DM35424_FUNC_BLOCK_SYNCBUS** 0x0001
- Function Block Constant for SyncBus.*

 - **#define DM35424_FUNC_BLOCK_EXT_CLOCKING** 0x0002
- Function Block Constant for Global Clocking.*

 - **#define DM35424_FUNC_BLOCK_CLK0003** 0x0003
- Function Block Constant for External Clocking (0003).*

 - **#define DM35424_FUNC_BLOCK_CAPTWIN** 0x0005
- Function Block Constant for Capture Window.*

 - **#define DM35424_FUNC_BLOCK_ADC** 0x1000
- Function Block Constant for ADC.*

 - **#define DM35424_FUNC_BLOCK_ADC1001** 0x1001
- Function Block Constant for 10 MHz ADC (1001).*

 - **#define DM35424_FUNC_BLOCK_ADC1002** 0x1002
- Function Block Constant for SDM35541-based ADC (1002).*

 - **#define DM35424_FUNC_BLOCK_DAC** 0x2000
- Function Block Constant for DAC.*

 - **#define DM35424_FUNC_BLOCK_DAC2001** 0x2001
- Function Block Constant for High Speed DAC (2001).*

 - **#define DM35424_FUNC_BLOCK_DIO** 0x3000
- Function Block Constant for DIO.*

 - **#define DM35424_FUNC_BLOCK_ADIO** 0x3001
- Function Block Constant for ADIO.*

 - **#define DM35424_FUNC_BLOCK_ADIO3010** 0x3010
- Function Block Constant for ADIO3010.*

 - **#define DM35424_FUNC_BLOCK_USART** 0x4000
- Function Block Constant for Synchronous/Asynchronous Serial Port.*

 - **#define DM35424_FUNC_BLOCK_REF_ADJUST** 0xF000
- Function Block Constant for Reference Adjustment.*

 - **#define DM35424_FUNC_BLOCK_TEMPERATURE_SENSOR** 0xF001
- Function Block Constant for Temperature Sensor.*

 - **#define DM35424_FUNC_BLOCK_FLASH_PROGRAMMER** 0xF002
- Function Block Constant for Module Firmware Programmer.*

 - **#define DM35424_FUNC_BLOCK_CLK_GEN** 0xF003
- Function Block Constant for Clock Generator.*

 - **#define DM35424_FUNC_BLOCK_DIN3011** 0x3011
- Function Block Constant for Digital Input (3011).*

 - **#define DM35424_FUNC_BLOCK_DOT3012** 0x3012
- Function Block Constant for Digital Output (3012).*

 - **#define DM35424_FUNC_BLOCK_INC3200** 0x3200
- Function Block Constant for Incremental Encoder (3200).*

 - **#define DM35424_FUNC_BLOCK_PWM3100** 0x3100
- Function Block Constant for PWM (3100).*

 - **#define DM35424_FUNC_BLOCK_CLK0004** 0x0004
- Function Block Constant for Programmable Clock (0004).*

 - **#define DM35424_FUNC_BLOCK_I2C3300** 0x3300
- Function Block Constant for I2C Bus Master (3300).*

 - **#define DM35424_MAX_FB** 62
- Maximum possible number of function blocks on a board.*

- #define **MAX_DMA_BUFFERS** 16
Maximum possible number of DMA buffers for any function block.
- #define **MAX_DMA_CHANNELS** 32
Maximum possible number of DMA channels for any function block.
- #define **DM35424_DMA_MAX_BUFFER_SIZE** 0xFFFFFC
Maximum possible DMA buffer size.
- #define **DM35424_BOARD_ACK_INTERRUPT** 0x1
Value to write to the EOI register to acknowledge interrupts.
- #define **DM35424_BOARD_RESET_VALUE** 0xAA
Value to write to the Reset register in order to reset the board.
- #define **DM35424_FIFO_ACCESS_FB_REVISION** 0x01
Minimum function block revision that supports direct FIFO read/write access.

Enumerations

- enum **DM35424_Clock_Sources** {
[DM35424_CLK_SRC_IMMEDIATE](#) , [DM35424_CLK_SRC_NEVER](#) , [DM35424_CLK_SRC_BUS2](#) ,
[DM35424_CLK_SRC_BUS3](#) ,
[DM35424_CLK_SRC_BUS4](#) , [DM35424_CLK_SRC_BUS5](#) , [DM35424_CLK_SRC_BUS6](#) , [DM35424_CLK_SRC_BUS7](#)
,
[DM35424_CLK_SRC_CHAN_THRESH](#) = 0x08 , [DM35424_CLK_SRC_CHAN_THRESH_INV](#) = 0x09 ,
[DM35424_CLK_SRC_BUS2_INV](#) = 0x0A , [DM35424_CLK_SRC_BUS3_INV](#) ,
[DM35424_CLK_SRC_BUS4_INV](#) , [DM35424_CLK_SRC_BUS5_INV](#) , [DM35424_CLK_SRC_BUS6_INV](#) ,
[DM35424_CLK_SRC_BUS7_INV](#) }
Possible clock sources used by function blocks. Note that some clock sources may not be available on your particular board. Check the hardware manual to verify which clock sources can be used.
- enum **DM35424_Clock_Buses** {
[DM35424_CLK_BUS2](#) = 2 , [DM35424_CLK_BUS3](#) , [DM35424_CLK_BUS4](#) , [DM35424_CLK_BUS5](#) ,
[DM35424_CLK_BUS6](#) , [DM35424_CLK_BUS7](#) }
Clock buses available to the function block.

6.57.1 Detailed Description

Defines for the DM35424. Values for the general board, not specific to a particular function block.

Id

[dm35424_types.h](#) 149930 2025-09-25 18:44:06Z bkorpacz

Definition in file [dm35424_types.h](#).

6.57.2 Enumeration Type Documentation

6.57.2.1 DM35424_Clock_Buses

enum [DM35424_Clock_Buses](#)

Clock buses available to the function block.

Enumerator

DM35424_CLK_BUS2	Clock Bus 2.
DM35424_CLK_BUS3	Clock Bus 3.
DM35424_CLK_BUS4	Clock Bus 4.
DM35424_CLK_BUS5	Clock Bus 5.
DM35424_CLK_BUS6	Clock Bus 6.
DM35424_CLK_BUS7	Clock Bus 7.

Definition at line 359 of file [dm35424_types.h](#).

6.57.2.2 DM35424_Clock_Sources

enum [DM35424_Clock_Sources](#)

Possible clock sources used by function blocks. Note that some clock sources may not be available on your particular board. Check the hardware manual to verify which clock sources can be used.

DM35424_General_Definitions

Enumerator

DM35424_CLK_SRC_IMMEDIATE	Clock Source - Immediate (0x00)
DM35424_CLK_SRC_NEVER	Clock Source - Never (0x01)
DM35424_CLK_SRC_BUS2	Clock Source - Bus 2 (0x02)
DM35424_CLK_SRC_BUS3	Clock Source - Bus 3 (0x03)
DM35424_CLK_SRC_BUS4	Clock Source - Bus 4 (0x04)
DM35424_CLK_SRC_BUS5	Clock Source - Bus 5 (0x05)
DM35424_CLK_SRC_BUS6	Clock Source - Bus 6 (0x06)
DM35424_CLK_SRC_BUS7	Clock Source - Bus 7 (0x07)
DM35424_CLK_SRC_CHAN_THRESH	Clock Source - Threshold Exceeded (0x08)
DM35424_CLK_SRC_CHAN_THRESH_INV	Clock Source - Threshold Inverse (None Exceeded) (0x09)
DM35424_CLK_SRC_BUS2_INV	Clock Source - Bus 2 Inverse (0x0A)
DM35424_CLK_SRC_BUS3_INV	Clock Source - Bus 3 Inverse (0x0B)
DM35424_CLK_SRC_BUS4_INV	Clock Source - Bus 4 Inverse (0x0C)
DM35424_CLK_SRC_BUS5_INV	Clock Source - Bus 5 Inverse (0x0D)
DM35424_CLK_SRC_BUS6_INV	Clock Source - Bus 6 Inverse (0x0E)
DM35424_CLK_SRC_BUS7_INV	Clock Source - Bus 7 Inverse (0x0F)

Definition at line 271 of file [dm35424_types.h](#).

6.58 dm35424_types.h

[Go to the documentation of this file.](#)

```

00001
00010
00011 //-----
00012 //  COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC.  ALL RIGHTS RESERVED.
00013 //
00014 //  This software package is dual-licensed.  Source code that is compiled for
00015 //  kernel mode execution is licensed under the GNU General Public License
00016 //  version 2.  For a copy of this license, refer to the file
00017 //  LICENSE GPLv2.TXT (which should be included with this software) or contact
00018 //  the Free Software Foundation.  Source code that is compiled for user mode
00019 //  execution is licensed under the RTD End-User Software License Agreement.
00020 //  For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00021 //  Technologies, Inc.  Using this software indicates agreement with the
00022 //  license terms listed above.
00023 //-----
00024
00025 #ifndef __DM35424_TYPES_H__
00026 #define __DM35424_TYPES_H__
00027
00032
00037 #define DM35424_SUBTYPE_00      0
00038
00043 #define DM35424_SUBTYPE_01      1
00044
00049 #define DM35424_SUBTYPE_02      2
00050
00055 #define DM35424_SUBTYPE_03      3
00056
00057
00062 #define DM35424_SUBTYPE_INVALID 0xFF
00063
00068 #define DM35424_FUNC_BLOCK_INVALID      0x0000
00069
00074 #define DM35424_FUNC_BLOCK_INVALID2     0xFFFF
00075
00080 #define DM35424_FUNC_BLOCK_SYNCBUS      0x0001
00081
00086 #define DM35424_FUNC_BLOCK_EXT_CLOCKING 0x0002
00087
00092 #define DM35424_FUNC_BLOCK_CLK0003      0x0003
00093
00098 #define DM35424_FUNC_BLOCK_CAPTWIN      0x0005
00099
00104 #define DM35424_FUNC_BLOCK_ADC          0x1000
00105
00110 #define DM35424_FUNC_BLOCK_ADC1001      0x1001
00111
00116 #define DM35424_FUNC_BLOCK_ADC1002      0x1002
00117
00122 #define DM35424_FUNC_BLOCK_DAC          0x2000
00123
00128 #define DM35424_FUNC_BLOCK_DAC2001      0x2001
00129
00134 #define DM35424_FUNC_BLOCK_DIO          0x3000
00135
00140 #define DM35424_FUNC_BLOCK_ADIO         0x3001
00141
00146 #define DM35424_FUNC_BLOCK_ADIO3010     0x3010
00147
00152 #define DM35424_FUNC_BLOCK_USART        0x4000
00153
00158 #define DM35424_FUNC_BLOCK_REF_ADJUST   0xF000
00159
00164 #define DM35424_FUNC_BLOCK_TEMPERATURE_SENSOR 0xF001
00165
00170 #define DM35424_FUNC_BLOCK_FLASH_PROGRAMMER 0xF002
00171
00176 #define DM35424_FUNC_BLOCK_CLK_GEN      0xF003
00177
00182 #define DM35424_FUNC_BLOCK_DIN3011      0x3011
00183
00188 #define DM35424_FUNC_BLOCK_DOT3012      0x3012
00189
00194 #define DM35424_FUNC_BLOCK_INC3200      0x3200
00195
00200 #define DM35424_FUNC_BLOCK_PWM3100      0x3100
00201
00206 #define DM35424_FUNC_BLOCK_CLK0004      0x0004
00207
00212 #define DM35424_FUNC_BLOCK_I2C3300      0x3300
00213
00214

```

```

00219 #define DM35424_MAX_FB          62
00220
00225 #define MAX_DMA_BUFFERS          16
00226
00231 #define MAX_DMA_CHANNELS          32
00232
00233
00238 #define DM35424_DMA_MAX_BUFFER_SIZE 0xFFFFFC
00239
00240
00245 #define DM35424_BOARD_ACK_INTERRUPT 0x1
00246
00251 #define DM35424_BOARD_RESET_VALUE 0xAA
00252
00258 #define DM35424_FIFO_ACCESS_FB_REVISION 0x01
00259
00263
00264
00271 enum DM35424_Clock_Sources {
00272
00276     DM35424_CLK_SRC_IMMEDIATE,
00277
00281     DM35424_CLK_SRC_NEVER,
00282
00286     DM35424_CLK_SRC_BUS2,
00287
00291     DM35424_CLK_SRC_BUS3,
00292
00296     DM35424_CLK_SRC_BUS4,
00297
00301     DM35424_CLK_SRC_BUS5,
00302
00306     DM35424_CLK_SRC_BUS6,
00307
00311     DM35424_CLK_SRC_BUS7,
00312
00316     DM35424_CLK_SRC_CHAN_THRESH = 0x08,
00317
00321     DM35424_CLK_SRC_CHAN_THRESH_INV = 0x09,
00322
00326     DM35424_CLK_SRC_BUS2_INV = 0x0A,
00327
00331     DM35424_CLK_SRC_BUS3_INV,
00332
00336     DM35424_CLK_SRC_BUS4_INV,
00337
00341     DM35424_CLK_SRC_BUS5_INV,
00342
00346     DM35424_CLK_SRC_BUS6_INV,
00347
00351     DM35424_CLK_SRC_BUS7_INV
00352
00353 };
00354
00359 enum DM35424_Clock_Buses {
00360
00365     DM35424_CLK_BUS2 = 2,
00366
00371     DM35424_CLK_BUS3,
00372
00377     DM35424_CLK_BUS4,
00378
00383     DM35424_CLK_BUS5,
00384
00389     DM35424_CLK_BUS6,
00390
00395     DM35424_CLK_BUS7
00396
00397 };
00398
00401
00402 #endif

```

6.59 include/dm35424_util_library.h File Reference

Definitions for the DM35424 Utilities library, various helper functions.

```

#include <time.h>
#include <sys/time.h>

```

Enumerations

- enum [DM35424_Waveforms](#) { [DM35424_SINE_WAVE](#) , [DM35424_SQUARE_WAVE](#) , [DM35424_SAWTOOTH_WAVE](#) }

List of possible waveforms that can be generated for DAC purposes.

Functions

- uint32_t [DM35424_Get_Maskable](#) (uint16_t data, uint16_t mask)
Return a 32-bit maskable register value from the data and mask.
- void [DM35424_Micro_Sleep](#) (unsigned long microseconds)
Sleep for a specified number of microseconds.
- long [DM35424_Get_Time_Diff](#) (struct timeval last, struct timeval first)
Calculate the time difference between the two timeval structs, in microseconds.
- int [DM35424_Generate_Signal_Data](#) (enum [DM35424_Waveforms](#) waveform, int32_t *data, uint32_t data_count, int32_t max, int32_t minimum, int32_t offset, uint32_t mask)
Generate data with a specific wave pattern. This is useful for producing recognizable waves for DAC output.
- void [DM35424_Check_Result](#) (int return_val, char *message)
Check the result of an operation, usually a library call. If the result is non-zero, then it is an error and output the passed message.

6.59.1 Detailed Description

Definitions for the DM35424 Utilities library, various helper functions.

Id

[dm35424_util_library.h](#) 139458 2023-09-20 18:22:05Z lfrankfield

Definition in file [dm35424_util_library.h](#).

6.60 dm35424_util_library.h

[Go to the documentation of this file.](#)

```
00001
00010
00011 //-----
00012 //  COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC.  ALL RIGHTS RESERVED.
00013 //
00014 //  This software package is dual-licensed.  Source code that is compiled for
00015 //  kernel mode execution is licensed under the GNU General Public License
00016 //  version 2.  For a copy of this license, refer to the file
00017 //  LICENSE_GPLv2.TXT (which should be included with this software) or contact
00018 //  the Free Software Foundation.  Source code that is compiled for user mode
00019 //  execution is licensed under the RTD End-User Software License Agreement.
00020 //  For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00021 //  Technologies, Inc.  Using this software indicates agreement with the
00022 //  license terms listed above.
00023 //-----
00024
00025
00026 #ifndef _DM35424_UTIL_H_
00027 #define _DM35424_UTIL_H_
00028
00029
00030 #include <time.h>
00031 #include <sys/time.h>
00032
```

```
00033 #ifdef __cplusplus
00034 extern "C" {
00035 #endif
00036
00041
00046 enum DM35424_Waveforms {
00047     DM35424_SINE_WAVE,
00052     DM35424_SQUARE_WAVE,
00057     DM35424_SAWTOOTH_WAVE
00062 };
00063
00064
00086 uint32_t DM35424_Get_Maskable(uint16_t data, uint16_t mask);
00087
00088
00103 void DM35424_Micro_Sleep(unsigned long microseconds);
00104
00105
00128 long DM35424_Get_Time_Diff(struct timeval last, struct timeval first);
00129
00130
00131
00190 int DM35424_Generate_Signal_Data(enum DM35424_Waveforms waveform,
00191                                 int32_t *data,
00192                                 uint32_t data_count,
00193                                 int32_t max,
00194                                 int32_t minimum,
00195                                 int32_t offset,
00196                                 uint32_t mask);
00197
00198
00220 void DM35424_Check_Result(int return_val, char *message);
00221
00222
00223
00227
00228 #ifdef __cplusplus
00229 }
00230 #endif
00231
00232 #endif
```


Index

- access
 - dm35424_iocctl_region_modify, [200](#)
 - dm35424_iocctl_region_readwrite, [202](#)
- AD_MODE_OPTION
 - DM35424 Example Programs Constants, [146](#)
- ADC_NUM_OPTION
 - DM35424 Example Programs Constants, [146](#)
- ADC_OPTION
 - DM35424 Example Programs Constants, [146](#)
- allocated
 - dm35424_pci_region, [204](#)
- ASCII_FILE_NAME
 - dm35424_adc_continuous_dma.c, [390](#)
- BAUD_OPTION
 - DM35424 Example Programs Constants, [146](#)
- BIN2TXT_OPTION
 - DM35424 Example Programs Constants, [146](#)
- BIN_FILE_NAME
 - dm35424_adc_continuous_dma.c, [390](#)
- BINARY_OPTION
 - DM35424 Example Programs Constants, [146](#)
- board
 - dm35424_adc_continuous_dma.c, [395](#)
 - dm35424_adc_test.c, [213](#)
 - dm35424_dio.c, [431](#)
- buffer
 - dm35424_dma_descriptor, [191](#)
 - dm35424_iocctl_dma, [197](#)
- buffer_count
 - dm35424_adc_continuous_dma.c, [395](#)
 - dm35424_adc_test.c, [213](#)
- buffer_ptr
 - dm35424_iocctl_dma, [197](#)
- buffer_size
 - dm35424_dma_descriptor, [191](#)
 - dm35424_iocctl_dma, [197](#)
- BUFFER_SIZE_BYTES
 - dm35424_adc.c, [372](#)
 - dm35424_adc_continuous_dma.c, [390](#)
 - dm35424_adc_test.c, [209](#)
 - dm35424_dac_dma.c, [419](#)
- buffer_size_bytes
 - dm35424_adc_continuous_dma.c, [396](#)
 - dm35424_adc_test.c, [213](#)
- BUFFER_SIZE_SAMPLES
 - dm35424_adc.c, [372](#)
 - dm35424_adc_continuous_dma.c, [390](#)
 - dm35424_adc_test.c, [209](#)
 - dm35424_dac_dma.c, [419](#)
- buffer_start_offset
 - DM35424_DMA_Descriptor, [190](#)
- bus_addr
 - dm35424_dma_descriptor, [192](#)
- cdev
 - dm35424_device_descriptor, [187](#)
- channel
 - dm35424_dma_descriptor, [192](#)
 - dm35424_iocctl_dma, [197](#)
- CHANNELS_OPTION
 - DM35424 Example Programs Constants, [145](#)
- CLK_40MHZ
 - DM35424 Board Macros, [147](#)
- control_offset
 - DM35424_DMA_Descriptor, [190](#)
 - DM35424_Function_Block, [193](#)
- convert_bin_to_txt
 - dm35424_adc_continuous_dma.c, [391](#)
- COUNT_OPTION
 - DM35424 Example Programs Constants, [146](#)
- DAC_NUM_OPTION
 - DM35424 Example Programs Constants, [146](#)
- DAC_OPTION
 - DM35424 Example Programs Constants, [146](#)
- DAC_RATE
 - dm35424_adc.c, [372](#)
- DAT_FILE_NAME_PREFIX
 - dm35424_adc_test.c, [209](#)
- DAT_FILE_NAME_SUFFIX
 - dm35424_adc_test.c, [209](#)
- data
 - dm35424_pci_access_request, [203](#)
- data16
 - dm35424_pci_access_request, [203](#)
- data32
 - dm35424_pci_access_request, [203](#)
- data8
 - dm35424_pci_access_request, [203](#)
- DEFAULT_DAC_NUM
 - dm35424_dac.c, [410](#)
- DEFAULT_DAC_TO_USE
 - dm35424_dac_dma.c, [419](#)
- DEFAULT_RATE
 - dm35424_adc.c, [372](#)
 - dm35424_adc_continuous_dma.c, [390](#)
 - dm35424_adc_test.c, [209](#)
 - dm35424_dac_dma.c, [419](#)
 - dm35424_ref_adjust.c, [441](#)

- device_id
 - dm35424_device_descriptor, [187](#)
- device_index
 - dm35424_device_descriptor, [187](#)
- device_lock
 - dm35424_device_descriptor, [187](#)
- DM35424 ADC Library Constants, [7](#)
 - DM35424_ADC_CLK_BUS_SRC_CHAN_THRESH, [10](#)
 - DM35424_ADC_CLK_BUS_SRC_DISABLE, [10](#)
 - DM35424_ADC_CLK_BUS_SRC_PACER_TICK, [11](#)
 - DM35424_ADC_CLK_BUS_SRC_POST_STOP_BUFF_FULL, [10](#)
 - DM35424_ADC_CLK_BUS_SRC_PRE_START_BUFF_FULL, [10](#)
 - DM35424_ADC_CLK_BUS_SRC_SAMPLE_TAKEN, [10](#)
 - DM35424_ADC_CLK_BUS_SRC_SAMPLING_COMPLETED, [11](#)
 - DM35424_ADC_CLK_BUS_SRC_START_TRIG, [10](#)
 - DM35424_ADC_CLK_BUS_SRC_STOP_TRIG, [10](#)
 - DM35424_Adc_Clock_Events, [10](#)
 - DM35424_ADC_GAIN_05, [11](#)
 - DM35424_ADC_GAIN_1, [11](#)
 - DM35424_ADC_GAIN_128, [11](#)
 - DM35424_ADC_GAIN_16, [11](#)
 - DM35424_ADC_GAIN_2, [11](#)
 - DM35424_ADC_GAIN_32, [11](#)
 - DM35424_ADC_GAIN_4, [11](#)
 - DM35424_ADC_GAIN_64, [11](#)
 - DM35424_ADC_GAIN_8, [11](#)
 - DM35424_ADC_INPUT_DAC_LOOPBACK, [12](#)
 - DM35424_ADC_INPUT_DIFFERENTIAL, [12](#)
 - DM35424_ADC_INPUT_SINGLE_ENDED_NEG, [12](#)
 - DM35424_ADC_INPUT_SINGLE_ENDED_POS, [12](#)
 - DM35424_ADC_MODE_CONFIG_HIGH_RES, [13](#)
 - DM35424_ADC_MODE_CONFIG_HIGH_SPEED, [13](#)
 - DM35424_ADC_MODE_CONFIG_LOW_POWER, [13](#)
 - DM35424_ADC_MODE_CONFIG_LOW_SPEED, [13](#)
 - DM35424_ADC_RNG_BIPOLAR_156mV, [12](#)
 - DM35424_ADC_RNG_BIPOLAR_19mV, [12](#)
 - DM35424_ADC_RNG_BIPOLAR_1_25V, [12](#)
 - DM35424_ADC_RNG_BIPOLAR_2_5V, [12](#)
 - DM35424_ADC_RNG_BIPOLAR_312mV, [12](#)
 - DM35424_ADC_RNG_BIPOLAR_39mV, [12](#)
 - DM35424_ADC_RNG_BIPOLAR_625mV, [12](#)
 - DM35424_ADC_RNG_BIPOLAR_78mV, [12](#)
 - DM35424_ADC_RNG_UNIPOLAR_5V, [12](#)
 - DM35424_Gains, [11](#)
 - DM35424_Input_Mode, [11](#)
 - DM35424_Input_Ranges, [12](#)
 - DM35424_Sampling_Mode, [12](#)
- DM35424 ADC Public Library Functions, [13](#)
 - DM35424_Adc_Ad_Config_Get_Mode, [16](#)
 - DM35424_Adc_Ad_Config_Set_Mode, [17](#)
 - DM35424_Adc_Channel_Find_Interrupt, [18](#)
 - DM35424_Adc_Channel_Get_Filter, [19](#)
 - DM35424_Adc_Channel_Get_Front_End_Config, [20](#)
 - DM35424_Adc_Channel_Get_Last_Sample, [21](#)
 - DM35424_Adc_Channel_Get_Thresholds, [22](#)
 - DM35424_Adc_Channel_Interrupt_Clear_Status, [20](#)
 - DM35424_Adc_Channel_Interrupt_Get_Config, [25](#)
 - DM35424_Adc_Channel_Interrupt_Get_Status, [26](#)
 - DM35424_Adc_Channel_Interrupt_Set_Config, [27](#)
 - DM35424_Adc_Channel_Reset, [28](#)
 - DM35424_Adc_Channel_Set_Filter, [29](#)
 - DM35424_Adc_Channel_Set_High_Threshold, [30](#)
 - DM35424_Adc_Channel_Set_Low_Threshold, [32](#)
 - DM35424_Adc_Channel_Setup, [33](#)
 - DM35424_Adc_Fifo_Channel_Read, [34](#)
 - DM35424_Adc_Get_Clock_Source_Global, [35](#)
 - DM35424_Adc_Get_Clock_Src, [36](#)
 - DM35424_Adc_Get_Mode_Status, [37](#)
 - DM35424_Adc_Get_Post_Stop_Samples, [38](#)
 - DM35424_Adc_Get_Pre_Trigger_Samples, [39](#)
 - DM35424_Adc_Get_Sample_Count, [40](#)
 - DM35424_Adc_Get_Start_Trigger, [41](#)
 - DM35424_Adc_Get_Stop_Trigger, [42](#)
 - DM35424_Adc_Initialize, [43](#)
 - DM35424_Adc_Interrupt_Clear_Status, [44](#)
 - DM35424_Adc_Interrupt_Get_Config, [45](#)
 - DM35424_Adc_Interrupt_Get_Status, [46](#)
 - DM35424_Adc_Interrupt_Set_Config, [47](#)
 - DM35424_Adc_Open, [48](#)
 - DM35424_Adc_Pause, [49](#)
 - DM35424_Adc_Reset, [50](#)
 - DM35424_Adc_Sample_To_Volts, [51](#)
 - DM35424_Adc_Set_Clk_Divider, [52](#)
 - DM35424_Adc_Set_Clock_Source_Global, [53](#)
 - DM35424_Adc_Set_Clock_Src, [54](#)
 - DM35424_Adc_Set_Post_Stop_Samples, [55](#)
 - DM35424_Adc_Set_Pre_Trigger_Samples, [56](#)
 - DM35424_Adc_Set_Sample_Rate, [57](#)
 - DM35424_Adc_Set_Start_Trigger, [58](#)
 - DM35424_Adc_Set_Stop_Trigger, [59](#)
 - DM35424_Adc_Start, [60](#)
 - DM35424_Adc_Start_Rearm, [61](#)
 - DM35424_Adc_Uninitialize, [62](#)
 - DM35424_Adc_Volts_To_Sample, [63](#)
- DM35424 Board Access Public Library Functions, [155](#)
 - DM35424_Dma_Initialize, [156](#)
 - DM35424_Dma_Read, [157](#)
 - DM35424_Dma_Write, [159](#)
 - DM35424_General_InstallISR, [160](#)
 - DM35424_General_RemoveISR, [161](#)
 - DM35424_General_SetISRPriority, [161](#)

- DM35424_General_WaitForInterrupt, [162](#)
- DM35424 Board Access Structures, [64](#)
 - DM35424_Board_Close, [65](#)
 - DM35424_Board_Open, [65](#)
 - DM35424_Dma, [66](#)
 - DM35424_Modify, [67](#)
 - DM35424_Read, [68](#)
 - DM35424_Write, [68](#)
- DM35424 Board Library Public Functions, [147](#)
 - DM35424_Function_Block_Open, [148](#)
 - DM35424_Function_Block_Open_Module, [148](#)
 - DM35424_Gbc_Ack_Interrupt, [149](#)
 - DM35424_Gbc_Board_Reset, [150](#)
 - DM35424_Gbc_Get_Format, [151](#)
 - DM35424_Gbc_Get_Fpga_Build, [151](#)
 - DM35424_Gbc_Get_Pdp_Number, [152](#)
 - DM35424_Gbc_Get_Revision, [153](#)
 - DM35424_Gbc_Get_Sys_Clock_Freq, [154](#)
- DM35424 Board Macros, [147](#)
 - CLK_40MHZ, [147](#)
- DM35424 Board Types, [177](#)
- DM35424 DAC Library Constants, [71](#)
 - DM35424_DAC_CLK_BUS_SRC_CHAN_MARKER, [72](#)
 - DM35424_DAC_CLK_BUS_SRC_CONV_COMPL, [72](#)
 - DM35424_DAC_CLK_BUS_SRC_CONVERSION_SENT, [72](#)
 - DM35424_DAC_CLK_BUS_SRC_DISABLE, [72](#)
 - DM35424_DAC_CLK_BUS_SRC_START_TRIG, [72](#)
 - DM35424_DAC_CLK_BUS_SRC_STOP_TRIG, [72](#)
 - DM35424_Dac_Clock_Events, [72](#)
- DM35424 DAC Library Public Functions, [73](#)
 - DM35424_Dac_Channel_Clear_Marker_Status, [75](#)
 - DM35424_Dac_Channel_Get_Marker_Config, [75](#)
 - DM35424_Dac_Channel_Get_Marker_Status, [76](#)
 - DM35424_Dac_Channel_Set_Marker_Config, [77](#)
 - DM35424_Dac_Conv_To_Volts, [78](#)
 - DM35424_Dac_Fifo_Channel_Write, [79](#)
 - DM35424_Dac_Get_Clock_Div, [80](#)
 - DM35424_Dac_Get_Clock_Src, [81](#)
 - DM35424_Dac_Get_Conversion_Count, [82](#)
 - DM35424_Dac_Get_Last_Conversion, [83](#)
 - DM35424_Dac_Get_Mode_Status, [84](#)
 - DM35424_Dac_Get_Post_Stop_Conversion_Count, [85](#)
 - DM35424_Dac_Get_Start_Trigger, [86](#)
 - DM35424_Dac_Get_Stop_Trigger, [87](#)
 - DM35424_Dac_Interrupt_Clear_Status, [88](#)
 - DM35424_Dac_Interrupt_Get_Config, [89](#)
 - DM35424_Dac_Interrupt_Get_Status, [89](#)
 - DM35424_Dac_Interrupt_Set_Config, [90](#)
 - DM35424_Dac_Open, [91](#)
 - DM35424_Dac_Pause, [92](#)
 - DM35424_Dac_Reset, [93](#)
 - DM35424_Dac_Set_Clock_Div, [94](#)
 - DM35424_Dac_Set_Clock_Source_Global, [95](#)
 - DM35424_Dac_Set_Clock_Src, [96](#)
 - DM35424_Dac_Set_Conversion_Rate, [97](#)
 - DM35424_Dac_Set_Last_Conversion, [98](#)
 - DM35424_Dac_Set_Post_Stop_Conversion_Count, [99](#)
 - DM35424_Dac_Set_Start_Trigger, [100](#)
 - DM35424_Dac_Set_Stop_Trigger, [101](#)
 - DM35424_Dac_Start, [102](#)
 - DM35424_Dac_Volts_To_Conv, [103](#)
- DM35424 DIO Public, [104](#)
 - DM35424_Dio_Get_Direction, [105](#)
 - DM35424_Dio_Get_Input_Value, [105](#)
 - DM35424_Dio_Get_Output_Value, [106](#)
 - DM35424_Dio_Open, [107](#)
 - DM35424_Dio_Set_Direction, [108](#)
 - DM35424_Dio_Set_Output_Value, [109](#)
- DM35424 DMA Public Library Constants, [111](#)
 - DM35424_FIFO_EMPTY, [112](#)
 - DM35424_FIFO_FULL, [112](#)
 - DM35424_FIFO_HAS_DATA, [112](#)
 - DM35424_Fifo_States, [112](#)
 - DM35424_FIFO_UNKNOWN, [112](#)
- DM35424 DMA Public Library Functions, [113](#)
 - DM35424_Dma_Buffer_Setup, [114](#)
 - DM35424_Dma_Buffer_Status, [115](#)
 - DM35424_Dma_Check_Buffer_Used, [117](#)
 - DM35424_Dma_Check_For_Error, [118](#)
 - DM35424_Dma_Clear, [119](#)
 - DM35424_Dma_Clear_Interrupt, [120](#)
 - DM35424_Dma_Configure_Interrupts, [122](#)
 - DM35424_Dma_Find_Interrupt, [123](#)
 - DM35424_Dma_Get_Current_Buffer_Count, [125](#)
 - DM35424_Dma_Get_Errors, [126](#)
 - DM35424_Dma_Get_Fifo_Counts, [127](#)
 - DM35424_Dma_Get_Fifo_State, [129](#)
 - DM35424_Dma_Get_Interrupt_Configuration, [130](#)
 - DM35424_Dma_Pause, [131](#)
 - DM35424_Dma_Reset_Buffer, [132](#)
 - DM35424_Dma_Setup, [133](#)
 - DM35424_Dma_Setup_Set_Direction, [135](#)
 - DM35424_Dma_Setup_Set_Used, [136](#)
 - DM35424_Dma_Start, [137](#)
 - DM35424_Dma_Status, [138](#)
 - DM35424_Dma_Stop, [140](#)
- DM35424 Driver Constants, [142](#)
- DM35424 Driver Enumerations, [142](#)
 - dm35424_pci_region_access_dir, [142](#)
 - DM35424_PCI_REGION_ACCESS_READ, [142](#)
 - DM35424_PCI_REGION_ACCESS_WRITE, [142](#)
- DM35424 Driver Structures, [143](#)
- DM35424 Example Programs Constants, [143](#)
 - AD_MODE_OPTION, [146](#)
 - ADC_NUM_OPTION, [146](#)
 - ADC_OPTION, [146](#)
 - BAUD_OPTION, [146](#)
 - BIN2TXT_OPTION, [146](#)
 - BINARY_OPTION, [146](#)

- CHANNELS_OPTION, 145
- COUNT_OPTION, 146
- DAC_NUM_OPTION, 146
- DAC_OPTION, 146
- DUMP_OPTION, 146
- EXTERNAL_OPTION, 146
- FILE_OPTION, 145
- HELP_OPTION, 145
- Help_Options, 145
- HOURS_OPTION, 146
- LOW_THRESHOLD_OPTION, 146
- MASTER_OPTION, 146
- MINOR_OPTION, 145
- MODE_OPTION, 146
- NOSTOP_OPTION, 145
- NUM_OPTION, 146
- OFFILE_OPTION, 146
- OUTPUT_ADC_OPTION, 146
- OUTPUT_RMS_OPTION, 146
- PACKED_OPTION, 146
- PATTERN_OPTION, 146
- PORT_OPTION, 146
- RANGE_OPTION, 146
- RATE_OPTION, 145
- RECEIVER_OPTION, 146
- REF_NUM_OPTION, 146
- REFCLK_OPTION, 146
- REFILL_FIFO_OPTION, 146
- SAMPLES_OPTION, 146
- SENDER_OPTION, 146
- SIZE_OPTION, 146
- SLAVE_OPTION, 146
- START_OPTION, 145
- STORE_OPTION, 146
- SYNC_CONN_OPTION, 146
- SYNC_TERM_OPTION, 146
- SYNCBUS_OPTION, 146
- TERM_OPTION, 146
- TEST_OPTION, 145
- USER_ID_OPTION, 146
- VERBOSE_OPTION, 146
- WAVE_OPTION, 145
- DM35424 ioctl macros, 155
- DM35424 PCI Region Structures, 69
 - DM35424_DMA_FUNCTIONS, 70
 - DM35424_DMA_INITIALIZE, 70
 - DM35424_DMA_READ, 70
 - DM35424_DMA_WRITE, 70
 - DM35424_PCI_REGION_ACCESS_16, 70
 - DM35424_PCI_REGION_ACCESS_32, 70
 - DM35424_PCI_REGION_ACCESS_8, 70
 - dm35424_pci_region_access_size, 70
 - DM35424_PCI_REGION_FB, 71
 - DM35424_PCI_REGION_GBC, 71
 - DM35424_PCI_REGION_GBC2, 71
 - dm35424_pci_region_num, 70
- DM35424 Reference Adjustment Library Constants, 163
 - DM35424_ADC_NON_VOL_TO_VOL, 164
 - DM35424_ADC_VOL_TO_NON_VOL, 164
 - DM35424_BOTH_NON_VOL_TO_VOL, 164
 - DM35424_BOTH_VOL_TO_NON_VOL, 164
 - DM35424_Copy_Directions, 164
 - DM35424_DAC_NON_VOL_TO_VOL, 164
 - DM35424_DAC_VOL_TO_NON_VOL, 164
- DM35424 Reference Adjustment Public Library Functions, 164
 - DM35424_Ref_Adjust_Copy_Data, 165
 - DM35424_Ref_Adjust_Open, 165
 - DM35424_Ref_Adjust_Write_Adc_To_NonVolatile, 166
 - DM35424_Ref_Adjust_Write_Adc_To_Volatile, 167
 - DM35424_Ref_Adjust_Write_Dac_To_NonVolatile, 168
 - DM35424_Ref_Adjust_Write_Dac_To_Volatile, 169
- DM35424 Register Offsets, 171
 - DM35424_OFFSET_FB_ADC_FIFO, 175
- DM35424 Temperature, 175
 - DM35424_Temperature_Open, 175
 - DM35424_Temperature_Read, 176
- DM35424 Utility Library Functions, 179
 - DM35424_Check_Result, 180
 - DM35424_Generate_Signal_Data, 180
 - DM35424_Get_Maskable, 182
 - DM35424_Get_Time_Diff, 182
 - DM35424_Micro_Sleep, 183
 - DM35424_SAWTOOTH_WAVE, 180
 - DM35424_SINE_WAVE, 179
 - DM35424_SQUARE_WAVE, 180
 - DM35424_Waveforms, 179
- dm35424_adc.c
 - BUFFER_SIZE_BYTES, 372
 - BUFFER_SIZE_SAMPLES, 372
 - DAC_RATE, 372
 - DEFAULT_RATE, 372
 - exit_program, 374
 - interrupt_count, 374
 - main, 373
 - program_name, 374
 - sigint_handler, 374
- DM35424_Adc_Ad_Config_Get_Mode
 - DM35424_ADC_Public_Library_Functions, 16
- DM35424_Adc_Ad_Config_Set_Mode
 - DM35424_ADC_Public_Library_Functions, 17
- DM35424_Adc_Channel_Find_Interrupt
 - DM35424_ADC_Public_Library_Functions, 18
- DM35424_Adc_Channel_Get_Filter
 - DM35424_ADC_Public_Library_Functions, 19
- DM35424_Adc_Channel_Get_Front_End_Config
 - DM35424_ADC_Public_Library_Functions, 20
- DM35424_Adc_Channel_Get_Last_Sample
 - DM35424_ADC_Public_Library_Functions, 21
- DM35424_Adc_Channel_Get_Thresholds
 - DM35424_ADC_Public_Library_Functions, 22
- DM35424_Adc_Channel_Interrupt_Clear_Status
 - DM35424_ADC_Public_Library_Functions, 24

- DM35424_Adc_Channel_Interrupt_Get_Config
 - DM35424 ADC Public Library Functions, [25](#)
- DM35424_Adc_Channel_Interrupt_Get_Status
 - DM35424 ADC Public Library Functions, [26](#)
- DM35424_Adc_Channel_Interrupt_Set_Config
 - DM35424 ADC Public Library Functions, [27](#)
- DM35424_Adc_Channel_Reset
 - DM35424 ADC Public Library Functions, [28](#)
- DM35424_Adc_Channel_Set_Filter
 - DM35424 ADC Public Library Functions, [29](#)
- DM35424_Adc_Channel_Set_High_Threshold
 - DM35424 ADC Public Library Functions, [30](#)
- DM35424_Adc_Channel_Set_Low_Threshold
 - DM35424 ADC Public Library Functions, [32](#)
- DM35424_Adc_Channel_Setup
 - DM35424 ADC Public Library Functions, [33](#)
- DM35424_ADC_CLK_BUS_SRC_CHAN_THRESH
 - DM35424 ADC Library Constants, [10](#)
- DM35424_ADC_CLK_BUS_SRC_DISABLE
 - DM35424 ADC Library Constants, [10](#)
- DM35424_ADC_CLK_BUS_SRC_PACER_TICK
 - DM35424 ADC Library Constants, [11](#)
- DM35424_ADC_CLK_BUS_SRC_POST_STOP_BUFF_FULL
 - DM35424 ADC Library Constants, [10](#)
- DM35424_ADC_CLK_BUS_SRC_PRE_START_BUFF_FULL
 - DM35424 ADC Library Constants, [10](#)
- DM35424_ADC_CLK_BUS_SRC_SAMPLE_TAKEN
 - DM35424 ADC Library Constants, [10](#)
- DM35424_ADC_CLK_BUS_SRC_SAMPLING_COMPLETE
 - DM35424 ADC Library Constants, [11](#)
- DM35424_ADC_CLK_BUS_SRC_START_TRIG
 - DM35424 ADC Library Constants, [10](#)
- DM35424_ADC_CLK_BUS_SRC_STOP_TRIG
 - DM35424 ADC Library Constants, [10](#)
- DM35424_Adc_Clock_Events
 - DM35424 ADC Library Constants, [10](#)
- dm35424_adc_continuous_dma.c
 - ASCII_FILE_NAME, [390](#)
 - BIN_FILE_NAME, [390](#)
 - board, [395](#)
 - buffer_count, [395](#)
 - BUFFER_SIZE_BYTES, [390](#)
 - buffer_size_bytes, [396](#)
 - BUFFER_SIZE_SAMPLES, [390](#)
 - convert_bin_to_txt, [391](#)
 - DEFAULT_RATE, [390](#)
 - dma_has_error, [396](#)
 - exit_program, [396](#)
 - ISR, [391](#)
 - local_buffer, [396](#)
 - main, [391](#)
 - my_adc, [396](#)
 - next_buffer, [396](#)
 - output_channel_status, [392](#)
 - program_name, [397](#)
 - setup_adc, [393](#)
 - setup_ctrlc_handler, [394](#)
 - setup_dacs_and_start, [394](#)
 - sigint_handler, [395](#)
- DM35424_Adc_Fifo_Channel_Read
 - DM35424 ADC Public Library Functions, [34](#)
- DM35424_ADC_GAIN_05
 - DM35424 ADC Library Constants, [11](#)
- DM35424_ADC_GAIN_1
 - DM35424 ADC Library Constants, [11](#)
- DM35424_ADC_GAIN_128
 - DM35424 ADC Library Constants, [11](#)
- DM35424_ADC_GAIN_16
 - DM35424 ADC Library Constants, [11](#)
- DM35424_ADC_GAIN_2
 - DM35424 ADC Library Constants, [11](#)
- DM35424_ADC_GAIN_32
 - DM35424 ADC Library Constants, [11](#)
- DM35424_ADC_GAIN_4
 - DM35424 ADC Library Constants, [11](#)
- DM35424_ADC_GAIN_64
 - DM35424 ADC Library Constants, [11](#)
- DM35424_ADC_GAIN_8
 - DM35424 ADC Library Constants, [11](#)
- DM35424_Adc_Get_Clock_Source_Global
 - DM35424 ADC Public Library Functions, [35](#)
- DM35424_Adc_Get_Clock_Src
 - DM35424 ADC Public Library Functions, [36](#)
- DM35424_Adc_Get_Mode_Status
 - DM35424 ADC Public Library Functions, [37](#)
- DM35424_Adc_Get_Post_Stop_Samples
 - DM35424 ADC Public Library Functions, [38](#)
- DM35424_Adc_Get_Pre_Trigger_Samples
 - DM35424 ADC Public Library Functions, [39](#)
- DM35424_Adc_Get_Sample_Count
 - DM35424 ADC Public Library Functions, [40](#)
- DM35424_Adc_Get_Start_Trigger
 - DM35424 ADC Public Library Functions, [41](#)
- DM35424_Adc_Get_Stop_Trigger
 - DM35424 ADC Public Library Functions, [42](#)
- DM35424_Adc_Initialize
 - DM35424 ADC Public Library Functions, [43](#)
- DM35424_ADC_INPUT_DAC_LOOPBACK
 - DM35424 ADC Library Constants, [12](#)
- DM35424_ADC_INPUT_DIFFERENTIAL
 - DM35424 ADC Library Constants, [12](#)
- DM35424_ADC_INPUT_SINGLE_ENDED_NEG
 - DM35424 ADC Library Constants, [12](#)
- DM35424_ADC_INPUT_SINGLE_ENDED_POS
 - DM35424 ADC Library Constants, [12](#)
- DM35424_Adc_Interrupt_Clear_Status
 - DM35424 ADC Public Library Functions, [44](#)
- DM35424_Adc_Interrupt_Get_Config
 - DM35424 ADC Public Library Functions, [45](#)
- DM35424_Adc_Interrupt_Get_Status
 - DM35424 ADC Public Library Functions, [46](#)
- DM35424_Adc_Interrupt_Set_Config
 - DM35424 ADC Public Library Functions, [47](#)
- DM35424_ADC_MODE_CONFIG_HIGH_RES
 - DM35424 ADC Library Constants, [13](#)
- DM35424_ADC_MODE_CONFIG_HIGH_SPEED

- DM35424 ADC Library Constants, [13](#)
- DM35424_ADC_MODE_CONFIG_LOW_POWER
 - DM35424 ADC Library Constants, [13](#)
- DM35424_ADC_MODE_CONFIG_LOW_SPEED
 - DM35424 ADC Library Constants, [13](#)
- DM35424_ADC_NON_VOL_TO_VOL
 - DM35424 Reference Adjustment Library Constants, [164](#)
- DM35424_Adc_Open
 - DM35424 ADC Public Library Functions, [48](#)
- DM35424_Adc_Pause
 - DM35424 ADC Public Library Functions, [49](#)
- DM35424_Adc_Reset
 - DM35424 ADC Public Library Functions, [50](#)
- DM35424_ADC_RNG_BIPOLAR_156mV
 - DM35424 ADC Library Constants, [12](#)
- DM35424_ADC_RNG_BIPOLAR_19mV
 - DM35424 ADC Library Constants, [12](#)
- DM35424_ADC_RNG_BIPOLAR_1_25V
 - DM35424 ADC Library Constants, [12](#)
- DM35424_ADC_RNG_BIPOLAR_2_5V
 - DM35424 ADC Library Constants, [12](#)
- DM35424_ADC_RNG_BIPOLAR_312mV
 - DM35424 ADC Library Constants, [12](#)
- DM35424_ADC_RNG_BIPOLAR_39mV
 - DM35424 ADC Library Constants, [12](#)
- DM35424_ADC_RNG_BIPOLAR_625mV
 - DM35424 ADC Library Constants, [12](#)
- DM35424_ADC_RNG_BIPOLAR_78mV
 - DM35424 ADC Library Constants, [12](#)
- DM35424_ADC_RNG_UNIPOLAR_5V
 - DM35424 ADC Library Constants, [12](#)
- DM35424_Adc_Sample_To_Volts
 - DM35424 ADC Public Library Functions, [51](#)
- DM35424_Adc_Set_Clk_Divider
 - DM35424 ADC Public Library Functions, [52](#)
- DM35424_Adc_Set_Clock_Source_Global
 - DM35424 ADC Public Library Functions, [53](#)
- DM35424_Adc_Set_Clock_Src
 - DM35424 ADC Public Library Functions, [54](#)
- DM35424_Adc_Set_Post_Stop_Samples
 - DM35424 ADC Public Library Functions, [55](#)
- DM35424_Adc_Set_Pre_Trigger_Samples
 - DM35424 ADC Public Library Functions, [56](#)
- DM35424_Adc_Set_Sample_Rate
 - DM35424 ADC Public Library Functions, [57](#)
- DM35424_Adc_Set_Start_Trigger
 - DM35424 ADC Public Library Functions, [58](#)
- DM35424_Adc_Set_Stop_Trigger
 - DM35424 ADC Public Library Functions, [59](#)
- DM35424_Adc_Start
 - DM35424 ADC Public Library Functions, [60](#)
- DM35424_Adc_Start_Rearm
 - DM35424 ADC Public Library Functions, [61](#)
- dm35424_adc_test.c
 - board, [213](#)
 - buffer_count, [213](#)
 - BUFFER_SIZE_BYTES, [209](#)
 - buffer_size_bytes, [213](#)
 - BUFFER_SIZE_SAMPLES, [209](#)
 - DAT_FILE_NAME_PREFIX, [209](#)
 - DAT_FILE_NAME_SUFFIX, [209](#)
 - DEFAULT_RATE, [209](#)
 - dma_has_error, [213](#)
 - exit_program, [213](#)
 - ISR, [210](#)
 - local_buffer, [213](#)
 - main, [210](#)
 - my_adc, [214](#)
 - output_channel_status, [211](#)
 - program_name, [214](#)
 - sigint_handler, [212](#)
- DM35424_Adc_Uninitialize
 - DM35424 ADC Public Library Functions, [62](#)
- DM35424_ADC_VOL_TO_NON_VOL
 - DM35424 Reference Adjustment Library Constants, [164](#)
- DM35424_Adc_Volts_To_Sample
 - DM35424 ADC Public Library Functions, [63](#)
- dm35424_board_access.h
 - DM35424LIB_API, [471](#)
- dm35424_board_checkout.c
 - program_name, [230](#)
 - run_test_9, [229](#)
 - sigint_handler, [229](#)
- DM35424_Board_Close
 - DM35424 Board Access Structures, [65](#)
- DM35424_Board_Descriptor, [185](#)
 - file_descriptor, [185](#)
 - isr, [185](#)
 - pid, [186](#)
- DM35424_Board_Open
 - DM35424 Board Access Structures, [65](#)
- DM35424_BOTH_NON_VOL_TO_VOL
 - DM35424 Reference Adjustment Library Constants, [164](#)
- DM35424_BOTH_VOL_TO_NON_VOL
 - DM35424 Reference Adjustment Library Constants, [164](#)
- dm35424_calibrate.c
 - exit_program, [325](#)
 - main, [323](#)
 - program_name, [325](#)
 - sigint_handler, [324](#)
- DM35424_Check_Result
 - DM35424 Utility Library Functions, [180](#)
- DM35424_CLK_BUS2
 - dm35424_types.h, [521](#)
- DM35424_CLK_BUS3
 - dm35424_types.h, [521](#)
- DM35424_CLK_BUS4
 - dm35424_types.h, [521](#)
- DM35424_CLK_BUS5
 - dm35424_types.h, [521](#)
- DM35424_CLK_BUS6
 - dm35424_types.h, [521](#)

- DM35424_CLK_BUS7
 - dm35424_types.h, [521](#)
- DM35424_CLK_SRC_BUS2
 - dm35424_types.h, [521](#)
- DM35424_CLK_SRC_BUS2_INV
 - dm35424_types.h, [521](#)
- DM35424_CLK_SRC_BUS3
 - dm35424_types.h, [521](#)
- DM35424_CLK_SRC_BUS3_INV
 - dm35424_types.h, [521](#)
- DM35424_CLK_SRC_BUS4
 - dm35424_types.h, [521](#)
- DM35424_CLK_SRC_BUS4_INV
 - dm35424_types.h, [521](#)
- DM35424_CLK_SRC_BUS5
 - dm35424_types.h, [521](#)
- DM35424_CLK_SRC_BUS5_INV
 - dm35424_types.h, [521](#)
- DM35424_CLK_SRC_BUS6
 - dm35424_types.h, [521](#)
- DM35424_CLK_SRC_BUS6_INV
 - dm35424_types.h, [521](#)
- DM35424_CLK_SRC_BUS7
 - dm35424_types.h, [521](#)
- DM35424_CLK_SRC_BUS7_INV
 - dm35424_types.h, [521](#)
- DM35424_CLK_SRC_CHAN_THRESH
 - dm35424_types.h, [521](#)
- DM35424_CLK_SRC_CHAN_THRESH_INV
 - dm35424_types.h, [521](#)
- DM35424_CLK_SRC_IMMEDIATE
 - dm35424_types.h, [521](#)
- DM35424_CLK_SRC_NEVER
 - dm35424_types.h, [521](#)
- DM35424_Clock_Buses
 - dm35424_types.h, [520](#)
- DM35424_Clock_Sources
 - dm35424_types.h, [521](#)
- DM35424_Copy_Directions
 - DM35424 Reference Adjustment Library Constants, [164](#)
- dm35424_dac.c
 - DEFAULT_DAC_NUM, [410](#)
 - main, [411](#)
 - program_name, [411](#)
- DM35424_Dac_Channel_Clear_Marker_Status
 - DM35424 DAC Library Public Functions, [75](#)
- DM35424_Dac_Channel_Get_Marker_Config
 - DM35424 DAC Library Public Functions, [75](#)
- DM35424_Dac_Channel_Get_Marker_Status
 - DM35424 DAC Library Public Functions, [76](#)
- DM35424_Dac_Channel_Set_Marker_Config
 - DM35424 DAC Library Public Functions, [77](#)
- DM35424_DAC_CLK_BUS_SRC_CHAN_MARKER
 - DM35424 DAC Library Constants, [72](#)
- DM35424_DAC_CLK_BUS_SRC_CONV_COMPL
 - DM35424 DAC Library Constants, [72](#)
- DM35424_DAC_CLK_BUS_SRC_CONVERSION_SENT
 - DM35424 DAC Library Constants, [72](#)
- DM35424_DAC_CLK_BUS_SRC_DISABLE
 - DM35424 DAC Library Constants, [72](#)
- DM35424_DAC_CLK_BUS_SRC_START_TRIG
 - DM35424 DAC Library Constants, [72](#)
- DM35424_DAC_CLK_BUS_SRC_STOP_TRIG
 - DM35424 DAC Library Constants, [72](#)
- DM35424_Dac_Clock_Events
 - DM35424 DAC Library Constants, [72](#)
- DM35424_Dac_Conv_To_Volts
 - DM35424 DAC Library Public Functions, [78](#)
- dm35424_dac_dma.c
 - BUFFER_SIZE_BYTES, [419](#)
 - BUFFER_SIZE_SAMPLES, [419](#)
 - DEFAULT_DAC_TO_USE, [419](#)
 - DEFAULT_RATE, [419](#)
 - exit_program, [421](#)
 - main, [420](#)
 - NUM_BUFFERS_TO_USE, [419](#)
 - program_name, [421](#)
 - sigint_handler, [420](#)
- DM35424_Dac_Fifo_Channel_Write
 - DM35424 DAC Library Public Functions, [79](#)
- DM35424_Dac_Get_Clock_Div
 - DM35424 DAC Library Public Functions, [80](#)
- DM35424_Dac_Get_Clock_Src
 - DM35424 DAC Library Public Functions, [81](#)
- DM35424_Dac_Get_Conversion_Count
 - DM35424 DAC Library Public Functions, [82](#)
- DM35424_Dac_Get_Last_Conversion
 - DM35424 DAC Library Public Functions, [83](#)
- DM35424_Dac_Get_Mode_Status
 - DM35424 DAC Library Public Functions, [84](#)
- DM35424_Dac_Get_Post_Stop_Conversion_Count
 - DM35424 DAC Library Public Functions, [85](#)
- DM35424_Dac_Get_Start_Trigger
 - DM35424 DAC Library Public Functions, [86](#)
- DM35424_Dac_Get_Stop_Trigger
 - DM35424 DAC Library Public Functions, [87](#)
- DM35424_Dac_Interrupt_Clear_Status
 - DM35424 DAC Library Public Functions, [88](#)
- DM35424_Dac_Interrupt_Get_Config
 - DM35424 DAC Library Public Functions, [89](#)
- DM35424_Dac_Interrupt_Get_Status
 - DM35424 DAC Library Public Functions, [89](#)
- DM35424_Dac_Interrupt_Set_Config
 - DM35424 DAC Library Public Functions, [90](#)
- DM35424_DAC_NON_VOL_TO_VOL
 - DM35424 Reference Adjustment Library Constants, [164](#)
- DM35424_Dac_Open
 - DM35424 DAC Library Public Functions, [91](#)
- DM35424_Dac_Pause
 - DM35424 DAC Library Public Functions, [92](#)
- DM35424_Dac_Reset
 - DM35424 DAC Library Public Functions, [93](#)
- DM35424_Dac_Set_Clock_Div
 - DM35424 DAC Library Public Functions, [94](#)

- DM35424_Dac_Set_Clock_Source_Global
 - DM35424 DAC Library Public Functions, [95](#)
- DM35424_Dac_Set_Clock_Src
 - DM35424 DAC Library Public Functions, [96](#)
- DM35424_Dac_Set_Conversion_Rate
 - DM35424 DAC Library Public Functions, [97](#)
- DM35424_Dac_Set_Last_Conversion
 - DM35424 DAC Library Public Functions, [98](#)
- DM35424_Dac_Set_Post_Stop_Conversion_Count
 - DM35424 DAC Library Public Functions, [99](#)
- DM35424_Dac_Set_Start_Trigger
 - DM35424 DAC Library Public Functions, [100](#)
- DM35424_Dac_Set_Stop_Trigger
 - DM35424 DAC Library Public Functions, [101](#)
- DM35424_Dac_Start
 - DM35424 DAC Library Public Functions, [102](#)
- DM35424_DAC_VOL_TO_NON_VOL
 - DM35424 Reference Adjustment Library Constants, [164](#)
- DM35424_Dac_Volts_To_Conv
 - DM35424 DAC Library Public Functions, [103](#)
- dm35424_device_descriptor, [186](#)
 - cdev, [187](#)
 - device_id, [187](#)
 - device_index, [187](#)
 - device_lock, [187](#)
 - dma_descr_list, [187](#)
 - dma_wait_queue, [187](#)
 - int_queue_count, [188](#)
 - int_queue_in_marker, [188](#)
 - int_queue_missed, [188](#)
 - int_queue_out_marker, [188](#)
 - int_wait_queue, [188](#)
 - interrupt_fb, [188](#)
 - irq_number, [189](#)
 - name, [189](#)
 - pci, [189](#)
 - pdev, [189](#)
 - reference_count, [189](#)
 - remove_isr_flag, [189](#)
- dm35424_dio.c
 - board, [431](#)
 - DM35424_DIO_DIRECTION, [430](#)
 - main, [430](#)
 - my_dio, [431](#)
 - program_name, [431](#)
- DM35424_DIO_DIRECTION
 - dm35424_dio.c, [430](#)
- DM35424_Dio_Get_Direction
 - DM35424 DIO Public, [105](#)
- DM35424_Dio_Get_Input_Value
 - DM35424 DIO Public, [105](#)
- DM35424_Dio_Get_Output_Value
 - DM35424 DIO Public, [106](#)
- DM35424_Dio_Open
 - DM35424 DIO Public, [107](#)
- DM35424_Dio_Set_Direction
 - DM35424 DIO Public, [108](#)
- DM35424_Dio_Set_Output_Value
 - DM35424 DIO Public, [109](#)
- DM35424_Dma
 - DM35424 Board Access Structures, [66](#)
- DM35424_Dma_Buffer_Setup
 - DM35424 DMA Public Library Functions, [114](#)
- DM35424_Dma_Buffer_Status
 - DM35424 DMA Public Library Functions, [115](#)
- DM35424_Dma_Check_Buffer_Used
 - DM35424 DMA Public Library Functions, [117](#)
- DM35424_Dma_Check_For_Error
 - DM35424 DMA Public Library Functions, [118](#)
- DM35424_Dma_Clear
 - DM35424 DMA Public Library Functions, [119](#)
- DM35424_Dma_Clear_Interrupt
 - DM35424 DMA Public Library Functions, [120](#)
- DM35424_Dma_Configure_Interrupts
 - DM35424 DMA Public Library Functions, [122](#)
- DM35424_DMA_Descriptor, [190](#)
 - buffer_start_offset, [190](#)
 - control_offset, [190](#)
 - num_buffers, [190](#)
- dm35424_dma_descriptor, [191](#)
 - buffer, [191](#)
 - buffer_size, [191](#)
 - bus_addr, [192](#)
 - channel, [192](#)
 - fb_num, [192](#)
 - list, [192](#)
 - virt_addr, [192](#)
- DM35424_Dma_Find_Interrupt
 - DM35424 DMA Public Library Functions, [123](#)
- DM35424_DMA_FUNCTIONS
 - DM35424 PCI Region Structures, [70](#)
- DM35424_Dma_Get_Current_Buffer_Count
 - DM35424 DMA Public Library Functions, [125](#)
- DM35424_Dma_Get_Errors
 - DM35424 DMA Public Library Functions, [126](#)
- DM35424_Dma_Get_Fifo_Counts
 - DM35424 DMA Public Library Functions, [127](#)
- DM35424_Dma_Get_Fifo_State
 - DM35424 DMA Public Library Functions, [129](#)
- DM35424_Dma_Get_Interrupt_Configuration
 - DM35424 DMA Public Library Functions, [130](#)
- DM35424_DMA_INITIALIZE
 - DM35424 PCI Region Structures, [70](#)
- DM35424_Dma_Initialize
 - DM35424 Board Access Public Library Functions, [156](#)
- DM35424_Dma_Pause
 - DM35424 DMA Public Library Functions, [131](#)
- DM35424_DMA_READ
 - DM35424 PCI Region Structures, [70](#)
- DM35424_Dma_Read
 - DM35424 Board Access Public Library Functions, [157](#)
- DM35424_Dma_Reset_Buffer
 - DM35424 DMA Public Library Functions, [132](#)

- DM35424_Dma_Setup
 - DM35424 DMA Public Library Functions, [133](#)
- DM35424_Dma_Setup_Set_Direction
 - DM35424 DMA Public Library Functions, [135](#)
- DM35424_Dma_Setup_Set_Used
 - DM35424 DMA Public Library Functions, [136](#)
- DM35424_Dma_Start
 - DM35424 DMA Public Library Functions, [137](#)
- DM35424_Dma_Status
 - DM35424 DMA Public Library Functions, [138](#)
- DM35424_Dma_Stop
 - DM35424 DMA Public Library Functions, [140](#)
- DM35424_DMA_WRITE
 - DM35424 PCI Region Structures, [70](#)
- DM35424_Dma_Write
 - DM35424 Board Access Public Library Functions, [159](#)
- DM35424_FIFO_EMPTY
 - DM35424 DMA Public Library Constants, [112](#)
- DM35424_FIFO_FULL
 - DM35424 DMA Public Library Constants, [112](#)
- DM35424_FIFO_HAS_DATA
 - DM35424 DMA Public Library Constants, [112](#)
- DM35424_Fifo_States
 - DM35424 DMA Public Library Constants, [112](#)
- DM35424_FIFO_UNKNOWN
 - DM35424 DMA Public Library Constants, [112](#)
- DM35424_Function_Block, [193](#)
 - control_offset, [193](#)
 - dma_channel, [193](#)
 - dma_offset, [193](#)
 - fb_num, [194](#)
 - fb_offset, [194](#)
 - num_dma_buffers, [194](#)
 - num_dma_channels, [194](#)
 - ordinal_fb_type_num, [194](#)
 - sub_type, [195](#)
 - type, [195](#)
 - type_revision, [195](#)
- DM35424_Function_Block_Open
 - DM35424 Board Library Public Functions, [148](#)
- DM35424_Function_Block_Open_Module
 - DM35424 Board Library Public Functions, [148](#)
- DM35424_Gains
 - DM35424 ADC Library Constants, [11](#)
- DM35424_Gbc_Ack_Interrupt
 - DM35424 Board Library Public Functions, [149](#)
- DM35424_Gbc_Board_Reset
 - DM35424 Board Library Public Functions, [150](#)
- DM35424_Gbc_Get_Format
 - DM35424 Board Library Public Functions, [151](#)
- DM35424_Gbc_Get_Fpga_Build
 - DM35424 Board Library Public Functions, [151](#)
- DM35424_Gbc_Get_Pdp_Number
 - DM35424 Board Library Public Functions, [152](#)
- DM35424_Gbc_Get_Revision
 - DM35424 Board Library Public Functions, [153](#)
- DM35424_Gbc_Get_Sys_Clock_Freq
 - DM35424 Board Library Public Functions, [154](#)
- DM35424_General_InstallISR
 - DM35424 Board Access Public Library Functions, [160](#)
- DM35424_General_RemoveISR
 - DM35424 Board Access Public Library Functions, [161](#)
- DM35424_General_SetISRPriority
 - DM35424 Board Access Public Library Functions, [161](#)
- DM35424_General_WaitForInterrupt
 - DM35424 Board Access Public Library Functions, [162](#)
- DM35424_Generate_Signal_Data
 - DM35424 Utility Library Functions, [180](#)
- DM35424_Get_Maskable
 - DM35424 Utility Library Functions, [182](#)
- DM35424_Get_Time_Diff
 - DM35424 Utility Library Functions, [182](#)
- DM35424_Input_Mode
 - DM35424 ADC Library Constants, [11](#)
- DM35424_Input_Ranges
 - DM35424 ADC Library Constants, [12](#)
- dm35424_ioctl_argument, [195](#)
 - dma, [196](#)
 - interrupt, [196](#)
 - modify, [196](#)
 - readwrite, [196](#)
- dm35424_ioctl_dma, [197](#)
 - buffer, [197](#)
 - buffer_ptr, [197](#)
 - buffer_size, [197](#)
 - channel, [197](#)
 - fb_num, [198](#)
 - function, [198](#)
 - num_buffers, [198](#)
 - pci, [198](#)
- dm35424_ioctl_interrupt_info_request, [198](#)
 - error_occurred, [199](#)
 - interrupt_fb, [199](#)
 - interrupts_remaining, [199](#)
 - valid_interrupt, [199](#)
- dm35424_ioctl_region_modify, [200](#)
 - access, [200](#)
 - mask, [200](#)
 - mask16, [201](#)
 - mask32, [201](#)
 - mask8, [201](#)
- dm35424_ioctl_region_readwrite, [201](#)
 - access, [202](#)
- dm35424_list_fb.c
 - main, [435](#)
 - program_name, [435](#)
- DM35424_Micro_Sleep
 - DM35424 Utility Library Functions, [183](#)
- DM35424_Modify
 - DM35424 Board Access Structures, [67](#)
- DM35424_OFFSET_FB_ADC_FIFO

- DM35424 Register Offsets, 175
- dm35424_oven_test.c
 - program_name, 337
 - sigint_handler, 337
- dm35424_pci_access_request, 202
 - data, 203
 - data16, 203
 - data32, 203
 - data8, 203
 - offset, 203
 - region, 203
 - size, 203
- dm35424_pci_region, 204
 - allocated, 204
 - io_addr, 204
 - length, 204
 - phys_addr, 205
 - virt_addr, 205
- DM35424_PCI_REGION_ACCESS_16
 - DM35424 PCI Region Structures, 70
- DM35424_PCI_REGION_ACCESS_32
 - DM35424 PCI Region Structures, 70
- DM35424_PCI_REGION_ACCESS_8
 - DM35424 PCI Region Structures, 70
- dm35424_pci_region_access_dir
 - DM35424 Driver Enumerations, 142
- DM35424_PCI_REGION_ACCESS_READ
 - DM35424 Driver Enumerations, 142
- dm35424_pci_region_access_size
 - DM35424 PCI Region Structures, 70
- DM35424_PCI_REGION_ACCESS_WRITE
 - DM35424 Driver Enumerations, 142
- DM35424_PCI_REGION_FB
 - DM35424 PCI Region Structures, 71
- DM35424_PCI_REGION_GBC
 - DM35424 PCI Region Structures, 71
- DM35424_PCI_REGION_GBC2
 - DM35424 PCI Region Structures, 71
- dm35424_pci_region_num
 - DM35424 PCI Region Structures, 70
- DM35424_Read
 - DM35424 Board Access Structures, 68
- dm35424_ref_adjust.c
 - DEFAULT_RATE, 441
 - exit_program, 443
 - getch, 441
 - keyboard_hit, 441
 - main, 441
 - MAX_REF_ADJUST, 441
 - program_name, 443
 - sigint_handler, 442
- DM35424_Ref_Adjust_Copy_Data
 - DM35424 Reference Adjustment Public Library Functions, 165
- DM35424_Ref_Adjust_Open
 - DM35424 Reference Adjustment Public Library Functions, 165
- dm35424_ref_adjust_test.c
 - exit_program, 361
 - main, 359
 - program_name, 361
 - sigint_handler, 360
- DM35424_Ref_Adjust_Write_Adc_To_NonVolatile
 - DM35424 Reference Adjustment Public Library Functions, 166
- DM35424_Ref_Adjust_Write_Adc_To_Volatile
 - DM35424 Reference Adjustment Public Library Functions, 167
- DM35424_Ref_Adjust_Write_Dac_To_NonVolatile
 - DM35424 Reference Adjustment Public Library Functions, 168
- DM35424_Ref_Adjust_Write_Dac_To_Volatile
 - DM35424 Reference Adjustment Public Library Functions, 169
- DM35424_Sampling_Mode
 - DM35424 ADC Library Constants, 12
- DM35424_SAWTOOTH_WAVE
 - DM35424 Utility Library Functions, 180
- DM35424_SINE_WAVE
 - DM35424 Utility Library Functions, 179
- DM35424_SQUARE_WAVE
 - DM35424 Utility Library Functions, 180
- dm35424_temperature.c
 - exit_program, 452
 - main, 451
 - program_name, 452
 - sigint_handler, 452
 - TEMP_FB_TO_OPEN, 451
- DM35424_Temperature_Open
 - DM35424 Temperature, 175
- DM35424_Temperature_Read
 - DM35424 Temperature, 176
- dm35424_types.h
 - DM35424_CLK_BUS2, 521
 - DM35424_CLK_BUS3, 521
 - DM35424_CLK_BUS4, 521
 - DM35424_CLK_BUS5, 521
 - DM35424_CLK_BUS6, 521
 - DM35424_CLK_BUS7, 521
 - DM35424_CLK_SRC_BUS2, 521
 - DM35424_CLK_SRC_BUS2_INV, 521
 - DM35424_CLK_SRC_BUS3, 521
 - DM35424_CLK_SRC_BUS3_INV, 521
 - DM35424_CLK_SRC_BUS4, 521
 - DM35424_CLK_SRC_BUS4_INV, 521
 - DM35424_CLK_SRC_BUS5, 521
 - DM35424_CLK_SRC_BUS5_INV, 521
 - DM35424_CLK_SRC_BUS6, 521
 - DM35424_CLK_SRC_BUS6_INV, 521
 - DM35424_CLK_SRC_BUS7, 521
 - DM35424_CLK_SRC_BUS7_INV, 521
 - DM35424_CLK_SRC_CHAN_THRESH, 521
 - DM35424_CLK_SRC_CHAN_THRESH_INV, 521
 - DM35424_CLK_SRC_IMMEDIATE, 521
 - DM35424_CLK_SRC_NEVER, 521
 - DM35424_Clock_Buses, 520

- DM35424_Clock_Sources, [521](#)
- DM35424_Waveforms
 - DM35424 Utility Library Functions, [179](#)
- DM35424_Write
 - DM35424 Board Access Structures, [68](#)
- DM35424LIB_API
 - dm35424_board_access.h, [471](#)
- dma
 - dm35424_ioctl_argument, [196](#)
- dma_channel
 - DM35424_Function_Block, [193](#)
- dma_descr_list
 - dm35424_device_descriptor, [187](#)
- dma_has_error
 - dm35424_adc_continuous_dma.c, [396](#)
 - dm35424_adc_test.c, [213](#)
- dma_offset
 - DM35424_Function_Block, [193](#)
- dma_wait_queue
 - dm35424_device_descriptor, [187](#)
- DUMP_OPTION
 - DM35424 Example Programs Constants, [146](#)
- error_occurred
 - dm35424_ioctl_interrupt_info_request, [199](#)
- examples/_non_public/dm35424_adc_test.c, [207](#), [214](#)
- examples/_non_public/dm35424_board_checkout.c, [227](#), [230](#)
- examples/_non_public/dm35424_calibrate.c, [322](#), [325](#)
- examples/_non_public/dm35424_oven_test.c, [336](#), [338](#)
- examples/_non_public/dm35424_ref_adjust_test.c, [358](#), [361](#)
- examples/dm35424_adc.c, [370](#), [375](#)
- examples/dm35424_adc_continuous_dma.c, [388](#), [397](#)
- examples/dm35424_dac.c, [409](#), [412](#)
- examples/dm35424_dac_dma.c, [417](#), [422](#)
- examples/dm35424_dio.c, [428](#), [431](#)
- examples/dm35424_list_fb.c, [434](#), [436](#)
- examples/dm35424_ref_adjust.c, [439](#), [444](#)
- examples/dm35424_temperature.c, [450](#), [453](#)
- exit_program
 - dm35424_adc.c, [374](#)
 - dm35424_adc_continuous_dma.c, [396](#)
 - dm35424_adc_test.c, [213](#)
 - dm35424_calibrate.c, [325](#)
 - dm35424_dac_dma.c, [421](#)
 - dm35424_ref_adjust.c, [443](#)
 - dm35424_ref_adjust_test.c, [361](#)
 - dm35424_temperature.c, [452](#)
- EXTERNAL_OPTION
 - DM35424 Example Programs Constants, [146](#)
- fb_num
 - dm35424_dma_descriptor, [192](#)
 - DM35424_Function_Block, [194](#)
 - dm35424_ioctl_dma, [198](#)
- fb_offset
 - DM35424_Function_Block, [194](#)
- file_descriptor
 - DM35424_Board_Descriptor, [185](#)
- FILE_OPTION
 - DM35424 Example Programs Constants, [145](#)
- function
 - dm35424_ioctl_dma, [198](#)
- getch
 - dm35424_ref_adjust.c, [441](#)
- HELP_OPTION
 - DM35424 Example Programs Constants, [145](#)
- Help_Options
 - DM35424 Example Programs Constants, [145](#)
- HOURS_OPTION
 - DM35424 Example Programs Constants, [146](#)
- include/dm35424.h, [455](#), [456](#)
- include/dm35424_adc_library.h, [457](#), [463](#)
- include/dm35424_board_access.h, [470](#), [472](#)
- include/dm35424_board_access_structs.h, [473](#), [474](#)
- include/dm35424_dac_library.h, [476](#), [480](#)
- include/dm35424_dio_library.h, [483](#), [484](#)
- include/dm35424_dma_library.h, [485](#), [488](#)
- include/dm35424_driver.h, [492](#), [493](#)
- include/dm35424_examples.h, [495](#), [497](#)
- include/dm35424_gbc_library.h, [500](#), [501](#)
- include/dm35424_ioctl.h, [502](#), [503](#)
- include/dm35424_os.h, [504](#), [505](#)
- include/dm35424_ref_adjust_library.h, [506](#), [508](#)
- include/dm35424_registers.h, [509](#), [514](#)
- include/dm35424_temperature_library.h, [517](#), [518](#)
- include/dm35424_types.h, [518](#), [522](#)
- include/dm35424_util_library.h, [523](#), [524](#)
- int_queue_count
 - dm35424_device_descriptor, [188](#)
- int_queue_in_marker
 - dm35424_device_descriptor, [188](#)
- int_queue_missed
 - dm35424_device_descriptor, [188](#)
- int_queue_out_marker
 - dm35424_device_descriptor, [188](#)
- int_wait_queue
 - dm35424_device_descriptor, [188](#)
- interrupt
 - dm35424_ioctl_argument, [196](#)
- interrupt_count
 - dm35424_adc.c, [374](#)
- interrupt_fb
 - dm35424_device_descriptor, [188](#)
 - dm35424_ioctl_interrupt_info_request, [199](#)
- interrupts_remaining
 - dm35424_ioctl_interrupt_info_request, [199](#)
- io_addr
 - dm35424_pci_region, [204](#)
- irq_number
 - dm35424_device_descriptor, [189](#)
- ISR
 - dm35424_adc_continuous_dma.c, [391](#)
 - dm35424_adc_test.c, [210](#)

- isr
 - DM35424_Board_Descriptor, 185
- keyboard_hit
 - dm35424_ref_adjust.c, 441
- length
 - dm35424_pci_region, 204
- list
 - dm35424_dma_descriptor, 192
- local_buffer
 - dm35424_adc_continuous_dma.c, 396
 - dm35424_adc_test.c, 213
- LOW_THRESHOLD_OPTION
 - DM35424 Example Programs Constants, 146
- main
 - dm35424_adc.c, 373
 - dm35424_adc_continuous_dma.c, 391
 - dm35424_adc_test.c, 210
 - dm35424_calibrate.c, 323
 - dm35424_dac.c, 411
 - dm35424_dac_dma.c, 420
 - dm35424_dio.c, 430
 - dm35424_list_fb.c, 435
 - dm35424_ref_adjust.c, 441
 - dm35424_ref_adjust_test.c, 359
 - dm35424_temperature.c, 451
- mask
 - dm35424_ioctl_region_modify, 200
- mask16
 - dm35424_ioctl_region_modify, 201
- mask32
 - dm35424_ioctl_region_modify, 201
- mask8
 - dm35424_ioctl_region_modify, 201
- MASTER_OPTION
 - DM35424 Example Programs Constants, 146
- MAX_REF_ADJUST
 - dm35424_ref_adjust.c, 441
- MINOR_OPTION
 - DM35424 Example Programs Constants, 145
- MODE_OPTION
 - DM35424 Example Programs Constants, 146
- modify
 - dm35424_ioctl_argument, 196
- my_adc
 - dm35424_adc_continuous_dma.c, 396
 - dm35424_adc_test.c, 214
- my_dio
 - dm35424_dio.c, 431
- name
 - dm35424_device_descriptor, 189
- next_buffer
 - dm35424_adc_continuous_dma.c, 396
- NOSTOP_OPTION
 - DM35424 Example Programs Constants, 145
- num_buffers
 - DM35424_DMA_Descriptor, 190
 - dm35424_ioctl_dma, 198
- NUM_BUFFERS_TO_USE
 - dm35424_dac_dma.c, 419
- num_dma_buffers
 - DM35424_Function_Block, 194
- num_dma_channels
 - DM35424_Function_Block, 194
- NUM_OPTION
 - DM35424 Example Programs Constants, 146
- offset
 - dm35424_pci_access_request, 203
- OFFILE_OPTION
 - DM35424 Example Programs Constants, 146
- ordinal_fb_type_num
 - DM35424_Function_Block, 194
- OUTPUT_ADC_OPTION
 - DM35424 Example Programs Constants, 146
- output_channel_status
 - dm35424_adc_continuous_dma.c, 392
 - dm35424_adc_test.c, 211
- OUTPUT_RMS_OPTION
 - DM35424 Example Programs Constants, 146
- PACKED_OPTION
 - DM35424 Example Programs Constants, 146
- PATTERN_OPTION
 - DM35424 Example Programs Constants, 146
- pci
 - dm35424_device_descriptor, 189
 - dm35424_ioctl_dma, 198
- pdev
 - dm35424_device_descriptor, 189
- phys_addr
 - dm35424_pci_region, 205
- pid
 - DM35424_Board_Descriptor, 186
- PORT_OPTION
 - DM35424 Example Programs Constants, 146
- program_name
 - dm35424_adc.c, 374
 - dm35424_adc_continuous_dma.c, 397
 - dm35424_adc_test.c, 214
 - dm35424_board_checkout.c, 230
 - dm35424_calibrate.c, 325
 - dm35424_dac.c, 411
 - dm35424_dac_dma.c, 421
 - dm35424_dio.c, 431
 - dm35424_list_fb.c, 435
 - dm35424_oven_test.c, 337
 - dm35424_ref_adjust.c, 443
 - dm35424_ref_adjust_test.c, 361
 - dm35424_temperature.c, 452
- RANGE_OPTION
 - DM35424 Example Programs Constants, 146
- RATE_OPTION
 - DM35424 Example Programs Constants, 145

- readwrite
 - dm35424_ioctl_argument, [196](#)
- RECEIVER_OPTION
 - DM35424 Example Programs Constants, [146](#)
- REF_NUM_OPTION
 - DM35424 Example Programs Constants, [146](#)
- REFCLK_OPTION
 - DM35424 Example Programs Constants, [146](#)
- reference_count
 - dm35424_device_descriptor, [189](#)
- REFILL_FIFO_OPTION
 - DM35424 Example Programs Constants, [146](#)
- region
 - dm35424_pci_access_request, [203](#)
- remove_isr_flag
 - dm35424_device_descriptor, [189](#)
- run_test_9
 - dm35424_board_checkout.c, [229](#)
- SAMPLES_OPTION
 - DM35424 Example Programs Constants, [146](#)
- SENDER_OPTION
 - DM35424 Example Programs Constants, [146](#)
- setup_adc
 - dm35424_adc_continuous_dma.c, [393](#)
- setup_ctrlc_handler
 - dm35424_adc_continuous_dma.c, [394](#)
- setup_dacs_and_start
 - dm35424_adc_continuous_dma.c, [394](#)
- sigint_handler
 - dm35424_adc.c, [374](#)
 - dm35424_adc_continuous_dma.c, [395](#)
 - dm35424_adc_test.c, [212](#)
 - dm35424_board_checkout.c, [229](#)
 - dm35424_calibrate.c, [324](#)
 - dm35424_dac_dma.c, [420](#)
 - dm35424_oven_test.c, [337](#)
 - dm35424_ref_adjust.c, [442](#)
 - dm35424_ref_adjust_test.c, [360](#)
 - dm35424_temperature.c, [452](#)
- size
 - dm35424_pci_access_request, [203](#)
- SIZE_OPTION
 - DM35424 Example Programs Constants, [146](#)
- SLAVE_OPTION
 - DM35424 Example Programs Constants, [146](#)
- START_OPTION
 - DM35424 Example Programs Constants, [145](#)
- STORE_OPTION
 - DM35424 Example Programs Constants, [146](#)
- sub_type
 - DM35424_Function_Block, [195](#)
- SYNC_CONN_OPTION
 - DM35424 Example Programs Constants, [146](#)
- SYNC_TERM_OPTION
 - DM35424 Example Programs Constants, [146](#)
- SYNCBUS_OPTION
 - DM35424 Example Programs Constants, [146](#)
- TEMP_FB_TO_OPEN
 - dm35424_temperature.c, [451](#)
- TERM_OPTION
 - DM35424 Example Programs Constants, [146](#)
- TEST_OPTION
 - DM35424 Example Programs Constants, [145](#)
- type
 - DM35424_Function_Block, [195](#)
- type_revision
 - DM35424_Function_Block, [195](#)
- USER_ID_OPTION
 - DM35424 Example Programs Constants, [146](#)
- valid_interrupt
 - dm35424_ioctl_interrupt_info_request, [199](#)
- VERBOSE_OPTION
 - DM35424 Example Programs Constants, [146](#)
- virt_addr
 - dm35424_dma_descriptor, [192](#)
 - dm35424_pci_region, [205](#)
- WAVE_OPTION
 - DM35424 Example Programs Constants, [145](#)